

现代 GPU 编程 · 面向 ML 系统 (小白图解版)

SUB 现代 GPU 编程 · 面向 ML 系统 (小白图解版)

基于 mlc.ai 公开教材《Modern GPU Programming for MLSys》的中文原创精读笔记
仅供学习 · 非逐字翻译 · 以原文为准

Contents

现代 GPU 编程 · 面向 ML 系统 (中文精读笔记 · 超级小白版)	11
目录	12
建议阅读路径	14
版权与免责声明	15
第 0 章 · 写给纯新手:GPU & ML Infra 极简入门	17
0.1 先说清楚: 这本书在讲什么, 你为什么需要先读这一章	17
0.2 一个贯穿全书的比喻:GPU 是一座” 超大工厂”	18
0.3 谁来干活: 线程的层级	18
0.4 东西放哪: 内存的层级	20
0.5 到底在算什么 (一): 矩阵乘 GEMM	21
0.6 到底在算什么 (二):Tensor Core 与 MMA	22
0.7 怎么喂数据: 异步搬运、流水线与” 重叠”	22
0.8 注意力 (Attention) 是什么, 为什么在硬件上难	23
0.9 这本书的两个” 新东西”:Blackwell 与 TIRx	24
0.10 术语速查表 (读后面遇到生词就回来翻)	24
0.11 该怎么读这本书	25
小结	25
延伸阅读	25
第 01 章 · GPU 执行模型	27
一、执行层级: 为什么不是” 一锅线程”	28
二、内存空间: 容量与速度的取舍	31
三、计算引擎:CUDA 核与张量核	35
四、GEMM 数据流水线: 把拼图拼起来	36
小结	38
延伸阅读	39
术语对照	39
第 02 章 · 什么决定 Kernel 的速度	41
一、屋顶线模型: 两条路取更慢的那条	42
二、常见 workload 的算术强度	45
三、算术强度低的时候怎么办	47
四、优化阶梯: 从” 可能” 到” 做到”	49
五、重叠是最大的杠杆	50
六、占用率与资源压力	52
小结	53
延伸阅读	54

术语对照	54
第 03 章 · 数据布局与其记号	57
一、形状-步长模型 (The Shape-Stride Model)	58
二、Tile 布局 (Tile Layout)	60
三、命名轴 (Named Axes)	61
四、分布式布局 (Distributed Layout)	64
五、Swizzle 布局 (Swizzle Layout)	66
小结	69
延伸阅读	70
术语对照	70
第 04 章 · 各代 GPU 的 Tensor Core 操作数布局	71
引子: 为什么“逻辑公式没变”还会算错	72
一、两条从未离场的约束	72
二、Ampere: 把数据拆碎摊在 warp 的各个 lane 上	74
三、Hopper: 让硬件“照说明书”自己去读数据	79
四、Blackwell: 把结果搬进新内存 TMEM	82
五、一个反复出现的片段:m8n8	85
六、贯穿三代的主线 (The Throughline)	85
小结	86
延伸阅读	86
术语对照	87
第 05 章 · 异步数据搬运:TMA	89
0. 为什么需要 TMA: 从「线程自己搬」说起	90
1. 一个线程发起, 硬件搬运整块 tile	91
2. Swizzle 布局: 不只是「搬到」, 还要「摆对」	92
3. 3D TMA: 在一次拷贝里同时完成「分块」和「swizzle」	94
4. 完成机制 (一): 加载用 mbarrier	96
5. 完成机制 (二): 存储用 commit group + wait group	97
6. 为什么 TMA 对流水线 (pipelining) 至关重要	98
小结	100
延伸阅读	100
术语对照	100
第 06 章 · 特殊内存:TMEM(Tensor Memory)	103
引子: 寄存器不够用,Blackwell 才另开了一条路	103
TMEM 是一个二维地址空间	105
分配: 把 TMEM 当成像 SMEM 一样要做预算的资源	107
读写 TMEM: 三条专用的异步通路	108
把整条数据通路串起来:TMEM 就在 Tensor Core 数据流的正中间	111
小结	112
延伸阅读	112

术语对照	112
第 07 章 · 异步协调:mbarrier	115
7.1 为什么需要一个显式的”完成信号”	115
7.2 mbarrier 到底是什么	117
7.3 相位追踪 (Phase Tracking)	120
7.4 同步规则 (Synchronization Rules)	122
小结	123
延伸阅读	124
术语对照	124
第 08 章 · 进阶: 集群启动控制 (CLC)	125
8.1 从静态调度到动态调度: 为什么需要 CLC	126
8.2 两条指令: 请求与查询	128
8.3 工作窃取循环	130
8.4 与常驻 GEMM 的关系	131
小结	132
延伸阅读	133
术语对照	133
第 09 章 · TIRx 入门	135
一、为什么需要 TIRx: 把「隐藏的决策」翻到台面上	135
二、运行环境与依赖	137
三、第一个 kernel: 单次 MMA 的 GEMM	137
四、三要素回读:scope / layout / dispatch	146
五、编译是怎么发生的	148
六、接下来去哪	149
小结	149
延伸阅读	149
术语对照	150
第 10 章 · TIRx Layout API	151
10.1 为什么需要 Layout API: 让记号变成对象	152
10.2 先看几个例子 (Layouts by Example)	153
10.3 交互演示 (Interactive Demo) 要点	154
10.4 TileLayout: 主力仿射布局	155
10.5 具名轴 (Named Axes): 名字本身就是语义	157
10.6 前向映射 (Forward Mapping):layout.apply()	158
10.7 案例 A: 张量核寄存器 tile	159
10.8 案例 B:Blackwell 张量内存 (TMEM)	161
10.9 缩放因子布局 (Scale Factor Layouts): 复制的真正用武之地	161
10.10 现成布局 (Ready-Made Layouts)	163
10.11 SwizzleLayout 与 ComposeLayout: 仿射之外的换序	163
10.12 为什么要 swizzle:bank 冲突的来由	164

10.13 swizzle 变换的数学形式	165
10.14 如何选择 swizzle 参数	166
10.15 元素的 bank 与 line	166
10.16 完整演算: 对 (8, 64) float16 tile 施加 128B swizzle	167
10.17 设计原理 (Design Rationale)	168
小结	168
延伸阅读	169
术语对照	169
第 11 章 · 构建分块 GEMM(Step 1-3)	171
本章导读: 从一个正确的 Tile 开始 (Building a Tiled GEMM)	171
GEMM: 问题定义与数据通路 (GEMM)	174
优化路线图 (Optimization Path)	177
第 1 步: 单 Tile 顺序 GEMM(Step 1: Sequential Single-Tile GEMM)	178
第 2 步:K 维循环累加 (Step 2: K-Loop Accumulation)	186
第 3 步: 空间分块, 多 CTA 并行 (Step 3: Spatial Tiling / Multi-CTA)	190
习题精解 (Exercises)	193
小结	197
延伸阅读	198
术语对照	198
第 12 章 · 用 TMA 为 GEMM 做流水线 (Step 4-6)	201
为什么基础版 GEMM 在浪费时间	202
Step 4:TMA 异步 Load	203
Step 5: 软件流水线 (PIPE_DEPTH=2)	208
Step 6: 持久化 kernel + tile 调度器	212
练习 (原书 Exercises)	214
小结	215
延伸阅读	215
术语对照	215
第 13 章 · 用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)	217
引言: 用 Warp 专门化与 Cluster 把 GEMM 推到极致 (Scaling GEMM with Warp Specialization and Clusters)	217
第 7 步:Warp 专门化 + 流水线 (Step 7: Warp Specialization + Pipeline)	221
第 8 步: 用 2-CTA Cluster 协同算一个大 Tile(Step 8: 2-CTA Cluster)	228
第 9 步: 多消费者 Warp 专门化 (Step 9: Multi-Consumer Warp Specialization)	234
端到端的优化成果 (End-to-End Result)	238
习题与自测 (Exercises)	241
小结	245
延伸阅读	246
术语对照	246

第 14 章 · Flash Attention 4	249
本章导读: 为什么 Attention 不是「再来一遍 GEMM」(Overview)	250
算法的形状 (Algorithm Shape)	253
Tile-原语图 (Tile-Primitive Graph)	255
Warp 的角色与作用域 (Warp Roles and Scopes)	258
怎么读后面的代码片段 (Reading the Fragments)	261
两个 MMA 阶段: 被 softmax 串起来的双矩阵乘 (The Two MMA Phases)	263
TMEM 的布局与复用 (TMEM Layout and Reuse)	267
barrier 如何把各个角色「接线」起来 (How Barriers Connect the Roles)	270
流水线结构: 同一时刻到底谁在干活 (Pipelining Structure)	274
重缩放与写回 (Rescaling and Writeback)	277
因果掩码 (Causal Masking)	279
GQA 支持 (GQA Support)	282
Tile 调度 (Tile Scheduling)	284
编译并验证 (Compile and Verify)	287
与 GEMM 的区别 (Differences from GEMM)	289
课后练习 (Exercises)	291
小结	294
延伸阅读	294
术语对照	294
第 15 章 · 参考资料 (附录)	297
15.1 这一部分是什么	297
15.2 三类需求 → 三个去处	298
15.3 两个”不在此处, 但你可能要找”的指针	300
15.4 建议的使用方式 (阅读顺序)	300
小结	301
延伸阅读	301
术语对照	301
第 16 章 · 调试 Warp 专门化 Kernel	303
16.1 心法: 把异步 kernel 看成一组「交接」	304
16.2 调试前: 先排除运行环境	305
16.3 标准调试流程 (Workflow)	306
16.4 「四要素表」: 可迁移的调试工作表	306
16.5 当编译失败时	308
16.6 检查生成的代码: 让 CUDA 当地图	309
16.7 何时必须重启 Python	310
16.8 症状地图 (Symptom Map)	310
16.9 死锁 (Deadlocks)	311
16.10 崩溃与 context 毒化	313
16.11 Step 7 参考骨架	314
16.12 结果错误 (Wrong Results)	315

16.13 正确但慢 (Correct but Slow)	316
16.14 提交一个好的 Issue	316
小结	317
延伸阅读	317
术语对照	317
第 17 章 · 编译器内部 (概览)	319
17.1 这一页是什么: 写给贡献者的入口	320
17.2 子页面一览: 目前只有「TIRx 下降流水线」	320
17.3 速览:TIRx 下降流水线在做什么	321
17.4 建议的阅读顺序	324
小结	324
延伸阅读	325
术语对照	325
第 18 章 · TIRx Lowering 流水线	327
18.1 这条流水线解决什么问题	328
18.2 它处在编译流程的哪个位置	329
18.3 流水线的 18 个 pass(逐个看)	329
18.4 深入 LowerTIRx: 核心下降是怎么发生的	332
18.5 一个完整的小例子:scale kernel 的下降全程	334
18.6 自己动手复现每个阶段	336
小结	337
延伸阅读	337
术语对照	337
第 19 章 · TIRx 语言参考 (概览)	339
19.1 这一页是什么	339
19.2 五个子页面分别讲什么	340
19.3 建议的阅读顺序	346
小结	347
延伸阅读	347
术语对照	348
第 20 章 · 解析器工具 (Parser utilities)	349
一、先搞清楚: 什么叫「解析期工具」	350
二、T.meta_var 一把 Python 值 inline 进 IR	351
三、@T.inline 一内联函数	353
四、@T.meta_class 一解析期状态对象	354
五、T.constexpr 一编译期 kernel 参数	356
六、四个工具的横向对比	357
小结	357
延伸阅读	357
术语对照	358

第 21 章 · 数据类型与表达式	359
21.0 为什么要把”类型”拆成两层?	359
21.1 表达式的 dtype	361
21.3 指针 (handle)	365
小结	368
延伸阅读	368
术语对照	368
第 22 章 · 缓冲区与内存	371
一、缓冲区的统一心智模型	372
三、共享内存 (shared memory)	376
四、寄存器 (registers)	380
六、缓冲区方法 (Buffer APIs)	384
第 23 章 · 控制流	389
一、if / else: 分支	390
二、统一控制流 vs 发散控制流 (核心)	392
三、循环家族	395
第 24 章 · CUDA C++ / PTX intrinsics	399
24.1 为什么需要「逃生通道」	400
24.2 调用后端内建函数 (backend intrinsics)	400
24.4 内联原始 CUDA(Inlining raw CUDA)	409
小结	410
延伸阅读	411
术语对照	411
术语对照表 (全书总表)	413
A · 架构与执行模型	413
B · 性能模型	413
C · 数据布局 (Layout)	414
D · Tensor Core 与 MMA	414
E · 内存层级与特殊内存	415
F · 异步搬运与协调	415
G · CLC 与调度	416
H · Warp 专门化与调试	416
I · TIRx 语言与控制流	417
J · 解析期 / 元编程	417
K · 编译器内部与下降	418
L · 参考与生态	418
M · 关键算法	418

现代 GPU 编程 · 面向 ML 系统 (中文精读笔记 · 超级小白版)

这是基于 mlc.ai 公开教材《Modern GPU Programming for ML Sys》(© 2026 MLC Community) 撰写的中文原创精读笔记 / 学习导读, 仅用于学习目的。

本笔记是对原书的转述、归纳与讲解, 并非原书的逐字翻译; 书中所有结论、代码示例、设计取舍均以原文为准。阅读原文请访问官方网站:

<https://mlc.ai/modern-gpu-programming-for-mlsys/>

这是“超级小白版”: 同一套内容, 有两个版本。一个是写给“已经懂点 GPU、想系统学”的工程师; 你现在看的这个版本更白一档——它是写给“会写代码, 但从没碰过 GPU、显卡、CUDA、并行”的程序员的。你会写 Python / C++ / 后端 / 数据处理都行, 但只要一提到“线程怎么并行”“内存为什么要分好几层”“矩阵乘法为什么是主角”就一脸懵——没关系, 这个版本就是为你写的。每个 GPU 专有名词第一次出现, 我都会停下来用大白话讲清楚”它是什么、为什么需要它”, 绝不假设你有任何 GPU 背景。

先说说这份笔记想带你去哪。一句话: 从“GPU 到底是怎么跑代码的”, 一直走到“怎么亲手写出真正快的程序”。

打个比方, 这就像学开车。我们会分三步走:

1. 先看 **Blackwell 架构**——这是 NVIDIA 一代 GPU 芯片的代号 (你可以理解成“某个型号的发动机”)。这一步是**先掀开引擎盖, 看清这台机器内部长什么样**: 它有哪些零件、各干什么。
2. 再学 **TIRx 编程模型**——这是一种专门用来给 GPU 写程序的语言/工具 (后面会细讲)。这一步是**学会握方向盘踩油门**, 也就是用什么语言、怎么去指挥这台机器干活。
3. 最后是 **GEMM 和 Flash Attention 实战**。GEMM 是“通用矩阵乘法”(General Matrix Multiply) 的缩写, 你先记住它就是“两个大表格相乘”——这是 AI 计算里最核心、跑得最多的一种运算; Flash Attention 则是大语言模型里一个关键的计算模块。这一步就是**真正上路开一圈**, 把前面学的全都用一遍。

末尾还塞了一整套“编译器内部”和“语言参考”, 相当于这台机器的**维修手册**——你不用一口气读完, 需要时翻回来查就行。

前置知识

前置知识: 完全没 GPU 基础? 那你来对地方了, 但别直接从第 1 章开始——请先花 20 分钟读第 0 章·写给纯新手: GPU & ML Infra 极简入门。

为什么非得先读第 0 章? 因为后面正文里会**满天飞**地出现一堆词: warp、内存层级、GEMM、Tensor Core……这些词在第 0 章里会被用最直白的大白话先讲一遍, 让你**心里先有个画面**。等你对它们有了感觉, 再读正文就会顺得多——不然就像没学过单词就去读文章, 每句都要查, 很容易劝退。

目录

下面就是全书的章节地图。每一行的**章节标题**点过去，是本仓库里对应的中文笔记；「**原文链接**」点过去，是 `mlc.ai` 的官方英文页面（想对照原书时再点）。
表格里“一句话简介”这一列，目前还塞了不少专有名词（`warp`、`TMA`、`Tensor Core`……）——**这些词你现在看不懂完全正常，不用慌**。它们都会在第 0 章和对应正文里从头讲起；这张目录只是让你心里先有个大致地图，知道每章大概在聊什么、彼此什么关系。

新手必读

章节	标题	一句话简介
00	写给纯新手:GPU & ML Infra 极简入门	零基础总入口：用大白话讲清线程层级 / 内存层级 / GEMM / Tensor Core / 重叠，先建立画面再读正文。

第一部分 · 硬件、内存与执行模型

章节	标题	一句话简介	原文链接
01	GPU 执行模型	从 SIMT、warp / CTA / cluster 到 Tensor Core、TMA、TMEM，建立现代 GPU 的整体心智模型。	原文
02	什么决定 Kernel 的速度	用屋顶线模型（算术强度定胜负）建立「先诊断、再优化」的工作流，核心杠杆是重叠。	原文
03	数据布局与其记号	形状 / 步长 / 视图 / 分块 / 命名轴 / swizzle 的统一记号，是后续所有 layout 讨论的语言。	原文
04	各代 GPU 的 Tensor Core 操作数布局	从 Ampere 到 Blackwell、MMA 操作数 / 累加器在寄存器与 TMEM 中的迭代布局演变。	原文
05	异步数据搬运:TMA	张量内存加速器：用张量映射描述符 + tile 坐标 + mbarrier 实现异步批量搬运与多级缓冲。	原文
06	特殊内存:TMEM(Tensor Memory)	Blackwell 专供 Tensor Core 累加的二维 (Lane × Col) 片上内存及其使用约束。	原文
07	异步协调:mbarrier	到达计数 + 相位位 + 字节计数三机一体的异步屏障，是所有流水线的协调中枢。	原文
08	进阶：集群启动控制 (CLC)	用持久化核 + 工作窃取式 tile 调度榨干尾部 SM，理解关键路径与尾部行为。	原文

第二部分 · TIRx 编程模型

章节	标题	一句话简介	原文链接
09	TIRx 入门	IR 层级的 GPU 编程 DSL, 用最小 GEMM 讲透 scope / layout / dispatch 三大要素。	原文
10	TIRx Layout API	把纸面布局记号变成编译器对象:TileLayout / SwizzleLayout / ComposeLayout。	原文

第三部分 · GEMM 实战 (Step 1→9)

章节	标题	一句话简介	原文链接
11	构建分块 GEMM(Step 1-3)	从最朴素的分块 GEMM 起步, 搭好寄存器 / 共享内存 / MMA 的基本骨架。	原文
12	用 TMA 为 GEMM 做流水线 (Step 4-6)	引入 TMA + mbarrier + 双缓冲 / 持久化 kernel, 让搬运与计算重叠起来。	原文
13	用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)	用 warp 专门化拆分生产者 / 消费者, 并借 cluster 跨 CTA 协作冲击峰值。	原文

第四部分 · Flash Attention 4

章节	标题	一句话简介	原文链接
14	Flash Attention 4	把前述全部机制 (TMA / TMEM / mbarrier / warp 专门化) 综合到一个完整的 FA4 内核中。	原文

第五部分 · 参考与附录

章节	标题	一句话简介	原文链接
15	参考资料 (附录)	一张「需求 → 去处」导航表, 告诉你查语言特性 / 看下降流水线 / 调内核时该翻哪一页。	原文
16	调试 Warp 专门化 Kernel	把内核看成一组「交接」, 用「角色 / 存储 / 交接 / 生命周期」四要素表对照生成的 CUDA 排错。	原文
17	编译器内部 (概览)	面向贡献者: 讲清 <code>tvm.compile(tir_pipeline="tirx")</code> 内部的下降流水线与关键阶段。	原文
18	TIRx Lowering 流水线	高层 TIRx 经 18 个模块级 pass 下降为 host/device 函数, 核心是 LowerTIRx。	原文

19	TIRx 语言参考 (概览)	语言特性分五块 (解析器工具 / 数据类型 / 缓冲区 / 控制流 / 内建) 的导读与速查。	原文
20	解析器工具 (Parser utilities)	T.meta_var / @T.inline / @T.meta_class / T.constexpr 四大解析期元编程工具。	原文
21	数据类型与表达式	dtype / type / Prim-Expr / 指针 / 向量化访存的语义与下降到 CUDA 的方式。	原文
22	缓冲区与内存	buffer = 指针 + 元数据; 静态 / 动态共享内存、竞技场分配、TMEM 列偏移。	原文
23	控制流	统一 vs 发散控制流、集体操作、elect_sync、各类同步与循环修饰符。	原文
24	CUDA C++ / PTX intrinsics	后端内建函数: 选举 / 具名屏障 / 栅栏 / 异步代理 / 蝶形归约等逃生口。	原文

建议阅读路径

每个人来这儿的起点都不一样, 所以我给你列了几条路。你先看看哪条最像自己, 照着走就行。

一句话先理解

一句话先理解: 如果你是”从没碰过 GPU 的程序员”, 那别犹豫, 直接走**路线 A**——从头老老实实读下去就好, 其他几条路是给已经有点底子的人留的。

- **路线 A · 从零开始打基础 (最适合纯新手, 也就是大多数读这个版本的你):** 先读 **第 0 章** 建立画面感, 然后 **01 一路读到 24**。怎么理解这个顺序? 第一部分 (01-08) 先把”硬件长什么样、代码在 GPU 上怎么跑”啃明白, 心里有个谱; 第二部分 (09-10) 学会怎么用 TIRx 这门语言把想法写成代码; 到了第三、第四部分 (11-14) 的实战, 再把前面学的那些零件一个一个拼起来, 造出真正能用的快程序。第五部分 (15-24) 你**不用从头读到尾**, 把它当成查字典的工具书, 卡住了再翻。
- **路线 B · 上来就想写 GEMM(给已经懂 GPU 的人):** 你要是 CUDA(NVIDIA 写 GPU 程序的传统语言) 和 GPU 架构本来就熟, 那别绕弯子, 直接跳到 **第 11 章**动手写”分块 GEMM”。写着写着碰到不认识的东西——比如 TMA、mbarrier、TMEM、warp 专门化 (这些词后面都会讲)——再回头翻 **第 05 到 08 章**补一补就好。
- **路线 C · 只想弄懂”性能优化大概是怎么个思路”:** 先读 **第 02 章**, 把”屋顶线模型”(一个判断程序到底卡在哪、是算得慢还是搬数据慢的分析工具) 和”重叠”(让搬数据和算数据同时进行, 别干等) 这两件事吃透, 养成”先诊断、再动手”的习惯; 然后看 **第 08 章**和 **第 13 章**, 慢慢就摸清怎么把硬件的性能压榨干净了。
- **路线 D · 想给 TIRx 或编译器本身贡献代码 (进阶):** 直接进 **第 17、18 章**, 先把编译器内部和”下降流水线”(把高层代码一步步翻译成 GPU 能执行的底层代码的过程) 看懂; **第 19 到 24 章**那套语言参考放手边, 要查语义随时翻。

- 要是你正在抓那种莫名其妙的 bug(数据被写坏、程序卡死不动、两段代码抢着读写同一块内存): 别慌, 先翻 **第 16 章** (调试 Warp 专门化 Kernel), 照着那张”四要素交接表”一项一项对过去——这类问题十有八九就是某个”交接”没对齐 (谁该先写、谁该等着读, 顺序乱了)。

提示

提示: 每章的核心概念都配了中英对照和一句话解释, 想查的话翻本仓库的 **术语对照表**就行——读着读着忘了某个词是啥, 去那儿一查最快。

版权与免责声明

- 原教材《Modern GPU Programming for ML Sys》的版权是 © 2026 MLC Community 的。
- 这个仓库是**非官方的中文学习笔记**, 东西都是社区里学习的人自己整理的, 难免有理解不到位的地方; 只要和原文对不上, **就以原文为准**。
- 这份笔记只用来学习和交流, 千万别拿去做商业用途。

第 0 章 · 写给纯新手:GPU & ML Infra 极简入门

本章是中文笔记新增的”地基”章, 原书没有对应章节。如果你会写代码 (Python / C++ / 后端 / 数据都行) 但几乎没碰过 GPU——没写过显卡程序、没听过 CUDA、不知道”并行”在硬件上到底是怎么回事——**请从这一章开始**。读完它, 你再翻后面任何一章, 那些满天飞的 warp、SMEM、tile、MMA 才会有画面感, 而不是一堆吓人的黑话。

我会假设你**完全没有 GPU 背景**: 每个新词第一次出现, 我都当场用大白话把”它是什么、为什么会有它”讲清楚, 绝不甩个英文缩写就往下跑。你只要能编程、肯动脑, 就一定能读懂。

本章要点

本章要点 (TL;DR)

下面这几条你现在看不太懂没关系, 这就是本章要逐条讲明白的东西。先扫一眼, 有个轮廓:

- GPU(图形处理器, 就是你电脑里那张”显卡”上的芯片) 不是”一颗更快的 CPU”, 而是一座**超大并行工厂**: 几万个很弱的小工人 (它们叫**线程**) 被编成班组一起干活。
- 三件事撑起整本书: **谁来干活** (线程是怎么分层组织的:thread / warp / lane / warpgroup / CTA / cluster)、**东西放哪** (数据放在哪种内存里: 寄存器 / 共享内存 / 全局内存 / 张量内存)、**到底在算什么** (矩阵乘 GEMM 和注意力 Attention 这两种运算)。
- GPU 跑得快的秘诀, 归根到底就一个词: **重叠 (overlap)**——让”搬数据”和”做计算”同时进行, 谁也别干等着谁。
- 这本书讲的是 NVIDIA(一家做 GPU 的公司) **最新的 Blackwell 架构**, 以及一种用来写 GPU 程序的语言 **TIRx**。这两个名字现在记不住完全没关系, 本章末尾会点一下。
- 这一章只求”听个大概、建立画面”。每个概念后面都有专门的章节细讲, 所以**看不懂的细节**先放过, 心里有个印象就行。

0.1 先说清楚: 这本书在讲什么, 你为什么需要先读这一章

这本书讲的是怎么把 GPU 榨干、写出”飞快”的计算程序——尤其是深度学习里最吃算力的两个运算: **矩阵乘法 (GEMM)** 和 **注意力 (Attention)**。

先解释一个会反复出现的词: **内核 (kernel)**。在 GPU 的世界里, ”kernel”不是操作系统那个内核, 而是指一段**专门跑在 GPU 上的函数/程序**。你在 CPU 上 (比如用 Python) 发起一次调用, 把活儿派给 GPU, 让那几万个线程一起跑这段代码——这一整段就叫一个 kernel。所以本书说的”写内核”, 你就理解成”写一段要在 GPU 上飞快跑的函数”。

问题在于: 原书默认你已经懂 GPU 是怎么工作的, 所以一上来就甩出 warp、共享内存、Tensor Core 这些词……对一个没写过 GPU 的人, 这就像没学过加减乘除就被拉去听微积分, 只会一脸懵。

所以这一章的任务很简单: **用大白话, 把后面会反复出现的那套基础词汇, 一次性给你讲明白**。不求深, 只求你心里有画面。读完这章, 你不会变成 GPU 专家, 但后面那些黑话你至少都”见过、知道

大概是干嘛的”。

0.2 一个贯穿全书的比喻:GPU 是一座” 超大工厂”

在讲任何术语之前, 我先给你一个贯穿全书的画面。后面每出现一个新概念, 你都可以把它挂回这座工厂里, 马上就有了着落。

把 GPU 想象成一座工厂, 你大概就抓住八成了:

- **工人 = 线程 (thread):** 线程就是” 一条正在执行的指令流”。数量极多 (一次能有几万个), 但每个都很弱、只会干很简单的活。和 CPU 那种” 少数几个强壮工人” 完全相反,GPU 是” 海量弱工人靠人多取胜”。
- **班组 = 线程的层级:** 这几万个工人不是一盘散沙, 而是 32 人一个**小班 (warp)**、几个小班一个**车间 (CTA)**……一层一层编起来, 方便分工和协作。
- **仓库 = 各级内存:** 数据存放的地方分好几档。有的” 仓库” 就在工位手边 (快但极小), 有的在车间角落 (中等), 有的在厂区外的大仓库 (巨大但很远、取一趟慢)。
- **专用机床 = Tensor Core(张量核):** 普通工人 (它们叫 **CUDA 核**) 啥都能干但做矩阵乘很慢; 而工厂里另有几台专门做” 矩阵乘” 的高速机床 (Tensor Core), 一开起来, 产量甩开普通工人一个数量级。
- **传送带 = 数据搬运引擎 (TMA):** 专门负责把原料从大仓库运到工位, 而且它运货的时候不占用工人的手——工人可以一边等货一边干别的。

这些词 (warp、CTA、Tensor Core、TMA……) 现在都只是先混个脸熟, 下面每一节会把它们一个个拆开讲。

关键

关键: 工厂效率高不高, 不在于某一个工人跑得多快, 而在于那几台高速机床别停。只要传送带能源源不断地把原料及时送到, 高速机床就能一直满负荷转。让机床不停转——这就是后面整本书一直在折腾的核心目标。

0.3 谁来干活: 线程的层级

一句话先理解

一句话先理解:GPU 把几万个线程像” 班级—小组—一个人” 那样**分层管起来**, 每一层都是为了让” 某个规模的协作” 变简单。

先说” 为什么要分层”。假设你手里有几万个工人, 你会怎么管? 肯定不会让他们一盘散沙地乱跑, 而是分班分组——这样” 两个人之间传句话” 和” 全班一起做件事” 才好安排。GPU 也一样: 有时两个线程要互相交换一个数, 有时一整个车间要共用一块数据。如果不分层, 这些协作就没法高效进行。所以 GPU 不把几万个线程当成一个扁平的大池子, 而是一层层嵌套地组织起来, 每一层都对应” 某个尺度上的协作”。

从小到大, 一共六层:

结构图

连接关系:

- Grid 网格 / (一次 kernel 的全部线程) → Cluster 集群 / (一组车间, 可跨多个 SM)
- Cluster 集群 / (一组车间, 可跨多个 SM) → CTA / Block 车间 / (跑在一个 SM 上, 调度的基本单位)
- CTA / Block 车间 / (跑在一个 SM 上, 调度的基本单位) → Warpgroup 班组 / (4 个 warp = 128 线程)
- Warpgroup 班组 / (4 个 warp = 128 线程) → Warp 小班 / (32 个线程, 同步执行)
- Warp 小班 / (32 个线程, 同步执行) → Thread 工人 / (最小执行单位)

逐个认识一下 (这张表是全书最该记住的一张, 建议反复回看):

名字	是什么	大白话
线程 / thread	最小的执行单位。它有自己的” 程序计数器”(记着自己跑到第几行) 和一小撮私有寄存器 (下文讲)	一个工人
lane(通道)	warp 里某个线程的编号 (0~31)	工人在小班里的座位号。注意:lane 不是一种新硬件, 它就是” 这个线程在它那个小班里排第几号”
warp(线程束)	32 个线程被硬件捆成一组, 同一时刻发同一条指令	一个 32 人小班, 动作整齐划一, 口令一响 32 个人同时迈步
warpgroup(线程束组)	4 个 warp = 128 个线程	4 个小班拼成的班组 (较新的架构才有这个概念)
CTA / block(线程块)	一批线程, 跑在同一个 SM(下面解释)上, 能共用一块” 共享内存”	一个车间
cluster(集群)	一组 CTA, 可以分布在不同 SM 上还能互相协作	几个车间联手
grid(网格)	一次 kernel 启动的所有线程	整个厂这一单活的全部人手

这里挑两个最容易绕晕的多说一句:

- lane(座位号) 有什么用? 既然一个 warp 里 32 个线程动作一模一样, 那怎么让它们各干各的一小份? 靠的就是 lane 号。每个线程用自己的座位号 (0~31) 去算” 我该处理哪一块数据”。打个比方:32 个人同时执行” 去搬第 N 个箱子”, 但每个人代入的 N 不同 (0 号搬第 0 箱、1 号搬第 1 箱……), 整体就把 32 个箱子一次搬完了。
- warp 为什么偏偏是 32? 这是 NVIDIA 硬件定死的数字, 你改不了, 记住就行。后面所有”32”几乎都和这个有关。

关键

关键: 这几层是” 谁装谁” 的包含关系, 但每一层具体装几个, 有的硬件定死、有的你自己定。硬性规定: 一个 warp 永远是 32 个线程, 一个 warpgroup 永远是 4 个 warp(= 128 线程)。可一个 CTA(车间) 里到底放几个 warp, 是你写 kernel 时自己定的——比如一个 128 线程的 CTA = 4 个 warp = 1 个 warpgroup; 一个 256 线程的 CTA = 8 个 warp = 2 个 warpgroup。所以别把 warpgroup 当成 CTA 和 warp 中间一个固定的硬层级, 它说白了就是” 把相邻的 4 个 warp 凑成一组” 的叫法, 方便某些指令一次指挥 128 个线程。

还有两个一定会撞见的词, 顺手认了:

- SM(流式多处理器 / Streaming Multiprocessor):GPU 里真正干活的” 车间厂房”。一颗 GPU 是由几十上百个 SM 拼出来的, 而上面说的一个 CTA(车间), 就跑在其中一个 SM 上。你

可以把 SM 理解成”GPU 内部的一个独立小处理器”，每个 SM 自带一批计算单元和它自己的那块共享内存。

- **SIMT(单指令多线程, 读作 sim-tee):** 这是描述 GPU 工作方式的核心词。意思是 warp 里 32 个线程发的是同一条指令, 但各自处理自己那份数据。换句话说, 你只写一份代码, 硬件让 32 个线程同时拿不同数据去跑它。这正是为什么 GPU 特别适合”对一大堆数据做同一种处理”(比如给一百万个数都加个 1)。

注意

注意:”32 个线程发同一条指令”并不等于”它们必须走同一个 if 分支”。硬件允许把某些 lane 临时”屏蔽”(停一下不动), 所以同一个 warp 里有的线程走 if、有的走 else 是可以的——只不过这时硬件得分两趟跑 (先跑走 if 的那批、再跑走 else 的那批), 比 32 个人步调一致要慢。这个现象叫分支发散 (branch divergence), 后面会提到, 知道”分支不一致会变慢”就够了。

0.4 东西放哪：内存的层级

一句话先理解

一句话先理解:GPU 的存储分好几层, 越靠近工人的越快但越小, 越远的越大但越慢; 写得快不快, 很大程度上就是看你有没有把数据放对层。

为什么内存要分层? 因为”又快又大”的存储造不出来 (造得出也贵得离谱)。所以硬件工程师妥协: 做一点点”飞快但极小”的, 再做一大块”巨大但慢”的, 中间再垫几层。你写程序时的本事, 就在于尽量让数据待在快的那几层, 少往慢的大仓库跑。这和 CPU 上”CPU 缓存命中率”的道理是一样的, 只不过 GPU 把这几层摊开来让你亲手管。

名字	在哪	谁能用	特点	工厂比喻
寄存器 / Register(RF)	紧贴计算单元	每个线程私有	最快、最小	工人手里攥着的几个零件
共享内存 / SMEM	SM 片上	一个 CTA 内共享	很快、不大	车间中间的公用工作台
全局内存 / GMEM(也叫 HBM)	显存	所有线程都能访问	很大、相对慢	厂区外的大仓库
张量内存 / TMEM	SM 片上 (Blackwell 新增)	给 Tensor Core 放累加结果用	专用	高速机床旁边的专属料台

挨个用程序员熟悉的东西打个比方:

- **寄存器:** 和 CPU 寄存器是一回事——CPU 上变量临时算一算放寄存器里最快。GPU 区别只在于每个线程都有自己的一份寄存器, 互不干扰。数量很少, 所以特别金贵。
- **共享内存 SMEM:** 你可以把它当成”一块要你手动管理的高速缓存”。CPU 缓存是硬件偷偷帮你缓存, 你管不着;GPU 的 SMEM 是你自己决定把哪块数据搬进来、给同一个车间 (CTA) 里的线程共用。它比大仓库快很多, 但只有几十 ~ 几百 KB, 放不下太多。
- **全局内存 GMEM(也叫 HBM, 显存):** 就是显卡上那几十 GB 的大内存, 所有线程都能读写, 容量大但路远访问慢。模型权重、输入数据一开始都躺在这里。

- **张量内存 TMEM**: 最新的 Blackwell 架构才有的一块小专用内存, 专门给高速机床 (Tensor Core) 放计算的中间结果, 这个 0.5 节会细说。

数据的典型流向就是: 从大仓库 (GMEM) 搬到工作台 (SMEM) → 工人/机床在快内存里计算 → 结果再写回大仓库 (GMEM)。整本书的优化, 几乎都绕着”怎么少搬、搬得巧、算得快”打转。

这里有两个”性能杀手”, 在 GPU 里极其要命、后面会反复出现, 先混个脸熟。它们都源于同一个事实: warp 里 32 个线程是一起去访存的, 所以它们要的地址”摆得齐不齐”会直接决定快慢。

- **访存合并 (coalescing)**: 当一个 warp 的 32 个线程同时去大仓库读数据时, 如果它们要的地址正好首尾相连 (连续), 硬件就能把这 32 次访问打包成 1 次搬完; 可如果它们要的地址东一个西一个, 就可能被拆成 32 次单独的访问。一次 vs 三十二次, 差几十倍。所以”让相邻线程读相邻地址”是 GPU 性能的头等大事。
- **bank 冲突 (bank conflict)**: 共享内存 (那块工作台) 在硬件上被切成 32 个并排的小格子, 叫”bank (存储体)”。32 个线程如果各读各的格子, 能同时进行, 飞快; 可如果好几个线程挤到同一个格子去读不同位置, 硬件就只能让它们排队一个个来, 慢。为了避开这种”挤同一个格子”的情况, 后面会专门去折腾数据在共享内存里的摆放方式 (术语叫”布局 layout”和”swizzle 换序”, 别急, 到时候会讲)。

关键

关键: GPU 编程里大量看起来很玄的”奇技淫巧”, 归根结底都在回答同一个问题——怎么把数据摆好, 让一个 warp 的 32 个线程读起来又快又不互相打架。

0.5 到底在算什么 (一): 矩阵乘 GEMM

一句话先理解

一句话先理解: 深度学习的算力几乎全花在”矩阵乘”上; 而大矩阵算不动, 就切成小方块 (tile) 一块块算。

深度学习里最核心、最吃算力的运算, 就是**矩阵乘法**。专业点叫 **GEMM** (General Matrix Multiply, 通用矩阵乘), 但本质就是你中学学过的 $C = A \times B$: 两个矩阵相乘, 得到第三个矩阵。

为什么说它是主角? 因为神经网络里你听过的那些层——全连接层、卷积、注意力——拆到底层, 绝大部分计算都是矩阵乘。所以”GPU 跑得快不快”, 很大程度上就等于”GEMM 跑得快不快”。本书后面一半篇幅, 都在教怎么把这一个运算榨到极致。

关键概念: tile (分块)。现实里的矩阵动辄 4096×4096 这么大, 一次根本算不完, 更塞不进前面说的那些”又快又小”的内存。怎么办? **切块**。把大矩阵切成一个个固定大小的小方块 (比如 128×128), 每次只搬进来一小块、算一小块, 算完换下一块。这个小方块就叫 **tile**。整本书你会看到无数次这个词——记住它就是”大矩阵切出来的、便于一次处理的小方块”就行。

还有一个公式你会反复看到: $D = A \cdot B + C$ 。别被它唬住, 它就读作”乘了再加”:

- A、B: 要相乘的两块小矩阵 (tile);
- C: 之前已经累加到现在的中间结果;
- D: 把这一步的乘积加上 C 之后的新结果。

为什么非得”加上 C”？这是切块带来的必然结果。回想矩阵乘的算法：结果矩阵里的每个位置，都等于一行乘一列、把一串乘积加起来。当我们把矩阵切成小块、一块一块算时，**最终某个位置的值，就是好多小块的乘积一点点累加出来的**。所以每算完一块，都要把它”加进”之前的累计值里。

那个从头累到尾、一直在被加的 C/D，就叫**累加器 (accumulator)**——你可以理解成”一个一直在累加的求和变量”，只不过它是一整块矩阵。

累加器放哪儿，其实是门大学问。它往往不小，而且要从第一块一路累到最后一块，**全程都得占着存储空间**。要是让它一直霸占着每个线程那点**本来就稀缺的寄存器**，工人手里就腾不出地方干别的活了。所以最新的 Blackwell 架构干脆给它单开了一块专用片上内存——就是 0.4 里提到的 **TMEM**，专门用来摆累加器，把宝贵的寄存器解放出来。（这条线索，第 6 章会细讲，现在有印象即可。）

0.6 到底在算什么 (二):Tensor Core 与 MMA

一句话先理解

一句话先理解:GPU 里有一种专做矩阵乘的”高速机床”叫 **Tensor Core**，它一口吞下两块 tile、直接吐出乘加结果；让它别闲着，就是性能的命门。

上一节说矩阵乘是主角。那 GPU 是用什么去算它的？这里要区分两类”算手”：

- **CUDA 核 (CUDA Core)**: 通用的标量计算单元，什么都能算（加减乘除、判断大小、算地址），相当于工厂里的普通工人。灵活，但让它一个数一个数地去拼矩阵乘，**慢**。
- **Tensor Core(张量核)**: 专门为矩阵乘打造的固定功能硬件——就是那台高速机床。”固定功能”意思是它只会干一件事（矩阵乘加），但干这件事快得离谱：它一次就能吞下两块 tile，直接算出 $D = A \cdot B + C$ ，吞吐量比一堆 CUDA 核高一个数量级以上。

Tensor Core 干的这个”小块矩阵乘加”动作，有个专门的名字叫 **MMA**(Matrix Multiply-Accumulate, 矩阵乘累加)——其实就是上一节那个 $D = A \cdot B + C$ ，只不过强调”由 Tensor Core 一气呵成地做”。你后面还会撞见 **wgmma**、**tcgen05** 之类的词，它们是**不同代 GPU 上发起 MMA 的具体指令名**；现在你只需要在心里翻译成”哦，这是在让 Tensor Core 算一次矩阵乘”就够了，不必记。

关键

关键: 现代 GPU 之所以能跑得动大模型，核心功臣就是 Tensor Core。后面大半本书的所有折腾，本质都是为了一个目的——**让 Tensor Core 一刻不停地有数据可算，别让这台贵机床闲着**。

0.7 怎么喂数据: 异步搬运、流水线与”重叠”

一句话先理解

一句话先理解:GPU 提速的终极武器是**重叠**——让”搬数据”和”做计算”同时进行，而不是搬完了再算、算完了再搬。

Tensor Core 这台高速机床再快，**没数据可算也只能干瞪眼**。而数据躺在又远又慢的大仓库 (GMEM) 里，搬过来要花时间。所以”怎么及时把数据喂上去”和”怎么算”一样重要，甚至更重要。下面这几个词，就是 GPU 用来解决”喂数据”问题的工具：

- **同步 vs 异步**: 这是关键的思路转变。**同步**的老办法是: 让计算线程自己跑去搬数据——可它弯腰搬货的那段时间就没法算了, 机床只能停着等, 白白浪费。**异步**的新办法是: 线程只是”下个搬运指令”(吩咐一声”把这块货运过来”), 然后**立刻回头继续算**, 搬货的脏活交给专门的硬件去干。”异步”在这里就是”我吩咐完不用等它做完, 接着干自己的”。
- **TMA(张量内存加速器 / Tensor Memory Accelerator)**: 就是上面说的那个专门搬货的硬件引擎 (工厂里的传送带), 负责在大仓库 (GMEM) 和工作台 (SMEM) 之间**成批、异步**地搬 tile, 搬的时候完全不占用工人的手。
- **mbarrier(异步屏障)**: 既然搬运是异步的、不等它做完就走了, 那怎么知道”货到底搬到没、能不能用了”? 总得有个通知机制。mbarrier 就是这个”**信号灯**”: 搬运引擎搬完了就把灯点亮, 计算方看到灯亮才开始用这批数据。它负责在”搬的人”和”算的人”之间对暗号, 避免有人拿到还没搬完的半成品。
- **流水线 / 重叠 (pipeline / overlap)**: 这是全书的灵魂。先想想不重叠会怎样: 如果老老实实”搬完这块 → 算完这块 → 写完这块 → 再搬下一块”一步步串着来, 那同一时刻永远只有一个环节在忙, 其它都在干等, 机床的利用率低得可怜。重叠的做法是让它们**像工厂流水线一样错开同时**进行: 这会儿工人在算第 k 块的时候, 传送带已经在搬第 k+1 块了, 收尾环节还在写第 k-1 块的结果。三件事同时进行, 谁也不等谁, 整体自然快得多。

结构图

分组:

- **串行**: 一个等一个 (**慢**): 「搬第 k 块」 「算第 k 块」 「写第 k 块」 「搬第 k+1 块」
- **重叠**: 三个引擎同时忙 (**快**): 「搬 k+1 块」 「算 k 块」 「写 k-1 块」

关键

关键: 记住这一句, 后面就不会迷路——慢内核和快内核的差距, 几乎全在”**重不重叠**”上。

0.8 注意力 (Attention) 是什么, 为什么在硬件上难

一句话先理解

一句话先理解: 注意力 = 两次矩阵乘, 中间夹一个 softmax; 难点是中间那个超大的”分数矩阵”既占地方又难算, 得想办法别把它整个存出来。

如今的大语言模型基本都是 **Transformer** 架构 (你只需知道这是当前主流大模型的”骨架”), 而它的核心运算就是**注意力 (Attention)**。抛开数学公式, 它干的事可以这么理解: 对句子里的每一个词, 都去打量序列里所有其他词, 算出”对当前这个词来说, 该多关注谁、少关注谁”, 然后照这个”关注程度”把大家的信息加权汇总成一个新表示。

落到计算上, 它就是**两次矩阵乘, 中间夹一个 softmax**:

1. $S = Q \cdot K^T$: 第一次矩阵乘, 算出一个**分数矩阵 (score matrix) S**, 里面每个数代表”某个词对另一个词的**关注分数**”。它的形状是 $n \times n$ ——所以序列一长, 这个矩阵就**大得吓人** (比如 8000 个词, S 就是 8000×8000)。
2. **softmax(S)**: 把 S 的**每一行**分数, 换算成一组**权重**。具体做法是: 这一行每个分数先取指数 $\exp$ (指数运算会把大的拉得更大), 再各自除以”这一行所有 $\exp$ 值的总和”。这么一弄, 效果是——分

数大的被进一步突出、所有权重都变成正数、而且一行加起来正好等于 1。因为”加起来等于 1”，所以叫**归一化**；你可以直接理解成”把一行分数换算成百分比占比”。

3. $O = \text{softmax}(S) \cdot V$: 第二次矩阵乘, 用上一步那组”百分比权重”去把信息加权汇总, 得到最终输出 O 。

难点到底在哪? 就在中间那个巨大的分数矩阵 S 。如果老老实实把它整个存进显存, 又占地方又慢 (8000×8000 个数可不少)。更麻烦的是 softmax 这一步本身: 它要除以”一整行的总和”, 而且为了计算稳定, 还得先找出这一行的最大值。这就好像逼着你**必须先把一整行的分数全算出来、凑齐了**, 才能开始做归一化——那不就等于非得把整个 S 都存下来吗? 这正是矛盾所在。

著名的 **Flash Attention**(一种高效注意力算法) 就是来破这两个难的。它的核心思路是: **坚决不把完整的 S 存下来**, 而是把 K 、 V 一块一块地**流式喂进来**, 边算边维护两个”running(运行中)的小统计量”——目前为止见过的最大值、目前为止的累计总和; 每来一块新数据, 就用这两个统计量把之前算的结果**按比例修正一下**。这样既不用存整个 S , 最后又能得到和”老老实实做完整 softmax”一模一样的答案。这套技巧怎么在硬件上高效实现, 正是本书第 14 章的高潮。

现在你只需记住一句: **注意力 = 两个矩阵乘 + 中间一个 softmax**, 而工程上的关键是想办法别把那个超大的分数矩阵真的存出来。

0.9 这本书的两个”新东西”:Blackwell 与 TIRx

最后两个名字, 扫个盲就行, 记不住也不影响读前几章:

- **Blackwell**:NVIDIA 最新一代 GPU 架构 (代表显卡如 B200)。这里的”架构”指的是一代芯片的硬件设计代号。GPU 是一代代往上演进的:**Ampere**(代表卡 A100)→ **Hopper**(H100)→ **Blackwell**(B200), 每一代都会多出一些新硬件能力 (比如 Hopper 那代带来了前面说的 TMA 和 warpgroup,Blackwell 这代带来了 TMEM)。本书专门讲 Blackwell, 所以你会读到不少”只有最新卡才有”的特性——别拿它去套老显卡。
- **TIRx**: 本书用来写 GPU 内核的**编程工具/语言**。它是一种 **DSL**——DSL(Domain-Specific Language, 领域专用语言) 就是”为某个特定领域量身定做的小语言”, 这里 TIRx 是嵌在 Python 里写的 (随开源项目 Apache TVM 一起发布)。它最大的特点是: **让你直接、明确地点名硬件** (比如”这块数据放进 SMEM”)”这一步交给 Tensor Core 算”), 而不像 PyTorch 那种高层框架把硬件细节全替你藏起来。正因为它把硬件摊开给你控制, 才适合用来榨干 GPU。第 9 章开始会专门讲它。

0.10 术语速查表 (读后面遇到生词就回来翻)

English	中文	一句话理解
thread	线程	一个最小的工人
lane	通道	线程在 warp 里的座位号 0~31
warp	线程束	32 个线程的小班, 同步执行
warpgroup	线程束组	4 个 warp = 128 线程
CTA / block	线程块 / 车间	跑在一个 SM 上的一批线程
cluster	集群	可跨 SM 协作的一组 CTA
grid	网格	一次 kernel 的全部线程
SM	流式多处理器	GPU 里真正干活的”车间/厂房”
SIMT	单指令多线程	一个 warp 同发一条指令、各算各的数据

Register / RF	寄存器	每个线程私有，最快最小的存储
SMEM	共享内存	一个 CTA 内共享的片上快内存
GMEM / HBM	全局内存 / 显存	所有线程可见，又大又慢
TMEM	张量内存	Blackwell 给 Tensor Core 放累加器的专用内存
coalescing	访存合并	连续地址的访问合并成一次，极大提速
bank conflict	bank 冲突	多线程挤同一存储体，被迫排队
GEMM	通用矩阵乘	$C = A \times B$ ，深度学习的算力主角
tile	分块 / 小方块	大矩阵切出来的固定大小小块
MMA	矩阵乘累加	Tensor Core 干的话： $D = A \cdot B + C$
accumulator	累加器	一直累加中间结果的那块数据
Tensor Core	张量核	专做矩阵乘的高速固定功能硬件
CUDA Core	CUDA 核	通用标量计算单元，啥都能算但慢
TMA	张量内存加速器	异步成批搬运 tile 的专用引擎
mbarrier	异步屏障	协调异步搬运/计算的“信号灯”
overlap / pipeline	重叠 / 流水线	让搬、算、写同时进行，快的关键
epilogue	收尾阶段	算完后把结果写回去的那一段
Attention	注意力	两个矩阵乘夹一个 softmax
swizzle	换序摆放	打乱数据摆放以避开 bank 冲突
Blackwell	(架构代号)	NVIDIA 最新一代 GPU 架构
TIRx	(编程语言)	本书用来写 GPU 内核的 Python 内嵌 DSL

提示

提示: 更全的术语对照见术语对照表。

0.11 该怎么读这本书

- **完全没基础**: 先把这一章看完 (细节不懂没关系), 再从 **第 1 章** 顺着往下读。每读到一章卡壳, 回这里的速查表瞄一眼。
- **会一点 CUDA**: 可以快速扫过本章, 直接进第 1 章; 遇到 Blackwell 专属的 TMA / TMEM / warpgroup 再回头查。
- **只想看 GEMM / Attention 实战**: 也建议先看完本章和第 1、2 章, 否则第 11-14 章会很吃力。

完整的阅读路线见首页 README。

小结

这一章不指望你记住所有细节, 只要在脑子里搭起三根支柱就够了:

1. **谁干活**: 线程被编成 warp(32)→ warpgroup(128)→ CTA(车间)→ cluster, 跑在一个个 SM 上。
2. **东西放哪**: 寄存器 (私有最快)→ 共享内存 SMEM(车间共用)→ 全局内存 GMEM(又大又慢), 外加 Blackwell 的 TMEM。
3. **在算什么、怎么算快**: 核心是矩阵乘 GEMM 和注意力 Attention; 让 Tensor Core 不停转、让搬运和计算**重叠**, 就是全书的主线。

带着这三根支柱, 翻开第 1 章吧——你会发现那些黑话, 忽然就有画面了。

延伸阅读

- 下一章: 第 1 章 · GPU 执行模型 (把本章的”谁干活、东西放哪”讲得更细)
- 速查: 术语对照表

第 01 章 · GPU 执行模型

原文:GPU Execution Model —Modern GPU Programming for ML Sys

本章要点

本章要点 (TL;DR)

别被下面这几个英文缩写吓到——它们都会在正文里一个个用大白话讲清楚。这里先放着，等你读完回头再看，会发现每条都懂了：

- GPU 里有**成千上万个线程** (thread, 就是“一条独立的执行流”，你可以先把它想成一个最小的“干活的小工”)。GPU 不是把它们一锅平铺，而是套了好几层、组成一个**嵌套层级**：线程 → 线程束 (warp) → 线程束组 (warpgroup) → CTA → 集群 (cluster) → 网格 (grid)。每多一层，都是为了让“某个范围内的小工互相配合”这件事变得更便宜。
- GPU 上跑的那段程序叫**核函数 (kernel)** (可以先理解成“一段会被几千个线程同时各跑一份的函数”)。一个 kernel 干的活，说穿了就是把数据在好几种**内存**之间来回搬、临时存：寄存器 (RF)、共享内存 (SMEM)、全局内存 (GMEM)，还有 Blackwell 这代新加的张量内存 (TMEM)。最典型的搬运路线是 $GMEM \rightarrow SMEM \rightarrow (\quad) \rightarrow \quad \rightarrow SMEM \rightarrow GMEM$ 。
- GPU 上算数的硬件分成两类：通用的 **CUDA 核 (CUDA core)**，管的是算下标、逐个元素加减、求和、if/else 这类杂活；**张量核 (Tensor Core)** 是块“专用电路”，一条指令就能算完一整块矩阵的 $D = A \cdot B + C$ ，算矩阵的速度比 CUDA 核快十倍以上。
- 还有专门负责**搬数据**的硬件 (比如 TMA)，它的活儿就是把数据喂给上面那些算数的硬件。搬数据的和算数的，是**两套独立的硬件，各干各的，谁也不用等谁** (这就叫“异步”)。
- 把这些零件拼成一条矩阵乘 (GEMM) 的流水线之后，程序快慢的全部差距，就落在一个词上——**重叠 (overlap)**：让搬数据、算数、收尾 (epilogue) 这三摊活儿**同时进行**，而不是排队一个等一个。

前置知识

前置知识：这是全书的基础章，几乎不需要任何 GPU 背景。你只要会写代码 (Python、C++、后端、数据，哪种都行)，知道“矩阵乘法”是什么 (就是中学线性代数里那个 $A \cdot B$)，就能往下读。至于 GPU 是怎么并行的、内存为什么分这么多层、矩阵乘为什么是主角——这些**本章会从零讲起**，不假设你碰过显卡、CUDA 或者任何硬件。要是连“GPU 大致在干嘛”都还没概念，可以先翻一下第 0 章·极简入门；不翻也行，本章会兜住你。

还有一点先说在前头：这一章会冒出一堆英文缩写 (SM、warp、CTA、TMEM……)。别**有压力**——每个词第一次出现时，我都会停下来用大白话把“它是啥、为什么要有它”讲明白，讲完了后面才接着用。你不需要提前背任何东西。

这是全书第一部分「硬件、内存与执行模型」的头一章。它就想干一件事：在你脑子里画出一张地图。

怎么画？分两步。第一步，把四块拼图——**执行层级** (线程是怎么分组的)、**内存空间** (数据存在哪)、**计算引擎** (谁来算)、**搬运引擎** (谁来搬数据)——**一块一块讲明白**。第二步，用一条 GEMM 流

水线把这四块拼图**串成一条线**，让你亲眼看见数据是怎么在硬件里一站一站流动、各个零件又是怎么配合的。

一句话先理解

一句话先理解: GEMM (General Matrix Multiply, 通用矩阵乘) 就是“算 $A \times B$ 这样两个矩阵相乘”。为什么它这么重要、值得一整章的流水线来讲它？因为深度学习模型里绝大部分的计算量，本质上都是一堆矩阵乘。把矩阵乘搞快了，模型就快了。所以全书都拿它当主角。

关键

关键: 全书就围着一个核心观点转——后面几乎所有的优化技巧，说到底都在回答同一个问题：“怎么把活儿合理地铺到这几块拼图上？”所以别把这一章当成可看可不看的背景。它是后面每一章都要用的一套词汇和坐标系，先打好这个底，后面才走得顺。

先看一眼 GPU 硬件长啥样。

一句话先理解

一句话先理解: 你可以把一颗 GPU 想成一座工厂，工厂里有几十个一模一样的“车间”。每个车间自己就能独立干活——有自己的工人、自己的小仓库、自己的机床。GPU 里这个“车间”，有个正经名字叫**流式多处理器 (Streaming Multiprocessor, 简称 SM)**。”多处理器”是因为一个 SM 里塞了一大把能并行干活的小单元。整章我们基本都盯着一个 **SM 内部**看，搞懂一个，就懂了全部 (几十个 SM 长得都一样)。

下面这张图，就是 Blackwell 这代 GPU 里一个 SM 的”地图总览”。图里那些名字 (warp、SMEM、TMEM、Tensor Core、TMA……) 你现在一个都不用认识——它们后面都会一个个讲。现在你只要有印象：**一个 SM 内部分成四摊东西**——能干活的执行资源、片上的几种内存、算数用的计算引擎、专门搬数据的搬运引擎。先扫一眼，有个轮廓就行：

结构图

分组：

- **Blackwell SM (流式多处理器)**
- **执行资源:** 「warp $\times 4 = \text{warpgroup}$ 」 「warp $\times 4 = \text{warpgroup}$ 」 「...(多个常驻 CTA)...」
- **片上内存:** 「寄存器文件 RF (每线程)」 「共享内存 SMEM (每 CTA)」 「张量内存 TMEM (每 CTA)」
- **计算引擎:** 「CUDA 核 (通用 SIMT ALU)」 「Tensor Core (tcgen05, 块粒度 MMA)」
- **搬运引擎:** 「TMA (批量异步拷贝)」 「GMEM $\rightleftharpoons$ SMEM」

注意

注意: 一颗 GPU = 一大堆这样的 SM 拼在一起 (回到工厂比喻，就是一座工厂里几十个车间)。这些 SM 共用一块超大的”中央仓库”，叫**全局内存 (HBM)**——后面会细讲，现在只要知道它”大但是远、所有 SM 都能用”就行。这一章我们重点看两件事：**单个 SM 内部**怎么运转，以及**几个 SM 凑在一起协作** (这种组合后面叫”集群”)时是怎么配合的。

一、执行层级：为什么不是“一锅线程”

1.1 嵌套层级的设计动机

一句话先理解

一句话先理解: GPU 不是把几千个线程“一锅平铺”地摆在一起, 而是把它们**套了好几层** (就像公司分成总部 → 部门 → 小组 → 个人)。这一节就讲清楚: 为什么要分层?

先说“线程”这个词。**线程 (thread)** 就是一条独立的执行流——它有自己的进度 (执行到第几行了)、自己的临时变量。你写后端代码时多半见过“多线程”, GPU 的线程也是这个意思, 只不过 GPU 一上来就是几千上万个线程一起跑。

很多人第一反应是: 那这几千个线程, 大概就是平铺着、谁跟谁都一样吧? **还真不是**。GPU 把它们套成了一个嵌套的层级。

干嘛这么费劲? 因为**线程之间要互相配合 (协作), 但配合的“远近”差别很大**。打个比方, 就像同事之间传话:

- 挨着坐的两个人, 扭头说一句就行——这是很“近”的协作。
- 一个小组里十几个人, 凑在一张小白板前对一下数据——这是中等距离。
- 两个不同楼层的人想共用同一份资料——这就“远”了, 得跑一趟或者走个共享盘。

具体到 GPU 上对应这么几种:

- 挨着的两个线程, 想直接交换一下各自寄存器里的值——很“近”。(寄存器后面会讲, 先理解成“线程自己手边最快的小存储”。)
- 同一组里的 128 个线程, 想合用一小块暂存数据——中等距离。
- 趴在两个不同 SM(车间) 上的两组线程, 想共享**操作数**——这就远了。(operand / 操作数 = 参与运算的输入数据, 比如矩阵乘 $A \times B$ 里的 A 和 B。)

关键就在这: 要是所有线程都平铺、没有层级, 硬件就只能拿“最贵、最慢的那套”通信手段去应付所有情况——哪怕两个线程其实就挨在一起, 也得走“跨楼层”那套流程, 纯属浪费。GPU 反过来: **分层, 每一层专门把“某个距离上的配合”做得又快又省**。挨着的走最快的近路, 隔得远的才走慢路。下面这张图, 就是 Blackwell 上从大到小整个层级的样子 (每个名字下面都标了它大概是多少线程):

结构图

连接关系:

- Grid 网格 / (整个 kernel 的所有线程) → Cluster 集群 / (一组协作的 CTA, 可跨 SM)
- Cluster 集群 / (一组协作的 CTA, 可跨 SM) → CTA / Thread Block / (硬件调度的基本单位, 跑在单个 SM 上)
- CTA / Thread Block / (硬件调度的基本单位, 跑在单个 SM 上) → Warpgroup 线程束组 / (连续 4 个 warp = 128 线程)
- Warpgroup 线程束组 / (连续 4 个 warp = 128 线程) → Warp 线程束 / (32 个线程, SIMT 执行)
- Warp 线程束 / (32 个线程, SIMT 执行) → Thread 线程 / (标量执行单元, 有独立 PC 和寄存器)

1.2 逐层拆解

我们从最小的”一个线程”起步，一层一层往上爬。下面这张表每一行就是一层。第一次读不用死记，看懂每一层”是干嘛的”就够了，后面用到哪个会再提醒你：

层级	规模	关键特性 (大白话)
线程 (thread)	1	最小的”干活小工”。它有自己的 程序计数器 / PC (就是”我执行到第几行了”的书签) 和自己的 寄存器 (register——线程私有、手边最快的一小块存储，类似 CPU 寄存器，但 GPU 里 每个线程各有一份)。它在自己所属的 warp 里有个 0-31 的编号，叫 lane ID / 通道号 (就当它是”队列里的座位号”)。
线程束 (warp)	32 个线程	GPU 真正的”最小调度单位”——硬件不是一个一个发指令，而是 32 个线程一起、同步执行同一条指令 。这种模式叫 SIMT (single instruction, multiple threads, 单指令多线程)。打个比方:warp 像一个 32 人的方阵，口令一喊,32 个人同时迈同一步 。但每个人脚下踩的数据 (寄存器) 是各自的，而且可以让其中某些人”这一步先别动”(这叫屏蔽 / mask)。
线程束组 (warpgroup)	4 个 warp = 128 线程	把 4 个 warp 绑成一组一起干。Hopper 这代引入，专门用来发起一种”一大块矩阵乘”的指令 (下文的 wgmma)。到了 Blackwell 它又多了个活儿: 128 个线程合伙搬 TMEM (把一块累加器数据搬进/搬出寄存器)。先记住”warpgroup = 4 个 warp 合伙干大活”就行。
CTA(Cooperative Thread Array, 就是 CUDA 里常说的 thread block / 线程块)	多个 warp	硬件调度的基本单位 ——你可以理解成”一个 CTA 是一整批被一起派到某个车间 (SM) 去干活的线程”。一个 CTA 整个跑在 单个 SM 上，并独占该 SM 里的一块共享内存。一个 SM 上可以同时住好几个 CTA，这时它们就得 平分 这个 SM 的共享内存。
集群 (cluster)	一组 CTA	把几个 CTA 绑成一伙。特别之处： 这几个 CTA 可以分布在不同的 SM(车间) 上，还能互相同步、互相读写对方的共享内存——这个”跨车间互相读写”的能力，后面会专门讲，叫分布式共享内存 (DSMEM)。
网格 (grid)	全部	一个 kernel 启动后 产生的所有线程的总和 ，就是一个 grid。它是最大的那一层。

一句话先理解

一句话先理解: 上表里 **SIMT** 这个词最容易让人误会，下面这段专门掰扯清楚——它讲的是”warp 里 32 个线程到底能不能各走各的 if/else 分支”。

关键

关键:warp 里 32 个线程”一起发射同一条指令”，听起来好像它们必须齐步走、连 `if/else` 都只能一起走同一边——**其实不用**。因为每个线程有自己的寄存器、还能被单独”屏蔽”掉，所以同一个 warp 里，不同线程完全可以走不同分支。代价是什么？**硬件没法真的同时跑两条分支**，只能**轮着串行来**：先跑 `if` 这边（把走 `else` 的那些线程暂时冻住），再跑 `else` 这边（反过来冻住走 `if` 的）。换句话说，分支一发散，这个 warp 的速度就会变慢——这在 GPU 里叫”分支发散（divergence）”，是个常见的性能坑。正是”允许 warp 内部各走各路”这一点，把 SIMT 和老式的 SIMD（那个是真的全员锁死、一步都不能差）从根上区分开了。

1.3 ”作用域 (scope)”：同一个操作由谁来发起

一句话先理解

一句话先理解：这一节讲一个贯穿全书的概念——**scope(作用域)**。它回答的问题特别简单：”某件事，到底是由几个线程来发起的？一个？一队 32 个？还是一组 128 个？”听着不起眼，但它直接决定了你的代码要怎么写。

想读懂 Blackwell，这个观点得先记牢。

早期的 GPU 上事情很简单：一个 kernel 里的各种操作，基本都是”同一组线程”在干。可到了 Blackwell，玩法变了——**不同操作，改由不同规模的线程来发起**。为什么要这么搞？因为**每种活儿都有它最顺手的人数**：有的活儿一个线程做就够了（人多了反而添乱），有的活儿（比如搬一大块数据）得拉上整整 128 个线程一起上才划算。下面这张表举了四个例子（表里的术语后面都会细讲，现在只看最右边”由谁来干”）：

操作	由谁来干（这就是它的 scope / 作用域 ）
TMA 拷贝 （TMA = 一个专门搬数据的硬件，负责在大仓库 GMEM 和小仓库 SMEM 之间批量搬运，后面细讲）	一个线程 喊一嗓子发起，剩下的搬运交给专门硬件去完成。
把 TMEM 里的数据加载到寄存器	整组 warpgroup(128 线程) 一起干：4 个 warp 各搬一片。
tcgen05 矩阵乘指令	由一个被选中（ elected ）的线程来提交。（elected = 硬件从一组线程里”选一个代表”出来干这件事，避免大家抢着干、重复发指令。）
跨 2 个 CTA 的矩阵乘	一次动作横跨两个 CTA （也就是跨了两个车间）。

”某个操作具体由哪组线程来发起”，书里给这件事起了个名字，就叫这个操作的 **scope(作用域)**。后面只要碰到一个新操作，你的第一反应就该是问：”它的 scope 是谁？”

关键

关键:scope 是全书反复出现的三大设计要素之一——**作用域 / scope、布局 / layout、派发 / dispatch**。往后每碰到一个 GPU 操作，你都应该先问一句：”它的 scope 是啥？单个线程？一个 warp？一个 warpgroup？还是跨 CTA？”把这个想明白了，你才知道同步该怎么写、数据该怎么分。layout 和 dispatch 是另外两位，后面的章节会一点点补齐。

二、内存空间：容量与速度的取舍

2.1 为什么有这么多种内存

一句话先理解

一句话先理解:GPU 不是只有一种内存, 而是搞了好几种, 从” 大但慢” 到” 小但快” 排成一队。这一节讲清楚: 为什么非得这么折腾?

线程算得再快, 手上没数据也只能干等着。所以光有” 算数的硬件” 没用, 还得有地方放数据、而且得让数据**离计算单元够近**(近 = 取数据快)。

这里有条躲不开的物理铁律: **没有哪种存储能又大又快**。又大又快又便宜的内存不存在——你想要大容量, 它就慢; 你想要快, 它就只能做得很小。这跟你电脑上” 硬盘大但慢、内存小但快、CPU 缓存更小更快” 是一个道理。既然没有完美的内存,GPU 干脆**多备几种**, 每种在” 大” 和” 快” 之间占一个不同的位置, 按需取用。所以, 一个 kernel 干的活, 核心就是**把数据在这几种内存之间倒腾**: 平时放在大而慢的里, 真要算了就提前挪到小而快的里。

下面这张表, 从” 最大最慢” 到” 最小最快” 列了 GPU 的四种内存。” 归属” 那列说的是” 这块内存归谁用”(整颗 GPU 共用? 一个 CTA 独占? 还是每个线程各一份?):

内存空间	归谁用	角色	说明 (大白话)
全局内存 (GMEM)	整颗 GPU 共用	数据的” 长期仓库”	容量最大 (就是那块 HBM 大仓库), 但离计算最远、取一次最慢。所有 SM 都从这里拿数据。
共享内存 (SMEM)	每个 CTA 一块 (在单个 SM 内)	一小块数据的” 工作台”	快得多的片上小存储, 像一块 自己手动管理的高速缓存 (scratchpad / 便笺本)。B200 上每个 SM 最多 228 KB——很小, 得省着用。
张量内存 (TMEM)	每个 CTA 一块	放矩阵乘的” 累加器”	Blackwell 新增, 专给张量核 (tcgen05) 用。(累加器 / accumulator: 矩阵乘是一块块累加出来的, 这块地方就专门存” 算到一半的中间和”。下节细讲。)
寄存器文件 (RF)	每个线程各一份	线程手边最快的小存储	最快, 但也最小。放线程自己的临时变量、还有收尾阶段 (epilogue) 要用的值。(fragment / 片段: 一块大数据切给单个线程, 暂存在它寄存器里的那一小片。)

这张表别当成四个互不相干的条目看。**你从上往下顺一遍, 会发现它其实悄悄画出了一条路线**——数据从最远最慢的 GMEM 出发, 一步步往计算单元身边凑 (GMEM → SMEM → 寄存器), 算完再原路退回去存好。本书里几乎每个 kernel 的数据走向, 都是这个套路:

结构图

要是这个 kernel 用了张量核 (那个算矩阵超快的专用电路), 那 **TMEM 就卡在这条路的中间**——计算正在进行的时候, 那个” 算到一半的累加器” 就停在 TMEM 里。

2.2 重点理解 TMEM:Blackwell 独有的设计

一句话先理解

一句话先理解:TMEM 是 Blackwell 这代全新的一种内存。它存在的全部理由,就是为了把那个又大又占地方的”矩阵乘累加器”从寄存器里挪出来。这一节讲它为什么值得单独造一块内存出来。

四种内存里,前三种(GMEM、SMEM、寄存器)在别的硬件上多少都有对应物,唯独 TMEM 是 Blackwell 全新的,以前压根没有。完整细节留到后面「Tensor Cores: tcgen05」那章,但它当初为啥被造出来,现在就值得搞明白——这背后是一笔很经典的取舍。

先搞清楚它要解决啥问题。矩阵乘是”一块块累加”出来的:算一块、加进结果里,再算一块、再加进去……那个一直存着”加到一半的结果”的地方,就叫累加器(accumulator)。早期 GPU 把这个累加器直接塞进寄存器里。麻烦就出在这儿——

1. 累加器本身很大(矩阵乘的中间结果不小)。
2. 而寄存器是 GPU 上极其稀缺的资源(还记得吗?每个线程才分到那么一点点)。
3. 于是累加器占得越多,能匀给别处用的寄存器就越少。
4. 寄存器一旦不够用,一个 SM 上能同时塞下的线程数就被压低了。

第4点要多说一句:一个 SM 能同时”住”多少线程,这个数叫占用率(occupancy)。占用率高有什么好?线程一多,某些线程在等数据时,硬件就能立刻切去跑别的线程,把等待的时间填满,SM 就不容易闲着。所以”累加器吃光寄存器 → 占用率被压低 → SM 利用不充分”——这是个甩了好多年都甩不掉的老瓶颈。

Blackwell 的解法:把 tcgen05(第五代张量核的指令)算出来的累加器,从寄存器里搬出去,挪到一块新内存 TMEM 里。TMEM 是每个 CTA 一块的”二维便笺本”,规格是 128 行(lane)×最多 512 个 32 位列,物理上就长在 SM 上。这样寄存器就解放出来了。

当然天下没有免费的午餐,代价是:kernel 进入收尾阶段之前,你得自己写代码把 TMEM 里的数据显式读回寄存器——硬件不会自动帮你搬。下文会反复强调这个”得自己动手”的特点。

结构图

分组:

- 早期架构:「寄存器文件 RF / 累加器(巨大)与其他值 / 抢占稀缺寄存器」
- Blackwell:「寄存器文件 RF / (收尾时显式读回)」「TMEM (128 × ≤512 列) / 累加器停在这 / (显式分配 / 释放)」

连接关系:

- TMEM (128 × ≤512 列) / 累加器停在这 / (显式分配 / 释放) $\xrightarrow{\text{显式加载(按 warpgroup 分布)}}$ 寄存器文件 RF / (收尾时显式读回)

就这”多出来的一步”,牵出了两个会贯穿全书的后果:

1. 读 TMEM 得你自己写代码来读,而且是按 warpgroup 分着读:一个 warpgroup 的 4 个 warp 合伙完成,各搬一片。(还记得上一节的 scope 吗?这就是”这个操作的作用域是 warpgroup”的活例子。)
2. TMEM 不像寄存器那样由硬件自动打理:你得亲手申请一块、用完亲手还回去(就像 C 语言里的 malloc/free,忘了释放就会出问题)。

注意

注意：把累加器从寄存器挪到 TMEM，是笔很典型的买卖——花一点写代码的麻烦，换硬件资源彼此解耦。好处是寄存器再也不被那个大累加器占满了；代价是程序员得自己惦记 TMEM 的生死（分配、释放），还得记得那次显式读回。这种“拿复杂度换性能”的权衡，后面你会一遍遍碰到。

2.3 跨集群的分布式共享内存 (DSMEM)

一句话先理解

一句话先理解：正常情况下，每个 CTA 只能用自己车间 (SM) 里的那块共享内存，够不着别人的。DSMEM 就是给“集群”开的一个特权——让同一个集群里的几个 CTA，能直接读写对方的共享内存，省得绕远路。

回想一下层级：整条链里，就**集群这一层**，成员能横跨好几个 SM。正是这个“能够得着别的 SM”的本事，给了它一项别的层级都没有的内存能力。

它解决啥问题？一个 CTA 趴在单个 SM 上，平时只能用自家这个 SM 那一小块共享内存。可前面说过，SMEM 很小、很金贵。处理大块数据时，一个 CTA 经常**装不下足够多的操作数**，或者想把已经费劲搬进来的数据让别人也用一遍（别人重新去大仓库 GMEM 拿太亏了）。这些事，单个 CTA 自己根本兜不住。

Hopper 给的答案叫线程块集群 (thread block cluster)：把一组 CTA 拴在一块，让它们配合得比平时更近一步——能一起同步，还能读写对方的共享内存。这种“互相读写对方 SMEM”的能力，就叫**分布式共享内存 (DSMEM, Distributed Shared Memory)**。到了 Blackwell，集群不但保留，还更强了：加了**动态调度**（见后续「集群启动控制」）和**2-CTA 协作式矩阵乘**。

具体怎么玩，拆成几步看（这里出现的“屏障”先简单理解，下面括号有解释）：

1. 一个 CTA 可以**直接指名**去访问对端 (peer)CTA 的共享内存——就是“我要写到你那块 SMEM 的某个位置”。
2. 一个线程点名对端 SMEM 里的某个地址，把一块 tile 从自己的 SMEM 整批拷过去。
3. 等所有字节都送到了，它再**升起一面“完成屏障”**告诉对方“数据齐了，你可以用了”。

这里的**屏障 (barrier / mbarrier)**是一种同步信号，作用就是让一方告诉另一方“我这步干完了，你可以接着干”——后面「Async Coordination: mbarriers」会专门讲。

第三部分那个 2-CTA 集群 GEMM，就是靠这套机制在两个 CTA 之间共享操作数 tile，**全程不用把数据绕一圈到大仓库 GMEM 再绕回来**——省了一大圈冤枉路。

下面这张图说一遍同样的意思：两个 CTA 分别住在 SM 0 和 SM 1，各自手里攥着 A、B 矩阵的一半。它们通过 DSMEM 去读对方手里那半 B（这就是绕开 GMEM 的近道），最后两个 CTA 合力算出一个 256×256 的输出 tile：

结构图

分组：

- **SM 0：**「CTA 0 / 持有 $A_0 + B_0$ 」
- **SM 1：**「CTA 1 / 持有 $A_1 + B_1$ 」

连接关系:

- CTA 0 / 持有 $A0 + B0$ $\xrightarrow{\text{DSMEM 读取对端 } B}$ CTA 1 / 持有 $A1 + B1$
- CTA 1 / 持有 $A1 + B1$ $\xrightarrow{\text{DSMEM 读取对端 } B}$ CTA 0 / 持有 $A0 + B0$
- CTA 0 / 持有 $A0 + B0 \rightarrow 256 \times 256$ 输出 tile / (由这一对 CTA 协作产出)
- CTA 1 / 持有 $A1 + B1 \rightarrow 256 \times 256$ 输出 tile / (由这一对 CTA 协作产出)

关键

关键:DSMEM 的价值, 一句话——它开了一条**绕开全局内存的近道**。以前两个块共享数据, 只能各自跑一趟 GMEM 把它读出来 (流量白白翻倍, 延迟还高); 现在直接 SMEM 拷给 SMEM, 又快又省。后面要讲的 2-CTA 协作式 MMA 和 TMA 多播, 底下踩的都是它。

三、计算引擎: CUDA 核与张量核

线程, 加上它们辛辛苦苦搬来的数据, 最后都得在”算数的硬件”上会合, 真正开算。这里有个关键: 一个 SM 给你的不是一种算数硬件, 而是**两种完全不一样的** (书里管它们叫”引擎”——你可以理解成两种不同的”计算机器”)。它俩各管一摊、互相搭台, 而这个分工几乎把”每个 kernel 该怎么写”都给定死了。

引擎	是什么	干什么活
CUDA 核 (CUDA core)	通用计算单元 (一个个普通的算术逻辑单元 ALU)	啥杂活都能干: 算下标、逐个元素加减乘、把一堆数求和 (归约)、跑 if/else (控制流)——也就是围着矩阵重活转的那些”胶水逻辑”。
张量核 (Tensor Core)	专用电路 (fixed-function, 只会干一件事但干得飞快)	只干一件事: 一条指令算完一整块矩阵的 $D = A \cdot B + C$ 。

一句话先理解

一句话先理解:这两种引擎的关系, 有点像”瑞士军刀 vs 专用机床”。CUDA 核是瑞士军刀, 啥都能干但不算特别快; 张量核是专用机床, 只会冲压矩阵乘这一种零件, 但冲压速度快到离谱。

3.1 为什么这个分工如此重要

道理很直白: **张量核算矩阵的速度, 比 CUDA 核快一大截**——快多少? 在”每秒能做多少次浮点运算 (FLOP/s)”这个指标上, 差着 **10 倍甚至更多**。

所以那些”本质就是大量矩阵乘”的运算——比如 GEMM、卷积、还有**注意力 (attention)** (Transformer 模型里的核心算子, 靠矩阵乘来算”每个词跟其他词有多相关”)——只有搬到张量核上跑, 才有可能逼近硬件的极限速度。用 CUDA 核硬算这些, 等于拿瑞士军刀去干机床的活, 慢得没法看。

关键

关键:顺着这个差距, 能推出一条贯穿全书的”第一性原理”——**想拿性能, 基本就是一件事: 把张量核喂饱, 别让它停**。这话很硬: CUDA 核哪怕优化到头, 也补不回那 10 倍的吞吐窟窿。这也就解释了, 为啥后面那么多章节都在死磕同一个问题: 怎么把数据够快地递到张量核嘴边, 让

它一刻都别闲着。

3.2 代际变化：张量核怎么被编程、结果停在哪

GPU 一代代升级，张量核这块电路一直都在。真正在变的是两件事：你用什么指令去驱动它，还有它算出来的结果存到哪儿。

- **Hopper**(上一代): 引入了一种”异步、由 warpgroup 发起”的矩阵乘指令，叫 **wgmma**(见 §1.2)。”异步”的意思是发起后不用傻等它算完，可以先去干别的——这是后面”重叠”的基础。
- **Blackwell**(本书主角这代): 第五代张量核 **tcgen05**，最大变化是把累加器放进 **TMEM** 而不是寄存器 (就是 §2.2 讲的那笔买卖)。本书有专门一章「Tensor Cores: tcgen05」细讲它。

3.3 集群对计算引擎的两项扩展

前面说集群把内存”撑大”了 (靠 DSMEM)。其实它对计算引擎也加了两招，这俩在后面的 GEMM 章节里会反复出现：

1. **2-CTA 协作式矩阵乘**: 让两个 CTA 各拿出自己那份 SMEM 里的操作数，拼成一块更大的 tile，一起交给张量核算。一次算更大块，效率更高。
2. **TMA 多播 (multicast)**: 很多时候好几个 CTA 要用到同一块数据。与其让它们各自从 GMEM 拉一遍 (白白重复占用带宽)，不如让搬运引擎只从 GMEM 读一次，然后一股脑分发给好几个 CTA。一次加载，人人有份，省流量。

这两招，底层踩的都是前面那个分布式共享内存 (DSMEM)。

四、GEMM 数据流水线：把拼图拼起来

到这儿，四块拼图——执行层级、内存、计算引擎、搬运引擎——我们一块块都摆出来了。现在终于能把它们拼到一起，看一条最典型的 **GEMM(矩阵乘) 流水线**：数据和计算到底是怎么配合着流动的。

一句话先理解

一句话先理解：大矩阵不会一次算完，而是切成一块块小 tile，一块一块算。这一节就跟踪”一块 tile”从进入到算完出去的完整旅程，看它要过哪三关。

4.1 单个 GEMM tile 的三个阶段

一块 GEMM tile 从头走到尾，要过三关：**加载** → **计算** → **收尾**。下面这张表先给个全貌——每一关由谁发起 (scope)、数据从哪流到哪。看不太懂没关系，表下面会一关一关用大白话讲：

阶段	scope(谁发起)	数据通路
① Load 加载	单个线程发起 TMA	GMEM --TMA--> SMEM(记录预期字节数，字节到齐后屏障翻转)
② Compute 计算	一个被选中的线程发起 tcgen05 MMA	SMEM -- --> Tensor Core -- --> TMEM(算完后发信号给屏障)

③ Epilogue 收尾	整个 warpgroup 协作	TMEM -- --> -- dtype--> SMEM --TMA store--> GMEM
---------------	-----------------	--

一关一关说：

1. **加载 / Load:** 用一次 **TMA 拷贝**，把一块 A 或 B 的 tile 从大仓库 GMEM 搬进小工作台 SMEM。这趟由一个线程喊一嗓子发起，而且它会先报个数：这趟一共要搬多少字节。然后字节一点点到位，TMA 引擎边搬边记进度，直到报的字节全到齐了，就翻一下那面”完成屏障”，等于喊一声”数据齐活，可以用了”。（为什么要先报字节数？因为搬运是异步的，得有个办法判断”到底搬完没有”——报个总数，数够了就算完。）
2. **计算 / Compute:** 用一次 **tcgen05 矩阵乘指令**，从 SMEM 把操作数读出来，交给张量核算，结果累加进一块 **TMEM**。这一关由一个被选中 (**elected**) 的线程发起，算完给屏障递个信号”我算完了”。
3. **收尾 / Epilogue:** 整个 **warpgroup(128 线程)** 一起上，把 TMEM 里那个累加器读回寄存器（还记得吗？TMEM 得自己显式读回），把结果转成输出需要的数据类型（比如从高精度的中间结果转成省空间的输出格式），最后写回 GMEM——一般先在 SMEM 里中转一下，再用一次 TMA 写出去。

注意

注意：回头对一下 §1.2 那张 scope 表，你会发现一件挺妙的事：这三关刚好各用了一种不同的作用域——加载是**单个线程**发起，计算是**单个被选中的线程**提交，收尾是**整个 warpgroup**一起干。前面说的那句”Blackwell 的关键操作不再由同一组线程发起”，这就是最实打实的例子。看，scope 这个概念真的到处都用得上。

4.2 真正的胜负手：重叠 (overlap)

照上面这个写法，三关看着像是**规规矩矩排队、一个接一个跑下来**的。可这里藏着全章**最要紧的一句话**：慢 kernel 和快 kernel 的差距，几乎全在”**重叠 / overlap**”这三个字上。

一句话先理解

一句话先理解：你有三个独立的工人（搬数据的 TMA、算数的张量核、收尾的 warpgroup）。”重叠”就是别让他们排队干等，而是**让三个人同时开工**——一个在搬下一块，一个在算这一块，一个在收拾上一块。这就跟洗衣服一样：洗衣机、烘干机、叠衣服，聪明的做法是三台/三步同时转，而不是洗完一批、烘完一批、叠完一批再开始下一批。

- **朴素 (naive) 写法：**老老实实按顺序来——加载，干等；计算，干等；存储。坏就坏在，每个工人等上一个干完的那段时间里，自己都在**干瞪眼、白白闲着**。三个工人里永远只有一个在动。
- **快速写法：**把这三步**流水线 (pipeline)** 起来。所谓流水线，就是错开节奏让大家同时忙：张量核正算第 **k** 块的同时，TMA 引擎已经在搬第 **k+1** 块了，收尾那头还在收拾第 **k-1** 块。这么一来，**同一时刻三个工人全都在忙**，谁都没闲着。

下面两张表对比一下。横轴是时间 (t0、t1、t2……)，每一行是一个工人，格子里有东西就表示”这个时刻它在干这块活”。

朴素写法 (串行, 工人大把闲着) —— 每个时刻只有一个工人在动, 其余都在等 (注意大片空格就是” 闲着”):

引擎	t0	t1	t2	t3	t4	t5	t6	t7	t8
TMA	[load k]					[load k+1]			
Tensor		[mma k]					[mma k+1]		
Epilog			[ep k]					[ep k+1]	

流水线写法 (重叠, 三个工人一起忙) —— 从 t2 开始, 同一个时刻三个工人都在干活, 机器全程不空转:

引擎	t0	t1	t2	t3	t4
TMA	[load k]	[load k+1]	[load k+2]	[load k+3]	...
Tensor		[mma k-1]	[mma k]	[mma k+1]	...
Epilog			[ep k-2]	[ep k-1]	[ep k]

关键

关键: 三个工人各干各的 (异步) 想同时忙、又不能乱套, 就得有人盯着交接——保证” 数据真的搬完了” 算数的才动手,” 真的算完了” 收尾的才动手。这套交接机制, 就是后续「Async Coordination: mbarriers」要讲的**屏障 (barrier) + 相位 (phase) 模型** (屏障就是前面提过的那个” 我干完了” 的同步信号)。第三部分那个”GEMM 阶梯 (GEMM ladder)”, 就是一级一级地把朴素 kernel 改造成深度流水线 kernel, 而整套改造踩的地基, 就是这个 mbarrier/phase 模型。这也是为什么本书要专门拨一章讲异步协调——没有它,” 重叠” 根本无从谈起。

小结

这一章把读懂整本书要用的” 坐标系” 搭好了。要是只让你记三句话, 就记这三句:

1. **执行是分层的。**线程 → warp → warpgroup → CTA → 集群 → 网格, 每一层都把” 某个距离上的配合” 变便宜。Blackwell 带来的关键转变是: **不同操作各有各的” 作用域 / scope”**(也就是” 由几个线程来发起”)——搬数据 (TMA) 由单个线程发起, 矩阵乘 (tcgen05) 由单个被选中的线程提交, 读 TMEM 和收尾 (epilogue) 由整组 warpgroup 分着干, 集群矩阵乘跨两个 CTA。scope 是本书三大核心概念 (scope / layout / dispatch) 里打头的那个。
2. **数据是分层暂存的。**GMEM(大而慢的总仓库)→ SMEM(快的小工作台)→ 寄存器 (最快但最小); 用张量核的 kernel 里,TMEM 还夹在中间替你拿着累加器。TMEM 是 Blackwell 独有的, 它拿” 得自己申请、自己释放、还得自己显式读回” 这点麻烦, 换来把那个又大又占地方的累加器从稀缺的寄存器里解放出来。再往上, 集群这一层又解锁了 DSMEM, 让几个 CTA 能绕开总仓库 GMEM 直接共享数据。
3. **算数的硬件被拆成几个各干各的引擎, 胜负全押在” 重叠” 上。**CUDA 核 (瑞士军刀) 管杂活, 张量核 (专用机床) 以 10 倍以上的速度扛矩阵重活,TMA 这类搬运引擎专门负责喂数据。把它们拼成 GEMM 的” 加载 → 计算 → 收尾” 三级流水线之后, 性能的秘诀就一句话: **让搬数据、算数、收尾这三个工人同时忙起来**——而它们之间能安全交接, 靠的就是 mbarrier/phase 模型。

一句话收尾: 本书后面几乎所有的优化, 翻来覆去都是同一件事的不同版本——怎么把活儿铺到这几块拼图上, 再让各个引擎重叠着一起跑。

接下来推荐读的章节 (都是本书后面的内容):

- **Tensor Cores: tcgen05** ——详解 tcgen05 计算与张量内存 (TMEM)。
- **Async Data Movement: TMA** ——讲解基于 TMA 的异步数据搬运。
- **Async Coordination: mbarriers** ——介绍协调这些引擎的 mbarrier 与相位 (phase) 模型。

延伸阅读

- 原文 (英文):GPU Execution Model —Modern GPU Programming for ML

建议: 原文里有好几个交互式演示 (Blackwell SM 全貌、执行层级逐层高亮、2-CTA 集群的 DSMEM 跳转、GEMM 三级流水线的数据路径追踪)。这份笔记用静态图把同样的内容画了出来, 但要说” 点一下某个动作, 亲眼看它在硬件单元之间怎么流” 那种直观劲儿, 静态图到底差点意思。想找那个感觉, 建议配着原书的交互演示一起看。

术语对照

中文	English / 缩写	含义速记
线程	thread	标量执行单元, 有独立 PC 与寄存器
线程束	warp	32 个线程, SIMT 执行
线程束组	warpgroup	4 个 warp = 128 线程; 发射 wgmma、访问 TMEM 的协作单位
协作线程阵列	CTA / thread block	硬件调度基本单位, 跑在单个 SM 上
集群	cluster	一组可跨 SM 协作的 CTA
网格	grid	一个 kernel 的全部线程
流式多处理器	SM	GPU 的基本计算核心模块
单指令多线程	SIMT	warp 内同发一条指令、各 lane 可独立屏蔽
全局内存	GMEM	设备级大容量 HBM, 所有 SM 共享
共享内存	SMEM	每 CTA 的低延迟片上便笺本 (B200 <=228 KB/SM)
张量内存	TMEM	Blackwell 新增, 每 CTA 的 128x<=512 列累加器便笺本
寄存器文件	RF	每线程最快存储, 放标量与块片段
分布式共享内存	DSMEM	集群内 CTA 互相读写对端 SMEM 的能力
CUDA 核	CUDA core	通用 SIMT ALU, 做胶水逻辑
张量核	Tensor Core	固定功能单元, 块粒度 $D=A \cdot B+C$
张量内存加速器	TMA	异步批量数据搬运引擎 (GMEM $\rightleftharpoons$ SMEM)
矩阵乘加	MMA	matrix multiply-accumulate
通用矩阵乘	GEMM	稠密矩阵乘法
收尾阶段	epilogue	把累加器转 dtype 并写回的尾部阶段
重叠	overlap	让多个异步引擎同时忙起来

屏障	barrier / mbarrier	协调异步引擎交接工作的同步原语
作用域	scope	执行某操作的那组线程

第 02 章 · 什么决定 Kernel 的速度

原文:What Makes a Kernel Fast

本章要点

本章要点 (TL;DR)

先解释几个名词, 后面会反复出现:

- **核函数 (kernel)**: 就是一段你写好、丢到 GPU 上去跑的程序。你可以先把它想成”一个跑在显卡上的函数”。本章一直在问的就是: 这个函数到底快不快、为啥。
- **字节 (byte)**: 数据的计量单位, 一个数占几个字节。”搬字节”就是把数据从一处内存挪到另一处。
- **FLOP**: floating point operation, 一次浮点运算 (一次加法或一次乘法都算一个 FLOP)。它衡量”算了多少数学活儿”。

这一章的核心结论:

- **屋顶线模型 (roofline model)** 给一个 kernel 设定了性能天花板。这个天花板要么由内存带宽 (memory bandwidth, 单位时间能搬多少字节) 决定, 要么由算力吞吐 (compute throughput, 单位时间能做多少次浮点运算) 决定——取两者中更慢的那条路。换句话说, 你的 kernel 要么卡在”数据喂不上来”, 要么卡在”算不过来”, 两者取更糟的那个。
- 决定”撞上哪个天花板”的, 是 **算术强度 (arithmetic intensity, AI)**: 每搬运一字节数据, 能做多少有用的浮点运算 (单位 FLOP/byte)。这是个比值: 搬得少、算得多 → 强度高; 搬得多、算得少 → 强度低。
- **算术强度低 → 内存受限 (memory-bound)**, 也就是被”搬数据”卡住了。出路是: 少搬字节、多复用数据、做算子融合 (fusion)、用更小的数据类型——本质都是”把 kernel 往屋顶线右边推”(右边是算力受限那侧, 后面会画图)。
- **算术强度高 → 可以做到算力受限 (compute-bound)**, 也就是被”算数学”卡住了 (这反而是好事, 说明数据不再拖后腿)。此时核心任务变成”别让张量核 (Tensor Core, GPU 里专门做矩阵乘加的硬件单元, 见第 0 章) 闲着”。
- 在现代 GPU kernel 里, 最大的提速杠杆是 **重叠 (overlap)**: 让搬数据和算数据同时进行, 而不是一件干完再干下一件。只要先后顺序允许, 就让 TMA(一个硬件自带的、专门成批搬数据的引擎)、张量核、收尾 (epilogue, 把结果转成最终格式再写回去的尾部阶段)、写回这几件事同时在跑。
- 一个性能数字脱离了天花板就没有意义。先估算算术强度、定位天花板、判断是内存受限还是算力受限, 再去优化真正卡脖子的那个资源——这是贯穿全书的优化 workflow。

前置知识

前置知识: 这一章会用到几个 GPU 概念——内存层级 (寄存器 / 共享内存 / 全局内存, 即”快但小”到”慢但大”的几层存储)、Tensor Core(张量核)、重叠 (overlap), 以及第 01 章

讲的**执行层级** (GPU 怎么把活儿分给一层层的硬件)。这些词第一次出现时我都会当场用大白话讲清楚, 所以你**没接触过 GPU 也能读**。但如果想更踏实, 可以先翻一下第 0 章·极简入门和第 01 章。

第 01 章带你认了认硬件: GPU 有哪几层执行单元、有哪些内存、有哪些专门算数和搬数据的引擎, 等于先给你画了张地图。这一章我们聊个更实在的问题——一个 **kernel** 到底快不快, 是什么说了算?

先说结论: 不是什么玄学技巧, 也不是“经验丰富才会的手感”。它是一套能让你“拿数字算账”的框架, 叫**屋顶线模型**。把它学会, 你还没动手写代码, 就能先判断出劲该往哪儿使——这一点对完全没碰过 GPU 的人尤其重要, 因为它让你不用靠瞎试也能心里有数。

关键

关键: 一个性能数字, 光盯着它本身看, 啥也说明不了。你得**拿它跟天花板比**, 它才有意义。

举个例子你就懂了。假设你的 **kernel** 测出来跑了 330 TFLOP/s (TFLOP/s = 每秒一万亿次浮点运算, 是衡量“算得多快”的速度单位), 听着是不是挺唬人? 可假设这颗 GPU 满血能跑到 2 PFLOP/s 左右 (PFLOP/s = 每秒一千万亿次, 是 TFLOP/s 的一千倍), 那 330 TFLOP/s 连六分之一的算力都没摸到, 大半个芯片都在那儿干瞪眼呢。说白了, 没有天花板当参照, 你压根分不清: 这 **kernel** 是已经榨到硬件极限了, 还是还有一大把性能躺着没人用。这就好比有人告诉你“这辆车开到了 60 迈”——你得先知道它最高能跑多少, 才知道这数字是快是慢。

那这天花板是从哪儿冒出来的? 这正是屋顶线模型的活儿。它的思路特简单: 一个 **kernel** 干的事, 无非两类——**搬字节** (把数据从内存读进来、再把结果写回去) 和**做算术** (对这些数据做加减乘除)。然后看是谁在拖后腿。要是数据喂不上来 (搬得太慢), 那内存带宽就是你的上限; 要是数据反复用、算术活儿也够重, 这才轮到算力吞吐来当上限。

注意

注意 (本章数字约定): 跟第 01 章一样, 我们还是拿一款具体的 GPU——**NVIDIA B200**——当例子, 而且天花板都取个整方便心算: 它的张量核满血吞吐算 **2 PFLOP/s** 上下, 内存带宽 (这里指 HBM3e, GPU 板上那块又大又快的主显存) 算 **8 TB/s** 上下 (TB/s = 每秒搬运的字节数, 以万亿字节计)。这些数会随着型号、时钟频率、散热限制、测量方式来回变, 所以你就把它们当成**数量级** (order-of-magnitude, 意思是“大致在这个量级”, 不必精确) 的参考, 别当成数据手册上一分不差的常数。

一、屋顶线模型: 两条路取更慢的那条

一句话先理解

一句话先理解: 你的 **kernel** 同时受两个天花板压制——“算得多快”和“喂得多快”。最终速度由更低的那个天花板说了算, 跟木桶装多少水由最短的那块板决定是一个道理。

前面说了, 每个 **kernel** 都干两件事: 搬数据、做算术。屋顶线模型的精髓就一句话——**这两条路谁慢, 谁就是你的上限**。我们一条条看这两个天花板:

- **算力天花板 / compute ceiling**: 就是硬件”算数学”能力的顶, 即每秒最多能做多少次浮点运算。比如 B200 上跑 GEMM(通用矩阵乘, 即两个大稠密矩阵相乘; 它是深度学习里最核心、最常见的运算, 后面会反复出现), 这个顶就是张量核的吞吐; 而要是个逐元素 (elementwise, 指对张量里每个数各做一遍同样的简单运算) 的小 kernel, 这个顶可能是 CUDA 核 (GPU 里的通用算术核, 负责一个线程一个线程地做普通标量运算, 相对张量核更”全能” 但做矩阵乘没那么猛) 的吞吐, 也可能是别的某个功能单元。
- **内存天花板 / memory ceiling**: 它等于**带宽 × 算术强度**。直觉是这样: 带宽是”每秒能搬多少字节”, 算术强度是”每搬一字节能换来多少次运算”, 两者一乘, 就是”靠搬数据这条路, 每秒最多能撑起多少次运算”。每搬一字节只算那么一丁点 (算术强度低), 内存带宽就成了瓶颈; 反过来, 一字节能撑起一大堆运算 (算术强度高), 那内存基本就不卡你了。
把这两条凑一块儿, 就是屋顶线那个基本不等式:

$$\text{FLOP/s} \leq \min(\text{FLOP/s}, \quad \times \quad)$$

这个不等式怎么读? 它说: 你实际能跑到的速度 (可达 FLOP/s), **永远不会超过**右边括号里那两个值中较小的一个。要么撞算力顶, 要么撞内存顶, 先撞上的那个就是你的天花板。
其中算术强度定义为:

$$= \text{FLOP} / (\text{:FLOP/byte})$$

也就是”这个 kernel 一共要做多少次浮点运算”除以”它一共要搬多少字节数据”。比值大, 说明数据用得很值 (搬一次反复算); 比值小, 说明搬来搬去却没算几下。

注意

注意: 一提算术强度, 你先得说清楚是按哪一级内存算的。GPU 的内存是分好几层的 (从慢到快、从大到小), ”搬的字节”按哪一层算, 结果就不一样。按 HBM(那块最大最远、但也最慢的主显存) 画屋顶线, 字节就指进出 HBM 的字节; 按 L2(介于中间的一层缓存) 算, 就是 L2 字节; 按 SMEM(共享内存, 离计算单元最近的一小块片上高速存储) 算, 就是共享内存字节。这一章如果不特别说明, 默认都在聊 **HBM 屋顶线**——因为进出主显存通常是最大的开销。

1.1 读懂这张图: 斜屋顶、平屋顶与脊点

一句话先理解

一句话先理解: 这张图就是把”两个天花板”画出来。一条斜线代表”搬数据的极限”, 一条平线代表”算数学的极限”, 哪条压在你头上, 你就被哪条卡住。

先教你怎么看这张图, 别急, 一步步来。**横轴是算术强度** (FLOP/byte, 越往右, 数据用得越值), **纵轴是你实际能跑到的性能** (FLOP/s, 越往上越快)。图上有两条”屋顶”(就是天花板的形象说法):

$$\begin{aligned} (\quad) : &= \times \\ (\quad) : &= \text{FLOP/s} \end{aligned}$$

为啥一条斜、一条平？内存屋顶是斜的，因为算术强度越高（横轴越往右），靠搬数据能撑起的性能就越高（线往上爬）——这是上面那个”带宽 × 算术强度”的直接画法。算力屋顶是平的，因为硬件每秒最多就能算这么多次，跟算术强度无关，所以画出来是一条水平线，封了顶。

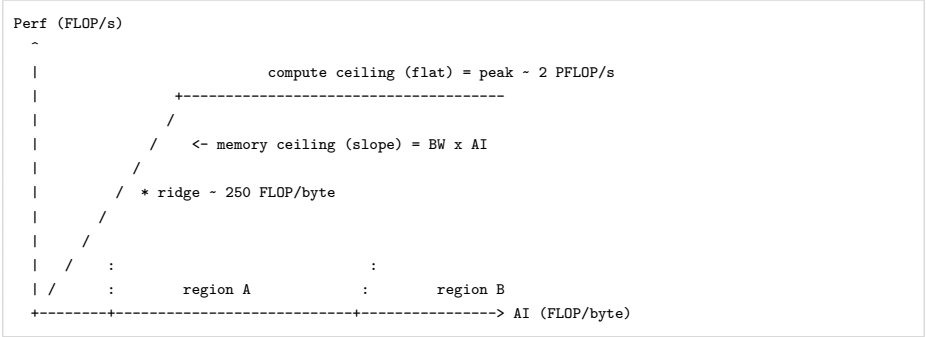
这两线会撞到一起，撞上的那个点叫**脊点 / ridge point**(就是斜线爬到顶、刚好碰上平线的拐角)：

= FLOP/s /

脊点就是那个临界值——算术强度到了这儿，你正好从”被搬数据卡住”切换到”被算数学卡住”。拿 B200 取整后的数字代进去算算：

-= 2000 TFLOP/s / 8 TB/s
- = 250 FLOP/byte

把斜屋顶、平屋顶、脊点这三样画一块儿，差不多就是下面这样：



图注：横轴是**算术强度 AI(FLOP/byte)**，纵轴是**可达性能 (FLOP/s)**。

标记	含义
memory ceiling(斜线)	内存屋顶 = 带宽 × 算术强度
compute ceiling(平线)	算力屋顶 = 峰值 FLOP/s ≈ 2 PFLOP/s
ridge(*)	脊点 ≈ 250 FLOP/byte，两条屋顶相交处
region A(脊点左侧)	内存受限区：被带宽卡住
region B(脊点右侧)	算力受限区：可被算力卡住

关键

关键：屋顶线真正值钱的，不是这张图长啥样，而是它一眼就能告诉你——到底是哪个资源在卡你。

- 算术强度**落在脊点左边** → **内存受限**：这会儿你再怎么打磨算术指令也是白搭，因为字节根本喂不上来。
- 算术强度**落在脊点右边** → **有机会做到算力受限**：这会儿抠那几个零碎字节也没用，你该干的是把算力单元喂饱，把性能顶到平屋顶上去。

一句话：内存受限的 kernel，不会因为你”数学指令写得漂亮”就变快；算力受限的 kernel，也

不会因为你”省了几个不相干的字节”就变快。所以优化的第一步永远是：先看清楚自己落在脊点哪一边。

二、常见 workload 的算术强度

一句话先理解

一句话先理解：你要解决的问题（算法）本身，基本就决定了它落在脊点哪一边——所以还没写代码，你就能预判自己会被哪个天花板卡住。

这里有个特别重要的发现：算术强度多半是算法自己定的，跟你怎么写实现关系不大（workload 就是”一类要干的计算任务”的意思）。这有啥好处？好处大了——你连一行 kernel 都还没写，就能先把它的算术强度估个八九不离十，提前知道劲该往哪儿使。下面挑三类最典型的 workload 看看。

Workload	算术强度	在脊点哪一侧	优化目标
逐元素 / 归约（GELU、RMSNorm）	低（常数级）	远在左侧，内存受限	逼近内存带宽屋顶
GEMM(矩阵乘)	高（随规模增长）	右侧，算力受限	喂饱张量核 + 重叠
注意力（Attention）	居中（取决于实现）	看情况	先抬高 AI，再做调度

2.1 逐元素与归约：天生内存受限

先解释这两个词。逐元素前面提过，就是对张量里每个数各做一遍同样的简单运算（比如每个数都乘一下、加一下）。归约（reduction）则是把一大堆数压成少数几个结果，比如求和、求平均、求最大值——”把很多数归并成一个”。

逐元素 kernel(像 GELU，一种激活函数，对每个数做一次固定的非线性变换) 和归约式 kernel(像 RMSNorm，一种归一化操作，要先把一行数平方求和、再拿来缩放每个数) 有个通病：它们要读写老大一个张量，可推到每个元素上，就做那么两三次 FLOP。

搬一大堆、算没几下，算术强度能不低吗？所以它们远远地趴在脊点左边（也就是被搬数据死死卡住）。这类 kernel 你别惦记着去够张量核屋顶——根本够不着——踏踏实实去贴内存带宽屋顶就对了。换个说法，你要操心的全是些很”机械”的访存（读写内存）问题：

```
-      (coalesced)?
-      ?
-      (producer)      (consumer)      ?
- dtype      ?
-      TMA      ?
```

逐条解释一下这几个术语（后面会再用到）：

- 合并（coalesced）：让相邻的线程去读相邻的内存地址。GPU 一次能成块搬一片连续数据，要是大家读的地址东一个西一个，硬件就得得分好多趟搬，带宽白白浪费。合并就是”让大家排好队、一次搬一整片”。
- producer / consumer:producer 是”生产数据的那个算子”,consumer 是”接着用这份数据的那个算子”。

- **dtype**: data type, 一个数用什么格式存 (比如 fp32 占 4 字节、fp16 占 2 字节)。dtype 越小, 搬的字节越少。
- **向量化访存**: 让一个线程一次读/写好几个相邻的数 (比如一次搬 4 个), 而不是一个一个来, 减少访存的次数。

关键

关键: 一个 kernel 要是既没法复用、又没法融合, 那内存屋顶就是它实打实的头顶, 谁也别想往上抬。这时候你的目标不是去“超过”屋顶——超不过去——而是把它“贴满”(怎么贴, 第四节细说)。

2.2 GEMM: 算术强度随规模增长

GEMM(矩阵乘) 是另一头的极端, 跟逐元素恰好相反。它的算术强度问题越大、它越高。为啥? 这里先解释一下矩阵乘为啥这么特别: 算两个矩阵相乘时, **结果里的每一个数, 都要用到输入矩阵里一整行和一整列**——也就是说, 输入里的每个数, 会被反复用在很多个输出上。这就给了“搬一次、算很多次”的天然机会。

具体到 GPU 上是这么干的: 把大矩阵切成一个个小方块 (叫 **tile**, 就是“瓦片/小块”的意思), 一块 **tile** 搬进片上快速存储之后, 能被反复拿去做好多次乘加 (**MMA, matrix multiply-accumulate, 矩阵乘加运算**, 即“乘一下再累加进结果”这个最基本的动作)——加载一回, 用上好多回, FLOP/byte 可不就上去了。

拿个方阵 fp16 矩阵乘来说 ($M = N = K$, 即三个维度都相等的方阵相乘), 理想情况下算术强度差不多是:

$$\begin{aligned} \text{AI} &\sim 2N^3 / (3 \cdot 2N^2) \\ &= N / 3 \quad \text{FLOP/byte} \end{aligned}$$

注意

注意 (这个估算有几个前提): 它假设 A、B 各只读一遍、C 只写一遍、**beta = 0**、片上复用一点不浪费, 而且没有任何多余的元数据、padding 或者冗余流量。实际 kernel 搬的数据比这个理想值多, 不过拿来“估个量级”还是够使的。

公式别背, 搞懂它咋来的就行。先看分母 $3 \cdot 2N^2$: 三块矩阵 (A、B、C), 每块 N^2 个元素, 每个元素 2 字节 (fp16)。再看分子 $2N^3$: 这是整个矩阵乘的浮点运算总数——每个输出元素得做 N 次乘加, 一次乘加算 2 个 FLOP。

代进去 $N = 4096$ 试试:

$$\text{AI} \sim 4096 / 3 \sim 1365 \text{ FLOP/byte}$$

1365 这个数, 远在脊点 (约 250 FLOP/byte) 的右边——也就是落在算力受限那一侧。所以结论很清楚: **大 GEMM 在 HBM 屋顶线底下, 是算力受限的**。既然如此, 它要优化就别去省 HBM 流量了 (那不是瓶颈), 该干的是: **把张量核使起来、喂饱它, 再让搬数据和算数据重叠着跑**——这么干才摸得到算力屋顶。

关键

关键：这正好解释了一个看着挺拧巴的现象——朴素 (naive, 指最直白、没做任何优化的)GEMM 算术强度明明很高, 却可能慢得要命。咋回事? 因为算法准许你跑得快, 不等于你的实现真的跑得快。代码写得糙一点, 张量核照样能大段大段地闲着没事干 (就好比你有台超跑, 但你一直挂在一档慢慢爬)。说到底, 算术强度只是给你画了个天花板, 够不够得着, 那是另一码事 (看第四节那个“优化阶梯”)。

2.3 注意力: 介于两者之间的混合问题

注意力 (Attention) 是大语言模型里的核心运算。一句话说明白它干啥: 一段文本被切成一个个 token(可以理解为“词”或“词片”), 注意力就是让**每个 token 去“看一眼”其他所有 token**, 据此决定哪些更相关。它正好卡在前面逐元素和 GEMM 这两中间。它的算术强度不是个定数, 好几样东西都能让它变: 序列长度 (文本有多长)、头维度 (head dimension, 每个“注意力头”处理的向量有多宽)、tiling(怎么切块)、掩码 (masking, 比如不让一个 token 看到它后面的词), 还有最要命的一样——**你有没有把中间张量物化 (materialize) 到内存里去。**

这里得解释一下**物化 (materialize)**: 就是把一个算出来的中间结果真的写进内存存起来 (而不是用完就扔、一直留在片上)。写进去再读回来, 就要多搬一大堆字节。

标准注意力最疼的地方, 就在**分数矩阵 / score matrix**——它是 query(查询) 和 key(键) 两两做点积 (向量对应位相乘再求和, 衡量两个 token 有多相关) 得到的一个中间大矩阵。kernel 要是把这个分数矩阵先写进 HBM、回头再读回来, 那等于让一个巨大的中间张量在主显存里白白跑了个来回, 搬运量巨大。**Flash Attention(书里拿 Flash Attention 4 举例)** 这个著名优化的高明就在这儿: 它压根不把整个分数矩阵存下来, 而是把要用的 tile **摞在片上** (片上存储就近用、用完即弃), 这趟 HBM 往返直接省了, 算术强度“噌”一下就上去了 (搬得少了, 比值自然大)。

关键

关键：所以优化注意力是个“两步走”的活——**前半段是屋顶线问题, 后半段是调度问题。**第一步先改算法, 把流向 HBM 的字节砍下来 (也就是把算术强度 AI 抬上去); 第二步再调度 kernel, 让剩下的搬运和计算重叠起来 (也就是让搬数据和算数据同时进行, 后面第五节细讲)。你品一品就会发现, 这不就是把 GEMM 那套打法直接搬过来用嘛。

三、算术强度低的时候怎么办

一个 kernel 落在脊点**左边**, 它就是内存受限的 (被搬数据卡住了)。这会儿张量核、CUDA 核很可能大段大段地干等着, 因为卡你脖子的是**字节** (数据喂不上来), 不是算术指令 (算力还有富余)。咋办? 两条路。

3.1 第一条路：抬高算术强度（高杠杆）

一句话先理解

一句话先理解：既然问题是”搬得太多、算得太少”，那就想办法让搬来的每个字节多干点活——这就是把算术强度抬上去。

这条路更划算（”高杠杆”意思是花小力气、收大效果），因为它能把 kernel 往算力受限那边推（脊点右边，那边反而是好事）。具体有三招。

第 1 招：算子融合（fusion）——这招最关键。算术强度低，最常见的病根就是”一个中间张量刚写进 HBM，下一个算子转头又把它读回来”——白白搬了两趟。怎么治？把上游的 producer（生产数据的算子）和下游的 consumer（用这份数据的算子）**塞进同一个 kernel 里**（这就叫”融合”）。这么一来，中间结果就一直待在片上的快速存储里，**那趟 HBM 往返直接没了**。这里顺带把 GPU 那几层片上存储交代清楚（都比 HBM 快得多）：

- **寄存器（register）：**最快，但每个线程私有、容量极小。跟 CPU 的寄存器是一个意思，只是 GPU 上每个线程都有自己的一份。
- **SMEM(shared memory, 共享内存)：**一个线程组（block）内的线程共用的一小块片上内存，需要你手动管理（自己决定往里放什么、什么时候放）。可以把它想成一块”程序员自己掌控的高速缓存”。
- **TMEM:**Blackwell 这代 GPU 上新增的、专门给张量核的累加器（暂存乘加中间和的地方）用的片上内存。

几个典型例子：

```
- GEMM +          epilogue
- normalization
-                ( Flash Attention)
```

第 2 招：为复用而分块（blocking for reuse）。”分块”就是前面说的把数据切成 tile。道理简单得很：一块 tile 搬进片上一次，趁它还没被新数据挤出去，能多用几次就多几次，这样每个字节就摊上了更多算术活儿（分母不变、分子变大，算术强度自然上去）。GEMM 算术强度高，靠的就是这个。别的 workload 只要也有”同一块 tile 反复用”的机会，这招照搬就行。

第 3 招：让每个数变小（换更小的 dtype）。从 fp32（4 字节）往下降到 fp16（2 字节）、fp8（1 字节）乃至 fp4，一箭双雕：每个数占的字节少了，搬运流量就小了；同时分母变小，FLOP/byte 还顺带抬高了。（dtype 后面的数字就是这个数占多少位，fp16 = 16 位 = 2 字节，以此类推。）

注意

注意：实际省下来的，通常没有按 dtype 字节比例算出来的那么多。因为低精度格式常常得额外拖着一些”附带信息”：比如元数据，或者缩放因子（scale factor，因为数太小存不下，得记一个倍数回头还原）。要么就得在算之前多走一道格式转换——**block-scaled fp8/fp4**（一种分块带缩放因子的低精度格式）就是这样。话虽如此，换更小的 dtype 还是把 kernel ”往屋顶线右边推”最干脆的办法之一。

3.2 第二条路：认了，就把内存屋顶贴满

不是每个 kernel 都救得回来。有些就是没东西可融、也没复用可榨：一次纯拷贝、一个简单的逐元素操作、对着一个大张量扫一遍做归约——它们生下来就是内存受限，你改不了。

关键

关键：碰上这种，别惦记着去”打败”屋顶（打不破），想办法把它贴满（saturate，意思是把带宽用到满、不留空闲）就行。怎么贴？

```
-
-
-      (coalesced)
-          tile      TMA
-                               (in flight),
-          ,          dtype
```

这里有两点新词解释一下：

- **TMA**:Tensor Memory Accelerator, 是 GPU 硬件自带的一个”专职搬运工”。以前搬数据得让计算线程自己动手，既占用线程又慢；TMA 能接管那种”成块、规则”的数据搬运，你只要下个指令告诉它搬哪块，它自己后台搬，线程腾出来干别的。
- **请求在途 (in flight)**: 内存访问是有延迟 (latency) 的——你下了”读取”指令，数据不会立刻到，得等一会儿。如果你一次只发一个请求、傻等它回来再发下一个，大把时间就浪费在等上了。”保持多个请求在途”就是一口气把好多读取请求都发出去，让它们排队同时在路上跑，这样数据就源源不断地回来，把等待时间盖掉了。这招叫”藏延迟”。

关键

关键：一个内存受限的 kernel 一旦把内存屋顶贴满了，你再去抠算力那边的优化就是白费功夫（瓶颈不在那儿）。这时候想让它更快，只剩一条路：回头改算法，让它少搬点字节。

四、优化阶梯：从”可能”到”做到”

一句话先理解

一句话先理解：屋顶线告诉你”理论上能跑多快”，但真要跑到那个速度，得一步步把实现做对——这一节就是讲那”一步步”长什么样。

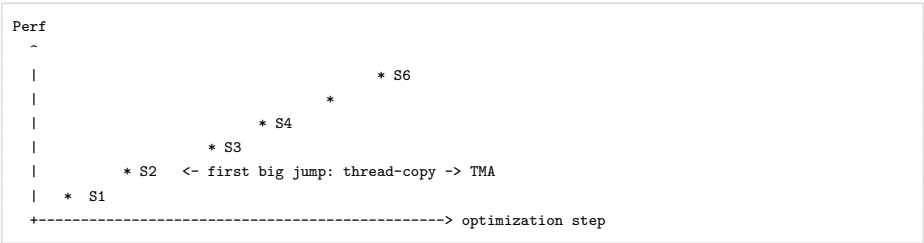
屋顶线只跟你说什么有可能做到，它可不管够到那个极限有多费劲。这两码事你得分清楚。

打个比方，一个大 fp16 GEMM 理论上是算力受限的。可这话的意思无非是”HBM 屋顶不是主要拦路虎”，绝不是说你随手写个实现就能摸到张量核屋顶。从”理论上能到”到”实际真跑到”，中间隔着一道坎；想填平它，你得用对指令、摆对布局 (layout, 指数数据在内存里怎么排布，排得对硬件读起来才顺)、做对暂存 (staging, 指提前把数据搬到离计算更近的地方备着)，还得把同步 (让多个并行的部分步调一致) 和调度 (安排谁先干谁后干) 都安排明白。

关键

关键: 这道坎, 本书第三部分在 B200 上拆成了一级一级的台阶 (就是后面「用 Warp 专精与集群扩展 GEMM」那一章)。这里有个点你留意: 每往上迈一级, **底层算法压根没动** (还是同一个矩阵乘), 变的只是 tile **怎么算、怎么调度**。

GEMM 优化阶梯: 性能一级一级往上走, 可**算法自始至终没变**, 变的只是 tile 怎么算、怎么调度。



台阶	名称	说明
S1	同步 tiled 基线	thread-copy 分块路径, 作为起点
S2	TMA 路径	第一次能测出来的大跳:thread-copy → TMA
S3	Warp 专精	分离 producer / consumer 角色
S4	C/TA 集群	更大的协作 tile 与多播
S6	多消费者执行	顶部: 多个消费者同时取用结果

- **第一次能测出明显提速的大跳**, 是从 **thread-copy 分块路径** (让计算线程自己一点点搬数据) 换到 **TMA 路径**。TMA 干的是这事儿: 那种”规则的 GMem → SMem tile 搬运”(GMem 就是前面说的全局内存/主显存 HBM,SMEM 是片上共享内存, 这里就是把数据从远处搬到近处), 本来压在线程身上, 现在交给前面讲过的那个硬件搬运工 TMA 去成批拷贝、喂给张量核。线程解放了去干别的, 搬运还更快了。
- **再往后的提速, 基本全靠重叠和调度**:TMA 提前把”后面才会用到的 tile”预取 (prefetch, 提前搬好备着) 进共享内存;tcgen05.mma(Blackwell 上那条异步发起张量核乘加的指令) 在后台异步地算;epilogue 这边同时把上一批算好的结果排干 (drain, 即慢慢写回去、腾出地方);最后再用软件流水线 (software pipelining, 用代码把”搬、算、写”这几段排成一条流水线, 让它们首尾交替地跑) 加 **warp 专精 (warp specialization**, 让不同的线程组各管一摊事, 后面第六节会讲) 把这几段编排到一块儿, 让好几个硬件引擎一起开工 (这就是”重叠”)。

注意

注意: 别想当然觉得”每爬一级都得比上一级快”。像 warp 专精这种步子, 可能会**先把资源砸进一个”眼下并不马上提速”的结构里**。可只要它能**解锁那些更简单结构压根做不到的重叠**, 这步就走对了——它是在给后面铺路呢 (就像重构代码, 有时单看那一步没变快, 但它让后续优化成为可能)。

五、重叠是最大的杠杆

一句话先理解

一句话先理解:GPU 里有好几个能各干各活的硬件引擎 (搬数据的、算数学的、写结果的)。如果你让它们一件接一件地干,大部分引擎大部分时间都在闲着;让它们同时开工,速度立刻上去——这就是”重叠”。

假设一个 GEMM 已经是算力受限、张量核也用上了,那还差点啥?多半就差在”空档”上——总有些引擎在那儿干等,时间白白溜走了。

来看看一个朴素 kernel 是咋跑的 (下面每行是一个动作,从上往下按顺序执行):

```
load tile k
compute tile k
store tile k
load tile k+1
compute tile k+1
store tile k+1
```

这里 load(加载)是把 tile k 搬进来,compute(计算)是算它,store(存储)是把结果写回去。这种一步压一步的排法,会把硬件晾在那儿大把空转:加载在跑的时候,张量核干瞪眼;张量核在算的时候,拷贝引擎没事干;轮到写回,前面俩可能又都在等。说白了,任何一个时刻,基本就一个引擎在干活,其他全闲着——就像流水线工厂里只开一个工位、其他工位的人都站着看。

流水线化 (pipelined) 的 kernel 就两样了,它让互不相干的几个阶段同时开跑 (关键在于:此刻该算的、该搬的、该写的,是不同批次的 tile,彼此不冲突,所以能并行):

```
load tile k+1 <-
compute tile k <-
store tile k-1 <-
```

朴素 (串行,引擎大量闲置)——任一时刻只有一个引擎在干活:

引擎	t0	t1	t2	t3	t4	t5
TMA	[load k]			[load k+1]		
Tensor		[mma k]			[mma k+1]	
Store			[st k]			[st k+1]

流水线 (重叠,多引擎齐忙)——同一时刻多个引擎都在干活:

引擎	t0	t1	t2	t3
TMA	[load k+1]	[load k+2]	[load k+3]	...
Tensor		[mma k]	[mma k+1]	...
Store			[st k-1]	[st k] ...

看出门道了吗?同一时刻 (比如 t1),TMA 在搬 k+2、张量核在算 k、写回引擎在排 k-1——三个引擎全在忙,谁也没闲着。这就是流水线把空档填满的办法。

这就是本书后面 Blackwell kernel 结构的主心骨:TMA 负责异步搬数据,tcgen05.mma 负责异步算张量核,epilogue 和写回管输出这头 (”异步”的意思是:发起这件事之后不用站在原地等它干完,可以接着去发起别的事,这正是能”重叠”的前提)。但几件事同时跑,就得有人协调”什么时候该轮到谁”,于是再拿 mbarrier 这么个对象把各个阶段拴到一起。

mbarrier 是啥：它是一个”异步屏障”对象，作用类似多线程编程里的信号量/栅栏——某件事干完了就在上面打个标记，等这件事的人看到标记再继续。比如张量核要算 tile k，就先在 mbarrier 上等”k 已经被 TMA 搬完了”这个信号，等到了再开算。靠它，每个消费者只在它真要用数据的那一刻，才去等一下，平时各忙各的，不互相等待。

关键

关键：这里你得想明白一件事——重点不是把依赖消掉，而是绕着依赖来排活儿。”依赖”就是”这件事必须等那件事干完才能干”。有些依赖是天生消不掉的:tile k 的 MMA(算)非得等 tile k 加载完才能动(数据还没来，算啥);tile k 的 epilogue(收尾写回)非得等 MMA 算完才能读结果。可妙就妙在——tile k+1 的加载往往可以趁着 tile k 的 MMA 还没算完就先跑起来(它俩用不同的引擎、碰的是不同数据，互不打架);tile k-1 的写回往往可以同时在那儿进行。正是因为有这些”能并行的空档”，后面好几章才专门来讲各种异步机制：

机制	作用	对应后续章节
TMA	GMEM → SMEM 的搬运	Async Data Movement: TMA
mbarrier	标记加载完成，做资源交接	Async Coordination: mbarriers
tcgen05	异步张量核计算	Special Memory: TMEM
TMEM	存放长生命周期的累加器	Special Memory: TMEM
warp 专精	分离 producer / consumer 角色	用 Warp 专精与集群扩展 GEMM
集群 (cluster)	更大的协作 tile 与多播 (multicast)	同上

这些机制长得各不一样，可它们都在伺候**同一个调度目标**：让有用的活儿**同时**跑在好几条硬件路径上，而不是挤在一条上。

六、占用率与资源压力

一句话先理解

一句话先理解：除了”重叠”，还有一招藏延迟——同一块硬件上多塞几摊活，谁卡住了就切去干别的，这叫高占用率。但它和重叠是两种风格，各有各的用武之地。

关键

关键：藏延迟的法子不止”重叠”这一招。还有个更老、更通用的机制，叫**占用率 (occupancy)**。

先说几个硬件名词，再说占用率。前面提过 GPU 把活儿分给一层层硬件，这里要用到两个：

- **SM(streaming multiprocessor, 流式多处理器):**GPU 内部一个能独立调度、独立干活的计算核心。一颗 GPU 上有很多个 SM。你可以粗略地把它类比为”一颗多核 CPU 里的一个核”，只不过它内部还能同时跑很多线程。
- **warp(线程束):**GPU 调度的最小单位，是 **32 个线程绑在一起的一个小班**。关键特点：这 32 个线程必须**齐步走**——同一时刻执行同一条指令，只是各自处理不同的数据。打个比方,warp 就像 32 个人组成的方阵，口令一响必须一起迈同一只脚，没法各走各的。
- **CTA(cooperative thread array, 协作线程阵列)：**也就是常说的一个 **block**，是一组

会被**整体**派到同一个 SM 上、能彼此协作 (共享 SMEM) 的线程。一个 CTA 里含若干个 warp。

那占用率是个啥？就是**同一时间常驻在一个 SM 上的活儿有多少** (常驻的 warp 越多, 占用率越高)。它咋帮你藏延迟呢？是这样: 一个 warp 卡住了 (stall, 比如在等内存数据回来), 调度器立马切到另一个已经准备好的 warp 去跑——就像 CPU 在多个线程间切换, 一个等 IO 就先跑别的。只要你手头总攒着一池子”招手就来”的独立 warp, 等待的时间就这么神不知鬼不觉地被别的活儿填满了。

那占用率被啥卡着？被**单个 SM 上有限的资源**卡着, 主要这四样: **寄存器、共享内存、warp 槽位、CTA 槽位** (后两样就是”一个 SM 最多同时容纳多少个 warp / 多少个 CTA”的名额)。一个 kernel 要是每个线程吃一大堆寄存器, 或者每个 CTA 啃掉一大块共享内存, 那它占用率一般就高不了——因为一个 SM 总共就那么多资源, 根本塞不下几个 CTA、几个 warp。

这里有个反常识的地方: **不少现代张量核 kernel 是成心”大手大脚”用资源、成心把占用率压低的**。多级共享内存流水线把 SMEM 吃掉, 大块的寄存器片段 (fragment, 指张量核运算时, 一个操作数被拆成小份分别放在各个线程的寄存器里, 那些小份就叫片段) 把寄存器吃掉,TMEM 分配把张量内存吃掉,warp 专精更狠, **一整个 warp 一整个 warp 地划走**, 专门指派给某个角色 (比如这几个 warp 只管搬数据, 那几个只管算)。图啥呢？

注意

注意 (这是成心做的取舍): 因为这些 kernel 换了套藏延迟的玩法。它们不再靠”一大堆不相干的 warp 常驻、谁卡了就切谁”来藏, 而是改成在**少数几个常驻 CTA 内部, 用明摆着的重叠 (第五节那套) 去藏**。所以你会看到个挺有意思的结果——一个占用率很低的 kernel, 只要它的流水线能让 **TMA、张量核、写回都没闲着, 照样能跑得飞快**。换句话说, 占用率低不一定是病, 关键看你有没有别的办法把硬件喂饱。

这两套玩法没有谁绝对更高明: 有些 kernel 因为访存不规则、或者腾不出地方做显式重叠, 就只能吃高占用率这碗饭; 另一些则**非得**靠深度暂存加专精不可, 因为只有这样才能喂得饱张量核。所以你真正该问的, 从来不是”占用率高不高”, 而是——**正在干活的那些硬件单元, 到底喂饱了没有？**

小结

这一章说来说去就一句话: **先看病, 再下药**。别一上来就埋头优化, 先搞清楚到底是哪儿慢。后面整本书都会一遍遍绕回这三个诊断问题:

-	kernel	?
-		?
-		?

情况不一样, 答案也就很清楚了:

Kernel 类型	诊断	优化方向
内存受限	落在脊点左侧, 被带宽卡住	减少字节 + 提升带宽利用: 融合、合并、向量化、TMA、更小 dtype

算力受限 GEMM	落在脊点右侧	先上张量核，再做重叠：暂存操作数、发异步 MMA、填满流水线、不卡住计算路径地排干结果
Flash Attention	先抬 AI 再调度	先把分数 / 概率 tile 留在片上抬高 AI；之后复用 GEMM 那套重叠工具 (tiled 搬运、SMEM 暂存、异步计算、资源交接)

这么一来，就有了一套能照着做的优化流程：

估算术强度 → 定位屋顶 → 判断是内存受限还是算力受限 → 只去优化那个真正设定天花板的资源。

少了这一步，优化 kernel 就成了拍脑袋瞎猜；有了它，你每动一处都说得清为什么——要是把算术强度抬上去了，要是把内存路径往带宽峰值推近了，要是把算力屋顶底下那点空闲时间给砍掉了。

延伸阅读

- 原文 (英文):[What Makes a Kernel Fast —Modern GPU Programming for ML](#) Sys
- 第 01 章「GPU 执行模型」：这一章接着用了那里定下的 B200 取整天花板 (≈ 2 PFLOP/s、 ≈ 8 TB/s)，还有执行层级、内存空间、计算和搬运引擎这些硬件底子。

建议：原文配了一张 B200 屋顶线图 (把内存屋顶、算力屋顶、脊点都标出来了)，还有一张 GEMM 优化历程图 (从同步 tiled 基线，一路经过 TMA、warp 专精、CTA 集群，走到多消费者执行的实测点)。这份笔记用静态示意图做了对应的表达，坐标轴比例只是示意，真要看准实测数值，请以原书图表为准。

术语对照

中文	English / 缩写	含义速记
屋顶线模型	roofline model	用“内存 / 算力取更慢”给 kernel 设上限
算术强度	arithmetic intensity (AI)	每搬一字节做多少 FLOP (FLOP/byte)
内存受限	memory-bound	落在脊点左侧，被带宽卡住
算力受限	compute-bound	落在脊点右侧，被算力卡住
脊点	ridge point	两条屋顶相交处 = 峰值 FLOP/s ÷ 带宽
内存带宽	memory bandwidth	单位时间能搬多少字节 (B200 HBM3e ≈ 8 TB/s)
算力吞吐	compute throughput	单位时间能做多少 FLOP (B200 fp16 ≈ 2 PFLOP/s)
算子融合	fusion	把 producer/consumer 合进一个 kernel，消除 HBM 往返
为复用而分块	blocking for reuse	一块 tile 加载一次、复用多次
物化	materialize	把中间张量真正写进内存

分数矩阵	score matrix	注意力里的中间矩阵, 易成 HBM 流量大户
闪存注意力	Flash Attention	把 tile 留片上、避免 HBM 往返以抬高 AI
合并访存	coalesced access	相邻线程访问相邻地址, 带宽利用高
软件流水线	software pipelining	用代码把多阶段排成重叠流水线
Warp 专精	warp specialization	把 warp 分工成 producer / consumer 角色
重叠	overlap	让多个异步引擎同时忙起来
占用率	occupancy	一个 SM 上常驻的工作量, 靠多 warp 藏延迟
通用矩阵乘	GEMM	稠密矩阵乘法
矩阵乘加	MMA	matrix multiply-accumulate
收尾阶段	epilogue	把累加器转 dtype 并写回的尾部阶段

第 03 章 · 数据布局与其记号

原文:[Data Layout and Its Notation](#)

本章要点

本章要点 (TL;DR)

- **数据布局 (data layout / 数据布局)** 干的事, 就是告诉你某个逻辑下标对应的数, 到底躺在物理上的哪个位置。别小看它: 访存能不能合并 (coalescing)、会不会撞 bank、某个硬件引擎读不读得了一块 tile, 全看它怎么摆。同一批数字, 光是换个摆法, 在同一块 GPU 上性能就能差一个数量级。
- 全书就用一套紧凑记号描述所有布局: $S[(\text{shape}) : (\text{strides})]$, 说白了就是「形状: 步长」做点积。这玩意儿你早就见过——PyTorch/NumPy 里的 `shape + stride` 就是它, 这里只是把它搬到了 GPU 上。
- 给步长贴上 **命名轴 (named axes / 命名轴)** (像 `@m`、`@laneid`、`@reg`、`@gpuid_x`、`@TLane` 这些), 同一套记号就能描述数据在内存、寄存器、warp 通道、TMEM、乃至整个 GPU 网络上是怎么铺开的。
- 再加一个 **复制项 (replication term / 复制项)** $R[n : \text{stride}]$, 就能表达「同一份数据被广播或拷贝到好几个地方」。分布式分片里的副本是它, Blackwell block-scaled MMA 在 TMEM 里的 `warpx4` 广播也是它。
- **Swizzle(地址异或重排 / swizzle)** 专门用来消掉共享内存的 bank 冲突, 让同一块 tile 按行读、按列读都能跑得快。注意它不属于仿射布局 $S[\dots]$, 而是单独叠在上面的一层非仿射变换。

前置知识

前置知识: 读这一章前, 最好先懂 warp 与 lane(通道)、寄存器 vs 共享内存、内存 bank 与 bank 冲突, 以及 Tensor Core 大概是干嘛的。没把握的话, 先翻一下第 0 章·极简入门。本章会默认你已经认识这些词。

平时写程序, 我们脑子里基本只有张量 (tensor, 你可以先粗暴地理解成「多维数组」, 比如一个二维数组就是矩阵) 的「逻辑形状」, 比如一个 (M, N) 的矩阵—— M 行 N 列。可形状只说清了一件事: 一共有多少个数。至于这些数字、这些字节, 实际摆在内存的哪个角落, 形状一个字都没提。

平时写后端、写业务, 你确实不用关心这个——数组就是数组, 取下标就完事了, 内存怎么摆是底层的事。但到了 GPU 上, 情况彻底反过来了: **硬件最在意的恰恰就是「字节到底摆在哪」**。摆得好和摆得坏, 同一段计算能差出十倍速度。这一章就是来回答这件事的。

一句话先理解

一句话先理解: 这一章讲的「数据布局」, 本质就是一张「翻译表」——你给它一个逻辑下标 (i, j) (第 i 行第 j 列), 它告诉你这个数实际存在物理上的哪个位置。

数据布局 (data layout) 回答的问题很直接: 逻辑下标 ($i, j, \hat{e}$) 对应的那个元素, 到底存在哪儿? 答案不止「内存」一种, 可能是全局内存, 可能是共享内存, 可能是寄存器, 也可能是某种专用的硬件存储——这些名词你现在不认识没关系, 本章会一个个讲到, 这里只要先有个印象:**GPU 上「存数据的地方」有好几种, 不是只有一块内存。**

那物理摆放为什么这么要紧? 原文给了三条理由。这三条里出现了几个 GPU 专有名词, 我先当场用大白话讲清楚, 你别被吓住:

1. **访存能不能合并 (coalescing)**。先解释 warp: GPU 干活不是一个线程单独斗, 而是把 **32 个线程绑成一个最小作战单位**, 这一组 32 个线程就叫一个 **warp**。warp 里每个线程有个编号 (0 到 31), 这个编号就叫 **lane(通道)**。现在关键来了: 这 32 个线程同时去内存里取数 (load) 时, 如果它们要取的 32 个地址刚好 **连成一整片**, 硬件就能一把把它们 **合并成 1 次内存读取**——这叫「合并访存」, 快得很; 可要是这 32 个地址东一个西一个, 散在内存各处, 硬件没法合并, 就得 **拆成多达 32 次单独的读取**。一次和三十二次, 差距大得吓人。所以数据摆得连不连续, 直接决定快慢。
2. **会不会撞 bank(bank conflict)**。GPU 有一块特别快的「共享内存」, 它在物理上被切成了若干个 **bank(存储体, 你可以想成「32 个并排的小柜台」)**。32 个线程同时来取数, 如果它们分别落在 **不同的柜台**, 就能 32 个一起办、并行处理; 可要是好几个线程挤到 **同一个柜台** 要不同的东西, 这个柜台一次只能服务一个人, 只能 **排队挨个来**, 瞬间慢下来。这种「挤同一个柜台」就叫 bank 冲突。
3. **引擎读不得了**。GPU 里有一块专门算矩阵乘法的硬件单元, 叫 **Tensor Core(张量核心)**——为什么 GPU 要为「矩阵乘」专门造一块硬件? 因为深度学习里绝大部分计算就是一堆矩阵乘, 把它做成专用硬件就能飞快。但这块硬件很挑食: 它只认 **特定摆法** 的数据。一块从大矩阵上切下来的小方块 (下文叫 **tile / 分块**), 它的字节怎么排, 直接决定 Tensor Core 肯不肯把它当成一个合法的「操作数 (operand, 就是参与运算的输入数据)」吃进去。摆错了, 这块昂贵的硬件根本用不上。

这一章的讲法是一层一层往上垒: 先搭最朴素的「形状-步长」模型, 再一点点往上加东西 (tile、命名轴、复制项), 最后才到 swizzle。讲到后面你会发现一个让人安心的事实: 整章翻来覆去, 其实就是 **同一个想法换着花样讲**——把它学透一次, 后面全是变奏。

一、形状-步长模型 (The Shape-Stride Model)

咱们从最简单的布局讲起。说到底, 描述一个布局只要两样东西: 一个 **形状 (shape)**——告诉你每个维度有多大; 加上一组对应的 **步长 (stride)**——告诉你每个维度的下标加 1, 地址要往前走多远。本书用一套很紧凑的记号把这俩写在一起, 长这样:

```
S[(shape) : (strides)]
```

S 你就读成「一个布局」, 中间的冒号把它劈成两半, 左边是形状、右边是步长。

那怎么用它算出「某个下标存在哪儿」? 办法简单到出乎意料: 把「下标」和「步长」**对应相乘再加起来** (这个运算数学上叫「点积」, 你不熟这个词也没关系, 就是「逐个相乘后求和」)。拿一个行主序 (row-major, 意思是「一行一行地、从左到右、连续地」往内存里铺) 的 4×4 矩阵来说:

```
S[(4, 4) : (4, 1)]      addr(i, j) = i.4 + j.1
```

步长 (4, 1) 是啥意思? 其实特别好懂, 你跟着想一遍就通:

- 行下标 *i* 每加 1, 意味着「往下挪一整行」。一行有 4 个元素, 所以地址得往前 **蹦 4 步**——这就是第一个步长 4。
- 列下标 *j* 每加 1, 意味着「在同一行里往右挪一格」, 紧挨着的下一个, 所以地址只 **往前挪 1 步**——这就是第二个步长 1。

就这么个经典模型, 没有任何玄机。(如果你听过 CuTe 这个库, 这套记号就是它的行主序简化版; 没听过完全不影响。)

你其实早就用过它

别把它当成什么新鲜玩意儿, 这套东西你很可能早就用过了。只要你写过 PyTorch 或 NumPy(就算没写过, 下面的代码也很好懂), 你就已经在用「形状 + 步长」了。这些库里的张量, 说白了就是一个 shape 加一个 stride, 盖在一段扁平的 (也就是一维的、拉直的) 存储缓冲区上面——底层永远是一条直线排列的数, shape 和 stride 只是教你怎么在这条直线上「按下标找位置」:

```
import torch
t = torch.arange(12).reshape(3, 4) #      0-11      ,      3      4
t.shape      # torch.Size([3, 4]) <-      :3      4
t.stride()   # (4, 1)              <-      :      S[(3, 4) : (4, 1)]
```

看到没, `t.stride()` 直接就吐出了 (4, 1)——和我们手推的一模一样。一旦你学会用「shape + stride」这个视角看张量, 很多「重排」操作为什么 **压根不碰数据、快得几乎不要钱**, 一下就通了。它们干的事无非是 **改写那几个步长数字**, 然后在同一段存储上给你返回一个 **视图 (view, 可以理解成「换一副眼镜看同一块内存」, 数据本体没动)**。最典型的就是转置 / permute(把行和列对调):

```
tt = t.permute(1, 0)          #      0      1      ,      t.T
tt.shape                     # torch.Size([4, 3]) <-      3x4      4x3
tt.stride()                  # (1, 4) <-      (4,1)      (1,4)
tt.data_ptr() == t.data_ptr() # True <-      !
```

最后那行 `data_ptr()` 是「这块张量的数据从哪个内存地址开始」。两者相等, 铁证如山: **转置前后是同一字节, 根本没搬家**。`t.permute(1, 0)` 不过就是同一块内存上换了组步长 `S[(4, 3) : (1, 4)]` 而已。换句话说, **所谓转置, 就是把步长换了换, 一个字节都没挪**。`reshape` / `view` 作用在连续张量上也是同样的套路——给老存储套上一副新 shape、新 stride 的眼镜, 如此而已。

为什么要在 GPU 笔记里花这么大篇幅讲 PyTorch? 因为这是你 **唯一一个已经熟的锚点**。记住「布局 = shape + stride、改布局往往不动数据」这件事, 后面 GPU 上那些花里胡哨的写法, 本质都没逃出这个框。

注意

注意:NumPy 的行为跟这一模一样, 唯一的区别是它的 `.strides` 按「字节」算, 而 PyTorch 的 `.stride()` 按「元素」算。

零拷贝推理的边界

下面这张图把「逻辑视图」和「物理存储」的关系串了起来:

结构图

连接关系:

- 逻辑下标 $(i, j) \xrightarrow{\text{下标} \cdot \text{步长} (\text{点积})} \text{线性地址 } \text{addr}$
- 线性地址 $\text{addr} \rightarrow \text{扁平存储缓冲区 } [0][1][2][3][4] \dots [11]$

GPU 上的布局也是这么回事。一块 `tile` 的映射, 不管是映到内存, 还是用后面要讲的「命名轴」映到通道和寄存器 (这俩词后面会讲, 先不急), 骨子里都是同一回事: 「固定缓冲区上的一条步长规则」。所以 **重排一块 `tile`, 多数时候只是换了个布局, 并没有真去拷贝数据**——和 PyTorch 转置一个道理。

关键

关键: 但「零拷贝」这件好事, 是有边界的, 别以为永远免费。它只在「同一块连续地址空间上换个看法」时才成立。到了 GPU 上, 这件事只在一种情况下管用: 新看法跟现有的字节排布、跟「谁拥有这块数据」的安排还对得上。一旦你动了「**哪个线程、哪个寄存器拥有某个元素**」(注意: GPU 上一个元素可能不是躺在公共内存里, 而是被某个具体线程的某个寄存器「私人持有」, 后文会讲), 或者改了共享内存的 `swizzle` (最后一节才讲), 那就 **躲不掉了**, 必须老老实实搬数据。搬数据靠的是 `load` (从内存读)、`store` (写回内存)、`shuffle` (线程之间互相交换数据)、`ldmatrix`、`transpose` 这些专门的操作——这些名字你现在记不住没关系, 只要知道「真搬数据是要花成本的」就够了。后面好几章的优化, 根子都扎在这条边界上。

二、Tile 布局 (Tile Layout)

前面讲的都是「整个张量」的布局。但现实里, GPU 上跑的程序 (一段在 GPU 上执行的函数, 术语叫 **kernel / 核函数**, 你就理解成「丢给 GPU 去并行跑的那段代码」) 几乎从不会一口气啃下整块大矩阵。

为什么不能一口气啃? 因为那块又快又小的存储装不下整个大矩阵——就像你不可能把整个数据库一次性塞进 CPU 缓存。所以 GPU 的标准做法是: 把大矩阵 **切成一个个更小的方块**, 这些小方块叫 **tile (分块)**, 然后丢给硬件的不同部分, 分头去加载、变换、计算。这个「切块」的动作就叫 **tiling (分块)**。

好消息是: **搞 tiling, 不需要学任何新概念**。它还是一个布局, 无非就是把下标拆细、多写几个维度罢了。

举个例子, 把一个 8×8 矩阵切成若干 2×4 的小块 (每块 2 行 4 列)。横着 2 块、竖着 4 块, 一共 8 块。本来用 (i, j) 两个数就能定位的元素, 现在我们改用 **4 个数** 来定位: $(\text{tile_row}, \text{row_in_tile}, \text{tile_col}, \text{col_in_tile})$ ——意思是「第几块行、块内第几行、第几块列、块内第几列」。所以这是一个 **4 维** 布局。步长怎么挑? 有个明确目标: 让每个 `tile` 内部的元素在物理内存里连续摆放 (这样硬件一次就能把一整块 `tile` 利索地读进来):

```
S(4, 2, 2, 4) : (16, 4, 8, 1)
```

映射就分两步, 慢慢来:

1. **先把逻辑 (i, j) 拆成四个数 $(i//2, i\%2, j//4, j\%4)$** 。这里 `//` 是整除、`%` 是取余 (和 Python 一样)。整除给出「第几块」(外坐标), 取余给出「块内第几个」(内坐标)。

- **TMEM(Tensor Memory):** 一种专给 Tensor Core 用的片上存储, 是 NVIDIA 较新的 GPU(Blackwell 代, 后面会再提) 才有的。你现在只要知道「这又是一种存数据的地儿」就行, 细节后面讲。

现在问题来了: 数据能待的地方这么多, 我们怎么用 **同一套记号**把它们全部描述清楚, 而不是每种情况发明一套写法? 作者这招很巧: **给每个步长系数贴一个「轴标签」**, 用这个标签说清楚——这个下标加 1 时, 是「在内存里走」, 还是「换一个线程」, 还是「换一个寄存器」。

标签	含义
@m	普通内存 (memory)
@laneid	warp 通道索引, 即 thread_index % warp_size
@reg	每通道私有的寄存器 (register)
@warpid	warp 索引
@TLane / @TCol	TMEM 的通道 / 列坐标

贴好标签, 一个行主序的 8×16 内存 tile 就写成这样:

```
S[(8, 16) : (16@m, 1@m)]
```

(两个步长都带 @m, 说明它们都指向线性内存。)

读法速成: 手把手念一行 S[...]

你可能已经被后面那种 S[(8,4,2):(4@laneid,1@laneid,1@reg)] 的写法吓到了——一堆 @ 符号, 看着像天书。别慌, 我跟你保证: 它再花哨, **念法永远是固定的那么几步**。把这套套路记死, 后面所有表达式你都能像查字典一样一个字一个字读出来, 而不用害怕。

关键

关键:S[(): ()] 就念成一句话——「这块数据有这么大 (形状); 每个下标 +1, 就在某个空间里走这么远 (步长)」。

第 1 步: 看中括号。 S[...] 就代表”一个布局”。中间的冒号 : 把它劈成两半: **左边是形状, 右边是步长**, 两边的项一一对应。

第 2 步: 读形状。 (8, 4, 2) 就是说这块数据是 8 × 4 × 2, 三个维度, 没别的。

第 3 步: 读步长里的 N@axis(这是命门)。 N@axis 念作:「**这一维的下标每 +1, 就沿 axis 这个轴走 N 步**」。关键全在 @ 后面那个轴——它告诉你”走在哪个空间里”:

轴标签	走在哪
@m	内存地址上
@laneid	warp 的 32 个通道 (lane) 上
@reg	某个通道自己的寄存器上
@TLane / @TCol	TMEM 的通道 / 列上

第 4 步: 同一个轴出现多次, 就把它加起来。这一步容易漏, 留心。要是有两个维度的步长都带 @laneid, 说明这俩维度 **合伙决定**「落在哪个 lane(哪个线程)」——把它们各自算出的「下标 × 步长」加到一起, 才是最终的 lane 号。(为什么会有两维合伙? 因为一个 warp 有 32 个 lane, 有时候用「8×4」这样两维的形状去铺这 32 个 lane, 比用一维更顺手。)

把这套套到那个吓人的例子上:S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]。假设我想知道逻辑元素 (i=2, j=3, r=1) 到底住哪:

- 前两维都带 @laneid, 合起来算 lane:lane = $2 \times 4 + 3 \times 1 = 11$;
- 第三维带 @reg:reg = $1 \times 1 = 1$ 。
- 结论: 元素 (2,3,1) 住在第 11 号 lane 的 1 号寄存器里。

(顺手验证一下: 第一维 $8 \times$ 第二维 $4 = 32$, 正好等于一个 warp 的 32 个 lane——所以这块数据刚好摊满一整个 warp, 每个 lane 手里攥 2 个元素。)

最后一块拼图: + R[n : stride]。有时你会看到 $S[\dots] + R[\dots]$, 那个 R 是复制 (replication): 把前面 $S[\dots]$ 定好位置的那份数据, 再沿某个轴抄 n 份, 相邻两份隔 stride 那么远。比如 $+ R[4 : 32@TLane]$ 念作“沿 TMEM 通道轴复制 4 份、每份隔 32 个通道”, 也就是第 0、32、64、96 号通道拿的是同一个值。

记住这套念法, 后面 @TLane、@gguid_x、 $R[\dots]$ 再怎么排列组合, 你都能一个字一个字拆开读懂, 而不用死背结论。

标签真正发光的时刻: 数据跨线程分布

光给一个内存 tile 贴个 @m, 你看不出这套标签有啥了不起——不就是「内存地址」嘛。标签真正出彩的时刻, 是描述那种 **CPU 世界里根本没有**的情形: 一块数据不是整整齐齐躺在一块公共内存里, 而是 **被拆成几十份, 分别塞进几十个线程各自的私有寄存器里**。这种「数据散在好多个线程上」的局面, 用普通的 shape/stride 根本没法表达, 而命名轴一招就搞定。看这个例子:

```
S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]
```

这个布局不再指向线性内存了, 它把「行、列」映射到了 **lane ID** 和 **每个通道里的寄存器**上:

- 前两维 (形状 8 和 4) 的步长都带 @laneid。也就是说, 这俩逻辑维度合起来决定「数据落在 warp 的哪个通道」(lane = $0 \cdot 4 + 1 \cdot 1$, 一共 $8 \times 4 = 32$ 个 lane, 正好把一个 warp 填满)。
- 最后一维 (形状 2) 的步长带 @reg。意思是这一维顺着「同一个通道内的寄存器」展开, 说白了就是每个 lane 手里攥着 2 个元素 r0 和 r1。

结构图

连接关系:

- 逻辑张量 (8, 4, 2) $\rightarrow$ @laneid 维 ($8 \times 4 = 32$): 决定落在哪个 lane
- 逻辑张量 (8, 4, 2) $\rightarrow$ @reg 维 (2): 决定在该 lane 的哪个寄存器
- @laneid 维 ($8 \times 4 = 32$): 决定落在哪个 lane $\rightarrow$ warp 的 32 个通道
- @reg 维 (2): 决定在该 lane 的哪个寄存器 $\rightarrow$ 每通道私有寄存器 r0 / r1

关键

关键: 别小看这个例子——它恰好就是 Tensor Core 实际用的「寄存器片段 (register fragment)」布局。所谓寄存器片段, 就是「一块矩阵被拆开、分散存放在各个线程的寄存器里的那一份份小碎片」; Tensor Core 算矩阵乘时, 输入数据就是以这种碎片形式分散在一个 warp 的各个线程手里的。后面《Tensor Core Operand Layouts Across GPU Generations》那一章会反复用到。这里你只要体会到一点: 同一套 $S[\dots]$ 记号, 既能描述「数据在内存里怎么摆」, 又能描述「哪个线程、哪个寄存器拿着哪个元素」——这套记号的厉害之处, 正在于此。

注意

注意: 原书在这儿也配了交互演示, 点一下格子就能看到它是由哪个 lane / register 持有的。想把「逻辑元素 ↔ 物理 lane/reg」这层对应关系摸熟, 去原书玩一玩这个演示。

四、分布式布局 (Distributed Layout)

命名轴最妙的一点是: 同一套记号, 能在好几个完全不同的层级上描述「数据放在哪」——小到「一个线程的寄存器」, 大到「机房里的好几台 GPU」。前面我们用它讲了单张显卡内部的 lane 和 register, 现在把同一个想法直接往外放大, 放大到一整片 GPU 网格 (mesh, 就是把多张 GPU 排成一个网格阵列, 比如 2×2 四张卡一起干活) 上。

为什么会有多张卡一起干活的场景? 因为现在的大模型, 一张卡的内存根本装不下, 只能把模型和数据 切开, 分摊到好几张卡上——这就是分布式训练 / 推理。那怎么用命名轴描述这种「数据摊在哪张卡上」? 加两个新轴就行:

- @gguid_x / @gguid_y 这两个轴, 说的是「数据落在网格里的哪张卡上」(x 是横坐标,y 是纵坐标);
- 有了它俩, 这套记号就能精确描述「把一个大张量切开、分摊到多张卡」的 分片 (sharding) 模式了。

用 R[...] 描述复制

命名轴几乎啥都能描述, 但还缺一块拼图: 复制 (replication)。什么叫复制? 就是「同一份数据, 被原样拷到好几个地方, 每处都存一份一模一样的」。分布式里这很常见——有些数据每张卡都要用, 那就干脆每张卡都存一份副本, 省得用的时候还要去别的卡上要。为描述这种情形, 作者添了个新记号:

R[n : stride]

R 就读成「复制」。**n** 是复制几份, **stride** 是相邻两份副本之间隔多远 (也照样能带轴标签, 告诉你副本是沿哪个轴铺开的)。比如 R[2 : 1@gguid_x], 念作「沿 @gguid_x 轴 (横向) 复制 2 份」。

把分片和复制拼一块儿, 一个表达式就能一口气说清两件事: 既说「张量怎么切开分摊到各张卡」, 又说「哪些卡之间互为副本」。比如下面这个, 在 2×2 的 GPU 网格上, 先纵向分片、再横向复制:

S[(2, 4, 8) : (1@gguid_y, 8@m, 1@m)] + R[2 : 1@gguid_x]

下面这张图, 画的就是这个「分片 + 复制」模式落在 2×2 网格上是什么样:

横轴 @gguid_x 是复制方向, 纵轴 @gguid_y 是分片方向。网格上每台 GPU 手里拿着啥, 看下表:

	@gguid_x = 0	@gguid_x = 1(复制方向)
@gguid_y = 0	GPU(0,0): 分片 A	GPU(1,0): 分片 A 副本
@gguid_y = 1	GPU(0,1): 分片 B	GPU(1,1): 分片 B 副本

- S[...] 里的 1@gguid_y → 张量沿 y 轴在两台设备间「分片」(分片 A / 分片 B);
- R[2 : 1@gguid_x] → 这个分片再沿 x 轴「复制」一份给配对的设备 (副本)。

注意

注意: 原书的交互演示能点格子看它由哪台 (或哪几台) 设备持有, 还能在「完全分片 / 分片 + 副本 / 分片 + 偏移」三种布局之间来回切。静态图没法还原这种切换, 想看个明白就去原书体验一下。

4.1 kernel 内的复制:TMEM 中的 Scale Factor

一句话先理解

一句话先理解: 这一小节是个稍微硬核的实例, 讲「复制」不光用在多张卡之间, 也用在一张卡内部——硬件把一份数据「广播」给好几个线程组共用。看不太懂细节没关系, 抓住「复制 = 同一份数据让多处共享」这一个点就够了。

复制维度 $R[\dots]$ 可不光是给「多张卡」用的。同样这套写法, 也能描述单张卡内部、一段 kernel 跑的时候发生的事——硬件把同一份数据广播给一个 warp 之外的别的 warp 共用。这里拿一个具体例子:Blackwell(NVIDIA 较新一代 GPU 架构代号) 上的 block-scaled MMA。这里头 MMA 是 matrix-multiply-accumulate(矩阵乘加) 的缩写, 就是 Tensor Core 干的核心活;block-scaled 是一种节省内存的技巧, 简单说就是给一批数配一个「缩放因子」, 用的时候再乘回去。

这个缩放因子 (scale factor) 存在前面提过的 TMEM(那块专给 Tensor Core 用的存储) 里。妙的地方在于: 一个逻辑上有 128 行的缩放向量, 物理上只占了 32 条 TMEM 通道 (可以理解成 32 个槽位)——也就是说,128 行被压进了 32 个槽位里。

- 逻辑行 $r \rightarrow$ TMEM 通道 $r \% 32$;
- $r // 32$ 沿列方向展开。

接着是关键「广播」一步。这压好的 32 条 TMEM 通道, 会沿着 TMEM 的 Tlane 轴复制成 4 份, 从 32 条铺成 128 条。为什么要复制成 4 份? 因为读这份数据的是一个 warpgroup——4 个 warp 凑成的一组, 共 $4 \times 32 = 128$ 个线程。让每一份副本对上一个 warp, 这样这 4 个 warp 各自都能在自己那 32 条通道的窗口里读到一份完整的缩放因子, 谁也不用跑去别的 warp 那儿要。这种「一份数据复制给 4 个 warp 共用」就叫一次 $\text{warp} \times 4$ 广播, 用复制维度写出来就是:

S[(32, $\hat{e}$) : (1 $\hat{e}$ Tlane, $\hat{e}$)] + R[4 : 32 $\hat{e}$ Tlane]

这就给出了 4 份副本, 相邻两份隔 32 个 TMEM 通道。换句话说,TMEM 通道 1、1+32、1+64、1+96 拿的是同一个缩放值。

存储阶段:128 逻辑行 $\rightarrow$ 32 TMEM 通道

- 行 $r \rightarrow$ TMEM lane $r \% 32$, 列偏移看 $r // 32$;
- 总共只动用了 32 条通道 (lane 0 $\cdots$ lane 31)。

广播阶段: $\text{warp} \times 4$ 复制 ($R[4 : 32\hat{e}\text{Tlane}]$)

通道 1、1+32、1+64、1+96 拿的是同一份缩放值; 每个 warp 的 32 通道窗口各自分到一份副本:

	warp0	warp1	warp2	warp3
TMEM 通道范围	0..31	32..63	64..95	96..127

关键

关键: 跟 GPU 网格那回事一样, 复制维度本身一个新数据都不带。它只是声明「同一个值, 同时坐在 4 个 TMEM 通道上」, 就像 @gpuid_x 把一行广播到整个 GPU 网格那样。真正去读它的, 是这些 warp 里的线程。

注意

注意: 每列内部的字节怎么打包 (scale_vec 的 1X/2X/4X 模式), 还有 cta_group::2 的拆分, 这些留到《Tensor Core Operand Layouts Across GPU Generations》再细说。原书在这儿也有交互演示, 把「压紧塞进 32 个 TMEM 通道」和「warp_x4 广播到 128 个读取通道」这两步连着演给你看, 值得一看。

要是你熟悉 CuTe, 本章的记号这么理解就行: 它就是 CuTe 的一个 **行主序变体**, 只不过多塞了「显式的硬件命名轴」和「专门的复制结构」。

五、Swizzle 布局 (Swizzle Layout)

这一章的最后一个布局, 是专门为治一个具体的硬件毛病而生的。先讲毛病, 再讲药方——这样你才明白为什么需要它。

问题:bank 冲突

先把舞台搭好。GPU 有一块叫 **共享内存 (shared memory, 常简写 SMEM)** 的存储——它比普通内存快得多, 而且一个线程块里的线程可以拿它当「公共白板」互相传数据。但它有个物理结构: 被切成了好几个 **bank(存储体, 前面用「并排的小柜台」打过比方)**。

规则是这样的: 一个 warp 的若干线程同时来访问共享内存时——

- 如果它们恰好落在 **不同的 bank(不同柜台)**, 那就 **同时办完**, 飞快;
- 可一旦好几个线程挤去访问 **同一个 bank 里的不同地址** (同一个柜台、要不同的东西), 这个柜台一次只能服务一个, 硬件只能让它们 **排队、一个接一个来**。

这种排队就是 **bank 冲突 (bank conflict)**, 它会本本该一步完成的访问拖成好几步, 白白浪费时间。

在矩阵程序里, 这个坑几乎绕不过去。为什么? 因为算矩阵时, 我们老是要对同一块 tile **既按行读、又按列读** (比如矩阵乘法, 一个矩阵取行、另一个取列)。而一种摆法没法同时讨好行和列, 于是你被逼进一个两难:

布局	行访问	列访问
利于行的布局	高效	产生 bank 冲突
利于列的布局	产生 bank 冲突	高效

Swizzle(混洗 / 异或重排) 这招, 就是冲着同时讨好行和列、打破这个两难来的。

思路：用 XOR 把地址打散

一句话先理解

一句话先理解:swizzle 就是「故意把数据存的位置打乱一下」，乱得恰到好处，使得无论你按行抓还是按列抓，抓到的那一组都正好分散在不同的柜台 (bank) 上，谁也不挤谁。

Swizzle 的核心，就是 在算地址时再做一道手脚，把地址打散重排。最典型的玩法，是拿列索引跟行索引做一次 异或 (XOR，就是按位「相同得 0、不同得 1」的那个位运算，很多语言里写作 ^)。别纠结 XOR 的数学，你只要知道它有个好性质：能把原本规规矩矩、容易撞车的一组地址，搅成一组分散开的地址。这么一搅，无论 你按行读还是按列读，落到的 bank 都被打散开了，不再扎堆。

下面拿一个 8×8 tile，把「朴素行主序」(不做任何手脚) 和「XOR swizzle」(做了手脚) 摆一块儿对比。假设只有 8 个 bank，我们盯住「读某一整列那 8 个元素」时，它们分别落进哪些 bank：

读「第 c 列」那 8 个元素，各自落进哪个 bank：

行	朴素行主序 (bank = c)	XOR swizzle(bank = c XOR row)
行 0	bank c	bank (c XOR 0)
行 1	bank c	bank (c XOR 1)
...	...(全是同一个 bank)	...
行 7	bank c	bank (c XOR 7)
结果	8 个元素全落进同一个 bank → 串行化成 8 个周期	同一列被打散到 8 个不同 bank → 1 个周期读完

关键

关键:swizzle 给的「无冲突保证」是 带条件的。只有当「元素宽度、swizzle 模式、访问模式」三者对得上 (也就是凑成某个引擎 descriptor 期望的那一组) 时，它才成立；并不是随便什么元素宽度、什么对齐方式都灵。一句话：模式选错，保证作废。

真实硬件：分段 + atom

上面那个 8 个 bank 的 8×8 只是方便讲解的玩具。真实 GPU 的 bank 多得多，一般是 32 个。为了在真实规模上也好用，swizzle 不会把一整块 tile 当成铁板一块来打乱 (那样太死板)，而是先把内存切成一小段一小段 (segment)，然后在 每一段内部各自施加打乱模式。

最常见的一种叫 SWIZZLE_128B，它以 128 字节为一段来组织。为什么偏偏挑 128 字节？因为它正好对得上 32-bank 的硬件：32 个 bank，每个 bank 一次处理 4 字节，32 × 4 = 128 字节，刚好一段铺满所有 bank。它打乱地址的规则，核心就一行：

```
physical_sector = logical_sector XOR row
```

(sector 你就理解成「段里的一小块」,row 是行号。) 在每个 128 字节段里头，光靠这一条 XOR 规则，就能把每一列都打散到不同的 bank 上，消掉冲突。

不同的 swizzle 模式 (atom 大小)

把上面的思路再抽象一层: 硬件先定一个 又小、又会反复出现的基本图案, 叫 **atom**(原子单元)——你可以把它想成「贴瓷砖时那一块标准瓷砖」。打乱规则就施加在这一块 atom 上; 然后整块大 tile 就拿这块选定的 atom 一格一格平铺 (**tiling**) 出来, 像铺地砖一样铺满。不同的 swizzle 模式, 区别就在于这块「瓷砖」多大。

模式	atom 形状	适用条件 (连续维度至少)
SWIZZLE_128B	8 × 128 B	128 字节 (fp16 下即 64 个元素)
SWIZZLE_64B	8 × 64 B	64 字节
SWIZZLE_32B	8 × 32 B	32 字节
16 B 交错 (interleaved) 模式	原书未给出具体 atom 形状	用于更小的连续维度

该选哪个模式?

一条经验法则:tile 能填满哪块最大的「瓷砖」(atom), 就挑哪块。

- 一块 N 字节的 atom, 要求 tile 的 连续维度 (就是内存里连着摆的那一维, 通常是「行方向」) 至少有 N 字节宽, 而且得是 N 的整数倍——不然瓷砖铺不满、对不齐。
- 所以 SWIZZLE_128B 只有在一行至少跨 128 字节 (也就是 64 个 float16 元素) 时才用得上。
- 而一旦用得上, 它就是首选。它那个 8 × 128 B 的 atom 刚好铺满一整条 128 字节的 bank line, 所以能一把就把一列打散到全部 32 个 bank; 在 fp16 下,8 行 8 列读起来都不撞 bank。
- 要是问题形状把连续维度逼得很小,128 B atom 填不满, 那就退一档, 换 SWIZZLE_64B 或 SWIZZLE_32B——原则不变, 就是「一行能盖得住的最大 atom」。

Swizzle 与 S[...] 记号的关系

关键

关键: 好消息——这些被打乱过的地址, 你 根本不用自己去手算, 工具会替你算。但有个概念得分清楚:swizzle 不属于 S[...] 那套布局。前面 S[...] 算地址用的都是「下标乘步长再相加」这种规规矩矩的线性运算 (数学上叫 仿射映射 / affine, 你理解成「只有乘和加、没有 XOR 这种位运算」就行); 而 swizzle 里有 XOR, 不是这种规矩运算, 所以它是 非仿射 (non-affine) 的、单独叠在 S[...] 上头的一层。

换句话说, 算最终地址要分两步:S[...] 布局先把元素安顿到一个普通的线性内存 (0m) 地址, 然后 swizzle 再把这个地址 XOR 打乱一下, 得到真正下手的位置。在 TIRx(本书配套的一个编译工具) 的 Layout API 里, 这两层「叠在一起」写成一个函数调用:

```
ComposeLayout(swizzle, tile)
```

ComposeLayout(swizzle, tile) 字面意思就是「把 swizzle 这一层和 tile 布局这一层组合成一个整体」。组合好之后, 你要操心的就只剩一件事: 凡是会碰这块 tile 的代码, 都得统一用同一个 swizzle 模式 (不然有人按打乱前的地址写、有人按打乱后的地址读, 数据就对不上了)。除此之外的算地址细节, 全甩给这个「组合后的布局」去自动摆平。

结构图

连接关系:

- 逻辑下标 $\xrightarrow{S[\dots] \text{ 仿射映射}}$ 线性 @m 地址
- 线性 @m 地址 $\xrightarrow{\text{swizzle 非仿射置换}}$ 实际 SMEM 地址

与 tiling、TMA 的会合

这个「组合后的布局」，恰好也正是 **硬件搬数据时要照着填的那张图纸**。swizzle 和 tiling 这两件事，就在这儿合流了。这里要引入一个新名词 **TMA**——它是 GPU 上一个专门负责「成批搬运 tile」的硬件搬运工（全称 Tensor Memory Accelerator，细节在《Async Data Movement: TMA》那章），你现在只要知道「它能一次性把一块 tile 从大内存搬进共享内存」就够了。

- 给 TMA 下指令用的是一个 **TMA descriptor**（可以理解成一张「搬运说明书」），它本身就是多维的，所以一张说明书一次就能把两件事都交代清楚：tile 的那些 atom「瓷砖」怎么平铺，外加每块瓷砖内部怎么 swizzle 打乱；
- 于是一次 TMA 搬运，就能 **一块瓷砖一块瓷砖地把整个 tile 铺进共享内存，并在写进去的同时顺手把 swizzle 也做掉**——不用先搬一遍、再单独跑一遍打乱，一步到位，省时间。

注意

注意：至于每个引擎到底「要哪种」swizzle，这是 **看代际 (generation)** 而定的，正好就是下一章的内容。

小结

这一章一层一层往上垒，最后搭出了一套贯穿全书的「布局语言」。回头看，每一层都只是在前一层上加了一点点东西：

1. **形状-步长模型** $S[(\text{shape}):(\text{strides})]$ 是地基——逻辑下标跟步长做点积，就得到物理地址。它其实就是 PyTorch/NumPy 张量背后那套 shape+stride。转置、reshape 为啥能零拷贝？就因为它们只改步长、不碰数据。
2. **Tile 布局**没添任何新概念，无非就是把下标拆成「外坐标 / 内坐标」、再多写几个维度。
3. **命名轴** (@m、@laneid、@reg、@warpid、@TLane、@TCol、@gpuid_x/y) 把同一套记号推到了内存、寄存器、通道、TMEM，乃至整个 GPU 网格，让「数据在硬件资源上怎么铺」也能精确写出来。
4. **复制项** $R[n:\text{stride}]$ 描述「同一份数据被广播到好几处」，分布式分片里的副本是它，Blackwell block-scaled MMA 在 TMEM 里的 warpx4 广播也是它。复制维度本身不带新数据。
5. **Swizzle** 拿 XOR 把地址重排，消掉 SMEM 的 bank 冲突，让按行读、按列读都快。它是叠在 $S[\dots]$ 上头的一层非仿射变换 ($\text{ComposeLayout}(\text{swizzle}, \text{tile})$)，按 atom(8×N B) 分段施加；挑法是「能填满的最大 atom 优先」，默认就用 SWIZZLE_128B。

最后，记牢贯穿全章那句话：**同样一批数字，换个物理摆法，在同一块 GPU 上跑出来的性能能差一个数量级**。而 $S[\dots] + \text{命名轴} + R[\dots] + \text{swizzle}$ ，就是用来精确说清、并攥住这种物理摆放的家伙事儿。

本章局限说明: 原书一共有 7 处交互演示 (tile 拆分、lane/reg 分布、GPU 网格分布、TMEM warp×4 广播、8×8 bank 冲突对比、SWIZZLE_128B 分段、各 swizzle 格式 atom 浏览)。这份笔记已经用 ASCII 图和表格把它们的核心要点重搭出来了, 但点击切换、逐周期步进这类交互, 静态文档实在还原不了。想真正吃透, 最好配着原书的交互演示一起看。

延伸阅读

- 原文章节:[Data Layout and Its Notation](#)
- 相关后续章节 (原书内部引用):
 - *Tensor Core Operand Layouts Across GPU Generations* ——寄存器片段、block-scaled MMA、scale_vec、cta_group::2
 - *Async Data Movement: TMA* ——TMA descriptor 如何一次性完成 atom 平铺 + swizzle
 - *TIRx Layout API* ——ComposeLayout(swizzle, tile) 的实际接口
- 概念背景:CuTe(本章记号是其行主序变体)、PyTorch/NumPy 的 stride 模型

术语对照

中文	English
数据布局	data layout
形状	shape
步长	stride / strides
视图	view
分块 / 瓦片	tile / tiling
命名轴	named axes
warp 通道	lane / laneid
寄存器	register / reg
复制 (广播)	replication
分片	sharding
GPU 网格	GPU mesh
缩放因子	scale factor
访存合并	coalescing
内存 bank	memory bank
bank 冲突	bank conflict
共享内存	shared memory (SMEM)
混洗 / 异或重排	swizzle / swizzling
原子单元	atom
仿射 (映射)	affine (map)
张量核心	Tensor Core
寄存器片段	register fragment

第 04 章 · 各代 GPU 的 Tensor Core 操作数布局

原文:Tensor Core Operand Layouts Across GPU Generations

本章要点

本章要点 (TL;DR)

先用大白话铺一句:GPU(显卡) 是一种擅长”一次干很多算术”的芯片;在做 AI 计算时,最累的活几乎都是**矩阵乘法**(两个数字表格相乘)。于是 NVIDIA 在 GPU 里塞了一块专门干这件事的硬件,叫 **Tensor Core**。Ampere、Hopper、Blackwell 是它三代芯片的代号(就像手机的”几代”),由老到新。这一章讲的就是:这块硬件三代下来,”数据要摆成什么样它才肯算”是怎么一路变化的。

- 三代以来,Tensor Core 干的活始终是同一件: $D = A \cdot B + C$ (把矩阵 A、B 相乘,再加上 C,这叫**矩阵乘累加 / MMA**)。真正在变的,是**数据(操作数)怎么喂进 Tensor Core**、它支持哪些尺寸和数据类型、以及算出来的结果(**累加器 / accumulator**)放在哪块内存里。
- **Ampere(最老这代)**: 数据被拆碎、摊在一组线程(后面会解释 warp / lane)各自的小存储格(寄存器)里,这种摊法叫**寄存器片段(register fragment)**。流程是:用一条叫 `ldmatrix` 的特殊指令把数据从共享内存(SMEM)搬进寄存器,再用 `mma.sync` 在寄存器之间算,结果也一直留在寄存器里。
- **Hopper(中间这代)**: 新指令 `wgmma` 可以直接从共享内存读数据,靠一份叫**矩阵描述符(matrix descriptor)**的”说明书”告诉硬件数据是怎么摆的。但算出来的结果仍然放在寄存器里。
- **Blackwell(最新这代)**: 读数据的方式基本沿用 Hopper,但把结果搬进了一块新内存 **TMEM(Tensor Memory)**;某些省内存的计算模式还会多出一类”缩放因子(scale factor)”数据,也要经 TMEM 中转。
- 还有两条最基础的内存规矩,三代从没消失过:**全局内存合并访问(coalescing)**和**共享内存 bank 冲突(bank conflict)**——这两名词后面第一节会从零讲清楚。一句话先记住:数据怎么摆,不是 Tensor Core 程序的”装饰”,它本身就是硬件接口的一部分。摆错了,硬件不会报错,只会默默读错字节、或者读得很慢。

前置知识

前置知识:这一章会大量提到几个 GPU 基本概念,我先在这里用一句话各自交代一下,正文第一次用到时还会再展开。你**不需要**提前精通,扫一眼有个印象就行:

- **线程(thread)**:GPU 上最小的”干活的人”。和 CPU 程序里的线程类似,但 GPU 会同时跑成千上万个。
- **warp(线程束)**:GPU 把线程按 **32 个一组**捆在一起调度,这一组就叫一个 warp。可以想

成”32 个人组成的一个班，必须迈同一歩走”。

- **lane(通道):warp** 这个 32 人班里，每个人的编号，从 0 到 31。说”lane 5”就是指这个 warp 里第 5 号线程。
- **寄存器 (register):** 每个线程私有的、速度最快的一丁点存储格，类似 CPU 寄存器，但每个线程各有一份。
- **寄存器片段 (register fragment):** 一块数据被拆碎，分别塞进 warp 里 32 个 lane 各自的寄存器中，合起来这一摊就叫片段。
- **共享内存 (SMEM) / 全局内存 (GMEM) / TMEM:**GPU 上几种不同的内存，容量和速度各不相同，后面会逐一解释。
- **MMA:** 矩阵乘累加， $D = A \cdot B + C$ ，就是 Tensor Core 干的那件核心算术。
- **第 3 章的布局记号:** 形如 $S[(8,4):(\dots)]$ 的一套写法，用来精确描述”数据在内存里怎么摆”。本章会用到，但每次出现我都会翻成人话，你看不懂符号也不影响理解结论。

想打更扎实的底子，可以先翻第 0 章·极简入门和第 3 章《数据布局》。

引子：为什么”逻辑公式没变”还会算错

先解释两个词，后面会反复用。**tile**：一个大矩阵太大，放不进高速内存，所以会切成一个个小方块来处理，这种小方块就叫 tile(瓦片)，你就理解成”矩阵切出来的一小块”。**kernel(核函数)**：在 GPU 上跑的那段程序，相当于”GPU 版的一个函数”。本章说”一个 kernel”时，你脑子里就想”一段在显卡上执行的代码”。

好，现在来看 Tensor Core(GPU 里专门做矩阵乘法的那块硬件)。你远远地看它，会觉得它特别稳：喂进去 A、B 两块 tile，再带上累加器 C(存放”要累加进去的那部分结果”的一块数据)，它就吐出来一个结果 D。这个样子打很早的型号 (Volta 那一代) 起就没变过。你可凑近一看就会发现，**这个运算周围的种种细节，其实一直在悄悄变。**

这一变，就给我们惹出两个很实在的麻烦。

1. **同一段 GPU 代码，在这一代芯片上飞快，换一代可能就拖泥带水了。**为什么？因为数据进出 Tensor Core 走的”路线”换了。
2. **数据摆放 (布局) 要是摆错了，程序干脆给你算出个错答案。**注意，哪怕你写的逻辑明明白白还是 $D = A \cdot B + C$ ，结果照样能错。

第 2 点听着很反直觉：逻辑对的，怎么会算错？说白了，**Tensor Core 根本不认什么”抽象矩阵”，它只认”按某个固定格式摆好的一堆字节”**。在它眼里，矩阵的第 0 个元素天经地义就该躺在某个 lane 的某个寄存器格里，第 1 个该在另一个位置，后面以此类推。你的数据只要没按它想要的样子摆，它就照样去乘——只不过把”压根不该相乘的两个元素”给乘到一块儿去了，结果自然是错的。打个比方：就像你给一台只认固定格式的读卡机塞了一张字段顺序错乱的卡，它不会报错，只会把”姓名”当成”金额”读走。

所以这一章，我们就顺着这根「**布局契约 / layout contract**」(也就是”硬件和你之间约定好的数据摆放规矩”)的线，把三代硬件挨个走一遍。描述这套规矩要用到第 3 章的布局记号;Blackwell 的 TMEM 细节比较多，单独放在《Special Memory: TMEM》一章里讲。

一、两条从未离场的约束

一句话先理解

一句话先理解: GPU 跑得快不快, 很多时候不取决于”算得快不快”, 而取决于”取数据快不快”。这一节讲两条最基础的取数据规矩——它们和 Tensor Core 没半点关系, 但任何 GPU 代码都躲不掉, 理解它们是看懂后面一切的地基。

其实早在 Tensor Core 出现之前, 就有两条最朴素的内存约束在悄悄左右着 GPU 代码的数据摆放。它们不挑对象, 对**任何** GPU 程序都管用, 哪怕这段程序跟 Tensor Core 八竿子打不着。我们先把这两条掰扯清楚, 后面 Tensor Core 那套故事才好接得上。

1. 全局内存合并访问 / global memory coalescing

先补一个概念: **全局内存 (GMEM)**, 就是显卡上那块最大、但也最慢的内存 (平时说”这张卡有 24GB 显存”, 说的就是它)。GPU 算东西时, 数据一开始都躺在这里。

再建立个直觉。前面说过, 一个 warp 是 32 个线程一组、必须同步行动。当这 32 个线程同时伸手去 GMEM 取数据时, 内存系统最乐意看到的, 是这 32 个要取的地址**挨得近近的**, 凑在少数几个连续、对齐的内存段里。为什么? 因为内存一次搬运是”成段搬”的, 不是”一个字节一个字节搬”的。打个比方: 你让 32 个人去仓库取货, 如果他们要的货都堆在同一个货架上, 保管员一趟就搬回来了; 要是散落在 32 个不同角落, 那就得跑 32 趟。

- 地址又连续又对齐 → 一次、顶多几次搬运就全取回来了。这种理想情况就叫**合并访问 (coalescing)**。
- 地址东一个西一个 → 同样这点数据, 硬要拆成一大堆搬运才取得完, 带宽和时间双双浪费。

2. 共享内存 bank 冲突 / shared memory bank conflict

再补一个概念: **共享内存 (SMEM)**, 是 GPU 上一块小而极快的片上内存, 由”同一组线程”共用。你可以把它当成一块**需要程序员自己手动管理的高速缓存**: 数据先从慢吞吞的 GMEM 搬进 SMEM, 之后反复用的时候就从 SMEM 快速取, 省得每次都去远处的 GMEM 跑一趟。

但 SMEM 也有自己的脾气。它在物理上被切成 **32 个 bank**(可以理解为 32 个能”同时独立服务”的小窗口)。麻烦就出在这儿: 一个 warp 里好几个线程要读的地址**各不相同, 却偏偏都落在同一个 bank 上**, 这时这个 bank 一次只能服务一个, 硬件只好让它们**排着队一个一个来**——这就叫**bank 冲突**, 本来能并行的事被迫串行, 速度直接打折。

注意

注意一个看着特别老实、人畜无害的”普通二维数组”, 也可能因为它在 bank 上的落点不巧而慢得离谱。慢, 慢的不是数据本身, 而是”地址落到哪个 bank”这套映射关系没安排好。

解药:swizzle / 地址混洗

那 bank 冲突怎么救? 常用的招数叫 **swizzle / 地址混洗**。名字唬人, 干的事其实很朴素: **故意把数据在内存里的物理摆放打乱一下**, 让原本会挤在同一个 bank 的那些访问, 被摊开到不同 bank 上去。

关键是,它**不动你的逻辑数据**——你代码里”第3行第5列”还是那个元素,值一个都没变;swizzle只是在”逻辑坐标 → 物理地址”这一步偷偷加了一道重新排列,让访问横着摊到好几个 bank 上,别再一窝蜂全挤一个 bank。

结构图

连接关系:

- 物理地址 / (打散到 32 个 bank) → 读写都能 / 避开 bank 冲突

图:swizzle 只改物理摆放,不改逻辑坐标。

Tensor Core 额外加的第三条约束

用 Tensor Core 的代码,在上面两条之上,还要**再背一条**:数据(操作数)得摆成 **Tensor Core 指令自己点名要的那个格式**。本章接下来的主线,就是看这”第三条约束”怎么从 Ampere 一路演变到 Hopper、再到 Blackwell。

关键

关键一句话先记住:前两条(合并访问、bank 冲突)三代都跑不掉;真正在变的是第三条,以及”摆数据这件脏活到底归谁干”——是程序员自己一行行算出来,还是直接甩给硬件去管。

二、Ampere: 把数据拆碎摊在 warp 的各个 lane 上

一句话先理解

一句话先理解:在 Ampere 这代,Tensor Core 只认寄存器里的数据。所以一切麻烦都来自一件事——你得先想办法把矩阵从内存里”拆碎、摊到 32 个线程各自的寄存器里”,摆成它指定的样子,它才肯算。

Ampere 这代 GPU 上,挑大梁的 Tensor Core 指令叫 `mma.sync.aligned.m16n8k*` 这一家子(`mma` 就是矩阵乘累加那条指令;后面那串字母数字是尺寸规格,先不用管)。它是 **warp 级**的——意思是一整个 warp 的 32 个线程**一起**协作执行这一条指令,不是每个线程各干各的。想搞懂这一代,你只要盯死一个问题: **这条指令到底从哪儿读数据、又往哪儿写结果? 答案就俩字——寄存器。**

A、B 两个输入,连同 C/D 累加器(结果),统统是**摊在 warp 那 32 个 lane 各自寄存器里的”寄存器片段”**。在这里,共享内存说白了只是个中转站。换句话说,矩阵乘还没开跑,数据 tile 就得先从 SMEM 搬进寄存器,而且得严丝合缝地摆成指令要的那个片段格式。

Ampere 的数据通路 (数据从哪到哪)

步骤	数据走向	说明
1	共享内存 → <code>ldmatrix</code> → 寄存器	用特殊指令 <code>ldmatrix</code> 把 tile 搬进寄存器片段
2	寄存器 → <code>mma.sync</code> → 寄存器	在寄存器之间做矩阵乘,结果还落在寄存器
3	寄存器 → 普通 <code>store</code> → 共享内存	把结果写回去

结构图

连接关系:

- 共享内存 SMEM $\xrightarrow{\text{ldmatrix / (warp 协作)}}$ 寄存器片段 / (32 lane 上的 fragment)
- 寄存器片段 / (32 lane 上的 fragment) $\xrightarrow{\text{mma.sync}}$ 累加器寄存器片段
- 累加器寄存器片段 $\xrightarrow{\text{普通 store / (无专用反向指令)}}$ SMEM / GMEM

图: Ampere 的 Tensor Core 通路, 输入靠 *ldmatrix*, 输出靠普通 *store*。

Ampere 的数据摆放这套故事, 大半都能从这条通路里顺出来: 程序得先把 *tile* 用一种”方便高效加载”的姿势写进 SMEM, 然后靠 *ldmatrix* 把它捏成 *mma.sync* 想吃的那种寄存器片段。

2.1 Ampere Tensor Core 到底期望什么

一句话先理解

一句话先理解: Tensor Core 对”哪个 lane 拿哪几个数”有一套死规矩。这一小节就是把这套规矩摊开给你看。你不用记公式, 只要体会到”它的规矩有多死板、多具体”就够了——正是这种死板, 才让”摆错 = 算错”。

Ampere Tensor Core 读的那种寄存器片段, 是拿 **8×8 的小方块**当最小积木一块块拼出来的 (也就是把数据按 8 行 8 列一组来组织)。这些 8×8 的小单元, 就是 *ldmatrix* 加载、MMA 吃进去的基本单位。

我们拿个具体例子上手: *mma.m16n8k16*(这个名字的意思是: 这条指令一次算的小矩阵, M 方向 16、N 方向 8、K 方向 16)。输入数据类型是 *fp16/bf16*(两种 16 位的浮点格式, AI 里常用), 累加用 *fp32*(32 位浮点, 精度更高)。它算出来的累加器 *tile* 是 **16 行 × 8 列**, 会按一个雷打不动的固定模式推到 32 个 lane 上。别急, 这个模式我们一步步拆。

先看 C/D 累加器, 问一个具体问题: 第 1 号 lane, 手里到底攥着原矩阵的哪几个元素? 它攥着的行编号是:

```
1 / 4
1 / 4 + 8
```

它攥着的列是:

```
2 * (1 % 4)
2 * (1 % 4) + 1
```

公式你别背 (1 / 4 是整除取商, 1 % 4 是取余数), 记住结论就够了: 每个 lane 手里都攥着 4 个 **fp32 累加值**。这 4 个值, 来自两行——把 16 行劈成上下两个”8 行半区”, 每个半区贡献一行——每行又取相邻的两列。还有个特别好记的规律: 连着的 4 个 lane, 正好把某一行的全部 8 列凑齐。(因为列是 $2 * (1 \% 4)$ 和它 $+ 1, 1 \% 4$ 取 0/1/2/3 刚好覆盖 0~7 列。)

下面这张表, 把 16×8 累加器到底怎么分给各个 lane 摆了出来 (每格写的是它归哪个 lane)。就拿 lane 0-3 这一组说: 它们的 $1/4 = 0$, 所以管的是第 0 行 (再加上第二个寄存器对应的第 8 行):

行	列 0, 列 1	列 2, 列 3	列 4, 列 5	列 6, 列 7	备注
行 0	L0	L1	L2	L3	行 = 1/4, lane 0-3 占第 0 行
行 1	L4	L5	L6	L7	lane 4-7 占第 1 行
...	...	...	...	...	每 4 个 lane 覆盖一行
行 7	L28	L29	L30	L31	lane 28-31 占第 7 行
行 8	L0	L1	L2	L3	行 = 1/4 + 8, 同一 lane 的第二个寄存器
...	...	...	...	...	
行 15	L28	L29	L30	L31	

(每格里的那个 lane, 同时握着这一格对应的两列; 比如行 0 的”列 0, 列 1” 格里写着 L0, 意思就是 lane 0 握着第 0 行的第 0、1 两列。)

说明: lane 0 同时持有 (行 0, 列 0)、(行 0, 列 1)、(行 8, 列 0)、(行 8, 列 1) 四个值; lane 1 持有列 2/3; 以此类推。每个 lane 拿 4 个 fp32 值, 连续 4 个 lane 刚好拼出某行的 8 列; 32 个 lane 覆盖上半区 (行 0-7) 与下半区 (行 8-15)。

补一句矩阵乘的基本记号: 做 $A \cdot B$ 时, A 是 $M \times K$ 、 B 是 $K \times N$ 、结果是 $M \times N$ 。**K 是被”乘加掉”的那个公共维度**。知道这个, 下面就好懂了。A 和 B 的摆法跟累加器大同小异, 只是各有各的侧重:

- **A 操作数:** M 这一侧 (行) 怎么切, 跟上面累加器一模一样; K 这一维则沿着 1 % 4 和每个 lane 手里那几个寄存器铺开。对 fp16/bf16 这种 16 位数据来说, 一个 **32 位寄存器正好塞得下 2 个 K 方向的值**。
- **B 操作数:** K 的摆法必须和 A 对上 (否则乘加时配不成对), 然后把 N 这一侧沿着 lane 分组和寄存器铺开。

关键

关键具体数字会随指令尺寸、数据类型变来变去, 但有条原则万变不离其宗:**Tensor Core 要的就是一个特定的”每个 lane 拿哪几个值”的摆法**。值只要没待在对的 lane、对的寄存器里, 指令就会把不该乘的两个元素乘到一块儿——逻辑没错, 答案却错。

如果用第 3 章那套布局记号把这个 8×8 片段精确写出来, 长这样 (看不懂符号没关系, 下面翻译):

```
S[(8, 4, 2) : (4@laneid, 1@laneid, 1@m)]
```

这堆符号别被它唬住, 你只要抓住:**@laneid** 表示”这一维是按 lane 编号铺开的”, **@m** 表示”这一维是在单个 lane 内部、按寄存器槽铺开的”。于是:

- 两个 **@laneid** 项凑一块儿, 讲的是行、列的碎片是怎么撒到 **32 个 lane** 上的;
- 最后那个 **@m** 项, 讲的是在**某一个 lane** 内部, 值该塞进它的哪个寄存器格。

2.2 ldmatrix: 从 SMEM 到寄存器片段的桥

一句话先理解

一句话先理解: 上一节我们看到, Tensor Core 要的那个”哪个 lane 拿哪几个数”的摆法非常刁钻。ldmatrix 就是 NVIDIA 专门造出来、一条指令就帮你把数据按那个刁钻摆法摆好的工具。

ldmatrix 要解决的事儿很明确: 怎么把数据从共享内存里取出来, 正好变成 Tensor Core 想要的那个寄存器片段? 它就是架在这两者中间的那座桥。它是一条 warp 协作式加载——一整个 warp 的 32 个 lane 一起上, 一条指令就能把一个、甚至好几个 8×8 的 16 位矩阵, 从 SMEM 搬进 mma.sync 想要的那种”散在各 lane 上”的寄存器布局。

指令写出来长这样 (// 后面是注释)。你不用认识每个字段, 看注释知道它在干嘛即可:

```
ldmatrix.sync.aligned.m8n8.x1.shared.b16    //      1    8x8
ldmatrix.sync.aligned.m8n8.x2.shared.b16    //      2
ldmatrix.sync.aligned.m8n8.x4.shared.b16    //      4
```

(m8n8 说明每块是 8×8;x1/x2/x4 是一次搬几块;b16 指元素是 16 位;.shared 指数据源在共享内存。)

这里头有个关键约定: 每一行数据在内存里的起始地址, 是由某个特定 lane 负责”报”出来的。具体说, 第 m 个矩阵的第 r 行, 它的起始地址由 lane m*8 + r 提供。这么一算就是:

形态	加载矩阵数	用到的 lane(提供行地址)
.x1	1	lane 0-7
.x2	2	lane 0-15
.x4	4	lane 0-31

加载完, 结果直接就摆成了 MMA 想要的片段。在最基础的 8×8 情形下, lane 1 拿到手的, 正好就是 Tensor Core 指望它持有的那一对行/列——一步到位, 不用你再手工挪。

为什么非它不可反过来想一下: 你要是不用 ldmatrix, 改用一堆普通的、每个线程各读各的内存读取指令 (ld.shared) 去凑, 那就得自己一字不差地把上一节那套刁钻的散射摆法手工算出来、拼出来——又烦又极易出错。ldmatrix 的价值, 就是把”从共享内存搬到片段”这套重排, 压缩成一条 warp 协作指令一把搞定。

那 .trans 是干啥用的? 它会在加载的同时顺手把每个 8×8 矩阵转置一下 (行列对调)。什么时候用得上? 就是当数据在 SMEM 里存的方向, 跟 MMA 指令想要的方向正好拧着的时候, 靠它来掰正, 省得你单独再做一次转置。

结构图

分组:

- SMEM: 「8x8 16-bit tile / (行基址由 lane 提供)」

连接关系:

- 8x8 16-bit tile / (行基址由 lane 提供) $\xrightarrow{\text{ldmatrix / (.trans 可转置)}}$ warp 寄存器片段 / (Tensor Core 期望布局)

图 (等价于原书 *ldstmatrix.svg*): *ldmatrix* 把 8×8 SMEM tile 装进 warp 寄存器片段; Ampere 上反方向只能用普通 *store*, 专用的 *stmatrix* 要到 Hopper 才出现。

2.3 把 Ampere 片段写回去

`mma.sync` 算完之后, 结果 (累加器) 还是一个寄存器片段——也就是说, 它依然是“散在 32 个 lane 上的一堆碎值”, 并不是一块整整齐齐的矩阵。收尾阶段 (英文 *epilogue*, 指矩阵乘完后做缩放、写回等扫尾工作的那一段代码) 要干的, 就是把这堆碎值收拢起来、写回内存。

注意

注意这儿有个坑: Ampere 上压根没有一条“和 *ldmatrix* 反着来”的专用指令 (也就是没有专门把片段一把写回 SMEM 的指令)。所以程序只能拿普通的、每个线程各写各的存储指令凑合; 有时还得动用 *warp shuffle* (让同一 warp 的线程之间直接交换寄存器值的一种操作) 或局部重排, 先把散乱的片段理顺, 再写进 SMEM/GMEM, 凑成一个有用的布局。

这套模型, 好处是够简单, 代价是把一大摊子摆数据的脏活全甩给了程序员: 输入侧用 *ldmatrix* 造片段, 中间计算读写寄存器片段, 输出侧又得靠普通 *store* 把片段一点点收拾出去。记住这个“脏活归谁干”的问题——后面 Hopper、Blackwell 改的主要就是它。

2.4 Ampere 上的 swizzle

一句话先理解

一句话先理解: 同一块数据, 你“写进去时”和“读出来时”走的访问花样不一样, 这两种花样会在 bank 上打架, 导致冲突。swizzle 就是用来化解这场打架的。

Ampere 代码天生就甩不掉共享内存 swizzle。为什么? 一句话: 同一块 SMEM tile, 往里写时是一种访问花样, 往外读时又是另一种花样。这两种花样一打架, bank 冲突 (前面讲过: 访问挤进同一个 bank、被迫排队) 就找上门了。

来看个典型场景。先补一个词: 行主序 (row-major), 指矩阵在内存里“一行接一行地连续存放”(C/Python 数组默认就是这样)。我们从全局内存按行把数据灌进 tile, 行主序让这个写操作既能合并、又对 bank 友好, 看着挺美。可坏就坏在读这一步: *ldmatrix* 读这块 tile 时, 走的往往是沿着列方向, 或者横跨好几个 8×8 小块。在朴素行主序下, 这些读会一股脑全撞进同一个 bank。

给你算笔账就懂了——一块 8 行 64 列的 float16 (16 位 = 2 字节) tile, 它一行有多大:

$$64 * 2 = 128$$

128 字节, 恰好就是共享内存全部 32 个 bank 绕一圈的宽度 (32 个 bank $\times$ 每个 4 字节 = 128 字节)。坏就坏在这儿: 沿着某个固定列往下走, 每跨一行地址就往前蹦 128 字节, 而 128 字节正好是“绕一整圈”, 于是落到的 bank 编号转一圈又回到原点。结果呢, 这 8 行全塌进同一个 bank, 8 个访问被迫排队, 变成 8 路冲突 (速度大约慢 8 倍)。

关键

关键你大概会想: “那我把数据改成列主序 (一列一列连续存) 不就完了?”——没那么便宜。改列主序通常只是把冲突挪了个地方: 列式读是好了, 可行式写又变差了。说白了就是按下葫芦浮

起瓢。

真正管用的解法是 **XOR swizzle**(用异或运算来打乱位置)。先说一句异或**XOR** 是位运算, 记号常写成 $\sim$, 特点是”用它打乱再用同一个数打乱一次就能还原”, 所以特别适合做这种可逆的地址重排。它的思路是: 让物理列的位置, 跟着行号一起变动。最简单的一版长这样:

```
physical_col = logical_col XOR row
```

怎么理解这一行? 对第 **row** 行的第 **logical_col** 列, 我们不把它放在物理的第 **logical_col** 列, 而是放到 **logical_col XOR row** 这一列。于是不同行里”同一逻辑列”的元素, 被推到了不同的物理列、也就是不同的 **bank** 上, 排队问题就化解了。妙就妙在: 逻辑 **tile** 半个字节没动 (你代码看到的还是原来那个矩阵), 光把物理摆放重排了一下, 就让**行式写**和 **Tensor Core** 的**列式读**俩一块儿躲开了 **bank** 冲突。下面这张表, 把三种方案摆一块儿对比一下:

方案	行式写 (从 GMEM 填充)	列式读 (给 ldmatrix)
朴素行主序	合并、 bank 友好	塌到同一 bank , 8 路冲突
朴素列主序	写变差	读变好
XOR swizzle	仍合并、 bank 友好	散到多个 bank

沿同一列 **col_c** 往下读时, 两种方案落到的 **bank** 对比如下:

访问	朴素行主序 (每行 +128B, bank 索引循环重复)	XOR swizzle (<code>physical_col = logical_col XOR row</code>)
<code>row0 col_c</code>	bank k	bank (<code>k XOR 0</code>)
<code>row1 col_c</code>	bank k (冲突!)	bank (<code>k XOR 1</code>)
...	...	...
<code>row7 col_c</code>	bank k (8 路冲突)	散开, 无冲突

图 (等价于原书 *swizzle_conflict.svg*): 朴素行主序下行写横铺、列读撞 **bank**; **XOR swizzle** 在保住合并行写的同时, 把列读打散到各 **bank**。

承上启下这里记住一点: 在 Ampere 上, 这套 **swizzle** 基本上全靠你自己手写共享内存的地址计算 (像上面那行 **XOR** 一样, 一处一处算清楚) 来实现。到了往后的世代, 它会摇身一变, 变成硬件直接能读懂的”说明书”(描述符) 格式里的一个选项, 不用你再手算。这正是接下来 Hopper 最值得看的地方。

三、Hopper: 让硬件照”说明书”自己去读数据

一句话先理解

一句话先理解: Ampere 要你先吭哧吭哧把数据搬进寄存器, **Tensor Core** 才肯算。Hopper 进步了一档: 你只要写一份”说明书”告诉硬件”数据在共享内存里是怎么摆的”, 硬件就能自己照着去读, 省掉了那一大步手工搬运。

Hopper 这代, 动刀子动的是 **Tensor Core** 通路的**输入侧** (读数据那一头)。它不再死磕”每个数据都得先用 **ldmatrix** 塞进寄存器”, 而是让新指令 **wgmma** 直接从共享内存里读数据。

- **B 操作数**: 从一份 **SMEM 矩阵描述符** (描述符就是下面要讲的那份”说明书”)。
- **A 操作数**: 既能从 SMEM 描述符读, 也能从寄存器读——这对应指令的 **.ss** 和 **.rs** 两种写法 (**s** = 来自共享内存 **shared**, **r** = 来自寄存器 **register**; **.ss** 是 A、B 都从共享内存读, **.rs** 是 A 从寄存器、B 从共享内存)。

注意

注意可别误会, 这只是把”从共享内存来的数据还要先 **ldmatrix** 一道”那个显式步骤给省了, 对数据怎么摆的要求一条没少。Tensor Core 照旧要求数据按精确的格式摆在共享内存里。真正变了的只是”怎么告诉硬件这个格式”: 以前埋在你手写的一堆地址计算里, 现在改成写一份矩阵描述符, 直接说给硬件听。

3.1 Hopper Tensor Core 期望什么

先讲讲这个描述符到底是个啥。Hopper 的 **SMEM 矩阵描述符**, 你就把它当成一块共享内存里矩阵 **tile** 的一份小巧”说明书”——本质上是一个打包好的小数据结构, 里面填着几个字段。它的活儿, 就是告诉 **wgmma**: **逻辑上的”第几行第几列”, 对应到共享内存里实际的哪个字节地址**。说明书里写着这么几个字段:

字段	作用
start address(起始地址)	tile 在 SMEM 的基址
leading dimension offset(主维偏移)	沿主维前进的步长
stride dimension offset(步维偏移)	沿另一维前进的步长
swizzle mode(swizzle 模式)	选定 atom 形状 + atom 内的 XOR 排列
base offset(基偏移)	附加偏移

关键

关键同样这几个字段, 具体怎么解读, 得看数据的 **major mode / 主序**——也就是”它在内存里是按哪个维度连续存放的”。对 **K 主序**的 **tile**(沿 **K** 方向连续), 一个步长是沿 **K** 走、另一个沿 **M** 走; 换成 **MN 主序**的 **tile**, 这俩角色正好对调。你只要记住: 同一份描述符字段, 在不同主序下”指向哪个方向”会变, 所以填描述符时必须和数据实际的存放方式对上。

上面那个 **swizzle 模式**字段, 取值就是从下面这几种预设格式里挑一个 (**NONE** 是不打乱, **32B/64B/128B** 是不同力度的打乱, 数字越大、一次打乱涉及的字节范围越大):

SWIZZLE_NONE
SWIZZLE_32B
SWIZZLE_64B
SWIZZLE_128B

swizzle 模式一选, 就**两件事**当场拍板:

1. 描述符用多大的 **atom 形状 (atom shape)**——**atom** 就是”做 swizzle 的最小单元方块”, 你可以理解成”以多大一格为单位来打乱”;
2. 在每个 **atom** 内部, 到底施加哪一种 **XOR 排列** (还是上一节那个异或打乱的思路, 只是这里由硬件按预设格式来做)。

举个例子:128 字节 **swizzle 模式**会把数据看成一张网格, 每格是一个”8 行 × 128 字节” 的 atom, 打乱就在每个 atom 内部各管各的进行。

结构图

连接关系:

- 选中某个 atom / (如 8 行 × 128B) $\xrightarrow{\text{swizzle mode / 选 atom 内字节位置}}$ 最终 SMEM 字节地址

图 (等价于原书 *smem_descriptor.svg*): 描述符的 *stride* 选 *atom*, *swizzle* 选 *atom* 内的字节位置。

3.2 数据是谁写进去的?TMA 与 wgmma 必须” 对暗号”

描述符这份说明书再聪明, 共享内存里的字节也总得先有人按规矩码进去。这活儿一般交给 **TMA / Tensor Memory Accelerator**(Hopper 新增的一个硬件搬运引擎, 专门成块地、高效地把数据从全局内存 GMEM 搬进共享内存 SMEM——你可以把它想成一台” 批量搬货的传送带”)。由它来把这块 SMEM tile 填上。

这里有条要命的规矩:**TMA 往里写时用的 swizzle 格式, 必须跟后面 wgmma 描述符读时用的 一模一样**。这就跟对暗号一个道理——写的人和读的人得用同一套打乱规则, 数据才对得上:

- TMA 写进去的要是 128 字节的 swizzled tile, 那 **wgmma** 描述符也得按 128 字节的 swizzled tile 来读。
- 一旦**描述符和数据对不上**, Tensor Core 读到的就是一堆被打乱的操作数。

结构图

连接关系:

- SMEM tile / (128B swizzled) $\xrightarrow{\text{wgmma 描述符 (读) / 必须也 =128B}}$ Tensor Core

图:*TMA* 写入与 *wgmma* 读取必须命名同一种 *swizzle* 格式, 否则读到乱码。

这正是 **Hopper 比 Ampere 最关键的那一步转变**。swizzle 不再窝在某段手写的 SMEM 索引计算里了。Hopper 把它提拔成了一等公民——一种有名有姓的描述符格式: 写 tile 的 TMA load 和读 tile 的 wgmma, 引用的是同一份格式, 自然也就对得上暗号。

3.3 Hopper 的输出仍用寄存器

Hopper 虽然把” 读数据” 这一头翻新了, 可有一样东西没动: **算出来的结果 (累加器) 还是住在寄存器里**。

wgmma 把累加器写进每个线程的寄存器片段。片段多大、占几个寄存器, 看指令尺寸——比如 m64nNk16 里那个 N(结果矩阵的列数), N 越大, 累加器吃掉的寄存器就越多。但骨子里跟 Ampere 没两样: 收尾阶段拿到手的, 还是一个散在各 lane 上的寄存器片段。

所以 Hopper 是个**混血儿**——读数据是新办法, 写结果是老办法:

阶段	Ampere	Hopper	变化
----	--------	--------	----

输入操作数	寄存器片段 (经 ldmatrix)	SMEM 描述符直读 (swizzle 由硬件描述)	改了
累加器/输出	寄存器片段	寄存器片段	没变

换句话说,到了 Hopper 这代,处理输出累加器仍然是一桩”摆寄存器”的活儿。这个输出侧的故事,得等到 Blackwell 才会被改写。

四、Blackwell: 把结果搬进新内存 TMEM

一句话先理解

一句话先理解:前两代,算出来的结果一直堆在寄存器里,可寄存器又少又金贵,结果一大就吃紧。Blackwell 新挖了一块专门的内存 TMEM 来放结果,把寄存器解放出来——但也因此多了几道”管理 TMEM”的手续。

读数据这一边,Blackwell 基本照搬了 Hopper 那套 SMEM 描述符的路子:A、B 还是在共享内存里按 Tensor Core 想要的格式摆好 (指令家族换了个名字,叫 tcgen05);某些模式下,A 甚至能直接从 TMEM 读。这里先把 TMEM 交代清楚:TMEM(Tensor Memory) 是 Blackwell 新增的一块片上内存,专门给 Tensor Core 暂存计算结果 (累加器) 用——你就理解成”一块新开辟的、离 Tensor Core 很近的小内存”。

真正翻天覆地的变化,落在结果 (累加器) 身上。搁以前,累加器是个”成天泡在寄存器里”的片段;Blackwell 的 tcgen05.mma 指令干脆把它改成写进 TMEM:

- 计算阶段:累加器一直待在 TMEM 里,不占寄存器。
- 收尾阶段:再用 tcgen05.ld 指令把它从 TMEM 装回寄存器,好让收尾代码处理。

结构图

连接关系:

- TMEM / (累加器驻留于此) $\xrightarrow{\text{tcgen05.ld}}$ 寄存器片段 / (交给 epilogue)

图:Blackwell 把累加器的”家”从寄存器搬到了 TMEM。

这么一搬,等于把”结果怎么摆”这个难题,从寄存器挪到了 TMEM。于是程序现在多了几桩手续要办:

1. 申请一块 TMEM(像申请内存一样,先要到地方);
2. 挑对 TMEM 里的摆法;
3. 等 MMA 算完 (因为结果是异步写进 TMEM 的,得确认写完了才能取);
4. 走对应的 tcgen05.ld 路径,把结果片段取回寄存器,交给收尾阶段。

注意

注意至于 cta_group::1 和 cta_group::2 这两种模式怎么把累加器拆到一个或两个 CTA 上 (CTA = cooperative thread array, 其实就是”一个线程 block”,一批一起协作的线程),那是《Tensor Cores: tcgen05》那章的事,这里不展开。本章里跟前几代差得最远的,其实是下面要讲的 block-scaled 缩放因子布局——接着看。

4.1 TMEM 里的缩放因子布局 (scale factor layout)

先说说这个缩放因子是干嘛、从哪冒出来的。为了省内存、跑得更快,新硬件支持用**特别低精度**的格式存数据,比如 **mxfp8**(一种 8 位)、**nvfp4**(一种 4 位)。位数这么少,直接存数值会严重不准,所以业界的做法是:把数据分成一小块一小块,每一小块配一个”缩放因子”(相当于给这一块单独定一个倍率),用的时候乘回去还原。这种模式叫 **block-scaled MMA**(分块缩放的矩阵乘)。

代价就是:除了原来的 A、B,MMA 现在还得**多读两块缩放因子数据**:

```
SFA(M, SFK) // A           ,SF = Scale Factor
SFB(N, SFK) // B
```

这里的 **SFK**,说的是沿 K 方向**一共切了多少个缩放块** (每块一个因子,所以有多少块就有多少个因子)。

关键

关键一句话先记住:数据 A、B 住在 **SMEM**,而缩放因子要住在 **TMEM**。住的地方不一样,搬进来的路线自然也不一样。

那为什么缩放因子搬运得分两步?因为前面说过的那台”传送带”**TMA** 只会从 **GMEM** 搬到 **SMEM**,不会直接往 **TMEM** 里写。所以缩放因子只能绕个弯,分两步走:

步骤	数据通路	说明
第一步	GMEM -TMA→ SMEM	传送带 TMA 先从 GMEM 搬到 SMEM
第二步	SMEM -tcgen05.cp→ TMEM	再用拷贝指令 tcgen05.cp 从 SMEM 搬进 TMEM(cp = copy)

只有等这第二步拷贝干完,缩放因子才算真正到位——也就是落进了 **tcgen05.mma** 指望去读它们的那块内存。

结构图

连接关系:

- SMEM $\xrightarrow{\text{tcgen05.cp}}$ TMEM
- TMEM $\xrightarrow{\text{读取}}$ tcgen05.mma

图:缩放因子的两步搬运路径 (数据 A/B 是 GMEM→SMEM 一步直达,缩放因子要多绕一步进 TMEM)。

4.2 缩放因子的 TMEM 坐标:warp_x4 广播

一句话先理解

一句话先理解:同一份缩放因子,后面会有”分布在不同 lane 上的很多线程”都要用到。与其让它们大老远去同一处抢着读,不如**预先复制几份**摆开,让大家就近取。这就是这一节”复制四份”的动机。

缩放因子在 TMEM 里怎么摆,用的是 TMEM 自己那套硬件坐标 **Lane** 和 **Col**(在布局记号里写成 **TLane** 和 **TCol**,你就理解成 TMEM 里的”行号”和”列号”)。

它的核心套路一句话就能说清: **先把一份缩放数据压成一小份,再复制四份铺满整个空间**。说细点: 一个 **128 行**的缩放向量先被压实进一个 **32 行**的组 (从 128 压到 32), 然后这一组**原封不动复制到 TMEM 的四个”32 行窗口”** 里。写成布局记号是这样 (看不懂照旧不影响, 下面翻译):

```
S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
```

你只需对着这两部分看:

- **分片项 (shard, 前半截):** 负责摆好那一份基础的 32 行组——第 **r** 行就放在 **TLane = r**;
- **复制项 (replica, 后半截):** 负责把这份组**复制 4 份**, 分别放到 lane 偏移 0、32、64、96 这四个起点上——也就是 **TLane = r + 32*q**,q 取 0、1、2、3 各一份; 列 (**TCol**) 不动。

这套就叫 **warp $\times$ 4 广播模式** (warp $\times$ 4, 即复制成 4 份覆盖): 同一份压实的缩放组被复制到四处, 这样整个 128 行的 TMEM 空间里, 不管哪个线程在哪个 lane, 都能就近看到它要的缩放因子。

同一个 32 行缩放组被广播 4 份, 铺满 128-lane 的 TMEM 空间:

TMEM 窗口	TLane	0 ...31	32 ...63	64 ...95	96 ...127
内容		基组	副本	副本	副本
副本编号		q=0	q=1	q=2	q=3

图:warp $\times$ 4 把压实的 32-lane 缩放组, 复制到四个 32-lane 窗口, 覆盖 128-lane TMEM 空间。

4.3 TCol 单元内部的字节打包 (scale_vec 模式)

一句话先理解

一句话先理解: 上一节决定了”缩放因子放在哪些 lane”, 这一节再放大一级, 看最小的一个 4 字节格子里到底塞了几个缩放值。这是布局的最后一层细节。

广播管的是”放到哪些 lane”, 可每个 32 位 (= 4 字节) 的 TCol 格子里头, 还有一层更细的讲究——这 4 个字节里到底装了几个缩放值。这就看 **scale_vec 模式** (scale vector, 缩放向量打包方式) 怎么定了:

模式	4 字节单元内的打包方式
1X	一个缩放值, 广播填满整个 32-bit 单元
2X	打包两个缩放值, 每个各复制一份
4X	打包四个 K-block 缩放值

4 字节 (32-bit)TCol 单元内部, 三种模式的字节摆放:

模式	字节 0	字节 1	字节 2	字节 3	含义
1X	S0	S0	S0	S0	一个值, 广播
2X	S0	S0	S1	S1	两个值, 各复制
4X	S0	S1	S2	S3	四个 K-block 值

图 (等价于原书 sf_scale_vec.svg):scale_vec 字节打包的三种模式。

注意

注意这种字节打包, 在 Ampere 和 Hopper 上根本找不到对应物。道理也简单: 那两代压根就没有给 tcgen05 block-scaled MMA 用的那种 TMEM 缩放因子操作数, 既然东西都没有, 自然也谈不上怎么打包。

4.4 cta_group::2 下缩放因子跟着数据走

前面提过,cta_group::2 是”两个 CTA(两批线程) 合伙算一道 MMA”的模式。在这种模式下, 你记住一条原则就够了: 缩放因子永远跟着它要缩放的那块数据走。拆开看就是:

- **SFA** 缩放的是 A。A 既然按 M 维 (行方向) 被切成两半、分给两个 CTA, 那 SFA 也跟着按 M 切两半, 好对上各个 CTA 手里的那些 A 行。
 - **SFB** 缩放的是 B。B 既然被这两个 CTA 共用 (两边都要用到同一份 B), 那 SFB 就多播给这两个 CTA(multicast, 即一份数据同时发给多个目标, 省得各搬一遍)。
- (想看更细的, 翻《Tensor Cores: tcgen05》。)

五、一个反复出现的片段:m8n8

三代一路讲下来, 周围的内存路线换了一茬又一茬, 可有个老面孔始终赖着没走:m8n8 风格的寄存器片段 (就是第 2.1 节那种”以 8×8 为单位、散在各 lane 上”的摆法)。只不过它每一代登场的位置不太一样。

世代	m8n8 片段扮演的角色
Ampere	ldmatrix 构建它, 好让 mma.sync 读取 (输入侧)
Hopper	wgmma 把累加器写成寄存器片段, 交给 epilogue(输出侧)
Blackwell	计算阶段累加器在 TMEM; 但 tcgen05.ld 在 epilogue 处理与存储之前, 把它装回寄存器片段

关键

关键片段从来没消失过, 变的只是它露脸的地方。早些的世代里, 累加器整个计算阶段都泡在片段里; 到了 Blackwell, 它基本只在 TMEM 和收尾阶段交接的那个当口才冒一下头。

六、贯穿三代的主线 (The Throughline)

结构图

分组:

- **Ampere**: 「输入: 显式构建寄存器片段 / (ldmatrix)」 「swizzle: 主要靠 kernel 手写索引数学」 「累加器: 寄存器」
- **Hopper**: 「输入: SMEM 描述符直读 / (wgmma)」 「swizzle: 具名描述符格式 / (TMA 与 wgmma 共享)」 「累加器: 寄存器」
- **Blackwell**: 「输入: SMEM 操作数 (+ 部分 TMEM A)」 「累加器: 搬进 TMEM / (tcgen05.mma / tcgen05.ld)」 「block-scaled: 缩放因子经 TMEM 暂存」

连接关系:

- Ampere → Hopper

把三代横向对比成一张表:

维度	Ampere	Hopper	Blackwell
主力 MMA 指令	<code>mma.sync.aligned.m16n8k*</code>	<code>wgmma(.ss/.rs)</code>	<code>tcgen05.mma</code>
输入操作数来源	寄存器片段 (经 <code>ldmatrix</code>)	SMEM 描述符直读	SMEM 描述符 (+ 部分 TMEM A)
swizzle 谁负责	kernel 手写索引数学	硬件描述符格式 (TMA + <code>wgmma</code> 共享)	硬件描述符格式
累加器位置	寄存器	寄存器	TMEM
输出取回	普通 <code>store</code>	epilogue 读寄存器片段	<code>tcgen05.ld</code> 从 TMEM 取回
缩放因子 (block-scaled)	无	无	经 <code>tcgen05.cp</code> 暂存到 TMEM

关键

核心论断千万别以为有了描述符就高枕无忧了。描述符根本没替你摆数据” 摆数据” 这活儿免掉, 它只是把那份契约写清楚了而已。该你兜的底还得自己兜: 数据搬运路线、内存里的摆法、Tensor Core 指令, 这三样必须**严丝合缝地对齐**。写 `swizzle` 数据的那个 TMA 描述符、读这块数据的那个 MMA 描述符、还有你声明的那个布局——它们说的得是同一种物理摆放, 一个字都不能差。

而且最要命的一点是: 就算有一环没对上, 硬件也**照跑不误**, 一声不吭, 不报错。它就默默地给你读错字节 (算出错误答案), 或者慢吞吞地读 (性能崩掉)。正因为这样, **数据怎么摆, 压根不是 Tensor Core 程序上面可有可无的装饰, 它本身就是硬件指令接口的一部分**。

小结

- Tensor Core 干的事三代都没动 (还是 $D = A \cdot B + C$), 可**数据进出走哪条道、结果最后落在哪块内存**, 每代都在变。而这恰恰决定了你的代码算得对不对、跑得快不快。
- **Ampere(最老)**: 一切围着**寄存器片段**打转 (数据拆碎摊在各 lane 的寄存器里)。`ldmatrix` 造片段、`mma.sync` 算、普通 `store` 写回;`swizzle` 靠你手写地址计算 (典型就是 `__mma_store` = `__mma_store` XOR `__mma_store`) 来躲 bank 冲突。
- **Hopper(中间)**:`wgmma` 改成靠 **SMEM 矩阵描述符** (一份说明书) 直接读数据,`swizzle` 也升级成**有名有姓的预设格式** (写数据的 TMA 和读数据的 `wgmma` 必须对上暗号); 不过结果还窝在寄存器里。
- **Blackwell(最新)**: 读数据接着用 SMEM 描述符, 但**结果搬进了新内存 TMEM**(`tcgen05.mma` 写、`tcgen05.ld` 取回); 分块缩放 (block-scaled) 那类低精度模式多出来的缩放因子, 得经 `tcgen05.cp` 分两步挪进 TMEM, 还用上了 `warpx4` 广播 + `scale_vec` 字节打包这套独门摆法。
- 两条老规矩——**全局内存合并和共享内存 bank 冲突**——从头到尾都在。三代怎么演变, 说到底就是把” **第三条规矩 (Tensor Core 要的数据摆法)**” 从程序员的手工活, 一点点交给硬件描述符契约去管。描述符让契约更清楚了, 可一分钱没替你出: **数据怎么摆, 自始至终都是硬件指令接口的一部分**。

延伸阅读

- 原文 (英文):[Tensor Core Operand Layouts Across GPU Generations](#)
 - 相关章节 (本书): 《数据布局 (Data Layout)》——本章布局记号的来源; 《Special Memory: TMEM》——Blackwell TMEM 细节; 《Tensor Cores: tcgen05》——cta_group::1/2 累加器拆分与缩放因子多播细节。
-

术语对照

中文	English
矩阵乘累加	MMA / matrix-multiply-accumulate
寄存器片段	register fragment
线程束 / 通道	warp / lane
共享内存	SMEM(shared memory)
全局内存合并访问	global memory coalescing
共享内存 bank 冲突	shared memory bank conflict
地址混洗	swizzle
矩阵描述符	matrix descriptor
主序 (K 主序 / MN 主序)	major mode(K-major / MN-major)
张量内存	TMEM(Tensor Memory)
缩放因子 (块缩放)	scale factor(block-scaled)
收尾阶段	epilogue
协作线程阵列	CTA(cooperative thread array)
张量内存加速器	TMA(Tensor Memory Accelerator)
多播	multicast

第 05 章 · 异步数据搬运:TMA

原文:Async Data Movement: TMA

本章要点

本章要点 (TL;DR)

- 这一章只讲一件事:GPU 怎么把数据从”大仓库”搬到”工作台”上,搬得又快又不耽误干活。
- **TMA(Tensor Memory Accelerator / 张量内存加速器)** 就是 GPU 里一个专门搬数据的硬件引擎。你可以把它想成办公室里的”搬运工外包公司”:你只要派「一个工人(一个线程)」喊一嗓子下个单,这家公司就在后台默默把一整箱货(一块矩形数据 tile)从大仓库(全局内存 GMEM)搬到你手边的工作台(共享内存 SMEM)。搬的过程中不挡你的道,你这一屋子人(整个线程块 CTA)该算的继续算。
- 整件事靠一张「**搬运清单**」说了算,这张清单的正式名字叫 **张量映射描述符 (tensor-map descriptor)**:货架长什么样(形状)、一排排怎么隔开(步长)、每件货多重(每个元素几个字节)、一次搬多大一箱(tile多大),还有最关键的 **swizzle(交织)摆放方式**,全写在这张清单里。
- 搬进来(加载 / load)的时候,TMA 顺手就帮你把货摆好了:箱子一落到工作台,里面的货就已经按照下游机器(Tensor Core,后面会讲,是 GPU 里专算矩阵乘的硬件)最顺手的方式排好了,省得你再单独花时间重新码一遍。前提是:搬运清单、工作台上的摆法、下游机器读取的预期,这三家得事先约定好同一套摆放规矩。
- 怎么知道货到了 / 搬走了?搬进来(加载)靠一个叫 **mbarrier** 的”到货通知器”(它还会数够不够字节数);搬出去(存储 / store)靠另一套叫 **commit group** 加 **wait group** 的机制。这俩盯的是两个不同的交接时刻,后面会分开讲。
- **TMA 真正的看家本领,是配合”流水线 (pipelining)”**:让搬运工在后台先把「下一箱货」搬进来,同时让计算单元在前台算「眼下这一箱」,中间用通知器把两边对齐。这样机器永远不闲着。

前置知识

前置知识:这一章会反复用到几个词,你只要先有个大概印象就行,正文每个第一次出现的地方我都会当场再讲一遍:

- **GMEM / SMEM:**GPU 上两层存数据的地方。GMEM(全局内存)是”大仓库”,容量大但离计算单元远、取数据慢;SMEM(共享内存)是”工作台”,容量小但贴着计算单元、取数据飞快。
- **tile:** 把一个大矩阵切成的小方块。矩阵太大一次装不进工作台,只能切成小块一块块搬、一块块算。
- **GEMM:** 通用矩阵乘(就是 A 矩阵乘 B 矩阵)。它是深度学习里最核心、最费算力的运算,

所以是本章的”主角”。

- **mbarrier**: 一个”异步到货通知器”,专门用来回答”后台那批数据到底搬完没有”。

没把握的话,先翻一下第 0 章·极简入门;mbarrier 的细节可以看第 7 章。本章不会假设你有任何 GPU 背景,放心读。

0. 为什么需要 TMA: 从「线程自己搬」说起

想搞懂 TMA,咱们先看看没有它的时候,搬数据这活儿是怎么干的、又有多别扭。先理解”痛”在哪,你才会明白 TMA 为什么是个好东西。

一句话先理解

一句话先理解:GPU 算得飞快,但前提是数据得喂得上。要是计算单元天天等数据,那再快的算力也是浪费。TMA 解决的就是”喂数据”这件事。

先问一句:一个矩阵乘 (GEMM)、或者一个注意力 (attention,Transformer 模型里那个核心运算) 的 **核函数 (kernel, 就是”一段跑在 GPU 上的程序”)**,跑得快不快,到底卡在哪儿?

答案是:卡在 **Tensor Core 有没有数据吃**。这里先解释一下什么是 Tensor Core——它是 GPU 里专门用来算矩阵乘的一块硬件, **唯一的本事就是疯狂算矩阵乘,而且算得极快**。深度学习里矩阵乘占了绝大部分算力,所以 GPU 专门造了这么个”矩阵乘专用机床”。

明白了这个,卡点就好理解了:只要数据源源不断喂上,Tensor Core 这台机床就能一刻不停地算,这种状态我们叫 **计算受限 (compute-bound)**——意思是”算力已经榨干了,瓶颈是算不是别的”,这是我们追求的好状态。可这有个前提: **下一批要算的输入数据 (参与矩阵乘的 A、B 矩阵切出来的小方块,术语叫操作数 operand 的 tile) 得及时端到机床边上**。端晚了,机床就只能停在那儿干瞪眼空转——这就浪费了。

那 **过去没有 TMA 的时候,数据是怎么搬的?** 答案是:让干活的线程自己抽空去搬。具体说,GPU 上成千上万个线程在并行干活;搬数据时,每个线程自己算好”我负责的那块数据在大仓库 GMEM 的哪个地址”,发一条 load 指令把它读出来,再发一条 store 指令把它写到工作台 SMEM 上。

这么干能用是能用,但暗里要付两笔账:

1. **干活的”指令配额”被白白占用了**。GPU 里线程是按 **warp** 为单位调度的——一个 warp 就是 **32 个绑在一起、必须齐步走的线程** (可以想成 32 个人组成的一个小方阵,喊一个口令大家一起迈同一步)。这个小方阵每发一条指令都是有成本的。本该把这些宝贵的指令名额拿去喂 Tensor Core 算数,结果全耗在”算地址、读、写”这些纯搬运的杂活上了。
2. **计算线程的”思路”被搬运打断了**。这批 warp 本来应该一门心思盯着喂 Tensor Core,现在却被迫在计算指令里掺进一大堆搬运指令,节奏全乱了。

关键

关键:这两点合起来,问题就清楚了——”**搬数据**”和”**做计算**”在抢同一拨人手、同一份硬件资源 (具体说是抢”发指令”的名额,以及抢 **寄存器**——寄存器是每个线程私有的一小块超高速存储,相当于线程手边的草稿纸,见第 0 章)。TMA 的招数很干脆: **把搬运这件事整个外包给一个专职的硬件搬运引擎**,这样计算线程就彻底腾出手来,专心算数就行,再也不用兼职当搬

运工。

下面这张图把”自己搬”和”外包搬”两条路摆一块比一比 (图里 SMEM 就是前面说的工作台):

结构图

分组:

- **旧方式: 线程自己搬:**「每个线程 / 算地址 + load + store」「SMEM」「占用 warp 指令 / 占用寄存器 / 污染计算指令流」
- **TMA 方式: 硬件搬:**「一个线程 / 发起一次拷贝指令」「TMA 引擎 / (异步搬整块 tile)」「SMEM」「其余线程继续干活 / 不占计算指令流」

连接关系:

- TMA 引擎 / (异步搬整块 tile) → SMEM

注意

注意: 原文这儿有个交互演示 (*TMA copying a tile from global memory to shared memory*, 即”TMA 把一块 tile 从全局内存搬到共享内存”), 你能切换 swizzle 模式, 把鼠标停在源数据某个格子上, 看它最后落到工作台 SMEM 的哪个位置。静态笔记没法还原这种交互, 想直观感受就去翻原书; 后面我会用文字和 Mermaid 图把要点画给你看。

1. 一个线程发起, 硬件搬运整块 tile

1.1 发起线程做什么、不做什么

一次 TMA 拷贝, 是从一个**发起线程 (issuing thread, 就是”负责下单的那一个线程”)**开头的。这里要特别强调一个反直觉的点: 这个线程**不会**自己一个一个地去遍历、搬运 tile 里的每个元素。它干的活更像是去前台「下单」——递给硬件一份「搬运清单」, 然后转身就走; 真正一箱箱搬货的力气活, 全甩给 TMA 引擎在后台干。

换句话说: 发起线程的工作量, 跟 tile 是大是小几乎没关系。不管要搬 100 个元素还是 10000 个, 它都只是”喊一嗓子下个单”, 一条指令的事。这正是 TMA 省事的根源。

这份「搬运清单」的主体, 就是 **张量映射描述符 (tensor-map descriptor)**——名字听着唬人, 其实就是一张描述”这批数据长什么样、怎么搬”的表。它写清了这么几件事:

描述内容 (英文术语)	大白话含义
tensor shape(张量形状)	大仓库里那个完整矩阵有多大, 比如 4096×4096
strides(步长)	矩阵在内存里一行隔一行该跳多远——内存其实是一长条, 矩阵是被”压扁”塞进去的, 步长就是告诉硬件”换行时地址该加多少”, 这样才能正确寻址
element size(元素大小)	每个数占几个字节 (比如 float16 占 2 字节)
tile shape(tile 形状)	每次搬一小块, 这一小块多大, 比如 64×64
swizzle mode(交织模式)	货搬到工作台后按哪种方式摆放 (这是本章重点, 第 2 节细讲)

一句话先理解

一句话先理解: 描述符里这些信息, 大部分在核函数一开始就定好、**整个过程都不变**——因为大矩阵的形状、每个数多大, 这些是固定的。

光有这张清单还不够。真要发起某一次具体的拷贝, 还得临时补上两个「这一次专属的坐标」:

- 这回到底从大矩阵的**哪个位置**抠出一块 tile(术语叫 tile 坐标 / tile coordinates, 相当于”我要第 3 行第 5 列那一小块”)。
- 抠来的这块 tile, **放到工作台 SMEM 的哪个具体地址上**。

这么对比就清楚了: **描述符回答的是”这个大矩阵长啥样、怎么排的”(固定不变)**, 坐标回答的是**”这一次具体搬哪一块、搬到哪儿”(每次都可能不一样)**。一个是常量, 一个是这次调用的参数。

指令一发出去, 拷贝就**异步 (asynchronous)**地跑起来了。”异步”这个词后面会反复出现, 这里先讲清楚: 它的意思是”我下了单就不管了, 接着干别的, 搬运在后台自己进行”——跟你点了外卖不会站在门口干等、而是继续做事是一个道理。所以发起线程下完单接着往下走它的, **CTA**(一个线程块, 即一组互相协作、能共用同一块工作台 SMEM 的线程, 见第 0 章) 里其他线程也照样干活, 没人停下来等。这会儿搬运的事全归 TMA 引擎管, 不再是一串普普通通的 load/store 指令在那儿循环。

1.2 同一个逻辑操作, 两条实现路径

注意, 有了 TMA 之后,”自己搬”那条老路并没有消失——它俩是**并存的两个选项**。同样是「把这块 tile 拷过去」这件事, 核函数现在有两种写法:

结构图

连接关系:

- 逻辑操作: 「拷贝这块 tile」→ 线程拷贝 (thread copy) / 线程协作 load/store
- 逻辑操作: 「拷贝这块 tile」→ TMA 拷贝 (TMA copy) / 一个线程发起, 硬件搬运
- 线程拷贝 (thread copy) / 线程协作 load/store → 优点: 对每次访问有直接控制 / 缺点: 吃线程指令和寄存器
- TMA 拷贝 (TMA copy) / 一个线程发起, 硬件搬运 → 优点: 适合大而规整的 tile / (Tensor Core 操作数 tile 的天然选择)

这两条路,”怎么确认搬完了”的规矩不一样, 跑出来的性能也不一样。到底走哪条, 这个选择本身有个名字, 叫**分发 (dispatch) 决策**——你可以理解成”派活儿”: 这趟活儿是派给普通线程, 还是派给 TMA。原文把”一次拷贝”这件事拆成三个互不打架的概念, 这么一拆就清楚多了:

- **布局 (layout):** 核函数想要数据在内存里排成啥样 (形状、摆放方式)。
- **作用域 (scope):** 哪些线程、哪些 CTA 来掺和这次拷贝 (谁负责)。
- **分发 (dispatch):** 这次拷贝用普通线程代码来做, 还是丢给 TMA 来做 (派给谁)。

这三个概念彼此独立, 组合起来才描述清楚”一次拷贝到底怎么发生”。

2. Swizzle 布局: 不只是「搬到」, 还要「摆对」

一句话先理解

一句话先理解:swizzle(交织) 就是”换一种方式摆货”, 目的是让计算单元后面去取数据时不会”挤在一起排队”。它只改”东西摆哪”, 不改”东西是什么”。

2.1 为什么搬到位还不够

把 tile 搬进工作台 SMEM, 其实只算干完一半。为啥? 因为 Tensor Core 这台机床不光要求工作台上装的是**对的货**, 还要求这些货**摆成对的样子**。摆得不对, 它去取的时候就慢。

慢在哪儿? 这里得讲一个重要的硬件细节:**SMEM bank 冲突**。工作台 SMEM 在硬件上被切成了好多个并排的小格子, 每个叫一个 **bank(存储体)**。它的妙处是: **只要不同线程取的数据落在不同的 bank 上, 大家就能同时取, 飞快**; 可一旦多个线程同时想取的数据挤在了**同一个 bank 上**, 硬件就只能让它们排队、一个一个来——这就叫 **bank 冲突**, 是 SMEM 变慢的头号原因 (见第 0 章)。打个比方: 超市有 32 个收银台 (bank), 顾客分散到不同台子结账就很快; 要是所有人都挤一个台子, 后面就得排长队。

那怎么避免大家挤一个 bank? 这时候 **TMA swizzle(交织)** 就上场了。TMA 往工作台写这块 tile 的同时, 能顺手**把它在 SMEM 里的摆放位置打乱重排一下** (术语叫 permute, 就是”做一次位置置换”)。注意: **逻辑上看**, 这块 tile 还是规规矩矩一个矩形, 该是第几行第几列的元素还是第几行第几列; 只是它们**物理上**落在 SMEM 的哪些格子, 被交织故意错开了, 好让后面取数据时正好散落在不同 bank 上, 避开排队。

关键

关键: 用哪种 swizzle 方式, 前面那张”搬运清单”(描述符) 里就写好了。所以发起线程**根本不用自己动手做交织**——字节落进工作台那一刻, TMA 引擎已经顺手替你摆好了。这是 TMA 又一个省事的地方: 搬运和”摆对”一步到位。

2.2 「一致约定」是硬性前提

swizzle 是省事, 但带来一条**绝对不能破的铁律: 有三方必须对”货怎么摆”达成完全一致的口径**。哪三方? 看下图。

结构图

这条铁律有多硬、不守会多惨? 你想想这个场景: TMA 拿”A 方式”把数据交织着写进去, 可后面真正去算矩阵乘的指令 (叫 **MMA**, 全称矩阵乘累加 matrix multiply-accumulate, 就是 Tensor Core 执行的那条核心指令, ”一边乘一边把结果累加起来”) 却以为数据是按”B 方式”摆的, 于是按”B 方式”去读。这时候硬件会怎样? **它不会报错, 它就老老实实照你说的”B 方式”去读**——因为硬件没法知道你写的时候用的是”A 方式”。结果呢? 读出来的字节全错位, 算出来的数字跟着错, 而且**一声不吭、半点提示都没有**。这种 bug 极难查, 因为程序照跑不误, 只是结果是错的。

举个例子。假设核函数声明某个操作数 tile 按 **128 字节 swizzle 布局**来存, 那么: TMA 描述符里就必须写对应的 128 字节 swizzle 模式, 后面 MMA 指令也必须按这同一套排布去读。三方说好同一套规矩, 数据才对得上。这也正是”布局记号 (layout notation)”存在的意义——它不是给人看

着玩的注释,而是一份”白纸黑字的合同”:你在编程框架(DSL)里写下的布局,必须跟 TMA 描述符、跟 Tensor Core 指令实际用的硬件布局,严丝合缝、字字对上。

换个角度想:swizzle 从不改变逻辑上的 tile。后面 MMA 要算的,还是同一个逻辑上的 A、B 矩阵小块,数值一个没变。swizzle 只管这块 tile 在物理上怎么跨着那些 bank 摆。一句话:它改的是”某个元素被放进了哪个物理格子”,而不是”这个元素本身是几”。理解这一点,你就不会被 swizzle 吓到——它本质只是个”换地方摆”的小动作。

3. 3D TMA: 在一次拷贝里同时完成「分块」和「swizzle」

一句话先理解

一句话先理解:前面讲的搬运是把一块平平整整的方块搬过去;但 Tensor Core 想要的摆法更讲究,得把方块切成一小颗一小颗”标准包装盒”码好。3D TMA 就是让搬运的同时,顺手把货码进这些标准盒子里。

3.1 问题:Tensor Core 想要的是「分块进 swizzle 原子」的布局

最普通的 TMA 拷贝是 2D 的——搬的是一块扁平的二维 tile,就是个规规矩矩的矩形,想象成一张表格直接搬过去。

可 Tensor Core 想要的工作台布局往往没这么省心:它要这块 tile 被切成一颗一颗的”标准小盒子”码放。这个”标准小盒子”有个术语,叫 swizzle 原子(atom)——”原子”在这里就是”不可再分的最小标准单元”的意思(借用了化学里”原子是最小单位”的说法,跟物理无关)。具体到 GPU,这个原子的尺寸是 8×128 字节(《数据布局及其记号》那章详细讲过),你可以理解成”硬件规定的、最顺手的一个标准搬运/摆放单位”。

现在矛盾来了:一块扁平的大方块,和”切成一颗颗标准盒子码好”这两种摆法对不上,怎么办?TMA 的解法挺妙:给搬运清单再加一个维度——从二维升到三维。

3.2 3D TMA 的寻址方式:(group, row, col)

3D TMA 把工作台 SMEM 当成一个三维的货架来寻址,用三个坐标 (group, row, col) 来精确点到任意一个位置:

- group(第几个盒子): 在一颗颗”标准盒子”(原子)之间跨步,相当于”走到第几个盒子”。
- row / col(盒子内的行、列): 定位某一个盒子内部的具体位置。

打个比方:这就像图书馆找书,要先说”第几个书架(group)”,再说”这个书架上的第几行第几列(row, col)”。

有了这三维坐标,一次 3D 拷贝就能一口气把原本两件分开的事都办了:

1. 分块(tile): 把这块大 tile 一个盒子一个盒子地铺开摆好。
2. swizzle(交织): 在每个盒子内部顺手做好交织。

最妙的是落地结果:数据刚搬进工作台的那一刻,就已经是 MMA 直接想要的样子了,不用再额外花一趟功夫去分块或者交织。一步到位。

结构图

连接关系:

- 3D TMA 拷贝 / 寻址 = (group, row, col) → atom 0 / (内部已 swizzle)
- 3D TMA 拷贝 / 寻址 = (group, row, col) → atom 1 / (内部已 swizzle)
- 3D TMA 拷贝 / 寻址 = (group, row, col) → atom 2 / (内部已 swizzle)
- atom 0 / (内部已 swizzle) → SMEM: 已是 MMA 期望布局
- atom 1 / (内部已 swizzle) → SMEM: 已是 MMA 期望布局
- atom 2 / (内部已 swizzle) → SMEM: 已是 MMA 期望布局

注意

注意: 原文配了个交互演示 *a 3D TMA copy, addressed as (group, row, col), tiling into swizzled shared memory*(即”一次按 (group, row, col) 寻址、把数据分块码进已交织共享内存的 3D TMA 拷贝”)。想直观体会这个寻址过程, 去原书点点看。

3.3 swizzle 格式的选择: 取决于 tile 能不能「填满」原子

那到底该选哪种 swizzle 格式 (128 字节 / 64 字节 / 32 字节) 呢? 这事跟分块绑得死死的。背后就两条道理, 慢慢说:

- **道理一:swizzle 越宽越好 (只要用得上)。**swizzle 越宽, 就越能把同一列数据摊薄到更多 bank 上, 后面读的时候就越不容易撞车排队。所以只要塞得下, **128 字节 swizzle 永远是默认首选**。回想前面超市的比方: 把顾客摊到 32 个收银台, 当然比摊到 8 个更不容易排队。
- **道理二: 但宽 swizzle 有” 门槛”, tile 太瘦就用不了。**一个 N 字节的原子 (标准盒子), 要求 tile 那条”连续摆放的方向”(术语叫**连续维度**, 就是内存里地址连着排的那一维) **正好能把这个盒子填满**。要是某块 tile 因为形状限制太”瘦”, 连续维度凑不够 128 字节、填不满这个大盒子, 那 **128 字节 swizzle 就用不成**, 只能退而求其次, 降到 64 字节; 还不够就再降到 32 字节。

经验法则: 在”能把盒子填满”的前提下, 挑**最大的那个 swizzle 格式**。又想散得开、又不能有填不满的空当, 取这两者的平衡。

原文举了个特别直观的例子: 拿一个 **16 × 16 的 tile** 直接套 128 字节 swizzle, 会撞车 (bank 冲突); 非得把它切成正好对上原子的 **16 × 8 分组**, 才能做到**无冲突 (conflict-free, 即完全没有 bank 冲突、取数据全程不用排队)**。

下面这张图帮你建立「填满 vs 填不满」的直觉:

结构图

连接关系:

- 连续维度 / (字节数) $\xrightarrow{\text{填满 128 字节}}$ 128B swizzle / (散布到最多 bank, 默认)
- 连续维度 / (字节数) $\xrightarrow{\text{只填 64 字节}}$ 降级到 64B swizzle
- 连续维度 / (字节数) $\xrightarrow{\text{只填 32 字节}}$ 降级到 32B swizzle
- $16 \times 16 \text{ tile} + 128\text{B swizzle} \xrightarrow{\text{直接铺}}$ 有 bank 冲突

- $16 \times 16 \text{ tile} + 128\text{B swizzle}$ $\xrightarrow{\text{切成 } 16 \times 8 \text{ 分组 (匹配原子)}}$ conflict-free

swizzle 格式	什么时候用	数据散到 bank 的程度
128 字节	连续维度能填满 128B 时 (默认首选)	最广 (最不容易排队)
64 字节	tile 偏小、填不满 128B 时降一档	中等
32 字节	tile 更小、连 64B 都填不满时再降一档	较窄

4. 完成机制 (一): 加载用 mbarrier

一句话先理解

一句话先理解: 既然搬运是”下了单就不管、后台慢慢搬”,那总得有个办法知道”货到底到没到”,才能安全地去用。这一节讲的就是”到货通知”怎么做。

4.1 为什么「发出指令」不等于「可以读」

别忘了前面反复强调的:TMA 拷贝是**异步**的。这就带来一个必须小心的陷阱: 指令发出去那一刻,数据根本还没到位 (后台正搬着呢)。

所以,后面要用这块数据的代码 (我们叫它**消费者 / consumer**,就是”等着用货的人”) **绝对不能**一看见 TMA 指令发出去了,就急吼吼跑去读工作台上的 tile——这时候去读,读到的多半是上一批的残留数据或者半成品。必须等 TMA 引擎真把字节**全写完了**,这块 tile 才安全、才能读。这就像外卖,你不能听见”已下单”就去开门拿,得等”已送达”。

那怎么知道引擎到底写完没? 对 TMA **加载 (load, 即”从大仓库往工作台搬进来”)** 来说,负责报”搬完了”这个信儿的,就是 **mbarrier**(细节看《异步协调:mbarriers》那章)。你可以把 **mbarrier** 理解成一个**专门的到货通知器**: 生产数据的一方搬完了就来这儿”打个卡”,等货的一方就守在这儿等”打卡齐了”的信号。

4.2 标准时序

下面这张时序图把”下单 → 后台搬 → 通知到货 → 安全读取”整个过程的五步走画了出来。先扫一眼,后面 4.3 会把最关键的那一步掰开揉碎讲:

时序图 (按时间顺序)

1. 备注:1. 初始化/复用流水线阶段的 **mbarrier**
2. 发起线程 → **mbarrier**:2. **mbarrier.arrive.expect_tx(bytes)** / (告知预期字节数 + 完成自身 arrive)
3. 发起线程 → TMA 引擎:3. 发起 TMA load
4. TMA 引擎 → **mbarrier**:4. 字节到达时做 **complete-tx** 更新
5. 备注: 满足两条件才翻转 **phase**: / **arrival** 计数满足且待传字节数归零
6. 消费者 (MMA 路径) → **mbarrier**:5. 等待 **barrier** 的目标 **phase**
7. 备注: 等待完成 → **SMEM tile** 就绪,MMA 可安全读取

4.3 关键 API: `mbarrier.arrive.expect_tx`

这一节出现一条 GPU 专用指令, 你不需要会写, 只要看懂它” 在干嘛” 就行。它长这样:

```
mbarrier.arrive.expect_tx(bytes)
```

逐段拆给你看这行在说什么:

- `mbarrier` —— 操作的是前面那个” 到货通知器”。
- `arrive` —— ” 到达/打卡” 的意思。
- `expect_tx` —— `tx` 是 `transfer`(传输) 的缩写,`expect_tx` 就是” 预告这次要传多少字节”。
- `(bytes)` —— 括号里填的就是这次预计要搬进来的字节总数。

所以这一条指令, 一口气干了两件事:

1. 登记” 这次预计要搬进来多少字节”(比如要搬 8192 字节, 就把这个数告诉通知器);
2. 顺手替发起线程在通知器上” 打个到达的卡”。

为什么要” 预告字节数”? 因为通知器要靠它来判断货齐没齐——它会一边收 TMA 引擎” 我又写进去了多少字节” 的汇报, 一边对照这个预告的总数, 直到对上, 才算” 真的搬完了”。这是一种很可靠的” 按量验收”。

关键

关键: 可别误会——调了这条指令**绝不等于通知器就亮绿灯了**。它只是” 预告 + 打卡”。真正放行, 还得等 TMA 引擎把字节一点点写够、回话说” 该到的字节全齐了”。

用术语说: 通知器有个状态叫 **phase(相位)**, 你可以理解成一个” 轮次开关”, 非得下面两个条件同时满足, 它才会翻转 (从” 还没好” 翻到” 好了”):

1. 该打卡的线程都打卡了 (arrival 计数到位);
2. 预告的字节一个不差地全搬到了 (待传字节数 pending byte count 清零)。

接下来, 等货的消费者就守在这个通知器上等。一旦等到它翻转 (到达目标 phase), 就说明工作台上的 tile 已经备齐, 这时候算矩阵乘的 MMA 那条路才能放心去读。两个条件缺一不可, 正是这个机制可靠的原因——既保证” 人到齐”, 又保证” 货到齐”。

原文这儿还配了张同步流程图 (`tma_sync_flow.png`), 意思跟上面那张时序图一模一样: 生产者 (producer, 生产数据的一方) 是 TMA 引擎, 消费者 (consumer, 等着用数据的一方) 是 MMA 那条路 (或者随便哪段要读工作台 tile 的代码), 通知器 `mbarrier` 就是横在它俩中间那个说一不二的交接关卡。这套” 一方生产、一方消费、中间靠 barrier 交接” 的模型在 GPU 异步编程里到处都是, 不只 TMA 用——你这里学会了, 后面看别的异步场景也是同一套路。

5. 完成机制 (二): 存储用 `commit group + wait group`

一句话先理解

一句话先理解: 搬进来要” 到货通知”(因为有人等着用); 搬出去关心的是另一码事——” 我这块工作台啥时候能空出来再用”, 所以用的是另一套更简单的” 批量验收” 机制。

6. 为什么 TMA 对流水线 (pipelining) 至关重要

一句话先理解

一句话先理解: 前面所有铺垫, 都是为了这一节。TMA 最大的价值, 是让”搬下一批”和”算这一批”同时发生、互不耽误——这就是流水线。

前面铺垫了这么多,TMA 真正大显身手的时刻, 就是它当上**流水线 (pipelining)** 里一环的时候。先解释流水线是啥: 它就是工厂流水线那个意思——**别等一道工序干完再开始下一道, 而是让多道工序同时进行、首尾搭接**, 这样整体吞吐量最高。

放到 GPU 上, 核心思路一句话就够:**Tensor Core 正埋头算着「当前这块 tile」的时候, 核函数就抢先把「下一块 tile」的搬运给发出去了**。搬运在后台闷头跑, 计算在前台连轴转, 两件事重叠进行; 等「下一块 tile」终于轮到要被算了, 它其实早就搬好躺在工作台上了——这时候那个到货通知器 (barrier) 负责把”搬好了”和”该算了”这两条线对接上。这么一来,Tensor Core 几乎永远不停停下来等数据, 算力就被榨得满满当当。

为什么这招管用: 回想第 0 节的痛点——计算单元最怕的就是”等数据干瞪眼”。流水线的整个目的, 就是用”提前搬”把这段等待时间藏起来 (术语叫”隐藏延迟”)。搬运再慢, 只要它是在你算上一块的同时偷偷进行的, 你就感觉不到它的耗时。

一个典型的矩阵乘 (GEMM) 循环就翻来覆去地用这套结构, 它有个专门的名字, 叫**多级缓冲 (multi-stage buffering)**——”多级”指的是工作台上同时备好几块格子轮流用, 下面这张图画的就是最简单的两块格子 (两级) 的情形:

结构图

分组:

- **SMEM 多级缓冲:** 「Stage 0 / 正被 MMA 消费」「Stage 1 / 正被 TMA 填充」

连接关系:

- Stage 0 / 正被 MMA 消费 → Tensor Core (MMA) / 前台: 算当前 tile
- TMA 引擎 / 后台: 搬未来 tile → Stage 1 / 正被 TMA 填充

这里出现一个新词 **stage(阶段 / 一级缓冲)**: 你就把它理解成”工作台上的一块格子”。图里有两块格子, 一块正被 Tensor Core 拿去算 (Stage 0), 另一块正被 TMA 在后台填新数据 (Stage 1)。循环每往前走一步, 这几块格子的角色就**轮一次班 (rotate)**——刚算完的那块腾出来让 TMA 填, 刚填好的那块交给 Tensor Core 算, 如此循环。就像两个传菜口轮流上菜, 厨房和餐桌都不用停。

要让这套轮班不出乱子, 规矩就两条 (都靠通知器 barrier 把关):

- **Tensor Core 想读某块格子之前**, 先等这块格子的”到货通知”(确认 TMA 真把数据搬齐了, 别读到半成品);
- **TMA 想往某块格子写新数据 (覆盖旧的) 之前**, 先确认上一轮用这块格子的消费者已经读完了 (别把人家还没看完的数据给盖掉了)。

关键

关键：讲到这儿，你就能明白为啥在 Blackwell、Hopper(NVIDIA 较新的两代 GPU 架构;Hopper 就是大名鼎鼎的 H100 那一代,Blackwell 比它更新) 风格的核函数里,TMA 和 mbarrier 几乎总是成对出现、形影不离。道理很简单:TMA 提供的是” 异步搬运引擎”，只管把货搬过来;mbarrier 提供的是” 精确判断货啥时候就绪” 的本事，只管通知。一个负责搬、一个负责通知，缺了任何一个，这套流水线都转不起来——搬了不通知，没人敢用；通知了没东西搬，通知个寂寞。所以它俩是天生的搭档。

小结

回头把这一章串成一句人话:GPU 算得飞快，但前提是数据喂得上;TMA 就是那个专门负责” 又快又不耽误干活地喂数据” 的硬件搬运工。下面是要点回顾：

- **TMA 就是个硬件搬运引擎：**派一个线程喊一声下个单，硬件就在后台异步把整块矩形 tile 搬好，把” 搬运” 这件杂活彻底从计算线程的活儿里拎出去——计算单元 (Tensor Core) 就不会被搬运拖后腿了。
- **张量映射描述符 (那张” 搬运清单”)** 是核心抽象：形状、步长、元素大小、tile 形状、swizzle 摆法全装在里面；它把” 一次拷贝” 从” 一长串自己算地址、自己 load/store 的循环” 简化成” 一份清单 + 喊一声发起”。
- **swizzle 在搬进来的路上顺手就把货摆对了：**数据落进工作台那一刻，已经摆成 Tensor Core 最顺手的样子，省去单独重排。但有铁律：搬运清单、工作台布局、读数据的 MMA 这三方必须说好同一套摆法，否则硬件会” 忠实地” 给你算出错误结果，还不报错。
- **3D TMA 用 (group, row, col) 三维坐标寻址，一次拷贝就把” 按标准盒子 (原子) 分块” 和” 盒子内交织” 两件事一块办了；**swizzle 格式 (128/64/32 字节) 选哪个，看 tile 那条连续维度能不能填满对应的盒子——经验法则是：能填满的前提下挑最大的。
- **” 确认搬完了” 分两套机制：**搬进来 (加载) 用带字节核对的 mbarrier(人到齐 + 字节到齐，通知器才翻转放行)；搬出去 (存储) 用 commit group + wait group(打包一批、等它排空)。两套盯的是不同的交接时刻。
- **TMA 的终极价值落在流水线上：**前台算这一块、后台搬下一块、通知器把两边对齐——这正是现代矩阵乘 (GEMM) 和注意力 (attention) 核函数能把 Tensor Core 喂得满满当当、不留空闲的关键。

延伸阅读

- 原文:[Async Data Movement: TMA —Modern GPU Programming for ML](#) Sys
- 相关章节 (原文中引用)：《What Makes a Kernel Fast》(核函数性能受限因素)、《Data Layout and Its Notation》(数据布局与 swizzle 原子)、《Async Coordination: mbarriers》(mbarrier 的异步协调模型)。

术语对照

中文	English
----	---------

张量内存加速器	Tensor Memory Accelerator (TMA)
张量映射描述符	tensor-map descriptor
tile 坐标	tile coordinates
交织 (模式)	swizzle (mode)
swizzle 原子	swizzle atom
共享内存	shared memory (SMEM)
全局内存	global memory (GMEM)
异步屏障	mbarrier
相位	phase
字节计数追踪	byte-count tracking
提交组 / 等待组	commit group / wait group
矩阵乘累加	MMA (matrix multiply-accumulate)
协作线程阵列	CTA (Cooperative Thread Array)
通用矩阵乘	GEMM
计算受限	compute-bound
流水线	pipelining
多级缓冲	multi-stage buffering
bank 冲突	bank conflict
分发 (决策)	dispatch
作用域	scope

第 06 章 · 特殊内存:TMEM(Tensor Memory)

原文:Special Memory: TMEM

本章要点

本章要点 (TL;DR)

别急, 下面几条要点里有不少生词 (TMEM、Tensor Core、累加器、SMEM……), 现在看不懂完全正常——这一章后面每个词都会用大白话从头讲一遍。这里先囫圇扫一眼, 留个印象就好:

- **TMEM(Tensor Memory, 张量内存)** 是 Blackwell 这一代显卡才有的一块”片上小内存”(片上 = 直接长在 GPU 芯片里, 离计算单元很近、所以特别快)。它专门伺候新一代的矩阵乘硬件指令族 **tcgen05**。它的形状是一张二维表格:**128 行 (叫 Lane) × 最多 512 列 (叫 Col)**, 每个格子宽 32 位 (也就是 4 个字节, 正好装一个常见的 float)。这块”便笺纸”(scratchpad, 意思是临时打草稿的地方) 在每个 SM(GPU 内部的一个独立计算核心, 后面会细讲) 上各有一块。
- 它来到世上的头号任务, 就是给”寄存器”减负。寄存器是每个线程私有的、最快但极少的存储 (类比: CPU 里的寄存器)。以前的显卡 (Hopper 及更早) 做矩阵乘时, 算出来的中间结果 (叫累加器 **accumulator**) 得全程占着寄存器; Blackwell 改成把累加器直接写进 TMEM, 这样寄存器不被挤爆, 也能让矩阵乘硬件一次处理更大的数据块、跑得更快。
- **TMEM 靠「二维坐标 (Lane, Col)」找位置**——给一个”第几行、第几列”, 而不是像普通内存那样给一个”第几个字节”。在本书的布局记号 (TIRx) 里, 这两根坐标轴写作 **TLane** 和 **TCol**。
- **TMEM 得你自己写代码去申请、用完自己还** (以线程块为单位、每次按 32 列起批)。它不像寄存器那样编译器替你安排好——你得把它当成一种”数量有限、要精打细算做预算”的资源, 和共享内存 (SMEM) 一个性质。
- **平时读写共享内存的那套指令根本碰不到 TMEM**。数据想进想出, 只能走专用、而且是**异步** (asynchronous, 指令发出去后不等做完就先返回, 你得另外等它) 的 **tcgen05** 指令: **tcgen05.ld** (从 TMEM 搬回寄存器)、**tcgen05.st** (从寄存器写进 TMEM)、**tcgen05.cp** (从共享内存批量拷进 TMEM), 每条还得配上各自专门的”等它做完”的机制。

前置知识

前置知识: 这一章会反复用到几个 GPU 基本概念——寄存器、共享内存、线程束 (warp)、Tensor Core、矩阵乘。本章每个词第一次出现时都会用大白话当场解释一遍, 所以就算你完全没碰过 GPU 也能往下读。但如果你想先打个底、读起来更顺, 可以花十分钟翻一下第 0 章 · 极简入门; 任何时候卡在某个术语上, 也可以随手查术语对照表。

引子: 寄存器不够用,Blackwell 才另开了一条路

一句话先理解

一句话先理解:GPU 上做矩阵乘, 算到一半的中间结果总得有地方暂存。以前一直塞在” 寄存器” 里, 但寄存器太少不够用;Blackwell 就专门加了 TMEM 这块新内存来接这个活儿。这一节就讲清楚” 为什么寄存器不够用”。

要搞懂 TMEM 为什么会出现, 咱们先问一个最朴素的问题: 矩阵乘一边算, 算到一半的中间结果, 到底该放哪儿?

先解释两个词。GPU 做深度学习, 核心工作量就是**矩阵乘法**(两个大矩阵相乘), 所以厂商专门造了一块硬件来加速它, 叫 **Tensor Core**(张量核心)——你可以把它想成” 专门飞快地算矩阵乘的小电路”。它干活的方式叫 **MMA(Matrix-Multiply-Accumulate, 矩阵乘累加)**: 不是一口气把整个矩阵乘完, 而是一小块一小块地乘, 然后把每一小块的结果**累加**到一个 running total 上。这个一直被累加、存放当前总和的东西, 就叫**累加器 (accumulator)**。

那这个累加器存哪儿? 在 Hopper 和更早的 GPU 上, 答案很简单——放**寄存器**里。寄存器是 GPU 上最快但也最少的一种存储, **每个线程私有一份** (类比 CPU 寄存器, 但 GPU 上成千上万个线程, 每个都自带一小撮)。整个流程也好理解:

1. MMA 指令一算完, 就吐出一个**寄存器片段 (register fragment)**——所谓” 片段”, 是因为一整个累加器矩阵被切成很多小份, **每个线程的寄存器里只放其中一小份**, 拼起来才是完整的累加器;
2. 接下来整个**计算阶段 (compute phase)**, 内核 (kernel, 就是跑在 GPU 上的那段程序) 就让这个片段**一直赖在寄存器里**, 反复往上累加;
3. 等到收尾的 **epilogue(尾声阶段)**——专门负责给最终结果做类型转换、逐元素加工、再写回去——内核才把片段从寄存器读出来, 做类型转换 (cast, 比如把高精度的 float32 转成省空间的 float16)、做逐元素操作, 最后写回内存。

这一套听着挺顺。可问题来了: **寄存器又少又金贵, 而且是一个线程一个线程死死分好的**——总量就那么点, 分给这个变量多了, 分给那个就少了。这就埋下了一个绕不过去的矛盾。

关键

关键:Tensor Core 就好「大 tile」这一口。这里的 **tile(瓦片)** 指的是从大矩阵里切出来的一个小方块——因为矩阵太大, 没法一次性塞进 GPU, 只能切成小块一块一块算。tile 切得越大, Tensor Core 每次能干的活越多、利用率越高、整体越快 (这叫吞吐高)。可 tile 一大, 要暂存的累加器片段也跟着变大; 片段一变大, 它在每个线程的寄存器里就占得越多, 把线程本来要用的其他变量全挤出去了。说白了, 「想用大 tile 跑得更快」和「累加器全塞寄存器」这两件事, 直接顶上了——鱼和熊掌不可兼得。

这个矛盾, 画张表就一目了然。每个线程能用的寄存器数量是固定死的 (” 寄存器文件” 就是一个线程那一整堆寄存器的统称)。同一块寄存器文件, 在小 tile 和大 tile 两种情况下被占成什么样, 对比一下:

场景	累加器片段占用	留给其他工作变量的空间	结果
小 tile	较小	充足	舒服
大 tile	几乎占满整个寄存器文件	被严重挤压	溢出 / 降低占用率

表里两个词解释一下:” 溢出 (spill)”——寄存器实在装不下,编译器只好把一部分变量临时挪到慢得多的内存里,跑起来就拖后腿;” 占用率 (occupancy)”——简单说就是 GPU 上同时能跑多少个线程,每个线程吃的寄存器越多,能同时挤上去的线程就越少,GPU 就越闲。两样都是越差越糟。

Blackwell(NVIDIA 2024 年这一代 GPU 架构的代号)就是冲着这个矛盾下的刀。思路很直接:既然寄存器装不下大累加器,那干脆**别让累加器一直占着寄存器**。所以新指令 `tcgen05.mma` 不再把累加器留在寄存器里,而是直接写进一块**全新加的片上内存——也就是 TMEM**。这块内存,早期的 NVIDIA GPU 压根儿没有,是这一代才新长出来的。

这么一改,好处立竿见影:**Blackwell 能开更大的 Tensor Core tile 了,还不用拿每个线程的寄存器去硬扛整个累加器**。那代价呢?天下没有免费的午餐——TMEM 不像寄存器那样白送给你,编译器替你打理好。下面这些活儿,都得你在内核代码里**亲自动手**:

- 自己去**分配 (allocate)** 一块 TMEM(就像写代码时手动 `malloc` 一块内存);
- 用**对的布局 (layout)** 去寻址 (布局 = 数据在内存里怎么摆放的规则,后面会细讲);
- 用**对的指令**把数据搬进搬出 (普通的内存读写指令在这儿不好使);
- 一个线程块用完了,记得把这块 TMEM **释放 (free)** 还回去 (就像手动 `free`,不然别人没得用)。

这里出现了一个新词 **CTA**,后面会反复用到,先记一下:CTA(Cooperative Thread Array,协作线程阵列) 其实就是一个**线程块 (thread block)**——一组会被一起调度、能互相协作、共享同一块快内存的线程。你暂时把”CTA”和”线程块”当成同义词就行。

下面我们就一条一条来看:TMEM 长什么样、怎么分配、怎么读写。

TMEM 是一个二维地址空间

一句话先理解

一句话先理解:你平时用的内存都是”一维”的——给一个地址(第几个字节)就能定位,像一条长长的纸带。TMEM 偏不,它是”二维”的,像一张 Excel 表格,你得给”第几行、第几列”才能定位到一个格子。

先记一句话:**TMEM 不是一条扁平的字节数组,它是一张二维网格 (像棋盘 / Excel 表)**。普通内存那套「给个线性字节偏移(第几个字节)就能定位」的玩法,在这儿行不通——你必须给一对坐标。硬件给 TMEM 的两根坐标轴起了名字,一根叫 **Lane**(可以理解成”行”),一根叫 **Col**(Column 的缩写,就是”列”):

维度	含义	规模	备注
Lane	行 (纵向)	128 个 Lane 行	对应 TIRx 记号里的 TLane 轴
Col	列 (横向)	最多 512 个 Col 列	每个 Col 是一个 32 位的列 ,对应 TIRx 记号里的 TCol 轴

这个形状可不是拍脑袋定的: **矩阵乘指令 `tcgen05.mma` 往里写累加器时,用的就是这套二维结构**,因为矩阵本身就是二维的,用二维内存来装它最顺手。所以你想定位 TMEM 里的某个位置,得给一对坐标——一个 Lane、一个 Col,而不是一个字节偏移。

下面这张表就把这张二维网格画了出来 (对应原书的 `tmem_grid` 图): 纵着看是 Lane(128 行), 横着看是 Col(最多 512 列, 每个 Col 宽 32 位)。每个格子, 就是一个 (Lane, Col) 坐标对应的一个存储位置。

	Col 0	Col 1	Col 2	...	Col N-1
Lane 0				...	
Lane 1				...	
Lane 2				...	
...				...	
Lane 127				...	

- 竖着的行:128 个 Lane, 也就是 **TLane** 轴。
- 横着的列: 最多 512 个 Col, 也就是 **TCol** 轴, 每个 Col 宽 32 位。

在 TIRx 布局记号里怎么写

在动手前先说清楚 **TIRx** 是个啥: 它是本书用来描述 GPU 程序的一套**领域专用语言 (DSL)**, 你可以粗略地把它理解成”一种专门用来写 GPU 张量计算的小语言”(IR 即 Intermediate Representation, 中间表示, 是编译器内部表示程序的一种形式)。它最讲究的事情之一, 就是**精确描述一块数据在内存里是怎么摆的**——这套描述规则就叫**布局 (layout)**。

为什么布局这么重要? 因为内存就是一堆格子, 同一份数据, 你可以横着摆、竖着摆、隔几个摆一个……摆法不同, 读写时算地址的方式也不同。布局就是把”这份数据具体怎么摆”白纸黑字写下来, 免得读写双方理解不一致、把数据读错位。

所以在 TIRx 里声明一块 TMEM 缓冲区时, 你得给它配一个**能把 Lane、Col 这两根硬件坐标说清楚的布局**。在布局记号里 (完整定义在原书「Data Layout and Its Notation」那一章), 我们把 TMEM 的 Lane 轴起名叫 **TLane**, Col 轴起名叫 **TCol**。

注意

注意:TLane / TCol 不过是布局轴的名字, **并不是要替换官方硬件术语 Lane / Col**。起这两个名, 纯粹是想在 DSL 里把 TMEM 的两个维度**写明白**, 顺便跟其他张量布局用上同一套写法。

举个例子, 一个累加器 tile 可以这么写:

```
S[(128, N) : (1@TLane, 1@TCol)]
```

这串记号别被它吓到, 拆开看其实很简单——它在说”有一块叫 S 的数据, 形状是这样、摆法是这样”:

- 前半截 (128, N)——讲的是**形状**: 这个 tile 有 **128 行** (沿 Lane 这一维)、**N 列** (沿 Col 这一维)。
- 后半截 (1@TLane, 1@TCol)——讲的是**步长 (stride)**。步长这个词解释一下: 它表示”逻辑上挪一格, 物理上要在某根轴上走几步”。**1@TLane** 的意思是, 逻辑上往下走一行, 物理上就在 **TLane** 轴上前进 1;**1@TCol** 的意思是, 逻辑上往右走一列, 物理上就在 **TCol** 轴上前进 1。
- 合起来看: **这就是最直来直去、最没花样的一种布局**——一行挨一行沿 **TLane** 排, 一列挨一列沿 **TCol** 排, 中间不搞任何重排、不搞交错。可以理解成”二维数组怎么想就怎么放, 所见即所得”。

关键

关键: 千万别把 TMEM 当成「Tensor Core 背后那个看不见的仓库」。它是整个 tile 布局里堂堂正正的一员: 内核得给它起名、从里头分配列, 还得用一个跟 `tcgen05` 指令读写方式严丝合缝的布局。布局一旦对不上, 数据就错位了。

分配: 把 TMEM 当成像 SMEM 一样要做预算的资源

一句话先理解

一句话先理解: 用 TMEM 之前得先”申请”一块, 用完得”归还”, 全靠你写代码手动管——很像 C 里的 `malloc/free`, 而不像普通局部变量那样系统自动帮你搞定。

内核要想用 TMEM, 得先里头占块地方。这点跟寄存器完全两码事: **寄存器是编译器替你分好的, TMEM 你得自己张口去要。**

分配的规矩不多, 记住这几条就行:

- **以 CTA(也就是线程块) 为单位申请:** 由这个 CTA 里某一个 **warp** 出面去要一段 TMEM 列区间。这里的 **warp(线程束)** 是 GPU 的一个核心概念, 先讲明白: GPU 调度线程不是一个一个调度的, 而是 **32 个线程绑成一捆**一起调度、一起执行同一条指令——这一捆 32 个线程就叫一个 **warp**。打个比方, warp 就像”32 个人组成的方阵, 必须迈同一步、做同一个动作”。所以”由某一个 warp 出面申请”, 意思就是由这 32 个线程作为一个整体去申请。
- **32 列起步:** 你想要多少列, 硬件都会**向上取整到 32 的倍数** (比如你要 5 列, 实际给你 32 列; 要 40 列, 给你 64 列)。
- **要完会给你一个基址 (base TMEM address):** 基址就是”你占下那块地的起始坐标”。往后所有 `tcgen05` 指令, 都靠这个基址去定位你占下的那块区域 (类比 `malloc` 返回给你的那个指针)。
- **用完务必释放 (free):** CTA 干完活、要结束的时候, 把这块地还回去, 好让后面的 CTA 能用。忘了还, 就相当于内存泄漏。

想真正理解它, 最好的办法就是把 **TMEM 当成和共享内存一个性质的、CTA 级别、得精打细算做预算的资源**——也就是说, 一个 CTA 能用的 TMEM 是有限的一块, 你得想清楚这块地怎么分着用:

结构图

分组:

- 一个 CTA 的资源预算
- **SMEM 预算 (必须塞进 SMEM 容量内):** 「TMA 暂存的矩阵操作数」「中间缓冲」「...」
- **TMEM 预算 (必须塞进 TMEM 容量内):** 「MMA 累加器 (列)」 「block-scaled 缩放因子」 「临时暂存 (staging)」

上图里出现了一个新词 **SMEM(Shared Memory, 共享内存)**, 顺手讲清楚: 它是一个 **CTA 内部所有线程共享的一块片上快内存** (片上, 所以比主存快很多)。你可以把它类比成”一块由程序员手动管理的高速缓存”——CPU 的缓存是硬件自动帮你填的, 而 SMEM 里放什么、什么时候放, 全得你自己写代码安排。图想表达的意思是: 一个 CTA 同时背着两本”预算账”——SMEM 一本、TMEM 一本, 各自都有容量上限, 里头要塞的东西都得精打细算。

这么一来,TMEM 就成了**内核做资源规划 (resource planning)** 时绕不开的一项。具体怎么取舍,都是很实在的账:

决策	收益	代价
用更大的累加器 tile	提升 Tensor Core 吞吐	消耗更多 TMEM 列
启用 block-scaled MMA(分块缩放)	支持更细粒度的量化缩放	需要额外 TMEM 空间放缩放因子

表里的 **block-scaled MMA(分块缩放矩阵乘)** 这里先简单点一下、后面还会再提: 为了省内存、跑得更快, 常常会用低精度的数 (比如 8 位) 来做矩阵乘, 这叫量化; 但低精度容易掉精度, 于是给每一小块数据各配一个”缩放因子”来补偿——这就是”分块缩放”。问题是这些缩放因子也得找地方放, 而它们就放在 TMEM 里, 自然要多吃一份预算。

这些用途加到一块, **全都得挤进那点有限的 TMEM 预算里**——道理跟你必须把所有数据塞进 SMEM 那本预算里, 一模一样。

注意

注意:TMEM 的总量是有天花板的——每个 SM 就一块 (128 × 512 个 32 位列), 不能再多。这里又冒出个 **SM(Streaming Multiprocessor, 流式多处理器)**, 它是 GPU 内部的一个相对独立的计算核心; 一块 GPU 由几十上百个 SM 拼成, 每个 SM 自带自己的寄存器、SMEM、TMEM, 并发地各干各的。正因为每个 SM 的 TMEM 就这么一小块, 它和占用率、SMEM 用量这几样凑在一起, 共同决定了一个内核能开多大 tile、能在一个 SM 上同时跑几个 CTA。

读写 TMEM: 三条专用的异步通路

一句话先理解

一句话先理解:TMEM 是个”自成一国”的特殊内存, 平时读写内存的指令进不去。数据想进出 TMEM, 只有三条专门修的”路”, 而且这三条路都是异步的——发车了不等到站, 你得自己盯着它什么时候到。

前面那句话再敲一遍黑板: **平时读写共享内存的那套指令 (ld.shared / st.shared, 字面意思就是 load / store shared, 从/往共享内存读写) 根本够不着 TMEM**。TMEM 是一块独立的地址空间 (可以理解成”另一个国家、另一套门牌号”), 数据想进想出, 只能走专门的 **tcgen05** 指令。这样的专用通路总共**三条**。

下图把 Blackwell Tensor Core 的整条数据通路都画出来了, 你一眼就能看到 TMEM 正好卡在最中间这个枢纽位置:

结构图

连接关系:

- 全局内存 GMEM $\xrightarrow{\text{TMA}}$ 共享内存 SMEM
- 共享内存 SMEM $\xrightarrow{\text{tcgen05.cp} / (\text{批量拷贝缩放因子})}$ TMEM / 128 Lane × 最多 512 Col
- 共享内存 SMEM $\xrightarrow{\text{tcgen05.mma} / \text{读 SMEM 操作数, 累加写入 TMEM}}$ TMEM / 128 Lane × 最多 512 Col
- 寄存器 Registers $\xrightarrow{\text{tcgen05.st} / (\text{寄存器} \rightarrow \text{TMEM})}$ TMEM / 128 Lane × 最多 512 Col

- $\text{TMEM} / 128 \text{ Lane} \times \text{最多 } 512 \text{ Col} \xrightarrow{\text{tcgen05.ld} / (\text{TMEM} \rightarrow \text{寄存器})} \text{寄存器 Registers}$
- $\text{寄存器 Registers} \xrightarrow{\text{epilogue:cast/逐元素/写回}} \text{输出到 GMEM}$

图里又出现两个新词，先解释：**GMEM(Global Memory, 全局内存)**就是 GPU 的“主存”——容量最大、但离计算单元最远、也最慢的那块内存，你的输入数据、输出结果最终都待在这儿（类比 CPU 世界的内存条）。**TMA(Tensor Memory Accelerator, 张量内存加速器)**是一块专门负责在 GMEM 和 SMEM 之间**异步搬运大块数据**的硬件——你只要吩咐它”把这一大块从那儿搬到这儿”，它自己后台搬，不占用线程去一格一格地拷。

整条链路一句话串下来：**TMA** 把矩阵操作数（operand，就是参与矩阵乘的那两个输入矩阵）从 GMEM 搬进 SMEM 暂存 → `tcgen05.mma` 读这些操作数，把乘累加的结果写进 TMEM（要是用了分块缩放，缩放因子也搁在 TMEM 里）→ 计算阶段一结束，`tcgen05.ld` 把累加器从 TMEM 搬回寄存器 → `epilogue` 在寄存器里做类型转换等收尾，把最终结果写回 GMEM。

通路一：`tcgen05.ld`(TMEM → 寄存器)

这条用得最勤，**epilogue** 一定会走这条（`.ld` 就是 load 的缩写，读取的意思）。回想一下：**MMA** 阶段把累加器算在了 TMEM 里，可 `epilogue` 要做的类型转换、逐元素操作、写回，都得在寄存器里干——所以必须先把累加器从 TMEM 搬回**寄存器片段**，才好接着收尾。

这条通路有个挺要紧的脾气：它是「一群人分着干」的，不是一个线程包圆。

关键

关键：在 DSL 这一层，你写”加载一次 TMEM”看着是一条指令，但它其实是**整个 warp-group 凑一块儿分头干的**。这里又来个新词 **warpgroup(线程束组)**：它是 **4 个 warp 合成的更大协作单位**，一共 **128 个线程**（ 4×32 ）。这条加载在编译时会被拆（**lower**，即降级展开成更底层的指令）成**四条 warp 级的 `tcgen05.ld`**，一个 warp 摊一条。每个 warp 负责 128 个 Lane 行里的 **32 行**，四个 warp 一拼，正好把整个 Lane 维度（也就是布局记号里的 **TLane 轴**，共 128 行）铺满——典型的”分工合作，各管一段”。

整个 **TLane 维度**一共 128 行，一个 **warpgroup**(4 个 warp) 分着扛，每个 warp 发一条 `tcgen05.ld`，各管 32 行：

warp	负责的 Lane 行	发出的指令
warp 0	Lane 0 .. 31	<code>tcgen05.ld</code>
warp 1	Lane 32 .. 63	<code>tcgen05.ld</code>
warp 2	Lane 64 .. 95	<code>tcgen05.ld</code>
warp 3	Lane 96 .. 127	<code>tcgen05.ld</code>

这条指令本身还带着一大家子**加载形状 (load shape)** 可选，什么 `.16x64b`、`.16x128b`、`.16x256b`、`.32x32b`、`.16x32bx2` 等等（这些后缀就是在说”这一趟读多大一块、长宽各多少”），外加一个**重复因子 (repeat factor)**，从 `.x1` 一路到 `.x128`(表示”这个动作连做几遍”)。你不用记这些具体值，只要抓住一点：**你挑哪个形状，就决定了这一趟读几列 TMEM、每个线程能分到几个寄存器**。

不过最该上心的，不是这些形状，而是**读回来的寄存器片段到底长啥样**——也就是”哪个线程拿到了 TMEM 里哪个位置的数”。在常见的 `epilogue` 路径里，规律是这样：

关键

关键:lane 1(这里的 lane 指”线程在 warp 里的编号”,0 到 31) 拿到的值, 来自 **TMEM 第 1 / 4 行** (整数除法) 的那两列。换句话说, 每 4 个相邻的线程共享同一个 TMEM 行。

这有啥讲究? 讲究就在于: 它给出的形态, 跟早期 GPU 上 MMA 直接暴露的那种「一个 lane 一份累加器片段」长得一模一样 (细节见原书「Tensor Core Operand Layouts Across GPU Generations」)。也就是说, 虽然底层换了块内存, 但摆到寄存器里的样子和老款一致。这份**前后一致**, 可太值钱了——下面这条注意就讲它值在哪。

注意

注意: 正因为片段的样子没变,Blackwell 的 epilogue 能直接捡起 Ampere(2020 年那代)mma、或 Hopper(2022 年那代)wgmma 时代那套「在寄存器里做类型转换 + 写回」的老代码接着用——哪怕这一代累加器其实是住在 TMEM 里的。说白了:TMEM 不过是给累加器换了个住处, 可一旦搬回寄存器, 上层软件几乎一行都不用改。对工程师来说, 这意味着迁移成本极低。

把 1 / 4 这个对应关系画出来, 大概是下面这个样子 (对应原书 `tcgen05_ldst` 那张图,m8n8 片段): 每 4 个 lane 共用同一个 TMEM 行, 每个 lane 从这一行里取 2 列 (也就是 2 个值) 放进自己的寄存器。

TMEM 行 (row = 1 / 4)	由哪些 lane 读取	每个 lane 拿到
row 0	lane 0, 1, 2, 3	该行的 2 列 (2 个值)
row 1	lane 4, 5, 6, 7	该行的 2 列 (2 个值)
row 2	lane 8 .. 11	该行的 2 列 (2 个值)
...	...	...

通路二:tcgen05.st(寄存器 → TMEM)

`tcgen05.st` 就是 `tcgen05.ld` 反过来跑 (.st 即 store, 写入): 线程手里已经攥着一个寄存器片段, 想把它塞回 TMEM。

啥时候用得上? 比方说某些操作数或者中间值, 得先在寄存器里过一道手 (staging, 中转 / 暂存的意思) 加工一下, 再写进 TMEM, 留给后面某条 `tcgen05` 指令接着用。

通路三:tcgen05.cp(共享内存 → TMEM)

`tcgen05.cp` 走的是批量拷贝 (bulk copy, 一次搬一大块) 这条路 (.cp 即 copy), 它最常干的活儿就是搬前面提过的分块缩放矩阵乘的那些缩放因子。流程分两步:

1. 先用 **TMA**(那块异步搬运硬件) 或者普通线程代码, 把缩放数据 (scale data) 先备到**共享内存 (SMEM)** 里;
2. 再让 `tcgen05.cp` 把它从 SMEM 搬进 TMEM, 并且摆成 **Tensor Core** 指望的那种 **TMEM 布局** (摆错了 Tensor Core 就读不对)。

三条通路有个共同点: 全是异步的

先说为什么要异步, 你才知道这套麻烦事是图个啥。搬数据慢, 如果指令必须”原地干等到搬完才返回”, 线程就被白白卡住、啥也干不了。异步的好处是: 指令一发就立刻返回, 线程可以趁数据在后台

搬的工夫去干别的活，等真要用结果时再回头确认它到了——这样硬件利用率高得多。代价就是：你得自己负责”确认它到了”。

关键

关键:tcgen05.ld、tcgen05.st、tcgen05.cp 这三条全是异步 (asynchronous) 的。也就是说，指令很可能在数据还没真正搬完的时候就先返回了。所以在你真正去用这份结果、或者回头再动那块内存之前，一定得用对的”等它完成”的机制把它等住，否则就会用到还没搬好的半截数据 (细节见原书「Async Coordination: mbarriers」)。

这里特别容易踩坑：到底用哪种方式等，得看你发的是哪条指令——三条指令各有各的等法，不能混用。单拉个表对照：

指令	完成 / 等待机制
tcgen05.ld	通过 tcgen05.wait::ld 等待完成
tcgen05.st	通过 tcgen05.wait::st 等待完成
tcgen05.cp	像 tcgen05.mma 一样，通过 commit group(提交组)+mbarrier 完成

表里两个词解释一下:commit group(提交组) 是把若干条异步操作”打包成一组”提交，然后整组一起等;mbarrier(memory barrier 的一种，异步屏障) 是一个专门用来”等异步操作做完”的同步对象——你可以把它想象成一个计数器闹钟：操作做完会去敲它一下，等它被敲够了次数，线程就知道”可以放行了”。

还有一点很容易漏：

注意

注意：要是数据得从一组线程 (生产者 producer，负责写数据的) 交到另一组线程 (消费者 consumer，负责读数据的) 手上，那光「等数据搬完」还不够，内核多半还得补一道栅栏 (fence)。栅栏是一种同步原语，作用是强制规定内存读写的先后可见顺序——它保证”我写好的东西，你那边一定能按正确顺序看到”。少了这道栅栏，接收方就可能读到旧数据，或者读得乱七八糟、半新半旧。

把整条数据通路串起来:TMEM 就在 Tensor Core 数据流的正中间

一句话先理解

一句话先理解：这一节不引入新东西，就是把前面所有零件按”数据从进到出”的顺序排成一条流水线，让你看清 TMEM 在整条链路里的位置。

最后退一步，咱们从头到尾捋一遍:TMEM 就坐在 Blackwell Tensor Core 数据通路的正中间这个枢纽位置。数据从最慢最大的 GMEM 出发，一站站往里走，最后从 GMEM 出去：

阶段	谁在动	数据从哪到哪
1. 预取操作数	TMA	GMEM → SMEM
2. (block-scaled) 准备缩放因子	TMA/线程 + tcgen05.cp	GMEM/SMEM → TMEM
3. 矩阵乘累加	tcgen05.mma	读 SMEM 操作数，累加写入 TMEM

4. 取回累加器	<code>tcgen05.ld</code>	TMEM → 寄存器
5. 尾声处理与写回	<code>epilogue</code> (寄存器级 <code>cast</code> /逐元素)	寄存器 → GMEM

这套设计的来龙去脉，一句话就能讲透：**寄存器想做大，物理上难、成本上也贵，与其硬磕，Blackwell 干脆另加一块专给 Tensor Core 用的二维便笺内存 (TMEM)，把「装大累加器」这副担子从寄存器肩上卸下来，扔给了 TMEM。**代价就是：本来编译器替你打理的累加器存储，现在得内核自己动手去分配、寻址、搬运、释放。可回报也实打实——不用牺牲寄存器，就能把 tile 开得更大、Tensor Core 跑得更满。一句话：多干一点手工活，换来更高的性能。

小结

读到这儿，把全章浓缩成五句话带走：

- **TMEM 是 Blackwell 这代 GPU 才有、专给 `tcgen05` 矩阵乘指令用的片上二维内存** (每个 SM 一块,128 行 Lane × 最多 512 列 Col, 每列 32 位)。它来到世上就一个目的：**把”大 tile 撑爆寄存器”这份压力卸掉**——让累加器不必一直霸占着寄存器。
- 它靠**二维坐标 (Lane, Col)** 找位置 (在 TIRx 布局记号里写作 `TLane / TCol`), 不是普通内存那种”第几个字节”。它是 **tile 布局里堂堂正正的一员**, 不是躲在背后的后备仓库; 它的布局必须跟 `tcgen05` 指令读写它的方式严丝合缝地对上。
- 它得跟共享内存一样**你自己写代码显式分配、显式释放** (以 CTA / 线程块为单位、容量按 32 列向上取整、申请后返回一个基址), 还得把它算进内核那本有限的资源预算里。
- 数据进出 TMEM 只有三条**异步专用通路**:`tcgen05.ld`(从 TMEM 搬回寄存器, 是 `epilogue` 的主路径, 一个 warpgroup 拆成四个 warp 分头干, lane 1 对应 TMEM 第 1/4 行)、`tcgen05.st`(从寄存器写回 TMEM)、`tcgen05.cp`(从共享内存批量拷贝, 常用来搬缩放因子)。三条各等各的 (分别用 `wait::ld`、`wait::st`、`commit group + mbarrier`), 跨线程交接数据时还得补一道栅栏保证顺序。
- 还有一条特别值钱的「前后一致」：**累加器虽然搬进了 TMEM, 但 `tcgen05.ld` 把它取回寄存器之后, 片段的样子和 Ampere / Hopper 那两代一模一样, 所以 `epilogue` 那套”类型转换 + 写回”的老代码, 几乎能原封不动接着用, 迁移成本极低。**

延伸阅读

- 原文:[Special Memory: TMEM —Modern GPU Programming for ML](#) Sys
- 相关章节 (原书内部引用):
 - Data Layout and Its Notation(布局记号 `TLane/TCol` 的定义)
 - Tensor Core Operand Layouts Across GPU Generations(各代 Tensor Core 操作数/片段布局的连续性)
 - Async Coordination: mbarriers(异步完成与 `mbarrier` 协调机制)

术语对照

中文	English	说明
----	---------	----

张量内存	TMEM(Tensor Memory)	Blackwell 专属、tcgen05 专用的片上二维内存
张量核心	Tensor Core	执行矩阵乘累加的专用硬件单元
累加器	accumulator	MMA 矩阵乘累加的结果存储
寄存器片段	register fragment	累加器/操作数在每个线程寄存器里的那一份
线程束	warp	32 个线程组成的调度单位
warp 组	warpgroup	4 个 warp 组成的更大调度/协作单位
线程块	CTA(Cooperative Thread Array)	协作线程阵列, 即一个 thread block
共享内存	SMEM(Shared Memory)	CTA 内共享的片上内存
全局内存	GMEM(Global Memory)	设备主存
矩阵乘累加	MMA / GEMM	Matrix-Multiply-Accumulate / 通用矩阵乘
张量内存加速器	TMA(Tensor Memory Accelerator)	负责 GMEM↔SMEM 异步大块搬运的硬件
分块缩放 MMA	block-scaled MMA	带分块缩放因子的低精度 MMA
尾声阶段	epilogue	MMA 后做 cast/逐元素/写回的阶段
占用率	occupancy	SM 上并发活跃 warp/CTA 的比例
内存栅栏	fence	保证跨线程写入可见顺序的同步原语
异步屏障	mbarrier	用于异步操作完成协调的屏障对象

第 07 章 · 异步协调:mbarrier

原文:Async Coordination: mbarriers

本章要点

本章要点 (TL;DR)

- 先解释两个名字,后面会反复出现。**TMA** 是 GPU 上一个专门搬数据的硬件小帮手 (后面 7.1 会细讲);**Tensor Core** 是 GPU 上专门算矩阵乘法的硬件单元 (深度学习里算得最多的就是矩阵乘)。
- **TMA 和 Tensor Core 都是异步的。**”异步”这个词你写后端可能熟:就是”我把活儿交出去,但不在原地等它做完,先去干别的”。在这里就是——发起线程 (GPU 上同时跑成千上万个执行流,每个叫一个”线程”)发出一条指令,把活儿往硬件帮手手里一塞,扭头就接着往下跑;真正的搬数据、算矩阵在后台跟你的程序**同时**进行。所以别再靠”代码写在前面”就一口咬定数据备好了。
- **mbarrier 就是那个”完成信号”**:它把五花八门的异步交接,捏成一个特别简单的套路——干活的人 **arrive**(到达),用数据的人 **wait**(等待)。
- 一个 **mbarrier** 内部记两笔账:**到达计数器** (还差几个人报到),还有专门给 TMA 用的**字节计数** (还差多少字节没搬完)。两笔账都归零,这一轮才算结。
- 每个屏障还带一个**相位位 / phase bit**(phase 就是”第几轮””第几相”的意思),每过一轮翻一下。就靠它,同一个屏障才能在循环里反复用,而不会把”上一轮做完了”误当成”这一轮做完了”。
- 流水线 GEMM(GEMM 就是矩阵乘法,深度学习里算得最多的核心操作;”流水线”就是像工厂流水线那样,让搬数据和算数据一前一后叠着跑)里,屏障要几个,看的是**可复用的流水级 (stage, 流水线上的一个工位)**有几个,跟 K 维一共有多少个 tile 没关系。每个 stage 配一个屏障,再加上寄存器里的一个相位值,就齐活了。

前置知识

前置知识:读这一章前,最好先懂**异步搬运 (TMA)**、**warp/线程**、**共享内存 (SMEM)**,以及**流水线 / 重叠 (overlap)**这几个概念。没把握的话,先翻一下第 0 章·极简入门。本章会默认你已经认识这些词。

7.1 为什么需要一个显式的”完成信号”

想搞懂 **mbarrier**, 我们得先弄明白:它到底在替我们擦什么屁股?

一句话先理解

一句话先理解: GPU 里有几个”专职硬件帮手”，干活特别快但都是异步的（交了活就不等）；异步带来速度，也带来一个新问题——后面的代码可能在数据还没到位时就抢跑。这一节就是讲这个问题。

先补一点最基础的背景，后面才好讲。GPU 的内存是**分好几层的**，跟 CPU 那套”寄存器 - 缓存 - 内存”很像，只不过分得更细、而且很多层要程序员手动管。这一章会碰到这几层，先混个脸熟：

- **全局内存 (global memory):** GPU 的”主存”，容量大但访问慢。可以类比作服务器的内存。
- **共享内存 (SMEM, shared memory):** 一小块片上（就在计算单元旁边）的高速内存，速度快得多但很小。你可以把它想成”一块由程序员手动管理的高速缓存”——CPU 的缓存是硬件自动帮你填的，这块得你自己写代码往里搬数据。一组协作的线程（GPU 把线程按组管理，一组叫一个 block）共用同一块 SMEM。

为什么要分层、为什么要费劲往 SMEM 搬？因为算得快没用，数据喂不上来照样卡。把要反复用的数据先搬到又近又快的 SMEM，计算单元取数据才不会被慢吞吞的全局内存拖死。这就是下面这两个”帮手”存在的理由。

现代 GPU（尤其是 Hopper / Blackwell 这一代，Hopper 和 Blackwell 是英伟达较新的两代数据中心 GPU 架构代号，可以理解成”第几代显卡”）上，最累的两类活儿都是**异步**干的：

- **TMA / Tensor Memory Accelerator:** 一个专门搬数据的硬件引擎。它的活儿就是在全局内存和共享内存之间搬大块的张量数据。这里的”张量 (tensor)”你就理解成多维数组 / 大矩阵；一个 **tile** 是从大矩阵上切下来的一个小方块（矩阵太大，放不进小小的 SMEM，只能切成小块一块一块搬）。有了 TMA，搬数据这件苦力活就不用让计算线程去干了，腾出线程专心算。
- **Tensor Core 上的 tcgen05 MMA:** **Tensor Core** 是 GPU 里专做矩阵乘法的硬件单元（普通运算单元也能算矩阵乘，但 Tensor Core 快几个数量级，是深度学习提速的关键）；**MMA** 是 Matrix-Multiply-Accumulate（矩阵乘 - 累加）的缩写，也就是”算一段矩阵乘法，把结果累加进一个累加器里”；**tcgen05** 是 Blackwell 这代上这套 MMA 指令的名字。它算完会把累加结果写进 **TMEM (Tensor Memory)**——Blackwell 上专门留给 Tensor Core 用的一块片上内存。

”异步”在这里到底啥意思？说白了就是：**kernel**（kernel 指的是在 GPU 上跑的那段程序，相当于 GPU 版的”函数”）发出一条 TMA load（load 就是”把数据从全局内存读进 SMEM”这个动作）之后，发它的那个线程**根本不会停下来等它搬完**。它把活儿往 TMA 引擎手里一塞，扭头自己接着往下跑。真正的搬运，是在后台跟你程序的后半段**同时**进行的。MMA 也是一个道理：发出去就走，Tensor Core 在后台慢慢算。

这么干的好处一目了然：**搬数据和算数据能叠在一起跑 (overlap)**，谁也不敢误谁——想把 GPU 的算力榨干，全靠这个。可天下没有免费的午餐，它也带来一个绕不开的大麻烦：

关键

关键: 异步说白了就是——”代码写在前面”不代表”事情已经做完”。前面那条异步操作还没收尾，后面的代码就可能抢跑了。换句话说，**你再也不能靠”谁写在前面”来判断数据准没准备好**。

不加约束的话，马上就会冒出三种经典的数据竞争 (race)：

(”数据竞争 / race”是并发编程里的老概念：两件事没排好先后，谁先谁后全看运气，结果时错时错。GPU 上这个问题被异步放大了。)

出错场景	后果
TMA 还在往 SMEM tile 写数据,MMA 就开始读这块 tile	MMA 读到 残缺的数据 (搬了一半的半成品)
Tensor Core 还没把累加器 (accumulator, 就是”边乘边累加结果”的那块存储) 写完, 尾声阶段 (epilogue) 就去读 TMEM	epilogue(epilogue 指 kernel 最后的”收尾”那段代码, 把算好的结果搬回去) 读到 错误的值
kernel 等在了一个永远不会满足的条件上	kernel 永远卡住, 推进不下去 (死锁)

那怎么办? 办法很朴素: **每有一处异步交接 (handoff), 就在那儿插一个明确的完成信号, 让 kernel 知道”这步真做完了”**。mbarrier 就是这个信号。它的玩法特别统一:

```
(producer):           ->  barrier   "arrive"(  )
(consumer):           ->  barrier   "wait"(  )
```

就这一套”到达 / 等待”, 能同时管三种交接:

- 1. **TMA → MMA**: 这块 tile 搬好了没?
- 2. **MMA → epilogue**: 累加器算完了没, 能读了没?
- 3. **跨流水级复用缓冲区**: 这块 buffer 腾出来了没, 能拿去覆写了没?

注意

注意:mbarrier 可不是那种”用一次就扔”的一次性标志位。它身上带一个相位位, 每完成一轮就翻一下。正是靠它, 同一个屏障才能在几百次循环里反复复用, 而不会把”上一轮做完了”当成”这一轮做完了”。

7.2 mbarrier 到底是什么

一句话先理解

一句话先理解:mbarrier 就是 SMEM 里的一个小账本, 记着”这一轮还差谁、到第几轮了”;干活的人记一笔”我完事了”, 用数据的人先翻账本确认”齐活了”再动手。

mbarrier 是 memory barrier(内存屏障) 的缩写。”屏障 / barrier” 是并发里的一个经典概念: 一道关卡, 大家都到齐了才放行。你就把它想成一个**摆在共享内存 (SMEM) 里的小账本**, 专门记同步状态。它本身只是 SMEM 里的一小块数据, 但 GPU 有专门的硬件指令去读写它, 所以又快又靠谱。这本账上主要有两栏:

- **到达计数器 / arrival counter**: 这一轮还差几个人没报到。
- **相位位 / phase bit**: 现在轮到第几轮了 (用 0、1 来回切)。

结构图

分组:

- **mbarrier(位于 SMEM)**: 「到达计数器 / arrival counter / (还差几个到达)」 「相位位 / phase bit / (0 / 1 交替)」

原书这里有个交互演示: 能看到 `mbarrier` 的到达计数器、相位位, 还能点 `init / arrive / wait` 三个操作, 挨个字段单独看。这份静态笔记用上面那张图做了等价表达, 但想体会动态过程, 还是去原书的在线演示玩一玩最直观。

7.2.1 init: 初始化

屏障这辈子从 `init(initialize, 初始化)` 开始——就是用之前先把账本设好初始值。这一步, `kernel` 只要交代清楚一件事: **这一轮总共要等几个人报到** (也就是期望到达数)。初始化一完成:

- 屏障停在 **相位 0**;
- 计数器被设成你说的那个期望到达数。

从这一刻起, 屏障就开始“点名等人”了。它得等所有该报到的生产者、用资源的人都喊一声“我干完了”, 这一轮才算凑齐。

7.2.2 arrive: 到达 (三条不同的路径)

“到达 / `arrive`”就是干活的人在账本上记一笔“我这边完事了”, 于是计数器减一, 屏障还要等的活儿就少一点。但有个点你得记住: **不同角色“到达”的姿势是不一样的**——这个区别特别关键。原书一共讲了三条到达路径, 下面挨个看。

路径一: TMA 的 `tx-count` 到达 (字节预算)

一句话先理解

一句话先理解: 搬数据光“线程喊一声开始”还不够, 得等字节真的搬到位。所以 TMA 这条路除了报到, 还额外记一笔“这次要搬多少字节”, 搬够了才算数。

说到 TMA load, 它常用的报到方式是 **`tx-count` 到达** (`tx` 是 `transaction`“事务”的缩写, 这里你就理解成“这次传输涉及多少字节”)。代码长这样:

```
//          :expect_tx
mbarrier.arrive.expect_tx(bytes);
```

这是一条 GPU 指令 (不是普通 C++ 函数); `mbarrier.arrive` 表示“在这个屏障上报到”, `.expect_tx(bytes)` 表示“预告这次要传 `bytes` 个字节”。别看就一行, 它一口气干了两件事:

1. 替发起线程在屏障上报一次到;
2. 顺手告诉屏障: 这趟 TMA 引擎大概要搬多少字节 (也就是先记一笔字节预算)。

关键

关键: 光是“发起线程报了到”, 屏障还不算完成。它还得等 TMA 引擎把这笔字节账“还清”——真把那么多字节搬到位才行。所以相位要翻转, 得同时满足两条: 普通到达计数归零, 而且待传输的字节计数也归零。

这也是为什么不能把 `expect_tx` 想成简单的“又报了一次到”。它真正的活儿, 是给这次异步拷贝先挂一笔字节预算; 之后 TMA 引擎每搬完一批数据, 硬件就自动靠 `complete-tx` (`complete`

transaction,”这批字节传完了”)把对应的字节数从预算里扣掉。一直扣到字节归零、到达也归零,这个屏障才算彻底交差。打个比方:expect_tx 像点外卖时写下”一共 10 份”,complete_tx 像快递员每送到一份就划掉一份,全划完了你才算收齐。

时序图 (按时间顺序)

1. 发起线程 → mbarrier:arrive.expect_tx(bytes)
2. 备注: 到达计数 -1 / 同时登记 byte 预算 = bytes
3. 发起线程 → TMA 引擎: 发起异步拷贝 (立即返回)
4. TMA 引擎 → mbarrier:complete_tx(每搬完一批就扣减字节)
5. 备注: 当到达计数 ==0 且字节计数 ==0 / 相位翻转,wait 才放行

路径二:Tensor Core 的 commit 到达

换到 Tensor Core 这边,报到的方式就完全是另一套了。记牢一句话:你发一条 tcgen05 MMA,它自己是不会去碰任何屏障的。也就是说,Tensor Core 算完矩阵乘,默认不会主动通知任何人。你得手动把一次屏障到达,挂到 MMA 的”提交(commit)”路径上(commit 在这里指”把这一批攒着的 MMA 工作正式提交给硬件去算”),比如:

```
//          : barrier          MMA    commit
tcgen05.commit.mbarrier::arrive;
```

这条指令的意思是:”把这批 MMA 提交出去,并且约定——等它们全算完,就替我去这个 mbarrier 上报个到。”挂好之后,等这一批提交出去的 MMA 全算完,Tensor Core 就会自动替你去屏障上报到。

注意

注意:要是 kernel 忘了挂这个 commit 到达,那等在这个屏障上的消费者就会傻等一辈子,永远等不到。这是写 tcgen05 特别容易栽的一个坑。

路径三:普通线程的直接到达

普通线程也能自己直接去屏障上报到。这一般用在两种场合:

- 这个普通线程本身就扮演生产者;
- 一伙线程想宣布:我们把某个资源用完了。

举个最常见的例子:某个消费者把一块 SMEM 缓冲区读完之后,可以在屏障上”到达”一下,意思就是告诉生产者——这块 buffer 我腾出来了,你拿去复用吧。

7.2.3 wait: 等待 (消费者这一侧)

”消费者(consumer)”就是要用这份数据 / 资源的那一方;对应地,干活产出数据的那一方叫”生产者(producer)”——这两个词后面会一直用,就是经典的生产者 - 消费者模型。wait(等待)是一套协议里消费者要干的事。消费者会死等,直到屏障转到”这一轮该到的那个相位”(相位是啥下一节细讲,现在你就理解成”轮次对上了”),然后才敢去读数据、或者复用这个屏障护着的资源。

把上面三条到达路径凑一块看，本章最要紧的一个洞见就浮出来了：

关键

关键：异步硬件不光是”自己闷头跑在程序前面”，它还会**反过来通过屏障，把‘我干完了’回报给程序**。TMA 报的是”SMEM tile 准备好了”；Tensor Core 报的是”TMEM 结果算好了”；普通线程报的是”这块 buffer 我不占了”。情况五花八门，最后全被收成同一个形状：**生产者 arrive，消费者 wait**。

7.3 相位追踪 (Phase Tracking)

7.3.1 为什么必须有相位

一句话先理解

一句话先理解：同一个屏障要在循环里反复用上百次，怎么分清”这是新一轮做完了”还是”上一轮的旧状态”？靠一个会来回翻的标记位——相位。

屏障**一般不会用一次就重新分配一个**。先解释下这里的循环：矩阵乘法会沿着一个叫 **K 维**的方向，把大矩阵切成很多个 tile，一个一个 tile 地算、累加起来 (K 维就是两个相乘矩阵”对齐相消”的那个维度，具体细节这章不用深究)。做成”流水线 (pipelined)”是指：一边搬下一个 tile，一边算当前 tile，叠着跑省时间。这样一个 K 维循环，可能要把”搬一块 - 算一块”这种同一种交接重复**几百遍**。每次迭代都新建一个 SMEM 屏障？既不现实，也太浪费 (SMEM 本来就小)。所以真实做法是：**kernel 只擦着固定的一小撮屏障，随着循环往前推，翻来覆去地复用它们**。

那复用怎么才不出乱子？全靠**相位位**。

屏障每凑齐当前这一轮的全部到达，就把**相位翻一下**：0 变 1, 1 变 0，循环往复 (就像个布尔开关)。**wait** 这边则会盯着看：**消费者期望的那个相位，跟现在对上了没**。这个期望相位，kernel 自己存在**寄存器里**——寄存器 (register) 你可以类比 CPU 寄存器：离计算单元最近、访问最快的一小撮存储；只不过在 GPU 上，**每个线程都有自己私有的一份寄存器**，互不干扰。某个 stage 等到一轮之后，kernel 在拿它跑下一轮之前，会**顺手把本地存的相位值也翻一下**，好跟屏障下一轮的相位对得上。

结构图

分组：

- 同一个被复用的 **barrier**：「迭代 0 / 等待相位 0 / 完成后翻转到 1」「迭代 1 / 等待相位 1 / 完成后翻转到 0」「迭代 2 / 等待相位 0 / ...」「迭代 3 / 等待相位 1 / ...」

原书这里又是个**交互演示**：一个屏障跨好几次流水迭代被反复用，每完成一轮，你就能看到相位位翻一下。上图是它的静态版，**想看动起来的样子，配合原书交互演示一起看**。

7.3.2 没有相位会出什么乱子

设想一个屏障刚给一次 TMA load 用过，而且已经完成了。下一轮循环又拿**同一个屏障来复用，可偏偏没追踪相位**——这下消费者一瞅，发现屏障是”完成”状态，就乐呵呵地以为”新的 load 已经就绪了”，冲上去读数据。可它看到的其实是**上一轮遗留的完成状态**，新数据压根还没搬过来呢。

相位位干的就是把这两轮**隔开**这件事。下面这张表是**最简单的情形**：只有一个屏障，每次迭代都用它，所以期望相位每轮都翻一次。

迭代	期望相位
迭代 0	相位 0
迭代 1	相位 1
迭代 2	相位 0
迭代 3	相位 1
...	0/1 交替继续

注意

注意:“每迭代翻一次相位”是有前提的——前提是只有一个屏障在那儿连轴复用。下一节你就会看到，一旦有好几个 **stage**、好几个**屏障轮班**上, 某个屏障得隔好几次迭代才轮到一回。所以**单盯着某一个屏障**, 它的相位翻得没那么勤——千万别把这两种情形的相位序列混为一谈。

7.3.3 真实流水线里的记账: 按 stage 来记

真实流水线里, 这本账一般是一个 **stage 一本 (per stage)** 地记。再强调一下 stage 是啥: 为了让”搬”和”算”叠着跑,kernel 会准备好几块 SMEM 缓冲区轮流用——TMA 往第 2 块搬的同时,Tensor Core 在算第 1 块。每块这样的缓冲区 (连同配套的屏障) 就是一个 stage, 相当于流水线上的一个工位。kernel 手里攥着:

- 固定几个 **SMEM 流水级 (stage)**;
- 跟 stage 一对一配好的固定几个屏障;
- 寄存器里存的一小撮**相位值**。

这里要分清两个词: **逻辑迭代**指你代码里 for 循环跑的第几次 (0、1、2、3……, 可能上百次); **物理 stage** 指实际用的是哪块缓冲区 (就那么 2、3 块, 轮流用)。循环往前推的时候, **每个逻辑迭代都会被派到某个物理 stage 上** (比如 2 级流水时: 第 0 次用 stage 0, 第 1 次用 stage 1, 第 2 次又回到 stage 0……)。相位值干的活儿, 就是告诉 **wait**: 你这会儿等的, 到底是这个物理屏障的哪一轮——是它第一次被用, 还是第二次回来用。

:	0	1	2	3	4	5	...
stage:	s0	s1	s0	s1	s0	s1	(2)
:	p0	p0	p1	p1	p0	p0	(stage)

关键

关键: 这一下就讲通了后面那个问题: 为什么”用 TMA 做流水线化 GEMM”的代码, **用不着给每个 K tile 都配一个屏障?** 因为它只要每个可复用 **stage** 配一个屏障, 再搭上相位追踪, 就够使了。

- **stage** 索引管的是: 这回该用哪块 SMEM 缓冲区、哪个屏障;
- **相位值**管的是: 把”这个 stage 这一次用”和”上一次用”区分开。

试一试 (让你的 agent 陪你练): 丢给它一个两级流水线, 让它追四次迭代。每次迭代都把这几样列出来:stage 索引、本地相位值、屏障啥时候翻转, 还有——要是复用 stage 之前忘了翻相位, 会捅出什么娄子。

7.4 同步规则 (Synchronization Rules)

把屏障和相位这套机制吃透之后, 你会发现 Tensor Core kernel 里的同步其实很有规律, 基本就是“照章办事”。这套章程一句话就能交代清楚:

只要一条执行路径生产了数据、或者腾出了某个资源, 而另一条路径接下来要用, 那这次交接就必须白纸黑字写出来。

原书把它分成三种常见情形。

情形一: 线程代码 → 异步引擎

假设线程往 SMEM 写了点东西, 紧跟着一条 TMA store(store 就是 load 的反向: 把 SMEM 里的数据写回全局内存) 或 MMA 就要来读这块 SMEM。这里有个反直觉的地方: 在 GPU 这种大规模并行硬件上, 一个线程刚写下的值, 别的执行单元不一定马上就能看见 (可能还卡在某层缓存里没刷出去)。所以 kernel 得保证: 线程写进去的内容, 在引擎读之前已经是看得见的。想做到这点, 得靠合适的线程级同步, 或者一道栅栏 (fence)——fence 是一条“挡一下”的指令, 意思是“在这条线之前写的东西, 必须先让大家都看见, 才准往后走”。

至于具体用哪条指令, 要看这次交接的作用域 (scope, 指“要让多大范围内的执行单元看见”: 一个 block 内? 还是更大?), 但道理从头到尾就一个:

注意

注意: 生产线程没写完之前, 引擎绝对不许看到那块 SMEM 缓冲区。

情形二:TMA → MMA

TMA load 是异步往一块 SMEM tile 里灌数据的。就算看到”TMA 指令已经发出去了”,MMA 也别想当然地认为 tile 备好了。所以:

- TMA 操作必须绑一个 mbarrier;
- MMA 动这块 tile 之前, 必须先在那个屏障上 wait。

情形三:MMA → epilogue

tcgen05 MMA 是异步把结果写进 TMEM 的。Tensor Core 没把活儿干完,epilogue 就别急着去读累加器。所以:

- MMA 的 commit 路径要在一个完成屏障上 arrive;
- epilogue 读 TMEM 之前, 先在那个屏障上 wait。

结构图

分组:

- 情形二:TMA 到 MMA:「TMA load / (异步填充 SMEM tile)」 「mbarrier」 「MMA 读取 tile」
- 情形三:MMA 到 epilogue:「tcgen05 MMA / (异步写 TMEM)」 「mbarrier」 「epilogue 读取 TMEM」

连接关系:

- mbarrier $\xrightarrow{\text{wait 通过后}}$ MMA 读取 tile
- mbarrier $\xrightarrow{\text{wait 通过后}}$ epilogue 读取 TMEM

原书这里又有个交互演示: 一个 TMA load 通过 **mbarrier** 发出完成信号,MMA 路径等屏障放行了才去读 SMEM tile。Tensor Core $\rightarrow$ epilogue 那次交接长得一模一样, 唯一的区别是: 那边的到达, 是 **Tensor Core** 的 **commit** 路径来报的, 不是 TMA。想看它动起来, 去翻原书的交互演示。

7.4.1 同一套机制, 也能拿来”回收资源”

前面屏障都在回答”数据备好了没”; 但同一套 **arrive/wait**, 反过来也能回答”地方腾出来了没”。为什么需要这个? 因为 SMEM 缓冲区就那么几块, 要循环复用——你想把新数据搬进某块缓冲区, 前提是上一拨用它的人已经读完、不再需要它了, 否则就会把别人还在用的数据覆盖掉。所以屏障一样能发”资源腾出来了”的信号:

- 一个 SMEM stage, 旧 tile 的所有消费者还没用完之前, 不许覆写;
- 一块 TMEM 区域, 上一个占用的人还没读写完之前, 不许复用。

到了这两种场合, **arrive** 和 **wait** 的意思就换了个讲法:

- **arrive** = ”这资源我用完了”;
- **wait** = ”好了, 这资源现在可以放心交给下一个 stage 了”。

关键

关键: 这才是读流水线 GEMM kernel 里那堆同步语句的正确打开方式——那一片 **wait** 和 **arrive** 绝不是到处乱撒的防御代码。每一条都对着一次明明白白的所有权交接 (**ownership transfer**): 一块 tile 备好了、一个累加器能读了、或者一块 buffer 能回收了。把这些交接点一个一个认出来, 再绕的控制流也就读顺了。

小结

- TMA 和 Tensor Core 的异步让搬数据和算数据能叠着跑, 代价是”代码先后顺序”再也证明不了数据就绪了。所以每一处异步交接, 都得配一个明确的完成信号。
- **mbarrier** 是个摆在 SMEM 里的硬件同步对象, 主要记两笔账: 到达计数器和相位位; 碰上 TMA, 还多记一笔字节计数 (**tx-count**)。完成的条件是: 到达归零 而且 字节归零。

- 三条到达路径各玩各的:TMA 用 `expect_tx`(顺手设字节预算)、Tensor Core 用 `tcgen05.commit.mbarrier` 了挂就死等)、普通线程可以直接 `arrive`(常拿来发”资源腾出来了”的信号)。
- 相位位让同一个屏障在循环里能放心复用: 每过一轮翻一次, 消费者把期望相位存在寄存器里、每轮也翻一下, 就不会把旧的完成当成新的完成。
- 真实流水线按 **stage 记账**:stage 有几个, 屏障就几个, 跟 K tile 一共多少个没关系;**stage 索引** 管选缓冲区和屏障, 相位值管区分这一轮和上一轮。
- 把所有 `wait` / `arrive` 都当成**所有权交接**来读 (数据就绪 / 累加器可读 / buffer 可回收), 再复杂的 kernel, 控制流也读得明明白白。

延伸阅读

- 原文:[Async Coordination: mbarriers —Modern GPU Programming for ML](#)Sys(含本章三处交互演示:mbarrier 状态视图、相位翻转、TMA→MMA 完成信号)
- 相关章节:**Async Data Movement: TMA**(异步数据搬运)、**Tensor Cores: tcgen05**(张量核心)、**Pipelining GEMM with TMA**(用 TMA 做流水线化 GEMM)——想搞懂最后那章里”为啥屏障数 = stage 数”, 得先把本章啃下来。

术语对照

中文	English
内存屏障 / 异步屏障	mbarrier (memory barrier)
到达计数器	arrival counter
相位位	phase bit
字节计数 / 事务计数	tx-count / byte count
期望字节预算	expect_tx
完成事务回报	complete-tx
到达 / 等待	arrive / wait
提交 (MMA 工作组)	commit
异步数据搬运引擎	TMA (Tensor Memory Accelerator)
张量核心	Tensor Core
张量内存	TMEM (Tensor Memory)
共享内存	SMEM (shared memory)
矩阵乘累加	MMA (Matrix-Multiply-Accumulate)
尾声阶段	epilogue
流水级	stage
程序顺序	program order
访存与计算重叠	overlap
所有权转移	ownership transfer

第 08 章 · 进阶：集群启动控制 (CLC)

原文: [Advanced: Cluster Launch Control](#)

本章要点

本章要点 (TL;DR)

别急，这一章里有一堆 GPU 专有名词，下面这几条先囫圇看一眼有个印象就行，正文里我会把每个词都掰开揉碎讲明白。

- 先认识几个最基础的词。**GPU 干活的最小调度单位叫 CTA(也叫“线程块” / thread block)**——你可以把它想成“一小队一起被派去干活的线程”。GPU 上真正执行计算的硬件核心叫 **SM(流式多处理器)**，一颗 GPU 上有几十上百个 SM，CTA 就是被丢到这些 SM 上去跑的。我们要算的大矩阵会被切成一个个小方块，每个小方块叫一个 **tile(分块)**。
- **常驻核 / persistent kernel**: 传统做法是“有多少个 tile 就开多少个 CTA，一个 CTA 算一块 tile，算完就退出”。常驻核换了个思路——只开一小批“活得久、不退出”的 CTA(或集群)，让它们像工人一样循环不停地一块接一块地吞 tile。于是核心问题就只剩一句：**手上这块算完了，下一块从哪里来?**(这里的“集群 / cluster”是指一起启动、还能互相同步配合的一组 CTA，后面会细讲。)
- **集群启动控制 / Cluster Launch Control(CLC)**: 这是 Blackwell(NVIDIA 比较新的一代 GPU 架构代号，排在上一代 Hopper 之后) 新加的一个硬件功能。它能让一个已经在跑的集群，在运行过程中向 GPU 内部一个叫**网格调度器 / grid scheduler**(负责派活儿的硬件模块) 去“讨”一份还没开始干的活儿。说白了，它就是一条硬件级的“抢活儿”通道——抢活儿这个套路在计算机里有个专门叫法，叫**工作窃取 (work-stealing)**: 谁先干完手头的，谁就去把别人还没来得及干的活儿抢过来自己干。
- CLC 用起来很简单，对外就给**两条机器指令**(这种贴近硬件的底层指令在 NVIDIA 这里叫 PTX 指令): 一条用来异步地发出请求，一条用来把结果读回来。“请求完成了没有”这个信号，靠一个叫 **mbarrier**(内存屏障) 的东西来通知——它是一种“异步操作干完后用来打信号、喊一嗓子‘我好了’”的同步小工具。这套报信机制是直接借用 **TMA**(GPU 里专门负责异步搬数据的硬件引擎) 早就在用的那一套，**没有另外发明任何新的等待方式**，你学一套就够用。
- 它最大的好处是**改善“尾部行为”(tail behavior)**——“尾部”指的是一批活儿快干完、收尾的那一小段时间。当各块 tile 算得有快有慢、或者 tile 的总数没法被 SM 数整除分得不匀时，先干完的集群马上就能再去拉一份新活儿，而不用在旁边干瞪眼空等别人。
- 最后提醒一句别搞混:“集群”这个概念从 Hopper 那一代起就已经是 GPU 的一种**启动单位**了(不是 CLC 发明的);CLC 是 Blackwell 才补上的新本事，它让这些**集群被派到哪儿去算，从“开跑前就钉死”变成了“跑起来再动态决定”**。

前置知识

前置知识: 这一章会反复用到 CTA(线程块)、cluster(集群)、tile(分块)、SM(干活的硬件核心)、关键路径与尾部、常驻核 (persistent kernel) 这几个概念。上面 TL;DR 里都已经初步解释过, 正文里还会再展开, 所以你不~~需要~~提前精通它们; 但如果你想先有个更系统的底子, 可以翻一下第 0 章·极简入门。读不懂某个词时, 回到本章前面找一下它第一次出现的地方, 那里一定有大白话解释。

8.1 从静态调度到动态调度: 为什么需要 CLC

一句话先理解

一句话先理解: GPU 算一个大矩阵, 得把它切成很多小块 (tile) 分头算。问题在于”谁来算下一块”。传统做法是开跑前就排死; CLC 让 GPU 跑起来再现场动态分配, 从而避免有人累死、有人闲死。

想搞懂 CLC, 得先把**常驻核**这个执行模式弄明白。这套玩法, 前面《用 warp 特化与集群扩展 GEMM》那一章是一步步搭起来的 (GEMM = 通用矩阵乘法, 是深度学习里出现最频繁、最吃算力的运算, 所以它是 GPU 编程里当之无愧的主角)。

先看传统 GPU kernel(kernel 就是”一段在 GPU 上跑的程序”, 类比你写的一个函数) 是怎么干活的。GPU 一次启动的全部 CTA, 合起来叫一个**网格 / grid**(你就理解成”这一次任务派出去的所有工人的总集合”)。传统 kernel 把这事当成一锤子买卖: 有多少个 tile 要算, 就一口气启动同样多的 CTA, 一个 CTA 负责包一块 tile, 算完就退出走人。

常驻核 (persistent = ”常驻、不退出”) 不这么玩, 它换了个思路:

- 它不按 tile 数量去开 CTA, 而是只开一小批**活得久、算完一块也不退出**的 CTA(或集群)。这批工人的数量, 大致调到”每个 SM 上差不多蹲一个干活的”这个量级就好——但说清楚: 它**并不要求** SM 和工人严格一对一, 不需要这种死板的保证, 大概齐够用即可。
- 每个集群算完一块 tile, 不退出, 而是接着去算下一块, 算完再下一块, 就这么循环转圈, 一直转到整个矩阵的所有 tile 都算完为止。

为什么要费这个劲搞”常驻”? 因为反复启动、退出 CTA 本身是有开销的; 让一小批工人常驻下来不停干, 就省掉了这部分反复开关的成本。但这么一改, 核一旦变成常驻的, 整个调度问题就被压成了一句话:

关键

关键: 一个集群算完当前 tile 之后, 下一个 tile 从哪里来?

静态公式调度: 简单但尾部会拉胯

最省事的答案是**静态公式 / static formula**。”静态”的意思是: 每个工人该算哪几块 tile, 在程序还没开跑之前就用一个固定公式算死了, 跑的时候不再变。怎么算? 每个 CTA 都有自己的编号 (cta_id, 就像工号), 拿这个编号直接定出它要算的第一块 tile; 算完一块往后跳, 每次都按一个固定的步子往后挪——这个固定步子叫**网格步长 / grid stride**, 它的大小正好等于工人的总数。

为什么步子要等于工人总数？这样能保证每个工人都不重不漏地分到”间隔着的”那些 tile。下面这段伪代码就是这个意思，我逐行讲：

```
tile_id = cta_id           #           ,   "           tile"
while tile_id < num_tiles: #           ,
    compute_tile(tile_id)  #           tile
    tile_id += grid_size   #   "           "           ,           tile
```

举个具体例子：假设有 4 个工人、12 块 tile。0 号工人就算第 0、4、8 块,1 号工人算第 1、5、9 块……每人各算 3 块，正好分完。

这招好不好用，得看情况：

- 写起来简单。哪个 CTA 该算哪几块 tile，在 kernel 真正开跑之前就**全排明白了**，不需要任何运行时协调。
- 要是每块 tile 算起来都差不多累（耗时相近），而且 tile 总数又恰好能被 SM 数整除（刚好分得匀），那它跑得挺香。
- 可问题就出在「活儿在干干之前就钉死了」这件事上。万一有几块 tile 算得特别慢，或者最后剩下的几块怎么也分不匀，就会撞上一个很尴尬的场面：**一些 SM 早早把自己那份干完了，却因为活儿早就钉死、不能去帮别人，只能在旁边干瞪眼；另一些还在那儿吭哧吭哧啃尾巴**。整批任务的总耗时，就被这几个最慢的拖到了最后——这就是前面说的”尾部”被拖长了。

CLC: 把分配推迟到运行时再决定

CLC 就是来收拾这个烂摊子的。它只改了一件事：不再开跑前就把整张排班表钉死，而是让一个正在跑的常驻集群，跑着跑着、在运行时（也就是程序已经在 GPU 上跑起来的过程中）向**硬件里的网格调度器**现要一份活儿——要的是”本来还排着队、但还没真正启动的另一个集群”该干的那份。

换个角度想：静态公式是”上班第一天就把全年排班表贴墙上，谁也别改”；CLC 是”谁手头干完了，就到调度台前现场领下一单”。后者显然更能让大家都不闲着。

结构图

分组：

- **分组**：「CTA 0: tile 0,4,8...」「早干完 → 空等」「CTA 1: tile 1,5,9...」「还在啃尾巴」
- **CLC 动态调度（运行时讨要）**：「集群干完手上 tile」「向调度器 / try_cancel」「接管该坐标继续算」「退出循环」

连接关系：

- 向调度器 / try_cancel $\xrightarrow{\text{成功: 拿到坐标}}$ 接管该坐标继续算
- 向调度器 / try_cancel $\xrightarrow{\text{失败: 队列空了}}$ 退出循环
- 接管该坐标继续算 → 集群干完手上 tile

这一要，结果无非两种：

- **要到了** → 当前集群就”接管”那个集群坐标，掉头去算它对应的 tile。（后面会讲到，图里那个 try_cancel 就是”我来要活儿”这条请求指令的名字，别被它字面吓到，下一节细说。）
- **要不到** → 说明排队的活儿已经被领光了，没活儿可抢了，循环到这儿就收工退出。

注意

注意:CLC 和”线程块集群”根本是两码事,千万别混。线程块集群 (thread block cluster) 说的是”一起启动、能做集群级同步、还能互相访问对方共享内存的那么一组 CTA”这个概念本身,它是 Hopper 那一代带来的(见《GPU 执行模型》)。而 CLC 则是 Blackwell 才加上的本事,它管的不是”集群是什么”,而是”这些集群该被派去算哪块、怎么调度”——让这件事从死的变成活的。换句话说,集群一直就是 GPU 的启动单位;CLC 干的只是让一个正在跑的集群,去把另一次”还没真正发生的启动”取消掉,然后把那份坐标接过来自己干。

这里有个心智模型特别关键,先记牢,它能帮你看懂后面所有协议细节:CLC 不是那种”从一个软件维护的任务队列里取出一个抽象任务给你”的机制。它干的事更贴近硬件、也更实在——它是把一次本来排着队、还没发生的集群启动给取消掉 (英文叫 **cancel a pending cluster launch**),然后把那个被取消的集群本来该用的坐标,转手交给现在来要活儿的这个集群。记住这一句,后面那一堆看着绕的协议规则,追到根上全是从它长出来的。

8.2 两条指令: 请求与查询

一句话先理解

一句话先理解:CLC 用起来一共就两件事——”发一个请求去要活儿”和”过一会儿来查结果”,对应两类底层指令。下面会逐条解释每个怪名字到底在干嘛。

前面提过,CLC 这套机器指令 (PTX 指令) 很底层,名字也长得吓人,但别怕,逻辑极简单。它对外就给你**两条指令**:一条用来异步地发出请求 (指令名里带 **try_cancel**,意思是”尝试取消一次待启动来抢活儿”),一条用来把响应结果读回来 (指令名里带 **query_cancel**,意思是”查询那次取消的结果”)。

这里有个小地方得提一句:读响应的 **query_cancel** 实际又拆成两个变体,一个用来查”这次到底抢没抢到”,另一个用来把坐标取出来。所以下面你会看到 3 个具体的指令名。别被”3”这个数字吓着——往大了说,要干的还是”发请求”和”查结果”这两件事而已。

请求指令:clusterlaunchcontrol.try_cancel.async

先看这条”发请求”的指令。下面表格里有几个 GPU 专有词,我在表格后面统一给你讲透,你先扫一眼有个轮廓:

维度	说明
语义	请求调度器取消一个待定集群的启动,并把那个集群的坐标返回给调用方
响应位置	结果会被写进 共享内存 ,是一条 16 字节 的小记录
同步性	异步 ——指令一发出就立刻返回,不会卡在原地等响应到达
完成信号	通过 mbarrier 来通知,沿用和 TMA 一模一样的 barrier + phase (屏障 + 相位)模型;”响应到了”这件事,靠 barrier 的字节数完成 (byte-count completion) 机制来宣布

把上面几个词逐个讲明白:

- ```
// :
mbarrier_wait(clc_barrier, phase) // 1) ,
if query_cancel.is_canceled(response): // 2) : ?
 (x, y, z) = query_cancel.get_first_ctaid(response) // 3) ,
 next_tile = decode_tile_coord(x, y, z) // " tile"
else:
 break // 4) -> ,
```

逐行说人话: 第 1 行先在屏障上等响应真的到了 (否则读到的是空的); 第 2 行用 `is_canceled` 问”抢到没”; 第 3 行只有抢到时才执行, 把三维坐标取出来; 紧接着换算成具体 `tile`; `else` 分支则表示没活儿了, 直接结束循环。

#### 注意

**注意:** 这个协议里没有用”哨兵值 / `sentinel`” 这种约定俗成的魔法数字来表示”没活儿了”。(解释一下: 很多程序里习惯用某个特殊值, 比如 `-1`, 来表示”到头了 / 没有了”, 这种特殊值就叫”哨兵值”。) CLC 没走这条路, 它纯粹是靠那个真/假谓词来分支: 谓词为 `true`, 坐标就有效; 谓词为 `false`, 工作窃取循环就到头。

为什么不用哨兵值、偏要用谓词? 还得绕回 CLC 的本质: 硬件不是从一个软件队列里给你发个抽象任务, 而是在取消一次还没发生的集群启动。既然是这样, ”成功的响应” 天然就捎带着一个货真价实的集群坐标; 而”失败的响应” 无非就是在说”启动队列已经被掏空了”。一个真/假就把这两种情况说清楚了, 根本用不着再额外约定什么魔法数字。这么一想, 全都顺了。

## 8.3 工作窃取循环

#### 一句话先理解

**一句话先理解:** 把”发请求”和”算当前块”在时间上重叠起来, 这样等你算完手头这块, 下一块的归属早就查清楚了, 几乎不用等。这就是 CLC 性能好的全部秘密。

有了前面那两条指令, 整个常驻调度器 (就是那段决定”下一块算谁”的逻辑) 就缩成了一个很短的循环。而这个循环里最妙的一笔, 不在于用了什么高深指令, 而在于那条”发请求”的指令到底摆在循环的哪个位置。先卖个关子, 往下慢慢看, 你会发现位置摆对了, 性能就白捡了。

#### 循环的结构

##### 时序图 (按时间顺序)

1. 备注: 手上有「当前 `tile`」要算
2. 常驻集群 → 网格调度器: 1. 为「下一个 `tile`」发 `try_cancel` (异步)
3. 备注: 2. 在请求飞行途中, 先算「当前 `tile`」
4. 网格调度器 → `mbarrier`: 响应就绪 → 字节数完成
5. 常驻集群 → `mbarrier`: 3. 算完后, `wait` 响应 `barrier`
6. 常驻集群 → 常驻集群: 4. `query is_canceled?`
7. 常驻集群 → 常驻集群: 5a. `get_first_ctaid` → 解码坐标 → 作为下一个 `tile`, 继续
8. 常驻集群 → 常驻集群: 5b. 没活儿了 → 退出

用大白话列出来, 就这五步:

1. 先为”可能还有的下一块 `tile`”发一发 `try_cancel` (把要活儿的请求扔出去)。
2. 趁这个请求还在路上飞着 (`in flight`, 意思是”已经发出、但还没回来”的那段时间), 别干等, 赶紧把当前这块 `tile` 算了。
3. 当前这块算完了, 再回到屏障上 `wait` 一下那个响应。

4. 查一下刚才那次抢活儿到底成没成。
5. 成了，就拿返回的坐标接着算下一块；没成，就退出循环。

为什么要”先请求，再计算”？

关键

关键：集群绝不会傻等着当前 tile 算完了、才去要下一份活儿。它的套路反过来——先把请求发出去，再去算当前这块。这么一安排，”向调度器发请求并等它回话”和”埋头算当前这块 tile”这两件事就重叠 (overlap) 到一块儿同时进行了。(overlap 是 GPU 编程里一个核心思路：让”等待”和”计算”在时间上叠在一起、互相遮掩，从而把等待的时间”藏”进计算里。) 结果就是：等你把当前 tile 吭哧吭哧算完，下一块的归属答案多半早就回来、在那儿候着了，你几乎不用专门干等。

这个道理，跟常驻核在别的地方用异步拷贝 (TMA)、用 Tensor Core(GPU 里专门做矩阵乘法的加速单元，深度学习的算力主力) 的 barrier 时完全是同一个意思，核心就一句话：别让又慢又长的”高延迟操作”直愣愣地堵在关键路径上。(高延迟操作 = 发出后要等很久才有结果的操作，比如跨芯片搬数据、向调度器要活儿；关键路径 / critical path = 决定整体快慢、怎么也省不掉的那条最长的依赖链路。东西堵在关键路径上，整体就会被它拖慢。)CLC 无非是把这套早就验证过的老办法，挪到了”tile 调度”这件事上——早早地把下一份活儿要上，手头先把当前这份算了，真到要用结果的时候再回头去取。

下面拿一张时间轴表来帮你看清这个重叠 (表格做了简化)：每一行是一个并行进行的角色，每一列是时间往前走，从 t0 到 t3。

| 角色 \ 时间   | t0                   | t1       | t2           | t3                          |
|-----------|----------------------|----------|--------------|-----------------------------|
| 集群 (关键路径) | 发 try_cancel(异步，不阻塞) | 算当前 tile | 算当前 tile     | wait barrier(此时多半已就绪，几乎不阻塞) |
| 调度器 (后台)  | 收到请求                 | 在后台找下一块  | 响应就绪 → 字节数完成 | —                           |

看明白了吧：发请求和算当前 tile 在 t1-t2 完全叠在一起，所以等集群算完、走到 t3 去 wait 的时候，响应早就准备好了，基本不卡。

### 8.4 与常驻 GEMM 的关系

《用 warp 特化与集群扩展 GEMM》那一章 (GEMM = 通用矩阵乘法，前面说过，是深度学习里最核心、最吃算力的运算)，主线讲解时用的是静态调度器。为啥用它？理由很实在：静态调度器好讲、好懂——下一块 tile 是哪块，拿循环里的当前状态套个公式就算出来了，不涉及任何运行时的协调。比如那章里有个叫 ClusterPersistentScheduler2D 的调度器 (名字直译就是”二维集群常驻调度器”)，就是按前面讲的网格步长那套路子，在输出 tile 的二维空间上一块一块往下分的。

而 CLC，就是这套静态分配机制的”动态版替身”——功能定位完全一样 (都是回答”下一块算谁”)，只是把决策时机从开跑前挪到了运行时。这儿要重点强调一句，这也是工程上最值钱的一点：外层那个循环一个字都不用改——每个常驻集群干的还是那套老活儿”算一块输出 tile，再往下挪一块”。真正变的，就只有’下一块 tile 从哪来’这一件事：

| 维度                   | 静态调度器              | CLC 动态调度   |
|----------------------|--------------------|------------|
| 下一个 tile 来源          | 公式算出 (grid-stride) | 硬件工作窃取返回   |
| 决策时机                 | 启动时一次性定死           | 运行时按需分配    |
| tile 计算体 (tile body) | 相同的常驻 GEMM 主体      | 完全相同       |
| 尾部不均时                | 早干完的 SM 空等         | 早干完的集群继续拉活 |
| tile 成本不均时           | 假设开跑前的分配「足够好」      | 不需要这个假设    |

在两种场景下 CLC 优势最明显

1) 启动的尾巴 (tail of the launch) ”尾巴”指的就是整批活儿快收尾的那一小段时间。在静态排班下,这一段剩的活儿往往七零八落、分布不均:有些 SM 早把分给自己的 tile 全啃光了,有些却还压着好几块没动。换成 CLC,先干完的那个集群不会闲着,它直接掉头去要”另一个还没启动的待集群”的坐标——只要启动队列里还剩着活儿,先收工的人就能不停地拉新 tile 接着干,把原本该闲着的时间也利用起来了。整批任务的尾巴因此被显著缩短。

2)tile 一块比一块累 (各块耗时不均) 为什么会出现 tile 算起来有快有慢? 因为有些 GEMM tile 走的代码路径跟别人不一样:可能是碰上了矩阵的边界 (boundaries, 边角上那些不满一整块的部分要特殊处理)、用了掩码 (masking, 跳过某些不需要算的位置)、用了稀疏 (sparsity, 矩阵里大量是 0、可以省着算)、用了分组调度 (grouped scheduling, 把若干小矩阵乘打包在一起),也可能是围着主矩阵乘顺手做了点融合 (fused, 把别的运算合并进来一起算)。这些情况都会让某些 tile 明显比别的慢。静态排班的麻烦就在这:它必须假设——在程序还没真跑、压根看不出这些耗时差异的时候,它分的活儿就已经分得”足够均匀”了。这显然是个很冒险的假设。而 CLC 压根不需要这个假设——它根本不预先猜谁快谁慢,而是等某个集群真的闲下来了,才现场再给它派一块新的,谁有空谁多干,天然就把负载均衡了。

在 TIRx 里怎么暴露

(TIRx 是这本书所用编译框架里的一层中间表示 / 编程接口,你可以先粗略理解成”上层写 GPU 程序时面对的那套抽象”,不必深究细节。)

就冲上面这些好处,在 TIRx 里可以把 CLC 包装成一个动态 tile 调度器给上层用。对写程序的人来讲,最爽的是:算 tile 的那部分代码根本不用动——tile 主体还是静态调度器用的那套常驻 GEMM 主体,一行不改。变的只有调度器内部,它从原来的

”按公式算出我的下一块 tile 坐标”

悄悄换成了

”向硬件去要下一个可用的集群坐标”。

到头来的效果是:外面看还是那个常驻循环,只不过”分活儿”的法子,从”启动时就钉死的死排班”,换成了”硬件实时说了算的活调度”。一处小改动,白捡一个硬件级的动态负载均衡。

小结

- 常驻核 + CLC 这一套组合,把 GPU 上”下一块 tile 算谁”的调度决策,从”编译/启动期就钉死”往后挪到了”运行时由硬件现场派”。它要治的病很清楚:活儿分得不均,导致收尾时一帮工人干瞪眼空转、白拖慢整体。

- CLC 的实现相当克制: 对外就**两条底层指令**, 外加**白嫖现成的 mbarrier 完成报信机制**。它没有另起炉灶发明什么新的等待原语, 而是把”异步发请求 + 在 barrier 上等响应”这套早在 TMA、Tensor Core 上验证过的老范式, 原封不动搬到了调度上——你学一套, 处处通用。
- 它的协议是**靠真/假谓词驱动的**, 而不是靠哨兵魔法值。为啥? 还是那句根本逻辑——CLC 的本质是”取消一次还没发生的集群启动”, 不是”从软件队列里取一个任务”。所以抢到了就直接给你一个真坐标, 没抢到就说明队列已空, 一个真/假足矣。
- 它**设计上最漂亮的一手**, 是”先请求、后计算”这个循环顺序: 把”等调度器回话”的那点延迟, 悄悄塞进了”算当前 tile”的时间里一起消化掉, 不让它单独堵在关键路径上。这跟整套异步 GPU 编程的核心思路一脉相承——能藏起来的等待, 就别让它露在外面拖后腿。
- 从工程角度看, CLC 对上层 (比如 TIRx) **几乎是零打扰**: 算 tile 的代码一个字不动, 只要把调度器从”套公式算下一块”换成”问硬件要下一块”, 就白捡一个硬件级的动态负载均衡。

**一句话:**CLC = 给常驻核接上一条**硬件级的”抢活儿”(工作窃取) 通道**, 让先干完的集群在运行时把”还没启动的集群”那份坐标抢过来自己算。这样一来, 当各块 tile 算得有快有慢、或数量分不均时, 收尾阶段的性能就能明显好上一截——因为再也没有人闲着干瞪眼了。

延伸阅读

- 本章原文:[Advanced: Cluster Launch Control —Modern GPU Programming for MLSys](#)
- 相关章节 (同书):
  - 《GPU 执行模型》——线程块集群、分布式共享内存的来历 (Hopper)
  - 《异步协调:mbarriers》——barrier + phase + 字节数完成模型, CLC 的完成信号复用了它
  - 《异步内存:TMA》——同样的「异步请求 + barrier 等待」范式
  - 《用 warp 特化与集群扩展 GEMM》——常驻 GEMM 与静态 ClusterPersistentScheduler2D 的来龙去脉

术语对照

| 中文          | English / 缩写                 |
|-------------|------------------------------|
| 集群启动控制      | Cluster Launch Control (CLC) |
| 常驻核         | persistent kernel            |
| 线程块集群       | thread block cluster         |
| 工作窃取        | work-stealing                |
| 网格调度器       | grid scheduler               |
| 网格步长        | grid stride                  |
| 静态调度器       | static scheduler             |
| 动态 tile 调度器 | dynamic tile scheduler       |
| 尾部行为        | tail behavior                |
| 关键路径        | critical path                |
| 谓词          | predicate                    |
| 哨兵值         | sentinel                     |
| 内存屏障        | mbarrier                     |
| 相位 (位)      | phase (bit)                  |
| 字节数完成       | byte-count completion        |

|        |             |
|--------|-------------|
| 输出分块   | output tile |
| 线程块    | CTA         |
| 矩阵乘    | GEMM / MMA  |
| 张量内存访问 | TMA         |

# 第 09 章 · TIRx 入门

原文:[Introduction to TIRx](#)

## 本章要点

### 本章要点 (TL;DR)

别急，下面这几句里有不少新词，你现在看不懂没关系——正文里每一个都会当场用大白话讲清楚。这里先让你对「这章要干嘛」有个轮廓：

- 先说「kernel」是啥：在 GPU 编程里，我们把「跑在 GPU 上的那段函数」叫一个 **kernel(核函数)**。你可以把它理解成「一段会被成千上万个线程同时执行的代码」。本章从头到尾就在研究怎么写一个 kernel。
- **TIRx(张量 IR neXt / Tensor IR neXt)** 是一种「嵌在 Python 里的小语言」(术语叫 DSL, 领域专用语言)，专门用来写 GPU kernel。它特别在哪？一方面，它让你能**直接指挥硬件的每一个细节** (具体哪些细节，正文会一个个讲)；另一方面，你写下的这些指挥命令，不是散落在一堆难懂的代码里，而是被整理成一种编译器**看得懂、能加工**的结构化形式 (这种形式就叫 **IR, 中间表示**，后面会解释)。
- 一个 kernel 到底怎么跑，关键就那么几件事。TIRx 把它们**全摆到台面上**，归成三大设计要素，你把这三个词记住，这章就抓住一半了：**作用域 (scope)**——这活儿归谁干；**布局 (layout)**——数据块放在哪、怎么摆；**派发 (dispatch)**——具体走哪条硬件路去执行。
- 整章就盯着一个**能真跑起来的、最小的矩阵乘程序** (术语叫 GEMM, 通用矩阵乘)：算一个  $128 \times 128$  大小的结果方块，公式是  $D = A \cdot B^T$  ( $A$  乘以  $B$  的转置)，而且整个矩阵乘核心就用一句 `Tx.gemm_async` 写完。
- 别看这程序小，上面那三大要素它一个不落全演示到了。说白了，全书后面的章节，做的就是「把这三个要素放大到更大规模」这一件事——所以这一章打好底，后面就顺了。
- 编译 (把你写的代码变成 GPU 能跑的机器指令) 就一句 `tvm.compile(mod, target="cuda", tir_pipeline="tirx")`。整个过程**对你不藏一点东西**：你既能看到中间的结构化表示 (IR)，也能看到最后生成的底层 CUDA 代码。

## 前置知识

**前置知识：**这一章会用到一些 GPU 的基础概念。我会在正文里第一次出现时当场解释，但如果你想先有个整体印象，可以先翻一下第 0 章 · 极简入门。涉及的概念主要是：GPU 上线程是怎么分层组织的、内存为什么要分成好几层、还有「Tensor Core」这种专做矩阵乘的硬件大概是干嘛的。完全没接触过也别怕，跟着读就行。

## 一、为什么需要 TIRx: 把「隐藏的决策」翻到台面上

### 一句话先理解

**一句话先理解:** GPU 这东西算得快, 但你得用一种「它听得懂」的方式去指挥它; 现有的指挥方式 (裸写 CUDA) 能干活, 但有个毛病——「这个程序到底打算怎么跑」这件事, 看代码根本看不出来。TIRx 就是来治这个毛病的。

前面第一部分 (Part I), 我们把硬件是什么讲透了。可光认识硬件还不行——你想让它真干活, 总得有个写代码的法子才行。

先把两个绕不开的名词说清楚。CUDA 是 NVIDIA 给自家 GPU 配的那套编程语言/工具 (你可以粗略理解成「写 GPU 程序的 C++ 方言」); PTX 则更底层一点, 是 GPU 的一种「类汇编」中间指令 (比 CUDA 更贴近硬件, 更难读)。这两样, 就是当下指挥 GPU 最直接的两种手段。

最直接的法子, 就是裸写 CUDA 或者 PTX。事实上, 不少追求极致性能的 kernel 就是这么一行行手搓出来的。那它差在哪? 差在: 决定这个 kernel 怎么跑的那几件关键事, 在裸代码里你几乎看不出来。不信你看下面这三个问题, 它们正是写 GPU 程序时最要命的几个决定, 可裸代码里偏偏最看不清:

- **谁在干活:** GPU 上同时有成千上万个线程 (线程 = 一条独立的执行流, 后面会细讲), 某个操作到底是哪些线程在执行?
- **数据放哪:** GPU 的内存不是一整块, 而是分了好几层 (快但小、慢但大, 后面会讲为什么)。你的每一块数据, 究竟放在哪一层?
- **走哪条路:** 同一件事 (比如「搬一块数据」), GPU 上常常有好几种硬件实现方式可选。这条计算实际走的是哪条硬件路径? 是让普通线程一个个搬? 还是用 TMA (一个专门负责「异步、批量搬数据」的硬件引擎, 你可以理解成 GPU 里的搬运专用机器人)? 还是交给 Tensor Core (一块专门负责矩阵乘法的硬件电路, 算矩阵乘特别快)?

裸写 CUDA/PTX 的时候, 上面这几个选择都藏到哪去了呢? 埋在某个函数一长串看不懂的参数里, 埋在程序员手算出来的内存地址里, 还有一堆「圈内人都这么写、但代码里没明说」的约定俗成里。于是就成了这样: 别人 (甚至几个月后的你自己) 来读这段代码, 基本得靠「考古」, 才能猜出当初到底想干嘛。

那 TIRx 是怎么破这个局的? 注意, 它**没有**走「把硬件细节全藏起来、让你写得轻松」那条路——那是 PyTorch 这类高层框架的玩法 (省心, 但你也别想精细控制硬件)。TIRx 偏要反着来: 它**照样**让你直接指挥硬件的每个细节, 但同时把上面那三类关键决策, 从「埋在代码里靠猜」改成了「明明白白写在 IR 上」。换句话说, 既要底层的控制力, 又要看得清。

### 关键

**关键:** 一句话说清 TIRx 的思路——底层控制力一点不丢, 同时还让编译器看得懂。这里多解释一个词: **编译器**就是那个「把你写的代码翻译成 GPU 能执行的机器指令」的程序。道理很简单: 你做的那些决策只要明着写出来, 编译器就能拿它去做三件事——**检查 (check)** 你写得对不对、**下降 (lower)** (把你写的高层意图一步步翻译成底层真指令, 这个翻译过程在编译界就叫「lowering / 下降」)、还有**调度 (schedule)** (安排指令以什么顺序、怎么并行地跑)。

这三类决策, TIRx 给起了三个名字, 后面整本书都会反复用到。现在记不住没关系, 正文里每个都会陪着代码讲透:



| 设计要素          | 它回答的问题                                 | 在代码里体现为                                                          |
|---------------|----------------------------------------|------------------------------------------------------------------|
| 作用域 (scope)   | 谁 (哪些线程) 来执行这个操作?                      | CTA(一个线程块)/ warpgroup(4 个 warp = 128 线程)/ 单个选出来的线程等 (这些层级正文会逐个讲) |
| 布局 (layout)   | 操作数 tile(就是「大矩阵切出来的小方块」) 放在哪里、按什么方式排布? | SMEM(一种片上共享内存) 的特殊排布、TMEM 的 TLane/TCol 排布、寄存器视图等                 |
| 派发 (dispatch) | 走哪条硬件路径来执行?                            | dispatch="tcgen05"、拷贝走普通线程还是走 TMA                                |

这三件事, 本章不想干巴巴地讲概念。咱换个法子: 直接搬一个完整、能跑的 kernel 出来看。先让它跑起来, 再回头一行一行读, 看看 scope、layout、dispatch 分别是怎么把这段代码捏成现在这样的, 以及它最后又是怎么编出来的。

注意

注意: 本章就盯着「一个 kernel + 三个要素」, 故意把焦距收得很窄。这个 kernel 用到的张量布局模型, 留到后面《TIRx 布局 API》专门讲; 完整的语言特性, 放在《TIRx 语言参考》里。

二、运行环境与依赖

想真把本章的例子跑起来, 你得有一块 Blackwell 架构的 GPU(目标是 sm\_100a, 比如 B200)。TIRx 编译器是跟着 Apache TVM 一起发的, 就在 wheel 包里的 tvm.tirx 模块, 安装的时候记得配上 CUDA 版的 PyTorch:

```
pip install apache-tvm
```

装完跑一行命令, 确认能正常 import 就行:

```
python -c "import tvm, tvm.tirx; print(tvm.__version__)"
```

注意

注意: 这套环境配好, 全书的例子都能跑。手头没有 Blackwell GPU 也别慌——代码是跑不了, 但你完全可以当「阅读理解」来读。本章真正值钱的地方, 本来就在于搞懂这一个 kernel 是怎么对应到 scope、layout、dispatch 上的。

三、第一个 kernel: 单次 MMA 的 GEMM

一句话先理解

一句话先理解: 我们要写的这个程序, 就是算一次矩阵乘法  $D = A \cdot B^T$ 。它被砍到了不能再小, 但「麻雀虽小五脏俱全」——前面说的三大要素它全用上了。

先说说为什么矩阵乘 (GEMM) 是主角。你可能会问: 讲 GPU 怎么老拿矩阵乘举例? 因为深度学习里绝大部分的计算量 (神经网络的每一层、Transformer 的注意力) 归根到底都是在做大矩阵相

乘,GPU 的硬件也是专门为它优化的。所以学会写好一个矩阵乘 kernel, 基本就摸到 GPU 编程的命门了。

咱要看的这个 kernel, 是把 GEMM(通用矩阵乘, 这里算的是  $D = A \cdot B^T$ , 也就是 A 乘 B 的转置) 砍到不能再砍的最小版——可即便这么小, 它还是刚好能驱动起一块 Tensor Core(前面说过, 就是那块专做矩阵乘的硬件)。规格很清楚:

- 计算单个  $128 \times 128$  大小的输出方块 (我们把这种「从大矩阵里切出来的小方块」叫一个 **tile**);
- 公式为  $D = A \cdot B^T$ , 其中  $K = 64$ (K 是参与相乘的那个「中间维度」的长度, 稍后用到);
- 整段矩阵乘计算从头到尾, 核心就用一个 `Tx.gemm_async` 这样的 tile 操作来表达。

### 3.1 「一个 tile 操作 $\neq$ 一条硬件指令」

这一点想明白了, 你就抓住 TIRx 一半的价值了。`Tx.gemm_async` 看着就是「一句话」, 可它并不会变成一条硬件指令——它背后会被翻译成好几条。为啥会这样? 咱算笔账 (先解释两个词):

- 硬件做矩阵乘时, 有个最核心的操作叫 **MMA(Matrix Multiply-Accumulate, 矩阵乘累加)**, 就是 Tensor Core 干的那件事: 把两小块矩阵相乘, 结果直接累加到一个地方。
- 但 Tensor Core 这块硬件一次只能「咬」固定大小的一小口。具体来说, 它在 K 方向上一次只能处理 16 那么长——这个「一次最多处理 16」就是硬件的最小粒度, 术语叫 **K-atom(K 方向的原子粒度) = 16**。「原子」在这里就是「不可再分的最小单位」的意思。
- 而我们这个 tile 的  $K = 64$ , 比硬件一口能吃的 16 大。所以编译器会自动把它拆成一小串、沿着 K 方向一步步走的 `tcgen05.mma` 指令 (`tcgen05` 就是 Blackwell 这代 GPU 上 Tensor Core 指令的名字;  $64 \div 16 = 4$ , 所以拆成 4 步)。

#### 关键

**关键:**DSL 值钱的地方就在这儿——你写的是「一块数据 (tile) 要做什么」, 而不是「一条条硬件指令」。具体怎么沿着 K 方向把它拆成 4 步、怎么生成正确的 `tcgen05.mma` 指令序列, 那全是编译器的活, 你这个写代码的人根本不用操心。这就像写 Python 时你不用管循环到底编译成哪几条机器指令一样。

光发一个 MMA 还不够, 一个完整的 kernel 周边还得干一圈杂活。先把几层内存的名字认一下 (GPU 内存为什么分这么多层, 3.3 里会细讲, 这里先混个脸熟): **GMEM(Global Memory, 全局内存)** 是 GPU 上最大、但也最慢的那块主显存; **SMEM(Shared Memory, 共享内存)** 是一小块在芯片上、速度快很多的内存; **TMEM(Tensor Memory, 张量内存)** 是 Blackwell 这代 GPU 新增的、专门给 Tensor Core 做累加用的存储; **寄存器 (RF, Register File)** 则是每个线程私有的、最快的那一丁点存储。

顺着数据流走一遍这圈杂活就清楚了: 先把 SMEM 和 TMEM 分配好 (占好地方), 把 A、B 两个输入矩阵从 GMEM 搬进更快的 SMEM, 接着发起那个 tile MMA、让结果在 **TMEM 累加器 (accumulator)**, 就是「边算边把结果累加进去」的那块存储) 里算出来, 算完再经过寄存器把结果读回来, 最后写回 GMEM 输出。

这 kernel 虽小, 它可是后面《构建分块 GEMM》那条优化升级路上的 **第 1 步 (Step 1)**。到那一章, 我们会带着完整讲解再回来找它。

### 3.2 起手式: 统一的 import

每个 TIRx kernel 开头那几行 import 都大同小异, 先扫一眼混个脸熟:

```
import tvm
from tvm.script import tirx as T # T:TIRx (device_entry Buffer ptx)
from tvm.script.tirx import tile as Tx # Tx:tile (cta.copy gemm_async wg.copy_async ê)
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode #
↪ TMA/swizzle
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg # DSL: TLane/TCol
↪ tid_in_wg
```

这几行你不用记, 扫一眼知道有这么回事就行。记住它们各管一摊:T 管「最底层的语言/硬件」——比如声明 kernel 入口、声明数据缓冲区 (buffer)、直接发底层 PTX 指令这些;Tx 管「tile 操作」——比如拷贝一块数据、做一次矩阵乘;tvm.tirx.layout 则是一套专门用来「描述数据怎么摆放」的小工具。后面代码里看到 T.xxx 和 Tx.xxx, 就知道它俩分别来自哪、管什么层级了。

提示

提示: 上面那行 from tvm.script.tirx import tile as Tx 里的 as Tx, 就是 Python 的「起别名」语法, 把一个很长的名字简写成 Tx。所以后文满屏的 Tx.xxx, 本质就是在调用这套 tile 操作库里的函数。

### 3.3 kernel 的整体骨架

整个 kernel 被装进一个小函数 hgemv\_v1(M, N, K) 里。它的作用类似一个「生成器」: 你喂它三个数字 (矩阵的规模 M、N、K), 它就吐给你一个写好的 kernel(在 TVM 里这个 kernel 对象叫 PrimFunc, 你理解成「一个编译器能处理的函数」即可)。本章挑的规模是 M=N=128, K=64, 这个尺寸刚好只产生一个输出 tile。第一版之所以能让你「一口气读完」, 靠的就是这个小尺寸——没有循环、没有多块拼接, 就一块。

下面把它拆成几块 (从 a 到 j), 每块只摘最要紧的那几行, 逐行用大白话讲它在干嘛 (完整源码看原文)。

#### (a) 数据类型与分块常量

```
a_type = b_type = d_type = tvm.DataType("float16") # A/B/D fp16
acc_type = tvm.DataType("float32") # fp32()

BLK_M, BLK_N, BLK_K = 128, 128, 64 # CTA
MMA_M, MMA_N, MMA_K = 128, 128, 16 # : MMA tile , K-atom=16
```

先解释两个新词。fp16 / fp32 是浮点数的精度:fp16 是 16 位的「半精度」浮点数 (省空间、算得快, 但精度低一点),fp32 是 32 位的「单精度」(更准, 但更占地方)。这里 A、B、D 都用 fp16, 但累加器 (就是边算边攒结果的那块地方) 特意用了更准的 fp32——为什么? 因为矩阵乘要把很多个乘积加起来, 加的次数多了, 误差会越攒越大, 所以攒结果的地方用高精度, 能少丢精度。

再看 BLK\_M, BLK\_N, BLK\_K = 128, 128, 64 这行: 它定义的是「一个线程块负责算多大的一块」(BLK 是 block 的缩写)。

## 注意

注意:MMA\_M/N/K = 128, 128, 16 这行在这儿纯粹是「写给人看的注释」, 就为了告诉你底层硬件那个 MMA 一口能吃多大 (尤其注意 K 方向是 16, 正好对应前面说的 K-atom=16)。它们压根不会真的传进 `gemm_async` 这个函数——`gemm_async` 自己会根据你给的输入方块和累加器方块的大小, 把该用的 MMA 形状反推出来。既然根本用不上, 后面几步索性就把这几个常量删了。这也透着 TIRx 的一点小脾气: 形状只要编译器自己能推出来, 你就别手动喂。

## (b) 为 A、B 准备 swizzled SMEM 布局

```
A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_M, BLK_K))
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_N, BLK_K))
```

**布局 (layout)** 这个要素, 头一回露面就在这儿: 这两行的意思是「给 A、B 在 SMEM 里规定一种特殊的摆放方式」, 这种方式叫 **128B swizzle** 布局。

**swizzle**(混排) 是干嘛使的? 顾名思义, 就是把数据存进 SMEM 时故意打乱一下摆放的地址, 不老老实实按顺序放。为什么要自找麻烦打乱它? 两个原因:

1. **躲开 bank 冲突**。这里得先讲讲 SMEM 的结构: SMEM 在硬件上被切成了若干个并列的「存储体」(术语叫 **bank**), 好处是多个线程可以同时各访问各自的 bank、互不耽误。但如果一群线程偏偏都去挤同一个 bank, 硬件就只能让它们排队一个个来, 速度直接掉下去——这种「撞车排队」就叫 **bank 冲突**。把数据地址 **swizzle** 打乱后, 线程们的访问就能均匀散到不同 bank 上, 不撞车。
2. **满足 Tensor Core 的摆放要求**。`tcgen05.mma` 这条硬件指令对「操作数数据该怎么摆」是有硬性规定的, **swizzle** 后的布局正好对它的胃口。

具体它怎么个打乱法, 《TIRx 布局 API》那一章会细说; 眼下你只要记住一句话就够了: **这就是 `tcgen05.mma` 想要的那种布局**。

## (c) 进入 device 入口、计算线程坐标

```
@T.prim_func
def kernel(A: T.Buffer((M, K), a_type),
 B: T.Buffer((N, K), b_type),
 D: T.Buffer((M, N), d_type)):
 T.device_entry()
 bx, by = T.cta_id([M // BLK_M, N // BLK_N]) # CTA
 wg_id = T.wargroup_id([1]) # wargroup -> wg_id 0()
 warp_id = T.warp_id_in_wg([4]) # wargroup warp (0-3)
 lane_id = T.lane_id([32]) # warp lane (0-31)
```

这段是第一次出现 GPU 的「线程分层」概念, 值得停下来好好讲。

## 一句话先理解

**一句话先理解:** GPU 上同时有几万个线程在跑, 为了管得过来, 硬件把它们分成了一层套一层的小组。下面这几行代码, 就是在问「我这个线程, 在这套分层结构里坐在哪个位置?」

GPU 的线程是这样分层组织的 (从大到小):

- **grid(网格)**: 一次启动 kernel 涉及的所有线程的总集合, 最外面那一层。

- **CTA(也叫 block, 线程块)**: grid 被切成若干个 CTA。一个 CTA 是「一组能互相协作、能共享那块 SMEM」的线程。它是协作的基本单位。
- **warpgroup(warp 组)**: 一个 CTA 里又分成若干个 warpgroup, 每个 warpgroup 是 4 个 warp = 128 个线程。
- **warp(线程束)**: 一个 warpgroup 里有 4 个 warp, 每个 warp 固定是 32 个线程。**warp 是 GPU 调度的最小单位**——这 32 个线程在硬件上必须**齐步走、同一时刻执行同一条指令**。打个比方: 一个 warp 就像 32 个人排成的方阵, 口令一响所有人必须同时迈同一只脚, 谁也不能掉队、不能自己走自己的。
- **lane(线程 / 通道)**: warp 里的单个线程, 编号 0~31。lane 就是「方阵里的某一个人」。  
所以这几行代码做的, 就是让当前正在执行的线程**报出自己的坐标**:
- **T.cta\_id(...)** ——我所在的 CTA 在整个 grid 里排第几 (本例 grid 就 1×1 一个 CTA, 所以 bx, by 都是 0)。
- **T.warpgroup\_id(...)** ——我在 CTA 里属于第几个 warpgroup(本例只有 1 个, 恒为 0)。
- **T.warp\_id\_in\_wg(...)** ——我在 warpgroup 里是第几个 warp(0~3)。
- **T.lane\_id(...)** ——我在自己这个 warp 里是第几号线程 (0~31)。

#### 关键

**关键:** 这几行, 合起来就是**作用域 (scope)** 的那套「坐标系」。**cta\_id / warpgroup\_id / warp\_id\_in\_wg / lane\_id** 一层套一层, 从「整个 grid」一路定位到「某一个具体线程」。后面凡是要判断「这活儿到底归谁干」(比如写 `if warp_id == 0`, 意思是「只让 0 号 warp 干这件事」), 全靠这套坐标来说话。这就是 **scope** 在代码里的真身。

还有个安排是故意的: 第一步让 **M** 正好等于 **BLK\_M**, **N** 正好等于 **BLK\_N**, 这样整个 **grid** 就只有 **1×1 一个 CTA**, 每个 CTA 在大矩阵里的起始偏移 (**m\_st**, **n\_st**) 自然全是 0(因为就一块, 起点就是 0)。要等到后面的 Step 3, 才会把矩阵放大、grid 里塞进多个 CTA。

#### (d) SMEM 分配: 用 pool 显式排布

```
pool = T.SMEMPool() # ,
tmem_addr = pool.alloc((1,), "uint32") # TMEM
mma_bar = pool.alloc((1,), "uint64", align=8) # MMA (mbarrier), 8B
pool.move_base_to(1024) # 1024,
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout) # A SMEM, swizzle
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout) # B SMEM, swizzle
pool.commit() # ,
```

这段在干嘛? **手动安排 SMEM 里的每一块地方**。你可以把 SMEM 想象成一长条空货架, **SMEMPool** (共享内存池) 就是你手里管理这条货架的工具。这段一行行做的事是:

- **pool.alloc(...)** ——在货架上「划出一格」放某样东西。这里先划了两个小格: 一格存 **TMEM** 的基址 (**tmem\_addr**), 一格存一个「屏障」(**mma\_bar**, 马上下一段讲它是什么); **align=8** 是要求这一格的起始地址按 8 字节对齐 (某些硬件结构对地址对齐有要求)。
- **pool.move\_base\_to(1024)** ——把「接下来分配的起点」挪到第 1024 字节处, 等于给前面那些小元数据留出 1024 字节的空间, 免得后面的大块数据跟它们挤在一起。
- 再 **alloc** 出 **Asmem**、**Bsmem** 两大块, 分别放 A、B 矩阵——注意它俩在划格的同时就带上了 **layout=A\_layout/B\_layout**, 也就是上一步定义的那套 **swizzle** 布局。

- `pool.commit()` ——「拍板提交」, 整个货架的布局到此定下来。

这一段最能让你尝到 TIRx「直接指挥硬件」是个什么味儿:SMEM 不是系统背着你偷偷分好的(高层框架就是那么干的), 而是你拿着 `SMEMPool` 一格一格亲手摆出来的。还有个点要留意——`Asmem/Bsmem` 在分配的那一刻, 就绑上了上一步定义的那套 `swizzle` 布局。布局这个要素, 到这儿又落地了一回。

### (e) 屏障 + TMEM 初始化 (只让 warp 0 干)

```
if warp_id == 0: # : warp 0
 if lane_id == 0: # : lane 0
 T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # mbarrier, 1
 T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_cols=512, cta_group=1) # TMEM(512)

T.ptx.fence.proxy_async("shared::cta") # fence: SMEM
T.ptx.fence.mbarrier_init() # mbarrier fence
T.cuda.cta_sync() # CTA ,
```

这段先讲一个贯穿全章的写法: 用 `if` 把活儿派给特定线程。`if warp_id == 0:` 意思是「下面这段, 只有 0 号 warp 的线程才执行」; 再套一层 `if lane_id == 0:`, 就进一步收窄到「只有 0 号 warp 里的 0 号线程」一个人。为什么有些事只让一个线程干? 因为像「初始化一个屏障」「申请一块 TMEM」这种事, 本质上是「设置一次就够」的全局动作, 要是让 128 个线程都抢着各做一遍, 既浪费又会出错。这正是 `scope`(作用域) 在起作用——明确划定「谁来干」。

逐行看:

- `T.ptx.mbarrier.init(...)` ——初始化一个 `mbarrier`(内存屏障)。屏障是什么? 后面我们要发起的 MMA 是异步的(发出去之后, 硬件在后台慢慢算, 代码不会原地干等)。可问题来了: 后面要读结果的时候, 怎么知道「它算完了没」? `mbarrier` 就是干这个的——它像一个「完成信号灯」, 异步操作做完会去「敲」一下它, 等结果的线程则盯着它, 灯一亮就知道可以往下走了。`init(..., 1)` 里的 1 是说「我等 1 个完成信号」。
- `T.ptx.tcgen05.alloc(...)` ——申请一块 TMEM(那块给 Tensor Core 累加用的特殊存储), 这里要了 512 列。
- `T.ptx.fence.* / T.cuda.cta_sync()` ——这几行是「同步栅栏」。`cta_sync()` 的意思是「CTA 里所有线程在这里集合、都到齐了再一起往下走」。为什么需要它? 因为前面只有个别线程做了初始化, 得有这么一道「集合点」, 确保这些初始化对全体线程都已生效、可见, 大家才好接着干。`fence` 系列则是更细的内存可见性保证, 确保某些写入对后续的异步操作可见。

### 注意

**注意:** 一看到 `T.ptx.*` 这个前缀, 你就该立刻反应过来: 这是在直接发 PTX 指令 (PTX 前面讲过, 是 GPU 最贴近硬件的那层指令)。这正是 TIRx 跟高层框架彻底分家的地方——`mbarrier`、`tcgen05.alloc`、`fence` 这些最底层的硬件概念, 它一个都不替你藏, 你想用就直接写; 它只不过把这些指令包装成了「结构化、编译器看得懂」的形式罢了。

### (f) 声明 TMEM 累加器视图

```
tmem = T.decl_buffer(
 (128, 512), "float32", scope="tmem", allocated_addr=tmем_addr[0],
```

```
layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]) # ->TLane, ->TCol
)
```

这段在声明「累加器在 TMEM 里长什么样」。`T.decl_buffer(...)` 就是「声明一块数据缓冲区」, 这里声明的是  $128 \times 512$  的 fp32 数据, `scope="tmem"` 说明它住在 TMEM 里, `allocated_addr=tmem_addr[0]` 把它指向上一步申请到的那块 TMEM 地址。

重点在最后那行 `layout=TileLayout(...)`, 这一行大概是**布局**要素最有代表性的一处了。它说: 这块累加器的 128 行对应到一个叫 **TLane** 的轴, 512 列对应到一个叫 **TCol** 的轴。

这里的 **TLane/TCol** 到底是啥? 它们是 TIRx 专门给 TMEM 这种特殊存储起的名字, 叫**命名轴 (named axes)**。为什么要起新名字、不直接叫「行」「列»? 因为 TMEM 的物理结构跟普通内存不一样——它是为 Tensor Core 量身定做的, 数据在里面的排布方式跟硬件电路一一咬合。所以这里的 **TLane**(可以联想成「对应到哪条 lane / 哪个线程通道」) 和 **TCol**(对应到哪一列) 是**跟硬件布局对齐的逻辑轴**, 而不是你平时直觉里的那种行列。你暂时只要知道「这是 TMEM 累加器该有的标准摆法」就够了。

### (g) 加载: 全员协作把 GMEM 拷到 SMEM

```
m_st = T.meta_var(bx * BLK_M) # CTA (0)
n_st = T.meta_var(by * BLK_N) # CTA (0)
phase_mma: T.int32 = 0 # mbarrier (phase)

Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :]) # CTA :128
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync() #
```

这段把 A、B 从慢的 GMEM 搬进快的 SMEM。逐行看:

- `m_st / n_st` ——本 CTA 负责的那块在大矩阵里的起始行/列偏移。`T.meta_var(...)` 你理解成「定义一个编译期就能算出来的常量」即可; 本例就一块, 所以都是 0。
- `Tx.cta.copy(Asmem[:, :], A[...])` ——把 A 的一块拷进 `Asmem`。这里的 `[:, :]` 是 Python 切片, 表示「整块」。
- `T.cuda.cta_sync()` ——又一道「集合点」: 等 CTA 里所有线程都把自己负责的那部分搬完, 大家到齐了再往下走。不然有的线程数据还没到位, 后面就该算错了。

#### 关键

**关键:** 注意 `Tx.cta.copy` 里那个 `cta`——它表示这是 **CTA 作用域**的拷贝, 意思是 CTA 里 128 个线程**全员一起上**, 分头把这块数据搬完 (人多力量大, 大批量搬运就该全员上)。这是 `scope` 要素的一个典型样本: 同样是「拷贝」这件事, 作用域前缀一换 (比如换成 `wg` 就是 `warpgroup` 来搬), 干活的到底是哪一群线程也就跟着换了。

### (h) 计算: 单个 elected 线程发射 MMA

```
if warp_id == 0:
 if T.ptx.elect_sync(): # warp
 Tx.gemm_async(
 tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=False, # ()
 dispatch="tcgen05", # : Blackwell Tensor Core
```

```

 cta_group=1
)
 T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1) # ,
T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # MMA

```

这是**整段 kernel 的核心**——真正的矩阵乘就发生在这里。它也是三要素难得同框的一幕。

先解释 `T.ptx.select_sync()`：它的意思是「在当前 warp 的 32 个线程里**选举出唯一**一个当代表」，被选中的那个线程进入 `if`，其余 31 个跳过。配合外层的 `if warp_id == 0`，最终就是「全 kernel 里只有一个线程」来执行里面这段。再看里面那句 `Tx.gemm_async(...)`，这就是那句「一句话写完矩阵乘」的核心调用：把 `Asmem`、`Bsmem` 相乘，结果写进 `tmem` 累加器；`accum=False` 表示「这是第一次写，直接覆盖，不在原有基础上累加」；`dispatch="tcgen05"` 指定走 Tensor Core 这条路。最后 `T.ptx.tcgen05.commit(...)` 是「提交」，告诉硬件「算完了记得去敲一下那个屏障 `mma_bar`」。kernel 外面那行 `try_wait(...)` 则是真正在「盯着信号灯等结果算完」。

现在把三要素在这段里一个个拎出来看：

- **scope(谁来干)**：`if warp_id == 0` 再加上 `select_sync()`，意思就是由**选举出来的那一个线程**来发射这次 MMA。你可能纳闷：矩阵乘这么大的活，一个线程哪忙得过来？关键在于——它发出去的每条 `tcgen05.mma` 都是一次**协作式 (cooperative)** 的启动：真正的计算是 Tensor Core 那块硬件一整组协同完成的，软件这边根本不用 128 个线程一起喊，只要派一个线程过去「点个火」把硬件启动起来就够了。所以这里反而是「一个线程」最合适。
- **layout(数据怎么摆)**：输入 `Asmem/Bsmem` 走前面铺好的 `swizzle SMEM` 布局，累加器 `tmem` 走 `TLane/TCol` 布局——这些前面都早早安排好了，到这里直接用。
- **dispatch(走哪条路)**：`dispatch="tcgen05"`，白纸黑字选定了 Blackwell Tensor Core 这条硬件路径。

(i) 写回：**TMEM → 寄存器 → GMEM**

```

Dreg = T.alloc_local((BLK_N,), acc_type) # fp32 ()
Dreg_f16 = T.alloc_local((BLK_N,), d_type) # fp16
Dreg_wg = Dreg.view(128, BLK_N,
 layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)])) # 128 warpgroup
 ↪
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # warpgroup :128
T.ptx.tcgen05.wait.ld() # TMEM load
Tx.cast(Dreg_f16[:, :], Dreg[:, :]) # fp32 -> fp16
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id) #
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :]) # GMEM

```

结果现在还在 **TMEM** 累加器里，得把它「请出来」写回 **GMEM**。但 **TMEM** 不能直接写到 **GMEM**，中间得过一道寄存器。这段就是走「**TMEM → 寄存器 → GMEM**」这条回家路。逐行看：

- `Dreg = T.alloc_local(...)` ——申请一块寄存器 (`alloc_local` 就是「在每个线程私有的寄存器里开一小块」)，用来暂存这个线程负责的那一行 `fp32` 结果。`Dreg_f16` 是转成 `fp16` 后的版本。
- `Dreg_wg = Dreg.view(..., layout=...)` ——给寄存器套一个「视图」，关键是用 `tid_in_wg` 把「第几行」对应到「`warpgroup` 里第几号线程」。`tid_in_wg` 就是「线程在 `warpgroup` 内的编号 (0~127)」。



- `Tx.wg.copy_async(...)` ——warpgroup 的 128 个线程一起，把 TMEM 累加器读回寄存器（注意前缀 `wg` = warpgroup 作用域）。
- `T.ptx.tcgen05.wait.ld()` ——等这次读取真正完成（又一次「等异步操作做完」）。
- `Tx.cast(...)` ——把 `fp32` 结果转成 `fp16` (`cast` 就是类型转换)。
- 最后 `Tx.copy(D[m_thr, ...], ...)` ——每个线程把自己那一行写回 GMEM 的输出 `D;m_thr` 是这个线程负责的输出行号，由它的 `warp` 号和 `lane` 号算出来。

关键

关键:`Tx.wg.copy_async` 是 **warpgroup 作用域**——warpgroup 的 128 个线程把 TMEM 累加器一行一行 (**row by row**) 读回来。门道在 `Dreg_wg` 这个视图: 它靠 `tid_in_wg` 把「某一行」对应到「warpgroup 里的某个具体线程」, 于是 128 个线程**每人刚好分到一行** (术语叫一行 **row fragment**, 行片段)。这是 **layout** 和 **scope** 配合得最漂亮的一幕——**布局**回答「哪个线程负责哪一行」, **作用域**回答「这 128 个线程一块儿上」, 两者一搭, 活儿就均匀分下去了。

(j) 释放 TMEM

```
T.cuda.cta_sync()
if warp_id == 0:
 T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1) #
 T.ptx.tcgen05.dealloc(tmemb_addr[0], n_cols=512, cta_group=1) # TMEM
```

这段是收尾打扫。先 `cta_sync()` 让大家集合一下 (确保结果都写完了), 再由 0 号 warp 把之前申请的那块 TMEM 释放掉 (`relinquish_alloc_permit` 交还许可、`dealloc` 真正释放)。

为什么要专门写这几行? 因为 TMEM 这东西又小又金贵, 它是你前面在 (e) 步**手动申请**来的, 系统不会替你自动回收。所以用完必须自己手动还回去, 不然就「占着茅坑」、别人没法用了——这又是 TIRx「直接指挥硬件、连内存回收都得你自己管」这股风格的一处体现。

3.4 整体数据流示意

读到这儿, 各块代码都过了一遍, 咱用一张图把它们串成一条完整的数据流 (顺手标上每一段是谁在干活):

结构图

连接关系:

- SMEM: `Asmem` / (swizzle 128B 布局)  $\xrightarrow{\text{Tx.gemm\_async} / \text{单 elected 线程发射} / \text{dispatch=tcgen05}}$  TMEM 累加器 / (TLane/TCol 布局, `fp32`)
- SMEM: `Bsmem` / (swizzle 128B 布局)  $\rightarrow$  TMEM 累加器 / (TLane/TCol 布局, `fp32`)
- TMEM 累加器 / (TLane/TCol 布局, `fp32`)  $\xrightarrow{\text{Tx.wg.copy\_async} / \text{warpgroup 作用域} \cdot \text{逐行}}$  寄存器 `RF` / (每线程 1 行片段)
- 寄存器 `RF` / (每线程 1 行片段)  $\xrightarrow{\text{Tx.cast: fp32} \rightarrow \text{fp16}}$  寄存器 (`fp16`)
- 寄存器 (`fp16`)  $\xrightarrow{\text{Tx.copy} / \text{每线程写自己那一行}}$  GMEM: `D` (`M` $\times$ `N`, `fp16`)

**图说：**顺着这条数据流走，一共冒出来三种不一样的**作用域**——CTA 级拷贝 (128 线程全员搬,GMEM→SMEM)、单个 elected 线程发射 MMA(协作式硬件 MMA,SMEM→TMEM)、warpgroup 级读回 (128 线程逐行取累加器,TMEM→RF)。同一个 kernel 里，不同的操作各挑各的作用域，这就是 scope 要素活生生的样子。图里颜色是按存储层级的：灰 =GMEM、蓝 =SMEM、橙 =TMEM、绿 = 寄存器。

### 3.5 先跑起来：用 torch 对拍验证

在逐行细抠 kernel 之前，先干件让人踏实的事——确认它真能跑，而且算得对。怎么确认「算得对」？办法很朴素：同一道矩阵乘，我们另用一个**绝对可信**的工具 (PyTorch, 深度学习里最常用的库) 再算一遍当标准答案，然后比对两边结果一不一样。这种「拿可信实现当参照、比对结果」的做法，行话叫**对拍**。流程很短：

```
import torch

target = tvn.target.Target("cuda") # sm_100a,arch
device = torch.device('cuda')

M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
 # tir_pipeline="tirx" TIRx ; Executable
 ex = tvn.compile(tvn.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")

A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)
D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)

ex.mod(A_tensor, B_tensor, D_tensor) # torch ,

D_ref = (A_tensor.float() @ B_tensor.float().T).half() # torch
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")
```

#### 注意

**注意：**这段里有两个对开发者特别贴心的小设计。一是 **arch 自动探测**：你只写 `target="cuda"` 就够了，编译器会自己上设备上摸出 `sm_100a` 这类具体架构。二是 `ex.mod(...)` 直接吃 **torch 张量**，中间你不用手动转一道。还有一点，容差为啥给到 `rtol=2e-2`, `atol=1e-2` 这么松？因为 `fp16` 算起来本身就带数值误差，卡得太死反倒容易误判。

## 四、三要素回读:scope / layout / dispatch

kernel 跑通了，现在咱换个角度回头再读一遍，边读边问自己一个问题：它每一行，到底「拍了什么板」？说白了，整个 kernel 不过就是沿着三个设计要素做的一连串选择。每个操作都在回答同样三个问题——谁来跑、数据 住哪、怎么执行——这三个答案，恰好就是 `scope`、`layout`、`dispatch`。

关键

**关键:** 原文这块有个交互演示——你点一下 Scope / Layout / Dispatch 按钮,kernel 里归这个要素管的代码行就会高亮起来。静态笔记没法复现这个交互, 不过下面三张表已经把「每个要素管哪几行」列清楚了, 效果是一样的。

4.1 Scope: 谁来执行?

| 操作                                 | 作用域                  | 参与线程               | 为什么是这个作用域                                                                         |
|------------------------------------|----------------------|--------------------|-----------------------------------------------------------------------------------|
| <code>Tx.cta.copy(...)</code>      | <b>CTA</b>           | 全部 128 线程          | GMEM→SMEM 是大批量搬运, 全员一起上最快                                                         |
| <code>Tx.gemm_async(...)</code>    | 单个 <b>elected</b> 线程 | 1 个线程发起            | 下降出的每条 <code>tcgen05.mma</code> 本身就是协作式 MMA 启动, 真正算的是 Tensor Core 硬件, 软件派一个线程点火就够 |
| <code>Tx.wg.copy_async(...)</code> | <b>warpgroup</b>     | warpgroup 的 128 线程 | TMEM 读回是按行切的, 每个线程拿一行片段最顺                                                         |

4.2 Layout: 每块 tile 住在哪、怎么排?

| tile                         | 存储层级 | 布局                          | 说明                                               |
|------------------------------|------|-----------------------------|--------------------------------------------------|
| A、B 操作数                      | SMEM | 128B <b>swizzle</b> 布局      | <code>tcgen05.mma</code> 想要的就是这个摆法, 顺带躲开 bank 冲突 |
| 累加器                          | TMEM | <b>TLane / TCol</b> 命名轴布局   | 跟 Tensor Core 硬件的累加布局一一对应                        |
| 寄存器读回视图 <code>Dreg_wg</code> | RF   | 行映射到 <code>tid_in_wg</code> | 让 <code>warpgroup</code> 里每个线程刚好「占一行」            |

4.3 Dispatch: 走哪条硬件路径?

| 操作                                                       | dispatch 选择    | 含义                                                            |
|----------------------------------------------------------|----------------|---------------------------------------------------------------|
| <code>Tx.gemm_async(..., dispatch="tcgen05", ...)</code> | <b>tcgen05</b> | 走 Blackwell Tensor Core 这条路                                   |
| 拷贝操作 (本章)                                                | 普通线程拷贝         | 第一版就用最朴素的线程拷贝                                                 |
| 拷贝操作 (后续章节)                                              | <b>TMA</b>     | 后面的 GEMM 步骤会把拷贝换成 TMA, 可周围的 <b>scope</b> 和 <b>layout</b> 一点不动 |

关键

**关键:**dispatch 想换就换, 这是 TIRx 的一大长处。想「换条硬件路」(比如线程拷贝 → TMA)? 这是个局部、正交的小改动: 只动 dispatch, 作用域不碰, 布局也不碰。正因为「三要素互不打架」这个设计,kernel 从第一版一路升级到生产级, 才能拆成一小步一小步稳稳地走, 而不是每次都推倒重来。

**动手练 (让你的 agent 搭把手):** 从第一个 kernel 里挑三行——一行拷贝、一行 MMA、一行 TMEM 读回。让 agent 给每行贴上 scope / layout / dispatch 三个标签, 然后你对着代码里的 guard(像 if warp\_id == 0)、buffer 布局、还有 dispatch= 参数, 核一核它贴得对不对。

## 五、编译是怎么发生的

其实前面跑测试那会儿, `tvm.compile(...)` 已经悄悄帮我们编译过一次了。编译, 就是「把你写的这段高层代码翻译成 GPU 真能执行的机器指令」。这里咱凑近瞧瞧那一步背后到底干了啥。做法很短: 把写好的 kernel(PrimFunc) 装进一个 IRModule(你理解成「一个装着若干函数的编译单元」), 扔给 `tvm.compile(...)` 就完事:

```
target = tvm.target.Target("cuda")
ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")
```

`tir_pipeline="tirx"` 这个参数, 启动的就是 **TIRx 下降流水线**。它的核心 pass 是 **LowerTIRx**:

### 结构图

连接关系:

- IRModule → LowerTIRx / 按 scope/layout/dispatch 契约 / 把每个 tile 原语兑现成指令
- LowerTIRx / 按 scope/layout/dispatch 契约 / 把每个 tile 原语兑现成指令 → host/device 拆分
- host/device 拆分 → finalize
- finalize → Executable / (可直接调用)

先解释 **pass** 这个词: 编译器干活不是一蹴而就的, 而是分成一道道工序、每道工序对程序做一种特定的加工, 每一道这样的工序就叫一个 **pass**。 `tir_pipeline="tirx"` 就是选用「TIRx 这条由若干 pass 串起来的流水线」。

- **LowerTIRx** 是这条流水线里真正的主角 (名字里的 Lower 就是前面讲过的「下降/翻译成底层」)。它一个一个地啃你写的那些 tile 操作, 读懂每个操作背后的 **scope / layout / dispatch** 三套约定, 然后照着把它「兑现」成一条条真正的硬件指令。换句话说, 前面咱反复聊的那三个设计要素, 就是在这一步被翻译成真指令的——比如那句 `Tx.gemm_async`, 正是在这儿被展开成前面算过的、沿 K 方向走 4 步的 `tcgen05.mma` 序列。
- 再往后是两步常规活儿:**host/device 拆分**加一个 **finalize**, 最后产出一个能直接启动的模块。
- 顺带提一句, 你也可以把 `tvm.compile` 写在 `with target:` 块里头, 这样 kernel 会自动继承外层的 target 上下文,target 就不用再传一遍了。

### 5.1 一切都可检视

这套流程有个特别招人喜欢的脾气: 它对你不藏一点东西。编译结果你能从两个层级上看:

```
kernel.show() # TIRx(TVMScript)
print(kernel.script()) # ,
```

```
CUDA C :
print(ex.mod.imports[0].inspect_source())
```

**注意**

**注意:** 这种「往上能读 IR, 往下能读生成的 CUDA C」的透明度, 不管是调 bug 还是学东西, 都太管用了——高层的意图 (IR) 你看得到, 它最后落成啥样的底层代码, 你照样看得到。本节只是个「速写」; 完整的下降故事 (所有 pass、tile 原语的 dispatch 怎么解析、host/device 怎么拆) 留到《编译器内部》那一章再讲。

六、接下来去哪

就这么一个 kernel, 已经够咱认全 scope、layout、dispatch 了, 还顺手看着它编译、跑起来。那接下来去哪? 这三个设计要素, 加上 kernel 本身, 各自都拽出了一个后续章节:

| 后续章节        | 内容                                                                                         | 什么时候去看                            |
|-------------|--------------------------------------------------------------------------------------------|-----------------------------------|
| TIRx 布局 API | TileLayout、命名轴、swizzle 这套张量布局模型——本章 A/B/累加器怎么摆, 全建在它上面                                     | 要是你觉得 layout 是三要素里最摸不透的一个, 就从这儿入手 |
| TIRx 语言参考   | 完整的语言特性: 解析器工具、数据类型、buffer 与内存、控制流、线程同步                                                    | 当你想要一本完整的「词典」, 而不只是「导览」时          |
| 构建分块 GEMM   | 把本章 kernel 当 Step 1, 一路加上 K 循环累加、空间分块、TMA、warp 专门化 (warp specialization), 最后建成真正能用的 kernel | 想看同样这三个要素怎么扩到生产级 kernel? 这是最顺的下一站 |

小结

- **TIRx 是一种「IR 层级」的 GPU 编程 DSL:** 它特别在哪? 一边照样让你直接点名硬件 (线程、SMEM、TMEM、屏障、tcgen05 MMA), 一边又把决定 kernel 怎么跑的那几个关键决策抬到**结构化 IR** 上, 这样编译器才有得检查、有得下降、有得调度。
- 全章的认知抓手, 就是 **scope / layout / dispatch 三大设计要素**: 分别回答「谁来干」「数据住哪」「走哪条硬件路」。这三个问题抓住了, 你就有了一把通读任意 TIRx kernel 的万能钥匙。
- 一个只做一次 MMA 的最小 GEMM, 就把三要素一次性演全了:Tx.cta.copy(CTA 作用域加载)→ Tx.gemm\_async(dispatch="tcgen05")(单线程发射、走 tcgen05 路径)→ Tx.wg.copy\_async(warp 逐行读回)。
- **核心思路**是「写 tile, 不写指令»:Tx.gemm\_async 就一句,LowerTIRx 会沿 K-atom=16 把它展成好几条 tcgen05.mma——这些你一条都不用手写。
- **三要素互不打架**, 正是 TIRx 好演进的根子: 后面想把线程拷贝换成 TMA, 只动 dispatch 就行,scope 和 layout 一个都不用动。
- 编译流程 tvml.compile(..., tir\_pipeline="tirx") **全程透明**:IR 能看 (.show() / .script()), 生成的 CUDA C 也能看 (inspect\_source())。
- 说到底, 本书后面那些章节, 干的就是一件小事——把这 kernel 里的三个要素「放大到真实规模」。

延伸阅读

- 原文章节:[Introduction to TIRx](#)
- 全书主页:[Modern GPU Programming for ML](#) Sys
- 相关后续章节 (原书内):TIRx 布局 API、TIRx 语言参考、构建分块 GEMM、编译器内部
- 原书的交互演示 (Scope/Layout/Dispatch 行高亮) 无法在本笔记中复现, [建议直接到原文页面](#) 体验。

术语对照

| 中文                       | English                          | 说明                                    |
|--------------------------|----------------------------------|---------------------------------------|
| 张量 IR neXt               | TIRx (Tensor IR neXt)            | 本章主角:IR 层级的 GPU 编程 Python DSL         |
| 作用域                      | scope                            | 三要素之一: 哪些线程执行某操作                      |
| 布局                       | layout                           | 三要素之一:tile 数据住在哪、怎么排                  |
| 派发                       | dispatch                         | 三要素之一: 走哪条硬件路径                        |
| 中间表示                     | IR (Intermediate Representation) | 编译器可处理的结构化程序表示                        |
| 下降                       | lower / lowering                 | 把高层 IR 翻译为更低层指令的过程                    |
| 共享内存                     | SMEM (Shared Memory)             | CTA 内共享的片上内存                          |
| 张量内存                     | TMEM (Tensor Memory)             | Blackwell 上专供 Tensor Core 累加的内存       |
| 全局内存                     | GMEM (Global Memory)             | 设备全局显存                                |
| 寄存器文件                    | RF (Register File)               | 每线程私有寄存器                              |
| 线程块                      | CTA (Cooperative Thread Array)   | 一组协作线程, 对应 CUDA block                 |
| warp 组                   | warpgroup                        | 4 个 warp(128 线程) 组成的协作单元              |
| 线程束                      | warp                             | 32 个 lane 组成的执行单元                     |
| 通用矩阵乘                    | GEMM                             | General Matrix Multiply               |
| 矩阵乘累加                    | MMA (Matrix Multiply-Accumulate) | Tensor Core 的核心操作                     |
| Blackwell Tensor Core 路径 | tcgen05                          | 第 5 代 Tensor Core 指令族 (tcgen05.mma 等) |
| 混排 / 搅动                  | swizzle                          | SMEM 地址重排, 避免 bank 冲突                 |
| 命名轴                      | named axes (TLane / TCol)        | 布局 DSL 中与硬件对应的逻辑轴                     |
| 协作式                      | cooperative                      | 一条指令由整组线程协同完成                         |
| 屏障                       | mbarrier                         | 异步操作完成同步的内存屏障                         |
| 张量内存访问                   | TMA (Tensor Memory Accelerator)  | 硬件异步批量拷贝引擎                            |
| warp 专门化                 | warp specialization              | 让不同 warp 承担不同角色的优化手法                  |

## 第 10 章 · TIRx Layout API

原文:[TIRx Layout API](#)

### 本章要点

#### 本章要点 (TL;DR)

- 上一章我们在纸上发明了一套记号来描述布局。这一章干的事说白了就一句话: 把那套纸面记号原样搬进编译器, 变成你能 new 出来的对象。这样一来, 你在纸上画的布局, 跟内核里敲的代码, 几乎就是同一行字。
- 要记的东西不多, 就三个: `TileLayout` (仿射布局)、`SwizzleLayout` (拿 XOR 给共享内存换序)、`ComposeLayout` (把 swizzle 叠到 tile 上面)。
- 整套设计的魂就一句话: 一个布局, 把一个逻辑坐标, 映射到一个甚至好几个物理坐标。方向记牢——是「逻辑  $\rightarrow$  物理」, 而且物理那头是一个集合。为什么非得是集合? 因为数据要广播 (broadcast) 的时候, 只有“一对多”才说得通。
- `TileLayout` 分三步走, 有点像填快递单: `shard` / 分片 (写作 `S[...]`, 公式里记成 `D`) 先把基础位置定下来; `replica` / 复制 (写作 `R[...]`, 记成 `R`) 再把它抄送到别的地方; `offset` / 偏移 (记成 `O`) 最后整体挪个位置。一句话公式:  $L(x) = \{ D(x) + r + O \mid r \in R \}$ 。
- 布局里的轴 (axis) 可不是匿名的维度编号, 每一条都是有名有姓的真实硬件坐标, 比如 `laneid`、`warpid`、`TLane`、`TCol`、`m`、`Bank`。另外配了几个现成的构造器 (`tmem_datapath_layout`、`tcgen05_atom_layout`、`wg_local_layout`), 把高频的硬件布局打包好让你直接拿来用——但别紧张, 它们吐出来的还是普通 `TileLayout`, 没藏什么黑魔法。

### 前置知识

**前置知识:** 读这一章前, 最好先懂第 3 章的布局记号 (shape/stride 那一套)、第 9 章的 TIRx 基础, 以及 `warp` / `lane`、`TMEM`、`swizzle` 这几个词。没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 3、9 章。本章会默认你已经认识这些词。

不过你放心, 这一章里凡是出现 GPU 专有名词, 我都会在它第一次露面的时候, 当场用大白话给你讲清楚“它是什么、为什么要有它”。你只要会写代码就行, 不需要任何 GPU 背景。

### 一句话先理解

**一句话先理解:** 这一章讲的“布局 (layout)”, 你可以先粗暴地理解成一张座位表——它规定“我这块数据里的第几个元素, 该坐到硬件的哪个座位上去”。座位可能是某个线程的私有存储, 也可能是片上内存的某个格子。这一章就是教你怎么用代码把这张座位表写出来。

### 先扫盲: 几个本章反复出现的硬件词

你完全没碰过 GPU 也没关系, 下面这几个词先有个模糊印象就够, 后文用到时我还会再细讲:

- **线程 (thread)**: GPU 干活的最小单位, 跟你熟悉的 CPU 线程类似, 但 GPU 会成千上万个线程一起跑。
- **lane / 通道**: 32 个线程被硬件捆成一小班, 班里每个线程有个 0-31 的编号, 这个编号就叫 lane。
- **warp / 线程束**: 上面说的那“一小班 32 个线程”, 整体就叫一个 warp。记住数字: 1 个 warp = 32 个 lane。
- **寄存器 (register)**: 每个线程私有的、速度最快的一丁点存储, 类似 CPU 寄存器, 但这里是“每个线程各有一份”。
- **共享内存 (SMEM)**: 一个线程组内部大家共用的一块片上小内存, 可以理解成一块“程序员手动管理的高速缓存”。
- **TMEM(张量内存)**: 新一代 GPU(Blackwell) 上专门伺候矩阵乘硬件的一种特种内存。
- **矩阵乘**: 为什么矩阵乘总当主角? 因为深度学习模型说到底, 绝大部分计算量都是一堆大矩阵相乘, GPU 专门造了硬件来加速它。

## 10.1 为什么需要 Layout API: 让记号变成对象

### 一句话先理解

**一句话先理解**: 上一章我们在纸上发明了一套“座位表”的写法, 但那只是写在纸上的文字。这一节讲的是: 怎么让这套纸面写法直接变成代码里能 `new` 出来的真对象, 这样纸上画的和代码里敲的就是同一行字。

先把上一章那套记号在脑子里过一遍。它就三块拼起来: 一个 **tile**(把一个大矩阵切成小方块, 每一小块就叫一个 tile; 因为整块矩阵太大放不下, GPU 总是切成小块一块一块处理) 的形状, 一组挂在**具名轴 (named axis)** 上的**步长 (stride)**, 外加一个可选的复制项。

这里有两个词要先解释清楚:

- **步长 (stride)**: 就是“逻辑上往前挪一格, 物理上要跨多远”。比如一个数组按行存, 行内挪一格地址 +1, 换到下一行地址要 +(一行的宽度), 这个“+ 多远”就是步长。会写代码的你对一维数组下标换算肯定不陌生, 这就是同一回事。
- **具名轴 (named axis)**: 普通数组的步长是挂在一个扁平的内存地址上的; 而这里的步长是挂在一条有名字的硬件坐标上, 比如挂在 `laneid`(线程编号) 上、或者挂在 `warpid`(第几个 warp) 上。换句话说, 步长不再是“地址 + 几”, 而是“线程编号 + 几”或者“warp 编号 + 几”。后面 10.5 会专门讲这个, 这里先有个印象。

那个可选的复制项是专门留给“被拷贝、而不是被切开”的数据用的 (后面 10.4.2 细讲)。这一章要干的, 就是把这套记号坐实成编译器真能跑的 API。

好处一句话就能说明白: **纸上写的, 跟代码里写的, 一模一样**。比方说你在文档里写下这么一行:

```
S[(128, 256) : (1@TLane, 1@TCol)]
```

这一行可不是写给人看的说明, 它真的会**构造出一个 TileLayout 对象**。这个对象能“挂 (attach)”到 **buffer**(缓冲区, 就是一块装数据的内存, 你可以理解成一个带类型和形状的数组) 身上。



一旦挂上去，凡是碰这个 buffer 的 tile 操作，都能直接从布局里读出”这块数据该摆在硬件的哪个角落”。

好处在哪？**摆放规则只写一次、只检查一次，后面编译器反反复复地用。**你不用在每个算子里都把”每个元素住哪儿”重抄一遍。这就跟你在代码里把一段逻辑抽成一个函数、到处复用是一个道理——只是这里复用的是”数据该怎么摆”这条规则。

那布局是啥时候挂上去的？两个时机：要么从**内存池 (memory pool, 一块预先划好、由你自己管理的内存, 从里头一块一块抠出来用)**里分配的时候，要么声明 buffer 的时候。

```
pool.alloc(shape, dtype, layout=layout) #
T.decl_buffer(shape, dtype, scope=scope, layout=layout) # buffer
```

从这一刻起,buffer 就自己”背着”物理摆放信息到处走了。tile 操作再也不用一遍遍啰嗦每个元素在哪。

所有布局对象都住在同一个模块里：

```
from tvn.tirx.layout import (
 TileLayout, SwizzleLayout, ComposeLayout, #
 S, R, # shard / replica
 laneid, warpidx, tid_in_wg, #
 TLane, TCol, m, # TMEM
 tcgen05_atom_layout, tmem_datapath_layout, #
)
```

### 关键

**关键：**整套 API 就围着一个念头转——一个逻辑下标，对应的未必是单个物理地址，而是一组挂在具名轴上的物理坐标，也就是一个集合。

为什么强调”集合”而不是”一个地址”？因为有些数据需要**广播 (broadcast)**——也就是同一份数据要同时出现在好几个线程、好几块内存区域里给大家共用。如果一个逻辑元素只能对应一个物理地址，那”同一份数据同时出现在多处”就没法表达了。所以这里特意设计成”一对多”。

话说回来，**大多数时候这集合里就一个元素**（就是普通的一对一摆放）。只有用上复制时，同一个逻辑元素才会在好几个物理位置同时露面。布局之所以拆成 shard / replica / offset 三块，正是为了把这件事支撑起来：shard 把元素摆好，replica 把它拷到别处，offset 再整体平移一下。

## 10.2 先看几个例子 (Layouts by Example)

别急着抠机制，先看几个典型例子，把手感找到再说。下面每个例子你都先不用看懂代码细节，只要跟着我用大白话读一遍，体会”哦，原来一行布局就是一张座位表”这个感觉就行。

**例 1:TMEM(Blackwell 这一代 GPU 上专给张量核用的特种内存, 见第 0 章) 里的累加器 (accumulator, 矩阵乘里不断把一小批一小批的乘积加进去、攒成最终结果的那块存储)——只摆放, 不复制。**

```
acc = TileLayout(S[(128, 256) : (1@TLane, 1@TCol)])
```

这行怎么读?S[...] 就是”摆放规格”的意思(后面会细讲)。(128, 256)是逻辑形状,128行256列;1@TLane 你读成”挂到 TLane 这条轴上,步长为1”。所以整行很直白:逻辑的行扔到 TLane 这条硬件坐标上,逻辑的列扔到 TCol 上。TLane、TCol 是 TMEM 这块特种内存的两个坐标方向,你暂时把它当成”TMEM 的行号和列号”就行。顺嘴提一句命名:TMEM 那一章里硬件坐标叫 Lane 和 Col,到了 TIRx 记号里就改写成 TLane、TCol,其实是同一个东西。

**例 2: 块缩放 (block-scaled)MMA(matrix-multiply-accumulate, 张量核上”乘一批 + 加进去”一气呵成的矩阵乘累加指令) 的缩放因子布局——这回用上了复制。**

```
scale_factor_layout = TileLayout(
 S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
)
```

怎么读?S[...] 那段是 shard(摆放),R[...] 那段是 replica(复制)。shard 先在 TMEM 里放下一个 32 行的组。replica 接着以 32 个 lane(通道,即 warp 内 0-31 的线程编号,见第 0 章)为间隔,把这个组整体抄送 4 份。这么一来,一个 32 行的组就把 128 个 lane(= 4 × 32)的整片 TMEM 铺满了。注意:抄送出来的 4 份是同一份数据,不是 4 份不同的数据,这正是”复制/广播”的意思。

**例 3: 张量核 (Tensor Core,GPU 上专门做矩阵乘的硬件单元;深度学习的算力主要就靠它) 的寄存器片段 (fragment——一个 tile 不是整块放在一处,而是被拆碎、每个线程的私有寄存器里揣一小片,这一小片就叫 fragment)——散落在 lane 和 warp(线程束,一个 warp = 32 个线程的小班,见第 0 章)上。**

```
frag = TileLayout(
 S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
)
```

这里有两点得留个心。第一,同一条物理轴可以出现不止一次——你看,这里就有两个不同的项(后面会管它叫 iter)都往 laneid 上塞数据。这没问题,它俩在 laneid 上的贡献最后会加到一起。第二,要是你没写明往哪条轴放(例子里最后那个光秃秃的 1,只有步长没写 @),那它默认就落到内存轴 m 上(m 就是”普通线性内存地址”那条轴,寄存器槽常用它,后面 10.5 表格里有)。

**例 4: 真实内核里,常见的硬件布局一般直接调构造器拿。**

```
acc = tmem_datapath_layout("D", 128, 256)
ld = tcgen05_atom_layout("32x32b", (128, 64), "float32")
```

这些构造器返回的依然是普通的 TileLayout 对象。它们不过是替你把常用布局打了个包,不是另起炉灶搞出来的新东西。所以你想干啥都行:检视 (inspect) 它返回的布局、把它跟别的布局拼起来,或者碰上特别古怪的形状,索性退回去手写底层的 S[...] / R[...]。

## 10.3 交互演示 (Interactive Demo) 要点

原文在这儿放了个交互式演示。你可以挑一个预置布局,改改逻辑形状,改改 S / R 项,再选个 dtype 和 swizzle 模式,然后点中一个元素,看到它到底落到了哪个、或者哪几个物理坐标上。

可惜这是个交互组件,静态笔记里搬不过来。不过没关系,下面这张”数据流”图就把它在背后干的活儿概括清楚了——而且这张图基本就是后面整套 API 的精确版,值得你记住。图里几个词先简单说一下:展平 (flatten) 就是把多维下标压成一个一维数字(跟你把二维数组按行拍扁成一维数

组算下标是一回事); **行主序 (row-major)** 就是”先走完一行再换下一行”的那种排法 (C 语言数组、NumPy 默认都是这个); **replica** 就是前面说的复制。

#### 结构图

连接关系:

- 1. 展平 flatten / (行主序 row-major) → 2. 按 shard extents 切分 / 得到 c0, c1, ...
- 2. 按 shard extents 切分 / 得到 c0, c1, ... → 3. 各分量乘步长累加到具名轴 /  $ck * sk @ ak$ , 再加 offset
- 3. 各分量乘步长累加到具名轴 /  $ck * sk @ ak$ , 再加 offset → 有 replica?
- 有 replica?  $\xrightarrow{\text{无}}$  单一物理坐标 / {...}
- 有 replica?  $\xrightarrow{\text{有}}$  枚举所有副本组合 / 得到坐标集合 {...}, {...}

#### 注意

**注意:** 这个演示最值钱的地方, 就是它把一件事画给你看了: 同一列访问, 不加 swizzle 的时候全挤在一个 bank(共享内存被切成的小块存储体, 后面 10.12 细讲) 里; 一旦配上对路的 swizzle(一种把数据打散摆放的换序手法, 后面 10.11 起细讲), 立马就散到不同 bank 上去了。这两个词你现在不懂没关系, 先记着”挤在一起 = 慢、散开 = 快”就行, 后面会从头讲。有条件的话, 强烈建议去原书亲手玩一玩。

## 10.4 TileLayout: 主力仿射布局

#### 一句话先理解

**一句话先理解:** TileLayout 是这一章最主要、你最常用的那个布局类。它只会做一种很”乖”的运算——拿坐标乘个数、再加起来。下面把它内部的三个零件 (shard / replica / offset) 拆开慢慢看。

TileLayout 是你最常打交道的布局对象, 它属于**仿射 (affine)** 那一类。”仿射”是个数学术语, 但别被吓到, 说白了就是”乘个步长再加起来”这种规规矩矩的线性变换, 没有取模、没有位运算这些花活儿 (那些花活儿要等到后面 swizzle 才出场)。它的写法跟正文记号完全是一套:

```
TileLayout(S[shape : strides]) # shard
TileLayout(S[shape : strides] + R[replica_shape : replica_stride]) # shard + replica
TileLayout(S[shape : strides] + R[...] + offset) # shard + replica + offset
```

- S 项就是 **shard spec**(分片规格), 你可以一句话读出来:”拿一个这种形状的逻辑 tile, 按这些步长, 把它摊到具名轴上去”。
- 要是某个值得同时出现在好几个地方, 就拿 R 项把 shard 扩一扩。
- 最后还能再叠一个可选的 offset。

再往下挖一层: 这几个部件底层都靠一种叫 **iter**(迭代器, 你把它当成”一条走法说明书”就行) 的东西来表示。一个 iter 就是一个三元组:

```
(extent, stride, axis)
```

你可以把一个 iter 想象成”在某条具名轴上, 从 0 开始, 按固定步长走一趟”。这三个数的意思是:

- **extent**——走几步 (也就是这一维有多少个位置)。
- **stride**——每步迈多远 (就是前面说的步长)。
- **axis**——在哪条硬件坐标上走 (比如 laneid、warpid)。

打个比方,iter (8, 4, laneid) 的意思就是:”在 laneid 这条轴上走 8 步, 每步跨 4, 于是依次落在 laneid = 0, 4, 8, 12, ..., 28。”下面咱们把 shard / replica / offset 三块挨个拆开看。

#### 10.4.1 Shard(分片, 记作 D)

shard 就是 `S[...]` 写出来的那一块。它干的活儿是: 把逻辑下标摊到一个或几个 iter 上**切开**, 再算出**基础物理坐标**。看个例子:

```
S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
```

这里有 4 个 shard iter,extent 依次是 8, 2, 4, 2。它们的步长把数据分别送到 laneid、warpid、又一次 laneid、还有默认内存轴 m(最后那个没写 @ , 默认就是 m) 上。

别觉得陌生, 这其实就是你平时写多维数组时那套”**形状-步长**”**换算规则**的加强版。你算二维数组 `a[i][j]` 的地址是 `i*stride + j`, 对吧? 这里干的是同一类事, **唯一不一样的地方是: 步长现在挂到了”具名硬件轴”上** (比如算出来的不是一个内存地址, 而是”该去 laneid= 几、warpid= 几”), 而不再是挂在一个扁平的内存地址上。

#### 10.4.2 Replica(复制, 记作 R)

replica 说的是同一个逻辑元素的**额外物理副本** (就是同一份数据多复制几份摆到别处)。它有个要命的特点:**replica iter 跟你访问的是哪个逻辑元素一点关系都没有**。

这话什么意思?shard 是”你问第几个元素, 我告诉你它在哪”——结果**随你问的下标变**。而 replica 是”不管你问哪个元素, 我都额外在固定的几个地方各放一份”——结果**跟下标无关、永远是那套固定偏移**。比如:

```
R[2 : 4@warpid] # warpid , 4 warpid
```

读法:`R[2 : 4@warpid]` 表示在 warpid 这条轴上复制出 2 份, 相邻两份隔 4 个 warpid。也就是说, 原本在 warpid = w 的那份数据, 会同时再在 warpid = w+4 处出现一份。

##### 关键

**关键:** 别把复制看成图省事的小聪明, 它是在**老老实实在地照搬硬件本来的样子**。有些数据天生就得在 warp、lane 或者内存区域之间广播 (broadcast) 出去。而”逻辑 → 物理”这个方向, 恰好天生就能把这事说清楚——因为它本来就允许一个逻辑元素对应一组物理坐标。

#### 10.4.3 Offset(偏移, 记作 O)

offset(偏移) 就是一个固定坐标, 它会加到**每一个**结果上, 效果是把整张座位表整体挪个位置。比如:

```
5@warpid # warpid 5(warpid=0 , warpid=5)
```

offset 通常拿来干这么几件事: 把 tile 挪到指定的起点; 给某个独占用途圈一块地方; 或者描述一个”紧贴在另一个 tile 后头”的 tile(俩人共用同一块资源)。

10.4.4 三部分如何协同

一个布局会按固定的顺序走这三步, 次序乱不得:

- 1. **shard** 先把基础坐标算出来;
- 2. **replica** 再把这个坐标抄成零个或多个额外副本;
- 3. **offset** 最后把每个坐标都挪一挪。

对逻辑坐标  $x$ , 最终结果就是:

$$L(x) = \{ D(x) + r + 0 \mid r \in R \}$$

这公式看着唬人, 其实就是把上面三步用集合写法表达了一遍, 逐项翻译一下: $D(x)$  是 shard 给元素  $x$  算出的基础坐标 ( $D =$  ”定位”); $0$  是固定 offset(平移); $r$  遍历 replica 给出的每一个副本偏移(抄送); 整个  $\{ \dots \mid r \in R \}$  是”集合推导式”, 意思是”对每一个副本偏移  $r$ , 都算一个坐标  $D(x)+r+0$ , 把它们凑成一个集合”——如果你用过 Python 的列表推导式  $[f(r) \text{ for } r \text{ in } R]$ , 这就是同一个写法的数学版。这公式不用背, 记住”定位  $\rightarrow$  抄送  $\rightarrow$  平移”这三步就够了。要是没有 replica, $R$  里就只剩一个零偏移, 结果是个只装一个元素的集合; 要是有了 replica, 那每个副本位置都会摊上一个坐标。一个写全乎的布局长这样:

```
layout = TileLayout(
 S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)] # shard: tile
 + R[2 : 4@warpid] # replica: 4 warp
 + 5@warpid # offset: warpid=5
)
```

从左往右读就行:shard 把逻辑 tile 摆好,replica 在隔着 4 个 warp ID 的地方又放一份副本,offset 把整体挪到从 warpid = 5 起步。要是你手头已经有现成的 iter 对象, 也可以直接拼:

```
TileLayout.from_iters(shard, replica, offset)
```

不过大多数代码还是爱用  $S[\dots] / R[\dots]$  记号, 因为它读起来更像数学公式, 一眼就懂。

10.5 具名轴 (Named Axes): 名字本身就是语义

前面一直在念叨”具名轴”, 这里专门掰开讲讲。

先说为什么要有”名字”。普通数组里, 维度就是个匿名编号: 第 0 维、第 1 维……你得靠脑子记”第 0 维是行还是列”。但 GPU 的硬件坐标种类多、含义还容易混(线程编号?warp 编号? 内存地址?), 如果只用编号, 过两天自己都看不懂。所以这里给每条轴都起了**实名**, 名字本身就告诉你它指的是哪个硬件坐标。

布局里的轴**不是匿名的维度编号**，每一条都对应一个真实的硬件坐标，或者编译器层面的某个摆放坐标。常见的轴有这么几类（下面表格里的词，你不用全记住，扫一眼知道”哦，轴是分这么几大类的”就够，用到哪个回来查）：

| 类别            | 轴名                                             | 含义                                                          |
|---------------|------------------------------------------------|-------------------------------------------------------------|
| 网格轴 (grid)    | bx, by, bz                                     | 把工作分布到不同 CTA(cooperative thread array, 即一个 block, 见第 0 章) 上 |
| 集群轴 (cluster) | cbx, cby, cbz                                  | 在一个 CTA cluster(一组能协同的 block 集群) 内部摆放工作                     |
| 线程轴 (thread)  | tx, warpid, laneid, wgid, tid_in_wg, wid_in_wg | 描述在 CTA 或 warpgroup(几个 warp 凑成的协作组) 内部的归属                   |
| 内存轴           | m                                              | 默认的线性内存轴 (寄存器槽常用它)                                          |
| 二维便签          | P, F                                           | 用于二维 scratchpad 式摆放                                         |
| 共享内存          | Bank                                           | 命名共享内存的 bank                                                |
| TMEM 轴        | TLane, TCol                                    | TIRx 对 TMEM 的 Lane / Col 坐标的命名                              |

这里顺带解释表里几个生词:**CTA**(也叫 block) 是一组线程的集合,GPU 把活儿先切成一个个 CTA 发下去;**warpgroup** 是几个 warp 凑成的更大协作单位;**bank** 是共享内存切成的小块存储体 (10.12 细讲)。看不懂没关系, 知道” 它们都是 GPU 里某一级的’ 分组单位’ ” 即可。

关键

关键: 轴的名字本身就是布局的一部分。为啥这么强调? 因为两个数值一样的坐标, 硬件含义可能差着十万八千里:1@tx 不等于 1@tid\_in\_wg,1@laneid 也不等于 1@TLane——前者是” 线程往前挪 1 个”, 后者是” 在 warpgroup 内挪 1 个” 或” 在 TMEM 上挪 1 个”, 落点完全是两码事。打个你熟的比方: 就像 time += 1 里这个 1 到底是 1 秒还是 1 毫秒, 数字一样含义天差地别——这里干脆把单位 (轴名) 直接写进式子里, 你就不用再靠上下文去猜” 这一维到底指啥” 了。

10.6 前向映射 (Forward Mapping):layout.apply()

一句话先理解

一句话先理解: 这一节就是把前面” 座位表怎么算出来” 这件事, 落成一个你真能调用的函数。给它一个逻辑坐标, 它返回这个元素该坐到硬件的哪个/哪些座位上。

所谓” 求值一个布局”, 意思就是: 你扔给它一个逻辑坐标, 它给你算出对应的物理落点。要用的 API(函数) 是:

```
layout.apply(*coord) # coord, apply(3, 5) 3 5
```

- (**\*coord** 是 Python 的可变参数写法, 你按维度依次传下标就行。) 返回什么呢:
- 没复制的时候, 你拿到**一个坐标字典**;
  - 有复制的时候 (同一份数据被广播到多处), 你拿到**一组坐标字典** (一个列表, 里头每项是一个落点)。

坐标字典说白了就是把”轴名”映到”在那条轴上的整数位置”，长这样：

```
{"laneid": 7, "warpid": 2, "m": 1}
```

(读法：这个元素归 7 号 lane、第 2 个 warp，落在内存轴 m 的第 1 个槽。)

整个求值就四步，跟前面那张数据流图一对得上。你跟着走一遍就会发现，这就是”多维下标换算”的加强版，没什么神秘的：

- 展平 (flatten)**：先把逻辑坐标按行主序压成一个一维下标（就是把多维下标拍扁成一个数字，跟你算多维数组的线性下标完全一样）。对形状  $(S_0, S_1, \dots, S_{r-1})$  里的坐标  $(x_0, \dots, x_{r-1})$ ，公式是：  
“ $\text{flat} = x_0 * S_1 * S_2 * \dots * S_{r-1} + x_1 * S_2 * \dots * S_{r-1} + \dots + x_{r-2} * S_{r-1} + x_{r-1}$ ”
- 切分 (split)**：把上一步那个一维下标，按 shard 的 extents  $(e_0, \dots, e_{n-1})$  重新切成几段，得到分量  $c_0, \dots, c_{n-1}$  (依然是行主序)。
- 累加 (accumulate)**：每个分量乘上自己的步长，加到对应的轴上去。具体讲，要是第  $k$  个 shard iter 是三元组  $(e_k, s_k, a_k)$ ，那分量  $c_k$  就往轴  $a_k$  上贡献  $c_k * s_k$ 。同一条轴上的贡献全加起来，最后再补上 offset。
- 复制 (replica)**：把 replica iter 施加上去。每个 replica iter 都贡献一个跟逻辑坐标无关的额外偏移；要是有好几个 replica iter，布局就把它们所有的组合都枚举出来。

#### 注意

**注意**：这里藏着一个特别好用的推论——**布局压根不用把输入形状写死**。它只认一件事：逻辑 tile 的元素总数，得等于 shard extents 的乘积（也就是  $e_0 \times e_1 \times \dots \times e_{n-1}$ ）。为什么？因为”展平”把多维拍成了一个一维数字，而”切分”只看这个总数怎么切——只要总数对得上，中间的形状随便怎么摆都行。这个数一对上，光靠”展平 + 切分”就能把整个映射定义得清清楚楚。正是这一点，让一个布局能套用到形状不完全一样、但元素总数相同的多种 tile 上，相当通用。

## 10.7 案例 A：张量核寄存器 tile

### 一句话先理解

**一句话先理解**：这一节没有新概念，纯粹是带你拿前面那四步，对一个具体布局算一遍，把流程跑通。看着步骤多，其实每一步都是小学算术（乘、除、取余）。

光说不练假把式，咱们拿前面的公式跑一个完整的例子。假设有个逻辑  $(8, 16)$  的 tile，要分到 2 个 warp (每个 warp 32 个 lane) 上，每个 lane 自己拿一小段寄存器 (register，每个线程私有、最快的那点存储，见第 0 章) 片段。寄存器槽就用默认内存轴 m 来表示。布局长这样：

```
layout = TileLayout(
 S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
 + R[2 : 4@warpid]
 + 5@warpid
)
```

随便挑个逻辑元素 (i, j), 咱们按四步走一遍。(下面  $\text{floor}(a/b)$  是整除取下,  $a \bmod b$  是取余, 跟你代码里的  $a // b$  和  $a \% b$  一模一样。)

**第 1 步行主序展平:**  $\text{flat} = 16*i + j$

**第 2 步按 shard extents (8, 2, 4, 2) 切分:**

```
c0 = i
c1 = floor(j / 8)
c2 = floor(j / 2) mod 4
c3 = j mod 2
```

**第 3 步累加到各轴 (4@laneid, 1@warpid, 1@laneid, 1 对应 m):**

```
laneid = 4*c0 + c2
warpid = c1
m = c3
```

**加上 offset 5@warpid:**

```
laneid = 4*i + floor(j/2) mod 4
warpid = floor(j/8) + 5
m = j mod 2
```

**第 4 步施加 replica R[2 : 4@warpid](在 warpid 上加 0 或 4):**

```
laneid = 4*i + floor(j/2) mod 4
warpid = floor(j/8) + 5 + 4*r, r in {0, 1}
m = j mod 2
```

捋一捋结果:shard 把 tile 放到了 warp 5 和 6 上,replica 又把它拷到 warp 9 和 10。所以同一个逻辑元素,最后落在了**两组 warp 位置**上:

#### 结构图

连接关系:

- shard 落点 / warpid = 5, 6  $\xrightarrow{+4 \text{ (replica)}}$  replica 副本 / warpid = 9, 10

#### 关键

**关键:** 这个例子正好讲明白了为啥这套模型非得选”逻辑  $\rightarrow$  物理、而且物理那头是一个集合”这个方向。

倒过来想一下:假如你把方向反过来,做成”物理坐标  $\rightarrow$  逻辑坐标”的函数,会怎样?复制就写不出来了。因为复制意味着”warp 5、6、9、10 这好几个物理位置,装的都是同一个逻辑元素”——也就是好几个物理点都要映回同一个逻辑点。可你想想数学上”函数”的定义:一个输入只能有一个输出。多个物理点映到同一个逻辑点没问题,但要是反过来从这个逻辑点出发,它该往回指哪个物理点?指不清,这就**不是一个合法的函数**了。

而用”一个逻辑坐标  $\rightarrow$  一组物理坐标”这个方向,一点障碍都没有:一个输入(逻辑坐标),对应一个输出(物理坐标的集合),完全合法。这就是 API 死活要选”逻辑  $\rightarrow$  物理”、还让结果



是个集合的根本原因。

10.8 案例 B:Blackwell 张量内存 (TMEM)

一句话先理解

**一句话先理解:** 上个例子在描述”数据分给哪些线程”, 这个例子在描述”数据摆在内存的哪个格子”。重点是: **同一套 TileLayout, 这两件事都能干**, 只是把轴从线程轴换成内存轴而已。

上个例子用的是线程轴 (laneid, warpid 这种, 讲的是”谁负责这块数据”)。但同一套布局模型, 其实拿来描述**内存摆放** (讲的是”这块数据搁内存哪儿”)也照样行——轴不一定非得是线程轴, 换成内存轴一样玩得转。TMEM 这块特种内存靠硬件 Lane 和 Col 两个坐标来寻址 (你就当它是”行坐标”和”列坐标”), 在 TIRx 记号里就写成 **TLane**、**TCol**。看这个布局:

```
layout = TileLayout(
 S[(2, 128, 112) : (112@TCol, 1@TLane, 1@TCol)]
)
```

要是逻辑形状正好就是 (2, 128, 112), 那就省事了: 切出来的分量恰好就是逻辑坐标本身, 不用再折腾换算。对元素 (a, 1, c):

```
TLane = 1
TCol = 112*a + c
```

- extent 是 128、步长 1@TLane 的那个 iter, 负责把 128 行 TMEM Lane 填满;
- extent 是 2(步长 112@TCol) 和 extent 是 112(步长 1@TCol) 这两个 iter 凑一块儿, 一共盖住 224 列, 也就是  $TCol \in [0, 224)$ 。

| TLane \ TCol | [0, 111]      | [112, 223]    |
|--------------|---------------|---------------|
| TLane=0      | a=0 段 (112 列) | a=1 段 (112 列) |
| TLane=1      | a=0 段 (112 列) | a=1 段 (112 列) |
| ...          | ...           | ...           |
| TLane=127    | a=0 段 (112 列) | a=1 段 (112 列) |

图注:128 行 TMEM Lane 全部填满;TCol 共 224 列, 左半 [0, 111] 是 a=0 段, 右半 [112, 223] 是 a=1 段。

注意

**注意:** 这 224 列可不是手滑写错了, 是故意的——**TMEM 布局不一定非得凑成 2 的幂** (2 的幂就是 2、4、8、...、256 这种数; 很多底层代码图省事爱用 2 的幂, 但这里不强求)。举个例子, 一个块缩放 FP8 GEMM(general matrix multiply, 通用矩阵乘法;FP8 是一种只占 8 位的超省空间浮点数) 就可能专门挑 224 列的累加器: 因为要是塞满 256 列,TMEM 这块内存就装不下了, 没法把两个累加器阶段再加上缩放因子全放进去。这种”不规整”(非 2 的幂) 的形状,Layout API **张口就能写**——这正是它”支持通用形状”这个设计目标在发挥作用。

10.9 缩放因子布局 (Scale Factor Layouts): 复制的真正用武之地

一句话先理解

一句话先理解: 这一节是 replica(复制) 的”主场秀”。前面累加器是一一对一摆放, 用不上复制; 而缩放因子需要被好几组线程同时看到, 这正是复制天生要解决的问题。

先说说缩放因子是干嘛的: 块缩放 (block-scaled) 矩阵乘里, 为了用 FP8 这种超低精度还不丢太多精度, 会给每一小块数据配一个”缩放系数”, 算的时候乘回去。这个系数就叫**缩放因子 (scale factor)**。

前面那个累加器是**纯摆放**: 一个逻辑累加器元素, 只对一个 TMEM 坐标, 一对一。可块缩放 MMA 里的缩放因子就两码事了——**同一份缩放因子, 经常得让好几个 warp 窗口同时都能读到** (因为它们要拿同一个系数去缩放各自手里的数据)。这下复制可就大显身手了:

```
scale = TileLayout(
 S[(32, sf_per_mma) : (1@TLane, 1@TCol)] # shard: TMEM 32
 + R[4 : 32@TLane] # replica: 32 lane, 4
)
```

对逻辑缩放坐标 (r, s):

```
shard: TLane = r , TCol = s
replica : TLane = r + 32*q , TCol = s , q in {0,1,2,3}
```

这么一来, 这个 32 行组就在 TMEM lane 的 0-31、32-63、64-95、96-127 这四个窗口里同时现身了:

| lane 窗口    | TLane [0, 31] | TLane [32, 63] | TLane [64, 95] | TLane [96, 127] |
|------------|---------------|----------------|----------------|-----------------|
| 内容         | SF            | SF             | SF             | SF              |
| replica 偏移 | 原本 (+0)       | +32            | +64            | +96             |

图注: 四个 warp 大小的 lane 窗口看到的是同一份缩放因子组 (SF), 通过 replica 在 TLane 上分别偏移 0 / 32 / 64 / 96。

这其实就是「跨 GPU 世代的张量核操作数布局」那一章讲的 **warp<sub>x</sub>4 广播模式**: 四个 warp 大小的 TMEM lane 窗口, 看到的都是同一份缩放因子组。

当然, 真实场景里这个布局还要再复杂一些 (下面这段你扫一眼有个印象即可, 不影响主线)。这个”原子”(atom, 指最小的可复用布局单元) 会跟外层 iter 拼到一起, 把它沿着矩阵的 M 行方向、以及 K 方向的多个缩放因子组铺开。另外, 看缩放因子是什么 **dtype**(data type, 数据类型, 比如 fp8 / fp16), **好几个缩放因子还可能挤进同一个 32 位 TCol 单元**——比方说 fp8 缩放因子只占 8 位, 4 个正好打包塞进一个 32 位列单元。除此之外还有两个可选的 iter: 一个”步长为 0 的复用”iter(步长为 0 意味着”原地不动地重复用同一份”), 一个”流水深度”iter, 分别用来描述跨多次 MMA 复用同一缩放因子, 以及**双缓冲** (double buffering, 准备两块缓冲区轮流用, 一块在算、另一块同时在搬数据, 省得互相等)。

关键

**关键:** 这里最该记住的一点是——同一个 **TileLayout** 模型, 把两种压根不一样的情况一锅端了。累加器是 TMEM 里的”单点摆放”; 缩放因子是同一片 TMEM 地址空间里的”复制摆放”。就一套模型, 两类需求全搞定。

10.10 现成布局 (Ready-Made Layouts)

写内核 (**kernel**, 就是真正跑在 GPU 上的那段函数) 的时候, 谁也不乐意每个硬件布局都从头一个 **S[...]** 一个 **R[...]** 地手敲。所以 TIRx 替那些天天用到的布局备好了现成的**构造器 (constructor, 就是替你 new 出对象的工厂函数)**, 直接调就完事——跟你用标准库里的现成函数、不自己造轮子是一个意思:

| 构造器                                                                | 返回内容                                                                   | 关键参数说明                                                                                           |
|--------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>tmem_datapath_layout(datapath, rows, cols)</code>            | <code>tcgen05.mma</code> 写出的 TMEM 累加器布局                                | <code>datapath</code> 选择行摆放模式: "D" 对应 M=128 的恒等式摆放, "F" 对应 M=64 的散布式摆放                           |
| <code>tcgen05_atom_layout(instr_shape, tensor_shape, dtype)</code> | 由一个 <code>tcgen05.ld / tcgen05.st</code> 原子搬运的寄存器 <code>tile</code> 布局 | <code>instr_shape</code> 如 <code>.32x32b</code> , <code>.16x64b</code> , <code>.16x128b</code> 等 |
| <code>wg_local_layout(cols, rows=128)</code>                       | <code>warpgroup</code> 本地寄存器 <code>tile</code>                         | 通常每个线程一行, 落在 <code>tid_in_wg</code> 上                                                            |

这里有个关于 `tcgen05_atom_layout` 的小门道值得留意: 它在两个不同层次上长得不一样。先解释两个词:**DSL** 是 domain-specific language(领域专用语言), 这里就指你写内核的那层高级写法; **降级 (lowering)** 是编译器把高级写法一步步翻译成底层真指令的过程 (就像高级语言被编译成汇编)。

在 DSL 这层, `tcgen05_atom_layout` 看上去就是一整块 **warpgroup 分布式 tile**(一整个 `warpgroup` 共同持有的一块数据); 可一旦经过降级, 它就被拆成**四条由各 warp 协同执行 (warp-collective) 的 `tcgen05.ld / tcgen05.st` 指令** (`ld` = load 读, `st` = store 写), 一个 **warp 摊一条**, 每条管自己那 32 个 TMEM lane。一句话: 上层看是一整块, 下层其实是 4 个 warp 分头干活。

结构图

连接关系:

- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → warp 1: `tcgen05.ld/st` (lane 32-63)
- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → warp 2: `tcgen05.ld/st` (lane 64-95)
- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → warp 3: `tcgen05.ld/st` (lane 96-127)

注意

**注意:** 这些 helper 不过是替你省掉手写常见硬件映射的功夫, 它们并没有把模型藏起来——每个 helper 吐出来的, 都是拿前面那套 **S / R** 部件拼成的普通 **TileLayout**。

## 10.11 SwizzleLayout 与 ComposeLayout: 仿射之外的换序

### 一句话先理解

**一句话先理解:** 到目前为止那套”乘步长再相加”的乘运算(仿射)有个天花板,搞不定一种叫 swizzle 的打乱摆放。所以 TIRx 把 swizzle 单独做成一个对象,再用 ComposeLayout 把它”叠”在 TileLayout 上面。这一节先讲清这个分工,具体为什么要 swizzle、怎么算,留给后面 10.12 起。

TileLayout 是仿射的:它能描述具名轴上的步长、复制和偏移。线程片段、TMEM tile、紧凑缩放因子布局这一大类摆放,它都拿得下。

可有一样它就是搞不定,那就是**共享内存 (SMEM)**,一个 block 内线程共享的片上内存,见第 0 章)的 swizzle。swizzle(换序)就是故意把数据在存储体之间打乱摆放,目的是躲开一种叫 bank 冲突 (bank conflict) 的性能坑——多个线程同时去挤同一块存储体内的不同地址,硬件没法并行,只能排队一个个来,慢一大截(后面 10.12 会从头讲)。问题是,这种打乱**根本不是”乘步长再相加”那种乖模式**,而是对内存地址来一次基于 XOR 的置换 (permutation,就是把地址重新洗牌排序)。XOR 是按位异或,典型的位运算,硬塞进仿射这种纯加乘的模型里,别扭得很。

所以 TIRx 索性把 swizzle 单拎出来做成一个布局对象,再让它跟 tile 布局组合着用:

```
swizzle = SwizzleLayout(...) #
layout = ComposeLayout(swizzle, tile) # swizzle tile
```

执行顺序一目了然:先让 tile 布局把线性内存地址算出来,再让 swizzle 把这个地址重排一遍。把这两层拆开,比硬把 XOR 置换往仿射模型里塞,要干净利落得多。

### 结构图

连接关系:

- 线性内存地址  $m \xrightarrow{\text{SwizzleLayout(XOR 置换)}}$  换序后地址  $addr$
- 换序后地址  $addr \rightarrow$  实际共享内存落点 / (bank 被打散)

## 10.12 为什么要 swizzle:bank 冲突的来由

### 一句话先理解

**一句话先理解:** 这一节讲”病因”——为什么不做 swizzle 就会慢。把病因搞懂了,下一节那个看着吓人的 XOR 公式你就知道它在治什么了。

得先把 bank 冲突是怎么来的讲清楚,你才明白 swizzle 到底在治啥病。

共享内存被硬件切成 32 个 bank(你就理解成 32 个并排的小抽屉),每个 bank 里一个”字(word)”是 4 字节。**为什么要切成 32 个?** 因为一个 warp 正好 32 个线程,理想情况下这 32 个线程各取各的抽屉,32 个抽屉同时开,一拍就全拿到,这是最快的。

规矩是这样:一次访问里,要是好几个 lane(线程)同时去碰**同一个抽屉 (bank) 里的不同地址**,这一个抽屉一次只能拿一个东西,硬件就只能排着队一个一个来——这就叫这次访问被串行化 (serialized) 了,本来一拍能干完的活儿,现在要好几拍,慢一大截。(注意:大家碰同一个抽屉的同一个地址不算冲突,那叫广播,硬件支持;冲突特指同抽屉、不同地址。)

麻烦就麻烦在: 一个老老实实、按常理排的行主序 `tile`, 从结构上天生就会撞上这种冲突。咱们拿一个 (8, 64) 的 `float16`(16 位、2 字节的半精度浮点数)`tile`、行主序布局来瞧瞧:

```
TileLayout(S[(8, 64) : (64@m, 1@m)])
```

逻辑元素 (i, j) 的线性地址是  $m = 64 \times i + j$  (就是  $\times 64 +$  , 标准的行主序换算)。一行有 64 个 `float16`, 也就是  $64 \times 2 = 128$  字节——而 32 个 bank  $\times$  每 bank 4 字节 = 128 字节, 所以一行正好铺满 32 个 bank 一整圈 (术语叫一条 “bank line”)。

问题就卡在这儿: bank 索引是按地址每 4 字节绕一圈 (0,1,...,31, 然后又回 0)。假设一个 warp 沿着某一列往下读 (也就是 j 固定、i 从 0 到 7 变), 每往下挪一行, 地址就整整跨过 128 字节。可 128 字节正好是 bank 绕一整圈的长度, 跨完一圈又精准回到出发的那个 bank。于是这一列 8 行读下来, 8 个元素全挤在同一个 bank (同一个抽屉) 上——这就是结构性的冲突, 躲都躲不掉。

那 `swizzle` 怎么治? 思路就一句: 让地址的低位 (决定落哪个 bank 的那几位) 去掺进更高的行号信息。这么一搅, 原本不同行算出来落到同一个 bank 的那一列, 就被推到不同 bank 上去了——抽屉散开, 大家就能并行拿了。

### 10.13 swizzle 变换的数学形式

一句话先理解

一句话先理解: `swizzle` 本质上就是对地址做一次 “按位异或洗牌”。这一节把它写成公式。你只要记住一件事: 它只改地址里决定 bank 的那几位, 数据本身一个没动、也没丢。

思路讲明白了, 咱们把 `swizzle` 的数学形式摆出来。别一看位运算就发怵——这里用到的就一个 **XOR (按位异或, 记作 ^)**: 两个二进制位相同得 0、不同得 1, C / Python 里就是 ^。先弄清每个参数是干啥的。一个 `SwizzleLayout` 由三个整数管着:

```
per_element = M #
swizzle_len = B # XOR
atom_len = S # XOR
```

输入是线性元素地址 `m`, 变换分几步走。先拿大白话过一遍 (>> 是右移、<< 是左移、& 是按位与, 都是你熟悉的位运算): **m 最低的 M 位原封不动** (这是为了让一小撮连续的元素继续挨在一起, 方便硬件一次成批读); 剩下的高位拽下来当临时值  $x = m \gg M$ ; 接着把 `x` 里 `[S, S+B)` 那段位组异或到 `[0, B)` 这段上 (这一步就是 “把高位行信息掺进低位” 的实现); 最后再把刚才没动的低 `M` 位拼回去。写成伪代码就是:

```
mask = (1 << B) - 1
low = m & ((1 << M) - 1) # M ,
x = m >> M #
x2 = x ^ ((x >> S) & mask) # [S, S+B) XOR [0, B)
addr = (x2 << M) | low # M
```

## 注意

**注意:** 这套变换要成立, 前提是  $S \geq B$ 。还有一点务必记牢:**swizzle 不会动 tile 里到底有哪些逻辑元素**, 它只动这些元素落到共享内存的哪个位置。MMA 读到的照样是同一个逻辑 tile,swizzle 不过是把背后那套物理 bank 排布优化了一下罢了。

## 10.14 如何选择 swizzle 参数

好消息是: 平时你基本不用亲自去定这三个参数——它们会根据 **dtype(数据类型)** 和 **共享内存 swizzle 模式自动推出来**。常见的 swizzle 模式就 32 字节、64 字节、128 字节这三种 (这个数字数大致对应”打乱的力度”)。

不过道理还是值得搞明白。**per\_element(也就是 M**, 前面说过它管”最低多少位保持不动”) 怎么选? 原则是: **让一个”向量”大小的小组保持连续**。这里的”向量”指硬件一次能成批搬运的一小撮连续数据 (GPU 喜欢一次读一整块, 而不是一个一个抠, 这样最快)。拿 float16 说, 一个 16 字节的向量能装 8 个元素 ( $16 \div 2 = 8$ ), 要让这 8 个连号元素不被打散, 就得保住最低 3 位不动, 所以  $M = \log_2(8) = 3$ 。再配上 128 字节 swizzle, 布局就这么写:

```
SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
```

这套参数一箭双雕: 既保住了 16 字节向量组不被拆散, 又足够把更大范围的共享内存地址模式搅乱, 从而把列方向的 bank 冲突消掉。于是, 一个带 swizzle 的共享内存分配就长这样:

```
tile = TileLayout(S[(8, 64) : (64@m, 1@m)])
swizzle = SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
layout = ComposeLayout(swizzle, tile) # SMEM buffer
```

## 关键

**关键:** 绝大多数代码根本不该自己去手算这些参数——dtype 和 descriptor 模式一般早就把它们定死了。这里的 **descriptor(描述符)** 你就理解成”一张告诉硬件该怎么读写这块数据的配置表”。**TMA(Tensor Memory Accelerator)** 是新一代 GPU 上专门负责高效搬运大块数据的硬件部件, 它搬数据时也按一张 descriptor 来认地址布局。你真正得操心的就一件事:**TIRx** 布局里的 **swizzle**、**TMA** 搬数据时认的 **descriptor**、还有 **MMA** 指令期望的数据排布, 这三家必须对得上、保持一致——它们仨各自按各自的理解去读同一块内存, 只要有一家对不上, 读出来就是乱的。

## 10.15 元素的 bank 与 line

怎么验证一个 swizzle 是不是真管用? 办法挺简单: 把 swizzle 之后的元素地址换算回”它落在 32 个抽屉里的哪一个”, 看看一整列读下来是不是散到了不同抽屉。下面给出从”元素地址”到”bank 编号”的换算。设 **addr** 是 swizzle 后的元素地址 (以”第几个元素”计), **b** 是每个元素的字节数:

```
byte = addr * b #
bank = floor(byte / 4) mod 32 # 32 bank, bank 4
line = floor(byte / 128) # 128 bank line
```

对  $\text{float16}, b = 2, \text{bank}$  公式就简化成  $\text{bank} = \text{floor}(\text{addr} / 2) \bmod 32$ 。下面那段演算用的就是这个。

10.16 完整演算: 对 (8, 64) float16 tile 施加 128B swizzle

一句话先理解

一句话先理解: 这一节把前面所有零件串起来跑一遍真账。结论会非常直观: 加了 swizzle, 原来全挤在一个抽屉的一列, 变成散在 8 个抽屉; 不加, 8 行死死压在同一个抽屉。看完这节你会对 swizzle” 到底有没有用” 彻底踏实。

理论讲完了, 咱们把整件事从头到尾算一遍, 看看冲突是不是真没了。还是那个行主序  $\text{float16}$   $\text{tile}: m = 64 * i + j$ , 套上 `SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)`。把它代进上一节那个变换公式:

```
x = m >> 3
addr = ((x ^ ((x >> 3) & 7)) << 3) | (m & 7)
```

令  $q = \text{floor}(j/8)$ 、 $r = j \bmod 8$ (就是把列号  $j$  拆成” 第几组 8 个” 和” 组内第几个”), 可化简为:

```
addr = 64*i + 8*(q XOR i) + r
```

关键就在  $q \text{ XOR } i$  这一项: 行号  $i$  被 XOR 掺进了地址里。盯住列  $j = 0$  看 (这时  $q = 0$ 、 $r = 0$ ), 得到  $\text{addr} = 64 * i + 8 * (0 \text{ XOR } i) = 64 * i + 8 * i = 72 * i$ 。咱们一行一行地算它落哪个 bank(用前面那条  $\text{bank} = \text{floor}(\text{addr}/2) \bmod 32$ ):

| 行 i | addr = 72*i | bank = floor(addr/2) mod 32 |
|-----|-------------|-----------------------------|
| 0   | 0           | 0                           |
| 1   | 72          | 4                           |
| 2   | 144         | 8                           |
| 3   | 216         | 12                          |
| 4   | 288         | 16                          |
| 5   | 360         | 20                          |
| 6   | 432         | 24                          |
| 7   | 504         | 28                          |

看结果: 这一列现在散在 8 个互不相同的 bank 上, 冲突没了。  
再对比一下不加 swizzle 的情形: 同一列的地址是  $m = 64 * i$ , 于是  $\text{bank} = \text{floor}(64 * i / 2) \bmod 32 = 0$ ——每一行全挤在 bank 0, 整次访问被彻底串行化。差距一眼就看出来了。

| 情况 (列 j=0)     | 8 行 i 落到的 bank                      | 结果          |
|----------------|-------------------------------------|-------------|
| 无 swizzle      | 全部塞进 bank 0                         | 串行化, 约 8x 慢 |
| 有 128B swizzle | 散到 bank 0, 4, 8, 12, 16, 20, 24, 28 | 无冲突         |

## 注意

**注意:** 可别把这个“无冲突”当成包治百病的护身符, 它是**有前提的**: 你得照着 `swizzle` 当初的设计意图来用。`dtype`、`swizzle` 宽度、访问形状, 统统得跟 TMA、MMA 的 `descriptor` 模式对上。一个 128 字节 `float16 swizzle`, 是专门冲着特定的 16 字节行块和张量核访问模式设计的——它可**不保证**你随便来个什么共享内存访问都能无冲突。

## 10.17 设计原理 (Design Rationale)

学到这儿, 咱们回头看看整套 Layout API 背后那三条设计取舍。你会发现, 前面每个例子其实都在替它们做注脚:

1. **支持通用形状**。硬件 tile 可不总是 2 的幂。全局张量、共享内存阶段、TMEM 累加器、缩放因子缓冲, 常常因为容量见顶或者算法上的取舍, 长成“不规整”的形状。布局模型把这种形状当家常便饭来对待 (还记得 10.8 那个 224 列的累加器吧)。
1. **映射方向是「逻辑坐标 → 物理坐标」**。这个方向为啥要紧? 因为复制太常见了: 一个逻辑元素可能同时住在好几个物理位置。”逻辑 → 物理”这个方向, 正好能把这点直接写成一个坐标集合 (回想一下 10.7 那个 warp 复制)。
1. **硬件轴明着写出来**。布局不玩匿名维度, 也不靠”回头看上下文”去猜每一维是啥意思。`tx`、`tid_in_wg`、`laneid`、`warpid`、`TLane`、`TCol` 彼此的区别, 统统白纸黑字写进布局本身。

最后还有一条很要紧的分工: **布局不管合法性、也不管可行性, 那不是它一个人扛的活儿**。布局就管一件事——”数据搁哪儿”; 至于”某个操作能不能合法、又高效地用上这种摆放”, 那是上层 **tile 原语 (primitive, 指那些封装好的基础 tile 操作)** 拍板的事。这就像编程里的”单一职责”原则: 布局只负责描述摆放, 别的事交给别的模块。这么一分工, Layout API 既保持了小巧, 又把足够的信息喂给编译器, 让编译器据此去派发 (**dispatch, 挑选并发出**) 真正该用的那条硬件指令。

## 小结

- TIRx Layout API 把上一章那套纸面布局记号**坐实成了对象**, 核心就三个类: `TileLayout`、`SwizzleLayout`、`ComposeLayout`。
- `TileLayout` 是仿射布局, 分三步走——**shard(S, 定位) + replica(R, 抄送) + offset(O, 平移)**, 底层全靠 (**extent, stride, axis**) 这种 **iter 三元组**来表示。一句话公式  $L(x) = \{ D(x) + r + O \mid r \in R \}$  抓住了魂: 一个逻辑坐标 → 一组物理坐标。
- `layout.apply()` 的求值就四步: **展平 → 按 shard extents 切分 → 乘步长累加到具名轴再加 offset → 枚举 replica 组合**。它不把输入形状写死, 只要”逻辑元素数 = shard extents 的乘积”就行。
- 轴是**具名硬件坐标**, 名字本身就带着意思 (记牢 `1@laneid`  $\neq$  `1@TLane`)。两个案例 (张量核寄存器 tile、Blackwell TMEM) 说明同一个模型既能描述线程摆放、也能描述内存摆放; 缩放因子布局则演示了 replica 怎么自然地把 **warp<sub>x</sub>4** 广播表达出来。
- `swizzle` 是 **XOR 置换、不属于仿射**, 所以单拎出来做成 `SwizzleLayout`, 再用 `ComposeLayout` 叠到 tile 上。它只动物理落点、不动逻辑内容, 靠”低位依赖高位”把 bank 打散, 从而消掉访问的 bank 冲突——但前提是得跟 TMA / MMA 的 `descriptor` 模式**严丝合缝地**对上, 否则白搭。



- 三条设计哲学串起来: 支持通用 (非 2 的幂) 形状、逻辑 → 物理的集合映射、把具名硬件轴明着写出来。布局只管”搁哪儿”, 合法性交给上层 tile 原语去判断。

延伸阅读

- 原文:[TIRx Layout API —Modern GPU Programming for ML](#) Sys
- 前置章节: 《数据布局及其记号 (Data Layout and Its Notation)》——本章记号的来源
- 相关章节: 《特种内存:TMEM(Special Memory: TMEM)》《跨 GPU 世代的张量核操作数布局 (Tensor Core Operand Layouts Across GPU Generations)》

术语对照

| 中文                      | English                             | 说明                                                   |
|-------------------------|-------------------------------------|------------------------------------------------------|
| 仿射布局                    | affine layout                       | <code>TileLayout</code> 表达的步长/复制/偏移类布局               |
| 分片 (规格)                 | <code>shard / shard spec</code>     | 由 <code>S[...]</code> 构建, 产出基础物理坐标                   |
| 复制 (规格)                 | <code>replica / replica spec</code> | 由 <code>R[...]</code> 构建, 生成与逻辑无关的额外副本               |
| 偏移                      | <code>offset</code>                 | 加到每个结果上的固定坐标                                         |
| 迭代器 (三元组)               | <code>iter</code>                   | <code>(extent, stride, axis)</code> , 描述一条具名轴上的带步长行走 |
| 具名轴                     | <code>named axis</code>             | 真实硬件坐标或编译器摆放坐标, 名字即语义                                |
| 前向映射                    | <code>forward mapping</code>        | 逻辑坐标 → 物理坐标 (集合) 的求值                                 |
| 换序 / 拌序                 | <code>swizzle</code>                | 基于 XOR 的共享内存地址置换, 用于避开 bank 冲突                       |
| bank 冲突                 | <code>bank conflict</code>          | 多 lane 访问同一 bank 不同地址导致的串行化                          |
| 块缩放 (MMA)               | <code>block-scaled (MMA)</code>     | 带缩放因子的矩阵乘累加                                          |
| 缩放因子                    | <code>scale factor</code>           | 块缩放 MMA 中的缩放系数                                       |
| 广播                      | <code>broadcast</code>              | 同一数据在 <code>warp/lane</code> /内存区域间被复制               |
| 降级                      | <code>lowering</code>               | DSL 层布局编译为具体硬件指令的过程                                  |
| 张量内存                    | <code>TMEM (Tensor Memory)</code>   | Blackwell 上由 <code>Lane/Col</code> 寻址的特种内存           |
| 线程束 / <code>warp</code> | <code>warp</code>                   | 32 个 lane 的执行单元                                      |
| <code>warp</code> 组     | <code>warpgroup</code>              | 多个 <code>warp</code> 组成的协作单元                         |



# 第 11 章 · 构建分块 GEMM(Step 1-3)

原文:构建分块 GEMM(Step 1-3)

## 本章要点

### 本章要点 (TL;DR)

- $\text{GEMM}(D = AB^T)$  是几乎所有深度学习算子的底层原语; 本章只追求**正确**, 性能留给后两章, 先建立一个可信的基线 (baseline)。
- 采用「一次只改一个决策」的增量式构建:Step 1 单 Tile 顺序 GEMM 建立基线契约,Step 2 加 K 维循环累加,Step 3 用多 CTA 网格做空间分块。
- 把每一步都读成对同一份「契约三要素」的编辑——**scope**(由谁执行)/ **layout**(操作数 tile 布局)/ **dispatch**(走哪条派发路径), 大多数步骤只动其中一项。
- 基线数据通路恒定不变: $\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{tcgen05.mma} \rightarrow \text{TMEM} \rightarrow \text{GMEM}$ ; 后续优化只改「每一跳怎么做得更高效」, 从不增删跳本身。
- 两个最隐蔽的正确性陷阱: 协作拷贝后 MMA 读 SMEM 前必须 `cta_sync`(等齐 + 发布写入); 复用 `mbarrier` 时 K 循环里必须 `phase_mma ^= 1`, 否则 `try_wait` 会提前放行, 静默算错。
- 累加器在 TMEM 中用 fp32 保存「运行和」压住舍入误差, 写回时才收窄 (cast) 为 fp16;Step 3 暂不复用相邻 CTA 的 operand, 带宽浪费留待持久化调度解决。

## 前置知识

**前置知识:** 这一章会带你第一次真正写出一个跑在 GPU 上的程序。你会写代码就行 (Python、C++、后端、数据, 哪种都成), GPU 本身一窍不通也没关系——下面这些 GPU 专有的词, 第一次出现时我都会当场用大白话讲清楚「它是什么、为什么需要它」。这里先给你列个最粗的概念清单, 有个印象就好, 看不懂也别慌, 正文会一个一个展开:

- **GPU 为什么快:** 它不像 CPU 那样靠几个很强的核心, 而是靠成千上万个很弱的核心一起上。所以 GPU 编程的核心永远是「怎么把活拆给海量线程并行干」。
- **线程的层级:** GPU 把线程一层层打包——最小是 **warp**(32 个线程为一组), 往上是 **warp-group**(4 个 warp = 128 个线程), 再往上是 **CTA / 线程块** (一组能共享同一块高速内存、彼此协作的线程)。这些词正文都会细讲。
- **内存的层级:** GPU 的内存也分快慢好几层 (离计算越近越快、但越小), 所以「数据现在待在哪一层、怎么搬到下一层」是性能的关键, 正文会逐层介绍 **GMEM / SMEM / TMEM**。
- **矩阵乘法 (GEMM):** 就是两个矩阵相乘。它是深度学习里最核心、最耗时的计算, 所以这本书拿它当主角。

想提前补补这些底子, 可以翻第 0 章·极简入门; 第 9 章讲的 TIRx scope / layout / dispatch 模型本章也会用到, 但放心, 正文会把它当新词重新解释一遍。

## 本章导读: 从一个正确的 Tile 开始 (Building a Tiled GEMM)

这一章是整本书的转折点。前面几章一直在讲抽象——TIRx 的 scope / layout / dispatch 这套模型。从这里开始, 我们要动真格的: 把这套模型落到一个真实的内核 (kernel) 上, 亲手写出第一个正确的分块 GEMM(tiled GEMM)。

### 为什么整本书都围着 GEMM 转

#### 一句话先理解

**一句话先理解:** 矩阵乘法是深度学习里最耗时的计算, 所以「把矩阵乘法写快」这一件事, 好处会摊到整个模型上——这就是它配当主角的原因。

GEMM(通用矩阵乘法 / General Matrix Multiplication) 是现代机器学习里最核心的一个计算。先别被名字唬住, 它就是「两个矩阵相乘」, 跟你高中学的矩阵乘法是一回事, 只不过矩阵特别大。你看, 神经网络里的线性层、注意力 (attention) 里的各种投影、还有一大堆卷积的实现, 扒到底层全是一次矩阵乘法。说白了, 模型在 GPU 上跑的时间, 绝大部分都花在 GEMM 上。

正因为它这么重要, 「算得对的 GEMM」和「算得快的 GEMM」之间的差距就特别要命。这个差距是什么? 其实就是「让芯片大半算力干等着」和「把芯片喂得饱饱的」之间的差距。

打个比方: GPU 里有一种叫 **Tensor Core** 的专用硬件, 它是芯片里**专门负责做矩阵乘法**的小电路 (就像有的 CPU 里有专门做加密、做视频解码的专用单元一样)。这玩意儿算矩阵乘特别猛, 但它要发挥威力, 前提是数据得**及时喂到它嘴边**。代码写得好, Tensor Core 一刻不停地干活, 硬件就被榨干了; 写得差, 数据老是供不上, Tensor Core 就大量闲着干等——算力白白浪费。本章后面所有的折腾, 本质上都是在想办法「别让 Tensor Core 饿着」。

#### 注意

**注意:** 本章的目标只有一个字——**对**。性能 (performance) 留给后面两章。这里我们要的, 是一个能算出正确答案的最小可信基线 (baseline)。

### 为什么不一步到位, 而要分步搭建

先说一个贯穿全书的词: **内核 (kernel)**。在 GPU 编程里, 「kernel」指的就是**你写的、要丢到 GPU 上去并行跑的那一段函数**。你可以把它类比成一个普通函数, 只不过它一被启动, GPU 就会拿成千上万个线程**同时**去执行它。本章要写的那个 GEMM, 就是一个 kernel。

好, 回到正题。一个能跑满硬件的 GEMM 内核, 会一下子把好几件难事全压你身上: 内存搬运、K 维累加、分块 (tiling, 就是把大矩阵切成小块来处理, 后面细讲)、还有 Tensor Core 的调度。一上来就写这种内核, 等于逼你**同时调试这一整套东西**。更要命的是, 你手里连个可信的对照都没有, 真出了错, 根本说不清是哪一环坏的。

所以更稳的走法是: 先从「能算对的最小内核」开始, 然后**一次只改一个决策**, 慢慢往上长。每一步只动一个地方, 一出问题就知道准是这步引进来的。这种一点一点搭起来的做法, 就是本章——乃至后面两章——从头到尾的套路。

#### 结构图

分组:

- **不可取的做法：一步到位：**「内存搬运 + K 累加 + 分块 + Tensor Core 调度 / (同时出错, 无从对比)」
- **本章采用的做法：增量构建：**「最小正确内核」「处理完整 K 维」「处理完整尺寸矩阵」

连接关系：

- 最小正确内核  $\xrightarrow{+ K \text{ 维累加}}$  处理完整 K 维
- 处理完整 K 维  $\xrightarrow{+ \text{跨 CTA 的空间分块}}$  处理完整尺寸矩阵

本章的三步路线图

这里又冒出来一个关键词 **tile**(分块 / 瓦片)。它就是**从大矩阵里切出来的一个小方块**。为什么要切？因为真实矩阵动辄几千乘几千，根本不可能一次性塞进 GPU 那点宝贵的高速内存里；只能切成一块块小方块，一块一块地搬进来、算、再搬出去。这种「切块处理」的思路，就是后面反复出现的 **tiling**(分块)。

本章从一个  $128 \times 128$  的输出 tile(也就是输出矩阵左上角那个  $128 \text{ 行} \times 128 \text{ 列}$  的小方块) 起步，一步步长成能算完整尺寸矩阵的内核。三步各管一摊：

| 步骤     | 名称             | 引入的核心变化                                |
|--------|----------------|----------------------------------------|
| Step 1 | 单 Tile 顺序 GEMM | 只算一个输出 tile，建立后续一切修改所基于的基线             |
| Step 2 | K 维循环累加        | 增加沿 K 维度的循环累加，处理完整的收缩 (contraction) 维度 |
| Step 3 | 跨 CTA 的空间分块    | 在多个 CTA 之间做空间分块，覆盖完整矩阵                 |

一句话先理解

**一句话先理解：**表里第一次出现了 **CTA** 这个词。它是 **Cooperative Thread Array**(协作线程阵列) 的缩写，也常叫「**线程块 (thread block)**」。你就把它理解成一小队互相协作的线程 (本章里一队是 128 个)，这一队线程能共用同一块片上高速内存、彼此配合干活。GPU 启动一个 kernel 时，会同时拉起很多很多个这样的小队。Step 3 干的事，就是让每个小队 (CTA) 各算输出矩阵的一小块，大家并行铺满整张矩阵。

Step 1 搭的是基线，后面每一步都只是在它上面「改一个地方」而已。

把每一步读成「对一份契约的编辑」

理解这条路线，有个特别好使的角度：把每一步都当成在改**同一份契约 (contract)**。「契约」这个词你别想复杂，它就是「我们事先约定好，这个操作要按什么规矩来执行」。这份契约说白了就三条 (这三个词后面会一直用，先混个脸熟)：

| 条款         | 含义           | 大白话                                            |
|------------|--------------|------------------------------------------------|
| scope(作用域) | 哪个执行单位来跑这个操作 | 这活儿派给「谁」干？一个线程？一队线程 (warpgroup)？还是一大堆队 (整个网格)？ |

|              |                 |                                             |
|--------------|-----------------|---------------------------------------------|
| layout(布局)   | 操作数 tile 使用哪种布局 | 数据在内存里 <b>怎么摆</b> ?(摆法不对, 硬件就读不顺)           |
| dispatch(分派) | 走哪条分派路径来真正执行它   | 用 <b>哪条硬件通路</b> 去真正执行它?(比如用普通线程搬, 还是用专用硬件搬) |

大多数步骤, 主要也就动这三条里**的一条**。所以本章每一步开头, 我都会先给你一张小卡片, 点明这步到底改了哪一条, 顺带把「想安全复用, 还得补哪些同步」标清楚。

注意

**注意:**「scope / layout / dispatch」这条主线, 会一路贯穿到后面两章。你要是把每个新内核都读成「它在上一个内核上改了契约里的哪一条」, 会比把它当成一个全新内核轻松太多。

本章在三章优化路径中的位置

本章是「同一条 GEMM 优化路径」三连章里的**头一章**。这三章共用一个内核, 一层叠一层、特性越加越多, 而不是每章都推倒重来:

结构图

连接关系:

- 本章: 构建正确的分块 GEMM / (线程拷贝 + 朴素数据路径) → 下一章: 用 TMA 做流水线 / (TMA 异步搬运 + 软件流水线)

- 本章:** 把一个正确的分块内核搭出来, 然后**就停手**——性能一概不碰。
- 下一章 (用 TMA 做 GEMM 流水线):** 把朴素的线程拷贝换成 TMA(Tensor Memory Accelerator, 「张量内存加速器」, 一个专门负责异步搬数据的硬件单元——它最大的好处是搬数据时**不占用线程**, 线程腾出来可以去干别的)。再用「流水线 (pipelining)」让搬数据和算数据**叠着干**。这里的流水线就跟工厂流水线一个意思: 一边的人在搬下一批料, 另一边的人同时在加工上一批, 谁也不用站着等谁, 而不是「搬完一批 → 停下来加工 → 再去搬」这样一段一段串着来。
- 再下一章 (用 Warp 特化与 Cluster 扩展 GEMM):** 再往上加 warp 特化 (warp specialization, 让不同的 warp 各干一种专职, 比如这队专管搬数据、那队专管算、另一队专管写回结果, 有点像流水线上分工种) 和 CTA cluster(把多个 CTA 编成一个能协作的「集群」, 让它们合力干一件更大的活)。

每一章都踩在前一章的肩膀上, 内核一路**攒特性**, 而不是从零再来一遍。这就是「一次改一个决策」放大到章节这个尺度的样子。

GEMM: 问题定义与数据通路 (GEMM)

GEMM 就是稠密矩阵相乘, 它埋在几乎每一个深度学习算子的底下。线性层、注意力里的各种投影、很多卷积的实现, 扒到最后都是一次矩阵乘法。正因为它无处不在, **只要把 GEMM kernel 写快, 好处就会摊到整个模型上**——这也正是本章要从零搭一个分块 GEMM 的理由。

本章采用的形状约定

本教程的所有例子都算  $D = AB^T$ , 三个矩阵的形状和元素定义如下:

| 矩阵  | 形状           | 含义                       |
|-----|--------------|--------------------------|
| $A$ | $M \times K$ | 左操作数 (operand)           |
| $B$ | $N \times K$ | 右操作数                     |
| $D$ | $M \times N$ | 输出 (accumulator 最终写出的结果) |

逐元素的定义是沿着收缩维  $K$  求和:

$$D[m,n] = \sum_k A[m,k] \cdot B[n,k]$$

注意

**注意:** 这个转置  $B^\top$ , **不是我们额外多做的一步**, 它是数据本来就那么存出来的结果。看  $B$  的形状:  $N \times K$ , 也就是「 $N$  行, 每行  $K$  个数」。这恰好就是线性层权重平时的摆法。所以沿  $K$  维做收缩时, 顺着内存把  $B$  读下来, 读到的天然就是  $B^\top$ , **根本不用挪一个字节**。换句话说, 选  $D = AB^\top$  这个写法, 是为了贴着真实权重布局、让访存顺手, 而不是平白多算一次转置。

性能度量:TFLOPS

一句话先理解

**一句话先理解:**TFLOPS 就是「这个 kernel 每秒能做多少万亿次浮点加减乘」, 数越大越快。它是我们衡量「快不快」的统一尺子。

全书衡量一个 kernel 快不快, 统一看吞吐量 **TFLOPS**(每秒万亿次浮点运算;TFLOPS = Tera Floating-point Operations per Second, 其中 Tera 就是「万亿」)。算法很简单: 一次「乘加」记 2 次浮点运算 (乘一次算一次、加一次再算一次), 再拿总浮点运算量除以墙钟时间 (wall-clock time, 就是你拿秒表拍的真实流逝时间):

$$\text{TFLOPS} = \frac{2 \times M \times N \times K}{t_{\text{seconds}} \times 10^{12}}$$

这个公式有几个点值得记住:

- 分子  $2 \times M \times N \times K$  是**算法本身定死的浮点运算量**, 只跟问题多大有关, 你 kernel 怎么写它都不变。
- 分母里能动的只有  $t_{\text{seconds}}$  这一个量。所以后面所有优化, 归根结底都是一件事: **同样多的运算, 用更短的时间跑完**,TFLOPS 自然就上去了。
- 至于除以  $10^{12}$ , 纯粹是把单位从 FLOPS 换成 TFLOPS。

GEMM 数据通路 (GEMM Data Path)

一句话先理解

**一句话先理解:**GPU 的内存分好几层, 离计算越近的越快但越小。这一节就是带你看一份数据, 如何从「又大又慢又远」的那层, 一步步搬到「又快又近」的那层去算, 算完再搬回去。

为什么要先讲这个？因为本章后面**每一项优化**，说到底都在回答两个问题：**数据现在待在哪块存储里，以及它怎么从这块搬到下一块**。这一点对刚接触 GPU 的人最反直觉：你可能以为「算得快不快」主要看计算单元强不强，其实**绝大多数时候瓶颈是搬数据**——计算单元算得飞快，数据却供不上，它只能干等。所以动手写代码之前，先把数据流捋清楚，绝对值得。

往大处看，Blackwell(NVIDIA 的一代 GPU 架构，本章代码即面向它；同一系列前几代叫 Hopper、Ampere——你不用记，知道是「不同年份的 GPU 型号代号」就行) 上的一个 GEMM kernel 翻来覆去就干两件事：

- 1. **搬 (move)**: 在不同层级的存储之间搬 tile(数据分块);
- 2. **算 (compute)**: 对搬进来的 tile 做矩阵乘累加。

下面这张图，跟着一个 tile 从输入一路走到输出，把它路过的每一层存储都标出来。这是一条**基线 (baseline) 通路**。后面每一步优化，改的都只是这条链上「某一跳具体怎么走」，但**这些跳本身，一个都不增、一个都不删**。

结构图

连接关系:

- GMEM / (全局显存)  $\xrightarrow{\text{① 搬入操作数 A、B 的 tile}}$  SMEM / (共享内存)
- SMEM / (共享内存)  $\xrightarrow{\text{② MMA 消费 SMEM 里的操作数}}$  tcgen05.mma / (MMA 计算单元)
- tcgen05.mma / (MMA 计算单元)  $\xrightarrow{\text{③ 把 accumulator (累加结果) 写入}}$  TMEM / (张量内存)
- TMEM / (张量内存)  $\xrightarrow{\text{④ epilogue 把 TMEM 读回}}$  registers / (寄存器)
- registers / (寄存器)  $\xrightarrow{\text{⑤ 把结果存回 GMEM}}$  GMEM / (全局显存)

把这条从左到右的链记死，它的关键节点是：

| 阶段              | 动作                             | 源 → 目的地       |
|-----------------|--------------------------------|---------------|
| ① 加载操作数         | 操作数 tile 进片上                   | GMEM → SMEM   |
| ② MMA 计算        | tcgen05.mma 消费 SMEM 里的操作数      | SMEM → MMA 单元 |
| ③ 写累加器          | MMA 把累加结果 (accumulator) 写进张量内存 | MMA 单元 → TMEM |
| ④ epilogue(尾处理) | 把累加结果读回寄存器                     | TMEM → 寄存器    |
| ⑤ 存回结果          | 输出写回全局显存                       | 寄存器 → GMEM    |

这里冒出来的几个名词，后面会反复用，先有个直观印象。如果你熟悉 CPU 的「内存 → L2/L1 缓存 → 寄存器」这套由慢到快、由大到小的层级，那 GPU 这套其实是一个道理，只是名字不一样：

- **GMEM / 全局显存 (Global Memory)**: GPU 上容量最大、但离计算最远、最慢的那层内存——类比 CPU 世界里的**主内存 (RAM)**。你的输入矩阵和最终输出结果都放在这儿。带宽吃紧，所以「少往 GMEM 跑」是性能优化的永恒主题。
- **SMEM / 共享内存 (Shared Memory)**: 一个 CTA(那一小队线程) 内部共享的片上**高速存储**，又快又近，但很小——你可以把它类比成「一块由你手动管理的高速缓存」。和 CPU 缓存最大的不同是: CPU 缓存是硬件自动帮你装填的，你管不着；而 SMEM 里放什么、什么时候搬进来，**全得你自己在代码里写明白**。操作数 tile 得先从 GMEM 搬到 SMEM，计算单元才好读。
- **tcgen05.mma**: Blackwell 上执行 MMA 的那条具体硬件指令。**MMA = Matrix Multiply-Accumulate, 矩阵乘累加**，就是「把两块小矩阵相乘、再把结果加到一个累加器上」这个动作——真正干乘加活的就是它 (前面说的 **Tensor Core** 就是执行它的硬件)。



- **TMEM / 张量内存 (Tensor Memory):**Blackwell 这代 GPU 新加的一层存储, 专门用来放 MMA 的累加结果。MMA 算完, 结果直接写进这里待着。
- **寄存器 (registers):** 每个线程私有的、最快的一小撮存储——这个概念和 CPU 寄存器几乎一样, 区别只在于 GPU 上每个线程各有一份自己的寄存器。最后要写出结果时, 数据得先从 TMEM 落到寄存器, 线程才能动它。
- **epilogue / 尾处理段:**kernel 收尾的那一段代码, 负责把 TMEM 里的累加结果搬回寄存器(顺手可以做点缩放、激活函数之类的后处理), 再写出到 GMEM。「epilogue」原意就是「尾声」, 记成「收尾环节」即可。

#### 注意

**注意:** 后面 Step 1 / Step 2 / Step 3 的所有优化, 都不会动「GMEM → SMEM → MMA → TMEM → 寄存器 → GMEM」这条链的整体形状。它们改的只是**每一跳怎么走得更快**——比如换 TMA 异步搬、用 mbarrier 搭流水线、让多个 CTA 在空间上分担。先把这条不变的主干刻进脑子, 再看每一步在哪一跳上做文章, 一下就清楚了。

## 优化路线图 (Optimization Path)

前面讲的那条「朴素数据通路 (plain data path)」——把操作数从 GMEM 搬到 SMEM、用 Tensor Core 算出来、再写回 GMEM——确实能算出**对**的答案。可对不等于快。这条最省事的通路, 几乎让整块芯片都在干等: 搬数据的时候计算单元闲着, 算数据的时候拷贝引擎闲着, 大半硬件都没喂饱。

接下来要干的, 说白了就是: 在「数据要走哪几层内存」这条主链不动的前提下, 一项一项把 Blackwell 的硬件本事补齐, 每次只加一个特性, 把那些空转的硬件填满。**每个特性都对应一个 TIRx 的 tile primitive(tile 原语)**。这样就不用从零重写 kernel, 只在一份代码上「一次改一处」地往前推。

#### 注意

**注意:** 这一小节其实是后面两章的「目录预览」。本章 (Step 1-3) 只管把正确的分块 GEMM 搭出来; 下面这六个优化特性, 会在《用 TMA 做 GEMM 流水线》和《用 warp 特化与 cluster 扩展 GEMM》里一个一个落地。读到这儿, 你心里有个「接下来往哪走」的地图就够了。

### 为什么要分步走, 而不是一步到位

朴素通路最大的好处, 是它**靠得住**: 它给的是一个正确结果, 后面所有优化都拿它当对照基准 (ground truth)。要是上来就写个能跑满芯片的 kernel, 你就得同时盯着内存搬运、累加、分块、Tensor Core 调度这一大坨, 而且手里连个可信的参照都没有, 错了都不知道找谁对。

所以这里的打法是: **先弄一个能算对的最小 kernel, 然后每次只加一个特性**。每加一项, 都能单独验两件事——「它有没有把答案搞坏」「它有没有带来该有的提速」。问题被圈在一个小范围里, 排查起来才不抓瞎。

### 六个优化特性概览

下面这六项, 按加入的先后排好, 正是后面两章要走的路。它们是一层叠一层, 不是谁替换谁。

提示

提示: 这张表里的术语 (TMA、流水线、warpgroup……) 现在看不懂**完全正常**, 它们都是后面两章的内容, 这里只是给你一张「接下来要往哪走」的地图。你扫一眼有个印象就行, 千万别卡在这儿。每一项的真正讲解, 都在它对应的那一章。

| 顺序 | 特性                                | 核心作用                                                                           | 解决的问题                                       |
|----|-----------------------------------|--------------------------------------------------------------------------------|---------------------------------------------|
| 1  | TMA 异步搬运 (TMA async movement)     | 用 Blackwell 的硬件拷贝通路在 GMEM ↔ SMEM 之间搬运 tile, 并用 mbarrier 跟踪拷贝完成                 | 让数据搬运不再占用线程, 可与计算解耦                         |
| 2  | 软件流水线 (Software pipelining)       | 用多个 SMEM stage(多缓冲), 让”取下一个 K tile”与”对当前 K tile 做 Tensor Core 计算”在时间上重叠        | 消除”搬完才能算、算完才能搬”的串行等待                        |
| 3  | 持久化调度 (Persistent scheduling)     | 维持一个固定数量的 CTA 池, 每个 CTA 通过 tile scheduler 连续处理多个输出 tile, 而不是”一个 tile 启动一个 CTA” | 减少 CTA 启动/销毁开销, 提高占用率                       |
| 4  | Warp 特化 (Warp specialization)     | 把生产者 (producer)、MMA 消费者 (MMA consumer)、写回 (write-back) 三种角色拆分到不同的 warpgroup 上  | 让不同职责的 warp 各司其职、并行流转                       |
| 5  | CTA 集群 (CTA clusters)             | 让两个 CTA 协作, 共同处理一个更大的 Blackwell MMA tile                                       | 突破单 CTA 的 tile 规模上限, 提升单次 MMA 的有效尺寸         |
| 6  | 多消费者执行 (Multi-consumer execution) | 用多个消费者 warpgroup 同时计算 tile 的不同部分                                               | 进一步提高计算密度 (compute density), 榨干 Tensor Core |

这些特性是如何叠加的

这六项, 你可以当成对同一条数据通路的「一层层加料」。从最素的「线程拷贝 + 单 stage + 单 CTA 单 tile」起步, 一路堆到「硬件异步搬运 + 多缓冲流水 + 持久化 + 角色特化 + 跨 CTA 协作 + 多消费者并行」:

结构图

连接关系:

- 朴素通路 (本章 Step 1-3) / 线程拷贝 GMEM→SMEM, 单 stage, 单 CTA 处理单 tile → + TMA 异步搬运 / (搬运交给硬件拷贝通路,mbarrier 跟踪完成)

注意

注意: 整条路线都绕着同一组「契约三要素」转——scope(谁来执行)、layout(操作数 tile 用什么布局)、dispatch(走哪条派发路径)。后面每一步, 基本都只是在这份契约上改一处: 动一个主要的点, 再顺带交代「为了安全复用还得补哪些同步」。本章的 Step 1 负责把这份契约的**基线版本**立起来, 之后的优化全是在这个基线上「打补丁」。

第 1 步: 单 Tile 顺序 GEMM(Step 1: Sequential Single-Tile GEMM)

一句话先理解

一句话先理解:Step 1 的目标是写一个小到极致、但五脏俱全的 GEMM——只算一个小方块,简单到一个循环都没有,好让你把数据走的每一步看得清清楚楚。

从零搭一个 GEMM, 聪明的做法不是一上来就奔着高性能去, 而是先写一个能把整条硬件数据通路完整跑一遍、却简单到一个循环都没有的版本。Step 1 就是这么个「最小可运行内核」: 它只算一个  $128 \times 128$  的输出 tile,K 也只有 64(也就是 A 是  $128 \times 64$ 、B 是  $128 \times 64$ , 输出是  $128 \times 128$  的小方块)。规模小到什么都不用循环——数据通路上的每一段 (GMEM、SMEM、TMEM、寄存器) 都恰好只出现一次。这样我们就能把每一跳 (hop, 就是数据从一层内存搬到下一层这「一步」) 单独看个明白, 不必同时跟循环、流水线缠在一起。

这一步确立了什么: 基线契约

写代码之前, 先把这个内核的「形状」说定。它定下来的, 就是后面每一步都要沿用的基线 (baseline):

| 维度              | 约定                                                                                               |
|-----------------|--------------------------------------------------------------------------------------------------|
| 范围 (Scope)      | 单个 warpgroup(4 个 warp = 128 个线程组成的执行单位; 一个 warp 是 32 个线程的小班, 见第 0 章) 顺序地走完整条通路, 一个阶段接一个阶段, 中间不重叠 |
| 数据布局 (Layout)   | A、B 两个操作数 tile 放在 SMEM; 累加器 (accumulator) 放在 TMEM; 结果经由寄存器 (register) 暂存后写回                      |
| 派发方式 (Dispatch) | 同步的 Tx.copy 负责搬运数据;tcgen05 负责执行 MMA(矩阵乘加)                                                        |

注意

注意: 这里特意挑了「同步搬运 + 顺序执行」, 而不是 TMA / 流水线。就是想先把正确性和数据通路的全貌讲透。性能优化是后面一项一项加的, 不归 Step 1 管。

单 Tile 的数据流向 (Single-Tile Dataflow)

基线契约定好之后, 下一件要钉死的, 是一个 tile 沿这条通路走的先后顺序。第一个内核把核心 GEMM 数据通路完整走一遍——也就是经典的  $GMEM \rightarrow SMEM \rightarrow TMEM \rightarrow \rightarrow GMEM$ ——外面一个循环都不套。它先分配工作内存, 加载操作数, 算出乘积, 写回结果, 最后收拾干净走人。

整个流程长这样:

结构图

连接关系:

- Asmem in SMEM / swizzled 布局  $\rightarrow$  tcgen05 MMA / 单线程发起 tile op
- Bsmem in SMEM / swizzled 布局  $\rightarrow$  tcgen05 MMA / 单线程发起 tile op
- tcgen05 MMA / 单线程发起 tile op  $\rightarrow$  fp32 accumulator /  $128 \times 128$  in TMEM
- fp32 accumulator /  $128 \times 128$  in TMEM  $\xrightarrow{\text{tcgen05.ld} / \text{异步加载}}$  寄存器 / 每线程 1 行

- 寄存器 / 每线程 1 行  $\xrightarrow{\text{cast fp32} \rightarrow \text{fp16}}$  D in GMEM / fp16 输出

对应的五步是:

1. **分配 (Allocate):** 开出 SMEM(用一个「池式分配器 / pool allocator」来划地盘, 概念上就跟你写代码时 `malloc` 一块内存差不多)、TMEM, 再加一个 `mbarrier`。这里第一次出现 `mbarrier`, 先解释一下: 它是 **memory barrier(内存屏障)** 的简称, 你可以把它理解成一个线程之间用的「信号灯 / 计数器」——某件事 (比如「MMA 算完了」) 做完时给它发个信号, 而其他在等这件事的线程就卡在它跟前, 等信号到了才放行。它是 GPU 上做线程间同步的关键工具, 本章后面会反复打交道。
2. **加载 (Load):** 128 个线程一起协作, 把 A、B 两个 tile 从 GMEM 拷到 SMEM(同步 `Tx.copy`)。「一起协作」意思是这 128 个线程各搬一小片, 凑起来才是完整的一块, 而不是某一个线程吭吭吭把整块搬完——这正是 GPU「人多力量大」的典型干法。
3. **计算 (Compute):** 挑一个线程出来发起 `Tx.gemm_async`(发起矩阵乘)+ `tcgen05.commit`(告诉硬件「算完了记得发信号」); 其他所有线程则卡在 `mbarrier` 上等结果出来。
4. **写回 (Writeback):** `warpgroup` 把结果从 TMEM 读进寄存器; 每个线程把 fp32 转 (cast) 成 fp16, 再写到 GMEM。
5. **释放 (Deallocate):** 把 TMEM 还回去。

## 第一个内核的四个部件 (Four Pieces)

完整内核就几十行, 但拆成四块来读更好啃: 内存分配、同步加载、MMA 派发、写回。这里用到的 API, 名字全来自 Part II 讲过的 TIRx tile primitive(分块原语)。

### 部件一: 内存分配

内核一开头, 先给操作数划出共享内存, 再给 TMEM 地址和 `mbarrier` 各留一个槽位:

```
pool = T.SMEMPool()
tmem_addr = pool.alloc((1,), "uint32") # TMEM (4)
mma_bar = pool.alloc((1,), "uint64", align=8) # mbarrier(8 , 8)
pool.move_base_to(1024) # 1024
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout) # 128x64 fp16
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout) # 128x64 fp16
pool.commit()
```

这段代码里有几个 GPU 专有的写法, 逐行说下它在干嘛:`pool.alloc(...)` 就是从 SMEM 这块地盘里划出一小段来用, 跟你平时申请一块缓冲区一样;`Asmem`、`Bsmem` 就是给 A、B 两个 tile 在 SMEM 里准备的落脚处;`pool.commit()` 表示「地盘划完了, 定下来」。

有两个细节值得多想一下:

- `pool.move_base_to(1024)` 为什么要「跳到偏移 1024」? 它把两块大的操作数 tile(`Asmem`、`Bsmem`) 推到 1024 这个偏移之后, 把前面那一小段低地址留给几个小东西 (TMEM 地址槽、`mbarrier`)。为什么非这么干? 因为搬运和计算硬件对数据的起始地址有对齐要求——好比有些硬件要求数据必须从「整 1024 字节的边界」开始放, 放歪了它就读不动。这种「对齐」要求在底层编程里很常见, GPU 这里尤其严格。

- **layout=A\_layout 为什么非得是 swizzled(交错) 布局?** 先说 **swizzle** 是什么: 它是一种「故意打乱顺序」的内存摆放方式。你可能觉得数据老老实实按行存最直观, 但 GPU 的硬件 (尤其多个线程同时读同一块 SMEM 时) 有个毛病: 如果大家都去挤同一条访问通道, 就会排队、变慢 (这种现象叫 bank conflict / 存储体冲突)。swizzle 就是把数据按一种特定规律错开摆, 让并发访问刚好均匀分散到不同通道, 避开堵车。这里向 **tma\_shared\_layout** 要的, 正是这样一种 swizzle 过的摆法, 好让搬运硬件和计算硬件都能**直接、高效地读**。这正是 Part II 一再强调的「布局即契约 (layout-as-contract)」: 数据怎么摆不是随便的, 它得同时迎合搬运硬件和计算硬件的胃口。

## 部件二: 同步加载

缓冲区开好了, 操作数还得真搬进 SMEM。第一版, 我们就让 CTA(协作线程阵列 / Cooperative Thread Array) 自己的线程去拷:

```
Tx.cta.copy(Asmem[:, :], A[:, :]) # CTA ,
Tx.cta.copy(Bsmem[:, :], B[:, :])
T.cuda.cta_sync() # , SMEM
```

因为这里就一个 tile( $M = N = 128, K = 64$ ), 把整个 A 和 B 拷过去, 加载就算干完了。**Tx.cta.copy(...)** 让整个 CTA 一起协作拷, 每个线程管自己那一片。

紧跟着的 **T.cuda.cta\_sync()** 一身二职: 它既等每个线程都拷完, 又把它们写进共享内存的内容发布 (**publish**) 出去。这样后面 MMA 去读 **Asmem**、**Bsmem** 时, 看到的才是一个填满的完整 tile, 而不是只填了一半的缓冲区。

### 注意

**注意:** 这种「线程自己来拷」的做法, 是我们头一个要换掉的东西。下一章 (用 TMA 做 GEMM 流水线) 就把它换成 TMA。这里先留着它, 纯粹是为了让 Step 1 简单直白。

## 部件三:MMA 派发

操作数现在已经躺在 SMEM 里了, 可以发起 MMA 了。注意: 发起它的, 只用挑出来的那一个线程:

```
if warp_id == 0: # : warpgroup warp 0
 if T.ptx.select_sync(): # : warp lane
 Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=False, dispatch="tcgen05", cta_group=1)
 T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
```

逐行解释下这段:**warp\_id** 是「当前线程属于第几个 warp」, **select\_sync()** 是一个 GPU 专用函数, 作用是「在一个 warp 的 32 个线程里, 挑出唯一一个来当代表」(elect 就是「选举」)。还有个词 **lane**, 它就是一个线程在自己所在 warp 里的编号 (0 到 31)——你把「lane」直接读成「线程」基本不会错。

所以这两层嵌套的 if, 是在一层层把「发起者」往小里掐: 外层 **if warp\_id == 0** 先只留下 warpgroup 的第 0 个 warp (把另外 3 个 warp 全挡在外面), 内层 **if T.ptx.select\_sync()** 再在这个

warp 的 32 个线程里挑出唯一一个。两层一叠,128 个线程最后就剩 1 个去跑 `Tx.gemm_async` 和 `tcgen05.commit`。

这里有个特别容易理解错的地方,得先掰开说清楚:

#### 注意

**注意:**「一个线程发起」绝不等于「一个线程在做乘法」。计算照样是一个完整的、tile 级别的 MMA。硬件会照着 SMEM 操作数布局和 TMEM 累加器布局,一起协作把整个 tile 的乘法做掉。关键就一句:`Tx.gemm_async` 是一个 **tile 操作 (tile operation)**,不是一条硬件指令。

为什么这么说? 因为  $K = 64$  的 tile,比硬件 MMA 的  $K$  原子 ( $MMA\_K = 16$ ) 宽多了。所以这一个 tile op 会被降级 (lower) 成一小串沿  $K$  维往前走的原始 `tcgen05.mma` 指令,整个 warpgroup 一起协作把每一条都驱动起来。

那为什么发起这串指令,只用一个线程? 因为底层每一条 `tcgen05.mma` 本身就是一个一发起就协作的操作:发一次,就能驱动整个 tile MMA 里的那个  $K$  原子。要是让 128 个线程都去发这串序列,同一份活就被重复发了 128 遍——纯属浪费。

最后,`accum=False` 这个标志是告诉 MMA: **直接覆盖 (overwrite)** TMEM 目标,而不是往里累加 (**add into**)。这里我们要的就是覆盖,因为压根没有上一轮的部分和 (partial sum) 要接着用。

#### 部件四: 写回 (Writeback)

##### 一句话先理解

**一句话先理解:** 结果现在还困在 TMEM 里、而且是高精度的 fp32; 写回前要把它「降级」成 fp16 再送回 GMEM。这一段就是干这个搬运 + 转换的活。

先说两个数据类型词:**fp32** 是 32 位浮点数 (精度高、占 4 字节),**fp16** 是 16 位浮点数 (精度低一半、只占 2 字节)。深度学习里常用 fp16 存数据来省内存、提速,但在累加一大堆数的时候用 fp32 更稳——这个取舍下面就会用到。

乘积现在躺在 TMEM 里,可调用方要的,是回到 GMEM、而且是 fp16 的结果。所以收尾阶段 (epilogue) 得把结果经寄存器带下来,顺手把类型从 fp32 转成 fp16:

```
Dreg = T.alloc_local((BLK_N,), acc_type) # fp32
Dreg_f16 = T.alloc_local((BLK_N,), d_type) # , fp16
Dreg_wg = Dreg.view(128, BLK_N, # warpgroup
 layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)]))
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # :TMEM -> (tcgen05.ld)
T.ptx.tcgen05.wait.ld() # ,
Tx.cast(Dreg_f16[:, :], Dreg[:, :]) # fp32 -> fp16
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id) #
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])
```

**累加器为什么用 fp32?** MMA 在 TMEM 里留下的,是一个  $128 \times 128$  的 **fp32** 累加器 tile。这个 fp32 是特意选的。GEMM 要沿  $K$  维把一大堆乘积加起来,用更高的精度存这个「运行和 (running sum)」,能把本来会越来越大的舍入误差摁住。但输出 D 是 fp16,所以这些值不能直接出去:得先落到寄存器,在那儿收窄 (narrow) 成 fp16,然后才能写到 GMEM。

**两个寄存器缓冲各管什么?** Dreg 是个每线程一份、长度为 BLK\_N 的缓冲;Dreg\_wg 则是同一批寄存器换了个布局后的 warpgroup 级视图 (view):

```
TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)])
```

这个布局把 tile 的第一维摊到 warpgroup 的线程上: 线程 0 拿第 0 行, 线程 1 拿第 1 行, 一直到第 127 行。第二维则留在每个线程自己的寄存器缓冲里, 所以每个线程攥着自己那一行的全部列。warpgroup 有 128 个线程,tile 有 128 行, 于是这个  $128 \times 128$  的输出正好一人一行, 分得清清楚楚。

照这个视图把累加器读出来, 就是 Tx.wg.copy\_async(Dreg\_wg, tmem) 干的活; 它会降级成 Blackwell 的 TMEM 加载通路 tcgen05.ld。这个加载是异步的, 所以在任何线程去碰 Dreg 之前, 都得先等 T.ptx.tcgen05.wait.ld() 返回。不然线程就会读到加载还没填好的寄存器。

等这个 wait 返回, 每个线程私有的 Dreg[:] 里, 就装着它那一行逻辑输出的 fp32 值了。线程把它们收窄成 fp16 存进 Dreg\_f16, 再算出自己该写哪个全局行:

```
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
```

然后写到 D[m\_thr, n\_st:n\_st + BLK\_N]。这些行在四个 warp 之间分得很整齐:

| Warp   | 负责写的行      |
|--------|------------|
| warp 0 | 第 0-31 行   |
| warp 1 | 第 32-63 行  |
| warp 2 | 第 64-95 行  |
| warp 3 | 第 96-127 行 |

完整内核 (Complete Kernel)

现在把四个部件拼成一个能跑的完整内核 ( $M = N = 128, K = 64$ )。先看导入:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg
```

内核照后面各步统一的 hgemm\_vX(M, N, K) 样式包起来。Step 1 跑  $M = N = 128, K = 64$ , 所以整个启动 (launch) 里就一个输出 tile。下面是内核主体, 关键的地方都加了中文注释:

```
def hgemm_v1(M, N, K):
 a_type = tvm.DataType("float16"); b_type = tvm.DataType("float16")
 d_type = tvm.DataType("float16"); acc_type = tvm.DataType("float32")
 BLK_M, BLK_N, BLK_K = 128, 128, 64
 # MMA_M/N/K MMA tile ; gemm_async
 # (/ tile MMA),
 MMA_M, MMA_N, MMA_K = 128, 128, 16
 # A B swizzled SMEM ,TMA tcgen05.mma
 A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_M, BLK_K))
 B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_N, BLK_K))

 @T.prim_func
 def kernel(A: T.Buffer((M, K), a_type),
```

```

 B: T.Buffer((N, K), b_type),
 D: T.Buffer((M, N), d_type)):
T.device_entry()
tile :M=BLK_M, N=BLK_N, grid 1x1; CTA tile
(m_st, n_st) 0 Step 3+ M/N
bx, by = T.cta_id([M // BLK_M, N // BLK_N])
wg_id = T.wargroup_id([1]) # wargroup, wg_id 0()
warp_id = T.warp_id_in_wg([4]); lane_id = T.lane_id([32])

--- SMEM ()---
pool = T.SMEMPool()
tmem_addr = pool.alloc((1,), "uint32")
mma_bar = pool.alloc((1,), "uint64", align=8)
pool.move_base_to(1024)
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
pool.commit()

--- + TMEM (warp 0)---
if warp_id == 0:
 if lane_id == 0:
 T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # 1
 T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512, cta_group=1)
 T.ptx.fence.proxy_async("shared::cta") # SMEM
 T.ptx.fence.mbarrier_init() # mbarrier
 T.cuda.cta_sync()

TMEM (alloc tmem_addr)
tmem = T.decl_buffer((128, 512), "float32", scope="tmem",
 allocated_addr=tmem_addr[0],
 layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))
m_st = T.meta_var(bx * BLK_M); n_st = T.meta_var(by * BLK_N)
phase_mma: T.int32 = 0

--- : GMEM -> SMEM()---
tile , Step 3 diff
Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :])
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync()

--- : MMA()---
if warp_id == 0:
 if T.ptx.select_sync():
 Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=False, dispatch="tcgen05", cta_group=1)
 T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
 T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # MMA

--- :TMEM -> -> GMEM()---
Dreg = T.alloc_local((BLK_N,), acc_type)
Dreg_f16 = T.alloc_local((BLK_N,), d_type)
Dreg_wg = Dreg.view(128, BLK_N,
 layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)]))
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
T.ptx.tcgen05.wait_ld()
Tx.cast(Dreg_f16[:, :], Dreg[:, :])
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)

```



```

Tx.copy(D[m_thr, n_st : n_st + BLK_M], Dreg_f16[:])

--- TMEM ---
T.cuda.cta_sync()
if warp_id == 0:
 T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
 T.ptx.tcgen05.dealloc(tmем_addr[0], n_cols=512, cta_group=1)

return kernel

```

### 注意

**注意:** `mbarrier.init(..., 1)` 的期望计数 (arrival count) 给的是 1, 因为要等的事件只有「MMA 完成」这一个。`tcgen05.commit` 会在整串 MMA 跑完后给这个 `mbarrier` 发个信号, 全体线程则卡在 `mbarrier.try_wait` 上, 等它翻 phase(相位)。

## 配套的编译与校验脚手架

后面每个 GEMM 步骤, 编译、运行、自检的套路都一样。所以这套脚手架在这儿**完整写一遍**, 之后就只贴内核本身。想跑哪个后续步骤, 把它对应的 `hgemm_vX` 和对应的问题规模换进来就行。

```

import torch
target = tvn.target.Target("cuda"); device = torch.device('cuda')
M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
 ex = tvn.compile(tvn.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")

A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)
D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)
ex.mod(A_tensor, B_tensor, D_tensor) # torch

D_ref = (A_tensor.float() @ B_tensor.float().T).half()
: K , K
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")

```

### 注意

**注意:** 有个坑容易踩——一个新的 Python 会话只能编译一个步骤, 换步骤之前得重启。原因是这些例子复用了内部名称, 而编译器又攥着一份按会话 (per-session) 走的状态。

Step 1 到 Step 3 都故意用很小的规模 (这里  $128 \times 128$ , Step 3 用  $256^3$ ), 就是为了让最开始这几次讲解好跟。后面「用 Warp 特化与 Cluster 扩展 GEMM」一节末尾那张跨步骤端到端结果表则反过来: 它把**每一个**步骤 (包括 Step 1 这套算法) 都放到统一的  $M = N = K = 4096$  规模下跑一遍, 这样各步骤之间的加速比才能直接比。

## 单 Tile 内核的局限 (Limits)

这个内核是**对**的——这正是 Step 1 想要的。但它的「对」只在一个特别窄的设定下才成立。有四个局限是**故意**埋进去的, 后面的优化路径会一个一个把它们拆掉:

| # | 局限                                          | 由哪一步解决（方向）             |
|---|---------------------------------------------|------------------------|
| 1 | 只处理 <b>单个 K tile</b> ，无法在大 K 上做收缩（contract） | 引入 K 维循环               |
| 2 | 只处理 <b>单个输出 tile</b> ，M、N 被钉死在 128          | 推广到多 tile / 网格（Step 3） |
| 3 | 用 <b>同步</b> 的 GMEM → SMEM 拷贝，而非 TMA         | 下一章换成 TMA              |
| 4 | 数据搬运与计算 <b>不重叠</b> ，两者永远不会同时进行              | 引入流水线（pipelining）      |

第 2 步:K 维循环累加 (Step 2: K-Loop Accumulation)

一句话先理解

一句话先理解: 真实矩阵的 K 维 (就是被「乘进去再加起来」的那个维度) 往往很长,Step 1 只能吃 64 那么一截。Step 2 就是套个循环,把 K 一截一截吃完、结果累加到一起。

先点明 **K 是什么维度**: 回顾  $D[m,n] = \sum_k A[m,k] \cdot B[n,k]$ , 这里被求和号  $\sum_k$  消掉的那个  $k$ , 就是 **K 维 (收缩维度 / contraction dimension)**。「收缩」是说:A 的一行和 B 的一行,要沿着 K 把对应元素乘起来再全加成一个数——这个维度在结果里「缩没了」。K 越长,要加的项越多。

第 1 步只能啃一个宽 64 的 K 切片 (K tile), 可真实矩阵的收缩维度常常远不止 64(几千都正常)。第 2 步要补的,就是这个最直接的短板: **输出 tile 还是只算一块,但让 K 维能横跨任意多个 64 宽的小块**。

思路其实很素: 把「加载 → MMA → 等待」这一套照 K 的块数重复跑,每次 MMA 都累加进**同一个 TMEM 槽位**。难点不在算,在**同步**。同一个 mbarrier 跨迭代反复用,就会冒出本章头一个真正的正确性陷阱: 代码要是把相位 (phase) 跟丢了,wait 可能在它该等的那次 MMA 还没算完时就提前放行,于是**不声不响**地算出错误结果。下面我们就把这个错怎么发生、又怎么躲开,讲个透。

这一步改变了什么: 复用既有布局

| 维度                   | 第 2 步的变化                                                                                                           |
|----------------------|--------------------------------------------------------------------------------------------------------------------|
| 作用范围 (Scope)         | <b>不变</b> , 仍然是单个 warpgroup。                                                                                       |
| 布局/复用 (Layout/reuse) | 同一对 SMEM tile 和同一个 TMEM 累加器槽位在整个 K-loop 中 <b>反复复用</b> 。不再分配任何新存储: 操作数 tile 像「流水」一样穿过这对固定缓冲区, 累加状态始终待在同一个 TMEM 槽位里。 |
| 同步 (Synchronization) | 被复用的 MMA barrier 必须在 <b>每个 K 块</b> 上正确推进相位, 否则后续的 wait 会观察到更早一次的完成信号。                                              |
| 调度 (Dispatch)        | <b>不变</b> 。                                                                                                        |

说到底, 第 2 步没添一个字节的新存储, 它全部的复杂度就压在一件事上: 「**怎么安全地把一个 mbarrier 反复用**」。

K-loop 的运行机制

第 1 步只在一个宽 64 的 K tile 上做收缩。这回我们保留它那唯一的输出 tile, 但放开 K, 让它按矩阵真实的需要爱伸多长伸多长。做法是以 **BLK\_K=64** 为步长沿 K 往前走, 每一轮把下一块 A、B 的

K 切片加载进 SMEM, 然后发起一次 `Tx.gemm_async`。

把这些零碎小块「缝」成一个完整点积, 靠的就是 `accum` 这个标志:

```
Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=(i != 0), # 0 :accum=False, TMEM
 dispatch="tcgen05", cta_group=1)
```

- 第一块 (`i==0`):`accum=False`, 把 TMEM 累加器初始化 (直接写进去, 旧值丢掉)。
- 后面每一块 (`i!=0`):`accum=True`, 把这一块的乘积加到已经躺在 TMEM 里的运行总和上。

同步: 相位翻转 (phase flip) 才是真正的考点

关键

关键: 先记住一句话——同一个 `mbarrier` 被反复用, 所以软件得自己记住「这一轮我该等它翻到哪一相」, 记错了就等错。

每次 MMA 完成, 我们用的都是**同一个** `mbarrier`。想安全地反复用它, 核心就一件事: 盯住「我们现在到底在等哪个相位」。

`mbarrier` 内部带一个 **1 比特的相位** (0 或 1)。每凑齐一次预期的到达 (arrival), 这个相位就翻到另一个值。微妙就微妙在 `try_wait` 的判定条件上:

注意

注意:`try_wait(bar, phase)` 会一直堵着, 直到 `barrier` 的内部相位跟你传进去的 `phase` 不一样为止。换句话说, 你传进去的那个参数, 代表的是「你预期马上要离开的那个旧相位」, 而不是「你想等到的那个新相位」。

下表把三次迭代的相位变化完整列出来:

| K 迭代 | 等待前本地的 phase_mma | try_wait 在等什么 | 等待后本地的更新      |
|------|------------------|---------------|---------------|
| 0    | 0                | barrier 翻到 1  | phase_mma = 1 |
| 1    | 1                | barrier 翻到 0  | phase_mma = 0 |
| 2    | 0                | barrier 翻到 1  | phase_mma = 1 |

让这张表一直对得上的, 就是循环里那行毫不起眼的代码:

```
T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # :barrier != phase_mma
phase_mma ^= 1 # ,
```

下面这张表, 把「本地 `phase_mma`」和「`barrier` 真实相位」怎么一块儿往前走, 摆得明明白白:

| 迭代   | 等待前本地 phase_mma | MMA 完成后 barrier 翻转 | try_wait 判定         | 放行 | ^=1 后本地更新 |
|------|-----------------|--------------------|---------------------|----|-----------|
| 迭代 0 | 0               | 0 → 1              | try_wait(.,0):0 ≠ 1 | 放行 | 1         |
| 迭代 1 | 1               | 1 → 0              | try_wait(.,1):1 ≠ 0 | 放行 | 0         |

|      |   |                   |                                           |    |   |
|------|---|-------------------|-------------------------------------------|----|---|
| 迭代 2 | 0 | $0 \rightarrow 1$ | <code>try_wait(..,0):0</code><br>$\neq 1$ | 放行 | 1 |
|------|---|-------------------|-------------------------------------------|----|---|

要是把 `phase_mma ^= 1` 删了会怎样？这就是这一步最该警惕的 bug:

- 第二轮迭代还是去调 `try_wait(bar, 0)`;
- 可 `barrier` 在第一次 MMA 之后早翻到相位 1 了;
- 于是 `try_wait` 一进来就发现「当前相位  $1 \neq$  参数 0」, 当场返回——可这会儿第二次 MMA 压根还没算完;
- 内核接着就去读一个**半成品**的累加器, 报出一个错误答案, 却一声不吭、不抛任何错。

结构图

连接关系:

- 错误时序 (缺少 `phase_mma ^= 1`) / 迭代 1:`try_wait(bar, 0) → barrier` 当前相位 = 1(上一轮 MMA 留下的)

注意

**注意:** 这是一个能**正常编译、正常跑**、却给出错误数值的 bug——不崩、不报错。正因为它这么藏得深, 这一行相位翻转才值得花这么多笔墨。一句话: 复用 `mbarrier` 的时候, 「盯住相位」永远是头等大事。

完整内核

完整内核说穿了就是「第 1 步 + K-loop + 相位翻转」叠到一起。导入部分跟之前一模一样:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg
```

内核包在 `hgemm_v2(M, N, K)` 里。网格 (grid, 本次启动里所有 CTA 排成的阵列;`[1, 1]` 即只有一个 CTA) 还是 `[1, 1]`, 因为我们依旧只算一个输出 tile。变大的只有它的 **K 跨度**:

```
BLK_M, BLK_N, BLK_K = 128, 128, 64
K_TILES = K // BLK_K # K 64
```

下面只摘跟第 2 步直接相关的核心结构, 长实现精简掉了:

1) 初始化 (照搬第 1 步): 分配 **TMEM**、初始化 `mbarrier`, 再做一次跨 CTA 同步。

```
if warp_id == 0:
 if lane_id == 0:
 T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # mbarrier =1
 T.ptx.tcg05.alloc(T.address_of(tmем_addr), n_cols=512, cta_group=1) # TMEM
```

```
T.ptx.fence.proxy_async("shared::cta")
T.ptx.fence.mbarrier_init()
T.cuda.cta_sync()
```

```
phase_mma: T.int32 = 0 # , 0
```

2)K-loop 主体 (这步的核心): 每一块都走一遍「加载 → 同步 → MMA → 等待 → 翻相位」。

```
for i in T.serial(K_TILES): # , K A/B
 # i K SMEM()
 Tx.cta.copy(Asmem[:, :], A[:, i*BLK_K:(i+1)*BLK_K])
 Tx.cta.copy(Bsmem[:, :], B[:, i*BLK_K:(i+1)*BLK_K])
 T.cuda.cta_sync()

 if warp_id == 0:
 if T.ptx.select_sync(): # lane MMA
 Tx.gemm_async(tmemb[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=(i != 0), # ,
 dispatch="tcgen05", cta_group=1)
 T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1) # MMA barrier

 T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # MMA
 phase_mma ^= 1 # (!)
```

3) 写回 (跟第 1 步一字不差): 从 TMEM 取出累加结果,cast 成 fp16, 写回 D, 最后把 TMEM 释放掉。

```
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # TMEM ->
T.ptx.tcgen05.wait.ld()
Tx.cast(Dreg_f16[:, :], Dreg[:, :]) # fp32 -> fp16
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :]) #
...
if warp_id == 0:
 T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
 T.ptx.tcgen05.dealloc(tmemb_addr[0], n_cols=512, cta_group=1) # TMEM
```

整个内核的数据流, 大致可以这么概括:

结构图

分组:

- **K-loop(i = 0 .. K\_TILES-1):**「A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta\_sync)」 「TMEM 累加器槽 (跨迭代复用)」 「try\_wait(phase\_mma) + phase\_mma ^= 1」

连接关系:

- A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta\_sync)   
 gemm\_async(accum = i!=0) 累加进同一个 → TMEM 累加器槽 (跨迭代复用)
- TMEM 累加器槽 (跨迭代复用) → try\_wait(phase\_mma) + phase\_mma ^= 1
- 全局内存 A, B → A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta\_sync)

- K-loop( $i = 0 \dots K\_TILES-1$ )  $\xrightarrow{\text{循环结束后, TMEM 中是完整点积}}$  寄存器  $\rightarrow$  cast  $\rightarrow$  fp16  $\rightarrow$  全局内存 D

你看，第 2 步在结构上对第 1 步的改动小得可怜：套一层 K-loop、把 `accum` 改成看块号、再加上 `phase_mma ^= 1` 那一行。可偏偏就是这一行，划开了「能跑」和「能跑对」这道分水岭。

### 第 3 步：空间分块，多 CTA 并行 (Step 3: Spatial Tiling / Multi-CTA)

#### 一句话先理解

**一句话先理解：**前两步只会算输出矩阵左上角那一个小方块。Step 3 让一大堆 CTA(小队) 同时开工，每队认领一个方块，大家并行把整张输出矩阵铺满。

第 2 步用 K-loop 搞定了收缩维度 (K 维) 的累加，可输出还是被钉死在一个  $128 \times 128$  的单 tile 上。真实的输出矩阵，可比一个 tile 大太多了。所以基础内核的最后一块拼图，就是让 M、N 这两个维度被许多 tile 一起铺满。

这里要引入一个词 **网格 (grid)**：它指的是这一次 **kernel** 启动时，所有 **CTA** 排成的那个阵列。前两步我们的网格是 `[1, 1]`——只有一个 CTA(一队线程)，所以只算一个方块。Step 3 把网格变成二维的一大片，比如  $2 \times 2$  就是 4 个 CTA。打个比方：网格就像一张作战地图，上面划了很多格子，每个格子派一队人 (一个 CTA) 去包干。

第 3 步的核心动作就一个：开一个二维的 CTA 网格，让**每个 CTA 包一个输出 tile**，然后让 GPU 把所有 tile 并行算出来。例子特意挑了  $M=N=K=256$ ，正好切成  $2 \times 2$  共 4 个 tile——既让 tile 索引的逻辑「不那么一眼就看穿」，又不至于多到把逻辑淹了。

#### 这一步到底改了什么

把内核拆成三个关注点来看，第 3 步只动了其中一个：

| 关注点           | 与第 2 步相比 | 说明                                                                              |
|---------------|----------|---------------------------------------------------------------------------------|
| 作用域 (Scope)   | 改变       | 从单个 CTA 变成二维 CTA 网格，每个 CTA 拥有一个 $128 \times 128$ 的输出 tile                       |
| 布局 (Layout)   | 不变       | 单个 CTA 内部的 SMEM $\rightarrow$ TMEM $\rightarrow$ 寄存器 (register) 数据通路 与第 2 步完全相同 |
| 分发 (Dispatch) | 不变       | MMA 的指令分发方式照旧                                                                   |

换句话说，**单个 CTA 内部的计算逻辑一行都没动**，变的只是「开几个 CTA」和「每个 CTA 算哪一块」。这恰恰是分块 GEMM 设计漂亮的地方：tile 级别的实现是一个能反复用的原子，要往空间上铺开，只要给它套上对的坐标偏移就行。

#### 网格映射：每个 CTA 找到自己那一块

网格的形状直接由分块方式推出来。既然一个 CTA 对一个  $128 \times 128$  的输出 tile，那一共要 CTA 数就是：

```
grid = [M // BLK_M, N // BLK_N]
```

跟第 2 步比, 真正多出来的活只有一件: 告诉每个 CTA, 它该算矩阵的哪一块切片。  
设 CTA 的二维坐标是 (bx, by), 那它负责的输出区域就是:

```
D[bx * BLK_M : (bx + 1) * BLK_M, # : bx
 by * BLK_N : (by + 1) * BLK_N] # : by
```

为了算出这块结果, 这个 CTA 的 K-loop 会反复加载 A 对应的「行带」和 B 对应的「列带」:

```
A[bx * BLK_M : (bx + 1) * BLK_M, k : k + BLK_K] # A CTA
B[by * BLK_N : (by + 1) * BLK_N, k : k + BLK_K] # B CTA
```

为什么 by 选的是 B 的”行”, 却变成了 D 的”列”

这一点最容易把人绕晕, 根子在于内核守的是  $D = A @ B.T$  这个约定——B 是先转置、再参与乘法的:

- bx 一下选中 A 的行和 D 的行, 这俩直接对上, 好理解。
- by 选的是 B 的行。可 B 要转置 (.T), 这些行一转, 就变成了列, 于是对到了 D 的列上。

下表把 2x2 网格里各个 CTA 归谁管, 摆得清清楚楚 (拿 M=N=256、BLK=128 举例)。行方向是 A 的行 (bx), 列方向是 B.T 的列 (by):

| A 的行方向 (bx) \ B.T 的列方向 (by) | by=0                         | by=1                           |
|-----------------------------|------------------------------|--------------------------------|
| bx=0                        | CTA (0,0) —D[0:128, 0:128]   | CTA (0,1) —D[0:128, 128:256]   |
| bx=1                        | CTA (1,0) —D[128:256, 0:128] | CTA (1,1) —D[128:256, 128:256] |

每个 CTA 各自跑一遍完整的 K-loop:  $CTA(bx, by) = \sum_k A[bx, k] @ B[by, k].T$ 。

注意

注意:bx 是行索引、by 是列索引, 但 by 是绕了 B.T 一圈才映射到 D 的列。要是你把 A、B 的切片偏移搞混, 矩阵乘法在数值上不会报错, 却会给你算出一个完全错的结果——这是多 CTA 分块里最常见的索引 bug。

一个 tile 一个 CTA: 能用, 但费

「一个 tile 配一个 CTA」是能跑通的最简单映射, 可它特别费内存带宽:

- 同一行上的所有 CTA(bx 相同、by 不同), 会各自从头到全局内存 (GMEM) 里去拉一模一样的 A 行带。
- 同一列上的所有 CTA(by 相同、bx 不同), 也各自重新加载一模一样的 B 列带。

说白了, 相邻 CTA 明明已经拉进来的数据, 彼此之间一点没省着用——同一条 A 行带, 被好几个 CTA 各自从最慢的 GMEM 重新拉了一遍, 纯属重复劳动。这步先忍着这份浪费 (记住本章只求对、不求快)。后面《用 TMA 做 GEMM 流水线》的第 6 步会上持久化调度 (persistent scheduling)

回头收拾它,让这些被多个 CTA 共用的数据尽量「赖在 L2 里保持热乎」,少往 GMEM 跑重复的活。

(这里又冒出个 **L2**: 它是介于「最慢的 GMEM」和「最快的 SMEM」之间的一层缓存,比 GMEM 快、被全 GPU 共享。「赖在 L2 里保持热乎」的意思就是: 如果好几个 CTA 先后要读同一份数据,让它待在 L2 缓存里别被挤走,后来的 CTA 就能从更快的 L2 拿,而不必每次都跑去最慢的 GMEM 重取。)

**和你的 agent 一起练:** 就用  $M=N=K=256$ 、 $BLK\_M=BLK\_N=128$ 、 $BLK\_K=64$  这套设置,让它盯着 CTA (1, 0) 和 CTA (0, 1) 跑。每个 CTA 都写清楚  $m\_st$ 、 $n\_st$ 、每轮 K 迭代加载的 A/B 切片、还有最后写回的 D 区域。再答一道: 既然内核算的是  $D = A @ B.T$ , 到底是 B 的哪些行,转过身变成了 D 的列?

### 完整内核: 在第 2 步上只改两处

第 3 步的内核骨子里还是第 2 步,就动了两处:

1. **网格形状:** 从 [1, 1] 改成  $[M // BLK\_M, N // BLK\_N]$ 。
2. **每个 CTA 的偏移:** 加载和写回的时候,都加上本 CTA 自己的  $m\_st$  和  $n\_st$ 。

内层的 K-loop 和写回 (writeback) 逻辑一个字没改。下面只把体现这两处改动的关键行挑出来,配上中文讲解;TMEM 分配、mbarrier 初始化这些样板代码跟第 2 步一样,这里就略了。

### 关键改动一: 二维网格 + 计算本 CTA 偏移

```
@T.prim_func
def kernel(
 A: T.Buffer((M, K), a_type),
 B: T.Buffer((N, K), b_type),
 D: T.Buffer((M, N), d_type),
):
 T.device_entry()
 # : CTA 128x128 tile
 bx, by = T.cta_id([M // BLK_M, N // BLK_N])
 ...
 # CTA tile D /
 m_st = T.meta_var(bx * BLK_M) # CTA tile
 n_st = T.meta_var(by * BLK_N) # CTA tile
```

这里  $T.cta\_id([...])$  顶掉了第 2 步那个写死的 [1, 1] 网格, 返回当前 CTA 的二维坐标 (bx, by);  $m\_st$ 、 $n\_st$  再把坐标翻译成本 CTA 的输出 tile 在全局矩阵里的起始行 / 列偏移 (注意是元素下标的偏移, 不是字节偏移)。

### 关键改动二: K-loop 里给 A/B 切片加偏移

```
for i in T.serial(K_TILES):
 # A: CTA m_st:m_st+BLK_M,K i
 Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_K:(i+1)*BLK_K])
 # B: CTA n_st:n_st+BLK_N(D),K i
 Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_K:(i+1)*BLK_K])
```



```

T.cuda.cta_sync()

if warp_id == 0:
 if T.ptx.select_sync():
 # accum=(i != 0): 0 , 2
 Tx.gemm_async(tmemb[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
 accum=(i != 0), dispatch="tcgen05", cta_group=1)
 T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)

T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
phase_mma ^= 1

```

跟第 2 步对一下, 这里就一处变了:A、B 切片在 M/N 方向的起点从 0 换成了 `m_st`、`n_st`。K 维照样靠循环变量 `i` 往前滑,`accum=(i != 0)` 的累加语义、`gemm_async` 的 MMA 调用、`mbarrier` 的等待握手, 统统原样不动。

### 关键改动三: 写回到正确的输出 tile

```

D : tile + warp + lane
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
n_st , CTA
Tx.copy(D[m_thr, n_st:n_st+BLK_N], Dreg_f16[:])

```

写回这块也只动了偏移: 行偏移从 `warp_id * 32 + lane_id` 变成 `m_st + warp_id * 32 + lane_id`, 列范围从 `0:BLK_N` 变成 `n_st:n_st+BLK_N`。`wargroup` 里的线程照旧各写一行, 只是现在落点是本 CTA 在 D 里那块专属区域, 而不是全局的左上角了。

### 整体数据流回顾

#### 结构图

分组:

- **K-loop** (`i = 0 .. K_TILES-1`): 「同步拷贝 A[m\_st 行带, i] → SMEM (Tx.cta.copy)」 「同步拷贝 B[n\_st 行带, i] → SMEM (Tx.cta.copy)」 「`gemm_async` 累加到 TMEM (`accum = i!=0`)」 「`mbarrier` 等待 MMA 完成」

连接关系:

- 同步拷贝 A[m\_st 行带, i] → SMEM (Tx.cta.copy) → 同步拷贝 B[n\_st 行带, i] → SMEM (Tx.cta.copy)
- 启动 [M//BLK\_M, N//BLK\_N] 个 CTA(本例  $2 \times 2 = 4$  个), 全部并行 → 每个 CTA(`bx,by`):`m_st = bx*128`, `n_st = by*128`

你看, 第 3 步只靠一层简单的坐标偏移, 就把一个「会算一个 tile」的内核, 扩成了「在 GPU 上一口气铺满整张输出矩阵」。这就是 GPU GEMM 的基本骨架:**tile 内部那条精细的数据通路一动不动, 往空间上铺开就全靠网格 + 偏移**。带宽浪费先搁着, 留给持久化调度去收拾; 但功能上, 这已经是一个能算任意 M、N 大小输出的完整分块 GEMM 了。

## 习题精解 (Exercises)

这一节用三道题, 把 Step 1-3 的核心要点串起来。三道题看着零散, 其实正好卡在从单 Tile 到 K 循环、再到多 CTA 这条路上三个最容易翻车、也最值得讲透的地方: **共享内存 (SMEM) 的可见性同步**、**mbarrier 相位 (phase) 的正确性陷阱**, 还有 **多 CTA 网格下的数据复用**。下面每道题都给完整推导和道理, 不只是丢个结论给你。

### 注意

**注意:** 做这几道题, 建议对着本章 Step 1-3 那三份完整 kernel 一起看。三个版本内层结构几乎一样, 差别就在「K 循环」和「网格 / 偏移」这两处, 所以同一套同步道理, 会在三步里一遍遍冒出来。

### 第 1 题: 为什么 Tx.gemm\_async 读 SMEM 前必须先 cta\_sync?

**题目:** 在 Step 1-3 里, Tx.copy(准确说是 Tx.cta.copy) 把 A、B tile 从全局内存 (GMEM) 搬进 SMEM, 搬完才做 MMA。那为什么在 Tx.gemm\_async 读这些 SMEM tile 之前, kernel 非得插一句 T.cuda.cta\_sync()?

**核心答案:** 因为这次拷贝是 **CTA 里一堆线程一起协作干的**, 而 MMA 又是 **挑一个线程发起、却要把整个 tile 都读进去的**。少了 cta\_sync, 发起 MMA 的那个线程什么保证都没有——它不知道别的线程是不是已经把各自那一片写完了、写的东西对自己是不是已经能看见。结果 MMA 很可能读到一个「只填了一半」的缓冲区。

想说透这件事, 得拆成两层看:

#### 1) Tx.cta.copy 是「大家分着搬」, 不是哪个线程一个人扛

```
Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :]) # CTA 128
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync() # +
```

Tx.cta.copy 让整个 CTA 一块儿搬: 128 个线程, 每人管 Asmem / Bsmem 的一小块。也就是说, 拷完之后, Asmem 这块 SMEM 里不同的区域, 是被 **不同线程** 分头写进去的。

#### 2) MMA 由挑出来的一个线程发起, 却要读「整块」tile

```
if warp_id == 0:
 if T.ptx elect_sync():
 # CTA
 Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :], ...)
```

发起 MMA 的就一个 lane。可 Tx.gemm\_async 是个 **tile 级操作**: 硬件 MMA 得把 Asmem、Bsmem 这两整块都读进去做协作矩阵乘。也就是说, 这一个线程发起的指令, 吃的是另外 127 个线程刚写下去的数据。

#### 3) 两个保证, 少一个都不行

线程之间不会自动对齐, 所以 T.cuda.cta\_sync() 在这儿一身二职:

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
| 它提供的保证                                 | 没有它会怎样                                  |
| 等齐（同步）：阻塞到 CTA 内所有线程都跑完拷贝              | 慢的线程可能还没搬完,MMA 就开始读                     |
| 公布（可见性 / fence）：让所有线程对 SMEM 的写入对其他线程可见 | 即使线程都「跑过了」拷贝语句，写入也可能还停在缓存/写缓冲里,MMA 读到旧值 |

第二条尤其要命：哪怕没有竞速，现代 GPU 的内存模型也**不担保**一个线程写进 SMEM 的东西，另一个线程立马就能看见。必须有一个同步 / 栅栏点，把这些写入「发布」出去。`cta_sync` 既是 barrier(等齐)，又顺手把发布写入这件事一起办了。

**一句话总结：**写 SMEM 的是「一帮人」，读 SMEM 的是「一个人」。这种跨线程的生产—消费关系，必须靠 `cta_sync` 一边等齐、一边公布。不然 MMA 读到的就是一块只填了一半、还可能根本看不全的缓冲区，然后悄没声地算错。

注意

注意：这条同步是「先写后读」型（SMEM 写入 → MMA 读取）。本章后面的 K 循环里还有一条反方向的依赖：拷下一轮数据之前，得先确认上一轮 MMA 已经把旧数据读完了，不然就会「新数据盖掉了还没读完的 buffer」。Step 2/3 还是用同一个 `cta_sync` 把两个方向一起挡住；真把这两类依赖拆开、做成流水线，那是后面 TMA 章节的事了。

第 2 题：删掉 `phase_mma ^= 1` 会发生什么？

题目：在 Step 2 里，要是把 K 循环里的 `phase_mma ^= 1` 删掉，会怎样？kernel 还会乖乖等每一次 MMA 吗，还是某次 `wait` 会提前放行？

**核心答案：**kernel 不会再正确地等了。从第二轮迭代起，`try_wait` 会当场提前返回，根本没等这轮的 MMA 算完。于是 kernel 读到一个只累加了部分 K 的「半成品」累加器，给出错误结果——而且一个错都不报。

想搞懂为什么，得先把 `mbarrier` 的相位语义讲清楚。这是整章头一个真正的正确性陷阱。

1)mbarrier 的相位只有 1 个比特，而且语义「反直觉」

一个 `mbarrier` 内部带一个 1 比特的相位（0 或 1）。每凑齐一次预期的「到达（arrival）」，相位就翻到另一个值。而等待原语的判断条件是：

|                                                  |                |                      |                 |                    |
|--------------------------------------------------|----------------|----------------------|-----------------|--------------------|
| <code>T.ptx.mbarrier.try_wait(bar, phase)</code> | <code>#</code> | <code>barrier</code> | <code>!=</code> | <code>phase</code> |
|--------------------------------------------------|----------------|----------------------|-----------------|--------------------|

注意这儿反直觉的地方：`try_wait(bar, phase)` 堵着的条件，是 `barrier` 内部相位**不等于**你传进去的 `phase`。所以你传进去的那个参数，应该是「你预期马上要被翻走的那个旧相位」，而不是你想等到的那个新相位。

2) 正确情形：相位一轮一轮翻

每轮迭代里，`phase_mma ^= 1` 这一行让本地记的相位跟着 `barrier` 一块儿翻，两边始终咬合：

| K 迭代 | wait 前的本地 <code>phase_mma</code> | <code>try_wait</code> 实际在等什么 | wait 后本地更新为                |
|------|----------------------------------|------------------------------|----------------------------|
| 0    | 0                                | <code>barrier</code> 翻到 1    | <code>phase_mma = 1</code> |

|   |   |              |               |
|---|---|--------------|---------------|
| 1 | 1 | barrier 翻到 0 | phase_mma = 0 |
| 2 | 0 | barrier 翻到 1 | phase_mma = 1 |

3) 删掉翻转后: 第二轮就出事

要是删了 `phase_mma ^= 1`, `phase_mma` 就永远卡在 0。一轮一轮跟一下:

```
0: try_wait(bar, 0) barrier =0, -> ,
 0 MMA barrier 1
1: try_wait(bar, 0) barrier 1 (),
 1 != 0 -> != phase -> wait ,
 1 MMA!
```

也就是说: 第一轮碰巧还对 (初始相位本来就是 0), 可从第二轮起, 每次 `try_wait` 都瞬间放行——因为本地一直拿 0 去比一个早就变成 1 的 `barrier`。kernel 不卡死, 会一路跑到底, 然后从 TMEM 读出一个还没把所有 K chunk 累加完的累加器, 给你个错答案。

这就是这类 bug 最阴的地方: 能编译、能跑、不报错, 数值还「看着差不离」(尤其 K 小、误差被相对容差放过去的时候), 实则是个不声不响的错误。`phase_mma ^= 1` 这一行毫不起眼, 却是整个 K 循环正确性的命门。

注意

注意: 为什么得本地自己记一个相位, 不能让硬件自动管? 因为同一个 `mbarrier` 被跨迭代复用——硬件就一个 1 比特状态在 0/1 之间来回翻, 软件必须自己盯住「我现在该等的是哪一相」。复用 `barrier` 的好处是零额外存储, 代价就是这份相位账, 得你自己记对。

第 3 题: Step 3 启动多少 CTA? 相邻 CTA 之间复用了哪些 tile?

题目: `M=N=4096`、`BLK_M=BLK_N=128` 的情况下, Step 3 会启动多少个 CTA? 相邻 CTA 之间, 哪些 operand tile 在逻辑上其实是能共用的? Step 3 有没有把这种复用用上?

1) CTA 数量: 1024 个

Step 3 的网格直接由 tiling 定下来, 一个 CTA 管一个  $128 \times 128$  输出 tile:

```
bx, by = T.cta_id([M // BLK_M, N // BLK_N]) # , CTA tile
```

代入数字:

```
tile = 4096 / 128 = 32
 = 32 x 32
CTA = 32 x 32 = 1024
```

你可以画成一张  $32 \times 32$  的输出 tile 网格, 一个格子就是一个 CTA。同一行 (bx 相同) 的 CTA 共用同一条 A 行带; 同一列 (by 相同) 的 CTA 共用同一条 B 列带:

| bx \ by | by=0       | by=1       | ... | by=31       |
|---------|------------|------------|-----|-------------|
| bx=0    | CTA (0,0)  | CTA (0,1)  | ... | CTA (0,31)  |
| bx=1    | CTA (1,0)  | CTA (1,1)  | ... | CTA (1,31)  |
| ...     | ...        | ...        | ... | ...         |
| bx=31   | CTA (31,0) | CTA (31,1) | ... | CTA (31,31) |

- 每一行 (如 bx=0 这一整行的 32 个 CTA) 共用同一条 A 行带。
- 每一列 (如 by=0 这一整列的 32 个 CTA) 共用同一条 B 列带。

2) 逻辑上可复用的 operand tile

回忆每个 CTA 加载的切片:CTA (bx, by) 在每一轮 K 里加载

|                                         |     |         |    |
|-----------------------------------------|-----|---------|----|
| A[bx*BLK_M : (bx+1)*BLK_M, k : k+BLK_K] | # A | ,       | bx |
| B[by*BLK_N : (by+1)*BLK_N, k : k+BLK_K] | # B | ( D ) , | by |

关键就一句:A 的切片只看 bx,B 的切片只看 by。于是天然就冒出两类复用机会:

| 复用方向        | 谁共享         | 共享的内容                 | 在 4096 例子里被重复加载的次数          |
|-------------|-------------|-----------------------|-----------------------------|
| 同一行 (bx 相同) | 整行 32 个 CTA | 相同的 A 行带 (所有 K chunk) | 同一份 A tile 被 32 个 CTA 各加载一次 |
| 同一列 (by 相同) | 整列 32 个 CTA | 相同的 B 列带 (所有 K chunk) | 同一份 B tile 被 32 个 CTA 各加载一次 |

换句话说, 理论上每一片 A 从 GMEM 读一次, 就够喂整行 32 个 CTA; 每一片 B 读一次, 就够喂整列。绝大部分 GMEM 流量, 本来是能省下来的。

3)Step 3 有没有把这种复用用上?——没有

没用上。Step 3 走的是「一 tile 一 CTA」这种最素的映射, 每个 CTA 各干各的, 自己的 A 行带、B 列带都从 GMEM 重新拉一遍, 彼此之间一点不共享。结果就是: 同一份 A tile, 同一行的 32 个 CTA 各读了一遍; 同一份 B tile, 同一列的 32 个 CTA 各读了一遍, 大把 GMEM 带宽白扔了。

原文对这点态度很明确: 这份浪费是故意先留着的。Step 3 要的只是把 M、N 铺开、让多 tile 并行跑起来, 把正确性和网格映射讲清楚。复用的事, 留给后面的「持久化调度 (persistent scheduling)」(TMA 章节的 Step 6) 去办——靠让共享的 operand 常驻 L2 缓存, 把相邻 CTA 重复读 GMEM 的开销压下去。

注意

注意: 这里要把两件事分清:「逻辑上有没有复用机会」和「kernel 实现到底兑没兑现」。复用机会是 GEMM 结构自带的 (A 切片只看 bx、B 切片只看 by), 它一直都在; 但 Step 3 只是让一堆 CTA 把同一份数据各从 GMEM 搬一遍, 压根没去兑现它。想明白这点, 你才能懂后面为什么还要持久化调度这类优化, 以及它们到底在省什么。

小结

本章从一个  $128 \times 128$ 、 $K = 64$  的单输出 tile 起步, 亲手把抽象的 TIRx scope / layout / dispatch 模型, 落到一个真能跑的分块 GEMM 内核上, 然后顺着「一次只改一个决策」的路子, 分三步把它养大:

- **Step 1(单 Tile 顺序 GEMM):** 用一个 warpgroup 顺序走完  $\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{MMA} \rightarrow \text{TMEM} \rightarrow \text{GMEM}$  这条完整通路, 把后面一切修改都依着的基线契约立起来。几个关键点:swizzled SMEM 布局 (布局即契约)、一个线程发起 tile 级 MMA(发起  $\neq$  一个线程在算)、还有 fp32 累加器写回时收合成 fp16。
- **Step 2(K 维循环累加):** 不添一个字节新存储, 套上 K-loop, 用 `accum=(i!=0)` 把零散的 K 块缝成完整点积。真正的难点在于反复复用同一个 mbarrier 时的相位记账: 少了 `phase_mma ^= 1`, 就会不声不响地算错。
- **Step 3(空间分块 / 多 CTA):** 单 CTA 内部的计算一行不动, 只靠二维网格 + 坐标偏移, 就把内核铺到任意 M、N。重点搞懂两件事: $D = A B^{\sim\{\text{top}\}}$  下 by 怎么经转置对到 D 的列, 以及「一 tile 一 CTA」带来的、特意先留着的那份带宽浪费。

最值钱的收获, 是一套能搬到别处用的方法论: 把每个新内核都读成「它在上一个内核上改了契约里的哪一条」, 同时对跨线程的生产—消费依赖 (SMEM 可见性、mbarrier 相位) 死盯不放——这类 bug 能编译、能跑、不报错, 却悄没声地给你错数值。

延伸阅读

- 原文:[构建分块 GEMM\(Step 1-3\)](#)
- 下一章《用 TMA 做 GEMM 流水线》: 把本章朴素的线程拷贝换成 TMA(Tensor Memory Accelerator), 并用多 SMEM stage 的软件流水线让数据搬运与计算重叠; 持久化调度 (Step 6) 回头解决本章 Step 3 暂留的 operand 复用 / GMEM 带宽浪费问题。
- 再下一章《用 Warp 特化与 Cluster 扩展 GEMM》: 引入 warp 特化 (producer / MMA consumer / writeback 分到不同 warpgroup) 与 CTA cluster(两个 CTA 协作完成更大的 MMA tile), 并在统一的  $M = N = K = 4096$  规模下给出跨步骤端到端加速比对比。
- 本书 Part II(TIRx tile primitive): 本章用到的 `Tx.copy` / `Tx.gemm_async` / `tma_shared_layout` / `TileLayout` 等原语词汇表的来源, 建议回看以理解「布局即契约」。

术语对照

| 英文                             | 中文        | 简要说明                                                        |
|--------------------------------|-----------|-------------------------------------------------------------|
| GEMM (General Matrix Multiply) | 通用矩阵乘法    | 稠密矩阵相乘, 本章计算 $D = A B^T$ ; 深度学习几乎所有算子的底层原语                  |
| Tile                           | 分块 / 瓦片   | 矩阵被切成的小块 (本章输出 tile 为 $128 \times 128$ ), 内核以 tile 为单位搬运与计算 |
| Tiling / Spatial tiling        | 分块 / 空间分块 | 把大矩阵切成多个 tile, Step 3 用多 CTA 网格在空间上并行覆盖完整输出                 |
| Contraction dimension (K)      | 收缩维度      | 矩阵乘法中被求和消除的维度; Step 2 沿 K 循环累加                              |
| Baseline                       | 基线        | 最小可信的正确实现, 作为后续所有优化的对照基准 (ground truth)                     |

|                                  |              |                                                   |
|----------------------------------|--------------|---------------------------------------------------|
| Scope                            | 作用域          | 契约三要素之一：由哪个执行单位 (warpgroup / CTA / 网格) 来跑操作       |
| Layout                           | 布局           | 契约三要素之一：操作数 tile 在内存中如何摆放 (如 swizzled SMEM)       |
| Dispatch                         | 分派 / 派发      | 契约三要素之一：走哪条派发路径真正执行操作 (如 tcgen05)                 |
| CTA (Cooperative Thread Array)   | 协作线程阵列 / 线程块 | 一组协作线程，共享 SMEM; Step 3 一个 CTA 负责一个输出 tile         |
| Warpgroup                        | warp 组       | 128 个线程 (4 个 warp) 组成的执行单位，本章顺序走完整条通路             |
| GMEM (Global Memory)             | 全局显存         | 容量最大，离计算最远的存储，输入与最终输出所在地                          |
| SMEM (Shared Memory)             | 共享内存         | CTA 内共享的片上高速存储，操作数 tile 先搬到此供 MMA 读取              |
| TMEM (Tensor Memory)             | 张量内存         | Blackwell 新增、专门存放 MMA 累加器的存储层级                    |
| MMA (Matrix Multiply-Accumulate) | 矩阵乘累加        | 真正执行乘加的运算; tcgen05.mma 是 Blackwell 上的对应指令         |
| Accumulator                      | 累加器          | 沿 K 维累加部分和的缓冲，本章在 TMEM 中以 fp32 保存以压制舍入误差          |
| Swizzle / Swizzled layout        | 交错布局         | 一种 SMEM 摆放方式，使 TMA 与 tcgen05.mma 都能直接读取           |
| mbarrier (Memory barrier)        | 内存屏障         | 用于跨线程 / 异步同步的硬件屏障，内部带 1 比特相位                      |
| Phase (flip)                     | 相位 (翻转)      | mbarrier 的 1 比特状态; 复用时需本地翻转 phase_mma ^= 1 才能正确等待 |
| try_wait                         | 等待原语         | 阻塞直到 barrier 内部相位!= 传入参数; 参数是「预期要离开的旧相位」          |
| cta_sync                         | CTA 同步       | 一身二职：等齐所有线程 + 发布 (publish) 它们的 SMEM 写入            |
| Epilogue                         | 尾处理段         | 内核收尾阶段，把 TMEM 累加结果读回寄存器、收窄类型后写回 GMEM              |
| accum flag                       | 累加标志         | gemm_async 参数: False 覆盖 (初始化)、True 累加进已有 TMEM 值   |
| TFLOPS                           | 每秒万亿次浮点运算    | 吞吐量度量：一次乘加算 2 次浮点运算, $2MNK/(t \times 10^{12})$    |
| TMA (Tensor Memory Accelerator)  | 张量内存加速器      | Blackwell 硬件异步拷贝贝通路，下一章用以替换本章的线程拷贝                |
| Persistent scheduling            | 持久化调度        | 固定 CTA 池连续处理多个 tile，后续用以复用 operand、减少 GMEM 重复读    |





# 第 12 章 · 用 TMA 为 GEMM 做流水线 (Step 4-6)

原文: [Pipelining GEMM with TMA](#)

## 本章要点

### 本章要点 (TL;DR)

别急, 先把几个会反复出现的词用大白话过一遍, 后面就不卡壳了:

- **GEMM**: 就是矩阵乘法 (通用矩阵乘法的简称)。深度学习里绝大部分算力都花在矩阵乘上, 所以它是当之无愧的主角。
- **张量核心 (Tensor Core)**: GPU 芯片上一块专门做矩阵乘的电路。它是整块芯片上最贵、最强的计算单元, 理应一刻不停地转; 让它空闲就是最大的浪费。
- **线程 (thread)**: GPU 干活的最小单位。和 CPU 不同, GPU 里有成千上万个线程同时跑, 这正是它快的原因。
- 把大矩阵切成小方块来算, 每个小方块就叫一个 **tile(瓦片)**。这是上一章的核心招数。

好, 正题:

- 上一章写出的那个”算得对”的分块 GEMM, 有个大毛病: 最贵的张量核心**大部分时间在空等**。流程是——线程搬一块数据进来、张量核心算这块、线程再搬下一块、张量核心继续等……搬数据和算数据本来是两套不同的硬件、可以同时干, 却被排成了一前一后的串行, 白白浪费。
- 本章不动数据本身怎么摆、算什么 (tile、内存布局、数学全都不变), 只改**什么时候做、由谁来指挥**。核心思想就一句话: **让一个硬件引擎跑在另一个引擎前面** (术语叫”异步交接”——一边还在算, 另一边已经把下一批料备好了)。
- **Step 4**: 把”从大显存 (GMEM, 显卡上那块大内存) 搬到片上小快内存 (SMEM)”这件苦力活, 从线程手里接过来, 交给一个叫 **TMA** 的专用搬运硬件。线程只要喊一嗓子 (发一条命令), 剩下的 TMA 自己干。搬完没搬完, 用一个叫 **mbarrier** 的”门门”按搬了多少字节来判断。不过这一步**每次搬完还是傻等**, 所以还没真正重叠起来。
- **Step 5**: 给 SMEM 准备**两块格子轮着用** (叫双缓冲 / 环形缓冲), 这样”算当前块”的同时,”下一块”有地方提前搬进来 (预取)。流水线的骨架就此搭好——但要注意, 这一步主循环**还是先等算完、再发下一笔搬运**, 真正两条线同时跑满, 要等下一章 Step 7。
- **Step 6**: 把整个程序改成**持久化 (persistent)** 形态: 不再”算一块就开一批新线程”, 而是固定养一池线程组, 让每个线程组**连着啃好几个 tile**。好处是省掉反复开张的开销, 还能挑一个聪明的处理顺序, 让数据赖在缓存里被反复利用。

## 前置知识

**前置知识:** 读这一章前, 最好先懂第 11 章那个分块 GEMM(把大矩阵切成小方块来算)。本章会反复用到这几个词:TMA(搬数据的专用硬件)、mbarrier(判断异步搬运完没完的”门闩”)、双缓冲/流水线(让”搬数据”和”算数据”两条线错开、同时跑)。这些词第一次出现时我都会现场用大白话讲清楚, 所以零基础也不用慌。如果连”线程””共享内存””warp”这些最基础的概念都还没概念, 建议先翻一下第 0 章·极简入门, 再回头看第 11 章。

这一章属于「第三部分·GEMM 实战 (Step 1→9)」。

上一章我们盯着一个问题:”算得对不对?”——结果对了就行。这一章换个问题:”算得满不满?”——也就是, 那块最贵的计算硬件, 有没有被喂饱、有没有一直在干活。说白了, 就是别让最贵的那块硬件闲着。

读之前, 先把终点记在心里。这三步, 一步干一件事:

- Step 4: 让一块**专门搬数据的硬件 (TMA)** 替线程去搬数据, 把线程解放出来;
- Step 5: 给数据准备**两块轮流用的格子 (双缓冲)**, 这样下一块能提前搬进来;
- Step 6: 把整个程序的启动方式重塑成**持久化**形态 (下面会细讲什么叫持久化)。

这里有个点要先说清楚, 免得你越读越晕: 这三步从头到尾, 数据**摆在哪儿、怎么摆** (也就是各种片上内存里的布局) **一行都没改过**。这里出现两个新名字:**TMEM**(张量内存, GPU 上一块专门存放矩阵乘”中间累加结果”的片上小内存), **寄存器 (register)**, 每个线程私有的、速度最快的一点点存储——和 CPU 的寄存器是一个意思, 只不过 GPU 里每个线程各有一份)。真正算新东西的, 从头到尾就一个: **让不同硬件单元之间异步交接活儿** (一个干着, 另一个提前把下一步备好)。剩下的全是老零件, 只是换个用法。

## 为什么基础版 GEMM 在浪费时间

## 一句话先理解

**一句话先理解:** 最贵的那块计算硬件 (张量核心) 本该一直转, 可现在它一半时间在干等——因为”搬数据”和”算数据”被排成了一前一后, 而它俩明明可以同时干。

先把问题摊开讲。前面说过, 张量核心是整块芯片上**最贵的家伙**, 理应一刻不停地转。可上一章那个写法本身没错的分块 GEMM, 偏偏让它大半时间都在干等。

为什么? 因为这个程序 (GPU 上跑的这种程序习惯叫 **kernel(核函数)**, 你就理解成”一段在 GPU 上跑的函数”)是”一个一个轮着来”的:

1. 线程把一块 tile 从大显存搬进共享内存 (SMEM, 片上那块又小又快的内存, 下面马上细讲);
2. 张量核心把这块嚼完;
3. 线程拷下一块;
4. 张量核心又在那儿等着。

每一步都得死等上一步收工才能动。可你仔细想想: **搬下一块 tile 和 算手里这块**, 根本是两套不同的硬件在干 (一个是搬运通路, 一个是计算单元), 它俩完全可以同时进行啊! 现在却傻乎乎地排队, 这就是浪费的根源。

要把这条缝补上, 不用动任何数据通路——tile 怎么切、数据怎么摆、算什么, 现成的全够用。要改的就两样: **什么时候做**, 和**谁来调度**。

下面拿两张表对一对，看看”串着跑”和”我们想要的重叠跑”差在哪。表的读法：每一行是一个**硬件引擎**（搬运的 / 计算的），每一列是一个**时间步**（t0、t1……像秒表一格一格地走），格子里写的是那一刻这个引擎在干什么（—表示它在空等）：

**基础版（串行，轮流上场）：**

| 引擎       | t0      | t1     | t2      | t3     | t4      |
|----------|---------|--------|---------|--------|---------|
| 搬运（拷贝硬件） | load k0 | —      | load k1 | —      | load k2 |
| 计算（张量核心） | —       | mma k0 | —       | mma k1 | —       |

看出来了吗？拷贝硬件干活的时候（t0、t2、t4），张量核心在歇；张量核心干活的时候（t1、t3），拷贝硬件又在歇。俩人从不一一起上，各自都有一半时间空等。**load k0**意思是”搬第 0 块”，**mma k0**意思是”算第 0 块”（mma 就是矩阵乘加，后面会讲）。

**理想版（重叠，各司其职）：**

| 引擎       | t0      | t1      | t2      | t3      | t4  |
|----------|---------|---------|---------|---------|-----|
| 搬运（拷贝硬件） | load k0 | load k1 | load k2 | load k3 | ... |
| 计算（张量核心） | —       | mma k0  | mma k1  | mma k2  | ... |

看出区别了吗？搬运那一行**一刻不停**，在算 k0 的同一个时间步里，k1 已经在搬了。等张量核心算完 k0，k1 早就备好，马上接着算。张量核心几乎一直在转——这就是我们要的。

关键

**关键：**本章三步是**渐进的**，别指望一步到位。Step 4 把搬运交给硬件（但还是等）；Step 5 搭出流水线骨架并预取（但还是等）；真正”两条线同时跑满”的重叠，要到下一章 Step 7。那时会用一个叫 **warp 专门化（warp specialization）** 的手法——简单说就是”分工”：让一拨线程专门搬数据（生产者），另一拨专门算（消费者），俩角色同时跑。本章先把地基打好。

注意

**注意：**从 Step 4 开始，所有例子都跑在完整规模  $M=N=K=4096$  的矩阵上（不再是前几步的小尺寸玩具例子）。端到端的计时数据统一放在下一章末尾的 *End-to-End Result* 表里。

### Step 4:TMA 异步 Load

这一步只改了”派发方式”

这一步就动一个地方：原来是线程一块一块**亲自**地搬（术语叫”同步拷贝”，意思是线程得守着搬完才能干别的），现在换成 **TMA load**——让 TMA 硬件去搬。别的全不动。

原书用三个维度来描述每一步改了什么，这里先把这三个词解释清楚，后面每步都会用这张表：

- **Scope(范围)**：这活儿由多大一拨线程协同完成。这里会出现 **warpgroup** 这个词——它是 4 个 warp、共 128 个线程组成的一个班组（warp 是 32 个线程的小队，见第 0 章；先记住  $\text{warpgroup} = 128$  个线程一组就行）。
- **Layout(布局)**：数据在各级片上内存里怎么摆。
- **Dispatch(派发)**：活儿具体怎么发出去、由谁执行。

拿这张表对一下 Step 4 改了哪一维，一目了然：

| 维度           | Step 4 的状态                                 |
|--------------|--------------------------------------------|
| Scope(范围)    | 不变，仍是一个 warpgroup(128 线程)                  |
| Layout(布局)   | 不变，同样的 SMEM / TMEM / 寄存器 tile              |
| Dispatch(派发) | 变了：从大显存搬到共享内存存这件事，从“线程亲自同步搬”改为“交给 TMA 引擎搬” |

同步拷贝 vs. TMA 命令：执行模型的本质差别

一句话先理解

一句话先理解：同步拷贝是线程自己撸起袖子一趟趟搬，搬的全程被占着脱不开身；TMA 拷贝是线程喊一嗓子“搬这块！”就走人，真正搬运交给专用硬件后台去跑。

代码就改那么几行，可背后干活的方式**根本是两码事**。

- 同步的 `Tx.copy`(改之前用的)：这活儿是 **CTA 里的线程亲自干的** (CTA 就是一组协同干活的线程，等同于“线程块 block”；你可以理解成“一个工作小组”)。算每个数据在内存里的地址、发出读写指令，全得线程一条条来，而且搬的整个过程线程都被占着，啥别的也干不了。
- TMA 拷贝 (改之后)：**只要一个线程发一条命令，就完事走人**。后面的搬运全归 TMA 这块专用硬件，它自己独立去跑。那些琐碎又费劲的事——算地址、把相邻线程的访问拼成一笔大传输 (术语叫“合并访存 / coalescing”)、为避开内存读写冲突而打乱数据摆放 (术语叫 **swizzle**)——早就一次性写进了一份 **TMA 描述符 (descriptor)**(你可以理解成一张“搬运说明书”)里，TMA 引擎照着这张说明书做就行，不用线程操心。

两段代码摆一块儿，差别看得清清楚楚。

**改前 (Step 3)**:128 个线程一起上手搬，搬完再用一句 `cta_sync`(意思是“全组线程在这里集合一下，确保大家都搬完了”)让写进 SMEM 的数据对所有线程都可见：

```
Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_K:(i+1)*BLK_K]) # 128 A
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_K:(i+1)*BLK_K]) # B
T.cuda.cta_sync() # :
```

**改后 (Step 4)**:只挑一个线程发起 TMA load，至于硬件搬完没搬完，交给一个叫 `mbarrier` 的“门闩”去盯着：

```
tid = warp_id * 32 + lane_id # " 128 "(0..127)
if tid == 0: # 0 TMA
 Tx.copy_async(Asmem, A[...], dispatch="tma") # : A (TMA)
 Tx.copy_async(Bsmem, B[...], dispatch="tma") # , B
 T.ptx.mbarrier.arrive.expect_tx(tma_bar, byte_count) # :
 T.ptx.mbarrier.try_wait(tma_bar, phase) # :
```

这里出现了几个 GPU 专用的“咒语”，逐个说一下：`copy_async` 是“发起一笔异步搬运”(异步 = 发出去就不管了，后台自己跑)；`dispatch="tma"` 是指定“这笔搬运走 TMA 硬件”；`mbarrier` 这套 (`arrive.expect_tx` / `try_wait`) 就是那个门闩的用法，下面专门讲。

## 为什么是 `tid == 0`, 而不是 `elect_sync()`?

这是这一步最容易栽跟头的地方。GPU 里“从一群线程中挑出一个代表来干某件事”是很常见的操作, 通常有个现成的 `intrinsic`(`intrinsic` 就是编译器内置的特殊小函数) 叫 `elect_sync()`。乍一看, 挑线程嘛, 用哪种写法不都一样? 真不一样。

问题出在“它到底挑出了几个”。`elect_sync` 是**每个 warp 各挑一个活跃 lane**(lane 就是 warp 内线程的编号, 0~31; 你把它理解成“warp 里的第几号位”就行)。可一个 `warpgroup` 里有 4 个 warp, 于是 `elect_sync()` 一下子挑出了 **4 个线程**, 这 4 个全冲进了 `load` 协议。

坏就坏在这。TMA 协议要往 `mbarrier` 门闩里**报一个“预计搬多少字节”的数**, 而这个数必须**不多不少、正正好好报一次**。现在 4 个线程各报一遍, 门闩里的计数立马乱套——它以为要等 4 倍的字节, 于是 `wait` 永远等不到那个正确的“放行点”。更阴的是, 这是个**不声不响的正确性 bug**: 程序照跑、不崩溃, 就是悄悄给你算出错误结果。这种 bug 最难查。

所以对的做法是: 用 `warpgroup` 内的全局编号, 精准点中**唯一一个线程**——也就是 `tid == 0` (第 0 号线程)。

### 关键

**关键:** 示意图里写的“Elected Thread(被选中的线程)”指的是启动 TMA 的那一个线程, 在我们代码里就是 `tid == 0`, 不是 `elect_sync()` 选出来的那批 lane。这俩名字像, 可别混了。

## Load 完成怎么知道: `mbarrier` + 字节数握手

### 一句话先理解

**一句话先理解:** 既然 TMA 是“发出去就走人、后台慢慢搬”, 那线程怎么知道数据到底搬完没有? 靠一个叫 `mbarrier` 的门闩, 它按“搬够了多少字节”来判断, 够了才放行。

换成 TMA, 其实一口气改了两件事: **谁来发起搬运** (这个看代码就懂), 以及**怎么判断搬完了** (这件事最容易被忽略, 可一错就是前面说的那种静默算错)。下面重点讲后一件。

- 还是用 `Tx.cta.copy`(线程亲自搬) 的时候: 大家一起搬, 后面补一句 `cta_sync()` (全组集合) 就够了——因为搬运是线程自己干的, 集合了就说明都搬完了。
- 换成 TMA 之后, `cta_sync()` 就不顶用了。为什么? 因为 `cta_sync()` 只会等“线程自己”干的活, 只管线程写进 SMEM 的那些数据。可现在数据是 **TMA 后台**在搬的, `cta_sync()` 对这后台传输**压根不知情**——tile 还在半道上呢, 它一看线程都没在搬东西, 就乐呵呵地“集合完毕、放行”了。结果就是 MMA 拿着半截数据去算。

正确的修法, 是把“搬完了”这件事用门闩**明明白白地盯住**:

1. 被选中的那个线程, 先用 `arrive.expect_tx(total_bytes)` 告诉门闩: 这回要等够这么多字节才算搬完。
2. TMA 引擎把这些字节实打实搬齐之后, 会去“敲”一下门闩; 门闩一看字节凑够了, 对应的 `mbarrier.try_wait(phase)` 才放行。
3. **非得到放行这一步**, SMEM 里的 tile 才能交给 MMA 去算。MMA 是 `matrix multiply-accumulate` (矩阵乘加) 的缩写——就是“两块小矩阵相乘、再累加到结果上”这个动作, 正是张量核心专门干的那堆活。



**Try with your agent(原书练习):** 为一个 K tile 追踪 Step 4 的 load/store 同步——哪个线程启动每条 TMA 命令、哪个 mbarrier 或 commit group 跟踪完成、哪个等待保护 MMA 读 Asmem/Bsmem、哪个等待保护 Dsmem 的复用? 为什么这里 `elect_sync()` 是错误的线程选择?

完整 kernel 的结构与关键点

完整的 kernel, 无非是把上面这套 TMA load/store 塞进 Step 3 的骨架里, 别的地方一概不碰。下面不照抄那一长串源码 (对零基础读者没意义), 只挑出**真正扛着 TMA 语义的那几行**, 逐行讲。

先看最外层的 K 循环。这里插一句背景: 矩阵乘是沿着 K 这个维度一块一块累加的, 所以要循环 K\_TILES 次, 每次算一块、累加到结果上。请特别注意这个循环里**每一轮都要等两次** (等搬完、等算完), 所以这一步**还谈不上真正的重叠**:

```
for k in range(K_TILES): # K
 k_st = T.meta_var(k * BLK_K) # K
 if tid == 0:
 tma_load(k_st) # 0 TMA
 T.ptx.mbarrier.try_wait(tma_bar.ptr_to([0]), phase_tma) # : ?(SMEM)
 if tid == 0:
 mma(accu=k != 0) # 0 (k==0
 ↪ ,)
 T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # : ?
 phase_tma ^= 1 # " "(
 phase_mma ^= 1
```

这里出现一个新东西: `phase`(相位) 和 `^= 1`(异或翻转, 效果就是在 0 和 1 之间来回切)。先简单理解: 同一个门闩会被反复使用, 它需要一个 0/1 的标记来区分“这是第几轮的放行”, 不然分不清等的是这一轮还是上一轮。Step 5 会把相位讲透, 这里先放着。

接着看那两个 `@T.inline` 辅助函数 (`@T.inline` 是个装饰器, 意思是“编译时把这个小函数的代码直接展开、塞进调用处”, 所以它能直接用周围 kernel 里的变量, 不用传参)。先看 `tma_load`, 它发两条搬运命令, 再顺手把“预计搬多少字节”报给门闩:

```
@T.inline
def tma_load(k_st):
 tma_config = T.meta_var({"dispatch": "tma", "cta_group": 1, # " ": TMA
 "mbar": tma_bar.ptr_to([0])})
 Tx.copy_async(Asmem[:, :], A[m_st:m_st+BLK_M, k_st:k_st+BLK_K], **tma_config) # A
 Tx.copy_async(Bsmem[:, :], B[n_st:n_st+BLK_N, k_st:k_st+BLK_K], **tma_config) # B
 T.ptx.mbarrier.arrive.expect_tx(
 tma_bar.ptr_to([0]),
 (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE) # : A B
```

逐行说: `tma_config` 是把“走 TMA、用 `tma_bar` 这个门闩”打包成一份配置; 两条 `copy_async` 分别搬 A、B 的当前块; 最后一行算出 A、B 两块加起来的总字节数 (块的行 × 列 × 每个数的字节数), 报给门闩——门闩就靠这个数判断“搬够了没”。

整段 kernel 里, 真正扛着 TMA 语义的就**五个点**。把这五个记住, 这一步也就拿下了:

1. **TMA 配置**:`{"dispatch": "tma", ...}`——告诉 `copy_async` 走 TMA 硬件”，并指定用哪个门闩汇报完成。
2. **字节数**:`(BLK_M * BLK_K + BLK_N * BLK_K) * 2`——A、B 两块输入 (operand, 即”操作数”，参与运算的输入数据) 加起来的字节数 (乘 2 是因为 fp16 每个数占 2 字节)，喂给门闩当作”要等够的量”。
3. **门闩初始化**:`init(tma_bar.ptr_to([0]), 1)`——用之前先把门闩立起来 (参数 1 表示”等 1 个到达信号”就放行)。
4. **@T.inline:tma\_load(...)** 和 **mma(...)** 是辅助小函数，编译时展开进主体，所以能直接用周围的变量。
5. **store 同步**:`epilogue`(收尾阶段，指 K 循环算完后把结果写回的那段代码) 先把 fp16 结果写进 Dsmem，用 `fence.proxy_async` 和 `warpgroup_sync` 这两道同步把线程的写入和 TMA 回写通路对齐 (确保 TMA 读到的是写好的数据)，再用 `commit_group()` / `wait_group(0)` 等回写跑完。

### 关键

**关键**:Step 4 的提速**不是靠重叠** (它每搬完一块还在傻等)。提速纯粹来自**换了搬运方式**:TMA 接管了大块数据的批量传输，把线程从”一条条指令吭哧吭哧搬数据”里彻底解放出来。光这一项，性能就往前挪了一截。一句话——把搬运这件苦差事从”必经之路”上拿掉，就是 Step 4 的全部价值。

所以眼下的局面是：**零件配好了，节奏还没踩上**。搬运和计算还是一个接一个轮着来。下一步，TMA 这套搬运通路一个字都不动，我们只去重新排时间表。

## Step 5: 软件流水线 (PIPE\_DEPTH=2)

为什么 Step 4 没法重叠？瓶颈在”存储”

### 一句话先理解

**一句话先理解**：两个引擎想同时干，可手头只有一块格子放数据——正算着的那块占着，新数据没地方搬进来。解法就是多备一块格子。

两个引擎明明各干各的、互不相干，Step 4 怎么就是重叠不起来？卡点不在硬件，在**没地方放数据**。

你想象一下：手头只有一对 SMEM 格子 (一块放 A、一块放 B)。下一块数据想提前搬进来，可它**没地方落脚**——当前的 MMA 还在读这对格子呢，你这会儿往里搬，就把人家正用着的数据给覆盖了。说白了，就是缺第二块空地。

Step 5 的招就是**双缓冲 (double buffering)**：多准备一块格子。这个名字你大概率熟——和图形界面里”一块屏幕在显示、另一块在后台画”是一个道理。这边也一样：一块格子在被 MMA 读 (算着)，另一块格子同时被 TMA 写 (搬下一块)，互不打架。

不过有句话得先说在前头，免得你期望过高：这一步单 `warpgroup` 的主循环**还是先等当前这块算完、再发下一笔搬运**，并没有真正同时跑。区别只在于：现在有了”两块分开的格子”，可以**预取** (提前把下一块先搬着)、可以轮着复用了。真正的同时跑，留给 Step 7。



| 维度       | Step 5 的状态                                                                                                                                                    |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scope    | 不变，仍是一个 warpgroup                                                                                                                                             |
| Layout   | 变了：原来一对 SMEM 格子，现在变成 PIPE_DEPTH 块格子组成的 <b>环形缓冲 (ring buffer)</b> ——就是几块格子排成一圈、用完一块转到下一块、转完一圈再回到开头，像跑道一样循环利用                                                   |
| Dispatch | 不变，仍是 TMA load + tcgen05 MMA(tcgen05 是 Blackwell——NVIDIA 较新一代 GPU 架构——这一代张量核心专用的矩阵乘法指令族；它算出的累加器，即”逐块往上累加的中间结果”，存在前面提到的 TMEM 里)；本步只加了预取和格子轮换，真正的完整重叠仍留给 Step 7 |

流水线骨架长什么样

PIPE\_DEPTH=2 说白了就是：开 2 个 SMEM stage(阶段 / 格子位)，让”搬数据”和”算数据”各有各的格子用，不再抢同一块。(stage 在这里你就理解成”环形缓冲里的一个槽位、一块格子”。)

下面这张图，画的是**双缓冲想撑起的那套理想流水线结构** (PIPE\_DEPTH=2)，注意它**不是**这个单 warpgroup kernel 真实跑起来的样子。别会错意——这一步只是先把”两块格子轮换 + 预取”的架子搭出来，真要两条线跑满重叠，得等 Step 7。图里每个 stage 就是一块 SMEM 格子，各个 K tile 轮着用它：

结构图

分组：

- stage 0 (一块 SMEM 格子)：「k0 → k2 → k4 → ...」
- stage 1 (另一块 SMEM 格子)：「k1 → k3 → k5 → ...」

连接关系：

- 启动：两次 TMA load 先填满两个 stage (prefetch) → stage 1 (另一块 SMEM 格子)
- stage 0 (一块 SMEM 格子)  $\xrightarrow{\text{等 stage} \rightarrow \text{跑 MMA} \rightarrow \text{等 MMA 读完} \rightarrow \text{空出后 load } k + \text{PIPE\_DEPTH}}$  stage 1
- 每轮循环：轮转复用两个 stage
- stage 1 (另一块 SMEM 格子) → 每轮循环：轮转复用两个 stage

注意

注意：这还不是并发的 TMA/MMA 时间表；它只是建立了环形缓冲结构,Step 7 会把这个结构拆成生产者/消费者两个角色，届时 TMA 和 MMA 才真正同时跑。

相对 Step 4 的四处改动

Step 5 的代码跟 Step 4 比，就差这四处 (都是为了”多一块格子轮着用”服务的)：

1. 存放 A、B 的 Asmem / Bsmem 前头多加了一个 PIPE\_DEPTH 维度——相当于从”一块格子”变成”一排 PIPE\_DEPTH 块格子”，每个 stage 各占一块。
2. 门闩 tma\_bar 从一个变成一个数组：一个 stage 配一个门闩 (因为现在有好几块格子同时在被搬，得分别盯着)。
3. 主 K 循环开跑之前，先把头两个 stage 提前预取好，这样循环一开始就有现成数据可算。

4. K 循环里用 `stage = k % PIPE_DEPTH` 来轮转选格子 (% 是取余,2 块格子时结果就是 0、1、0、1……循环): 等当前 `stage` 备好 → 在它上面算 MMA → 再把这块刚算完的格子拿去搬 `k + PIPE_DEPTH`(也就是”再往后 `PIPE_DEPTH` 块”的那块)。

流水线机制三件套

下面三段都是示意伪代码, 只为把机制讲清楚 (为了读着清爽,if `tid == 0` 守卫和 `try_wait` 的完整写法都略掉了); 带守卫的真实版本, 看后面”完整 kernel”那一节。

(1) 预取 (Prefetch): 主循环还没开跑, 先把头 `PIPE_DEPTH` 块格子都搬满, 这样循环第一轮一上来, 就有现成数据可算, 不用干等:

```
for s in range(min(PIPE_DEPTH, K_TILES)): # PIPE_DEPTH (,)
 tma_load(s, s * BLK_K) # s s stage
```

(2) 主循环: 每个 K tile, 先等它所在的格子备好、在上面把 MMA 算了; 然后趁这块格子刚腾空, 马上拿去搬”再往后 `PIPE_DEPTH` 块”的那块——这一手就是让搬运提前跑、为重叠埋伏笔:

```
stage = k % PIPE_DEPTH # (0 1)
wait(tma_bar[stage], phase_tma) #
mma(stage, accum) #
wait(mma_bar[0], phase_mma) # (MMA ,)
phase_mma ^= 1
tma_load(stage, next_k * BLK_K) # , ()
```

(3) 相位管理 (Phase management): 这块最容易把人绊倒, 先把”相位”是啥说清楚。

一句话先理解

一句话先理解: 同一个门闩会被反复使用, 它靠一个 0/1 的标记 (相位) 来区分”这是第几轮的放行”。轮到它再次被用之前, 就得把这个标记翻一下 (0→1 或 1→0), 否则它分不清你等的是新这一轮还是上一轮, 可能直接误放行。

为什么需要它? 因为门闩是循环复用的。同一个门闩, 第一轮、第二轮……都在用它。如果不做个区分,try\_wait 就没法判断”现在该等的, 到底是这一轮搬的数据, 还是上一轮残留的状态”。相位就是那个区分标记。

规则其实就一句话——一个门闩多久翻一次相位, 全看它隔多久才被再次用到。我们这两个门闩正好一对照:

- `mma_bar`(就一个门闩):MMA 的累加器只占一个 TMem 槽位, 所以算完这件事只需要一个门闩, 每一轮都会用到它。每轮都用到 → 当然每轮都得翻一次相位:

```
“python phase_mma ^= 1 # “
```

- `tma_bar`(是个数组,PIPE\_DEPTH 个): 每块格子配一个门闩。某一块格子的那个门闩, 得等环形缓冲转完一整圈、重新轮回到这块格子时, 才会被再次用到。所以 `phase_tma` 只在格子下标绕回起点 (转完一圈) 的那一下翻一次:

```
“python if stage == PIPE_DEPTH - 1: # = phase_tma ^= 1 # “
```

关键

**关键:** 记住这条规律——门闩多久被再次用到, 相位就多久翻一次。mma\_bar 每轮都用 → 每轮翻;tma\_bar[stage] 每 PIPE\_DEPTH 轮才轮回到同一个 → 转完一圈才翻。这是 mbarrier 相位机制最直接的推论。

**Try with your agent(原书练习):** 取 PIPE\_DEPTH=2、K\_TILES=5, 逐 k 追踪主循环——列出每轮的 stage、传给 try\_wait 的 phase\_tma/phase\_mma、以及是否发出新的预取。phase\_tma 究竟在哪里翻? 为什么最后两轮没有预取?(提示:next\_k = k + PIPE\_DEPTH < K\_TILES 不成立时就不再预取。)

完整 kernel: 关键差异

完整 kernel 把 Step 4 那条 TMA load/store 通路**原封不动地留着**, 外面只是又裹了一层”多块格子 + 相位”的逻辑。结构上的不同就集中在三处, 下面各挑几行最能说明问题的。

第一处, 双缓冲的内存布局——给 A、B 各加一个 stage 维度 (就是”从一块格子变一排格子”):

```
PIPE_DEPTH = 2
A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM,
 (PIPE_DEPTH, BLK_M, BLK_K)) # PIPE_DEPTH:
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM,
 (PIPE_DEPTH, BLK_N, BLK_K))
```

(SwizzleMode.SWIZZLE\_128B\_ATOM 是前面提过的 swizzle——为避开内存读写冲突而定的数据摆放方式, 这里照搬即可, 不用深究。)

第二处, 门闩初始化——算的那个门闩只要 1 个, 搬的那个每块格子配 1 个:

```
if warp_id == 0 and lane_id == 0:
 #
 T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # : 1
 for s in range(PIPE_DEPTH):
 T.ptx.mbarrier.init(tma_bar.ptr_to([s]), 1) # : 1
```

第三处, 主循环里比 Step 4 多了”顺手发下一笔预取”和”转完一圈才翻 phase\_tma”两步:

```
for k in range(K_TILES):
 stage = k % PIPE_DEPTH #
 T.ptx.mbarrier.try_wait(tma_bar.ptr_to([stage]), phase_tma) #
 if tid == 0:
 mma(stage, accum=(k != 0)) # ()
 T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) #
 phase_mma ^= 1 #
 next_k = k + PIPE_DEPTH #
 if next_k < K_TILES: # ()
 if tid == 0:
 tma_load(stage, next_k * BLK_K) #
 if stage == PIPE_DEPTH - 1: #
 phase_tma ^= 1 #
```

epilogue(算完后:TMEM 里的结果 → 寄存器 → Dsmem 暂存 → TMA 搬 → 大显存) 跟 Step 4 一模一样, 这里就不再展开了。

## Step 6: 持久化 kernel + tile 调度器

从” 优化单 tile 内” 到” 优化跨 tile”

### 一句话先理解

**一句话先理解:** 前面都在抠” 算一块 tile” 内部怎么快;Step 6 拉近镜头, 管的是” 一个个 tile 之间”——少开张几次、挑个好顺序, 把整体开销摊薄。

前面那些功夫, 都花在优化单个 tile 内部的活儿上。Step 6 把镜头往后一拉, 换了个尺度看——跨 tile 优化。

先回顾 Step 5 怎么干的: 每算一个  $128\times128$  的输出 tile, 就开一个 CTA(还记得吗,CTA 就是一组协同干活的线程, 等同 block)。对  $4096\times4096$  的输出, 要算 **1024 个 tile**, 那就是开 **1024 个互不相干的 CTA**, 一个接一个开张、干完一块就直接解散、消失。问题是: 每个 CTA 一开张都得做一堆准备工作 (分配内存、立门闩等), 开 1024 次就把这套准备重复了 1024 遍, 纯属浪费。

Step 6 换个活法: 不再” 算一块开一批”, 而是一开始就固定养一池 CTA, 让每个 CTA 留着不走、轮着去啃好几个 tile。这么一来, 能换两样好处:

1. **摊薄开张开销:** 分配 TMEM、初始化门闩、准备调度器状态这些准备工作, 原来在 1024 个一次性 CTA 上要重复 1024 遍, 现在每个常驻 CTA 只做一次, 这一次的成本摊到它经手的大约 7 个 tile 上, 平均下来就便宜多了。
2. **更聪明的处理顺序:** 既然” 哪个 CTA 算哪个 tile” 现在由程序内部决定了 (而不是一股脑全交给硬件随机分), 就能特意挑一个**让数据能被反复利用**的顺序来跑 (下面细讲)。

| 维度       | Step 6 的状态                                  |
|----------|---------------------------------------------|
| Scope    | 变了: 固定养一池常驻 CTA, 每个 CTA 靠一个调度器循环处理多个输出 tile |
| Layout   | 不变, 每个 tile 内部的 SMEM/TMEM/寄存器通路照旧           |
| Dispatch | 不变                                          |

持久化调度:grid 大小贴合硬件, 而非问题

先解释两个词。**grid(网格):** 启动一个 kernel 时, 你要告诉 GPU” 总共开多少个 CTA”, 这一大坨 CTA 整体就叫 grid。**SM(流多处理器):**GPU 内部的计算核心, 一块 GPU 上有很多个 SM, 每个 SM 同时只能容纳有限的 CTA(详见第 0 章)。

### 一句话先理解

**一句话先理解:** 以前” 有多少个 tile 就开多少个 CTA”(grid 大小跟着问题走); 持久化反过来——”GPU 有多少个 SM 就开多少个 CTA”(grid 大小跟着硬件走), 开出来的 CTA 不解散, 循环去领新 tile。

持久化 kernel 的核心想法, 一句话:**grid 开多大, 看硬件有多少 SM, 而不看问题有多少 tile**。

换句话说, 它就开 SM\_COUNT 个 CTA(大致一个 SM 配一个), 不管你输出 tile 总共多少个。图的就是一件事: 让每个 SM 手里始终有活干, 别让它空着。

注意

**注意:** 这里说”大致”是有讲究的——精确的”一个 SM 正好驻一个 CTA”没人保证, 具体还得看占用率 (occupancy, 即一个 SM 实际能同时塞下多少 CTA), 以及硬件怎么调度。

这一章的目标硬件是 NVIDIA B200, 它有 SM\_COUNT=148 个 SM。于是就开 148 个 CTA, 各自循环去处理一个叫 ClusterPersistentScheduler2D 的调度器派给它的那些 tile。

第二个好处, 来自调度器挑的那个**处理顺序**。这里把 l2\_group\_size 设成 8, 意思是”把相邻的 8 个 tile 凑成一组、一个挨一个地处理”。为什么这样能省? 看下面这张输出 tile 的网格图 (行 = M 方向, 列 = N 方向), 每格是一个 128×128 的输出 tile:

| M 方向\ N 方向 | N=0 | N=1 | N=2 | N=3 |
|------------|-----|-----|-----|-----|
| M=0        | T0  | T1  | T2  | T3  |
| M=1        | T4  | T5  | T6  | T7  |

表里的复用关系很清楚: **同一行** (同一个 M) 上的那几个 tile, 算的时候用的都是 A 矩阵的同一份行块; **同一列** (同一个 N) 上的, 用的都是 B 矩阵的同一份块。换句话说, 挨着的 tile 之间, 输入数据是大量重叠的。

道理就在这儿。这里要引入 **L2** 和 **HBM** 两个词:HBM 是显卡上那块大显存 (就是前面说的 GMEM, 容量大但相对慢),L2 是一块容量小、但快得多的**缓存** (数据从 HBM 拉一次后会暂存在 L2, 下次再用就能就近拿、不必再跑去 HBM)。既然挨着的 tile 共用大量输入, 那把这些”沾亲带故”的 tile 一个挨一个地处理, 它们共用的那份输入就能**一直赖在 L2 缓存里**, 后面的 tile 直接从 L2 拿, 不用一趟趟跑回慢吞吞的 HBM 重新拉。这就是 l2\_group\_size=8 想榨取的红利——它在 Step 3 时其实就摆在桌面上了, 只是当时没顺手吃掉。

接进调度器, 几行就够:

```
bx = T.cta_id([SM_COUNT]) # grid: CTA SM(tile grid)

tile_scheduler = ClusterPersistentScheduler2D(# : tile
 "ts",
 num_m_tiles=M // BLK_M, # M tile
 num_n_tiles=N // BLK_N, # N tile
 l2_group_size=8, # 8 tile (L2)
 num_clusters=SM_COUNT)
tile_scheduler.init(bx) # CTA bx (" ")
```

一个容易漏掉的正确性后果: 相位初始化要进循环

一个 CTA 现在要循环啃好几个 tile, 这就牵出一个**特别容易漏的坑**, 还是个静默 bug。每个 tile 都要重新跑一遍自己**全新的 K 循环**, 而前面讲过, 门门的相位必须从一个干净、已知的状态 (0) 起步, 否则 try\_wait 会判断错。

- Step 5 里, 一个 CTA 只管一个 tile, 所以 phase\_tma、phase\_mma 在最开头初始化一次就完事, 没毛病。
- 到了 Step 6 就不行了: 如果还放在最外面只初始化一次, 那么第 2 个 tile 一开始, 接手的就是**上一个 tile 留下的相位残值**, 节奏立马错位。所以这俩初始化必须**挪进 while tile\_scheduler.valid()** 这个 tile 循环里头, 让每个 tile 都从 0 重新起步。

```

while tile_scheduler.valid(): # tile ,
 phase_tma: T.int32 = 0 # tile
 phase_mma: T.int32 = 0
 ... # Step 5 K
 tile_scheduler.next_tile() # tile ,

```

(`tile_scheduler.valid()` 是问调度器” 还有没有派给我的 `tile`”;`next_tile()` 是” 我这块干完了, 给我下一块”。)

**完整 kernel: 结构上就是 Step 5 套了个外层 `tile` 循环**

从结构上讲, Step 6 不过是把 Step 5 那整条流水线整个套进一个 `tile` 级的外层循环罢了。新加的依赖就一个——调度器本身:

```

from tvn.tirx.lang.tile_scheduler import ClusterPersistentScheduler2D

```

骨架长这样 (内层那个 `K` 循环跟 Step 5 一字不差, 不再展开):

```

SM_COUNT = 148 # NVIDIA B200 SM

... Step 5 SMEM/ /TMEM , : CTA ...

while tile_scheduler.valid(): # : tile
 m_st = T.meta_var(tile_scheduler.m_idx * BLK_M) # tile, ()
 n_st = T.meta_var(tile_scheduler.n_idx * BLK_N) # ()
 phase_tma: T.int32 = 0 # tile ()
 phase_mma: T.int32 = 0
 # === : Step 5 ===
 # PIPE_DEPTH -> K (-> -> -> ->)-> epilogue
 T.cuda.cta_sync() # tile ,
 tile_scheduler.next_tile() # tile

:TMEM while (CTA)

```

### 关键

**关键:** 注意 `TMEM` 的分配 / 释放 (`alloc/dealloc`) 都在整个 `while` 循环之外——它们跟着 `CTA` 的整段生命周期走, 只在最开头分配一次、最末尾释放一次, 绝不在每个 `tile` 上重做。这正是前面说的” 摊薄开张开销” 落到代码上的样子。

## 练习 (原书 Exercises)

- Step 4:** `arrive.expect_tx` 用的字节数是  $(\text{BLK\_M} * \text{BLK\_K} + \text{BLK\_N} * \text{BLK\_K}) * 2$ 。如果这个字节数偏小或偏大, `mbarrier` 会等什么?(思路: 偏小则字节没凑齐它就提前释放, `MMA` 读到不完整数据; 偏大则永远凑不齐, `wait` 死等不放。)
- Step 5:** 为什么每个 `SMEM` stage 需要自己的 `TMA` barrier, 而不能两个 stage 共用一个 `tma_bar`?(思路: 两个 stage 的 load 在途时间不同, 共用会让” 哪个 stage 好了” 分不清, 相位语义也会乱。)

3. **Step 6:**4096×4096 输出、BLK\_M=BLK\_N=128, 有多少个输出 tile?SM\_COUNT=148 时每个持久化 CTA 平均处理几个?(算一下: 每边 4096/128 = 32 个, 共 32×32 = **1024** 个 tile;1024/148 ≈ **6.9**, 即平均约 7 个。)

小结

这一章的主线就一句话: **让一个硬件引擎跑在另一个前头**。下面三步, 就是把这句话一点点落到地上的过程:

| 步骤     | 改的维度     | 核心动作                                                                | 是否真重叠                         |
|--------|----------|---------------------------------------------------------------------|-------------------------------|
| Step 4 | Dispatch | 同步拷贝 → TMA 命令;mbarrier+ 字节数跟踪 load,commit/wait group 跟踪 store       | 否 (每 load 后仍等); 提速来自把搬运移出关键路径 |
| Step 5 | Layout   | SMEM 双缓冲 (PIPE_DEPTH=2 环形缓冲)+ 预取; 相位翻转节奏 ∝ barrier 槽位数              | 否 (单 warpgroup 仍等); 搭好流水线骨架   |
| Step 6 | Scope    | 持久化 kernel(— SM — CTA)+ tile 调度器;l2_group_size 控操作数 L2 复用; 相位初始化进循环 | 否; 摊薄启动开销 + 跨 tile 操作数复用      |

几条最该揣走的认知 (都是上面讲过的, 这里给你拎成一句话好记):

- **TMA 的价值有两层:**Step 4 先吃到的是”把搬数据这件苦力活从线程手里摘出去”; 至于真正的”搬和算同时跑”(重叠), 那是后话 (Step 7 的 warp 专门化)。
- **两套同步机制别串了:** 搬进来 (load) 用门闩 mbarrier + expect\_tx 报字节数; 搬回去 (store) 用 commit group + wait group。拿 cta\_sync()(全组集合) 去等 TMA load, 是个不声不响的错——因为它根本不知道后台还有 TMA 在搬。
- **挑线程发命令用 tid == 0, 别用 elect\_sync():** 后者每个 warp 各挑一个、一共挑出 4 个,4 个线程会把字节数重复报 4 遍, 直接把门闩计数搞坏, 静默算错。
- **相位翻转记个口诀:** 门闩每轮都被用到, 就每轮翻 (mma\_bar); 每转完一圈才被用到一次, 就绕圈翻 (tma\_bar[stage])。
- **持久化换来两份红利:** 摊薄每个 tile 的开张成本, 外加用聪明的 tile 顺序换来 L2 缓存复用; 代价是相位初始化和预取都得搬进 tile 外层循环里头。

下一章 (从 Step 7 起) 会上 warp 专门化, 把 TMA 和 MMA 拆成生产者、消费者两个角色。到那会儿, 这一章搭好的这套环形缓冲, 才真正两条线一起跑满。

延伸阅读

- 原文:[Pipelining GEMM with TMA —Modern GPU Programming for ML](#)Sys
- 本章末尾提到的端到端计时, 见下一章 *Scaling GEMM with Warp Specialization and Clusters* 的 *End-to-End Result* 表。

术语对照

| 中文         | English                         | 说明                                |
|------------|---------------------------------|-----------------------------------|
| 张量核心       | Tensor Core                     | 芯片上最贵、最该跑满的计算单元                   |
| 共享内存       | SMEM (shared memory)            | tile 在片上的落脚点                      |
| 线程束        | warp                            | 32 个 lane 一组                      |
| warpgroup  | warpgroup                       | 4 个 warp 一组, 共 128 线程             |
| 张量内存加速器    | TMA (Tensor Memory Accelerator) | 专门搬 GMEM↔SMEM tile 的硬件引擎          |
| 内存屏障       | mbarrier (memory barrier)       | 用字节数/到达数跟踪异步完成                    |
| 相位         | phase                           | mbarrier 的翻转位, 区分轮次               |
| 双缓冲        | double buffering                | PIPE_DEPTH=2 的两阶段环形缓冲             |
| 环形缓冲       | ring buffer                     | 多 stage 轮转复用的 SMEM 结构             |
| 预取         | prefetch                        | 主循环前先填满头几个 stage                  |
| 持久化 kernel | persistent kernel               | grid 按硬件 (SM 数) 定,CTA 循环处理多 tile  |
| tile 调度器   | tile scheduler                  | 在 kernel 内分配 tile 并挑复用友好的顺序       |
| warp 专门化   | warp specialization             | 把 TMA/MMA 拆成生产者/消费者 (Step 7, 下一章) |
| 提交组 / 等待组  | commit group / wait group       | TMA store 的完成跟踪机制                 |
| 张量内存       | TMEM (tensor memory)            | tcgen05 MMA 累加器所在的片上内存            |



# 第 13 章 · 用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)

原文:用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)

## 本章要点

### 本章要点 (TL;DR)

- 上一章那个流水线 GEMM(矩阵乘法), 还是把「加载 / 计算 / 写回」三件事全压在同一队线程 (一个 warpgroup) 身上, 结果三个干活的硬件引擎 (负责搬数据的 TMA、负责算矩阵乘的 Tensor Core、负责把结果写出去的回写路径) 只能你忙我闲、轮流空转。本章用 Step 7-9 三步, 一级一级把「谁跟谁协作」的范围撑大, 一路逼近业界最高的吞吐。下面这些名词 (warpgroup、TMA、Tensor Core……) 正文里都会从零讲清楚, 这里先囫圇看个大概就行。
- **Step 7(Warp 专门化 + 流水线):** 把 warp 拆成 TMA 生产者 / MMA 消费者 / 写回三个并发角色, 靠 4 个方向相反的 barrier(`tma2mma`、`mma2tma`、`mma2ld`、`ld2mma`) 和 `PipelineState` 串起来; 生产者和消费者的初始 phase 必须反着, 否则不是死锁就是静默的数据损坏。
- **Step 8(2-CTA Cluster):** 把两个 CTA 凑成 cluster, 靠 DSMEM 跨 CTA 共享操作数, 用一条 `cta_group=2` 的协同 MMA 算出  $256 \times 256$  的 tile, 把**算术强度大约翻一倍**——这是真收益, 不是把活儿重分了一遍。
- **Step 9(多消费者 Warp 专门化):** 再添一个 MMA 消费者, 拿「不同的  $A \times$  同一块 B」把每个 CTA 的计算密度翻倍, cluster 输出长到  $512 \times 256$ , 是本教程里最稠密、最优的那个 GEMM 变体。
- 三步只改「谁协作」, 不动「数据怎么摆」: SMEM / TMEM / 寄存器布局都沿用前两章的契约。端到端从朴素的 70 ms 一路压到 0.094 ms(约 744 $\times$ ), 跟 cuBLAS 打平; 收益越往后越小, 是逼近计算受限之后的边际递减。

## 前置知识

**前置知识:** 这一章会用到一串 GPU 专有名词——warp(线程束)、warp 专门化、cluster(集群)、TMEM(Tensor Core 专用的累加器内存)、mbarrier/phase(屏障与相位)。别担心, 每个词第一次出现时, 我都会当场用大白话给你讲明白「它是什么、为什么要有它」, 你不需要事先就懂。不过如果你想先建立一点点 GPU 的整体直觉 (线程怎么并行、内存为什么分好几层、矩阵乘为什么是主角), 花十分钟翻一下第 0 章·极简入门会让这一章读起来更顺。本章直接接着第 11-12 章往下走 (TMA 流水线、`tcgen05` MMA、TMEM 累加器这几样东西的「规矩」都是那两章定下的), 但即使你没读过那两章, 跟着本章的解释也能读懂。

## 引言: 用 Warp 专门化与 Cluster 把 GEMM 推到极致 (Scaling GEMM with Warp Specialization and Clusters)

### 上一章遗留的瓶颈: 一个 warpgroup 包揽所有事

上一章我们用 TMA 把 GEMM 做成了流水线, 跑得已经挺快了。可它骨子里有个毛病。

#### 一句话先理解

**一句话先理解:** GPU 上干活的最小集体不是「一个线程」, 而是一小撮线程绑在一起一起动; 而真正负责干重活的「硬件引擎」就那么几种, 关键是别让它们排队空等。

在动手之前, 先把这几个词捋清楚——后面整章都靠它们:

- **线程 (thread):** 跟你写后端代码时理解的线程一样, 就是一条独立的执行流。GPU 的特别之处是它一次能跑成千上万个。
- **warp(线程束):** GPU 不是一条线程一条线程地调度, 而是把 32 条线程捆成一束、让它们齐步走——这一束就叫一个 warp。打个比方, 就像 32 个人排成一个方阵, 喊一个口令大家一起迈同一步。为什么要这么设计? 因为这样硬件只需要为 32 条线程发一次指令, 省下大量调度开销。
- **warpgroup:** 再把 4 个 warp(也就是 128 条线程) 凑成一支更大的队伍, 叫一个 warpgroup。本章里它是「分配任务」的基本单位——我们会把不同的活派给不同的 warpgroup。

#### 注意

**注意:** 本章里 GPU 的硬件细节 (比如 32、128 这些数字, 还有下面要说的几种引擎) 都按 NVIDIA 较新的架构来讲。你不用记数字, 记住「线程 → 32 条凑成 warp → 4 个 warp 凑成 warpgroup」这个层级关系就够用了。

毛病出在哪? 上一章那条流水线从头到尾就只用了一个 warpgroup(一支 128 线程的队伍), 却要它一个人按顺序干完三件压根不是一回事的活——

1. **发起搬运 (load, 加载):** 把下一块要算的输入数据, 从又大又慢的「全局内存」搬进又小又快的「共享内存」。这里有几个新词:
  - **tile(分块):** 整个大矩阵太大, 塞不进快速内存, 所以我们把它切成一个个小方块, 一块一块地算。每个小方块就叫一个 tile。
  - **全局内存 (GMEM, Global Memory):** GPU 上容量最大、但访问最慢的那层内存, 整个 GPU 共用, 相当于「主存」。
  - **共享内存 (SMEM, Shared Memory):** 容量小得多、但快得多的一层片上内存, 由程序员手动安排往里放什么——你可以把它当成「一块需要你亲手管理的高速缓存」。为什么要先把数据搬进 SMEM 再算? 因为直接从龟速的 GMEM 取数, 算力会被活活饿死; 先搬进近在咫尺的 SMEM, 后面反复读才划算。
  - **TMA(张量内存加速器 / Tensor Memory Accelerator):** 一个专门负责搬数据的硬件引擎。它能在后台异步把一整块 tile 从 GMEM 搬到 SMEM, 搬运期间线程不用守着、可以去干别的——这正是它存在的意义。
2. **跑矩阵乘 (MMA, Matrix-Multiply-Accumulate, 矩阵乘累加):** 在搬进来的这块数据上做矩阵乘法。

- **Tensor Core**: 一个专做矩阵乘累加的硬件引擎，一条指令就能算一小块矩阵乘，比用普通运算单元一格一格乘快几个数量级。GEMM 之所以能在 GPU 上飞起来，全靠它。
- **tcgen05** 则是用来「指挥」Tensor Core 的那条指令的名字，后面会反复见到，你把它当成「发起一次矩阵乘」的命令就行。

3. **写回 (writeback)**: 把算好的结果搬出去，写回内存。

- **TMEM(张量内存 / Tensor Memory)**:Tensor Core 专用的一小块「累加器内存」，矩阵乘的中间累加结果就攒在这里。算完一整块 tile 之后，得把 TMEM 里的结果「排空」——读出来、转成最终格式、再写回 GMEM。

就算用软件流水线把这三步在时间上稍微叠了一点，根子上的问题还在: **三种硬件引擎 (TMA、Tensor Core、写回路径) 全堵在同一队线程身上**。这队线程就成了三个引擎共用的独木桥，谁想干活都得排队过这队线程，自然谁也快不起来。

症状: 引擎轮流空转

这瓶颈长啥样? 把三个引擎的忙闲按时间步摆开，一眼就看明白了 (行是引擎，列是时间步，表示在干活，空白表示闲着):

| 引擎          | t0 | t1 | t2 | t3 | t4 | t5 |
|-------------|----|----|----|----|----|----|
| TMA 单元      | 忙  |    |    | 忙  |    |    |
| Tensor Core |    | 忙  |    |    | 忙  |    |
| 写回路径        |    |    | 忙  |    |    | 忙  |

一队线程串着驱动三个引擎，任何时刻总有引擎在干等别人——三个永远凑不到同一拍上一起忙。

- Tensor Core 在算的时候,TMA 单元闲着;
- 结果往内存里排空的时候,轮到 Tensor Core 闲着;
- 每个引擎想动一下,都得隔着**同一组线程**去等别的引擎腾位置。

那怎么破? 思路就一句话: **别再让一队线程包圆了所有活**。让不同的 warp 各管一摊,引擎之间才能真正同时转起来。

三步走: 把「协作范围」一步步做大

这一章用三步,一点一点把「谁跟谁协作」的范围撑大。每一步都拆掉一个串行瓶颈,一路逼近业界最高的吞吐。

| 步骤     | 名称             | 做了什么                                 | 协作范围            | 输出 tile |
|--------|----------------|--------------------------------------|-----------------|---------|
| Step 7 | Warp 专门化 + 流水线 | 把 warp 划分成生产者 / 消费者 / 写回三种角色         | 单个 CTA 内部       | 128×128 |
| Step 8 | 2-CTA Cluster  | 把两个 CTA 组成一个 cluster,跨彼此的 SMEM 共享操作数 | 跨 2 个 CTA       | 256×256 |
| Step 9 | 多消费者 Warp 专门化  | 再加一个 MMA 消费者, 让一份暂存 tile 喂两倍的算力      | 跨 2 个 CTA, 双消费者 | 512×256 |

## 注意

**注意:** 这里冒出两个新词, 顺手讲明白:

- **CTA (Cooperative Thread Array, 协作线程数组):** 就是一个线程块 (thread block)——一批线程的集合 (本章里通常含 2~3 个 warpgroup), 它们跑在同一个物理核心上、能共用同一块 SMEM。你可以把一个 CTA 理解成「被分配到一起、能共享高速内存的一支小分队」。
- **cluster (集群):** 更上一层的结构, 把好几个 CTA 绑成一组。它的特别本事是: 让组里的 CTA 能互相读对方的 SMEM——本来每个 CTA 的 SMEM 是私有的, 谁也碰不到别人的; cluster 打通了这堵墙。为什么需要它? 后面 Step 8 你就会看到, 两个 CTA 一旦能共享 SMEM, 就能合伙算一块更大的 tile、把同一份数据复用给更多计算。

## 把三步看成同一个套路的不同尺度

要看懂这三步, 最好的角度是: 它们其实是同一个套路, 被一次次放大而已。

## 结构图

分组:

- **Step 7: 单个 CTA (输出  $128 \times 128$ ):** 「warpgroup A: TMA 搬 + MMA 算」 「warpgroup B: writeback 写回」
- **Step 8: 2-CTA Cluster (一个  $256 \times 256$  tile 横跨两者, 沿 cbx 轴展开):** 「CTA 0: SMEM 半块」 「CTA 1: SMEM 半块」
- **Step 9: 多消费者 (cluster 输出  $512 \times 256$ , 最稠密变体):** 「CTA 0: 消费者 1 + 消费者 2」 「CTA 1: 消费者 1 + 消费者 2」

连接关系:

- CTA 0: SMEM 半块  $\xrightarrow{\text{操作数 tile 横跨两个 CTA 的 SMEM}}$  CTA 1: SMEM 半块
- CTA 0: 消费者 1 + 消费者 2  $\xrightarrow{\text{一份暂存的 B tile 被两个消费者复用}}$  CTA 1: 消费者 1 + 消费者 2
- Step 7: 单个 CTA (输出  $128 \times 128$ )  $\xrightarrow{\text{协作范围跨出单个 CTA}}$  Step 8: 2-CTA Cluster (一个  $256 \times 256$  tile 横跨两者, 沿 cbx 轴展开)
- Step 8: 2-CTA Cluster (一个  $256 \times 256$  tile 横跨两者, 沿 cbx 轴展开)  $\xrightarrow{\text{计算密度再翻倍}}$  Step 9: 多消费者 (cluster 输出  $512 \times 256$ , 最稠密变体)

- **Step 7:** 整条流水线还窝在一个 CTA 里。TMA 和 MMA 合用一个 warpgroup, 写回甩给另一个 warpgroup。这是角色分工里最小、最朴素的样子。
- **Step 8:** 协作范围头一回跨出单个 CTA。两个 CTA 搭伙产出一个  $256 \times 256$  的 tile, 这块 tile 横跨它俩的 SMEM。
- **Step 9:** 再往死里榨一把算力密度。cluster 的输出长到  $512 \times 256$ , 每一块暂存的 B tile 都被两个消费者轮着用, 于是我们就拿到了全教程里计算最稠密的那个版本。

贯穿全章的不变量: 变的是谁协作, 不是数据怎么摆

整章有一样东西**自始至终没动**:SMEM、TMEM 和寄存器的布局, 还是老老实实守着前两章定下的那些契约。说白了, 这一章改的是**谁跟谁协作**, 而不是**数据怎么摆**。

注意

注意:Step 8 是协作范围头一次伸到单个 CTA 之外。也正因为这样, 它的**操作数 (operand)**——「操作数」就是喂给矩阵乘的两个输入矩阵 A 和 B——tile 被**劈成两半**, 分别放进两个**CTA 的共享内存**, 布局沿着 cluster 的 **cbx 轴** (**cbx** 就是 cluster 在 x 方向上给每个 CTA 的编号, 取 0 或 1) **横跨两个 CTA**。读 Step 8 的代码时, 脑子里一定得绷着这根弦: 同一个逻辑上完整的 tile, 物理上其实是拆散了、分头存在两个 CTA 各自的 SMEM 里的。

第 7 步:Warp 专门化 + 流水线 (Step 7: Warp Specialization + Pipeline)

一句话先理解

一句话先理解: 与其让一支队伍轮流把所有活都干完 (干这件时另一件就停着), 不如**一件活配一支专职队伍, 让它们各管一摊、同时开工**。这就是这一步要做的全部。

到目前为止, 我们的 GEMM 内核都是「单 warpgroup」结构: 同一批线程顺着「加载 → 计算 → 写回」走一遍。这写法有处要命的浪费——线程在用 TMA 搬数据的时候,Tensor Core 只能干瞪眼; 线程在跑矩阵乘 (MMA) 的时候,TMA 引擎又闲着。两个金贵的硬件引擎, 愣是凑不到一块儿同时忙。

这一步的核心就一个词:**warp 专门化 (warp specialization)**。它要治的病前面说了——与其让一支队伍轮着把所有活都干了, 不如**一件活配一支专门的 warp**(让某些 warp 只负责搬数据、另一些只负责算、再一些只负责写回), 让这些 warp **同时跑起来**, 再拿一套软件流水线把它们的节奏串起来。这是整条 GEMM 优化路上动静最大的一次架构改造, 后面第 8、9 步全是踩着它的肩膀往上走的。本节的测试矩阵规模统一用 **M=N=K=4096**(也就是三个 4096×4096 的矩阵相乘)。

本步改了什么、没改什么

| 维度              | 第 6 步 (及之前)                             | 第 7 步 (本步)                                                         |
|-----------------|-----------------------------------------|--------------------------------------------------------------------|
| 作用范围 (Scope)    | 一个 warpgroup 顺序走 load → MMA → writeback | 拆成 <b>3 个并发角色</b> :TMA 生产者 / MMA 消费者 / 写回, 用 full/empty barrier 衔接 |
| 布局 (Layout)     | 多级 SMEM stage + TMEM 累加器                | <b>不变</b> (沿用第 6 步)                                                |
| 指令派发 (Dispatch) | TMA 加载, tcgen05 MMA                     | <b>不变</b>                                                          |

说白了, 多级 SMEM 流水线和持久化调度器 ClusterPersistentScheduler2D 都是从第 5-6 步原样搬过来的, **真正新增的只有一件事: 把活按角色拆开**。这一节会一项一项讲清楚:

- warp 专门化: 把不同的活分给不同的 warp / warpgroup
- 高层 barrier 抽象:TMABar、TCGen05Bar、MBarrier
- PipelineState: 自动管 stage(槽位) 和 phase(相位)
- warpgroup\_sync 的 barrier ID: 让同步能精确到「一个 warpgroup」这个粒度

从顺序到并发：为什么要拆

讲角色和 barrier 之前，先把 warp 专门化到底要干掉哪个瓶颈看明白。下面拿时间步表格，把「专门化前」（第 4-6 步那一路）和「专门化后」（第 7 步）的引擎利用率摆一块儿比（行是引擎，列是时间步，格子是那一刻在干啥）。

**专门化前（单 warpgroup 轮着干）**——搬数据时 Tensor Core 闲，算的时候 TMA 闲，两个引擎永远错着峰：

| 引擎          | t0       | t1      | t2       | t3      | t4       |
|-------------|----------|---------|----------|---------|----------|
| TMA 引擎      | load k=0 |         | load k=1 |         | load k=2 |
| Tensor Core |          | MMA k=0 |          | MMA k=1 |          |

**专门化后（生产者/消费者并发）**——TMA 一边预取下一片,MMA 一边在算当前片，两个引擎同时满载：

| 引擎                  | t0       | t1       | t2       | t3        | t4  |
|---------------------|----------|----------|----------|-----------|-----|
| TMA 生产者<br>(warp 3) | load k=0 | load k=1 | load k=2 | load k=3  | ... |
| MMA 消费者<br>(warp 0) |          | MMA k=0  | MMA k=1  | MMA k=2   | ... |
| 写回 (WG 0)           |          |          |          | writeback | ... |

第 5、6 步靠双缓冲和持久化调度把基线往上抬了抬，但它们始终没把「加载」和「计算」拆成生产者、消费者两个独立角色。第 7 步的专门化打破了「轮流」这个魔咒:TMA 生产者 (warp 3) 趁着 MMA 消费者 (warp 0) 还在算当前片，就抢先把下一片的加载发出去了，于是两个引擎谁也不用干等谁。

这里要引入一个贯穿全章的关键概念:**barrier(屏障)**。

一句话先理解

**一句话先理解:**barrier 就是不同 warp 之间互相喊话的「信号灯」——一方喊「我这边好了」，另一方在那儿等着这声喊，听到才往下走。

为什么需要它？既然现在搬数据和算数据是两支队伍**同时**在跑，那就必须有个机制让它们对上拍子：算的那支不能在数据还没搬到时就抢跑（会读到垃圾），搬的那支也不能在数据还没被用完时就覆盖掉（会毁掉别人正用的数据）。barrier 就是这个「对拍子」的工具——它本质是一个内存里的小计数器，一方在上面「打卡 (arrive)」表示「我干完了」，另一方「等待 (wait)」直到对方打了卡才放行。后面你会反复看到 **arrive** 和 **wait** 这对操作。

那 load 和 MMA 之间怎么交接班？靠两个 barrier：

- **tma2mma**(TMA → MMA): 跟 MMA 说「你等的那片 SMEM 数据搬到位了，开算吧」。
- **mma2tma**(MMA → TMA): 跟 TMA 说「这块缓冲我读完了，你拿去装下一片吧」。

注意

**注意：**这里先解释两个词。**stage(槽位)**：我们不止准备一块 SMEM 缓冲，而是准备好几块轮着用，每一块就叫一个 stage;**PIPE\_DEPTH** 就是「准备几块」。**环形缓冲 (ring buffer)**：这几块缓冲首尾相接转圈用——用完最后一块就绕回第一块，像跑道一样，所以叫「环形」。为什么要好几块？因为这样 TMA 才能趁 MMA 在用第 0 块的时候，提前往第 1 块里装下一片数据，

两支队伍才不用互相干等。

知道这两词，下面这个细节就好懂了。时间线里有处乍一看像 bug——mma2tma 的释放箭头会「跨一个 stage」。别慌，这正是环形缓冲的正常现象。PIPE\_DEPTH=2 的时候只有两块 SMEM 缓冲 (stage 0、stage 1):TMA 把 k=0 装满 buffer 0,k=1 装满 buffer 1。等 MMA 算完 k=0、读完 buffer 0，它就发个 mma2tma，意思是「buffer 0 空了」。可这会儿真正惦记着 buffer 0 的是谁？是 **k=2**——因为 buffer 1 这会儿还被 k=1 占着呢。所以这个释放箭头会一路指到 k=2。说白了，跨一个 stage 纯粹是因为环上就两个槽，转一圈正好回到自己。

谁来干每件活:warp 角色分工

时间线讲清了为什么要拆；接下来说谁来干。当 WG\_NUMBER=2(内核用两个 warpgroup，下表简写 WG) 时，三件活这么分：

| 角色             | 位置                | 职责                    |
|----------------|-------------------|-----------------------|
| TMA 生产者        | WG 1,warp 3       | 持续用 TMA 加载 A、B 的 tile |
| MMA 消费者        | WG 1,warp 0       | 数据一就绪立刻跑 MMA          |
| 写回 (Writeback) | WG 0(全部 4 个 warp) | 读 TMEM 结果，写回 GMEM     |

4 个 barrier: 前向报「就绪」，后向报「释放」

三个并发角色，要配 4 个 barrier。这 4 个怎么记？它们正好分成方向相反的两组：

- 前向 (TMA → MMA → 写回) 传的是「数据就绪」：意思是「你等的那片到了」。
- 后向 (写回 → MMA → TMA) 传的是「缓冲释放」：意思是「你惦记的那个槽又腾空了」。

命名规则是 2 (source2destination)，记住这条，名字自己就能读懂——tma2mma 就是「TMA 给 MMA 发信号」那个 barrier。

| Barrier | 类型         | 方向        | 含义              |
|---------|------------|-----------|-----------------|
| tma2mma | TMABar     | TMA → MMA | 「SMEM 数据已就绪」    |
| mma2tma | TCGen05Bar | MMA → TMA | 「SMEM 缓冲可复用」    |
| mma2ld  | TCGen05Bar | MMA → 写回  | 「TMEM 结果已就绪」    |
| ld2mma  | MBarrier   | 写回 → MMA  | 「TMEM 已空，可写下一片」 |

为什么每个 barrier 的类型不一样?(TMABar、TCGen05Bar、MBarrier 是代码里给这三类 barrier 起的封装名字，你不用记，知道它们都是 barrier 的不同变体即可。) 类型取决于「生产者拿什么方式告诉别人自己干完了」：

- TMA 加载** 用 TMABar，它本质是一个带字节计数 (byte counting) 的 barrier：意思是它不数「来了几个人打卡」，而是数「搬到了多少字节」。TMA 硬件每搬完一笔数据，就把对应字节数累加上去；字节攒够了，barrier 就自动放行。妙处在于这一切是 **TMA 硬件自己干的**——消费者连一行「数据到了没」的轮询代码都不用写，坐等放行就行。
- TMA 存储 (store，把数据往外写回内存)** 使不了这招——往外写根本没有「该通知谁」这个对象。所以它只能退回到老办法:cp\_async.bulk.commit\_group() + wait\_group(0)，这两条指令合起来的意思就是「谁发起的写出，谁自己原地等到自己那笔写出彻底排空 (写完)」。
- MMA 用 TCGen05Bar**,MMA 一算完，由 tcgen05.commit() 这条指令替它在 barrier 上打卡。

## 注意

**注意：**这里埋了个第 8 步的伏笔——现在所有 `arrive` 都传 `cta_mask=0`，因为单 CTA 内核里没有别的 CTA 要通知。等第 8 步拼成 `cluster`，正是这个参数会变成非零，摇身一变成了「叫醒协作 CTA」的开关。

## PipelineState: 自动管理 stage 与 phase

4 个 barrier 负责告诉各角色什么时候某块缓冲就绪了。但还差一样：得有人记着眼下轮到哪块缓冲。这份记账活就交给 `PipelineState`（直译「流水线状态」，就是一个帮你记账的小对象）。环形缓冲要同时盯两件事：

1. **stage(槽位)：**现在在用第几块缓冲；
2. **phase(相位)：**现在在等这个槽位 barrier 的哪一「轮」（用 0 / 1 两个值循环标记）。

## 一句话先理解

**一句话先理解 phase：**同一块缓冲会被反复使用——这一轮装 `k=0`、下一轮装 `k=2`……barrier 怎么区分「我等的是这一轮还是上一轮的信号」？靠 `phase`。它就是个在 0 和 1 之间来回翻的开关：每用完一轮就翻一次。你可以把它想成厕所门上的「有人/无人」牌子，翻一下就换个状态，等的人看牌子决定能不能进。下面那个「初始 `phase` 必须设反」的坑，根子就在这个开关的初始朝向上。

这两个量你要是手动一块儿维护，那正是流水循环里最容易出 `off-by-one`（差一）的地方——偏偏这里只要差一个，整个内核就**死锁**。`PipelineState` 的活，就是把这两量绑一块儿自动往前推，省得你操这份心：

```
tma_ps = PipelineState(PIPE_DEPTH, phase=1) # : phase=1,
tma_ps.stage = stage
tma_ps.phase = phase(0 1)
tma_ps.advance() # stage(phase)
```

**初始 `phase` 设成几，**决定了某个角色「第一次 `wait`」时是直接放行还是当场卡住。而流水线两头的正确答案**正好相反**——这就是最容易翻车的地方：

- **`phase=1`(生产者)：**第一次 `wait(phase=1), barrier` 这会儿还停在 `phase 0, 0 != 1`，所以**立即放行**。这正合我们的意——缓冲一开始就是空的，生产者本来就该马上去填。
- **`phase=0`(消费者)：**第一次 `wait(phase=0), barrier` 也是 `phase 0, 0 == 0`，所以**卡住**。这也正合意——还没数据呢，消费者没东西可读，就该老老实实等生产者先 `arrive`。

## 注意

**注意：**两端要是设了同样的起始 `phase`，等着你的就是死锁，或者更糟——**静默的数据损坏**（算出错的结果你还看不出来）。这一个选择，必须做对。



## warpgroup\_sync 的 barrier ID: 按 warpgroup 同步

先说个背景:cta\_sync() 是一条「让整个 CTA 的所有线程在此集合、到齐了一起走」的同步指令——在 warp 专门化之前,所有线程走的是同一条代码路径,用它再合适不过。

可专门化一来就带了个特别容易踩的坑:一旦各个 warpgroup 走的代码路径不一样了(这个 warpgroup 在搬数据、那个在算、那个在写回),cta\_sync() 就会**死锁**(所有人卡死、谁也走不动)。为啥?因为 cta\_sync() 死死要求 **CTA 里所有线程**都到齐才放行;可现在某个 warpgroup 的分支里只有一部分线程会走到这一句,剩下的线程在别的分支里忙别的,人永远凑不齐,于是已经到的那批就永远干等。

我们要的是一个「只管一个 warpgroup、不牵扯别人」的同步。好在 GPU 硬件提供了 16 个可以分别使用的「命名 barrier」(编号 0-15),就像 16 盏可以各自独立点亮的信号灯。所以内核改用 warpgroup\_sync(10)——它借用 10 号这盏灯,只同步一个 warpgroup 内部的线程,不去惊动别的队伍。要是有多组 warpgroup 都得各自独立同步(比如第 9 步的多消费者),它们就用 warpgroup\_sync(wg\_id + 10) 各占一盏不同编号的灯(10、11、12……),省得几支队伍撞到同一盏灯上去互相干扰。

## 完整内核实现

这一步用 PIPE\_DEPTH=2——这是「还能让 load 和 compute 叠起来」的最小深度(更深能多藏点内存延迟,代价是更吃 SMEM;这个取舍后面「当第 7 步出问题」一节细聊)。把前面这些零件——角色、4 个 barrier、PipelineState、warpgroup 级同步——拼到一块儿,核心结构长这样。

### 提示

**提示:** 下面的代码读不懂每一行也没关系。这些函数名(T.ptx.xxx、Tx.xxx、tcgen05.xxx)都是 GPU 专用的底层操作,你只需顺着每行后面的中文注释,抓住「这一段在干哪件事、谁在等谁」的大意。我已经把又长又硬的实现摘掉,只留**关键骨架 + 中文讲解**。

### 1) 分配与初始化——4 个 barrier、SMEM 缓冲、TMEM 累加器都在这儿建好:

```
BLK_M, BLK_N, BLK_K = 128, 128, 64
PIPE_DEPTH = 2
WG_NUMBER = 2

4 barrier:
tma2mma = TMABar(pool, PIPE_DEPTH) # TMA->MMA, mbarrier
mma2tma = TCGen05Bar(pool, PIPE_DEPTH) # MMA->TMA,tcgen05.commit
mma2ld = TCGen05Bar(pool, 1) # MMA->
ld2mma = MBarrier(pool, 1) # ->MMA, mbarrier
...
ld2mma.init(128) # WG 0 128 arrive, 128
```

**2)TMEM 分配 + fence + cta\_sync**——这一段所有线程还走在同一条路上,所以可以放心用 cta\_sync();角色的分叉是在它之后才发生的:

```
if wg_id == 0 and warp_id == 0:
 T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_cols=512, cta_group=1)
T.ptx.fence.proxy_async("shared::cta")
```

```
T.ptx.fence.mbarrier_init()
T.cuda.cta_sync() # CTA
```

**3)WG 1 = TMA 生产者 (warp 3)+ MMA 消费者 (warp 0)**——这是整条流水线的心脏。盯一眼: 生产者和消费者的 PipelineState 起始 phase 是反着的:

```
if wg_id == 1:
 if warp_id == 3: # === TMA ===
 tma_ps = PipelineState(PIPE_DEPTH, phase=1) # phase=1: wait
 if T.filter(lane_id, T.ptx.elect_sync()): # TMA
 while tile_scheduler.valid():
 for k in range(K_TILES):
 mma2tma.wait(tma_ps.stage, tma_ps.phase) # MMA
 tma_load(k * BLK_K) # A B TMA copy_async
 tma2mma.arrive(tma_ps.stage, # arrive(
 (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE)
 tma_ps.advance()
 tile_scheduler.next_tile()

 elif warp_id == 0: # === MMA ===
 mma_ps = PipelineState(PIPE_DEPTH, phase=0) # phase=0: wait
 ld_ps = PipelineState(1, phase=1)
 if T.filter(lane_id, T.ptx.elect_sync()):
 while tile_scheduler.valid():
 ld2mma.wait(ld_ps.stage, ld_ps.phase) # TMEM
 ld_ps.advance()
 for k in range(K_TILES):
 tma2mma.wait(mma_ps.stage, mma_ps.phase) #
 Tx.gemm_async(tmем[:, :BLK_N],
 Asmem[mma_ps.stage], Bsmem[mma_ps.stage],
 accum=(k != 0), dispatch="tcgen05", cta_group=1)
 mma2tma.arrive(mma_ps.stage, cta_group=1, cta_mask=0) # TMA
 mma_ps.advance()
 mma2ld.arrive(0, cta_group=1, cta_mask=0) #
 tile_scheduler.next_tile()
```

**4)WG 0 = 写回**——读 TMEM、转 fp16、再用 TMA 存回 GMEM(细节下一节展开):

```
elif wg_id == 0:
 wb_ps = PipelineState(1, phase=0)
 while tile_scheduler.valid():
 mma2ld.wait(wb_ps.stage, wb_ps.phase) # MMA
 wb_ps.advance()
 Tx.wg.copy_async(reg_wg[:, :], tmем[:, :BLK_N]) # TMEM -> (warpgroup)
 T.ptx.tcgen05.wait.ld()
 ld2mma.arrive(0, cta_id=0, pred=True) # 128 arrive:TMEM
 Tx.cast(reg_f16[:, :], reg[:, :]) # fp32 -> fp16
 Tx.copy(Dsmem[warp_id*32 + lane_id, :], reg_f16[:, :])
 T.ptx.fence.proxy_async("shared::cta")
 T.cuda.warpgroup_sync(10) # warpgroup (cta_sync!)
 if warp_id == 0 and lane_id == 0:
 Tx.copy_async(D[...], Dsmem[:, :], dispatch="tma") # TMA store
 T.ptx.cp_async.bulk.commit_group()
 T.ptx.cp_async.bulk.wait_group(0) # store mbarrier ,
```

```
T.cuda.wargroup_sync(10)
tile_scheduler.next_tile()
```

### 注意

**注意:** 想跑某一步的内核, 直接复用第 1 步那套 `compile / run / check` 脚手架, 把 `hgemm_v1` 换成 `hgemm_v7/v8/v9` 就行。clustered 的那几步要求 `M`、`N` 是它 cluster tile 的整数倍 (第 8 步是  $256 \times 256$ , 第 9 步是  $512 \times 256$ ), 所以  $128 \times 128$  这种小尺寸一个 tile 都凑不出来。还有一点: 每个全新的 Python session 只编译一个步骤, 切换之前得重启内核——这些内核复用了内部命名, 编译器又攥着 per-session 的状态。

## 收尾阶段 (写回) 细节

第 7 步的 **epilogue(收尾)**——就是矩阵乘算完之后, 把累加结果转成最终格式、再写回内存的那最后一小段——能写得相当省事。这里要补一个新词: **寄存器 (register)**。寄存器是 GPU 上最快的一层存储, 但有个特点是**每个线程各有自己的一份、而且数量很有限** (类比 CPU 寄存器, 只是 GPU 上每条线程都独立拥有一套)。结果要先从 TMEM 读进寄存器、在寄存器里转格式, 才能写出去。第 7 步省事在哪? 因为输出只有 `BLK_N=128` 列, 写回 `wargroup` 一趟就能把整块 TMEM tile 读进寄存器 (寄存器装得下), 然后发一条 TMA store 就收工。第 8、9 步就没这福气了 (这也正是它俩后面非得搞「分块写回」的原因)。第 7 步的写回顺序是这样:

1. 等 `MMA::mma2ld.wait(phase)`。(第 8、9 步会在这儿加一个 `fence.after_thread_sync()` 当保守的双保险; 不过 MMA 完成的那个 `mbarrier` 本身已经把顺序兜住了, 大多数内核——包括 CUTLASS——都把它省了, 所以第 7 步也省。)
2. 读 TMEM → 寄存器: 每线程 128 个 fp32, `wargroup` 范围, 用 `Tx.copy_async + T.ptx.tcgen05.wait`。
3. 通知 `MMA::ld2mma.arrive(0, cta_id=0, pred=True)` (全 128 线程都 arrive), 意思是 TMEM 空出来了, 可以装下一片。这里两个 `kwarg` 在 clustered 步骤里会反复出现: `cta_id` 指明「给哪个 CTA 那份 barrier 副本发信号」(0 就是本 CTA 的本地 barrier; 第 8 步的协作 arrive 改用 `cta_mask` 去指 CTA-0); `pred` 是逐线程的谓词, 管这个线程到底算不算 arrive (这里是 `True`, 所以每个写回线程都计进 arrival 总数)。
4. `fp32` → `fp16` (在寄存器里转好)。
5. 寄存器 → `Dsmem`, 再用 `fence.proxy_async("shared::cta") + wargroup_sync(10)` 把写入刷出去。
6. TMA store: `Dsmem` → `GMEM`, 用 `cp_async.bulk.commit_group() + wait_group(0)`。

### 注意

**注意:** 第 8、9 步为啥不能也「一趟读完」? 卡在**寄存器压力 (register pressure)** 上——「寄存器压力大」就是说同时要占用的寄存器太多、快不够分了。第 8 步 `BLK_N=256`、第 9 步每个消费者 `MMA_N=256`, 意味着每个线程得让 256 个 fp32 数 ( $256 \times 4 = 1024$  字节) 同时占着寄存器。寄存器一旦不够用, 编译器只能把放不下的值临时甩到慢得多的「本地内存」里 (这叫**溢出 / spill**, 一旦发生性能就垮), 还会逼出更大的 `Dsmem` 缓冲。所以那两步把写回切成一小块一小块来, 每块 `EPI_N` 列 (`EPI_N=64`): 每轮只让 `EPI_N` 个 fp32 活在寄存器里, 配一条对应的小 TMA store——本质是拿「多发几条 store 指令」换「寄存器预算一直宽裕、不溢出」。

另外还有两个实现要点值得记一下：

- **持久化内核 (persistent kernel):**`bx = T.cta_id([SM_COUNT])`——一个 SM(流多处理器,GPU 上真正执行线程的核心, 见第 0 章) 一个 CTA, 在一个个 tile 之间循环。
- **L2 友好调度:**`ClusterPersistentScheduler2D` 按缓存局部性把 tile 的顺序重排了一遍。
- 「warp 专门化 + 软件流水线」这一对组合, 是高性能 GEMM 内核的通用套路,CUTLASS 那一派的设计也是这么来的。

当第 7 步出问题：常见坑与流水线深度调优

第 7 步是头一个让 TMA 加载、`tcgen05 MMA`、写回三个**同时在飞的** GEMM 内核。下面这些故障到第 8、9 步会原样再演一遍：

- barrier 计数对不上 (比如 `ld2mma.init` 没设成 128);
- 角色守卫 (role guard, 也就是 `if wg_id == ... / if warp_id == ...`) 放错了位置;
- 漏了 fence;
- staging 缓冲在 TMA store 还没排空的时候就被人拿去复用了。

**流水线深度调优。**第 7 步用的是最小值 `PIPE_DEPTH=2`。把它调到 4 或 6, 能让 TMA 生产者跑得离 MMA 消费者更远、多藏点内存延迟, 代价是更吃 SMEM——而 SMEM 拢共就那么多,B200 每个 SM 只有 228 KB。按 `BLK_M=BLK_N=128, BLK_K=64, fp16` 算笔账：

| 项目                       | 占用                                                         |
|--------------------------|------------------------------------------------------------|
| 每个流水线 stage(A + B)       | $(128 \times 64 + 128 \times 64) \times 2 = 32 \text{ KB}$ |
| Dsmem 写回 staging 缓冲 (固定) | 32 KB                                                      |
| PIPE_DEPTH=2 合计          | 约 $2 \times 32 + 32 = 96 \text{ KB}$                       |
| PIPE_DEPTH=4 合计          | 约 160 KB                                                   |
| PIPE_DEPTH=6 合计          | 约 224 KB(紧贴 228 KB 上限)                                     |

注意

**注意:** 想比 `PIPE_DEPTH=6` 再深, 就只能重新设计写回的 staging 策略, 否则 SMEM 直接爆掉。

到这儿,warp 专门化让一个 **CTA 里头**的线程协作了起来。下一步 (第 8 步) 要把这种协作**捅过 CTA 的边界**——让两个 CTA 一块儿啃一个更大的 tile。

第 8 步：用 2-CTA Cluster 协同算一个大 Tile(Step 8: 2-CTA Cluster)

第 7 步让 TMA、MMA、写回这几个引擎在流水线里叠了起来, 可每个 CTA 还是各干各的: 每个 CTA 独立算自己那块  $128 \times 128$  的输出 tile, 谁也蹭不到邻居已经搬进来的操作数,B 矩阵被一遍又一遍地重复加载。第 8 步就是要打破这种老死不相往来的局面。

思路是: 把两个 CTA 凑成一个 **cluster / 集群**, 让它俩能互相读对方的共享内存 (SMEM)。这么一来, 一条**协同的** `tcgen05 MMA` 指令就能算出横跨两个 CTA 的一整块  $256 \times 256$  输出 tile, 而同一份 B 搬进来, 现在能喂两倍的 MMA 计算。本步矩阵规模还是  $M=N=K=4096$ 。

注意

**注意:** 这一步真正赚到的不是「把活儿重新分一遍」, 而是实打实地把**算术强度 (arithmetic intensity)** 给提上来了。「算术强度」是个很关键的概念: 它衡量你**每从内存搬一字节数据, 能**

拿这字节做多少次计算。这个比值越高越好——因为搬数据慢、算数据快，同一份数据若能喂给更多计算，就等于把慢动作摊薄了。下文会专门讲它在这一步为什么大约翻了一倍。

这一步改了什么：作用域 + 布局 + 派发

这次改动，从三个维度就能概括：

| 维度              | 第 7 步 (单 CTA)     | 第 8 步 (2-CTA Cluster)                                        |
|-----------------|-------------------|--------------------------------------------------------------|
| 协作作用域 (Scope)   | 单个 CTA 内部协作       | 一个 cluster 内的两个 CTA 协作                                       |
| 数据布局 (Layout)   | 操作数全在本 CTA 的 SMEM | 操作数分散在两个 CTA 的 SMEM；由 CTA-0 持有共享的完成 barrier(通过 remote_view)  |
| 指令派发 (Dispatch) | MMA 用 cta_group=1 | MMA 带上 cta_group=2 / cta_mask, 让 tcgen05 作为一条跨 2-CTA 的协同指令运行 |

这一步会用到的关键技术点 (每一个下面都会展开讲)：

- **CTA cluster**: 把多个 CTA 绑成一组，合伙算一个更大的 tile。
- **跨 CTA 读 SMEM**: 用 `map_shared_rank` 这个操作去读邻居 CTA 的共享内存。这种「能跨 CTA 互访的共享内存」有个专门名字叫 **DSMEM(分布式共享内存 / distributed shared memory)**——「分布式」就是说同一份逻辑数据散落在好几个 CTA 的 SMEM 里、但彼此都看得见。
- **cta\_group=2 的协同 MMA**: 让一条矩阵乘指令横跨两个 CTA 一起算，产出  $256\times 256$  的 cluster tile。
- **跨 CTA 发 barrier 信号**: 用 `cta_mask`(一个二进制掩码，标记「要通知哪几个 CTA」) 一次把信号同时发给 cluster 里两个 CTA 的 barrier。

cluster tile 的形状与拆分

整套优化全靠一个硬件能力撑着：**cta\_group=2** 的时候，MMA 不再只能读自己这个 CTA 暂存 (stage) 的操作数 tile，而是被允许把**两个** CTA 的一块儿读了。每个 CTA 加载存储版 B 的一个 128 行切片，转置之后这 128 行就摇身变成 128 个**逻辑输出列**；协同 MMA 再把两个切片「缝」成一个完整的操作数。

提醒一句：本教程把 GEMM 存成  $D = A @ B.T$ ，其中**存储版的 B 形状是  $N\times K$** (注意是  $N\times K$ ，不是  $K\times N$ ，所以才要  $B.T$ )。两个 CTA 凑成 cluster 后，这块  $256\times 256$  的 tile 拆得特别干净：

- **A 沿行方向 (竖着) 切**:CTA-0 拿 A0(第 0~127 行),CTA-1 拿 A1(第 128~255 行)。竖着撮起来就是  $[A0; A1]$ ，一共 256 行。
- **存储版的 B 按行切**:CTA-0 加载 B 第 0~127 行,CTA-1 加载 B 第 128~255 行。因为算式里用的是  $B.T$ ，这两段「存储行切片」一转置，就成了逻辑右操作数的两段「128 列切片」。
- 有了 **cta\_group=2**,MMA 硬件会靠跨 CTA 的共享内存访问，从**两个** CTA 的 SMEM 里同时读 B，这样它眼里就是一整条完整的逻辑输出列跨度。
- 结果就是：两个 CTA 合力算出**一块**  $256\times 256$  的输出 tile，每个 CTA 负责写回其中一条  $128\times 256$  的横向条带 (row stripe)。

下面这张表画的是这俩 CTA 的 A、B 切片怎么拼成 256×256 的 cluster tile(每一行对应一个 CTA 负责的那条输出行条带):

| 持有方   | A 按行切        | 存储版 B(N×K)<br>按行切 | B.T 后的逻辑列   | 输出 tile<br>256×256(列 0-255) |
|-------|--------------|-------------------|-------------|-----------------------------|
| CTA-0 | A0 行 0-127   | B0 行 0-127        | 逻辑列 0-127   | 写出第 0-127 行 × 整 256 列       |
| CTA-1 | A1 行 128-255 | B1 行 128-255      | 逻辑列 128-255 | 写出第 128-255 行 × 整 256 列     |

注意:MMA(cta\_group=2) 会把两个 CTA 的 B 一块儿读, 把两段 128 列的逻辑切片缝成完整的 256 列; 两个 CTA 各写各的那条 128 行条带, 但每条都横跨整整 256 输出列。

再说明一下: 每个 CTA 手里只有自己那段 A 行切片、和一段存储版 B 行切片, 可靠着 DSMEM, 它还能跨 cluster 把对方那段 B 切片也读到。两段切片转置后合起来, 正好把整条输出列范围盖满, 于是两个 CTA 合力憋出一块完整的 256×256 输出 tile。

为什么这是真收益, 而不是把活儿挪了挪

这里值得停下来算笔账。每个 CTA 还是只加载 128×K 的 A 和 128×K 的 B, 所以**整个 cluster 暂存的操作数大概是单 CTA 的 2 倍**; 可它产出的是 256×256 的 tile, 对应的输出 FLOPs 大约是 128×128 tile 的 **4 倍** (面积涨了 4 倍)。

搬运量翻 2 倍, 计算量翻 4 倍——一除就明白了: 每搬一字节操作数,MMA 干的计算量大约翻了倍。凭啥能这样? 因为每个 CTA 的 B 切片, 通过协同 MMA 被「借」到了对方 CTA 的 A 切片上又复用了一遍。换句话说, **算术强度大约翻倍**。

这里要补一个判断瓶颈的概念: 一个程序如果大部分时间耗在「等数据从内存搬过来」上, 就叫**内存受限 (memory-bound)**; 如果大部分时间耗在「真正做计算」上, 就叫**计算受限 (compute-bound)**。我们这个 kernel 现在还偏内存受限——也就是算力其实在干等数据。对这种情况, 「提高算术强度」正是最对症的那根杠杆: 同样的搬运量喂更多计算, 被饿着的算力就利用起来了。后面 End-to-End 性能表里那个约 2.2× 的加速, 根子就在这儿: 同样多的字节, 喂了更多的计算。

tile 地址计算

工作单元现在变成了 cluster,tile 调度器 (scheduler) 也就得按 **cluster tile** 来数了。它吐出来的每个 (m\_idx, n\_idx) 都指着一整块 256×256 的区域, 再由 cluster 里两个 CTA 把这块分着干。那怎么把一个 cluster 坐标翻译成每个 CTA 实际该加载的切片? 代码长这样:

```
m_st = (m_idx * CTA_GROUP + cbx) * BLK_M # CTA
n_st = (n_idx * CTA_GROUP + cbx) * BLK_N # CTA MMA B
```

两个 CTA 啃的是**同一块** 256×256 cluster tile, 而 cbx(CTA 在 cluster 里的位置,0 或 1) 这一个坐标, 同时在两个轴上挑出本 CTA 该管的那份。注意 num\_m\_tiles = M // 256、num\_n\_tiles = N // 256 数的是 cluster tile, 不是单个 CTA 的 tile。

## 注意

**注意:**乍一看 `cbx` 在 `m_st` 和 `n_st` 里都冒出来了,像是「行偏移漏到列里去了」。其实两处都没错,只是意思不一样,得分开看:

- 写回路径上,`cbx` 只管 M 轴: 每个 CTA 攥着一条自己的 128 行条带 (CTA-0 写 `m_idx*256` 往后的 128 行,CTA-1 写紧挨着的下一段 128 行),但两个 CTA 都写整整 256 输出列。这就是为什么写回的列坐标取自 `cluster` 的 `n_idx(n_st_epi = n_idx*256 + no*128`, 里头没有 `cbx`),而不是取自各 CTA 的 `n_st`。
- `n_st` 里之所以带上 `cbx`,是因为每个 CTA 往 MMA 里喂的是不一样的存储版 B 行切片: 这里的 `cbx` 是个加载偏移,不是这个 CTA 的输出列偏移。

## 相比第 7 步的代码改动

跟第 7 步比,改动集中在六处,每一处都对得上前面「cluster 契约」里的某个要点:

```
1. Cluster : CTA cluster
cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1]) # cbx = CTA cluster (0 1)

2. MMA(cta_group=1)
Tx.gemm_async(..., cta_group=2)

3. CTA : (rank=1) Bsmem
B_remote = T.ptx.map_shared_rank(Bsmem, cta_id=1)

4. CTA barrier: CTA-0 barrier cluster
tma2mma_cta0 = T.decl_buffer(
 [CTA_GROUP], "uint64",
 data=T.ptx.map_shared_rank(tma2mma.ptr_to([0]), 0),
 scope="shared")

5. mma2tma / mma2ld arrive: cta_mask=0(CTA, 7)
cta_mask=3(cluster CTA)
mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3)
mma2ld.arrive(0, cta_group=CTA_GROUP, cta_mask=3)

6. cluster_sync cta_sync
T.cuda.cluster_sync()
```

## 作用域变成 cluster 后的几个关键变化

这六处改动,说到底都出自同一个转变:协作范围从单个 CTA 撑大到了整个 cluster。下面就把这个「撑大」落到实处,一条条说清楚——每个 CTA 怎么知道自己在哪、cluster 拿谁的 barrier 来协调、协同 MMA 究竟由哪个 CTA 发。

- **cluster 内的 CTA 编号:**`cbx` 告诉每个 CTA 它在 cluster 里的位置 (0 或 1)。CTA-0 管 A 的第 0~127 行,CTA-1 管第 128~255 行。
- **远程 barrier 视图 (remote view):**cluster 里每个 CTA 都有自己的 SMEM、自己的 barrier。那就冒出一个很自然的问题:CTA-1 要等 CTA-0 产出的东西,它该去碰谁的 barrier? 答案是——挑 CTA-0 的 barrier 当唯一的协调点,让 cluster 里随便哪个 CTA 都能访问它。

`map_shared_rank(tma2mma.ptr_to([0]), 0)` 返回一个指向 CTA-0 barrier、全 cluster 都看得见的指针，在 TIRx 封装层里就是 `tma2mma.remote_view(0)`。打这以后，每一次 arrive 和 wait，指的都是 CTA-0 那一份 barrier。

- **MMA 只由 CTA-0 发**：好多人会把 `cta_group=2` 脑补成「两个引擎同时点火、各跑各的」，其实不是。只有 CTA-0 发一条 `tcgen05.mma`，然后硬件驱动一条协同 MMA 横跨两个 CTA：从两个 SM 的 SMEM 读操作数，把累加结果写进两个 SM 的 TMEM。CTA-1 一条 MMA 都不发。（每个 SM 只有一个 `tcgen05` 引擎，所以 `cta_group=2` 是一条跨 SM 的 MMA，不是两个引擎并排各忙各的。）这就是代码里为啥要拿 `if cbx == 0`：把 MMA 守起来。
- **多播 arrive(multicast arrive)**：`tcgen05.commit(..., cta_group=2, cta_mask=3)` 同样只由 CTA-0 发，但它一发就同时给两个 CTA 的 barrier 都发了信号。`cta_mask=3`（二进制 11）的意思就是 CTA-0 和 CTA-1 都是目标。
- **ld2mma 的初始化计数**：`init(128 * CTA_GROUP)`——因为两个 CTA 的写回 warpgroup（各 128 个线程）都会来 arrive，所以等待计数得  $\times 2$ 。

## 实现要点

下面是 `hgemm_v8` 的关键骨架。完整源码很长，这里精简成几段有代表性的代码，加上中文讲解（长的实现部分用注释带过，不逐行抄）：

```
def hgemm_v8(M, N, K):
 CTA_GROUP = 2
 BLK_M, BLK_N, BLK_K = 128, 128, 64
 MMA_M, MMA_N = 256, 256 # MMA 256x256 cluster tile
 F16_SIZE = 2 # fp16 2

 @T.prim_func
 def kernel(A, B, D):
 bx = T.cta_id([SM_COUNT])
 cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1]) # CTA cluster

 # ---- barrier ----
 ld2mma.init(128 * CTA_GROUP) # CTA arrive, x2

 # ---- TMEM , cta_group ----
 if wg_id == 0 and warp_id == 0:
 T.ptx.tcgen05.alloc(..., n_cols=512, cta_group=CTA_GROUP)

 # ---- tile cluster tile ----
 tile_scheduler = ClusterPersistentScheduler2D(
 "ts", num_m_tiles=M // 256, num_n_tiles=N // 256, # //256
 l2_group_size=8, num_clusters=SM_COUNT // CTA_GROUP)
 tile_scheduler.init(bx // CTA_GROUP)
 m_st = (m_idx * CTA_GROUP + cbx) * BLK_M
 n_st = (n_idx * CTA_GROUP + cbx) * BLK_N

 # ---- CTA barrier : CTA-0 ----
 tma2mma.cta0 = tma2mma.remote_view(0)
```

生产者 (TMA) 和消费者 (MMA) 的核心逻辑，能拎出三条要点：



```
---- TMA (warp 3): CTA A B ----
arrive CTA :
tma2mma.cta0.arrive(
 tma_ps.stage,
 CTA_GROUP * (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE) # xCTA_GROUP

---- MMA (warp 0): CTA-0 MMA ----
if cbx == 0: # : MMA CTA-0
 Tx.gemm_async(
 tmem[:, :MMA_N], Asmem[...], Bsmem[...],
 accum=(k != 0), dispatch="tcgen05", cta_group=CTA_GROUP)
 mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3) # CTA
```

写回阶段得把 256 列掰成两块 128 列分头处理:

```
---- warpgroup:256 2 x128 ----
for no in T.unroll(2): # 256 ,
 Tx.wg.copy_async(reg_wg[:,], tmem[:, no*128:(no+1)*128]) # TMEM
 Tx.cast(reg_f16[:,], reg[:,]) # fp32 -> fp16
 # cluster n_idx, cbx:
 n_st_epi = n_idx * 256 + no * 128
 Tx.copy_async(D[m_st:m_st+BLK_M, n_st_epi:n_st_epi+128],
 Dsmem[:, :], dispatch="tma") # TMA store
 ld2mma.arrive(0, cta_id=0, pred=True) # MMA:TMEM

---- :cluster_sync cta_sync, CTA TMEM ----
T.cuda.cluster_sync()
if warp_id == 0:
 T.ptx.tcgen05.relinquish_alloc_permit(cta_group=CTA_GROUP)
 T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_group=CTA_GROUP)
```

为支持 2-CTA 而做的调整汇总

| 调整项            | 具体改动                                                  | 原因                                              |
|----------------|-------------------------------------------------------|-------------------------------------------------|
| 输出宽度           | CTA_GROUP=2,MMA_N = BLK_N *<br>CTA_GROUP = 256        | cluster tile 列宽翻倍                               |
| 等待计数           | ld2mma.init(128 * CTA_GROUP)                          | 两个 CTA 的写回 warpgroup 都要 arrive                  |
| TMA arrive 字节数 | CTA_GROUP * (BLK_M*BLK_K +<br>BLK_N*BLK_K) * F16_SIZE | 完成信号要把两个 CTA 加载的字节都算进去                          |
| TMEM 申请/释放     | tcgen05.alloc 与 tcgen05.dealloc<br>都用 cta_group=2     | 协同分配/回收跨两个 SM 的 TMEM                            |
| 写回分块           | 256 输出列拆成两块各 128 列                                    | 一次读全部 256 列 TMEM 会超出寄存器容量 (第 9 步进一步缩到 EPI_N=64) |
| 结尾同步           | 用 cluster_sync() 取代 cta_sync()                        | 确保所有 CTA 都干完后, 才回收 TMEM                         |

性能与排错

多出来的这点算术强度, 直接反映在墙钟时间上: 第 8 步在 4096<sup>3</sup> 规模下跑到 **0.104 ms**, 比第 1 步那个 70 ms 的朴素算法 (同规模) 快了约 **676×**(见 End-to-End 表)。这会儿 kernel 已经开始往计

算受限 (compute-bound) 那头靠了, 这正好给第 9 步埋下伏笔——下一步要再添一个 MMA 消费者, 让更多 Tensor Core 计算同时在飞 (in-flight)。

注意

**注意:** 要是你发现第 8 步**反倒比第 7 步还慢**, 那基本可以断定是某条新加的 cluster 契约填岔了一点点。优先查这三处:

1. TMA arrive 的字节数是不是  $\text{CTA\_GROUP} * (\text{BLK\_M} * \text{BLK\_K} + \text{BLK\_N} * \text{BLK\_K}) * \text{F16\_SIZE}$ ;
2. 调度器的维度是不是  $\text{num\_m\_tiles} = \text{M} // 256$ ,  $\text{num\_n\_tiles} = \text{N} // 256$  (对应  $256 \times 256$  的 cluster tile);
3. 写回是不是发了**两次** TMA store (每块 128 列一次), 而且每次都在 Dsmem 被重新占用前排空 (drain) 了。

cluster 把复用拉到了**跨 CTA** 这个层面; 而压轴的第 9 步掉头往里走, 给生产者再配一个 MMA 消费者, 在**每个 CTA 内部**接着榨计算密度。

第 9 步: 多消费者 Warp 专门化 (Step 9: Multi-Consumer Warp Specialization)

到第 8 步, MMA 单元已经喂得挺饱: TMA 在后台流水搬数据, MMA 几乎一刻不闲地算。可还藏着一处浪费——一个 **consumer warp** 消化一块暂存好 (staged) 的 B tile, 速度终归是有上限的; 而这块 B tile 其实就静静躺在 SMEM 里, 谁想读都能读。第 9 步抓的就是这个空子: 再添一个 MMA consumer, 让它拿**另一块 A** 去乘**同一块 B**。这么一来, 每个 CTA 的计算密度直接翻倍, cluster 的输出 tile 也从  $256 \times 256$  长成了  $512 \times 256$ 。规模还是  $\text{M} = \text{N} = \text{K} = 4096$ 。

这一步的核心, 三句话就说完了:

| 维度            | 第 8 步 → 第 9 步的变化                                 |
|---------------|--------------------------------------------------|
| 角色范围 (Scope)  | 一个 MMA consumer 变成两个, 靠 <b>warp_id</b> 区分        |
| 数据布局 (Layout) | 一块暂存的 B tile 被两个 consumer 复用; A 多出一个「consumer 轴」 |
| 调度 (Dispatch) | 不变                                               |

本节盯三件事:

- 多个 MMA warp(consumer) 怎么把吞吐顶上去;
- 多个写回 (writeback)warpgroup 怎么各用各自的独立 barrier slot;
- 这套结构正是本教程里**最优那个 GEMM 变体**用的骨架。

为什么只共享 B, 不共享 A

要看懂这一步, 关键得抓住整个设计赖以立住的那点**不对称 (asymmetry)**——这里「不对称」指的是 A 和 B 的待遇不一样: 两个 consumer(消费者, 这里就是两个负责算 MMA 的 warp) 各用**自己的一块 A**, 但去乘**同一块 staged**(已经暂存进 SMEM 的)B tile。一边各用各的、一边共用一份, 这就是「不对称」。

凭啥共享 B、不共享 A? 因为两个 consumer 管的是**不同的 M 行条带 (row stripe)**——它们的 A block 是货真价实两份不同的数据; 可对这两条 M 条带来说, 要乘的 B 却**一模一样**。于是:

- 一次 B 的 TMA load, 现在喂了 2× 的 MMA 工作量;
- 折算下来, 每个有用 FLOP 摊到的 B 加载成本砍了一半。

结构图

连接关系:

- B tile (同一份, 共享) → A\_block[0] (M 行条带 0)
- B tile (同一份, 共享) → A\_block[1] (M 行条带 1)
- A\_block[0] (M 行条带 0) → consumer0
- A\_block[1] (M 行条带 1) → consumer1
- consumer0  $\xrightarrow{\text{各写各的累加器列区间}}$  TMEM [0:256]
- consumer1  $\xrightarrow{\text{各写各的累加器列区间}}$  TMEM [256:512]

注意

**注意:** 之所以能共享 B, 根子上是因为两个 consumer 在 N 方向落在同一个 tile, 只在 M 方向错开。换句话说, 这是**唯一**说得通的共享法——原文的 Exercise 3 就是让你自己把这事想透 (反过来要是共享 A、各算各的 B, 那两份 B 是不同数据, 压根没法共享)。

多消费者的角色布局

添了第二个 consumer,kernel 里要安排的角色也跟着多了:MMA warp 从一个变俩, 还得再配一个写回 warpgroup, 去把多出来那份累加器排空 (drain)。配置上设 NUM\_CONSUMER=2、WG\_NUMBER=3, 于是整个 kernel 摊到三个 warpgroup(下表简称 WG):

| Warpgroup | Warp   | 角色                                                      |
|-----------|--------|---------------------------------------------------------|
| WG 2      | warp 0 | MMA consumer 0:Asmem[... , 0]<br>× B → TMEM 列 [0:256]   |
| WG 2      | warp 1 | MMA consumer 1:Asmem[... , 1]<br>× B → TMEM 列 [256:512] |
| WG 2      | warp 3 | TMA producer: 每个 stage 加载 2 块 A + 1 块 B                 |
| WG 0      | 全部     | consumer 0 的写回: 读 TMEM [0:256]                          |
| WG 1      | 全部     | consumer 1 的写回: 读 TMEM [256:512]                        |

看得出来,WG 2 内部还是「生产者和消费者住一块儿」的格局 (producer 在 warp 3, 两个 consumer 在 warp 0/1), 而 WG 0、WG 1 各包一个 consumer 的写回。这种**一对一绑定** (consumer  $i \leftrightarrow$  写回 WG  $i \leftrightarrow$  TMEM 列区间  $i$ ), 是后面所有 barrier 设计的地基。

下面这张图把数据流串起来看:

结构图

分组:

- **Warpgroup 2:** 「warp 3 / TMA producer」 「warp 0 / MMA consumer 0」 「warp 1 / MMA consumer 1」

连接关系:

- warp 3 / TMA producer  $\xrightarrow{2 \times A + 1 \times B \text{ 入 SMEM}}$  SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem
- SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem  $\xrightarrow{A0 + B \text{ 共享}}$  warp 0 / MMA consumer 0
- SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem  $\xrightarrow{A1 + B \text{ 共享}}$  warp 1 / MMA consumer 1
- warp 0 / MMA consumer 0  $\xrightarrow{\text{累加}}$  TMEM 列 [0:256]
- warp 1 / MMA consumer 1  $\xrightarrow{\text{累加}}$  TMEM 列 [256:512]
- TMEM 列 [0:256]  $\rightarrow$  WG 0 写回
- TMEM 列 [256:512]  $\rightarrow$  WG 1 写回
- WG 0 写回  $\rightarrow$  全局内存 D
- WG 1 写回  $\rightarrow$  全局内存 D

相比第 8 步具体改了什么

要支持第二个 consumer, kernel 只动了几处地方, 而且每一处改动都能追溯到同一个事实: 现在每个 stage 要喂饱、再排空两块 A、两段 TMEM, 而 B 还是共享一份。改动归得起三类: 多暂存一块 A、给每个 consumer 配一个独立的 barrier slot、为长高到 512×256 的 cluster tile 调整 tile 寻址。

| 改动点                | 第 8 步                                  | 第 9 步                                                                                   |
|--------------------|----------------------------------------|-----------------------------------------------------------------------------------------|
| A 的暂存形状            | (PIPE_DEPTH, BLK_M, BLK_K)             | (PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K), 每 stage 2 块 A                                 |
| TMA arrive 字节数     | $1 \times A + 1 \times B$              | $CTA\_GROUP * (NUM\_CONSUMER * BLK\_M * BLK\_K + BLK\_N * BLK\_K) * F16\_SIZE$ (多算一块 A) |
| MMA 选数据            | 固定                                     | 用 warp_id 选 A block 和 TMEM 列区间                                                          |
| mma2tma 初始化        | init(1)                                | init(NUM_CONSUMER), 每 stage 等两个 consumer 都 arrive                                       |
| mma2ld / ld2mma 深度 | 1                                      | depth=NUM_CONSUMER, 每个 consumer 一个独立 slot                                               |
| M 方向 tile 地址       | $(m\_idx * CTA\_GROUP + cbx) * BLK\_M$ | $(m\_idx * NUM\_CONSUMER * CTA\_GROUP + cbx) * BLK\_M$ (多一个 NUM_CONSUMER 因子)            |
| tile 调度器           | $num\_m\_tiles = M // 256$             | $num\_m\_tiles = M // 256 // NUM\_CONSUMER$ (cluster tile 变 512×256)                    |
| 写回                 | 一次性读全部列                                | 按 EPI_N 分块读, 减少同时存活在寄存器里的回读值                                                            |

注意

**注意:**mma2tma.init(NUM\_CONSUMER) 这处改动很要命——B tile 是两个 consumer 共用的, 所以 TMA 必须等两个 consumer 都用完 (各 arrive 一次, 合起来两次) 才能放心覆写这个

stage 的 SMEM。少等一个，就撞数据竞争。

关键代码片段精读

下面只挑跟「多 consumer」直接相关的几行核心，完整实现请看原文。

(1) A 多出一根 consumer 轴,B 仍是单份

```
A :[, consumer , M, K] stage A
Asmem = pool.alloc((PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K), a_type, layout=A_layout)
B : , consumer
Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type, layout=B_layout)

mma2ld = TCgen05Bar(pool, NUM_CONSUMER) # depth=2: consumer slot
ld2mma = MBarrier(pool, NUM_CONSUMER) # depth=2: consumer slot
```

(2) TMA producer: 每个 stage 搬两块 A、一块 B

```
@T.inline
def tma_load(k_offset):
 m_st_c1 = T.meta_var(m_st + CTA_GROUP * BLK_M) # A M : consumer 0
 ↪ CTA_GROUP*BLK_M=256
 Tx.copy_async(Asmem[tma_ps.stage, 0, :, :], A[m_st:m_st+BLK_M, ...], ...) # A block 0
 Tx.copy_async(Asmem[tma_ps.stage, 1, :, :], A[m_st_c1:m_st_c1+BLK_M, ...], ...) # A block 1
 Tx.copy_async(Bsmem[tma_ps.stage, :, :], B[n_st:n_st+BLK_N, ...], ...) # B
arrive A
tma2mma_cta0.arrive(tma_ps.stage,
 CTA_GROUP * (NUM_CONSUMER * BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE)
```

(3) MMA consumer: 用 warp\_id 路由 A block 与 TMEM 列

```
elif warp_id < NUM_CONSUMER: # warp 0/1 consumer
...
Tx.gemm_async(
 tmem[:, warp_id * MMA_N : warp_id * MMA_N + MMA_N], # warp_id TMEM
 Asmem[mma_ps.stage, warp_id, :, :], # warp_id A
 Bsmem[mma_ps.stage, :, :], # B ,
 accum=(k != 0), dispatch="tcgen05", cta_group=CTA_GROUP)
mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3) # stage, TMA
...
mma2ld.arrive(warp_id, cta_group=CTA_GROUP, cta_mask=3) # , warp_id WG
```

(4) 写回: 按 wg\_id 认领自己那段 TMEM, 并分块回读

```
elif wg_id < NUM_CONSUMER: # WG 0/1 consumer
 while tile_scheduler.valid():
 mma2ld.wait(wg_id, wb_ps.phase) # consumer slot
 ...
 for i in T.unroll(MMA_N // EPI_N): # 256 4 EPI_N=64
 col_st = T.meta_var(wg_id * MMA_N + i * EPI_N) # wg_id TMEM
 Tx.wg.copy_async(reg_wg[:,], tmem[:, col_st:col_st+EPI_N])
 ... # cast fp16 -> SMEM -> TMA GMEM
 ld2mma.arrive(wg_id, cta_id=0, pred=True) # MMA consumer:TMEM
```

注意

**注意:** 写回为啥要把 256 列拆成 4 个  $\text{EPI\_N}=64$  的小块一块块来? 因为从 TMEM 回读 (read-back) 出来的值, 得先落到寄存器、再做 cast。一口气把 256 列全读进来, 会让一大堆回读值同时挤在寄存器里, 把预算撑爆; 切成小块以后, 每轮迭代只让少量回读值活在寄存器里, 寄存器压力就好拿捏多了。

两对 barrier 怎么「按消费者」对接

第 9 步有个容易看走眼的细节:mma21d 和 1d2mma 并不是给每个 consumer 各建一个独立对象, 而是各自就一个对象, 只不过内部  $\text{depth}=\text{NUM\_CONSUMER}$ (也就是 2 个 slot)。两条「MMA  $\leftrightarrow$  写回」的通路, 靠 slot 下标来分:

| Barrier | 方向                   | slot 0                          | slot 1                          | MMA 侧按...索引 | 写回侧按...索引 |
|---------|----------------------|---------------------------------|---------------------------------|-------------|-----------|
| mma21d  | MMA $\rightarrow$ 写回 | 接 warp 0 $\leftrightarrow$ WG 0 | 接 warp 1 $\leftrightarrow$ WG 1 | warp_id     | wg_id     |
| 1d2mma  | 写回 $\rightarrow$ MMA | 接 WG 0 $\leftrightarrow$ warp 0 | 接 WG 1 $\leftrightarrow$ warp 1 | warp_id     | wg_id     |

一句话:MMA 侧用 warp\_id 选 slot, 写回侧用 wg\_id 选 slot。又因为角色布局上 consumer i 正好绑死写回 WG i, 两边的下标自然对得上——slot 0 永远连着「consumer 0  $\leftrightarrow$  WG 0」, slot 1 永远连着「consumer 1  $\leftrightarrow$  WG 1」。这套「一个对象 + 多个 slot」的写法, 比「一个 consumer 配一个对象」更省 SMEM; 往后想扩到更多 consumer, 把 depth 调大就完事。

端到端的优化成果 (End-to-End Result)

从 Step 1 一路打磨到 Step 9, 我们终于能把所有招数串成一条完整的链子, 回头看看: 从最朴素的实现走到「跟 cuBLAS 打平」, 中间到底跨过了多大一道坎。这一节不讲新东西, 就是回过头拿一组能横向比的基准数据, 把整章的脉络捋顺——每一步到底快了多少、为什么越往后收益越薄、还有哪几项技术才是真正的大头。

基准条件: 让数字能比的前提

性能数字只有拿「同一把尺子」量出来, 才谈得上比较。本节所有数据都来自同一组受控条件:

- 硬件:NVIDIA B200
- 问题规模: $M = N = K = 4096, \text{fp16}$
- 时钟锁频 (locked clocks), 避免动态频率波动干扰
- 计时方式:1000 次迭代的计时基准 (timed benchmark)

注意

**注意:** 这组数字请当成「特定条件下的一次 B200 参考跑分」来看, 别拿它当排行榜成绩。每一步里嵌的那个 `{.python .input}` 基准单元格算「冒烟基准 (smoke benchmark)」——看趋势、判断方向对不对挺好使, 但不足以拿它去宣称摸到了峰值性能。

完整的加速链路

下表把从朴素基线到 warp 专门化 cluster kernel 的每个里程碑都摆出来, 拿 cuBLAS 当参照:

| 步骤 | 技术                             | 耗时       | 相对加速            |
|----|--------------------------------|----------|-----------------|
| 1  | 同步加载 + MMA(Sync load + MMA)    | 70 ms    | 1×              |
| 2  | K 维循环累加 (K-loop accumulation)  | —        | 处理大于单个 tile 的 K |
| 3  | 空间分块 (Spatial tiling)          | 53.6 ms  | 约 1.3×          |
| 4  | TMA 异步加载 (TMA async load)      | 0.49 ms  | 约 142×          |
| 5  | 软件流水线 (Software pipeline)      | —        | 加载与计算重叠         |
| 6  | 持久化 kernel(Persistent kernel)  | —        | L2 缓存局部性        |
| 7  | warp 专门化 (Warp specialization) | 0.23 ms  | 约 309×          |
| 8  | 双 CTA cluster(2-CTA cluster)   | 0.104 ms | 约 676×          |
| 9  | 多消费者 (Multi-consumer)          | 0.094 ms | 约 744×          |
| —  | cuBLAS(参照)                     | 0.094 ms | 约 744×          |

表里带破折号那几行 (Step 2、5、6), 是为了把结构讲明白才列上去、但没单独计时的步骤。它们的价值在于「让后面的步骤变得可能」, 而不在于自己能拉出一个独立的耗时数。

关键澄清: 那个 70 ms 到底是啥

这一行最容易被读岔, 值得单拎出来说说。

70 ms 不是《构建分块 GEMM》那一节里「单 tile 的 Step 1 kernel」跑在 4096<sup>3</sup> 上的成绩——那个 kernel 一辈子只算一个 128×128 的 tile, 而且只在小规模下跑。

70 ms 其实是一个朴素的全尺寸基线: 它沿用同样「顺序、单 tile」那套思路, 但把问题放大到完整的 4096<sup>3</sup>。换句话说, Step 1~3 在《构建分块 GEMM》里是拿小规模 (128×128、256<sup>3</sup>) 讲的, 图的是一开始讲解尽量简单; 而本表里的 Step 1、Step 3 那两行, 是它们对应的全尺寸基准版。

这点理清很要紧, 因为整条加速链之所以能「端到端横着比」, 靠的正是包括 70 ms 在内的每一个数, 都测自同一个  $M = N = K = 4096$  的规模。

别把 142× 这个数读歪了

从 Step 1 到 Step 4 那约 142×, 是表里最唬人的一跳。但它究竟是谁的功劳, 得看清楚:

结构图

连接关系:

- Step 1(单 128×128 tile, grid 1×1, 软件拷贝) — 这一跳同时叠加了多项变化: K 维循环 (处理完整 K) + 空间分块 (把输出
- Step 4 (完整的分块并行 kernel + TMA)

也就是说, 142× 是「从一个 grid(网格, 一次内核启动里全部 CTA 的总集合, 见第 0 章)1×1 的单 tile kernel, 一路做到带 K 循环、空间分块、众多 CTA 并行、还用上 TMA 的完整 kernel」的

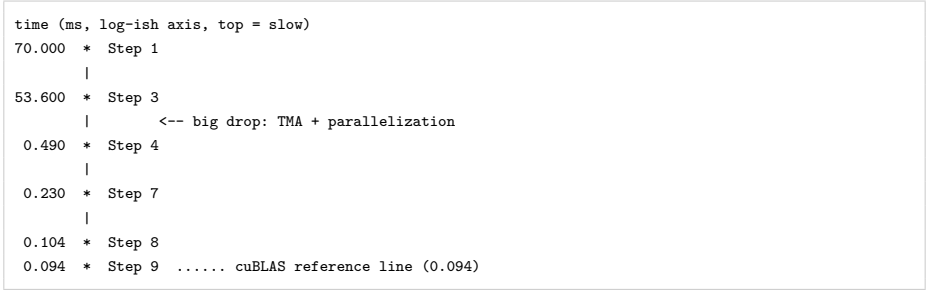
综合效果, 不是 TMA 一个人的功劳。真想把 TMA 的贡献单独拎出来, 得拿两个只在拷贝机制上有区别、其余全一样的全尺寸 kernel 去对比。

四项技术撑起了几乎全部收益

整条链子上, 真正起决定作用的就这四项技术:

- 1. **TMA 异步数据搬运 (TMA Async Data Movement)**: 拿硬件拷贝引擎换掉软件拷贝 (Step 1 → Step 4, 约 142×, 前面说了这是综合效果)。
- 1. **软件流水线 + warp 专门化 (Software Pipelining + Warp Specialization)**: 给加载和计算各分一个专职角色, 让两者叠起来 (Step 4 → Step 7, 约 2.2×)。
- 1. **CTA Cluster**: 靠双 SM 协作的 MMA, 让 B-tile 在多个 CTA 之间复用得更多 (本基准下 Step 7 → Step 8, 约 2.2×)。
- 1. **多消费者 (Multi-Consumer)**: 用两个 MMA warp 把计算密度顶上去 (Step 8 → Step 9, 约 10%)。

把这四项贡献分别标在各自实测的里程碑点上, 就能看到一条曲线: 从「同步分块 kernel」一级一级往下掉, 最后贴近 cuBLAS 的参照线。



图注: 纵轴是耗时 (ms, 可当作对数刻度, 越往上越慢), 自上而下每个点对应一个实测里程碑——Step 1 同步加载 + MMA(70 ms)、Step 3 空间分块 (53.6 ms)、Step 4 TMA 异步加载 (0.49 ms)、Step 7 warp 专门化 (0.23 ms)、Step 8 双 CTA cluster(0.104 ms)、Step 9 多消费者 (0.094 ms);Step 3 到 Step 4 那段「断崖式落差」来自 TMA + 并行化; 最底下那条虚线是 cuBLAS 参照线 (0.094 ms),Step 9 已经跟它打平。

注意

注意: 这条曲线只标了「实测的里程碑点」, 中间设计时的几步 (2、5、6) 没画上去, 但它们是后面每一步的结构性铺垫。

为什么收益越往后越小

看上面那张表会发现: 加速越往下越小。这背后是结构性的原因, 不是后期优化「不够卖力」。

道理是这样:

- 前期那几步打的是「内存瓶颈」: TMA 换掉软件拷贝、cluster 拉高算术强度。而 70 ms 里绝大部分时间偏偏就耗在内存搬运上, 所以打这些靶子回报最高。



- 到 Step 8, kernel 已经贴近 cuBLAS 了 (0.104 vs 0.094 ms, 差约 10%), 也快摸到计算受限 (compute-bound)。这意味着几乎没剩多少内存停顿 (memory stall) 还能拿来遮掩了。
- Step 9 的多消费者重叠, 无非是把那点所剩无几的缝隙再捡一捡。

所以, 最后那约 10% 的收益, 正是「快撞上计算天花板」时该有的样子: 它是一个差不多已经解决的问题的**边际递减 (diminishing return)**, 而不是优化使不上劲的信号。

可以拿一张对比表概括这种「瓶颈搬家」:

| 阶段         | 主要瓶颈             | 优化手段                 | 收益量级        |
|------------|------------------|----------------------|-------------|
| Step 1 → 4 | 内存搬运 (软件拷贝、低并行)  | TMA + 分块并行           | 极大 (约 142×) |
| Step 4 → 7 | 加载与计算无法重叠        | 软件流水线 + warp 专门化     | 中等 (约 2.2×) |
| Step 7 → 8 | B-tile 复用不足      | CTA cluster(双 SM 协作) | 中等 (约 2.2×) |
| Step 8 → 9 | 残余内存停顿 (已接近计算受限) | 多消费者重叠               | 小 (约 10%)   |

这些技术会原样带到下一章

本章搭起来的这整套家当——TMA 加载、tcgen05 MMA、TMEM 回读, 还有 warp 专门化的 barrier——会原封不动地搬到下一章。

Flash Attention(注意力机制, 大模型里算”每个词该关注哪些词”的核心算子) 会把这些全部复用, 然后再加一道难度: 它不再是干巴巴地重复单个 MMA, 而是在两个 MMA 阶段之间**塞进一步在线 softmax(online-softmax;softmax 是把一行分数归一化成概率的操作)**。换句话说, 我们这一章磨出来的「warp 专门化 + 异步流水线」骨架, 正好是下一章应付更复杂数据流的起跑线。

习题与自测 (Exercises)

这一小节是对 Step 7-9 的回扣练习。三道题都不是”背公式”那种, 而是逼你回到设计动机上: **初始 phase 为啥不能瞎设、TMA 字节数为啥要乘 CTA\_GROUP、为啥共享 B 而不是共享 A 才是唯一对的选法**。这三件事想通了, 就等于把整章的流水线契约 (pipeline contract) 在脑子里跑了一遍。下面一题一题给原创讲解和参考答案, 最后再附上”拉着你的 agent 一起做”的追踪练习法。

注意

注意: 做题时会反复用到一个心智模型——每个 mbarrier 都有”阶段 (phase)”和”计数 (count / byte count)”这两层状态。wait 比的是 phase, arrive 推的是 count; count 攒满了 phase 就翻转 (0↔1), 正等着这个 phase 的那一方才被放行。下面所有推理都搭在这个模型上。

第 1 题: 把两端 phase 都设成 0, 会怎么死锁?

题目: 在 Step 7 里, 要是把 TMA 和 MMA 两个 PipelineState 的初始 phase 都设成 0, 会发生什么? 把死锁场景画出来。

先回顾正确设定。原本 Step 7 里两端是反着的:

```
tma_ps = PipelineState(PIPE_DEPTH, phase=1) # : phase=1
mma_ps = PipelineState(PIPE_DEPTH, phase=0) # : phase=0
```

mbarrier 刚 init 完, 物理相位都是 0。wait(phase) 的规矩是: 当前相位 == 期望 phase 就卡住, != 就放行。于是:

| 角色      | 初始 phase | 首次 wait 看到的相位 | 比较结果   | 行为   | 是否正确               |
|---------|----------|---------------|--------|------|--------------------|
| 生产者 TMA | 1        | 0             | 0 != 1 | 立即放行 | 对: 缓冲区是空的, 应立刻开始装填 |
| 消费者 MMA | 0        | 0             | 0 == 0 | 阻塞   | 对: 还没数据, 应该等生产者到达  |

也就是说, 这个一边放行、一边卡住的相位差, 正是流水线能启动的前提: 生产者先开跑, 消费者后头等。

那两端都改成 phase=0 会咋样? 生产者这边也会在第一次 wait 时撞上 0 == 0 而卡住。看生产者循环里这一句就明白:

```
mma2tma.wait(tma_ps.stage, tma_ps.phase) # "
tma_load(k * BLK_K) # TMA load
```

如果 tma\_ps.phase=0, 生产者头一拍就卡在 mma2tma.wait 上, 死等”缓冲区被 MMA 释放”的信号。可缓冲区压根没填过, MMA 自然也没读过、更谈不上释放——于是 mma2tma.arrive 永远不会被叫起来。

把这个死锁闭环画成流程图, 一目了然 (俩人互相等对方先动手, 形成循环等待):

结构图

连接关系:

- TMA 生产者 / wait(mma2tma) 等缓冲区可复用 / (phase=0, 阻塞) / 从不执行 tma\_load 等 mma2tma.arrive (缓冲区释放), 但它从不发生 → MMA 消费者 / wait(tma2mma) 等数据就绪 / (phase=0, 阻塞) / 从不执行 gemm
- MMA 消费者 / wait(tma2mma) 等数据就绪 / (phase=0, 阻塞) / 从不执行 gemm 等 tma2mma.arrive (数据就绪), 但它从不发生 → TMA 生产者 / wait(mma2tma) 等缓冲区可复用 / (phase=0, 阻塞) / 从不执行 tma\_load

俩人互相等对方先动手 → 永久死锁。

- TMA 等 MMA 发 mma2tma(缓冲区释放), 才肯装数据;
- MMA 等 TMA 发 tma2mma(数据就绪), 才肯计算;
- 这俩 arrive 谁都不会先发生, 因为它们各自都卡在 wait 上动不了。

这就是个典型的循环等待 (circular wait), 死锁的四个必要条件全凑齐了。根子上的毛病是: 把生产者的初始 phase 设错, 等于不认”缓冲区一开始是空的、可以直接写”这个事实——你硬逼着生产者也去等一个”释放”信号, 可头一拍根本没东西需要释放。

注意

**注意：**反过来，要是两端都设成 `phase=1`，事情更糟——它可能不是干脆利落地死锁，而是**静默的数据损坏**：消费者第一次 `wait(phase=1)` 看到相位是 `0,0 != 1`，直接放行，于是它跑去读一块还没填过的垃圾缓冲区。死锁至少会卡住，你能察觉；静默错误却会算出错的结果还不露馅。所以原文才反复念叨，这一个选择”值得做对”。

第 2 题:cta\_group=2 时,TMA arrive 字节数为啥要乘 CTA\_GROUP?

题目:Step 8 里 `cta_group=2`,TMA arrive 的字节数是 `CTA_GROUP * (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE`。既然每个 CTA 只装自己那份数据，为啥还要乘 `CTA_GROUP`？

关键先讲清：这个字节数不是”我装了多少”，而是”**barrier 要攒够多少字节才算满**”。`TMABar` 是一个带字节计数 (byte-count) 的 `mbarrier`:TMA 硬件每搬完一笔数据，就把对应的字节数 arrive 到 `barrier` 上；累计字节数一够到设定的门槛,`barrier` 相位翻转，等待方 (MMA) 就被放行。所以这个数字是 `barrier` 的”满载线”，它必须等于**所有把它当就绪信号的那批 load 的字节总和**。

那这个总和为啥要跨两个 CTA 来算？因为 Step 8 用的是一枚集群级的共享 `barrier`——`CTA-0` 的 `barrier` 被选成唯一的协调点，两个 CTA 的 TMA load 全都 arrive 到它身上：

```
tma2mma_cta0 = tma2mma.remote_view(0) # CTA CTA-0 barrier

CTA TMA load tma2mma_cta0 barrier
Tx.copy_async(Asmem[...], A[...], cta_group=CTA_GROUP, mbar=tma2mma_cta0.ptr_to([stage]))
Tx.copy_async(Bsmem[...], B[...], cta_group=CTA_GROUP, mbar=tma2mma_cta0.ptr_to([stage]))
```

而协作式 MMA(`cta_group=2`) 得等两个 CTA 的 A、B 全到齐才能开算——它要跨两个 SM 的 SMEM 读操作数，拼成一块 `256×256` 的 tile。所以这枚 `barrier` 的”满”，就必须定义成:`CTA-0` 的 (A+B) 加上 `CTA-1` 的 (A+B) 全都落了地。

拿一张表，把”每个 CTA 装多少”和”`barrier` 要等多少”分清楚：

| 量            | 数值                                                | 含义                   |
|--------------|---------------------------------------------------|----------------------|
| 单 CTA 装的字节   | $(BLK\_M * BLK\_K + BLK\_N * BLK\_K) * F16\_SIZE$ | 这个 CTA 自己的 A 片 + B 片 |
| barrier 满载门槛 | $CTA\_GROUP * ( \quad )$                          | 集群里两个 CTA 的数据都到齐     |

**忘了乘 CTA\_GROUP 会咋样？** 那 `barrier` 的门槛就只设成了单 CTA 的字节数。于是随便哪个 CTA 的 load 先干完，字节数就提前攒满、相位提前翻转,MMA 被叫早了——这时另一个 CTA 的操作数八成还没搬到,MMA 读到半成品，结果就是竞态加错误输出（还时时对时错，极难复现）。这正是原文末尾”Step 8 反倒比 Step 7 慢 / 错”排查清单里头一条要查的点。

注意

**注意：**别被”每个 CTA 装自己的数据”带偏成”每个 CTA 各等各的”。这里是一枚**共享 barrier 等两份数据**，不是”两枚 barrier 各等一份”。乘 `CTA_GROUP` 编码的就是”协作”这件事本身——把两个 CTA 的就绪条件捏成一个原子的放行信号。Step 9 又添了第二块 A，门槛跟着变成 `CTA_GROUP * (NUM_CONSUMER * BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE`，逻辑一模一样:`barrier 门槛 = 所有挂在它上头的 load 的字节总和`。

第 3 题: 为什么共享 B 而不是 A 才对?

题目:Step 9 里每个 consumer 处理不同的 M 行, 却用同一块 B tile。为什么共享 B(而不是 A) 才是对的选法?

先看数学——共享什么, 是 GEMM 的结构定死的。本教程把 GEMM 写成  $D = A @ B.T$ 。输出 tile 上某个位置  $D[m, n]$ , 是拿 A 的第 m 行去点乘 B 的第 n 行 (转置后就是第 n 列) 得来的。Step 9 让两个 consumer 各算一条不同的 M 行条带, 但它们落在同一段 N 列范围里:

结构图

连接关系:

- B tile ( $N \times K$ ) / 两个 consumer 共用同一份  $\xrightarrow{\text{共享同一份 B}}$  consumer 0 / A block  $0 \times B \rightarrow D$  行  $[0:128]$  (M 行段 0)
- B tile ( $N \times K$ ) / 两个 consumer 共用同一份  $\xrightarrow{\text{共享同一份 B}}$  consumer 1 / A block  $1 \times B \rightarrow D$  行  $[128:256]$  (M 行段 1)

- 两个 consumer 覆盖不同的 M 行, 所以它们的 A block 是两份不同的真数据 (行 0-127 对行 128-255), 没法共享——硬要共享就会用错行、算错结果。
- 两个 consumer 覆盖同一段 N 列, 而 B 只跟 N、K 有关、跟 M 没关系, 所以两个 consumer 看到的 B 完全一样, 天生就能共用同一份 SMEM 拷贝。

为什么这是”唯一可行”的共享方向? 把四种可能全摆出来, 就一种立得住:

| 共享方案                    | 两个 consumer 算的是 | A 是否相同 | B 是否相同 | 能否共享                       |
|-------------------------|-----------------|--------|--------|----------------------------|
| 共享 B, A 各异 (Step 9 的选择) | 不同 M 行、同 N 列    | 否      | 是      | 正确                         |
| 共享 A, B 各异              | 同 M 行、不同 N 列    | 是      | 否      | 这是”沿 N 切”, 得另设计; A 相同才可共享  |
| 都共享                     | 完全相同的 tile      | 是      | 是      | 没意义 (两个 consumer 算同一块, 白干) |
| 都不共享                    | 完全不同的 tile      | 否      | 否      | 退化回两个独立 GEMM, 没有复用收益       |

Step 9 的目标, 就是在已经装进 SMEM 的那份 B 上榨出更多 MMA: B tile 一装好, 本来就静静躺在 SMEM 里, 谁都能读。让第二个 consumer 拿一块不同的 A 去乘同一块 B, 一次 B load 就喂了  $2 \times$  的 MMA 工作, B 的”每个有效 FLOP 摊到的加载成本”直接砍半。这正是把 kernel 从访存受限推向计算受限的那根杠杆。

注意

注意: 你大概会想”那反过来共享 A、让两个 consumer 各算不同 N 列, 行不行”。数学上确实能切 (沿 N 方向切), 可那是另一套布局——而且在本教程的设定里, B 早就因为集群被劈成两半、跨 CTA 共享掉了, N 方向的复用空间在 Step 8 的 cluster 那一步就吃干净了。Step 9 之所以挑在 M 方向叠 consumer、共享 B, 一来跟现成的 cluster 布局不打架, 二来正好白捡了”B 跟 M 无关”这个数学便宜。所以”共享 B”压根不是随手一挑, 而是被  $D = A @ B.T$  的

结构加上前面几步的布局一块儿锁死的、唯一顺理成章的选择。

拉着你的 agent 一起做: 追一个 K-tile 穿过四道 barrier

原文给了个特别好的动手练习, 这里说说怎么做、要盯住哪些点。

做法: 把 Step 7 的 kernel 整段甩给你的 agent, 让它追某一个 K-tile(比如 k=0 这一拍) 挨个穿过四道 barrier——tma2mma、mma2tma、mma2ld、ld2mma。每到一道 barrier, 就逼它回答四个问题:

- 1. 谁在等 (who waits): 哪个角色卡在这道 barrier 的 wait 上?
- 2. 谁来到 (who arrives): 哪个角色、用啥指令在它上面 arrive?
- 3. 哪块 tile 变得能安全读了 (what becomes safe to read):arrive 之后, 等待方现在能读哪块数据 / 哪段 TMEM?
- 4. 哪个缓冲区跟着能复用了 (which buffer is reusable): 这次握手放掉了哪个 SMEM/TMEM slot, 下一拍能重新占用?

自检答案 (拿来核对 agent 答得对不对):

| Barrier            | 谁等                   | 谁到达                     | 之后可安全读                | 随之可复用                                   |
|--------------------|----------------------|-------------------------|-----------------------|-----------------------------------------|
| tma2mma(前向, 数据就绪)  | MMA 消费者              | TMA 硬件按字节计数到达           | 该 stage 的 Asmem/Bsmem | —(刚被填满, 还不能复用)                          |
| mma2tma(后向, 缓冲释放)  | TMA 生产者              | MMA 用 tcgen05.commit 到达 | —(这是释放信号, 不产生新数据)     | 该 stage 的 Asmem/Bsmem, 供 k+2 拍的 load 复用 |
| mma2ld(前向, 结果就绪)   | Writeback warp-group | MMA 用 tcgen05.commit 到达 | TMEM 累加结果             | —(结果刚算完, 等读出)                           |
| ld2mma(后向,TMEM 释放) | MMA 消费者              | 128 个 writeback 线程全部到达  | —(释放信号)               | TMEM, 供下一个输出 tile 复用                    |

注意

注意: 盯住”前向 = 数据就绪、后向 = 缓冲释放”这条主线, 就不会乱。有个细节 agent 特别容易答岔——mma2tma 的跨一级释放: 因为 PIPE\_DEPTH=2 的环形缓冲就两个槽,MMA 算完 k=0(用的是 buffer 0) 后发出的 mma2tma, 放行的不是想用 buffer 1 的 k=1, 而是想再用一次 buffer 0 的 k=2。让 agent 把环形缓冲那两个 slot 老老实实画出来, 它自己就能看出这个”跳一拍”。另一个考点是 cta\_mask:Step 7 里是 arrive(..., cta\_mask=0)(没别的 CTA 要通知), 到 Step 8 就变成 cta\_mask=3(二进制 11, 一下叫醒集群里两个 CTA)——这正是单 CTA 流水线怎么平滑长成集群协作的那个关键参数。

小结

- 一条主线: 一步步把「协作范围」做大。本章从「单个 warpgroup 串着驱动三个引擎」这个瓶颈起步, 用 Step 7→8→9 三步, 把协作的边界从「单 CTA 内的角色分工」撑到「跨 2 个 CTA 的 cluster」, 再撑到「同一 CTA 内多消费者复用同一份 B」。每一步都拆掉一个串行瓶颈。

- **Step 7 拿到的关键:**warp 专门化让 TMA、MMA、写回三个引擎真正并起来跑; 搞懂 4 个 barrier 「前向报就绪、后向报释放」的意思, 以及 PipelineState 里生产者 (phase=1) 和消费者 (phase=0) 起始相位必须反着, 这是整套流水线契约的核心。
- **Step 8 拿到的关键:**cluster 真正赚的是**算术强度翻倍**——同样的字节喂更多计算。协同 MMA 只由 CTA-0 发一条 cta\_group=2 指令、横跨两个 SM; barrier 用 cta\_mask=3 多播、用 remote\_view 指向 CTA-0 的共享 barrier; TMA arrive 字节数要乘 CTA\_GROUP。
- **Step 9 拿到的关键:**共享 B 而不是共享 A, 是  $D = A @ B.T$  的数学结构加上前面 cluster 布局一块儿锁死的、唯一顺理成章的选择;mma2ld / ld2mma 用「一个对象 + 多个 slot(depth=NUM\_CONSUMER)」按消费者对接, 既省 SMEM 又好扩。
- **端到端来看:**四项技术 (TMA、软件流水线 + warp 专门化、CTA cluster、多消费者) 撑起了几乎全部加速; 收益越往后越小, 是「瓶颈从访存受限搬到计算受限」之后正常的边际递减, 不是优化没劲了。这套「warp 专门化 + 异步流水线」骨架会原样带到下一章的 Flash Attention。

延伸阅读

- 本章原文:用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)
- 上一章: 用 TMA 与 tcgen05 MMA 构建流水线化 GEMM(Step 4-6)——本章直接复用了那一章的多级 SMEM 流水线、TMEM 累加器布局和持久化调度器 ClusterPersistentScheduler2D, 建议先把那一章的 SMEM/TMEM 契约读透, 再回头看本章的角色拆分。
- 下一章:Flash Attention——会原封不动地复用本章的 TMA 加载、tcgen05 MMA、TMEM 回读和 warp 专门化 barrier, 再在两个 MMA 阶段之间塞进在线 softmax(online-softmax)。
- 对照参考:CUTLASS 中的 warp-specialized GEMM 设计与 Hopper/Blackwell 的 cluster(CGA)、TMA、tcgen05 编程模型文档。

术语对照

| 英文                                | 中文                  | 简要说明                                                        |
|-----------------------------------|---------------------|-------------------------------------------------------------|
| Warp Specialization               | Warp 专门化            | 把不同任务 (加载/计算/写回) 各自指派给专属 warp 并发执行, 而非让一队线程轮流干所有事           |
| Warpgroup                         | Warpgroup           | 由 4 个 warp、共 128 个线程组成的协作单元                                 |
| CTA (Cooperative Thread Array)    | CTA / 线程块           | 一个 thread block, 本章中常作为持久化内核里每个 SM 上的工作单元                   |
| Cluster                           | Cluster / 集群        | Hopper/Blackwell 引入的结构, 把多个 CTA 绑成一组、允许互相访问对方 SMEM          |
| DSMEM (Distributed Shared Memory) | 分布式共享内存             | cluster 内跨 CTA 可访问的共享内存, 通过 map_shared_rank 读取邻居 CTA 的 SMEM |
| TMA (Tensor Memory Accelerator)   | 张量内存加速器             | 硬件拷贝引擎, 异步搬运 tile 并能按字节计数自动在 barrier 上 arrive               |
| Tensor Core / MMA                 | Tensor Core / 矩阵乘累加 | 执行矩阵乘累加的硬件单元; 本章用 tcgen05 指令派发                              |
| TMEM (Tensor Memory)              | 张量内存                | Tensor Core 专用的累加器内存, MMA 写入、写回阶段回读                         |

|                               |             |                                                              |
|-------------------------------|-------------|--------------------------------------------------------------|
| Producer / Consumer           | 生产者 / 消费者   | TMA 加载数据为生产者,MMA 计算为消费者                                      |
| Barrier (mbarrier)            | 屏障          | 同步原语; 具有 phase(相位) 与 count/byte-count(计数) 两层状态               |
| Phase                         | 相位          | mbarrier 的 0/1 状态; 计数攒满后翻转, 等待该 phase 的一方被放行                 |
| Stage                         | 槽位 / 阶段     | 环形 (多缓冲) 流水线中的缓冲槽, 由 PIPE_DEPTH 决定数量                         |
| PipelineState                 | 流水线状态       | 把 stage 与 phase 绑在一起自动推进, 避免手动维护导致的差一 (off-by-one) 死锁        |
| cta_group=2                   | 协同组大小为 2    | 让 tcgen05 MMA 作为一条跨 2-CTA 的协同指令运行, 读两个 CTA 的 SMEM            |
| cta_mask                      | CTA 掩码      | 指定 arrive 信号要多播给哪些 CTA 的 barrier(如 3 = 二进制 11, 通知两个 CTA)     |
| remote_view / map_shared_rank | 远程视图        | 把某个 CTA 的 barrier/SMEM 指针映射成全 cluster 可见                     |
| Arithmetic Intensity          | 算术强度        | 每搬运一字节操作数所能做的计算量;cluster 把它约翻倍                               |
| Memory-bound / Compute-bound  | 访存受限 / 计算受限 | 性能瓶颈在数据搬运还是在计算; 本章把内核从前者推向后者                                 |
| Persistent Kernel             | 持久化内核       | 每个 SM 一个常驻 CTA, 在多个 tile 之间循环, 改善 L2 局部性                     |
| Epilogue / Writeback          | 收尾 / 写回     | 读 TMEM、转 fp16、用 TMA store 写回 GMEM 的阶段; 高列宽时按 EPI_N 分块以控寄存器压力 |
| Multi-Consumer                | 多消费者        | 多个 MMA 消费者复用同一块 staged B tile, 提升计算密度                        |
| Ring Buffer                   | 环形缓冲        | 多 stage 复用的缓冲结构;mma2tma 释放会「跳一拍」正源于此                         |





## 第 14 章 · Flash Attention 4

原文:Flash Attention 4

### 一句话先理解

**一句话先理解:** 这一章讲的是怎么把 Transformer 里那个叫「注意力 (Attention)」的核心计算, 高效地跑在一块现代显卡 (GPU) 上。如果你完全没碰过 GPU 也别慌——下面这几条要点里出现的所有专有名词 (GEMM、softmax、CUDA Core、TMEM……), 正文都会从零讲明白, 这里先扫一眼混个脸熟就行, 不用现在就懂。

### 本章要点

#### 本章要点 (TL;DR)

- 别把 Attention 当成「再写一遍矩阵乘法 (后面会用它的行话 GEMM, 即 General Matrix Multiply, 通用矩阵乘)」。它是**两段矩阵乘**中间夹着一段 softmax(一种「把一排数字归一化成概率」的运算, 后面细讲): 先算分数  $S = Q \cdot K^T$ , 再算输出  $O += P \cdot V$ , 中间那段 softmax 跑在 GPU 的「通用计算单元 (CUDA Core)」上。难就难在这段 softmax——你嘴上说在优化 Attention, 十有八九是在跟它较劲。
- **在线 softmax(online softmax)** 一句话说白了: 从不把整个  $S$  算出来摆那儿 (因为它太大, 显卡上的快速内存放不下), 而是把  $K/V$  一块一块喂进来, 边喂边加。每一行它都盯着三个「运行态」(running state, 可以理解成「一边流式处理一边随时更新的几个累计变量」): `row_max`、`row_sum` 和输出  $O$ 。麻烦在于, 只要 `row_max` 一变大, 之前加进  $O$  的东西尺度就错了, 得先**重缩放 (rescale, 即整体乘一个系数把比例修正回来)** 再往下加——这正是普通矩阵乘永远不用操心的事。
- 这套实现 (框架叫 TIRx) 把一个 kernel(运行在 GPU 上的一段程序) 拆成 4 个 warp-group(warp 组, 可以先理解成「4 支干不同活的施工队」, 后面会细讲), 按「干什么活」分工: WG3 专门驱动硬件引擎 (搬数据、做矩阵乘、写结果), WG0/WG1 各管一条  $Q$  流水线分支上的 softmax, WG2 做 correction(校正) 和 epilogue(收尾)。记住一条就够: 所有矩阵乘 (MMA) 指令都只从 WG3 里的一支队伍发出来。
- $S$ 、 $P$ 、 $O$  三块数据全挤在**同一块  $128 \times 512$  的 TMEM**(一种显卡上专给矩阵乘用的、特别小的高速内存) 里。靠一个「fp16 视图别名」的小技巧硬给  $P$  腾出地方 (后面有一整节讲这个, 现在不懂没关系)。这么挤还能不出错, 全靠一组 `mbarrier`(线程之间用来对暗号、保证「你做完我才开始」的同步器) 把时序管得死死的。
- **因果掩码** (让每个词只能看自己和前面的词)、**GQA**(一种省显存的注意力变体)、**tile 调度** (把任务分给显卡上的各个计算单元)——这三样都只在「边界」上动手脚, 计算核心几乎一行没碰。这三个词后面各有一节专门讲。
- 最关键的一点: 底层那套 TIRx 写法 (描述一块数据「是什么、谁来算、放哪儿、什么时候能用」的四件套) 和普通矩阵乘 **一模一样**。Flash Attention 4(后面简称 FA4) 不是另起炉

灶，只是在「广度」上铺得更开——多了些数据块，多了些「交接班」。

前置知识

**前置知识:** 这一章是全书的「集大成」，理想情况下你先懂 TMA / TMEM / mbarrier(第 5-8 章) 和矩阵乘的流水线(第 11-13 章)，以及注意力 / softmax 的基本算法，读起来最顺。**但即使你完全没读过前面、对 GPU 一无所知也没关系**——本章每个专有名词第一次出现时，我都会用大白话当场讲清「它是什么、为什么要有它」。真心建议先花十分钟翻一下第 0 章·极简入门，把「显卡为什么和 CPU 不一样、内存为什么分好几层」这两件事有个直观感受，后面会轻松很多。

本章导读: 为什么 Attention 不是「再来一遍 GEMM」(Overview)

在正式开讲前，先花点篇幅把「显卡到底长什么样」用大白话过一遍。你可以把一块 GPU 想象成一座工厂，里面有几个关键角色，这一章会反复用到:

一句话先理解

**一句话先理解:**GPU 之所以快，是因为它有成千上万个小小的**计算核心同时干活**；但代价是，数据从「又大又慢的仓库」搬到「又小又快的工作台」上，反而成了最费劲的事。所以 GPU 编程的一大半功夫，都花在「怎么把数据高效地搬来搬去、让计算核心别闲着」。

| GPU 里的角色        | 大白话解释                                                                  | 程序员熟悉的类比                 |
|-----------------|------------------------------------------------------------------------|--------------------------|
| GMEM(全局内存 / 显存) | 显卡上 <b>最大但最慢</b> 的那块内存，所有数据一开始都堆在这儿                                    | 像硬盘 / 主内存: 容量大, 但读写慢     |
| SMEM(共享内存)      | 芯片 <b>内部</b> 的一小块高速内存, 快得多但很小, 要程序员 <b>手动</b> 往里搬数据                    | 像一块「你得自己管理的高速缓存」         |
| TMA(张量内存加速器)    | 一个 <b>专门负责搬数据</b> 的硬件引擎, 把数据从 GMEM 异步搬到 SMEM(异步 = 发个命令它自己去搬, 你不用干等)    | 像一个后台 DMA / 异步拷贝线程       |
| Tensor Core     | 专门做 <b>矩阵乘法</b> 的硬件单元, 一条指令顶一大片乘加                                      | 像一个「矩阵乘专用协处理器」           |
| MMA(矩阵乘累加)      | Tensor Core 干的那件事: $C += A \cdot B$ , 边乘边往结果上累加                        | 就是 $C = C + A \otimes B$ |
| CUDA Core       | <b>通用</b> 计算核心, 啥都能算 (加减乘除、求指数……), 但不像 Tensor Core 那样专精矩阵乘             | 像 CPU 的普通 ALU            |
| TMEM(张量内存)      | 最新的 Blackwell 架构新加的一小块片上内存, <b>专门给 Tensor Core 存矩阵乘的输入和结果</b> , 容量小得可怜 | 像 Tensor Core 专属的一小块暂存区  |
| mbarrier(内存屏障)  | 线程之间用来 <b>对暗号、做同步</b> 的小工具: 「我做完了你才能开始」                                | 像并发编程里的信号量 / barrier     |

这些名字现在记不住没关系，正文每个都会再讲一遍。前面几章把这些零件挨个讲透了，还教会了怎么用它们拼出一个高性能的**矩阵乘 (GEMM)**——也就是  $C = A \cdot B$  这件最基础的事。

现在到了 Flash Attention, 这些积木终于要**一起**上场了。说白了, Attention(注意力) 就是「决定一个 Transformer 大模型跑不跑得起来」的那个核心计算，也是前面所有功夫第一次必须拧成一股

绳的地方。可它偏偏不是把矩阵乘照抄一遍那么简单——它带着一个普通矩阵乘从来没碰过的结构性难题。

核心矛盾：两个矩阵乘中间夹着一坨「真活儿」

先说说注意力到底在算什么（只讲到够你看懂代码为止）。一句话：给定一堆向量  $Q$  (查询)、 $K$  (键)、 $V$  (值)，注意力要算「每个  $Q$  跟每个  $K$  有多相关」，据此给  $V$  加权求和。这天然就是两次矩阵乘：先  $Q \cdot K^T$  算出「相关性分数」，再拿这些分数（归一化后）去乘  $V$ 。

普通矩阵乘就是「同一件事一遍遍重复」：每一步只往结果（累加器）里加，从头到尾干的是同一个动作。注意力不一样。它是两个矩阵乘，而且这俩中间还夹着一段非得在通用核心（CUDA Core）上做的、实打实的计算（也就是 softmax）：

结构图

连接关系：

- $Q, K, V \xrightarrow{TMA} SMEM$
- $SMEM \rightarrow \textcircled{1} \text{ Score MMA (Tensor Core): } S = Q \cdot K^T$
- $\textcircled{1} \text{ Score MMA (Tensor Core): } S = Q \cdot K^T \rightarrow \textcircled{2} \text{ Softmax (CUDA Core): } S \rightarrow P$ （新难点在这里）
- $\textcircled{2} \text{ Softmax (CUDA Core): } S \rightarrow P$ （新难点在这里） $\rightarrow \textcircled{3} \text{ Value MMA (Tensor Core): } O += P \cdot V$

这里先把 softmax 是什么讲清楚：它就是把一排数字变成「加起来等于 1 的概率」的运算——对每个数求  $\exp$  (自然指数)，再除以这一排  $\exp$  的总和。在注意力里，它把「相关性分数」变成「权重」，决定每个  $V$  占多大比重。

中间这段 softmax 为什么难缠？两个原因：

1. 它跑在通用核心（CUDA Core）上，正好卡在两个矩阵乘中间。求指数（ $\exp$ ）和「把一整行的数字加起来」（按行归约, row-wise reduction）都直接压在关键路径上（关键路径 = 拖慢全局、谁也绕不过去的那条计算链）。Tensor Core 想接着算第二个矩阵乘？对不起，得先等 softmax 把分数  $S$  揉成权重  $P$ 。
2. 它逼着你回头去改已经算好的结果。这正是普通矩阵乘从来不用面对的：矩阵乘的累加器只加不改；可注意力的输出累加器  $O$ ，会因为新的  $K/V$  流进来，被迫整体「换个比例」。下面马上解释这是怎么回事。

也正因为这样，有个听上去反直觉、但很实在的结论：嘴上说优化 Attention 的人，其实绝大多数时间都在优化 softmax——要么换种写法把  $\exp$  算得更省，要么想方设法让 softmax 和矩阵乘叠在一起（overlap，即让两件事在时间上同时进行、用一件事的计算把另一件事的等待藏起来）跑，而不是干等着它算完。

那个普通矩阵乘没有的「转折点」:running max 一变,O 就失真

一句话先理解

一句话先理解:softmax 求  $\exp$  之前,要先减去这一行的最大值 (防止  $\exp$  溢出成无穷大,这是个标准数值技巧)。可问题是——数据一块一块流进来,「这一行到目前为止的最大值」会越来越大;最大值一变,之前用旧最大值算出来的结果就全都「量错了尺子」,得回头重新缩放。这就是整章最核心的那个转折点。

想看懂后面所有代码,先抓住这一条主线就够了——盯着一个 **tile**(tile = 一小块数据,GPU 上习惯把大矩阵切成一块块来处理),跟着它在 **kernel** 里从头走到尾:

| 阶段          | 输入    | 计算单元        | 产物                | 含义                |
|-------------|-------|-------------|-------------------|-------------------|
| 输入加载        | Q、K、V | TMA         | SMEM 中的输入 tile    | 从 GMEM 搬到 SMEM    |
| ① Score MMA | Q、K   | Tensor Core | 分数 tile S(在 TMEM) | $S = Q \cdot K^T$ |
| ② Softmax   | S     | CUDA Core   | 分子 tile P         | 把分数变成 (未归一化的) 权重  |
| ③ Value MMA | P、V   | Tensor Core | 更新输出累加器 O         | $O += P \cdot V$  |

单看这张表,Attention 就像「两个矩阵乘法粘在一起」。可 Flash Attention 的在线 softmax(online softmax,即「不一次性算完,而是一块块流式地算」的 softmax)会带来一个普通矩阵乘永远碰不到的麻烦:

注意

**注意:** 只要那个一边流式处理一边更新的 softmax 最大值 (running max,「跑动中的最大值」) 一变,之前加进 O 的结果当场就处在错误的比例上。你得先拿新最大值把旧 O 重新缩放 (rescale) 一遍,下一次 Value MMA 才能安全地往里加。

说白了,Flash Attention 靠「分块一步步算 + 一路维护 running max / running sum」来躲开「一次性把整个 S 摆出来」这件事;代价就是:每来一块新 K/V,只要发现了更大的最大值,就得把历史累加值整体打个折,让先来的块和后来的块站到同一个尺度上。下面这段伪代码就点出了这条「rescale」主线 (具体的硬件细节先略掉,只看算法骨架;读不懂的 GPU 名词后面都会讲):

|                                    |                               |
|------------------------------------|-------------------------------|
| # softmax :m running max,1         | running sum( ),0              |
| for kv_block in key_value_blocks:  | # K/V                         |
| S = Q @ kv_block.K.T               | # (1) Score MMA(Tensor Core)  |
| m_new = max(m, rowmax(S))          | # running max                 |
| P = exp((S - m_new) / sqrt(d))     | # (2) Softmax(CUDA Core,exp ) |
| alpha = exp((m - m_new) / sqrt(d)) | # ->                          |
| l = l * alpha + rowsum(P)          | #                             |
| O = O * alpha + P @ kv_block.V     | # (3) rescale O, Value MMA    |
| m = m_new                          |                               |
| O = O / l                          | #                             |

看  $O = O * \alpha + \dots$  这一行,这就是前面反复念叨的那个转折点——**alpha** 就是「旧尺度换算到新尺度」的折扣系数。它把「回头改已经算好的结果」这件普通矩阵乘压根不用做的事,变成了每来一块 K/V 都要做一遍的日常动作。后面整章一大半的复杂度,追根溯源都是为了把这一行干净利落地实现出来。

## 本章的目标与组织方式

先把话说清楚：本章不会从零把 **Flash Attention 算法重新推一遍**。算法部分只讲到「够你看懂 kernel」为止；真正下功夫讲的，是那块全新的内容——这套算法怎么落地到 **GPU** 上，也就是怎么把它映射到具体的硬件零件和「各支施工队的分工」上。

所以本章是这么个讲法：

1. 先跟着一个 **tile** 把整条数据通路走完（就是上面那条  $Q, K, V \rightarrow S \rightarrow P \rightarrow O$ ，外加 rescale 这个转折点），把「算法到底要干什么」讲透。
2. 再看怎么把每个阶段分给不同的施工队（**warpgroup**），然后用角色分工、在线 softmax 的重缩放、因果掩码（causal masking）、GQA（分组查询注意力）这些机制，把各个阶段一段段接线串起来。

这条主线一句话就能说完：矩阵乘教会我们怎么把数据搬进来、怎么喂给 Tensor Core 累加；Attention 在这之上又逼我们多学一手——怎么在两个矩阵乘之间塞进一段 softmax，还要在 running max 一变就回头修正已经算好的输出。接下来所有代码，要解决的就是这件事。

## 算法的形状 (Algorithm Shape)

把一块块数据 (tile) 往各级存储里摆之前，得先搞清楚这些 tile 到底为哪个算法服务。换句话说，内存布局 (layout，即「哪块数据放在哪儿、按什么顺序排」) 不是拍脑袋设计的——它就是算法的计算流程往硬件上一投影的结果。所以这一节先把 **Flash Attention** 的核心计算逻辑讲明白，再回头看每个 tile「住哪儿」，因为「住哪儿」直接决定了你后面要写什么样的布局代码和同步代码。

针对一个 query 块 (query block，即一批  $Q$  向量)，Flash Attention 要算的，就是标准的缩放点积注意力 (scaled dot-product attention，即「先点积算分数、再除以  $\sqrt{d}$  缩放、做 softmax、最后乘  $V$ 」的标准注意力公式)：

$$O = \text{softmax}(QK^T / \sqrt{d}) V$$

## 为什么不能照着公式直算

要是真照公式字面去做，流程就是：先把完整的分数矩阵 (score matrix)  $S = QK^T$  算出来，对它做 softmax，再乘  $V$ 。可这偏偏是**唯一不能用的**做法，原因很简单：整个  $S$  实在太大了。

我们算笔账。序列长度  $\text{seq} = 4096$  的时候 (序列长度 = 一次处理多少个 token / 词)，每个注意力头 (head, Transformer 把注意力拆成多个并行的「头」，各看不同的特征) 的  $S$  差不多有 16M 个元素，用 fp32 (32 位单精度浮点) 存大概 **64 MB**。这个量级比片上的共享内存 (SMEM) 大了好几个数量级，也远远超过一块  $128 \times 512$  的 TMEM。一句话——片上那些快速内存根本没地方搁它，只能放在又大又慢的显存里，可那样每次都要来回搬，慢得没法看。

### 注意

**注意：**这里真正卡脖子的不是「算得慢」，而是「放不下」。**Flash Attention** 的核心思路就一条：永远不把完整的  $S$  实打实地摆出来 (materialize)。

流式更新与「重缩放」的由来

既然没法一口气算完整个  $S$ ,Flash Attention 就换个打法: 把  $K/V$  一块一块 (block) 流式喂进来, 边读边算。这就像你不可能把一个超大文件一次读进内存, 只能分批读、边读边累计统计量一样。问题来了——只看到局部数据, 怎么保证最后结果还对? 办法是给每一行维护三个「运行态 (running state, 即一边流式处理一边随时更新的累计变量)」, 用它们来概括「到目前为止看过的所有块」:

| 运行态     | 含义                         |
|---------|----------------------------|
| row_max | 到目前为止见过的最大分数 (每行一个)        |
| row_sum | softmax 分母的累加值 (每行一个)      |
| 0       | 输出累加器 (output accumulator) |

难点在哪? 每来一个新块,row\_max 都可能变大。一旦它变大, 之前用旧 max 算出来的那一堆结果就「量错尺子」了——它们是按旧尺度归一化的。所以在把新块的贡献加进来之前, 得先把旧的运行态「拉」到新尺度上。这就是 Flash Attention 里那个有名的在线重缩放 (online rescaling)。

单个块的更新逻辑可以写成如下伪代码:

```
S = Q_block @ K_block.T # tile()
m_new = max(row_max, rowmax(S)) #
scale = exp((row_max - m_new) / sqrt(d)) # ->
P = exp((S - m_new) / sqrt(d)) # softmax (numerator)
row_sum = row_sum * scale + rowsum(P) # :
O = O * scale + P @ V_block # : O
row_max = m_new #
```

这里有个挺妙的设计: 同一个 scale 因子干了两份活——它既缩放了运行分母 row\_sum, 又缩放了运行输出 O。为什么这么干就对? 因为有了这个统一的因子, 先来的块和后来的块最后都被换算到同一个尺度上, 这样它们才能正确地加到一起。

工程上的两处优化:exp2 与延迟归一化

用 exp2 顶替 exp。这里先解释一下:exp2(x) 就是 2 的 x 次方, 而我们平时说的 exp(x) 是 e 的 x 次方。显卡硬件里算 exp2(2 的幂) 比算 exp(e 的幂) 更省事、更快——这是硬件指令层面的现实。上面的伪代码为了好读, 写的是自然指数 exp 加上明晃晃的 /sqrt(d)。但真正的 kernel 走的是更省的路子: 它把 1/sqrt(d) 和 log2(e) 这两常量提前揉成一个常量 scale\_log2 = log2(e)/sqrt(d), 然后对着原始分数直接用硬件的 exp2 求所有指数, 靠的是这个恒等式 (就是高中学的换底公式):

```
exp(x / sqrt(d)) = exp2(x . scale_log2)
```

动机很朴素: 在这类硬件上,exp2 就是比自然指数 exp 快。

归一化是故意往后拖的。这点得说清楚: 伪代码里的 P 不是最终归一化好的注意力矩阵, 它只是当前这一块  $K/V$  的 softmax 分子。真正的归一化被有意往后推——非要等最后一块处理完了,kernel 才写出  $O / row\_sum$  当最终输出。这么做就是为了省掉「每块都除一遍」的反复折腾。

每个 tile 「住在哪儿」

光知道算法算什么只算搞懂了一半；另一半是**每个 tile 在 kernel 跑起来的时候待在哪种存储里**。后面这一半才是决定布局和同步代码的关键。**S、P、O** 都是 tile 值，各有各的「家」。这里要新引入一个内存层级——**寄存器 (register)**：它是**每个线程私有的、最快的那一小撮存储**，跟 CPU 的寄存器是一个意思，只不过 GPU 上成千上万个线程各有各的一份。CUDA Core 做的那些逐元素小计算（比如求 `exp`），数据都得先进寄存器才能算。

| Tile | 含义              | 驻留位置与数据通路                                                                                           |
|------|-----------------|-----------------------------------------------------------------------------------------------------|
| S    | 分数 tile         | 由 score MMA(算分数的那次矩阵乘) 算出后写入 <b>TMEM</b>                                                            |
| P    | softmax 分子 tile | softmax 把 S 从 TMEM 读进 <b>寄存器</b> ，算出 $P = \exp((S - m_{\text{new}})/\sqrt{d})$ ，再把 P 写回 <b>TMEM</b> |
| O    | 输出累加器 tile      | <b>value MMA</b> (算输出的那次矩阵乘) 从 TMEM 读 P、从 <b>SMEM</b> 读 V，把结果累加进 <b>TMEM</b> 里的 O                   |

把这条数据流画成图，谁读谁写就一目了然了：

结构图

连接关系：

- S 分数 tile / TMEM  $\xrightarrow{\text{读入寄存器}}$  寄存器：算 softmax
- 寄存器：算 softmax  $\xrightarrow{\text{exp2 + 重缩放}}$  P 分子 tile / TMEM
- P 分子 tile / TMEM  $\xrightarrow{\text{value MMA}}$  O 累加器 tile / TMEM
- O 累加器 tile / TMEM  $\xrightarrow{\text{最后一块后}}$  写出 O / row\_sum

注意

**注意：**前面说的「重缩放」也是一次 **tile 操作**，可不是几句标量记账那么轻巧。`row_max` 一变，旧的 O 就得从 TMEM 读出来、在寄存器里乘上 `scale`、再写回 TMEM，然后下一次 value MMA 才能往里累加。

这套结构会一直贯穿后面每一节：**一处 tile 摆放 + 一条硬件通路 + 一个用来宣告「下一个消费者可以开跑了」的 barrier**。把这三件套吃透，整章实现的骨架你也就抓住了。

Tile-原语图 (Tile-Primitive Graph)

上一节我们已经定下了三个运行状态 (**S、P、O**) 各自「住」在哪儿：**S** 在 **TMEM**，**P** 经寄存器在 **TMEM** 来回跑，**O** 累加在 **TMEM**。有了这张「数据住哪」的地图，这一节就把整个算法落成一条**具体的、能跑的 tile 搬运路径**——也就是所谓的「Tile-原语图」（原语 = primitive，指框架提供的一个个最小操作积木，比如「搬一块数据」「做一次矩阵乘」）。

tile-原语图说白了就是一张「生产者-消费者」依赖图（谁产出数据、谁消费数据，跟并发编程里的生产者-消费者是一个意思）。图里每个节点，就是某块数据待在某一级内存里的那个样子（比如「**SMEM** 里的 **Q**」）；每条边，就是一次操作——要么把数据从这儿搬到那儿，要么就地算出点新东西。这张图一画清楚，后面所有关于布局、同步、流水线的活儿，本质上就剩一件事了：给图上每条边补上对的同步（也就是「上一步真做完了，下一步才能动手」）。

一条 K/V block 的短路径

对一个 K/V block 来说, kernel 从上往下走的这条搬运路径, 几行就能概括:

| 步骤 | 数据落点          | 靠什么完成                                                 |
|----|---------------|-------------------------------------------------------|
| 起点 | GMEM 中的 Q、K、V | ——                                                    |
| 1  | SMEM 中的 Q、K、V | TMA load 搬入                                           |
| 2  | TMEM 中的 S     | score MMA: 计算 $QK^T$                                  |
| 3  | TMEM 中的 P     | softmax 分子: TMEM $\rightarrow$ 寄存器 $\rightarrow$ TMEM |
| 4  | TMEM 中的 O     | value MMA: 计算 $P \cdot V$                             |
| 5  | GMEM 中的 O     | 归一化、SMEM 暂存、TMA store 写回                              |

画成有向图更直观, 每条边都标了「靠哪条硬件路径完成」:

结构图

连接关系:

- $Q/K/V @ \text{SMEM} \xrightarrow{\text{score MMA } (QK^T)} S @ \text{TMEM}$
- $S @ \text{TMEM} \xrightarrow{\text{softmax: TMEM} \rightarrow \text{寄存器} \rightarrow \text{TMEM}} P @ \text{TMEM (fp16)}$
- $P @ \text{TMEM (fp16)} \xrightarrow{\text{value MMA } (P \cdot V)} O @ \text{TMEM}$
- $O @ \text{TMEM} \xrightarrow{\text{归一化} + \text{SMEM 暂存} + \text{TMA store}} O @ \text{GMEM}$

注意

**注意:** 这里的 **P** 不是最终归一化好的注意力权重, 只是当前这块 K/V 的 softmax 「分子」。真正的归一化 (除以 `row_sum`) 被特意拖到最后一块处理完才做, 所以图里只有  $O @ \text{TMEM} \rightarrow O @ \text{GMEM}$  这条边上才冒出「归一化」这几个字。

和 GEMM 的唯一本质区别

这张图跟 GEMM 的 tile-原语图比, 差别其实就「一行」:

- **GEMM:** 就是一条 **MMA 链**反复累加——**A·B** 一路 MMA 算下去, 中间没有任何非 MMA 的计算插进来。
- **FA4:** 有两段 MMA(score MMA 和 value MMA), 而 **softmax** 恰好卡在这条链中间。

你可以这么理解: 把 GEMM 的 MMA 链从中间「劈」一刀, 塞进一个 softmax 阶段。后面几乎所有比 GEMM 复杂的地方——多出来的 TMEM $\leftrightarrow$  寄存器流量、多出来的角色分工、多出来的 barrier——全是这「多出来的一段」连锁反应出来的后果。先把这一点攥住, 后面再乱的复杂度也有了主心骨。

展开成完整的生产者—消费者图

把上面那条短路径每一步都摊开, 标清「用哪个原语 + 走哪条硬件路径」, 就得到完整的依赖图。下表把 TIRx 原语 (代码里都以 **Tx**. 打头, 是这个框架封装好的高层积木) 一一对到底层真正执行的硬件指令上。表里会冒出几个底层指令名, 先在这儿打个底:**tcgen05** 是新一代 Tensor Core 那一族



指令的前缀 (`tcgen05.mma` 做矩阵乘, `tcgen05.ld` 从 TMEM 读数据); 看到它就知道「这是在直接驱动矩阵乘硬件」。这些名字你不用记, 知道大概在干嘛即可:

| 阶段             | tile 搬运 / 计算                          | TIRx 原语                                                  | 硬件路径                                          |
|----------------|---------------------------------------|----------------------------------------------------------|-----------------------------------------------|
| 加载 Q/K/V       | GMEM tile -> SMEM tile                | <code>Tx.copy_async(..., dispatch="tma")</code>          | TMA load                                      |
| Score MMA      | SMEM 中的 Q、K -> TMEM 中的 score tile S   | <code>Tx.warp.gemm_async(..., dispatch="tcgen05")</code> | <code>tcgen05.mma</code>                      |
| Softmax 读      | TMEM 中的 S -> warp-group 寄存器 tile      | <code>Tx.wg.copy_async(reg, tmem)</code>                 | <code>tcgen05.ld</code>                       |
| Softmax 写      | 寄存器中的分子 tile P -> fp16 TMEM 视图        | <code>Tx.copy_async(tmem_as_fp16, reg)</code>            | TMEM store, 随后 <code>tcgen05.wait.st()</code> |
| Value MMA      | TMEM 中的 P、SMEM 中的 V -> TMEM 中的输出累加器 0 | <code>Tx.warp.gemm_async(..., dispatch="tcgen05")</code> | <code>tcgen05.mma</code> (带一个 TMEM 操作数)       |
| Correction(校正) | TMEM 中的 0 -> 寄存器 -> TMEM 中的 0         | TMEM 回读、寄存器乘法、TMEM store                                 | <code>tcgen05.ld</code> / TMEM store          |
| Epilogue(收尾)   | 最终的 0(TMEM)-> 寄存器 -> SMEM -> GMEM     | TMEM 回读、Tx.copy、TMA store                                | <code>tcgen05.ld</code> + TMA store           |

几个细节值得单拎出来说:

- **Softmax 写回为什么非得来一句 `tcgen05.wait.st()` (等待写入完成)?** 因为把寄存器里的 P 写回 TMEM 这个动作是**异步的**——发出去之后程序会继续往下跑, 不保证数据已经真的落地。可后面紧跟的 value MMA 又要这块 P 当输入读。那怎么办? 必须先 **wait** (原地等一下), 确认写入真落地了, 才能放心让矩阵乘去消费——这是图里一条藏着的、但很关键的依赖边。
- **Value MMA 带一个「TMEM 操作数 (operand, 即送进矩阵乘的输入矩阵)」。** 普通矩阵乘的两个输入都在 SMEM; 可这里 P 在 TMEM、V 在 SMEM, 矩阵乘直接吃一个来自 TMEM 的输入, 省掉了把 P 再搬回 SMEM 那一趟。
- **Correction 这一行是「新冒出来的」。** `row_max` 一往上跳, 之前算的 0 就还停在旧 scale 上, 必须先把 TMEM 里的 0 读到寄存器、乘上校正系数、再写回 TMEM, 然后下一次 value MMA 才能接着往里累加。这正对应上一节强调的那句: `rescale` 是一次 tile 操作, 不是标量记账。

哪些边是 GEMM 里没有的

把这张图跟 GEMM 的链摆一块儿对比, 多出来的就是 **softmax** 和 **correction** 这两行。它俩有个共同毛病:

1. 都得来回跑 **TMEM -> 寄存器 -> TMEM** (GEMM 的 MMA 链里, 数据基本不用绕回寄存器再算)。
2. 都在 **score MMA** 和 **value MMA** 之间硬加了一道交接 (**handoff**)——也就是说, 两段 MMA 没法背靠背连上, 中间隔着「先读出来、在寄存器里算、再写回去」这一套。

FA4 为什么要更细的角色分工、更绕的同步? 根子就在这两条「绕回寄存器」的边上。数据要在不同硬件单元之间来回交接 (从 Tensor Core 到 CUDA Core 再回去), 那就得有人生产、有人消费, 中间还得有个同步器 (`mbarrier`) 来喊一嗓子: 「好了, 我做完了, 下一个消费者可以开跑了。」

注意

**注意：**读这张图，有个特别好用的练习——只盯那条短路径，对每一个箭头挨个回答四件事：**生产者**是哪个阶段、**消费者**是哪个阶段、**源 tile** 是谁、**目标 tile** 是谁、**走的哪条硬件路径**。一个箭头一个箭头过完，再问自己一句：哪些箭头在 GEMM 那章里**压根不存在**？答案就落在 softmax 读/写、还有 correction 这几条 TMEM↔ 寄存器的边上。

Warp 的角色与作用域 (Warp Roles and Scopes)

上一小节把「数据怎么流」讲完了——Q、K、V 怎么在各级内存之间倒腾，两次矩阵乘各在哪一步发生。数据通路定了之后，下一个该问的问题自然就是：**这每一个阶段，到底谁来干？**

答案是：把一组线程按「角色」分，而不是按「数据」分。在讲这个之前，得先把 GPU 线程的几个组织层级讲清楚——这是看懂后面所有分工的地基：

一句话先理解

**一句话先理解：**GPU 上的线程不是一个个散着跑的，而是一层**层打包成组**的。最小的硬件单位是 32 个线程绑成的一个 **warp**(线程束)，它们**必须齐步走**——同一时刻执行同一条指令，像一个 32 人的方阵迈同一只脚。再往上，几个 warp 组成 **warpgroup**，一大堆线程组成一个 **CTA**(线程块)。

| 层级                | 是多少                        | 大白话                              | 类比                |
|-------------------|----------------------------|----------------------------------|-------------------|
| 线程 (thread)       | 1 个                        | 最小的执行单元，有自己的寄存器                  | 像一个工人             |
| warp(线程束)         | 32 个线程                     | <b>必须齐步走</b> :32 个线程同一拍执行同一条指令   | 像一个必须迈同一步的 32 人方阵 |
| warpgroup(warp 组) | 4 个 warp = 128 线程          | 干同一类活的一支「施工队」                    | 像一个班组             |
| CTA(线程块)          | 这里是 4 个 warpgroup = 512 线程 | 一起被调度、 <b>能共享同一块 SMEM</b> 的一大群线程 | 像一个车间             |

为什么要管「warp 必须齐步走」这件事？因为它直接影响性能：如果一个 warp 里 32 个线程因为 **if** 分支各走各的路，硬件只能一拨拨轮流跑，效率大跌（这叫「分支发散」，后面因果掩码一节会专门躲它）。

回到 Flash Attention。这里每个 CTA 有 4 个 warpgroup，总共 512 个线程 (4 × 128)。FA4 的分工原则很要紧：

注意

**注意：**线程是按「这个 warpgroup 负责哪一类活」来分的，不是按「它碰哪块数据」来分。这就是看懂整个 kernel 结构的入口。

整体先记住一句话：

- **WG3** 是「开机器」的那个 **warpgroup**——TMA 加载、MMA 计算、TMA 存储这三类硬件引擎指令，都归它来发。
- **WG0、WG1、WG2** 是「埋头算寄存器数学」的 **warpgroup**——夹在两次引擎调用中间的那些运算:softmax、correction(校正/重缩放)、epilogue(收尾)，都归它们干。

换句话说,WG3 更像一个「调度台」,只管按按钮启动那些专用硬件单元 (TMA 引擎、Tensor Core);WG0~WG2 才是真正趴在通用 CUDA core 上做寄存器运算的「计算工」。

完整角色分配表

| 归属 (Owner)  | 角色 (Role)             | 具体做什么                          |
|-------------|-----------------------|--------------------------------|
| WG3, warp 1 | TMA load              | 把 Q、K、V 的 tile 从 GMEM 搬到 SMEM  |
| WG3, warp 0 | MMA                   | 发起 score MMA 和 value MMA 两类矩阵乘 |
| WG3, warp 2 | TMA store             | 把最终的 O tile 从 SMEM 写回 GMEM     |
| WG0         | Q stage 0 的 softmax   | 从 TMEM 读 S, 算出 P, 把 P 写回 TMEM  |
| WG1         | Q stage 1 的 softmax   | 对第二个 Q 流水线阶段做同样的工作             |
| WG2         | correction 与 epilogue | 在 TMEM 中重缩放 O、归一化、整理输出         |

能看出来,WG3 内部还按 warp 又细分了一层:warp 0 专盯 MMA,warp 1 专盯 load,warp 2 专盯 store。三类硬件引擎 (搬数据的 TMA、做矩阵乘的 Tensor Core、写回的 TMA),一类配一个 warp 守着,谁也不碍着谁。这就像三个工位各派一个班长盯着各自的机器。

关键澄清:「两个 Q stage」不是两个 attention head

表里 WG0 和 WG1 干的是「同一种活」(softmax),只不过各管 Q 流水线里的一个槽位。这地方特别容易看岔:

注意

注意: 千万别把「两个 Q stage」读成「两个 attention head」。它俩其实是 **Q 流水线里的两个槽位 (slot)**——WG0 占一个,WG1 占一个,这样两个 Q tile 就能同时「在飞」(in flight),凑成流水并行。softmax 之所以写了两份,就是因为有这俩流水槽,跟「有两个头」一点关系没有。

画张图,把这种「同种活、两个流水槽」的关系摆出来:

结构图

分组:

- **Q 流水线 (两个并发槽位)**
- **Q stage 0:** 「WG0: softmax」
- **Q stage 1:** 「WG1: softmax」

连接关系:

- **Q 流水线 (两个并发槽位) → 两个 Q tile 可同时在飞 → 隐藏延迟、提升吞吐**

代码里如何「认领」角色

kernel 怎么让「同一份代码」里的不同线程各干各的活？靠的是让每个线程先问自己两个问题：「我属于哪个 warpgroup？我在这个 warpgroup 里是第几个 warp？」拿到这两个编号，后面就能用 if 分支把活儿派下去：

```
wg_id = T.warpgroup_id([4]) # warpgroup? 0-3
warp_id = T.warp_id_in_wg([4]) # warpgroup warp?
```

逐行说：`T.warpgroup_id` 和 `T.warp_id_in_wg` 是框架提供的「身份查询」函数，每个线程调用它都会得到「自己」的编号——这跟多线程编程里用 `thread_id` 区分线程是一个套路，只不过这里查的是「我在哪个组、哪个 warp」。拿到 `wg_id` 后，代码里就会出现 `if wg_id == 3`：这样的分支，把不同的活分给不同的施工队。

注意

注意：读这份 kernel 有个实用窍门——先去找「角色分支」(role branch)，也就是那些基于 `wg_id / warp_id` 的 if 分支。一旦你锁定了某个角色分支，嵌在它里头的所有 tile 级原语 (load、MMA、softmax 等) 归哪支「队伍」管，就一目了然了。这是快速理清控制流的最佳切入点。

各个角色分支里头到底在干什么：

- **WG3 warp 1**: 发搬数据 (TMA load) 的命令。先由一个被选中的 lane(elected lane;lane 就是 warp 内某个线程的编号,0~31) 发出拷贝指令——为什么只让一个线程发？因为发指令这种事只需要一个人喊一嗓子,32 个人同时喊反而会重复添乱。接着 TMA 引擎就自己去搬 tile 了。
- **WG3 warp 0**: 发矩阵乘 (`tcgen05.mma`) 指令，也就是叫 Tensor Core 干活。
- **WG0 和 WG1**: 以整个 warpgroup 一起协作的方式跑 softmax——也就是 128 个线程一整组合力完成 (softmax 要对一整行求最大值、求和，本来就得多个线程配合)。
- **WG2**: 同样是整组 warpgroup 一起跑 correction(校正/重缩放) 和 epilogue(收尾)。

这里出现了「作用域 (scope)」这个词，它指的就是「这个操作由多大一拨线程协作完成」：可能是单个 lane、一个 warp、或一整个 warpgroup。后面读卡片时会反复用到这个概念。

一个关键的「不对称」，决定了整张 barrier 图

整个 kernel 的同步结构 (谁等谁的那张关系图)，被一个核心的「不对称」给定了形：

注意

注意：所有矩阵乘——不管是算分数的还是算输出的——都只从 **WG3 warp 0** 这一个地方发出来。WG0 和 WG1 自己从来不发矩阵乘；它们只负责「消费」已经算好的分数 tile，跑 softmax，再把结果 P 写回 TMEM。

为什么 softmax 非得拿同步器 (mbarrier) 包起来？根子就在这儿：发矩阵乘的 (WG3 warp 0) 和消费分数、跑 softmax 的 (WG0/WG1) 压根不是同一拨人。两边各干各的，中间就必须有个东西帮它们对上节奏——就像两个线程通过条件变量握手：「数据我备好了」「收到，我开始处理」。这条依赖链画成图是这样的：

结构图

连接关系:

- WG0 / WG1 / 跑 softmax / 生成 P  $\xrightarrow{\text{p\_o\_rescale 屏障 / 传递 P + 安全的 O 槽位}}$  WG3 warp 0 / 发起 value MMA

这里冒出来两个会贯穿整章的 mbarrier 名字, 务必记牢:

| 屏障名         | 作用                                                                                 |
|-------------|------------------------------------------------------------------------------------|
| s_ready     | 把 score tile 从 MMA warp(WG3 warp 0) 传递给 softmax(WG0/WG1), 表示「分数算好了, 可以做 softmax 了」 |
| p_o_rescale | 把 P 以及一个对 value MMA 安全的 O 槽位传递回去——这个 O 槽位要么已经完成重缩放, 要么因为不需要重缩放而被释放                 |

注意

**注意:**s\_ready 和 p\_o\_rescale 这两名字, 本章后面会反复出现。它们就是把「MMA 发起方」和「softmax 消费方」这条跨 warpgroup 依赖链拴在一起的两根绳——把它俩弄明白了,FA4 同步逻辑的骨架你就拿下了。

怎么读后面的代码片段 (Reading the Fragments)

本章后面要一段段拆的代码, 全是从 flash\_attention4.py 这个真实 kernel 里抠出来的片段 (fragment, 就是「节选的代码块」, 不是完整能跑的程序)。既然是节选, 就难免会用到一些「定义在别处、本章又没整段贴出来」的名字。第一次撞上一个不认识的变量, 人很容易就卡住了——所以这一小节的用处, 就是先把这些名字**一次性理一遍**, 给你一张随时能翻回来查的「术语速查表」。读代码读不懂某个变量时, 翻回这里查一下就行, 不必死记。

怎么用? 很简单: **现在不用背**。把这节当字典摆着, 后面哪段代码蹦出个陌生名字, 翻回来瞄一眼它是啥就行。

哪些名字「望文生义」, 会在用到时就地解释

有一类名字本身就挺「望文生义」的, 它们会在**第一次真正派上用场的地方**就地登场、就地讲清楚, 所以这里不细说, 先让你混个眼熟:

- wg\_id、warp\_id:warpgroup 编号、warpgroup 内的 warp 编号 (上一节已经见过)。
- BLK\_M / BLK\_N:tile 在 M / N 方向上的分块大小。
- HEAD\_DIM: 注意力头维度 \(\text{d}\)。
- kv\_stage:K/V 流水线的阶段 (stage) 索引。
- SMEM\_PIPE\_DEPTH\_\* / TMEM\_PIPE\_DEPTH: 各条流水线在 SMEM / TMEM 上的**深度** (同时能有几个 tile 在路上)。
- should\_accumulate:value MMA 这一步是「累加进旧 0」还是「初始化 0」的标志。
- CTA\_GROUP: 一个 MMA 由几个 CTA 协作完成 (本章固定为 1)。

这些等到对应片段出现时再讲, 反而更顺, 这里就先不抢戏了。

剩下这些名字：用到前先建个索引

真正得先打个底的，是下面这批——它们既不那么望文生义，又会反复出现。下表用中文把它们的意思重新捋了一遍，后面看代码随时翻回来查：

| 名字                                      | 含义                                                                                                                                                                                                    |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| q_stage、i_q                             | Q 流水线的阶段号，取值 0 或 1，表示用的是哪个 Q tile 槽位（SMEM_PIPE_DEPTH_Q = 2）。 <b>关键巧合</b> ：在 WG0/WG1 的 softmax 里，warpgroup 自己的 wg_id(0 或 1) 就正好等于这个阶段号，所以 S_region[q_stage]、P_region[wg_id]、O_region[i_q] 选中的是同一个 Q 阶段 |
| MMA_N                                   | score / output tile 的宽度，以 TMEM 列数计，这里是 128                                                                                                                                                            |
| MMA_K                                   | value MMA 在内层 K 维（也就是 P / V 的列方向）上每步前进的宽度，16 列                                                                                                                                                        |
| K_SPLIT                                 | value MMA 调度的切分点，等于 6 * MMA_K = 96(详见《两个 MMA 阶段》一节)。第一段 value MMA 负责列区间 0:K_SPLIT，即 0:96                                                                                                              |
| should_rescale                          | WG2 用的 <b>逐行 (per-row) 标志</b> ：在下一一次 value MMA 之前，旧的 0 是否需要先重缩放 (rescale)。它在整个 warp-group 内用 any_sync 归约成一个统一结论                                                                                       |
| rescale_threshold                       | 「行最大值变化太小就跳过 rescale」的阈值，本版 kernel 取 8.0；一旦判定跳过，acc_scale 会被直接设成 1.0(即「不缩放」)                                                                                                                          |
| scale_log2                              | 以 log2 为底表达的 softmax 缩放系数， $\log_2(e)/\sqrt{d}$ ，于是 $P = \exp_2((S - m) \cdot \text{scale\_log2})$                                                                                                    |
| acc_scale                               | softmax 算出来、要传给 WG2 的 <b>逐行重缩放因子</b>                                                                                                                                                                  |
| chunk_start / chunk_end、p_start / p_end | 当前正在 <b>读取 / 写入</b> 的那一段 32 宽 softmax chunk 的列区间                                                                                                                                                      |

几个值得提前点透的设计意图

光盯着表格，容易只记住「等号右边等于几」；可有几条背后的「为什么」，一旦你看穿了，后面读代码会顺很多。

① 三个数组，共用同一个阶段号——这是故意对齐的。q\_stage、wg\_id、i\_q 在 softmax 这条路上其实就是**同一个 0/1**，只不过分别出现在不同数组的下标位置上。想通这点有什么好处？以后你看到 S\_region[q\_stage]、P\_region[wg\_id]、O\_region[i\_q]，就不用担心它们指向三套八竿子打不着的东西——它们说的是同一个 Q 流水线槽位里的分数、概率、输出。用下面这张图把这个「三合一」记牢：

Q 流水线两个槽位（SMEM\_PIPE\_DEPTH\_Q = 2），同一列里的 S / P / O 属于同一个 Q tile，且 q\_stage == wg\_id == i\_q：

| 维度   | 槽位 0        | 槽位 1        |
|------|-------------|-------------|
| 阶段号  | 0           | 1           |
| 谁负责  | WG0 softmax | WG1 softmax |
| 分数 S | S_region[0] | S_region[1] |
| 概率 P | P_region[0] | P_region[1] |
| 输出 O | O_region[0] | O_region[1] |

## 注意

**注意:** 别把「两个 Q 阶段」看成「两个注意力头 (attention head)」。它们就是 Q 流水线里的两个槽位, 目的是让两个 Q tile 同时在路上 (in flight), WG0、WG1 各管一个。softmax 代码「写了两遍」, 纯粹是因为有这俩并行的槽位。

② **MMA\_N / MMA\_K / K\_SPLIT: 列方向上的尺寸语言。**这三个名字都是在说 TMEM 列方向上的「步长和边界」: MMA\_N = 128 是一个 tile 多宽, MMA\_K = 16 是 value MMA 每步往前啃几列, 而 K\_SPLIT = 6 \* MMA\_K = 96 把整条 value MMA 切成了两段。为什么要切? 这是后面《两个 MMA 阶段》那一节的重头戏——眼下先记住「第一段管 0:96、剩下的归第二段」这条边界就够了。

③ **scale\_log2: 为什么用 log2, 不用自然底数。**softmax 里要算  $\exp(S - m)$ , 可硬件上  $\exp_2$  (以 2 为底) 比  $\exp$  (以 e 为底) 省事。换底公式给出  $\exp(x) = \exp_2(x \cdot \log_2(e))$ , 于是把  $1/\sqrt{d}$  的缩放和  $\log_2(e)$  合成一个常数  $\text{scale\_log2} = \log_2(e)/\sqrt{d}$ , 最后  $P = \exp_2((S - m) \cdot \text{scale\_log2})$ 。一句话: 用  $\exp_2$  是为了贴合硬件指令, scale\_log2 就是「把所有缩放揉进一个系数」的产物。

④ **should\_rescale / rescale\_threshold / acc\_scale: rescale 这条主线的三件套。**还记得导读里那个 twist 吗——running max 一变, 旧 0 就处在错误尺度上, 得重缩放。这三个名字正是这件事的实现细节:

- **should\_rescale:** 要不要缩放 (先逐行判断, 再用 any\_sync 在 warpgroup 内归约成一个统一动作);
- **rescale\_threshold = 8.0:** 行最大值只动了一丁点 (小于阈值) 就跳过, 跳过时 acc\_scale 直接取 1.0;
- **acc\_scale:** 真要缩放时缩放多少, 由 softmax 算出来, 再通过 SMEM 邮箱 (mailbox) 递给 WG2 去执行。

串起来读就是: softmax(WG0/WG1) 负责拍板缩放因子, WG2 负责动手缩放——这也正好呼应上一节那句「softmax 跟 correction/epilogue 分属不同 warpgroup」的分工。

把这张表和这四点记住, 后面再生的片段也都有处可查。

## 两个 MMA 阶段: 被 softmax 串起来的双矩阵乘 (The Two MMA Phases)

Flash Attention 的内层循环里, 装的不是一个矩阵乘, 而是两个 MMA, 中间拿一段 softmax 串着。每来一个 K/V tile, kernel 都得把这三步挨个跑一遍:

### 结构图

连接关系:

- $S \xrightarrow{\text{softmax}} P$

你可以把它想成三个工人排成一条流水线: 第一个 MMA 产出分数 S, softmax 把 S 加工成分子 P, 第二个 MMA 拿 P 去更新输出累加器 0。这里有个关键设计: 除以 row\_sum 那步归一化, 被故意拖到了最后的收尾阶段 (epilogue)。非得等所有 K/V tile 都处理完、0 攒齐了, 才在最后统一除这一次。这恰恰是在线 softmax 的精髓: 边累加边维护统计量, 最后一锤子归一化, 中途绝不反复除。

怎么读懂每个 tile 算子：四问 + 一个 Handoff

前面讲矩阵乘那几章，我们用一张固定的「读卡」来逐个审视每个操作：作用域 / 布局 / 调度 (scope / layout / dispatch)。这套读卡特别好用，所以这里照搬，只是再添一行 Handoff(交接)，专门点出”是哪个(些)同步器 (mbarrier) 把这块 tile 递给下一个角色”。也就是对每个操作问四个问题：

| 问题           | 含义                                             |
|--------------|------------------------------------------------|
| Scope(作用域)   | 谁来执行它 (哪个 warpgroup / warp / lane，即多大一拨线程协作)   |
| Layout(布局)   | 输入输出 tile 各自住在哪里 (SMEM / TMEM / 寄存器)           |
| Dispatch(调度) | 用什么硬件指令路径来发射这个操作 (比如走 tcgen05 = 用 Tensor Core) |
| Handoff(交接)  | 用哪个同步器 (mbarrier) 把成果交给下一棒，通知它「可以开工了」          |

注意

注意：计算代码从来不直接写裸的 TMEM 列号。kernel 把它唯一那一块 TMEM 分配切成几个”按阶段命名的视图”——S\_region、P\_region、O\_region，然后拿流水线阶段去索引，比如 S\_region[q\_stage]、O\_region[i\_q]、P\_region[i\_q, 0:K\_SPLIT)。这些视图是 T.TMEMStages 在《TMEM 布局与复用》那节里定义的。现在你只要把每个 region 当成”同一块物理 TMEM 上的一个有名字的切片”就行。

第一阶段：分数 MMA(Score MMA)

每一轮 K/V 迭代，开场就是分数 MMA，它算是：

$$S = Q_{\text{block}} K_{\text{block}}^T$$

然后把 128 x 128 的分数 tile 写进 TMEM。核心 API 这么用：

```
Tx.warp.gemm_async(
 S_region[q_stage], # :S TMEM
 Q_smem[q_stage, 0:BLK_M, 0:HEAD_DIM], # A:Q, SMEM
 K_smem[kv_stage, 0:BLK_N, 0:HEAD_DIM], # B:K, SMEM
 dispatch="tcgen05", # Tensor Core(tcgen05)
 cta_group=CTA_GROUP,
)
if T.ptx.elect_sync(): # warp 32 " "
 s_ready.arrive(q_stage) # s_ready " ",
```

逐行说人话：Tx.warp.gemm\_async 就是「叫 Tensor Core 算  $S = Q \cdot K^T$ 」，名字里的 async(异步)意味着发完命令不傻等、接着往下走。dispatch="tcgen05" 是在说「这活儿交给 Tensor Core 硬件」。最后那个 if T.ptx.elect\_sync():elect\_sync 会从一个 warp 的 32 个线程里选出一个当代表，只让它执行 if 里那行；s\_ready.arrive(...) 则是这个代表在同步器 s\_ready 上「报到」，等于喊一声「分数 tile 我算好啦，谁等着用谁可以来了」。

按四问拆解：



- **Scope:** 由 WG3(warpgroup 3) 的 warp 0 发射; 再由一个被选中的 lane(elected lane) 在 s\_ready 上 arrive。
- **Layout:**Q、K 都在 SMEM, 结果 S 落到 TMEM(S\_region[q\_stage])。
- **Dispatch:**tcgen05。
- **Handoff:**s\_ready(交给 softmax)。

注意

**注意:** 整个交接, 就是”被选中的那一个线程在 s\_ready 上报到”这么一个动作。它对外喊一声: 这块分数 tile 算完了, 跑 softmax 的那支队伍可以放心去读了。为什么只让单个线程报到, 而不是整个 warp 一起报到? 因为同步器是靠「数到几个人报到了」来判断的——要是 32 个线程一窝蜂全报到, 计数就乱了, 后面的同步逻辑会判断错。

中间阶段: 夹在两个 MMA 之间的 softmax(Softmax Between MMAs)

两个 MMA 中间夹的就是 softmax, 它把分数 tile S 加工成分子 tile P。这是整条流水线里唯一一个在 GEMM 里完全找不到对应物的阶段——它压根不是矩阵乘, 而是逐行 (row-wise) 的标量运算。它的卡片是:

- **Scope:**WG0(对应 Q stage 0)/ WG1(对应 Q stage 1), 整个 warpgroup 都参与。
- **Layout:**S 在 TMEM → 读进寄存器 (registers) → 算出的 P 以 fp16 写回 TMEM(P\_region[wg\_id])。
- **Dispatch:** 用 tcgen05.ld 读出, 用 TMEM store 写回; 中间在寄存器里做逐行数学。
- **Handoff:** 等待 s\_ready; 完成后在 p\_o\_rescale(前 96 列) 和 p\_ready\_2(后 32 列) 上 arrive。

WG0/WG1 先等 s\_ready(确认分数 tile 备好了才动手, 不然会读到半成品), 然后一次搬一小块地把分数从 TMEM 读进寄存器 (为什么不一次全搬? 因为寄存器太小, 只能一块块来, 这块算完腾出来再搬下一块):

```
Tx.copy_async(#
 s_chunk[:, chunk_start : chunk_end], # :
 S_region[wg_id, chunk_start : chunk_end], # :TMEM tile
) # TMEM
```

Tx.copy\_async 就是「搬一块数据」,async 同样表示异步发起。chunk\_start : chunk\_end 是 Python 的切片语法, 选中分数 tile 的某一段列。S\_region[wg\_id, ...] 里用 wg\_id(0 或 1) 当下标——还记得吗,WG0 / WG1 的编号正好对应它负责的那条 Q 流水线分支, 所以这里直接拿自己的编号去选「归我管的那块分数」。

分数进了寄存器,softmax warpgroup 就严格按顺序干三件事:

1. 算出每行的行最大值 (row max) 和行和 (row sum);
2. 算出 softmax 分子 tile P;
3. 把 P 以 fp16 写回 TMEM。

第 3 步的写回长这样 (方向反过来, 从寄存器写回 TMEM):

```
Tx.copy_async(
 P_region[wg_id, p_start : p_end], # :TMEM P (fp16)
```

```
p_chunk[:, p_start : p_end], # : P
)
```

这里很自然会冒出一个疑问:**P 我们刚在寄存器里算出来了,干嘛还费劲写回 TMEM?**

答案是:下游的取值 MMA 要把 P 当一块**矩阵输入**来读,而矩阵乘指令读了散落在各个线程私有寄存器里的零散标量——Tensor Core 只认那种「共享、规整、能按矩阵格子寻址」的内存 (SMEM 或 TMEM)。打个比方:寄存器里的 P 像是 128 个人各自手里攥着自己那几个数,而 Tensor Core 要的是「一张摊在桌上、所有人都能按行列读取的整表」。在这个 kernel 里,P 唯一能被矩阵乘读进去的形态,就是 **P\_region**(它落在 TMEM 上)。所以这趟写回**不是白搬**,而是把 P 摆成「下一次矩阵乘唯一认得的形状」,不做不行。

## 第二阶段: 取值 MMA(Value MMA)

每一轮 K/V 迭代收尾的,是取值 MMA,它算的是:

$$O = O + P_{\text{block}} V_{\text{block}}$$

走到这一步,O 已经被调到当前 K/V 块该有的正确状态了——第一块时被初始化,后面每块时被重缩放 (rescale) 过——所以这个 MMA 剩下要干的就只有“累加”。它跟普通 GEMM 最大的区别,在于**操作数都住在哪**:A 操作数 P 在 TMEM,B 操作数 V 在 SMEM,累加器 O 也在 TMEM。

```
MMA: P 96 (0:K_SPLIT), V 96
Tx.warp.gemm_async(
 O_region[i_q], # ():0 TMEM
 P_region[i_q, 0:K_SPLIT], # A:P, TMEM
 V_smem[kv_stage, 0:K_SPLIT, 0:HEAD_DIM], # B:V, SMEM
 transB=True, # B(V)
 accum=should_accumulate, # " 0 " " 0"
 dispatch="tcgen05", # Tensor Core
 cta_group=CTA_GROUP,
)
MMA (accum True, p_ready_2),
K_SPLIT:BLK_N
```

**accum** 这个参数值得专门点一下:它是个开关,True 表示「把这次结果**累加到** 0 原来的值上」,False 表示「**直接覆盖**写入 (相当于初始化)」。下面注意框会讲它什么时候取 True、什么时候取 False。

卡片:

- **Scope**:WG3 的 warp 0。
- **Layout**:P 在 TMEM + V 在 SMEM → 0 在 TMEM(**O\_region[i\_q]**)。
- **Dispatch**:tcgen05,且带一个 TMEM 操作数。
- **Handoff**:等待 p\_o\_rescale、p\_ready\_2、kv\_load.full;完成后在 o\_ready 上 arrive(交给 epilogue)。

两个 MMA 在硬件上的根本区别:操作数位置

| 项目     | 分数 MMA(Score MMA) | 取值 MMA(Value MMA)  |
|--------|-------------------|--------------------|
| A 操作数  | Q, 来自 SMEM        | P, 来自 TMEM         |
| B 操作数  | K, 来自 SMEM        | V, 来自 SMEM         |
| 累加器/输出 | S 写入 TMEM         | O 累加进 TMEM         |
| 特点     | 两个操作数都从 SMEM 读    | 一个操作数 (P) 从 TMEM 读 |

看得出来: 分数 MMA 两个输入都从 SMEM 来; 而取值 MMA 把 P 这一路输入换成了从 TMEM 直接喂给 Tensor Core——这正是上一步非把 P 写回 TMEM 不可的根本原因。

注意

注意:accum=should\_accumulate 这个标志, 就是算法里”初始化还是累加”那个开关的实现: 一个 query 块的第一个 K/V tile 上它是 False(直接写, 等于初始化), 后面每个 tile 上都是 True(往上加)。

为什么把取值 MMA 拆成 96 + 32 两刀?

你会发现取值 MMA 不是一发打完的, 而是按 96 + 32 拆成两次发。流程是这样:

结构图

连接关系:

- softmax 写 P, 分成 4 个 32 列 chunk: chunk0 | chunk1 | chunk2 | chunk3 → 前 96 列 (chunk0~2)
- softmax 写 P, 分成 4 个 32 列 chunk: chunk0 | chunk1 | chunk2 | chunk3 → 后 32 列 (chunk3)
- 前 96 列 (chunk0~2) → 前三块就绪即可发射 / 第一个子 MMA (96 宽)
- 后 32 列 (chunk3) → 等 p\_ready\_2 再发射 / 第二个子 MMA (32 宽)
- 前三块就绪即可发射 / 第一个子 MMA (96 宽) → 两者在时间上重叠: / chunk3 的 exp + TMEM 写回与 96 宽 MMA 同时进行
- 等 p\_ready\_2 再发射 / 第二个子 MMA (32 宽) → 两者在时间上重叠: / chunk3 的 exp + TMEM 写回与 96 宽 MMA 同时进行

拆分的逻辑是这样:

- softmax 把 P 分成 4 个 32 列的 chunk 来写;
- 前 3 个 chunk(共 96 列) 一备好, 取值 MMA 就立刻对着 P 的前 96 列和 V 的对应行开跑;
- 最后 32 列得等 p\_ready\_2;
- 第二个 MMA 把这最后一块吃掉, 整个 tile 收尾。

拆开的根本目的, 就是别让 Tensor Core 闲着。假如把取值 MMA 当一条整指令来发, 那整个阶段就得干等着, 等到全部 4 个 32 列 chunk 都求完指数 (exp)、都存好为止。而”前三块一就绪就先开打”这么一来, kernel 就把最后一块 chunk 的 exp 计算和 TMEM 写回, 跟一条已经在飞的 96 宽 MMA 叠到一块儿了——本来要白白浪费的空闲时间, 这下变成了有用的计算。这是典型的”用细粒度流水化 (pipelining) 换吞吐”的手法。

TMEM 的布局与复用 (TMEM Layout and Reuse)

这一小节回答一个看着不起眼、实际牵动全局的问题:S、P、O 这三块中间结果, 到底搁哪儿? 答案有点出人意料——它们全被塞进**同一块** 128 × 512 的 TMEM 里。正因为挤在一起, 在这个 kernel 里同步器和内存布局就成了拆不开的一对: 地方太小逼着你「复用」(同一块内存先给这个用、用完再给那个用), 而复用要想不出错, 就只能靠同步器盯着「上一个真用完了没」。这跟手动管理一个很小的内存池、靠加锁防止读写冲突, 是一个道理。

注意

**注意:** 这里的 TMEM 是 Blackwell 架构带来的、专给 tcgen05 MMA 用的张量内存。它的容量小得可怜 (每个 CTA 只有 128 × 512 的 fp32 空间), 所以”省着用”在这儿不是什么优化技巧, 而是硬性约束。

三类 tile slot 共享同一块分配

你可以把这块 TMEM 想成一排 tile 槽位 (tile slot), 每一类数据占其中一段:

| 槽位类型                 | 存放内容                  | 数据含义                        |
|----------------------|-----------------------|-----------------------------|
| Score slots(分数槽)     | $S = QK^T$            | 注意力打分矩阵                     |
| Numerator slots(分子槽) | softmax 指数化之后的 P tile | 即 $\exp(S - \text{rowmax})$ |
| Output slots(输出槽)    | fp32 的 0 累加器          | 最终输出的累加结果                   |

关键在于: 这三类槽位**不是各自独立的缓冲区**, 而是同一块物理分配上的不同区域。这么共享不是为了代码好看, 纯粹是被容量逼的。

算笔账你就明白”被逼”这俩字一点不夸张。设 Q 方向的流水线深度 (Q-pipeline depth) 是 2: TMEM 总容量:128 行 × 512 列 (fp32)。逐项算账:

| 占用项      | 列数计算                                    | 列数         |
|----------|-----------------------------------------|------------|
| 两个 S 槽   | $2 \times \text{MMA\_N} = 2 \times 128$ | 256        |
| 两个 O 槽   | $2 \times \text{MMA\_N} = 2 \times 128$ | 256        |
| 合计已占用    |                                         | 512(正好占满!) |
| 留给 P 的空间 | 512 512                                 | 0          |

也就是说, 光 S 和 O(各两个流水线 stage) 就把 512 个 fp32 列**全吃光了**, 一列不剩。P 连个落脚的地方都没有。

P 如何”无中生有”:fp16 视图的别名 (aliasing) 技巧

先解释两个数据类型:**fp32** 是 32 位浮点数 (占 4 字节, 精度高),**fp16** 是 16 位浮点数 (占 2 字节, 精度低一半但够用)。注意力计算里很多地方用 fp16 就足够了, 还能省一半空间。

fp32 这个视角下既然腾不出地方,kernel 就玩了个花招: 用 **fp16 视图**把这块物理内存重新盖一遍。所谓「视图 (view)」, 就是**不搬动任何数据、只是换个角度去解读同一片内存字节**——这跟 C 里把一个 float\* 强转成 short\* 去读同一块内存是一模一样的思路。道理很简单:fp16 元素只有 fp32 的一半宽, 同样多的字节, 用 fp16 来数就能数出**两倍**的列。P 就住进这些「凭空冒出来」的列里 (其实是借住在 S、O 那片字节上, 后面会讲怎么靠时序保证不打架)。

下面这段代码就是这个别名 (aliasing, 即「两个名字指向同一块内存」) 技巧的核心: 先拿一个 fp32 视图, 然后把内存池的「下一个可分配位置」倒回起点 0, 再在一模一样的那片物理字节上拿一个 fp16 视图:

```
tmem_pool = T.TMEMPool(pool, total_cols=N_COLS_TMEM, cta_group=CTA_GROUP, tmem_addr=tmem_addr)
tmem = tmem_pool.alloc((128, N_COLS_TMEM), "float32") # fp32 : S 0
tmem_pool.move_base_to(0) # : " "
↪ 0,
tmem_as_f16 = tmem_pool.alloc((128, N_COLS_TMEM * 2), "float16") # fp16
↪ : , P
tmem_pool.commit() #
```

逐行讲: `alloc(...)` 就是「从内存池里划出一块」, 第二个参数是数据类型。第一次 `alloc` 划出 fp32 的 `tmem`; 然后 `move_base_to(0)` 是整段的命门——它把内存池「下次该从哪儿往下分」的游标强行拨回到 0, 于是第二次 `alloc` 又从同一个起点开始分, 得到的 `tmem_as_f16` 就跟 `tmem` 指向同一块物理内存, 只不过一个按 fp32(4 字节) 解读、一个按 fp16(2 字节) 解读。注意 fp16 那次申请的列数是 `N_COLS_TMEM * 2`(翻倍), 正对应「元素半宽 → 同样字节能数出两倍列」。

#### 注意

**注意:** 这套别名能安全跑, 唯一的前提是**时序**——每块区域都得严格等它“上一任租客”用完了, 才被重新启用。而保证这个时序的, 恰恰是同步器 (mbarrier)。所以在 FA4 里, 同步器不只是一个调度工具, 它从根上让这套「同一块字节轮流给不同数据用」的布局**合法、不出错**。下一小节会专门拿一张表来捋“每块区域复用之前, 得先等谁干完”。

### 用 T.TMEMStages 把字节切成按 stage 索引的区域

有了这两个视图, 内核再用 `T.TMEMStages` 把 S、P、O 各自切成几个「按阶段编号的区域」。你可以把 `T.TMEMStages` 理解成一个「带下标的数组工厂」: 给它起点列、每块多宽、分几个阶段、每阶段间隔多少, 它就生成一个能用 `[0]`、`[1]` 这样取的区域对象。这么做的好处是: 上层计算代码只管 `S_region[0]`、`O_region[1]` 这样按编号取, 根本不用关心「实际落在第几列」这种脏细节。

```
S fp32 , 0 ; MMA_N , SMEM_PIPE_DEPTH_Q , MMA_N
S_region = T.TMEMStages(tmem, col_start=0, width=MMA_N,
↪ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
O fp32 , S (MMA_N x)
O_region = T.TMEMStages(tmem, col_start=MMA_N * SMEM_PIPE_DEPTH_Q, width=MMA_N,
↪ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
P fp16 , (stride) x2 fp32
P_region = T.TMEMStages(tmem_as_f16, col_start=MMA_N, width=BLK_N,
↪ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N * 2)
```

这里唯一让别名“露馅”的地方, 就是 `P_region` 那个 `stride=MMA_N * 2` 里的 `* 2`:

- `S_region` 和 `O_region` 是按 **fp32** 的 `tmem` 列来量的;
- `P_region` 是按 **fp16** 的 `tmem_as_f16` 列来量的, 而 fp16 列只有 fp32 列的一半宽;
- 所以, P 在 stage 之间挪动时, 要想落到跟 **fp32** 布局对齐的同一批物理字节上, 跨 stage 的步长就得**翻倍**。

拿一张图, 就能直观看出三块区域在同一片字节上是怎么对应的 (以流水线深度 2 为例):

物理 TMEM 是 128 行 × 512 fp32 列;fp16 视图把同一片字节按半宽元素重新数, 列数翻倍 (0 ~ 1023)。两个视图在同一片字节上的对应关系如下:

| 字节区间 (按 fp32 列) | 0-128      | 128-256      | 256-384      | 384-512       |
|-----------------|------------|--------------|--------------|---------------|
| fp32 视图         | S 槽 0      | S 槽 1        | O 槽 0        | O 槽 1         |
| fp16 视图 (列号 ×2) | fp16 0-256 | fp16 256-512 | fp16 512-768 | fp16 768-1024 |
| P_region        |            | P 槽 0        | P 槽 1        |               |

P\_region 的 col\_start = MMA\_N = 128(fp16 列), 步长 = MMA\_N\*2 = 256(fp16 列); 跨一个 P 槽正好覆盖一个 fp32 槽的字节宽度。

注意

注意:P\_region 的 col\_start=MMA\_N 和 stride=MMA\_N\*2 都是按 fp16 列量的 (fp16 列只有 fp32 列一半宽)。所以 P 的字节不是简单” 接在 O 后头”, 而是借用了 fp32 视图里某些槽位对应的同一片字节——P 跟 S/O 能合法共存, 全靠 barrier 保证的时序, 而不是靠地址互不重叠。

可以这么理解:P 借住在那段已经算完、不再要的字节里——S 一算完,softmax 把它变成 P 写回去, 这段字节的” 身份” 就从 fp32 的 S 切成了 fp16 的 P, 靠的就是同一块内存被两个视图以不同宽度共用。

区域定义好之后, 计算代码反而很干净

底层那套别名换算虽然绕, 可一旦区域定义好了, 上层计算代码就根本不用操心原始列号。它只管按 stage / warpgroup 去读写:

```
S_region[q_stage] # S
S_region[wg_id, ...] # S
P_region[wg_id, ...] # P
O_region[i_q] # 0
```

整段计算代码从头到尾没碰过一个原始列索引。” 哪段字节归谁、fp16 步长要乘 2” 这些脏活, 全被塞进了 T.TMEMStages 的区域定义里。这正是好抽象该干的事: 把” 被容量逼出来的别名复杂度” 圈在一个角落, 让主干逻辑保持清清爽爽。

和你的 agent 一起试试

你可以让你的 AI agent 来讲讲这个 FA4 内核里的 fp32(tmem) 视图和 fp16(tmem\_as\_f16) 视图:S、P、O 各自落在哪块物理 TMEM 区域?P\_region 的 stride 为什么是 MMA\_N \* 2? 至于” 复用” 这个问题——某块区域要重新启用, 得先等哪些消费者干完——留到下一小节, 看完 barrier 表再回来对照。

barrier 如何把各个角色「接线」起来 (How Barriers Connect the Roles)

这是整个 kernel 里最难啃的一块, 所以最好一层一层剥开看, 别急。前面几节已经把活儿分到了不同的施工队:WG3 开硬件引擎 (搬数据、做矩阵乘、写回),WG0/WG1 做 softmax,WG2 做修正 (correction) 和收尾 (epilogue)。可问题是——这些队伍彼此不直接说话, 它们只能靠同步器 (mbarrier) 互相「递东西、报进度」。这一节要干的, 就是把这些同步器看成一张「谁把哪块数据交给谁」的接线图。说白了, 这就是一套精心设计的「生产者-消费者 + 锁」的协议, 只不过参与方多了点。

先抓主干: 只有 5 个搬运数据的 handoff

面对一长串同步器, 正确的打开方式是: 先只盯那几个真正在主计算路径上搬数据的交接 (handoff = 一次「我做完了, 轮到你」的交班), 其余的一律先当成「记账 (bookkeeping, 即纯粹管理资源回收、不直接搬业务数据)」, 以后用到再查。真正「数据就绪 (data-ready)」的交接, 就下面这 5 条:

| 交接 (handoff)                     | 含义                          |
|----------------------------------|-----------------------------|
| TMA load → score / value MMA     | Q、K 或 V 已经到达 SMEM, 可以喂给 MMA |
| score MMA → softmax              | S 已在 TMEM 里就绪               |
| softmax / correction → value MMA | P 已在 TMEM 里就绪, 且 0 可以安全累加   |
| value MMA → epilogue             | 最终的 0 已在 TMEM 里就绪           |
| epilogue → TMA store             | 0_smem 已就绪, 可以写回            |

除此之外的所有 barrier, 本质上都是流水线记账: 它们管的是释放某块 SMEM / TMEM / 暂存 (staging) 缓冲区, 好让另一个角色拿去复用。

注意

注意: 好消息是, 一个 barrier 不管是搬数据还是单纯记账, 读法都一样——都是一次「tile 交接」。每碰到一个 barrier, 就问三句: 谁生产了数据? 谁来消费? 两边都用完之后, 哪块缓冲区被腾出来了? 这三问问清楚, 再玄乎的 barrier 也就那么回事。

两个 MMA 的「就绪闸门」

把上面这些交接收一收, 关键就看两个 MMA 各自在等什么。这里要摆正一个心态: 这些 barrier 是「正确性闸门 (correctness gate)」, 不是「时间表 (schedule)」。它只回答「这个 MMA 开火前得满足哪些条件」, 至于「什么时候发生」它压根不管。

结构图

连接关系:

- score MMA → S (TMEM)
- value MMA → O (TMEM)

- score MMA 就等两样:Q、K 都到了 SMEM。等齐了就产出 S。
  - value MMA 得同时等三样:V 在 SMEM、softmax 给的 P tile、还有一个能安全写入的 0 槽位 (WG2 要么已经把旧 0 释放掉, 要么已经给它重缩放好了)。
- P 的就绪为什么被拆成两半? 就是因为上一节那个 96 + 32 调度:P 前 96 列一就位,value MMA 就能先开火, 最后 32 列由另一个闸门 p\_ready\_2 单独放行。

唯一的「异类」:softmax → correction 的标量信箱

有一个交接不走「整块 tile 就绪」这套路子:softmax 到 correction 这条边。它递过去的不是一块 tile(一大片数据),而是一个标量(就一个数)——在 K/V 循环里是缩放因子 `acc_scale`, 在 epilogue 里是最终的行和 `row_sum`。这个数通过一个只有一格的 SMEM「信箱 (mailbox)」交给 WG2。可以把它想成两个线程之间一个容量为 1 的队列:生产者放一个值进去,消费者取走。

正因为这一格信箱每轮迭代都得反复复用,所以得用一对 `full / empty` 同步器守着它 (`full = 「里头有货了,可以取」`,`empty = 「取空了,可以再放」`)——这是个标准的「生产者-消费者通道」:

结构图

连接关系:

- softmax 生产者: 1. 等 `softmax_corr.empty` (信箱空了才能复用) → 2. 把 `acc_scale / row_sum` 写入信箱
- 3. arrive `softmax_corr.full`  $\xrightarrow{\text{通知}}$  WG2 消费者: 4. 等 `softmax_corr.full`, 然后读信箱
- 5. arrive `softmax_corr.empty`  $\xrightarrow{\text{释放}}$  6. softmax 下一阶段可复用这一格信箱

这套握手一步步是这样:

1. softmax 先等 `softmax_corr.empty`, 确认信箱已经腾空, 才能复用这一格;
2. softmax 把 `acc_scale` 或最终的 `row_sum` 写进信箱;
3. softmax 在 `softmax_corr.full` 上 arrive(喊一声「写好了」);
4. WG2 等 `softmax_corr.full`, 然后把信箱读出来;
5. WG2 在 `softmax_corr.empty` 上 arrive(喊一声「读完了」);
6. 到下一阶段,softmax warpgroup 就能复用这一格信箱了。

注意

**注意:**`softmax_corr.empty` 的含义特别容易看岔, 务必盯清楚。它只说明「WG2 已经把这一格标量消费掉了」, 它绝不代表 P 就绪, 更不是放行 value MMA 的那道闸门。放行 value MMA 的闸门是 `p_o_rescale`(只在 P 前 96 列写好、且 0 槽位能安全累加时才触发)。把这俩搞混, 是一类经典「结果算错」bug 的源头。

完整 barrier 清单 (参考表)

主干捋清楚了, 下面这张完整清单就当查阅手册用。每一行还是老规矩——「谁 → 谁」、「什么变安全了」:

| Barrier                    | 生产者 → 消费者                      | 什么变得安全了                   |
|----------------------------|--------------------------------|---------------------------|
| <code>q_load.full</code>   | TMA load → score MMA           | Q 的 SMEM tile 可以喂 MMA     |
| <code>q_load.empty</code>  | 本 Q 阶段的所有 score MMA → TMA load | Q 的 SMEM 阶段可被下一个 task 复用  |
| <code>kv_load.full</code>  | TMA load → score / value MMA   | K 或 V 的 SMEM tile 可以喂 MMA |
| <code>kv_load.empty</code> | score / value MMA → TMA load   | K/V 的 SMEM 阶段可被复用         |
| <code>s_ready</code>       | score MMA → softmax            | TMEM 里的 S tile 可读         |



|                    |                           |                                     |
|--------------------|---------------------------|-------------------------------------|
| p_o_rescale        | softmax + WG2 → value MMA | P 的前 96 列已在 TMem, 且 0 槽位可安全累加       |
| p_ready_2          | softmax → value MMA       | P 的最后 1/4(32 列) 已在 TMem             |
| o_ready            | value MMA → epilogue      | 最终的 0 累加器就绪                         |
| softmax_corr.full  | softmax → WG2             | acc_scale 或最终 row_sum 已在 SMem 信箱里就绪 |
| softmax_corr.empty | WG2 → softmax             | WG2 读完后, 同一格信箱可复用                   |
| corr_epi.full      | epilogue → TMA store      | 0_smem 就绪, 可以写回                     |
| corr_epi.empty     | TMA store → epilogue      | 0_smem 阶段可被复用                       |

看「谁发信号」就能猜出 barrier 类型

跟矩阵乘那章一样, 同步器是哪种类型, 看「谁来发完成信号」就知道了。这里有三种, 差别在于「完成」是谁来宣布的:

| 信号来源      | 同步器类型      | 为什么是这样                                             |
|-----------|------------|----------------------------------------------------|
| 搬数据 (TMA) | TMABar     | 搬数据的硬件引擎自己会数搬了多少字节, 数够了就自动宣布「搬完了」                  |
| 矩阵乘完成     | TCGen05Bar | Tensor Core 那边由专门的 tcgen05.commit 指令给「一组算完的话」统一发信号 |
| 纯线程之间交班   | MBarrier   | 没有硬件帮忙, 得靠参与的线程自己一个个手动「报到」(arrive) 才算数             |

拆开看 softmax → value 的双闸门

softmax 到 value 这个交接值得再放大看一眼, 它用了两道闸门:

- p\_o\_rescale: P 前 96 列写好、且 0 tile 能安全累加时, 放行 value MMA 开跑;
- p\_ready\_2: 放行 P 的最后 32 列, 正好对上上一节那个 96 + 32 的 value-MMA 调度。

第一个 K/V 块最省心。这会儿还没有「旧 0」要 rescale, 所以 WG2 干脆对 p\_o\_rescale 直接预先 arrive(pre-arrive)——相当于一上来就把这道闸门打开, 反正也没活可干。

后面的块就得当心了。WG2 必须等到自己要么跳过了一次不必要的 rescale, 要么已经把旧 0 缩放完了, 才在 p\_o\_rescale 上 arrive。这里有个故意做得保守的「跳过判定」:

```
softmax : log2
delta = (m_old - m_new) * scale_log2
if delta > -rescale_threshold:
 # , rescale: max, acc_scale 1.0
 acc_scale = 1.0
else:
 # , exp2 , WG2 rescale 0
 acc_scale = exp2(delta)
```

说白了就是: 只有最大值「跳」得够大, 才走 exp2 路径、真正叫 WG2 去 rescale 0。

拿到 softmax 的信号后, WG2 还要用 any\_sync 把整个 warpgroup 里大家各自的 should\_rescale 汇总成一个统一决定:

```
warpgroup " ": " ",
if T.ptx.any_sync(should_rescale):
 rescale_0() # 0 , - -
```

为什么要 `any_sync`? 还记得 `warp` 必须齐步走吗——一组线程要么一起执行 `rescale_0()`, 要么一起跳过, 不能你做我不做。`T.ptx.any_sync(should_rescale)` 就是「问一圈: 有没有任何一个线程举手说需要缩放? 只要有一个举手, 大家就一起做」。这是把「每个线程各自的逐行判断」收敛成「全组统一动作」的标准手段。

这个「跳过」为什么重要? 因为 `rescale 0` 是一次横跨整个累加器的 **TMEM** → **寄存器 (RF)** → **TMEM** 读-改-写。要是阈值逻辑已经把 `acc_scale` 定成 1.0 了, 那这趟读-改-写就是**纯浪费**, 能跳当然要跳。

为什么所有新 **barrier** 都挤在 **softmax** 周围?

最后还有一个观察值得点破: 所有新增的 **barrier** 都挤在同一个地方。 `s_ready`、`p_o_rescale`、`p_ready_2`, 加上 `softmax_corr` 这一对, 全围着 **softmax** 转。

它们存在的理由只有一个:**score MMA** 和 **value MMA** 不再挨着了。在 **GEMM** 里两个 **MMA** 可以背靠背连发; 可在 **Attention** 里, 中间硬塞进了寄存器上的数学运算、**TMEM** 的回写、还有对输出的 `rescale`——这中间的每一步, 都得配一次自己的交接。说到底,**barrier** 的数量, 就是「softmax 这段非 **MMA** 工作」复杂度的直接写照。

和你的 **agent** 一起试试: 让它把一个 **K/V** 块完整走一遍 `s_ready` → `p_o_rescale` → `p_ready_2` → `o_ready`。每碰到一个 **barrier**, 都把四件事问清楚: 谁在等? 谁来 `arrive`? 哪块 **tile** 变得能读了? 之后哪块存储能被复用了?

流水线结构: 同一时刻到底谁在干活 (Pipelining Structure)

上一节讲的 **barrier** 回答的是「正确性」问题: 某个角色 (**role**) 消费一个 **tile** 之前, **哪些东西得先备齐**。可 **barrier** 有一件事不告诉你——**实际上到底谁和谁在同时跑**。这一节就来回答这第二个问题。

这俩真是两码事。一道正确性闸门啥时候被满足, 跟生产者啥时候真正在跑, 可以差出十万八千里——可能早得多, 也可能晚得多。换句话说, 「**S** 就绪了没」这个检查发生在哪一刻, 跟 「**S** 到底是啥时候算出来的」根本不绑定。**barrier** 图回答的是依赖关系, 这一节要画的是**时间线 (timeline)**。

没有「单一流水线深度」这回事

一句话先理解

**一句话先理解:**「流水线 (pipeline)」就是「上一道工序还没干完, 下一批料就先进来等着」, 像工厂流水线一样让各工位同时忙起来。「深度」就是「同时允许几批料在线上飞」。

矩阵乘里我们习惯问「流水线深度 (pipeline depth) 是几」——因为矩阵乘只有一条数据流, 所有东西都跟着同一个节奏走。**FA4** 这套就不行了: 不同的数据流前进的速度不一样, 所以**每条流各管各的环形缓冲 (ring buffer, 即一圈固定大小的缓冲槽, 转着圈复用)**, 各有各的深度。环形缓冲就像一个固定几格的循环队列: 填满了就得等最早那格被消费掉、空出来, 才能再往里填。

| 流水线 (tile 流) | 深度 | 含义                                         |
|--------------|----|--------------------------------------------|
| Q 流水线        | 2  | 一个 CTA 同时处理两个 Q stage:WG0 负责其中一个,WG1 负责另一个 |

|          |   |                                                                    |
|----------|---|--------------------------------------------------------------------|
| KV 流水线   | 3 | K、V 块在内层循环里不断流过，而同样的两个 Q stage 被反复复用                               |
| TMEM 流水线 | 2 | 每个 Q stage 拥有自己的一组 S/P/O 的 TMEM slot；等对应 barrier 触发后这些 slot 才被回收复用 |

注意

**注意:** 把这三条流水线分开看，是关键。Q 为什么是深度 2? 因为「两个 Q stage 并行算 softmax」(WG0 和 WG1 各管一个);KV 为什么是深度 3? 因为我们想让 TMA 多提前预取几个 K/V 块，把访存延迟藏起来；而 TMEM slot 的复用又各自被对应的 barrier phase 卡着。三套节奏彼此独立，没法用一个数字一概而论。

从「正确性闸门」切到「时间线视角」

原文这里给的是一张时间线示意图 (timeline)。它跟上一节的 barrier-flow 图回答的是两个不同问题:

- **barrier-flow 图:** 去这儿查「精确的生产者-消费者等待关系」(谁等谁)。
- **时间线图 (本节):** 去这儿看「大致同一时刻，哪些角色正活跃着」。

下面拿一张表，把一条「有代表性的流水线波次 (pipeline wave)」的时间线重画一遍: 行 = 角色 (对应代码里的一个 role branch), 列 = 大致的时间步 (从左往右推进), 格子 = 那一刻这个角色在干啥。

| 角色                           | t0 | t1     | t2 | t3      | t4           | t5     | t6        | ... |
|------------------------------|----|--------|----|---------|--------------|--------|-----------|-----|
| WG3<br>warp1<br>TMA<br>load  | Q0 | K[n-1] | Q1 | V[n-1]  | K[n-2]       | V[n-2] | K[n-3]    | ... |
| WG3<br>warp0<br>MMA          |    | S0     | S1 | PV0     | S0'          | PV1    | S1'       | ... |
| WG0/WG1<br>softmax           |    |        | P0 | P1      |              | P0'    | P1'       | ... |
| WG2<br>rescale               |    |        |    | release | rescale<br>O | ...    | normalize | ... |
| WG3<br>warp2<br>TMA<br>store |    |        |    |         |              |        | O 写回      | ... |

说明: $Sx$  = 第  $x$  个 Q stage 的 score MMA; $PVx$  = 第  $x$  个 Q stage 的 value MMA; 带撇号 (') 的是进了下一个 K/V 块之后的同名操作;MMA 这一行里,score 和 value 是交错着发的。你能看到 score、softmax、correction(rescale)、value 这几行在时间上是彼此重叠的 (同一列里好几个角色都在动), 而不是一个干完再轮下一个。

每一行对应的角色, 正好就是 kernel 里那几个分支:

- **WG3 warp 1:** 发起所有 TMA load(把 Q/K/V 从 GMEM 搬进 SMEM)。
- **WG3 warp 0:** 发起 score MMA 和 value MMA(两种 MMA 都归它管)。
- **WG0 和 WG1:** 为两个 Q stage 分别跑 softmax。
- **WG2:** 负责 0 的 release / rescale, 最后再做一次最终输出的归一化 (normalize)。
- **WG3 warp 2:** 发起 TMA store(把最终的 0 写回 GMEM)。

跟着一条波次从左往右走

把上面的时间线从左往右读一遍, 就是一次完整的流水线波次:

1. **load warp 先起步:** 先发 Q0、K[n-1]、Q1、V[n-1], 然后接着往下流式预取索引更低的 K/V 块 (K[n-2]、V[n-2]……)。留意一下,K/V 是从高索引往低索引走的。
2. **MMA warp 跟上:** 先发头几个 score MMA, 产出 S0 和 S1。
3. **WG0/WG1 接棒:** 把 S0、S1 经过 softmax 变成 P0、P1。

关键设计:MMA warp 必须把 score 和 value 交错发

这是本节最重要的一点。MMA warp 千万不能先把所有 score MMA 发完、再去发所有 value MMA。等两个 Q stage 都「热身 (primed)」好了, 它就开始交替发两种 MMA: 给当前 V 块发一个 value MMA, 紧接着给下一个 K 块发一个 score MMA, 就这么来回倒:

| 发射顺序 | 类型    | 运算            | 备注             |
|------|-------|---------------|----------------|
| 1    | score | $Q0 * K[n-1]$ |                |
| 2    | score | $Q1 * K[n-1]$ |                |
| 3    | value | $P0 * V[n-1]$ | 开始交错:value 进来了 |
| 4    | score | $Q0 * K[n-2]$ |                |
| 5    | value | $P1 * V[n-1]$ |                |
| 6    | score | $Q1 * K[n-2]$ |                |
| 7    | value | $P0 * V[n-2]$ |                |
| ...  | ...   | ...           |                |

**为什么非得这么交错?** 因为正是这种交错, 才让时间线里 score、softmax、correction、value 这四行叠到了一起, 而不是排成一队规规矩矩地一个接一个跑。要是先把全部 score 做完再做全部 value, Tensor Core 在两段之间就会空出一大段窗口, softmax 和 rescale 也没法跟 MMA 并行——流水线一下就退化成串行了。

WG2 那一行:release / rescale 的两种情形

时间线里 WG2 那一行写的是 release / rescale, 这两个词对应的就是我们之前见过的两种情况:

| 情形        | WG2 干什么                                   | 原因                               |
|-----------|-------------------------------------------|----------------------------------|
| 第一个 K/V 块 | 只参与 handoff, 让 value MMA 能往下走 (只 release) | 此时还没有「旧的 0」, 无需 rescale          |
| 后续 K/V 块  | 在 value MMA 累加进 0 之前, 可能先 rescale 旧的 0    | running max 变了, 旧 0 的比例失真, 需要先校正 |

而归一化 (normalization) 和 TMA store 只来这么一次——非得等整个 attention task 的最后一个 K/V 块处理完了, 才动手。

## 为什么 FA4 没法用「一条 GEMM 式流水线」来描述

收个尾, 回到本节的主旨: 没有哪条 GEMM 式的单一流水线能描述 FA4, 因为 Q、K/V、TMEM slot 各走各的独立时间表。这正是前面要拆出三个环形缓冲、三个深度的原因。

TIRx 的做法, 是把这些时间表明明白白地摊开: 每条流各用一组缓冲、各有一个 PipelineState 游标 (cursor, 即「现在转到这圈缓冲的第几格了」的指针)、各有自己的同步节拍, 而不是把整个 kernel 藏到一个庞大的「黑盒大函数」后面让你看不见内部。

### 注意

**注意:** 这是个明摆着的工程取舍。代价是「活动部件 (moving parts) 更多」——你得同时盯着好几个 ring、好几个游标、好几个 barrier; 但好处是复杂度始终摆在明面上、看得见也查得了 (visible and inspectable)。kernel 一出问题, 你能一条 tile 流一条 tile 流地去追, 而不是干瞪着一个黑盒。

## 重缩放与写回 (Rescaling and Writeback)

在线 softmax 的精髓就在「边算边修正」。每来一个新分数 tile, 每行的最大值  $m$  都可能被刷新。 $m$  一变, 之前加进 0 的那些结果就尴尬了——它们是用旧  $m$  算的, 整体偏大。这一小节讲两件事: 这一步非做不可的修正 (也就是重缩放), 还有 K/V 循环跑完之后, 怎么把最终结果归一化再写回显存。

### 为什么重缩放是「必须」而不是「优化」

不少人第一次看 Flash Attention, 会以为重缩放是个性能技巧, 图省事可以跳过。恰恰相反: 它是保证结果正确的硬性步骤, 跳不得。

顺一遍逻辑就明白了。在线 softmax 不是一口气看到所有分数的, 而是一块一块往上加。算每一块时, 用的是「当时已知的最大值  $m_{old}$ 」来做指数减法。等下一块带来个更大的  $m_{new}$ , 麻烦就来了: 之前加进 0 的每一项, 都比它该有的值大了一个倍数  $\exp(m_{new} - m_{old})$ 。不修正? 那先来的块就被「加权过头」了, 最终输出直接算错。

修正办法是把已经累加的 0 整体乘上一个缩小因子:

$$O_{old} \leftarrow O_{old} \cdot e^{(m_{old} - m_{new})/\sqrt{d}}$$

注意  $m_{old} - m_{new} \leq 0$ , 所以这个因子  $\leq 1$ , 刚好把之前偏大的部分缩回来。

### 注意

**注意:** 这里的  $m(\text{row\_max})$  存的是原始、没缩放过的  $QK^T$  分数的最大值, 所以指数项里得额外带上  $1/\sqrt{d}$  这个注意力缩放因子。这个约定后面写 LSE 时还会再用到一次。

## 这是一次完整的 TMEM → 寄存器 → TMEM tile 操作

还有一点也常被误解: 重缩放不是几个标量的小记账, 而是对整个 0 累加器 (accumulator) 来一次完整的「读出一乘一写回」。0 待在 TMEM 里, 得先用 `tcgen05.ld` 把整块 tile 读进寄存器, 在寄存器里逐元素乘上缩放因子, 再写回 TMEM。

这活儿拆给两个角色合作完成:

- **Softmax 角色:** 负责算出每行的缩放因子 `acc_scale`, 塞进 SMEM 「信箱 (mailbox)」里。
- **WG2(第二个 warpgroup):** 等 `softmax_corr.full` 信号, 从 TMEM 把当前的 `0` 读出来, 乘上缩放因子, 再写回 TMEM。

核心代码片段 (关键行加了中文注释):

```
RESCALE_TILE = T.meta_var(16) # 16 (, 0)
o_row = T.wg_reg_tile(RESCALE_TILE) #
Tx.copy_async(o_row, 0_region[i_q, d_start : d_start + RESCALE_TILE]) # 1 : 0 TMEM
↪
Tx.mul(o_row, o_row, acc_scale) # 2 :
Tx.copy_async(0_region[i_q, d_start : d_start + RESCALE_TILE], o_row) # 3 : TMEM
T.ptx.tcgen05.wait.st() # 4 : ,
```

逐行说人话: 这就是经典的「读出 → 修改 → 写回」三连。`Tx.copy_async( , )` 搬数据, 第一次是 TMEM→ 寄存器、第三次是寄存器 → TMEM(注意参数顺序是「目的地在前」)。`Tx.mul(o_row, o_row, acc_scale)` 把 `o_row` 整段乘上 `acc_scale`(那个折扣系数) 再存回 `o_row`。最后的 `wait.st()`(wait store) 是因为写回是异步的——不等它落地, 紧接着的取值 MMA 可能读到还没更新的旧 `0`, 结果就错了。整块 `0` 太大, 寄存器塞不下, 所以代码外层会用循环一次只搬 16 列 (`RESCALE_TILE`) 地分批做完。

Tile 原语读出卡: 修正 (rescale)

本书每个阶段都拿同一张「读出卡」来统一描述, 这一阶段的卡片是这样:

| 项目            | 内容                                                                                                                             |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|
| 作用域 (Scope)   | WG2, 整个 warpgroup                                                                                                              |
| 布局 (Layout)   | <code>0</code> 在 TMEM → 寄存器 → <code>0</code> 在 TMEM( <code>0_region[i_q]</code> )                                              |
| 派发 (Dispatch) | <code>tcgen05.ld</code> 读出, TMEM store 写回, 中间在寄存器里做乘法                                                                          |
| 交接 (Handoff)  | 等待 <code>softmax_corr.full</code> ; 完成后触发 <code>p_o_rescale</code> (→ value MMA) 和 <code>softmax_corr.empty</code> (→ softmax) |

端到端的同步流程

重缩放最微妙的地方, 就在于几个角色之间的握手 (handoff)。一整轮的同步, 可以画成下面这条流水线:

结构图

连接关系:

- SMEM 信箱  $\xrightarrow{\text{softmax\_corr.full (信箱已填好)}}$  WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM
- WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM  $\xrightarrow{\text{p\_o\_rescale (O 已就绪)}}$  WG3 的 value MMA: 消费 P, 累加进重缩放后的 `0` tile
- WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM  $\xrightarrow{\text{softmax\_corr.empty (信箱已读空)}}$  Softmax 可复用信箱, 进入下一轮

一步一步拆开看:

1. Softmax 把缩放因子写进 SMEM 信箱。

2. WG2 等 `softmax_corr.full`(确认信箱写好了)。
3. WG2 在 TMEM 里把 0 重缩放一遍。
4. WG2 在 `p_o_rescale` 上「到达 (arrive)」, 宣告 0 已经是新值了。
5. 到这时候, WG3 的 value MMA 才能安全地消费 P, 把结果累加进已经重缩放过的 0 tile。

整个循环靠 `softmax_corr.empty` 来闭合: WG2 读完信箱、把这个 SMEM 槽位释放掉, softmax 角色下一轮就能复用同一个信箱。

#### 注意

**注意:** 第 5 步那个顺序约束是关键——必须先把 0 重缩放, WG3 的 value MMA 才能往里加; 不然新一块结果就会被加到还没缩小的旧 0 上, 前面那番修正全白费。 `p_o_rescale` 这个 mbarrier, 干的就是强制这个先后次序的活。

### K/V 循环结束后: 从修正切换到收尾 (epilogue)

所有 K/V 块都处理完了, WG2 就从「修正」模式切到「收尾」模式, 把最终结果归一化、写回显存:

1. 等最终的 `row_sum` 和 `o_ready` 信号。
2. 从 TMEM 读出最终的 0。
3. 乘上  $1 / \text{row\_sum}$ ——这就是我们一开始故意往后拖的那次归一化 (除以分母这步留到最后统一做, 是在线 softmax 省算力的惯用招)。
4. 把结果转成 (cast)fp16。
5. 写进 `0_smem`。

最后, 由 WG3 的 TMA store warp 把 `0_smem` 通过 TMA 搬回 GMEM(全局显存)。

### 一个需要注意的限制: 只有前向输出, 没有 LSE

想拿这个 kernel 去扩展的人得留个心: 它只算前向输出 (forward output)。而训练场景下的前向传播, 一般还得存下反向用的对数和指数 (log-sum-exp, LSE)。

要补 LSE, 有个缩放细节绕不开。前面说过, 本 kernel 的约定是:

- `row_max` 存的是原始、没缩放过的  $\text{QK}^T$  分数的最大值;
- `row_sum` 累加的是  $\exp((S - \text{row\_max}) / \sqrt{d})$ 。

因为 `row_max` 身上没带  $1/\sqrt{d}$ , 所以你在拼自然对数形式的 LSE 时, 得把这个缩放因子重新补回 `row_max`:

$$\text{LSE}_i = \log(\text{row\_sum}_i) + \text{row\_max}_i / \sqrt{d}$$

#### 注意

**注意:** 这个实现「只出前向输出」, 不写 LSE。要是想支持训练的反向传播, 记得按上面的式子把  $1/\sqrt{d}$  补回去, 否则 LSE 的数值会差一个常数因子。

因果掩码 (Causal Masking)

先说因果注意力 (causal attention) 是干嘛的: 语言模型生成文字时, 预测下一个词只能依据前面已经出现的词, 不能偷看后面的词 (否则就作弊了)。落到注意力上就是一条硬规矩: 一个 query (可以理解成「当前这个位置」) 只能看「自己这个位置或更早位置」的 key。换句话说, 对第 i 个 query 来说, 它在分数矩阵里合法的列只到第 i 列, 后面的 key 全是「未来」, 必须当作不存在。

FA4 处理这条规矩的办法很利落: 已经成型的那条数据通路它一概不碰——两次矩阵乘法加中间的 softmax, 整套流程原封不动。因果模式只在两个地方动手脚, 一个图「省」, 一个图「准」:

- 省: 那些纯属未来、对当前 Q 块一点贡献都没有的 K/V 块, 整块整块直接跳过;
- 准: 对横跨对角线的那一两个块, 逐行把越界的列精确屏蔽掉。

下面这两手分别看。

先看「省」: 整块跳过对角线上方的 K/V 块

把分数矩阵想成一张二维网格: 行是 query, 列是 key, 对角线就是「query == key 位置」那条线。因果约束下, 对角线右上方 (列号大于行号) 整片区域都非法。

按块一划分, 很多 K/V 块会整块落在某个 Q 块所需范围的右上方——里面每一列都是未来, 对这个 Q 块的输出贡献是零。既然如此, 根本没必要把它们从 GMEM 搬进来, 更别说跑 MMA 和 softmax 了。

实现里用 `get_n_block_max(...)` 算出「当前 Q 块顶多用得到的最后一个 K/V 块」, 然后 K/V 循环对超出这个上界的块, 就干脆不加载、也不计算了。

```
Q , K/V
n_block_max = get_n_block_max(...)
for n_block in range(n_block_max): # , " "
 ... # : / / softmax /
```

这段没什么 GPU 玄机, 就是个普通的 for 循环——关键在于上界 `n_block_max` 被 `get_n_block_max` 提前算小了, 于是那些纯属「未来」的块连循环都进不去, 自然就不会被加载、不会被计算。省工省在「根本不做」。

注意

注意: 这一手纯粹是「省工」——被跳过的块, 连数据都不搬。因为上三角 (对角线右上方) 整块都被跳掉, 平均下来因果 kernel 只需算全注意力 (full attention) 大约一半的 K/V 块 (越靠后的 Q 块要遍历的 K/V 越多, 越靠前的越少); 这正是它比全注意力快的主要原因。

下图直观看一下哪些块被跳过、哪些块要逐列精修 (拿一个靠中间的 Q 块举例):  
以一个靠中间的 Q 块为例, 它对各 key 块的处理 (横轴是 key 块编号):

| key 块 | 0   | 1   | 2           | 3    | 4    | 5    |
|-------|-----|-----|-------------|------|------|------|
| 处理方式  |     |     |             |      |      |      |
| 含义    | 全保留 | 全保留 | 跨对角线 (逐列掩码) | 整块跳过 | 整块跳过 | 整块跳过 |

- 完全在对角线下方: 每一列都合法, 无需掩码
- 横跨对角线: 跑 MMA, 但 softmax 前要按行精确掩码
- 完全在对角线上方: 全是未来, `get_n_block_max` 让循环根本不会碰它



再看「准」：对横跨对角线的块逐行掩码

`get_n_block_max` 只能整块整块地取舍，可总有那么正好压在对角线上的块：这种块里一部分列合法、一部分列非法，既不能整块跳，也不能整块留。

对这类块的处理是：`score MMA` 照常跑、把分数算出来，但在求指数之前，先在 `softmax` 里把非法列屏蔽掉。具体是逐行干的：

- 1. 拿这一行的 `query` 位置，加上当前块的列偏移 (`block offset`)，推出这一行的列上限；
- 2. 列号在上限以内的，保留；
- 3. 列号超过上限的，在寄存器里直接置 `-inf`。

为什么置 `-inf` 而不是置 0？因为这一步是在 `exp2` 之前做的：

- $\text{exp2}(-\text{inf}) = 0$ ，这些非法列对 `softmax` 分子 (`P`) 的贡献自然就是零；
- 同时 `-inf` 也不会把这一行的 `row_max`(行最大值) 带歪，因为它永远当不上最大值。

这么一来，不管是行最大值，还是 `exp2` 分子的累加，非法列都被干干净净地剔掉了，后续运算压根不带它玩。

注意

注意：回顾一下前面——`row_max` 维护的是原始 (未缩放) $\text{QK}^T$  分数的逐行最大值，`P` 是  $\text{exp2}((S - \text{row\_max}) \cdot \text{scale\_log2})$ 。把非法列设成 `-inf`，正好让它既抬不高 `row_max`，又在 `exp2` 之后归零，一箭双雕。

实现技巧：用位掩码一次性处理，而不是逐元素分支

要是对每个元素都写一句 `if > then -inf` 的分支，在 GPU 上会慢得离谱——还记得前面说的吗，一个 `warp` 里 32 个线程必须齐步走，各走各的 `if` 分支 (分支发散) 会逼硬件一拨拨轮流跑，效率大跌。FA4 的做法是把「列上限」算成一个罩住整段 32 宽分数块的位掩码 (`bit mask`，即用 32 个 0/1 位表示「这些列留、那些列屏蔽」)，然后一把就把这段里所有越界列一起屏蔽掉，全程没有分支：

```
32 列上限，算成 32 位的掩码
mask_r2p(score_chunk, col_limit)
```

这就把「一个元素一个元素地判断」变成了「整段按位操作」，发散分支也就躲掉了。至于那些完全落在对角线下方的块 (每一列都合法)，连掩码都省了，直接照原样进 `softmax`。

下表把三类块各自怎么处理归纳一下：

| 块相对对角线的位置 | 是否加载/计算 | softmax 掩码                      | 说明                                        |
|-----------|---------|---------------------------------|-------------------------------------------|
| 完全在下方     | 是       | 不需要                             | 每列都合法，等同全注意力                              |
| 横跨对角线     | 是       | 需要，用 <code>mask_r2p</code> 按位屏蔽 | 部分列合法，逐行求列上限                              |
| 完全在上方     | 否       | ——                              | 由 <code>get_n_block_max</code> 直接跳过，数据都不搬 |

从 `tile` 原语视角看：因果模式只是「微调」，不是「重写」

站在 `tile` 级原语 (`tile primitive`) 的视角回头看，因果模式的改动相当克制：

- 它一条数据通路都没碰——score MMA 到 softmax、softmax 到 value MMA 的衔接全照旧；
- 它就干了两件事：**(1)** 用 `get_n_block_max` 把 K/V 循环的遍历次数 (trip count) 砍短了；**(2)** 在寄存器里的 softmax 流程中、夹在 score MMA 和 P 写回 (writeback) 之间，插了一步掩码操作。

结构图

分组：

- **全注意力**：「score MMA」「softmax (exp2 + 重缩放)」「P 写回」
- **因果注意力**：「score MMA」「掩码 mask\_r2p」「softmax」「P 写回」

连接关系：

- 因果注意力 → 同时 `get_n_block_max` 缩短整个 K/V 循环的遍历次数

注意

**注意：**这正是 FA4「tile 原语 + barrier」骨架的一个好处——像因果掩码这种功能变体，可以只往某一步原语内部塞逻辑 (这里是 softmax)，其余阶段和同步结构一概不用动。数据流、barrier 图都保持原样，改动被牢牢圈在局部。

GQA 支持 (GQA Support)

为什么需要 GQA

先补一点背景：标准的多头注意力里，Q、K、V 各有同样多的「头 (head)」，一一配对。「分组查询注意力 (Grouped Query Attention, GQA)」则让**多个查询头 (query head) 共用同一个 K/V 头**。动机很直接：大模型一个字一个字往外蹦 (自回归推理 / decoding) 的时候，真正拖慢速度的瓶颈是反复从显存里读 KV cache (缓存下来的历史 K、V)，而 KV cache 的大小跟 K/V 头的数量成正比。要是让每 4 个 Q 头共用 1 个 K/V 头，KV cache 直接缩到 1/4，读显存的压力跟着大降——这就是 GQA 流行的原因。

可这种共用给 kernel 带来一个「打包 (packing)」难题：既然我们对某个 K/V 头只想加载、只想留一个 K/V tile，那怎么让**好几个不同的 Q 头都从这同一个 tile 里取数算**? Flash Attention 4 的答案是——把**共用同一个 K/V 头的那一整组 Q 头**，一次性凑在一起处理，针对调度器派下来的单个 `kv_head_idx` 把活儿算完。

核心思路：重新解释 Q tile 的 128 行

先定义两个关键比例：

```
GQA_RATIO = num_qo_heads // num_kv_heads # K/V Q ()
SEQ_Q_PER_TILE = BLK_M // GQA_RATIO # Q tile
```

精髓就在于给 Q tile 的 128 行 (`BLK_M=128`) 换一种读法。标准的多头注意力里，这 128 行就是 128 个序列位置 (sequence position)。可在 GQA 下，拿 `GQA_RATIO=4` 来说，它们就不再是 128 个序列位置了，而是 **32 个序列位置 × 4 个查询头**，交错打包 (packed) 进同一块 tile，这样这 4 个 Q 头就一起骑在同一个 K/V tile 上了。

每一行怎么解码回「序列位置 + 头编号」：

```
seq_pos = row // GQA_RATIO # ->
q_head = row % GQA_RATIO # ->
```

下面用 GQA\_RATIO=4 直观看一下这 128 行是怎么排布的：  
GQA\_RATIO=4 时这 128 行的排布 (每个 seq 位置占连续 4 行，放它的 4 个 Q 头)：

| 行号<br>row | 0                      | 1 | 2 | 3 | 4                      | 5 | 6 | 7 | ... | 124                     | 125 | 126 | 127 |
|-----------|------------------------|---|---|---|------------------------|---|---|---|-----|-------------------------|-----|-----|-----|
| seq_pos0  |                        | 0 | 0 | 0 | 1                      | 1 | 1 | 1 | ... | 31                      | 31  | 31  | 31  |
| q_head0   |                        | 1 | 2 | 3 | 0                      | 1 | 2 | 3 | ... | 0                       | 1   | 2   | 3   |
| 分组        | \<- seq 0 的 4 个 Q 头 -> |   |   |   | \<- seq 1 的 4 个 Q 头 -> |   |   |   | ... | \<- seq 31 的 4 个 Q 头 -> |     |     |     |

注意

注意: 这种「序列位置在外、头在内」的交错排法, 让 MMA 看到的依旧是一块连续的 128 × HEAD\_DIM 操作数。计算单元根本不用知道 GQA 这回事。

Q 加载: 用 3D 视图无拷贝地完成打包

难点在于: 内存里的 Q 是自然布局 Q[batch, seq, qo\_head, dim], 可 MMA 后面要读的却是一块扁平的 128 × HEAD\_DIM SMEM tile。怎么把前者「塞进」后者, 又一点额外的数据搬运都不引入?

答案是给同一块 SMEM 套一个 **多维视图 (view)**——前面讲过, 视图就是「不搬数据、只换个角度解读同一段内存」(类似 NumPy 的 reshape: 数据没动, 只是改了怎么按下标去读它):

```
Q SMEM tile 4 :(, , , head_dim)
Q_smem_3d = Q_smem.view(SMEM_PIPE_DEPTH_Q, SEQ_Q_PER_TILE, GQA_RATIO, HEAD_DIM)

Tx.copy_async(
 Q_smem_3d[i_q, :, :, :], # : 3D
 Q[batch_idx,
 m_start : m_start + SEQ_Q_PER_TILE, # : SEQ_Q_PER_TILE
 kv_head_idx * GQA_RATIO : # : K/V
 (kv_head_idx + 1) * GQA_RATIO, # GQA_RATIO Q
 :],
 **tma_copy_q, # TMA
)
```

几个关键点:

- 目标侧的 (SEQ\_Q\_PER\_TILE, GQA\_RATIO) 这两维, 正好跟上面 seq\_pos / q\_head 的解码规则一一对上, 把 32×4 的逻辑结构原样映射进 128 行。

- 源侧在头维度上用  $\text{kv\_head\_idx} * \text{GQA\_RATIO} : (\text{kv\_head\_idx}+1) * \text{GQA\_RATIO}$ , 精准切出「归这个 K/V 头管的那一组 Q 头」。
- 整件事靠 TMA(Tensor Memory Accelerator) 的异步拷贝完成, 视图负责把两种布局对齐, 拷贝本身一气呵成。

K/V 不扩展 + O 存储对称还原

GQA 真正省钱的地方在于:K 和 V 在显存里压根不被复制扩展。 $\text{kv\_head\_idx}$  对应的那唯一一块 K/V tile, 被打包进 Q 行里的全部  $\text{GQA\_RATIO}$  个查询头一起复用。这正是 GQA 省带宽的本质——少读 K/V, 而不是把 K/V 复制成好几份去将就 Q。

输出侧跟输入侧完全对称:epilogue 之后, 同样用一个对应的 3D 视图, 把打包在一起的各行结果解包还原, 写回自然布局  $\text{O}[\text{batch}, \text{seq}, \text{qo\_head}, \text{dim}]$ 。

设计上的优雅:GQA 只活在边界上

这个方案最让人服气的一点, 是关注点切得干干净净:

| 位置                   | 是否感知 GQA    | 看到的数据形状                                                                                                                                                |
|----------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 加载边界               | 是 (3D 视图打包) | $\text{Q}[\text{batch}, \text{seq}, \text{qo\_head}, \text{dim}]$<br>$\leftrightarrow (\text{SEQ\_Q\_PER\_TILE}, \text{GQA\_RATIO}, \text{HEAD\_DIM})$ |
| 内部计算路径 (score MMA 等) | 否           | 普通的 $128 \times \text{HEAD\_DIM}$ Q tile                                                                                                               |
| O 存储边界               | 是 (3D 视图解包) | $(\text{SEQ\_Q\_PER\_TILE}, \text{GQA\_RATIO}, \text{HEAD\_DIM}) \leftrightarrow \text{O}[\text{batch}, \text{seq}, \text{qo\_head}, \text{dim}]$      |

也就是说,GQA 全被摁在 Q 加载和 O 存储这两个边界上解决掉了。一进计算路径,score MMA 看到的还是一块平平无奇的  $128 \times \text{HEAD\_DIM}$  Q tile, 后面整张 tile-primitive 计算图(softmax、PV MMA、流水线、mbarrier 同步等等) 一行都不用改。

拿一张流程图把这种「夹心」结构概括一下:

结构图

连接关系:

- $\text{Q SMEM tile} / 128 \times \text{HEAD\_DIM} / (32 \text{ seq} \times 4 \text{ head 打包}) \rightarrow \text{计算路径} / \text{score MMA} / \text{softmax} / \text{PV MMA} / (\text{完全不感知 GQA})$
- $\text{计算路径} / \text{score MMA} / \text{softmax} / \text{PV MMA} / (\text{完全不感知 GQA}) \xrightarrow{\text{3D 视图解包}} \text{O 显存} / [\text{batch}, \text{seq}, \text{qo\_head}, \text{dim}]$

注意

**注意:** 这种「边界上打包、核心里透明」的套路, 很值得搬到别的场景去——凡是只想在数据进出时改排布、又希望计算核心保持通用的需求, 都能靠零拷贝视图 (view) 把转换逻辑摁在加载/存储这两端。

Tile 调度 (Tile Scheduling)

调度器 (scheduler) 的核心职责就一件事: 把每个 CTA(还记得吗,CTA 就是一大群一起调度的线程, 这里一个 CTA 负责算一块输出) 派到一个具体的注意力任务上。这个任务用三元组 (batch, kv\_head, m\_block) 描述——也就是「第几个 batch(批次里的第几条样本)、第几个 KV 头、第几个 Q 块」。换句话说, 调度器只管「这个 CTA 负责算哪一块」, 至于这块怎么算, 它不操心。这跟把一堆任务分给线程池里的工人, 是同一类问题。

为什么要专门搞个调度器? 因为在 GPU 上, 所有 CTA 大致是一起启动、一起收尾的; 一个 kernel 的总耗时, 取决于**最后一个干完的那个 CTA**(木桶效应)。要是任务分得不匀, 有些 CTA 早早算完干等着、另一些还在埋头苦算, 整体就被那个最慢的拖下来了。要不要费心调度, 关键就看一件事: **这些任务的计算量是不是一样大**。而这一点, 又是掩码模式说了算。

两种调度策略对应两种掩码模式

Flash Attention 4 对「非因果 (non-causal)」和「因果 (causal)」这两种模式, 用了两套不同的调度器。

非因果模式:FlashAttentionLinearScheduler(线性调度器)

非因果注意力里, 每个 Q 块都得对**全部** K/V 块算一遍注意力。也就是说, 每个 tile 任务的活儿一样多、整整齐齐。负载这么均匀, 就用不着什么花活儿: 维护一个固定大小的 CTA 池, 每个 CTA 干完手头的任务, 就把任务索引**整体往前跳 num\_ctas 个** (也就是 GPU 上 CTA 的总数), 这样所有任务就被均匀铺给了所有 CTA。这是典型的「跨步划分 (strided partition)」——简单、没分支、负载天然就平。

因果模式:FlashAttentionLPTSchedular(最长处理时间优先调度器)

因果掩码把「均匀」这个前提彻底打破了, 各任务的活儿一下子变得极不平衡:

- 序列开头附近的 Q 块, 只能「看到」自己前面那一点 token, 所以大概只要处理 **一个** K/V 块;
- 序列末尾附近的 Q 块, 前面所有 token 都「看得到」, 所以要处理 **全部** K/V 块。

因果掩码下各 Q 块的工作量 (# = 需要计算的一个 K/V 块):

| Q block | 需要计算的 K/V 块 | 工作量 |
|---------|-------------|-----|
| 0(开头)   | #           | 最小  |
| 1       | ##          |     |
| 2       | ###         |     |
| ...     | ...         |     |
| N(末尾)   | #####       | 最大  |

这时候要还像非因果那样「傻乎乎地」平均切任务, 就会冒出严重的长尾: 分到「重」块的 CTA 会比分到「轻」块的 CTA 晚收工老半天,GPU 利用率被尾巴上那几个任务拖垮。

所以因果模式改用 **LPT(Longest-Processing-Time, 最长处理时间优先)** 调度。它的核心思想来自经典的多机调度算法: **先把最重的任务安排上**。把活儿大的 Q 块「前置 (front-load)」分配, 让重活先开工, 这样各 CTA 的完成时间 (finish time) 就被拉平了, 长尾也削掉了。

注意

注意:LPT 一边追求负载均衡, 一边还会尽量把相邻的 batch / head 任务凑在一起调度, 好保住 **L2 cache 局部性 (locality)**。也就是说, 它不是死板地按工作量贪心, 而是在「均衡」和「数据复用」之间找平衡——既要让重活先跑, 又不想老是切到八竿子打不着的数据上, 白白

糟蹋已经缓存好的 K/V。

两类策略对比

| 维度             | 非因果 (Linear)                  | 因果 (LPT)                   |
|----------------|-------------------------------|----------------------------|
| 调度器            | FlashAttentionLinearScheduler | FlashAttentionLPTScheduler |
| 各 tile 工作量     | 完全相等 (每个 Q 块都遍历全部 K/V)        | 高度不均 (开头轻、末尾重)             |
| 划分方式           | 固定 CTA 池, 步长为 num_ctas 跨步推进   | 最长任务优先、重块前置                |
| 优化目标           | 均匀铺满                          | 拉平完成时间 + 兼顾 L2 局部性         |
| next_tile() 行为 | 跳到下一个任务, 继续循环                 | 当前任务结束后, 退出循环              |

统一的循环接口

两个调度器内部策略差得远, 可它们对外露出来的循环接口一模一样。所以 kernel 主体代码根本不用管自己用的是哪种调度器:

```
while scheduler.valid():
 m_block_idx = scheduler.m_block_idx
 batch_idx = scheduler.batch_idx
 kv_head_idx = scheduler.head_idx
 # Q, K/V
 scheduler.next_tile()
 # tile()
```

两者唯一的行为差别, 全藏在 next\_tile() 里:

- 非因果模式:next\_tile() 让当前 CTA 跳到下一个任务, 循环接着转 (一个 CTA 这辈子可能要处理好几个 tile);
- 因果模式:next\_tile() 干完当前任务后直接把循环结束掉 (因为 LPT 早在 kernel 启动前就把每个 CTA 该干哪些活都排好了)。

结构图

连接关系:

- 读取 m\_block / batch / kv\_head 索引 → 计算这一个 tile / (TMA load → score MMA → softmax / → value MMA → correction → TMA store)
- 计算这一个 tile / (TMA load → score MMA → softmax / → value MMA → correction → TMA store) → scheduler.next\_tile()
- scheduler.next\_tile()  $\xrightarrow{\text{非因果: 跳到下一任务}}$  scheduler.valid()?
- scheduler.next\_tile()  $\xrightarrow{\text{因果: 结束}}$  循环退出
- scheduler.valid()?  $\xrightarrow{\text{否}}$  循环退出

调度只决定「算哪一块」，不决定「怎么算」

这一点是整个调度设计的精髓：不管用哪种调度器，它干的都**纯粹是调度决策**——只挑「这个 CTA 摊上哪一块注意力 tile」，至于这块 tile 内部怎么算，它**从不插手**。

循环体里跑的，始终是同一套本地原语 (local primitives)，跟调度策略半毛钱关系没有：

结构图

把「调度」和「计算」这么彻底地拆开，带来两个好处：一是 kernel 主体逻辑能保持单一、稳定，不用为不同掩码模式写两套计算路径；二是以后想加新的调度策略（比如针对别的稀疏掩码模式），只要再写一个实现相同循环接口的调度器就行，计算内核一点不用动。

编译并验证 (Compile and Verify)

前面所有小节给的都是「片段」——一块一块拆开来讲原理。这一节反过来：把整章拆出来的零件重新拼成一个能跑的整体，真刀真枪编译它、扔到 GPU 上跑，再拿一个「已知正确」的参考实现（这里用 PyTorch 自带的注意力）核对结果对不对。思路跟写单元测试一模一样：**自己实现的结果 vs 信得过的标准答案，看两者是否一致**。说白了，这是整章唯一一个「跑一下就能把全部内容验收掉」的代码单元格 (cell, 指 Jupyter Notebook 里一格可独立运行的代码)。

完整 kernel 的实现书里没有逐行铺开，而是收在 `tirx-kernels` 仓库的 `flash_attention4.py` 这一个文件里——本章讲过的每一块 (TMA 加载、两阶段 MMA、在线 softmax、rescale 与回写、causal 掩码、GQA、tile 调度) 都在那儿组装成了一份能直接 import 的源码。

与 GEMM 验证单元格的两点差异

要是你跑过上一章 GEMM 的验证单元格，会发现 Flash Attention 的入口几乎一个样，就两处不同：

| 维度   | GEMM 验证     | Flash Attention 4 验证                                                         |
|------|-------------|------------------------------------------------------------------------------|
| 入口函数 | 直接拿到 kernel | 更丰富的工厂函数 <code>get_flash_attention4_kernel</code> (需要传形状、GQA 头数、是否 causal 等) |
| 额外参数 | 无           | 多一个 <code>profiler_buf</code> ，供 kernel 内置的 profiler 写计时数据                   |

第一点，是因为 attention 的参数空间比 GEMM 大太多了 (序列长度、query/KV 头数、head dim、要不要 causal)，所以得有个工厂函数照着这些参数「现做」一份具体的 kernel。第二点，是因为这份 kernel 自带 profiler，跑的时候需要一块设备上的 buffer 来落计时点——所以调用时比 GEMM 多传一个张量。

跑通整章的那一个单元格

下面就是「一键验收全章」的代码。它干了四件事：备输入、编译、跑、核对。

```
import torch
import torch.nn.functional as F
import tvm
from tirx_kernels.attention.flash_attention4 import (
```

```

get_flash_attention4_kernel, PROFILER_BUFFER_SIZE)

GQA :32 query 8 KV
B, S, Hq, Hkv, D = 1, 1024, 32, 8, 128
Q = torch.randn(B, S, Hq, D, dtype=torch.float16, device="cuda")
K = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
V = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
O = torch.empty(B, S, Hq, D, dtype=torch.float16, device="cuda")
profiler buffer,
prof = torch.zeros(PROFILER_BUFFER_SIZE, dtype=torch.uint64, device="cuda")

#
kernel = get_flash_attention4_kernel(B, S, S, Hq, Hkv, D, is_causal=False)
target = tvn.target.Target("cuda")
with target:
 # tirtx TIR
 ex = tvn.compile(tvn.IRModule({"main": kernel}), target=target, tir_pipeline="tirtx")

ex.mod torch , ; prof
ex.mod(Q, K, V, O, prof)
torch.cuda.synchronize()

torch ;enable_gqa 32 query 8 KV
qt, kt, vt = (x.transpose(1, 2).float() for x in (Q, K, V))
ref = F.scaled_dot_product_attention(qt, kt, vt, enable_gqa=True).transpose(1, 2).half()
torch.testing.assert_close(O, ref, rtol=1e-2, atol=1e-2)
print(f"FA4: B={B} S={S} Hq={Hq} Hkv={Hkv} D={D}, non-causal -> PASS")

```

几个细节值得留意:

- **布局转换:** Q/K/V 在内核里用的是 (B, S, H, D), 而 PyTorch 的 `scaled_dot_product_attention` 要的是 (B, H, S, D), 所以参考实现先 `transpose(1, 2)` 把 head 维挪到前头, 算完再换回来。
- **GQA 对齐:** `enable_gqa=True` 让 PyTorch 用上跟我们 kernel 一样的「32 个 query 头共享 8 个 KV 头」语义, 不然两边算的压根不是一回事。
- **精度对齐:** 参考实现先 `.float()` 升到 fp32 再算, 最后 `.half()` 落回 fp16, 这样才跟 kernel 的输出 dtype 对得上。

**期望输出:** 最后一行打印 ... -> PASS。

## 为什么用 `rtol/atol` 而不是逐位相等

你可能会问: 都是算注意力, 为啥不要求结果分毫不差, 反倒给个 `rtol=1e-2`, `atol=1e-2` (相对误差、绝对误差容许到 1% 这个量级) 这么「松」的容差? 原因是: 浮点计算只要换个顺序、换个精度, 结果就会有微小出入 (这对所有数值程序都成立, 不是 bug)。就算 kernel 把在线 softmax 的累加放在 fp32 里做, 它跟「高精度参考实现」之间也还隔着好几层互相独立的近似来源。一项项拆开看:

### 结构图

连接关系:

- 高精度参考实现 → ① fp16 存储: 输入与中间操作数被存成 fp16, 带来舍入误差
- ① fp16 存储: 输入与中间操作数被存成 fp16, 带来舍入误差 → ② exp2 重写: softmax 用 exp2 改写 / ( $\text{scale\_log2} = \log_2(e)/\sqrt{d}$ ), 把每个指数都换算到 2 的幂上)



- ② `exp2` 重写:softmax 用 `exp2` 改写 / (`scale_log2 = log2(e)/√d`, 把每个指数都换算到 2 的幂上) → ③ 在线 softmax 重排: 按块累加并逐行 `rescale`, / 而不是一次性把所有块加在一起
- ③ 在线 softmax 重排: 按块累加并逐行 `rescale`, / 而不是一次性把所有块加在一起 → ④ 回写 `fp16`: 最终 O 在写回时被 `cast` 成 `fp16`
- ④ 回写 `fp16`: 最终 O 在写回时被 `cast` 成 `fp16` → kernel 实际输出

注意

**注意:** 这里挑的 `rtol/atol` 跟源 kernel 自带测试用的容差是一样的。它是冲着「上面四种误差叠加之后相对参考实现的总偏差」来设的, 不是只盯 `fp16` 舍入这一项。换句话说, 它得把 ①~④ 全罩住, 而不是只罩其中某一层。

如果真的 FAIL 了, 去哪里找问题

正因为这个容差是「按总误差量身定的」, 所以你要是看到的不是「擦边差一点」的临界值, 而是一次实打实的失败, 那它差不多就是个明确的路标: 问题八成出在 softmax 通路上, 而不是浮点精度本身。具体说, 得回头查本章那些用 `barrier` 串起来的「交接 (handoff)」:

| 可疑现象                                                            | 对应被破坏的交接点                             |
|-----------------------------------------------------------------|---------------------------------------|
| 漏等 <code>s_ready</code>                                         | <code>scores</code> 还没算好就进了 softmax   |
| 漏等 <code>p_o_rescale</code>                                     | <code>rescale</code> 与第二个 MMA 之间的同步丢失 |
| 漏等 <code>p_ready_2</code>                                       | 概率矩阵 P 还没就绪就被第二阶段 MMA 消费              |
| <code>row_max / row_sum</code> 更新后, <code>rescale</code> 步骤没有应用 | 在线 softmax 的运行缩放 (running scale) 没跟上  |

这些恰好就是本章一路用 `mbarrier` 小心护着的那几个 handoff。一次真正的数值失败, 说白了就是在告诉你: 某对 producer/consumer 之间的等待被漏了, 或者某次 `row_max/row_sum` 的更新没被 `rescale` 好好吸收掉。把验证失败当成「指向 softmax 路径的诊断信号」来读, 往往比死盯着浮点数据本身更快摸到根因。

与 GEMM 的区别 (Differences from GEMM)

前面几节我们已经看明白, Flash Attention 4 (简称 FA4) 在结构上其实是「站在矩阵乘 (GEMM) 肩膀上」的: 它复用的还是那同一套搬数据、做矩阵乘、再写回的硬件零件。那问题来了——底层硬件能耐没变, FA4 写起来怎么就比一个高性能矩阵乘难这么多?

本节的核心观点很简单: **FA4 难, 不是因为它换了硬件, 而是因为它要照看的数据块更多, 这些数据块之间的「交接班」也更多。** 所有复杂度, 最后都能追到本章开头点明的那个结构性变化上——多了第二个矩阵乘, 而且 softmax 还被夹在两个矩阵乘中间。

逐项对比: GEMM 与 FA4 在哪些轴上发生了变化

下面这张表把 FA4 相对 GEMM 变了的几个维度一条条列出来。看的时候建议带个视角: 每一行的差异, 归根到底都是「第二个 MMA + 中间那段 softmax」这一件事派生出来的。

| 对比维度                   | GEMM                                 | Flash Attention 4                                            |
|------------------------|--------------------------------------|--------------------------------------------------------------|
| MMA 阶段                 | 同一个 MMA 反复执行                         | 分成两段: 打分 MMA(score MMA) 和取值 MMA(value MMA)                   |
| MMA 之间的工作              | 除了流水线交接外没有别的事                        | 在线 softmax(online softmax)、掩码 (masking)、对 O 的重缩放 (rescaling) |
| 运行时状态                  | 只有累加器 (accumulator)                  | 行最大值 (row max)、行求和 (row sum)、O 累加器                           |
| 主要中间结果                 | 累加器所在的 TMEM tile                     | S、P、O 三块 TMEM tile 区域                                        |
| warp 角色分工              | TMA 生产者、MMA 消费者、写回 (writeback)       | TMA load、MMA、softmax、correction(校正)、TMA store                |
| 屏障 (barrier)           | 主要是 load / compute / writeback 之间的交接 | 额外多出 score / softmax / value / correction 等多组交接              |
| 调度单元 (scheduling unit) | 一个输出矩阵 tile                          | 一个注意力任务:(batch, kv_head, m_block)                            |

为什么会有这些差异: 从「两个 MMA」一路推下去

把上表每一行串起来看, 你会发现它们不是一堆零散的设计选择, 而是一条因果链:

结构图

连接关系:

- 起因: 多了第二个 MMA, 中间夹了 softmax → MMA 拆成两段 / (score / value)
- 起因: 多了第二个 MMA, 中间夹了 softmax → 两段之间要做 / softmax / 掩码 / 重缩放
- 起因: 多了第二个 MMA, 中间夹了 softmax → 需要维护跨块的 running 状态 / (max / sum / O)
- MMA 拆成两段 / (score / value) → 需要更多中间 tile / (S / P / O 三块)
- 两段之间要做 / softmax / 掩码 / 重缩放 → 需要新增 warp 角色 / (softmax / correction)
- 需要维护跨块的 running 状态 / (max / sum / O) → 需要更多 barrier / 把这些角色串起来
- 需要更多中间 tile / (S / P / O 三块) → 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」
- 需要新增 warp 角色 / (softmax / correction) → 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」
- 需要更多 barrier / 把这些角色串起来 → 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」

- **MMA 从一段变两段:**GEMM 全程就是把同一个  $C += A @ B$  重复跑很多遍;FA4 则是先算  $S = Q @ K^T$ (打分 MMA), 再算  $O += P @ V$ (取值 MMA)。两段 MMA 的输入输出都不一样, 自然就得上两套节奏。
- **两段 MMA 之间不再是「空档」:**GEMM 里, 两次 MMA 之间除了流水线交接啥也不干。FA4 偏要往这个夹缝里塞 online softmax(在线、分块地求 softmax)、给被掩盖的位置打掩码、行最大值更新后还要对已累加的 O 重缩放。这些可都是「真活儿」, 不只是搬数据。
- **运行时状态变多:**GEMM 只要维护一个累加器。FA4 因为 softmax 要在分块流式处理里保持数值稳定, 就得额外带上每行的 running max、running sum, 再加 O 累加器——三份状态都得跨

KV 分块一路更新。

- **中间结果从一块变三块:** 相应地,TMEM 里得给 S(打分结果)、P(softmax 之后的概率)、O(输出累加) 各留一块 tile 区域,而不是 GEMM 那一块累加器 tile。
- **warp 角色更细:**GEMM 大致就「TMA 生产者 / MMA 消费者 / 写回」三类角色。FA4 把分工拆得更碎——TMA load、MMA、softmax、correction、TMA store, 各管一摊, 这样上面那些额外的计算环节才能流水线化地并行起来。
- **屏障更多:** 角色多了, 交接点跟着就多了。除了 GEMM 里常见的 load/compute/writeback 交接,FA4 还得多加 score、softmax、value、correction 等好几组 mbarrier, 把「谁算完了、下一个角色才能消费」管得死死的。
- **调度单元升级:**GEMM 的调度单元就是一个输出矩阵 tile;FA4 的调度单元升成了一个完整的「注意力任务」, 用三元组 (batch, kv\_head, m\_block) 来标识——也就是「哪个 batch、哪个 KV 头、哪个 M 方向的块」。

底层契约其实一点没变

有一点要专门强调: 上面列了这么一大堆差异, 可 **FA4 靠的那套底层「描述规矩」(契约,contract), 从头到尾一点没变。** 也就是说, 描述这套 kernel 的「四件套」问题, 在矩阵乘和 FA4 里完全一样——这也正是本章反复用的那张读卡:

| 契约要素                     | 它回答的问题           | 大白话      |
|--------------------------|------------------|----------|
| tile 原语 (tile primitive) | 「搬运 / 计算的是哪一块数据」 | 这一步在干什么活 |
| 作用域 (scope)              | 「哪些线程在协作完成它」     | 谁来干      |
| 布局 (layout)              | 「这块数据存在哪里」       | 数据放哪儿    |
| 屏障 (barrier)             | 「下一个角色什么时候可以消费它」 | 什么时候能交班  |

注意

**注意:** 看懂这一点对学 FA4 太重要了——你不用去啃一套全新的硬件抽象。FA4 用的还是同样的 tile 原语、同样的 scope/layout/barrier 语义。区别只在「数量」上:tile 数值更多、角色之间的交接更多。一句话,FA4 是 GEMM 在「广度」上的铺开, 不是在「深度」上另起炉灶。

所以, 你要是已经把 GEMM 内核那套契约吃透了, 读 FA4 的正确姿势就是: 把它拆成「同样的契约 + 更多 tile + 更多 handoff」, 然后顺着上面那条因果链, 把新增的角色和屏障一个个对号入座就行。

课后练习 (Exercises)

这一小节拿几道问答, 把全章最容易绕晕的几个点串起来: 两段 MMA 之间多出来的那次 tile 交接、softmax 为什么要把 P 写回 TMEM、还有某个 barrier 到底“证明”了什么。下面每道题, 我先把”问的是啥、该往哪个方向想”讲清楚, 再给一份对着本章前文 (尤其是「两段 MMA」和「barrier 如何连接各角色」这两节) 推出来的参考答案。建议你先把答案盖上, 自己推一遍, 卡住了再翻回来看。

注意

**注意:** 这些练习真正要练的不是”背答案”, 而是把每个 tile 原语都套进同一张读卡——scope(谁在跑) / layout(tile 在哪) / dispatch(怎么发射) / handoff(用哪个 barrier

交棒)。哪天你能拿这四问拆开任意一个原语, 这章就算读通了。

第 1 题:FA4 相比 GEMM 多出来的 tile 交接

问题: 相比 GEMM,FA4 在两段 MMA 之间多出来一个全新的 tile 交接。请说出它的生产者 (producer)、所交接的 TMEM tile、消费者 (consumer)。

怎么想:GEMM 只有”一段反复跑的 MMA”, 累加器 tile 就在 MMA 内部循环里转, 中间没有别的角色插队。FA4 把一段 MMA 拆成了「score MMA」和「value MMA」两段, 中间硬塞了 softmax。规律是: 只要”角色变多”, 就一定”交接变多”——所以新交接肯定落在 score MMA 与 softmax、softmax 与 value MMA 这两条边上。题目问的是”两段 MMA 之间”那个 GEMM 压根没有的环节, 落点就是 score MMA 把分数 tile 递给 softmax 这一步。

参考答案:

| 要素         | 内容                                 |
|------------|------------------------------------|
| 生产者        | score MMA(由 WG3 warp 0 发射)         |
| TMEM tile  | 分数 tile S(写入 s_region[q_stage])    |
| 消费者        | softmax(WG0 / WG1, 整 warpgroup 读出) |
| 交接 barrier | s_ready                            |

整个交接说穿了就一句话:score MMA 算完, 由一个被选中的 lane(elect\_sync) 在 s\_ready 上 arrive, 喊一声”这块 S 已经躺在 TMEM 里了”,softmax warpgroup 这才能去读。GEMM 里没这一步, 是因为 GEMM 的 MMA 算完直接进下一轮累加, 中间没别的角色要看这块中间结果。

```
Tx.warp.gemm_async(S_region[q_stage], Q_smem[...], K_smem[...],
 dispatch="tcgen05", cta_group=CTA_GROUP)

if T.ptx.select_sync():
 s_ready.arrive(q_stage) # lane softmax
```

第 2 题: 为什么 softmax 要把 P 写回 TMEM

问题:softmax 在寄存器里刚算完分子 tile P, 为什么不直接把它留在寄存器里喂给 value MMA, 反而要写回 TMEM?

怎么想: 这题考的是”MMA 的操作数到底能读什么形状”。寄存器是每个线程私有、按线程散开的一堆标量; 而 MMA 是 Tensor Core 指令, 它要的是一个矩阵形状的 tile 操作数, 只认 SMEM 或 TMEM 这种共享、规整、能被硬件按矩阵布局寻址的地方。一堆散落在各线程私有寄存器里的 fp32 标量,Tensor Core 根本没法当矩阵来读。

参考答案: 因为 value MMA 要把 P 当矩阵操作数 (tile operand) 来吃, 而 MMA 读不了散在各线程的标量寄存器。本 kernel 里 P 唯一能被 MMA 消费的形态, 就是 P\_region——它是 fp16 TMEM 别名 tmem\_as\_f16 上的一个视图。所以这趟写回不是多此一举的搬运, 而是把 P 摆成”下一段 MMA 唯一真能读进去的形状”。顺便说一句, 写回时还顺手做了 fp32→fp16 的 cast, 既省了 TMEM 占用, 又对上了 Tensor Core 处理低精度操作数的吞吐。

可以这样对照记忆:

| 阶段 | P 的存放形态 | 谁能用它 |
|----|---------|------|
|----|---------|------|

|              |                        |                    |
|--------------|------------------------|--------------------|
| softmax 算完瞬间 | 各线程私有寄存器 (分散标量)        | 只有本线程的标量运算         |
| 写回之后         | P_region(fp16 TMEM 视图) | value MMA 当矩阵操作数读取 |

第 3 题:p\_o\_rescale 或 p\_ready\_2 到底证明了什么

问题: 从 p\_o\_rescale 和 p\_ready\_2 里挑一个, 说清楚这个 barrier 究竟”证明”了什么; 如果 value MMA 跳过对它的等待, 会出什么问题?

怎么想:value MMA 干的是把  $P \times V$  累加进 0。它要能安全开跑, 前提有两类: 一是**操作数真到位了** (P 这部分列已经写进 TMEM), 二是**累加目标安全** (0 这个槽位要么已经被 WG2 rescale 成新 max 下的正确状态, 要么因为压根不需要 rescale 而被放行)。这两个 barrier 分别守的, 就是 P 的两段 (96 + 32 切分) 和 0 的安全性。

参考答案 (以 p\_o\_rescale 为例):

- 它同时证明两件事成立:(1)P 的前 96 列已经写进 TMEM;(2)0 槽位对 value MMA 的累加是安全的——WG2 要么已经把旧 0 按新 row max rescale 完了, 要么判定本轮 max 跳得不够大、acc\_scale 保持 1.0、于是跳过了 rescale。第一个 K/V block 是特例: 没有旧 0 要 rescale,WG2 干脆直接预先 arrive p\_o\_rescale。
- 要是跳过这个等待会怎样:value MMA 可能在 P 前 96 列还没写完时就去读 TMEM, 读到陈旧/半成品数据; 更阴的是 0 还没被 rescale 就被加了进去——旧 0 是按旧 max 缩放的, 新一块直接加上去, 数值基准对不齐, 结果就是个悄无声息的**错误结果 (wrong-result bug)**, 通常还不报错、不崩溃, 排查起来要命。

注意

**注意:** 本章特意警告过一个经典坑——softmax\_corr.empty 只说明”WG2 已经把那个 scale/-sum 邮箱槽位消费掉了”, 它**不代表** P 就绪, **更不是**放行 value MMA 的门。真正放行 value MMA 的是 p\_o\_rescale。把这俩搞混, 正是 wrong-result bug 的常见来源。

要是你选 p\_ready\_2: 它证明的是 P 的最后 32 列 (最后四分之一) 已写进 TMEM, 对应 value MMA 那个 96 + 32 切分调度——第二个 sub-MMA 专吃这最后一段。跳过等它, 第二段 MMA 就会在最后 32 列还没写好时去读, 照样读到垃圾数据、结果出错。这个切分的意义就是别让 Tensor Core 空转: 前三块 chunk 一就绪就先发 96 宽的 MMA, 把最后一块的 exp 和 TMEM 写入跟正在飞的 MMA 叠到一起。

和你的 agent 一起练 (Try with your agent)

挑一个前文没给读卡的 tile 原语, 让 agent 替它补一张 scope / layout / dispatch / handoff 卡, 然后回到源码里, 用 guard、allocation、wait 这三处线索去核对它说得对不对。可以拿来练的对象有:

- epilogue 里的 Tx.copy\_async: 把最终 0 从 TMEM/寄存器搬到 0\_smem, 核对它是谁 (WG2 发起的、交给哪个 barrier(corr\_epi.full → TMA store)。
- fp32 → fp16 的 Tx.cast: 就是第 2 题里把 P 降精度写回 TMEM 的那一步, 核对它的精度转换是发生在 register 里、还是写入 TMEM 时。
- 第二个 gemm\_pv sub-MMA: 吃 P 最后 32 列的那段 value MMA, 核对它的 handoff 等的是不是 p\_ready\_2、accum 是不是 True。

核对的方法永远是同一套: `guard(if T.ptx.select_sync() / 角色分支 wg_id == ... 告诉你 scope)、allocation(S_region / P_region / O_region 视图告诉你 layout)、wait/arrive(barrier.wait / barrier.arrive 告诉你 handoff)`。把 agent 的回答一项一项对回源码, 别直接信它。

## 小结

本章顺着「跟一个 tile 走完整条数据通路」这条主线, 把 Flash Attention 4 怎么落到 GPU 上讲透了。核心脉络拢成几条:

- **算法层面:** Attention 的难点不在矩阵乘本身, 而在两段矩阵乘中间夹的那段 softmax。在线 softmax 靠「分块流式累加 + 维护 `row_max/row_sum/0`」躲开了「把整个 S 实打实摆出来」(物化), 代价就是行最大值一变大就得对旧 0 重缩放——这是贯穿全章的转折点。工程上又拿 `exp2`(配合 `scale_log2 = log2(e)/√d`) 顶替 `exp`, 还把归一化拖到最后统一做。
- **落地层面:** TIRx 按「干什么活」而不是「碰哪块数据」给 4 个 warpgroup 分角色, 把 S/P/O 挤进同一块 TMEM、靠 fp16 别名复用, 再用一组 mbarrier 把跨 warpgroup 的 handoff 接起来。所有新增的 barrier(`s_ready`、`p_o_rescale`、`p_ready_2`、`softmax_corr` 等) 都围着 softmax 转, 因为正是 softmax 把本来挨着的两段 MMA 给拆开了。
- **流水线层面:** Q(深度 2)、KV(深度 3)、TMEM slot(深度 2) 各跑各自的节奏, 没有「单一流水线深度」这一说; MMA warp 必须把 score 和 value 交错着发, 才能让 score / softmax / correction / value 这四类活在时间上叠起来。
- **功能扩展:** 因果掩码、GQA、tile 调度都被特意摺在「边界」上, 核心那张 tile-primitive 计算图原封不动——这正是「tile 原语 + barrier」骨架的厉害之处。
- **关键收获:** 读懂 FA4 的正确姿势, 是把它当成「跟 GEMM 同一套契约 + 更多 tile + 更多 handoff」, 对每个原语反复套那四问 `scope / layout / dispatch / handoff`, 同时盯紧 `softmax_corr.empty` 和 `p_o_rescale` 混淆这类 wrong-result bug。

## 延伸阅读

- 原文: [Flash Attention 4 —Modern GPU Programming for ML Systems](#)
- 前置章节 (第三部分 GEMM): 本章一路拿 GEMM 当参照系——GEMM 的 tile-primitive 图、scope/layout/dispatch 读卡, 还有 TMA / `tcgen05` MMA / TMEM / mbarrier 这些原语, 都是读懂 FA4 的必备底子。
- 硬件背景: Blackwell 架构的 TMEM(Tensor Memory) 和第 5 代 Tensor Core(`tcgen05`) 指令族, 是本章讨论 TMEM 别名复用、MMA 操作数位置的硬件前提。
- 参考实现: `tirx-kernels` 仓库的 `flash_attention4.py` 把本章讲过的全部组件 (TMA 加载、两段 MMA、在线 softmax、rescale 与回写、causal 掩码、GQA、tile 调度) 都收在一起, 可以直接 import、编译, 再拿 PyTorch 的 `scaled_dot_product_attention(enable_gqa=True)` 对拍验证。
- 进一步阅读: 想把 kernel 扩到训练场景, 得补写 `log-sum-exp(LSE)`, 还要记得按 `LSE_i = log(row_sum_i) + row_max_i/√d` 把 `1/√d` 缩放因子补回 `row_max`。

## 术语对照

| 英文                              | 中文                   | 简要说明                                                                                 |
|---------------------------------|----------------------|--------------------------------------------------------------------------------------|
| Flash Attention 4 (FA4)         | Flash Attention 4    | 用在线 softmax 实现、避免物化完整分数矩阵的高性能注意力 kernel                                              |
| Online softmax                  | 在线 softmax           | 按 K/V 块流式累加、逐行维护运行态、最后统一归一化的 softmax 算法                                              |
| Score MMA                       | 打分 / 分数 MMA          | 第一段矩阵乘 $S = Q \cdot K^T$ ，两个操作数都来自 SMEM                                              |
| Value MMA                       | 取值 MMA               | 第二段矩阵乘 $O += P \cdot V$ ，P 来自 TMEM、V 来自 SMEM                                         |
| Rescale / Correction            | 重缩放 / 校正             | row_max 变大后对旧 0 做的 TMEM→寄存器→TMEM 读-改-写，统一尺度                                          |
| row_max / row_sum               | 行最大值 / 行和            | 在线 softmax 逐行维护的两个运行态 (row_max 存未缩放分数的最大值)                                           |
| scale_log2                      | log2 缩放系数            | $\log_2(e)/\sqrt{d}$ ，使 $P = \exp_2((S - m) \cdot \text{scale\_log2})$ ，贴合硬件 exp2 指令 |
| acc_scale                       | 累加器缩放因子              | softmax 算出、经 SMEM 信箱传给 WG2 的逐行 rescale 因子                                            |
| rescale_threshold               | 重缩放阈值                | 行最大值变化小于此阈值 (本版 8.0) 则跳过 rescale, acc_scale 置 1.0                                    |
| TMEM (Tensor Memory)            | 张量内存                 | Blackwell 为 tcgen05 MMA 引入的片上内存，每 CTA 仅 128×512 fp32                                 |
| tcgen05                         | 第 5 代 Tensor Core 指令 | 驱动 MMA、TMEM 读 (tcgen05.ld)/写、commit 的指令族                                             |
| TMA (Tensor Memory Accelerator) | 张量内存加速器              | 在 GMEM↔SMEM 之间异步搬运 tile 的硬件引擎，按字节计数判完成                                               |
| Warpgroup (WG)                  | warp 组               | 128 线程为一组；本 kernel 每 CTA 有 4 个 (WG0-WG3)，按角色分工                                       |
| Q stage                         | Q 流水线阶段 / 槽位         | Q 流水线的 0/1 两个并发槽 (非两个 attention head), WG0/WG1 各管一个                                  |
| mbarrier                        | 内存屏障                 | 跨线程/角色的同步原语, FA4 用它把各 handoff 接线起来                                                   |
| s_ready                         | score 就绪屏障           | score MMA → softmax: S 已在 TMEM 可读                                                    |
| p_o_rescale                     | P/O 就绪屏障             | softmax+WG2 → value MMA: P 前 96 列就绪且 0 槽位可安全累加                                       |
| p_ready_2                       | P 尾段就绪屏障             | softmax → value MMA: P 最后 32 列就绪 (对应 96+32 切分)                                       |
| softmax_corr (full/empty)       | softmax-校正信箱屏障       | 保护单格 SMEM 信箱，在 softmax 与 WG2 间传 acc_scale/row_sum                                    |
| Causal masking                  | 因果掩码                 | query 只能关注自身及更早的 key；整块跳过 + 跨对角线逐行掩码                                                 |
| get_n_block_max                 | 最大 K/V 块上界           | 按 Q 块位置算出需遍历的最后一个 K/V 块，跳过对角线上方整块                                                    |
| mask_r2p                        | 列上限转位掩码              | 把逐行列上限转成 32 宽位掩码，一次性屏蔽越界列 (避免发散分支)                                                   |
| GQA (Grouped Query Attention)   | 分组查询注意力              | 多个 Q 头共享一个 K/V 头，缩小 KV cache、降带宽                                                     |
| GQA_RATIO                       | 分组比例                 | 每个 K/V 头被多少个 Q 头共享 (如 4)                                                             |
| LPT Scheduler                   | 最长处理时间优先调度器          | 因果模式用：重块前置以拉平各 CTA 完成时间，兼顾 L2 局部性                                                    |

|                      |          |                                                                    |
|----------------------|----------|--------------------------------------------------------------------|
| Linear Scheduler     | 线性调度器    | 非因果模式用：固定 CTA 池按 <code>num_ctas</code> 跨步均匀分配任务                    |
| Epilogue             | 收尾阶段     | K/V 循环结束后归一化（乘 <code>1/row_sum</code> ）、cast fp16、写 SMEM、TMA store |
| LSE (log-sum-exp)    | 对数和指数    | 训练反向所需的统计量；本 kernel 仅前向输出、不写 LSE                                   |
| Tile-primitive graph | tile-原语图 | 「生产者—消费者」依赖图：节点是某层存储里的 tile，边是搬运/计算                                |



# 第 15 章 · 参考资料 (附录)

原文:Reference

## 本章要点

### 本章要点 (TL;DR)

- 这一章是「第五部分 · 参考与附录」的**导读页**——你可以把它想成一本工具书最前面那张「目录 / 索引」。它不教新东西, 只回答一个问题: 你遇到具体问题时, 该去翻后面哪一页。
- 全书真正讲知识的主线是第一到第四部分 (Parts I-IV)。附录**不是拿来从头读到尾的**, 而是你写代码、调程序时**遇到问题随手翻**的工具箱。
- 附录主要有三块内容:**TIRx 语言参考** (查语法)、**编译器内部实现** (讲编译器怎么一步步把代码翻译成 GPU 能跑的指令, 这个过程叫「下降流水线」)、**Warp-Specialized 内核调试指南** (教你修那种特别难调的 GPU 程序)。
- 完整的 `tvm.tirx` Python API (就是那种把每个函数、每个参数都列全的字典式文档) 不在这本书里, 而是放在**上游 TVM 官方文档**中。
- 至于 TIRx 的底层语义和张量布局模型 (Layout API), 其实**第二部分**已经讲过了, 这里只是立个路牌指给你看。

## 前置知识

**前置知识:** 这一章里会冒出几个 GPU 专有名词, 别被吓到——它们在本章只是「预告」, 正文各章会细讲。这里我会把每个第一次出现的词都用大白话当场解释一遍, 所以你**零 GPU 基础也能读懂这一章**。如果想先有个整体印象, 可以翻一下第 0 章 · 极简入门。先记住三个最常出现的词就好:

- GPU 内核 (kernel):** 就是「一段跑在 GPU 上的小程序」。你平时写的代码跑在 CPU 上; 要让显卡帮你算, 就得把那段计算写成一个 kernel 丢给 GPU 跑。
- GEMM:** 就是「矩阵乘法」(General Matrix Multiply 的缩写)。为什么它这么重要? 因为深度学习里绝大部分计算——神经网络的每一层、注意力机制——本质上都是在做矩阵乘, 所以它是 GPU 上当之无愧的「主角」。
- warp specialization(warp 专精):** 一种写 GPU 程序的技巧, 意思是「让不同的线程小组各干各的活」——比如一组专门负责搬数据、另一组专门负责算。后面 15.2 会展开讲为什么要这么干。

## 15.1 这一部分是什么

先把整本书的脉络捋一遍。《Modern GPU Programming for ML Sys》真正讲东西的, 是 **Parts I-IV** 这四个部分。它的讲法大概是这样, 我们顺着读者第一次学的角度走一遍:

1. 先带你认识 GPU 硬件长什么样——显卡里有什么、为什么内存要分好几层、几千个线程怎么一起干活, 这些先有个直观印象。
2. 再引出 TIRx。TIRx 是这本书自创的一门「中间语言」。什么叫中间语言? 打个比方: 你写的是高层意图 (比如「把这两个矩阵乘起来」), 而 GPU 最终执行的是又细又底层的机器指令; 中间需要一种语言把高层意图描述清楚、又方便编译器往底层翻译, TIRx 就是夹在中间的这一层。它特别之处在于是「张量级」的——直接拿整块矩阵、整块张量当操作对象, 而不是一个数一个数地处理。
3. 接着讲编译器是怎么把你写的高层算子, 一步一步”降”成高效 GPU 代码的。这里的「算子」你就理解成「一个计算操作」(比如一次矩阵乘、一次加法); 「降」(lowering) 是个编译器术语, 指把高层、抽象的描述一层层翻译成越来越底层、越来越贴近硬件的代码, 就像把「我要喝水」逐步细化成「抬手、握杯、送到嘴边」那一连串具体动作。
4. 最后教你怎么写出 warp-specialized 的高性能内核, 也就是前面提到的「让不同线程小组分工」的那种高性能 GPU 程序。

这条线才是主菜。

那这一章所在的「参考与附录」呢? 定位完全不一样。原文说得很白: 这里放的, 是”你读着读着随手要查的东西”。说白了, 它不是让你从头啃到尾的, 而是搁在手边的一本工具书。你读正文卡住了, 或者真动手写内核、调内核的时候撞上某个具体问题, 翻到对应那一页查一下就行。

**关键**

**关键:** 附录是用来”哪里不会查哪里”的, 不是用来通读的。第一次学的话, 先把 Parts I-IV 顺顺当当读一遍, 附录就当踩坑时翻一翻的入口, 够用了。

### 15.2 三类需求 → 三个去处

附录是按”你想干嘛”来分的——你有什么需求, 就告诉你该往哪儿翻。下面这张表把需求和去处一一对上:

| 你的需求                                                         | 该翻哪一页                                                      |
|--------------------------------------------------------------|------------------------------------------------------------|
| 想查某个 TIRx 语言特性 (语法、内建函数、语义)                                  | TIRx 语言参考 (TIRx Language Reference)                        |
| 想搞懂编译器内部实现, 尤其是下降流水线 (lowering pipeline)                     | 编译器内部实现 (Compiler Internals)                               |
| 要调试异步 GEMM / FlashAttention 内核的 hang(挂死)、crash(崩溃)、结果算错或性能变慢 | Warp-Specialized 内核调试 (Debugging Warp-Specialized Kernels) |

**一句话先理解**

**一句话先理解:** 这张表就是「我遇到 X 类问题, 该去翻哪一页」。下面把三种最常见的场景挨个说清楚, 顺带把里头的 GPU 名词讲明白。

说白了, 这三块对着三种最常见的场景:

- **查语言:** 写 TIRx 的时候, 忘了某个语法、某个内建函数怎么用 → 翻语言参考。这跟你查一门编程语言的语法手册是一回事。
- **看原理:** 想弄明白”我写的高层算子, 最后到底怎么变成 GPU 指令的” → 翻编译器内部实现, 重点看 lowering pipeline(下降流水线, 就是上面说的那条「一层层往底层翻译」的流水线)。

- **修 bug:** 异步内核出毛病了 → 翻调试指南。

这里先解释一下什么叫 **FlashAttention**(简称 FA): 它是一个专门用来高效计算「注意力」(Transformer 大模型里最核心的那个操作) 的 GPU 内核, 出了名地快、也出了名地难写。

那这一节标题里的「**异步内核**」又是什么意思? 「异步」在这里指的是: **搬数据和做计算**这两件事不再排队一个接一个干, 而是**重叠着同时进行**——一边算着这一块, 一边后台偷偷把下一块数据搬进来。为什么要这么干? 因为 GPU 算得飞快, 但从远处的大内存搬数据进来却很慢; 要是「先搬完再算、算完再搬」, GPU 大半时间都在干等数据, 白白浪费算力。让搬运和计算重叠起来, 就能把这段等待时间「藏」掉。

具体来说, 这里说的异步内核, 是指那些用到下面这几样东西的 GEMM / FA 内核:

- **TMA:** 一个专门负责「异步搬数据」的硬件单元 (详见第 0 章)。有了它, 搬数据这活儿可以丢给硬件后台去办, 计算线程不用守着。
- **warp specialization:** 让不同的 warp 各司其职——有的 warp 专门搬数据、有的专门算。(warp 是 GPU 里 32 个线程绑在一起、必须步调一致行动的一个小组, 可以想象成「32 个人必须迈同一方的阵」; 后面正文会细讲。)
- **tcgen05 MMA:** 一条专门做矩阵乘加的硬件指令。MMA 是 Matrix-Multiply-Accumulate(矩阵乘加, 即「乘完顺手累加」) 的缩写, 是 GPU 里那块专门加速矩阵乘的硬件——**Tensor Core**——的核心指令。
- **TMEM**(Tensor Memory): Blackwell 这代 GPU 上专门留给 Tensor Core 用的一块「片上内存」(就是直接长在芯片上、离计算单元极近、所以存取极快的一小块内存)。

为什么要把这种内核单拎出来、专门给它一整节调试指南? 因为正是上面这种「异步 + 流水线化 + 多组分工」的内核, 是所有 GPU 程序里**最难调**的一类——好几件事同时在跑, 出了问题很难看清是哪一步错了。

**澄清一个容易混的点:** 这本书的 warp-specialized 调试指南, 讲的是**跑在 Blackwell 上的内核**。Blackwell 是 NVIDIA 较新的一代 GPU 架构, 代号写作 **sm\_100a**(这种 **sm\_xxx** 的写法是 NVIDIA 给每代架构起的编号)。它的矩阵乘用的是 Blackwell 那套 **tcgen05 MMA** 指令, 再配上刚说的 TMEM。注意, 这**不是** Hopper(上一代架构) 那一代的 **warpgroup MMA**(写作 **wgmma**, 是把 4 个 warp 凑成一组协同做矩阵乘的指令)。一句话: 两者是不同代显卡上的不同指令, 千万别搞混。

下面用一张 Mermaid 图, 把这个”对症下药”的过程画出来:

**结构图**

连接关系:

- 属于哪类? 查 TIRx 语法/特性 → TIRx 语言参考 / TIRx Language Reference
- 属于哪类? 理解编译器/下降流程 → 编译器内部实现 / Compiler Internals / (lowering pipeline)
- 属于哪类? 异步 GEMM/FA 内核 / hang/crash/算错/变慢 → Warp-Specialized / 内核调试指南
- 属于哪类? 完整 tvm.tirx Python API → 上游 TVM 官方文档 / (不在本书内)
- 属于哪类? TIRx 底层语义 / 张量布局模型 → 回到第二部分 / Intro to TIRx + Layout API

注意

**注意:** 图里的 R1/R2/R3, 是附录自己实打实包含的三个子页面。而 R4(完整 Python API) 和 R5(底层语义与布局) **并不在附录里**——作者只是顺手给你指个路, 告诉你”这俩该上哪儿找”, 免得你在附录里翻半天还找不着。

15.3 两个”不在这里, 但你可能要找”的指针

有两样东西, 你读着读着八成会去找, 可它们偏偏不在这儿。附录特意提了一句, 免得你白翻半天。

1. 完整的 `tvm.tirx` Python API

书里讲 TIRx, 只讲关键概念和怎么用。那种完整、一条一条列全的 `tvm.tirx` Python API 文档, 书里没有, 它放在上游 TVM(upstream TVM) 的官方文档里。(这里多说一句:TVM 是一个开源的深度学习编译器项目,TIRx 是建在它之上的;「上游」就是指 TVM 这个原始项目本身, 跟你平时说的「上游仓库」是一个意思。)

关键

**关键:** 你要是想查”某个函数的精确签名、有哪些参数、每个参数能填什么”这种字典式的细节, 该去翻 TVM 官方 API 文档, 别在这本书里找。说白了就是分工: 书负责把思想和用法讲透,API 文档负责把每个接口的细节列全。

2. TIRx 原生层 / native level 与张量布局模型 / Layout API

先把这两个名词点一下:**TIRx 原生层 (native level)** 指的是 TIRx 更贴近底层硬件的那一层语义, 也就是它最「原始、底层」的样子; **张量布局模型 (tensor layout model)** 则是描述「一块张量在内存里到底怎么摆放」的一套规则——同样一个矩阵, 数据在内存里横着排还是竖着排、怎么切块, 直接影响 GPU 读取快不快, 所以专门有一套模型 (配套的 API 叫 Layout API) 来管这件事。

这两块, **第二部分 (Part II) 早就讲过了**。附录在这儿只是给你指条路, 告诉你”回去看哪儿”, 省得你在附录里瞎翻:

| 内容                           | 在哪讲过                                  |
|------------------------------|---------------------------------------|
| TIRx 的原生/底层语义                | 第二部分 · TIRx 入门 (Introduction to TIRx) |
| 张量布局模型 (tensor layout model) | 第二部分 · TIRx 布局 API(TIRx Layout API)   |

15.4 建议的使用方式 (阅读顺序)

这是张导航页, 它自己没什么”先读这段、再读那段”的讲究。真正值得说道的, 是**怎么把附录揉进你整个学习过程里**。给你一套实在的用法:

1. **第一遍学:** 顺着主线把 Parts I-IV 读完。这阶段附录就当”卡住了再去查”的入口, 不用提前看, 更不用从头啃。
2. **写代码的时候:**TIRx 的某个语法或特性吃不准 → 先翻 **TIRx 语言参考**; 想要某个函数的精确用法 → 直奔 **TVM 官方文档**。

3. 想往深里钻原理: 等正文读明白了, 再去看 **编译器内部实现**, 顺着 lowering pipeline 把”我写的高层算子, 到底怎么一步步变成 GPU 代码”这一路走一遍。
4. 真上手调内核: 等你开始写或调异步 GEMM、FlashAttention 这类 warp-specialized 内核, 撞上挂死、算错、变慢的时候 → 重点啃 **Warp-Specialized 内核调试指南**。
5. 底层语义或布局忘了: 回 **第二部分**复习 TIRx 入门 (Intro to TIRx) 和布局 API(Layout API)。

小结

这一章是「参考与附录」的导航页, 记住三点就够了:

- 全书的主线在 **Parts I-IV**。附录是**手边随查的工具箱**, 哪里不会查哪里, 别从头通读。
- 附录按”你想干嘛”分成三个子页面:**TIRx 语言参考** (查语法特性)、**编译器内部实现** (看 lowering pipeline, 也就是「代码怎么一步步翻译成 GPU 指令」)、**Warp-Specialized 内核调试** (修那种异步 GEMM / FA 内核的挂死、崩溃、算错、变慢)。
- 两样最容易”找不着”的: 完整 `tvm.tirx` Python API 在**上游 TVM 文档**里;TIRx 底层语义和张量布局模型在**第二部分**。

把这张”对症下药”的导航表记牢, 真动起手来, 你就能很快翻到对的那一页。

延伸阅读

- 原文页面:[Reference —Modern GPU Programming for ML](#) Sys
- 完整 `tvm.tirx` Python API: 见上游 TVM 官方文档 (Apache TVM 项目文档)。
- 相关前置章节: 第二部分「TIRx 入门 (Introduction to TIRx)」与「TIRx 布局 API(TIRx Layout API)」。

术语对照

| 中文                    | English                            |
|-----------------------|------------------------------------|
| 下降流水线                 | lowering pipeline                  |
| 编译器内部实现               | Compiler Internals                 |
| TIRx 语言参考             | TIRx Language Reference            |
| Warp-Specialized 内核调试 | Debugging Warp-Specialized Kernels |
| 张量布局模型                | tensor layout model                |
| TIRx 布局 API           | TIRx Layout API                    |
| TIRx 原生层              | TIRx native level                  |
| 上游 TVM                | upstream TVM                       |
| 挂死                    | hang                               |
| 崩溃                    | crash                              |



# 第 16 章 · 调试 Warp 专门化 Kernel

原文: [Debugging Warp-Specialized Kernels](#)

## 本章要点

### 本章要点 (TL;DR)

先别被下面这些陌生词吓到——正文里每一个都会从头讲清楚。这里你只要先记住一句话：这种 GPU 程序 (kernel) 出错，绝大多数不是你的算法写错了，而是「几个工序之间交接班的时候掉了链子」。

- **先别急着重写 kernel。** (kernel 就是「跑在 GPU 上的那段程序」，你可以先这么理解。) 这类程序出运行期故障，十有八九不是算法写错了，而是某一次「交接 / handoff」掉了链子——就像流水线上前一道工序还没把零件放好，后一道工序就抢着去拿，自然出事。具体怎么坏的？无非这么几种：屏障 (barrier) 没初始化、到达计数对不上、集体操作被「角色守卫」挡掉了一半、屏障的相位 (phase) 过期了，或者一块存储还没用完就被人抢去复用了。(这些词现在不懂没关系，正文都会讲。)
- **动手之前，先填一张「角色 → 存储 → 交接 → 生命周期」四要素表。**把这四样东西白纸黑字写下来，再拿生成的 CUDA (GPU 真正执行的底层代码) 一项一项去对。千万别凭感觉乱改同步代码——那样越改越乱。
- **真正算数的是生成的 CUDA，它才是「事实来源 / source of truth」。**你在 Python 里写的那些高层判断 (比如「只让 0 号小组干这件事」)，编译之后会变成一堆跟线程编号有关的算术运算。所以先用一个叫 `inspect_source("cuda")` 的工具把生成代码存下来，用 `grep` 搜一下关键字，再回头去读你的 Python。
- **CUDA 一旦出错，会把整个运行环境 (context) 「毒化 / poison」掉。**意思是：出过一次严重错误之后，GPU 这一摊就脏了，后面你再跑别的代码也会跟着报错。撞上「非法内存访问」(程序访问了不该碰的显存地址) 怎么办？别犹豫，直接**重启 Python 进程**。不然你接着调的根本不是当前代码，而是上一次崩溃留下的残局。
- **症状只是线索，不是结论。**先把现象按「死锁 / 崩溃 / 结果错 / 正确但慢」分个类，再对着对应的检查清单一条一条排。还有一句话要刻在脑子里：**一次只改一个交接点——一次改一处，出问题才知道是哪儿的锅。**

## 前置知识

**前置知识：**这一章会用到一些 GPU 的基本概念。如果下面这几个词你完全没概念，强烈建议先翻一下第 0 章 · 极简入门垫一垫底：

- **GPU：**显卡里那块芯片，擅长「同一件事让成千上万个小工同时干」，所以特别适合深度学习里那种海量重复的计算。
- **线程 / thread：**GPU 上最小的「干活单位」，你可以理解成一个小工。GPU 会同时跑到千上万个线程。

- **warp(线程束)**: GPU 把线程 **32 个绑成一捆**一起调度, 这一捆就叫一个 warp。打个比方, warp 就像「32 个人组成的方阵, 必须迈同一步」——它们同一时刻执行同一条指令。
- **warp 专门化 / warp specialization**: 让不同的 warp 去干不同的活 (有的搬数据、有的算乘法、有的写结果), 分工协作, 而不是人人都干一样的事。
- **屏障 / barrier**: 一个「集合点」。线程跑到这儿会停下来等, 直到约定数量的人都到齐, 才一起放行——就是用来协调「谁先谁后」的工具。

本章后面还会反复出现 **mbarrier**(异步版的屏障)、**phase**(屏障的「相位」计数)、以及「生成的 CUDA」这些词, 正文第一次出现时都会现场解释。配套背景在第 13 章「用 Warp 专门化和 Cluster 扩展 GEMM」。

这一章是《Modern GPU Programming for ML Sys》参考与附录里的一篇「工程实战手册」。它不教新算法, 只干一件事: 把前面那些复杂的、好几个工序同时跑的 GPU 程序, 提炼成一套**能反复用、能直接上手的调试套路** (配套代码就是「用 Warp 专门化和 Cluster 扩展 GEMM」那一章里 GEMM 的步骤 7-9。GEMM = 通用矩阵乘法, 就是「两个矩阵相乘」, 它是深度学习里最核心、跑得最频繁的计算)。

#### 一句话先理解

**一句话先理解**: 这些 GPU 程序之所以难调, 是因为它们让**三件事同时进行、谁也不等谁**——一边在搬数据, 一边在算乘法, 一边在往外写结果。

具体来说, 这三件事是:

- **搬数据进来**: 用 TMA 把数据从「大而慢的显存」整块搬到「小而快的片上内存」。(TMA 是 GPU 里一个专门负责「整块整块搬运数据」的硬件引擎, 你可以把它想成一台自动传送带; 后面会再说这些内存的区别。)
- **算乘法**: 用 GPU 里的专用矩阵乘法硬件 (Tensor Core) 把搬进来的数据相乘并累加。
- **写结果出去**: 把算好的结果再写回内存。

这三件事**叠在一起 / overlap**跑 (overlap 就是「时间上重叠、并行进行」), 好处是快, 坏处是——一旦出 bug, 因为同时在跑的东西太多, 排查起来格外恼火。不过有个好消息: 这套调试方法不光对矩阵乘法好使, **Flash Attention** (一种让 Transformer 注意力机制跑得又快又省内存的实现) 里那几段交接, 照样能套上去。

## 16.1 心法: 把异步 kernel 看成一组「交接」

传统的 GPU 程序你是怎么调的? 基本上就是从头到尾把代码读一遍, 逮逻辑 bug。可 warp 专门化的程序不能这么干, 因为它的结构根本不一样。

它的玩法是这样: 把不同的 warp (前面讲过, 32 个线程绑成的一捆) 派去干不同的活, 也就是给它们分**角色 / role**——一组专门负责「搬数据进来」(发 TMA 加载), 一组专门负责「算乘法」(发 MMA, 即矩阵乘加指令), 一组专门负责「写结果出去」(写回)。你就把它们想成流水线上的几个工位, 各管一摊, 谁也别抢谁的活。

这里还会冒出一个词 **warp group(线程束组)**: 就是把 **4 个 warp 捆在一起**, 合起来  $4 \times 32 = 128$  个线程, 作为一个更大的协作单位。后面会经常用「WG0」「WG1」指代第 0、第 1 个 warp group。



那工位和工位之间靠什么协调? 靠**异步屏障 / mbarrier**。前面说过屏障是个「集合点」;mbarrier 就是它的「异步增强版」,专门用来跨工位喊话:「我这道工序干完啦,该你接手了。」(「异步」的意思是:发出信号的人不用站在原地干等,可以接着忙别的,这正是它能让三件事同时跑的关键。)

关键

**关键:** 在这种结构里,代码本身经常「怎么看都没毛病」,真正爱出事的地方是**角色之间的交接**。哪怕某个屏障的到达计数(就是「要等几个人到齐」那个数字)只写错了一个,整条流水线都能给你永久卡死——因为它会一直傻等那个永远凑不齐的人数。

所以这一章开篇就先立一条规矩:

注意

**注意:** 别一上来就重写 **kernel**。顺序应该是这样:(1) 先把运行环境查清楚,(2) 再去查看生成的 CUDA,(3) 最后才回头动 Python。环境和编译这两关一过,剩下的运行期故障基本就跑不出下面这几种「坏掉的交接」了(每一种正文都会细讲,现在扫一眼有印象就行):

- 屏障压根没初始化(没设好「集合点」)
- 到达计数写错了(等的人数对不上)
- 集体操作被「角色守卫」挡掉了一部分(本该所有人一起做的事,被「只让某些角色做」的判断给拦掉了一半人)
- 屏障的相位(phase)过期了(后面讲)
- **生产者 / producer**(干活产出数据的一方)刚写好的数据, **消费者 / consumer**(要用这份数据的一方)还没来得及看见呢,这块存储就被人抢去存别的东西了

这条心法记牢了——后面那一堆检查清单,说白了都只是它的展开版。

## 16.2 调试前: 先排除运行环境

很多事看着像「kernel 的 bug」,其实根本不在 kernel 里——是环境出了岔子。所以这一章的要求很明确: **第一步先把环境查干净**。两条命令就够了:

```
TVM (checkout)
python -c "import tvm, tvm.tirx; print(tvm.__file__, tvm.__version__)"

GPU (kernel Blackwell)
python -c "import torch; print(torch.cuda.get_device_name(), torch.cuda.get_device_capability())"
```

为什么偏偏查这两样?

- **这些 kernel 只认 Blackwell 这一代显卡(代号 sm\_100a)**。Blackwell 是 NVIDIA 显卡的一代「架构」(可以理解成「第几代芯片设计」),比它老的还有 Hopper、再老的 Ampere。本章用到的 **tcgen05**(Blackwell 上那套新的矩阵乘法指令)只有 Blackwell 这一档的硬件才支持。所以你的 GPU 要不是 Blackwell,代码怎么改都白搭——第二条命令就是用来确认显卡型号和它的「计算能力」(compute capability,衡量这块显卡支持哪些功能的版本号)。
- **Python 很可能偷偷导入了一个过期的 TVM**。(TVM 是本章用来生成 GPU 代码的那套编译框架。)万一 **tvm.\_\_file\_\_**(它会打印出「实际加载的 TVM 装在哪个文件夹」)指过去的是个

旧目录，你还以为早更新过了——那你跑的压根不是刚改的代码。这个坑最阴，经常一搭进去就是好几个钟头打水漂。

### 关键

**关键：**环境确认没问题之后，先跑一遍 kernel 的最小正确性检查（比如 `run_correctness()`），之后再管性能。先看「对不对」，再操心「快不快」，顺序别反了。

## 16.3 标准调试流程 (Workflow)

原文给了一条 8 步的工作流，我把它画成了下面这张流程图。它的精神用一句话就能说清：一步步收窄范围、拿生成代码对照、一次只动一个地方。

### 结构图

连接关系：

- 2. 编译是否失败？ $\xrightarrow{\text{是}}$  检查：已装的 API / target / / dispatch= / buffer 作用域 / (先别看运行期同步代码)
- 2. 编译是否失败？ $\xrightarrow{\text{否}}$  3. 保存 `inspect_source('cuda')` / grep: 角色守卫 / mbarrier\_init / / tcgen05 / cp.async.bulk.tensor / cta\_sync()
- 3. 保存 `inspect_source('cuda')` / grep: 角色守卫 / mbarrier\_init / / tcgen05 / cp.async.bulk.tensor / cta\_sync()  $\rightarrow$  4. 为出错路径写出 / 角色/存储/交接/生命周期四要素表
- 4. 为出错路径写出 / 角色/存储/交接/生命周期四要素表  $\rightarrow$  5. 拿生成的 CUDA 逐项核对该表 / (barrier init 在角色分支之前等)
- 5. 拿生成的 CUDA 逐项核对该表 / (barrier init 在角色分支之前等)  $\rightarrow$  6. 归类：死锁 / 崩溃 / / 结果错 / 正确但慢
- 6. 归类：死锁 / 崩溃 / / 结果错 / 正确但慢  $\rightarrow$  7. 一次只改一个交接点 / (init 计数 / phase / 守卫 / fence / drain ...)
- 7. 一次只改一个交接点 / (init 计数 / phase / 守卫 / fence / drain ...)  $\rightarrow$  8. 先重跑正确性，再测性能
- 8. 先重跑正确性，再测性能  $\xrightarrow{\text{仍未解决}}$  1. 用仍能复现的 / 最小 shape 复现故障

这几步里，有几个细节得拎出来反复念叨：

- **第 1 步：**要是故障是非法内存访问 / illegal memory access，下次跑之前务必先**重启 Python**(为啥?16.7 节专门讲)。
- **第 2 步：**编译挂了，先去看 API / target / dispatch / 作用域，别急着去抠运行期的同步代码。编译问题和同步问题压根是两码事。
- **第 3 步：**生成的 CUDA 存盘之后，先 **grep** 关键字字符串，再去读 **Python**。这是本章翻来覆去强调的招：把生成代码当成你的「地图」来用。
- **第 7 步：**一次只改一个交接点 (init 计数、arrive/wait 的 phase、角色守卫、fence、TMA store 的 drain、TMEM 的 alloc/dealloc、tile scheduler 的推进，挑一个改)。你要是一口气改好几处，真出了问题，根本分不清到底是哪一处起的作用。

16.4 「四要素表」：可迁移的调试工作表

这是整套方法的**心脏**，也是最该背下来的一段。它的思路很朴素：别急着读那一大坨代码，**先把这个 kernel「谁干什么、东西放哪、怎么交接、什么时候能扔」这四件事写在纸上**，心里有谱了，再去对代码。不管你面前是**哪个**这种「多工序同时跑」的 kernel，动手改代码之前，先老老实实把下面这张表填出来：

| 要素              | 要写下来的内容                                                                                                                                                                                                                                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 角色 / Roles      | 每一个异步操作，到底是谁发出来的——哪些线程 / warp / warpgroup / CTA。(CTA = 线程块，一批被分到一起、能共享同一块高速内存的线程，见第 0 章；后面会经常出现。)                                                                                                                                                                                                               |
| 存储 / Storage    | 每个 tile(把大矩阵切成的一个个小方块,GPU 一次处理一块)走到每一步时「住」在哪一层内存里。GPU 的内存是 <b>分层</b> 的，越快的越小，这里从慢到快依次是: <b>GMEM</b> (全局内存，就是显存，容量大但慢)、 <b>SMEM</b> (共享内存，一个 CTA 内的线程共用的一小块片上高速内存，你可以把它当成「一块要自己手动管理的高速缓存」)、 <b>TMEM</b> (矩阵乘法硬件 Tensor Core 专用的累加器内存)、 <b>寄存器 / register</b> (每个线程私有、最快的一点点存储，作用类似 CPU 的寄存器，但 GPU 里每个线程都有自己的一份)。 |
| 交接 / Handoff    | 谁生产、谁消费、用哪个屏障对象传信号、到达计数多少、相位 (phase) 多少，还有那个「让刚写的数据真正对别人可见」的 <b>fence</b> 或 <b>drain</b> 。(fence / drain 现在不懂没关系，后面 16.12 会细讲——一句话：它们是确保「我写的东西别人真能看到 / 真写完了」的 <b>保险措施</b> 。)                                                                                                                                    |
| 生命周期 / Lifetime | 每一块存储，最早什么时候能拿去存别的东西 (复用)、什么时候能被读回、什么时候能释放掉。                                                                                                                                                                                                                                                                    |

在往下之前，先把一个会反复出现的词讲清楚：**角色守卫 / role guard**。它其实就是代码里的一句 **if** 判断，用来「把活分给特定角色」。比如 `if ( 0 ) { ... }`，意思是「只有 0 号小组的线程才进来干这件事，别人绕道」。守卫写对了，分工就清清楚楚；守卫写错了——比如本该「所有人都做」的事被一句 **if** 拦掉了一半人——就出大事。这正是本章一半 bug 的来源。

表填完了，接下来**拿生成的 CUDA 去逐项验**这五件事 (每一条对应一类常见 bug，后面都会展开)：

1. **角色守卫**跟你表里的角色对得上 (谁该干的活，守卫确实只放谁进去)。
2. **屏障初始化** (设置「集合点」那一步) 是在被守卫的角色分支**外面** (在它前面)，不是塞在某个 **if** 分支里头。为啥重要?16.9 会讲——塞错地方会导致初始化根本没人执行，直接死锁。
3. **集体操作没被守卫不小心收容**。「集体操作 / collective」指的是「本该一整组线程一起做」的事；一旦被 lane / warp / warpgroup 守卫拦掉一部分人，人数就凑不齐了。(lane / **通道**就是线程在一个 warp 里的编号,0~31, 一共 32 个。) 这一类 bug 最隐蔽，后果也最狠。
4. **arrive / wait 的相位 (phase)** 跟你的交接表对得上。(arrive = 「我到集合点了」,wait = 「我在集合点等」;phase 是它们对暗号用的一个计数，后面 16.9 ⑥ 会细讲。)
5. **TMA store 的 drain**、**TMEM 的释放 (dealloc)**、**SMEM 的复用**，统统等到生命周期表点头说「现在可以了」之后才发生——也就是别在数据还有用的时候，就把它的地盘抢去干别的。

关键

**关键:** 同一张表, 套在矩阵乘法的「搬数据 → 乘法 → 写回」流水线上行 (也就是 `TMA` → `MMA` →     ), 套在 Flash Attention 的 `score` → `softmax` → `value` → `correction` 那几段交接上也行 (`score` = 分数矩阵; `Q·K` 两个矩阵相乘的结果; `softmax` = 把这些分数归一化成「概率」的那一步)。这就是「可迁移」的意思——方法本身一个字不用变, 你只要把表里的内容换成对应 `kernel` 的角色和存储就完事了。

下面这张图, 把矩阵乘法流水线的三个角色、以及它们之间的交接都画了出来。先看主线那三个实线箭头: 数据从 `GMEM` 被搬进 `SMEM`、在 `SMEM` 里被乘进 `TMEM` 当累加器、最后从 `TMEM` 写回去。再看那几条虚线——它们就是「交接信号」: 每条虚线上标的 `count=`     就是那个屏障的到达计数 (要等几个人到齐)。看一眼, 你就能摸到「四要素」在真实 `kernel` 里到底长啥样:

结构图

分组:

- 分组
- 分组
- 分组

连接关系:

- `GMEM (A/B)`  $\xrightarrow{\text{TMA load}}$  `SMEM`
- `SMEM`  $\xrightarrow{\text{tcgen05 MMA (累加)}}$  `TMEM`
- `TMEM`  $\xrightarrow{\text{写回 (结果 D)}}$  `SMEM / GMEM`

注意

**注意:** 重点盯紧那几个到达计数。`tma2mma/mma2tma/mma2ld` 都是 **1**(一个引擎或一条 `commit` 触发一下就够了), 而 `ld2mma` 是 **128**(得等 `WG0` 全部 128 个线程都到齐)。这些数对不对, 几乎就决定了 `kernel` 会不会死锁 (细节见 16.9 节)。

16.5 当编译失败时

编译期的问题, 要**抢在**运行期同步问题前头解决, 因为这俩根本不是一类 `bug`。原文给了一张对照表, 我重新理了一下:

| 症状                                                 | 大概率出问题的地方                          | 第一步该查什么                                                                                                                                                                      |
|----------------------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 报「未知 <code>TIRx</code> API」或属性错误                   | 装的 <code>wheel</code> 跟教程代码对不上     | 打印 <code>tvm.__file__</code> / <code>tvm.__version__</code> , 拿 <code>API</code> 名字去和「 <code>TIRx</code> 语言参考」核一遍。                                                           |
| 报「不支持的 <code>dispatch=</code> 」                    | 你选的 <code>target</code> 或原语撑不起这条路径 | 看 <code>dispatch</code> 参数和 <code>target</code> 能力; 本教程的 <code>tcgen05</code> 路径要 <code>Blackwell</code> 。                                                                   |
| <code>buffer</code> 作用域 ( <code>scope</code> ) 对不上 | 有个 <code>buffer</code> 走错了硬件路径     | 翻工作表的「存储」那行: <code>TMEM</code> 只能走 <code>tcgen05</code> 访问; <code>TMA</code> 的操作数 ( <code>operand</code> , 指令的输入数据) 得用兼容的 <code>GMEM/SMEM</code> 布局 ( <code>layout</code> )。 |

|                             |                     |                                                            |
|-----------------------------|---------------------|------------------------------------------------------------|
| 编译过了，可生成的 CUDA 里根本没有你要的那条路径 | dispatch 没按你以为的方式降级 | 改算法之前，先去生成的 CUDA 里 grep 一下 tcgen05 和 cp.async.bulk.tensor。 |
|-----------------------------|---------------------|------------------------------------------------------------|

关键

**关键：**最后一行是个超级容易栽的坑。一句话记牢：**编译过了，不等于生成了你想要的指令。**dispatch 完全可能悄没声地拐去一条慢路（没用 TMA、没用 Tensor Core），却照样编译成功。所以「去生成的 CUDA 里亲眼确认那条路径在不在」，这一步是躲不掉的。

16.6 检查生成的代码：让 CUDA 当地图

这是本章最实用的一个工程招数。先说清楚一个关键事实：你在 Python 里写的是「高层意图」，但 GPU 真正执行的是另一份**由编译器自动生成的底层代码**，它用的语言叫 **CUDA**（可以理解成「GPU 上的 C 语言方言」）。很多 bug 在 Python 里看不出来，但在这份生成的 CUDA 里一眼就能逮到。所以我们要做的，就是把这份生成的 CUDA **存成一个文件**，这样就能对它 grep（搜关键字）、对它 diff（和「正确版本」逐行比对）——调试效率立马上一个台阶。下面这段 Python 就是干这事的，逐行看：

```
from pathlib import Path

inspect_source("cuda") : CUDA ,
cuda_source = ex.mod.imports[0].inspect_source("cuda")
Path("artifacts").mkdir(exist_ok=True) # ()
Path("artifacts/my_kernel.cu").write_text(cuda_source, encoding="utf-8") # .cu
print(cuda_source) #
```

存盘之后，后面所有「grep 一下 xxx」的动作，搜的都是这个 **my\_kernel.cu** 文件。

16.6.1 TIRx 守卫 → CUDA 表达式的对照

关键就在这儿：你在 Python 里写的那些高层守卫（比如「只让 0 号小组进来」），过了编译会变成一堆看起来很陌生的、跟线程编号有关的算术运算（**threadIdx** 就是「当前线程的编号」）。如果你不认识这套对应关系，在生成的 CUDA 里就会一脸懵。所以你得先把下面这张映射表吃透，才能一眼认出「哦，这段位运算其实就是我那个角色守卫」：

| TIRx 写法       | 生成的 CUDA                              |
|---------------|---------------------------------------|
| wg_id == 0    | (warp_id_in_cta >> 2) == 0            |
| wg_id == 1    | (warp_id_in_cta >> 2) == 1            |
| warp_id == 0  | (warp_id_in_cta & 3) == 0             |
| warp_id == 3  | (warp_id_in_cta & 3) == 3             |
| lane_id == 0  | ((((int)threadIdx.x) % 32) == 0       |
| .init() 的内部守卫 | ((int)threadIdx.x) < 1(仅 CTA 的 0 号线程) |
| elect_sync()  | tvm_builtin_elect_one_sync_op()       |

这张表里有几处是位运算，程序员一看就懂，但要对上「为什么是这个数」得想一下：

- `wg_id`(是哪个 `warpgroup`) 凭啥用 `>> 2`? 因为一个 `warpgroup` 刚好 4 个 `warp`。把 `warp` 号「右移 2 位」在二进制里就等于「除以 4」, 商就是它归哪个 `warpgroup`。
- `warp_id`(在组内排第几) 凭啥用 `& 3`? `& 3` 是「取最低两位」, 而两位二进制能表示 0~3, 正好对应它在 `warpgroup` 里的 4 个位置之一。
- `.init()`(初始化屏障) 凭啥变成 `threadIdx.x < 1`? `threadIdx.x < 1` 就是「只有 0 号那一个线程」。换句话说, 这句的意思是「整个 CTA 里只有 0 号线程去做屏障初始化」——设集合点这种事, 一个人去设就够了, 人人都设反而乱套。这条太关键了: 它正好解释了后面 16.9 节那个「屏障初始化被 `wg_id` 守卫坑死」的经典死锁。

16.6.2 读 Python 前, 先 grep 这些字符串

| 在生成 CUDA 里搜                          | 它意味着什么 / 要检查什么                                                |
|--------------------------------------|---------------------------------------------------------------|
| <code>if (threadIdx.x &lt; 1)</code> | 单 CTA 线程守卫, 通常是 <code>barrier</code> 初始化所在处                   |
| <code>mbarrier_init</code>           | <code>barrier</code> 初始化存在, 且应当出现在角色分支之前                      |
| <code>tcgen05</code>                 | Tensor Core 路径确实生成了                                           |
| <code>cp.async.bulk.tensor</code>    | 拷贝确实降级成了 TMA                                                  |
| <code>cta_sync();</code>             | CTA 范围的 <code>barrier</code> ; 它绝不能待在 <code>wg_id</code> 分支里面 |

注意

**注意:** 这套「先 grep 后读」的路子, 等于把一个又大又长的生成 kernel, 变成了一张能导航的地图。你根本不用从头啃到尾, 顺着这几个锚点 (`barrier init`、角色守卫、`tcgen05`、TMA、`cta_sync`) 一路摸过去, 就能落到出问题的那个交接上。

16.7 何时必须重启 Python

这是一条单独的、但要命的纪律, 所以我专门拎出来说:

关键

**关键:**CUDA 出了错, 不会自己收拾干净。只要撞上一次非法内存访问、XID 错误, 或者「CUDA context 被毒化 / `poisoned`」, 那么后面那些跟它八竿子打不着的调用——哪怕你只是来一句 `torch.randn`——都可能跟着一路报错。

这后果很坑: 你要是不重启进程就接着调, 那你调的其实是上回崩溃留下的烂摊子, 根本不是你眼下这份代码。结果就是你对着一个早修好的 bug 来回折腾, 死活想不通。所以——

一碰到非法内存访问 / XID / context 毒化, 马上重启 Python 进程, 再去试下一个 `fix`。

16.8 症状地图 (Symptom Map)

拿到一个跑挂的 run, 先别忙着下结论, 按「现象」快速归个类再说。不过有句话得先打个预防针:

注意

**注意:** 症状只是线索, 不是最终诊断。它顶多帮你圈出「大概率是这一片出事了」, 真正的根因还得回头靠四要素表, 一项一项去对。

| 线索 (现象)                                       | 大概率区域                | 第一步该查什么                                                        |
|-----------------------------------------------|----------------------|----------------------------------------------------------------|
| kernel 挂住, 随后运行时报「unspecified launch failure」 | 死锁                   | barrier init 的位置、到达计数、cta_sync() 的位置、next_tile() 的参与情况         |
| 非法内存访问、XID, 或之后无关的 CUDA 调用也跟着失败               | 崩溃 / context 被毒化     | 重启 Python, 再查指针范围、存储生命周期、集体操作参与情况                              |
| 错误的行以「128 行」或「tile 大小」的条带 (stripe) 出现         | 同步竞争或 tile 索引错位      | 生产者/消费者 phase、scheduler 推进、以及哪个 warpgroup 拥有哪个行条带              |
| 出现 NaN 或明显非法的值                                | 描述符 / 操作数设置 / 累加未初始化 | SMEM/TMEM 描述符设置、swizzle(把数据按特定规律打散摆放以避免 bank 冲突)/layout、累加器初始化 |
| 有限但呈规律性的错误值                                   | 数据过期或仅部分可见           | 缺 fence、缺 TMA store drain, 或存储在生命周期允许之前被复用                     |
| 输出正确, 但没达到预期加速                                | dispatch 或资源问题       | 生成的 CUDA 路径、流水线深度、占用率 (occupancy)、寄存器溢出 (spill)                |

接下来这几节, 就把「死锁 / 崩溃 / 结果错 / 正确但慢」这四类挨个掰开了细讲。

16.9 死锁 (Deadlocks)

先说说什么叫**死锁 / deadlock**: 就是大家都在「等」, 但谁都等不到——A 等 B、B 等 A, 或者一群人在集合点傻等一个永远不会来的人, 于是整个程序卡死在那里、再也动不了。死锁是 warp 专门化 kernel 最家常便饭的故障。建议你照下面的顺序, 一条一条往下排:

① 到达计数 ≠ 初始化计数 (最常见)

这是头号嫌疑犯。先回忆一下: 屏障初始化时会定下「要等几个人到齐」(这个数叫到达计数); 后面线程用 arrive 表示「我到了」, 用 wait 表示「我在这儿等齐」。麻烦就出在「定的人数」和「实际来的人数」对不上。

举个典型例子: `MBarrier.init(128)` 这句话等于放出话来——「我得等满 128 个线程到了才放行」。可实际负责报到的 arrive 却被 `if warp_id == 0: if lane_id == 0:`(意思是「只有 0 号 warp 的 0 号线程才执行」) 这两层守卫给框住了, 到头来真正来报到的只有 1 个线程。这下好了, 这个屏障永远凑不够 128 个人, 在它那儿 wait 的所有线程也就永远等不到放行——卡死。

下面这张表, 把 GEMM 里几个 barrier 的「正确计数」列了出来。拿它跟你的 kernel 比一比, 看对不对得上:

| Barrier                     | init(计数) | 谁来到达                           | 实际到达数 |
|-----------------------------|----------|--------------------------------|-------|
| TMABar(tma→mma)             | 1        | TMA 引擎, 经 arrive(stage, bytes) | 1     |
| TCGen05Bar(mma→tma, mma→ld) | 1        | MMA warp, 经 tcgen05.commit     | 1     |
| MBarrier(ld→mma)            | 128      | WG0 全部线程, 经 arrive             | 128   |

关键

**关键:**128 这个数从哪来的? 它正好是一个 warpgroup 的线程数 (4 warp × 32 lane)。所以 init 计数是 128 的时候, arrive 必须让 WG0 的全体线程都来执行一遍, 绝不能被 lane 或 warp 守

卫栏掉一部分。

## ② 屏障初始化被塞进了「角色守卫」里

回想一下 16.6 节: 初始化屏障的 `.init()` 会被翻译成 `if threadIdx.x < 1;`, 意思就是**只有 CTA 的 0 号线程**才去做初始化。而这个 0 号线程, 偏偏归在 **WG0**(第 0 个 warpgroup) 名下。问题就出在这儿——如果你把初始化代码放进了「只有 WG1 才进得来」的分支:

```
// : WG1
if (wg_id == 1) { // 1 warpgroup
 mbarrier_init(...); // !
} 0 WG0, WG1
```

结果: 集合点压根没被设置好, 后面所有人在它那儿等, 全都白等 → 死锁。所以记牢一句: **所有屏障初始化都得放在最外层 (顶层 / top level)**, 别往任何角色守卫的 `if` 里塞。怎么验? 简单——在存好的生成 CUDA 里 `grep mbarrier_init`, 看看这些初始化语句都蹲在什么位置, 有没有不小心被某个 `if (wg_id == ...)` 包住。

## ③ cta\_sync() 蹲进了 warpgroup 分支里

`cta_sync()` 是一个「全员集合」指令——它要求**整个 CTA 的所有线程**都到齐了才放行 (熟悉 CUDA 的话, 它就等价于经典的 `__syncthreads()`)。既然要求全员到齐, 你就**绝不能**把它放进某个角色分支里。你要是把它放进 `if wg_id == 0:` 里头, 那 WG1 的线程压根走不到这个集合点, 而 WG0 的线程又在傻等 WG1 → 又死锁了。

### 关键

**关键:** 如果你只是想在单个 **warpgroup** 内部 (那 128 个线程之间) 做同步, 用专门的 `T.cuda.warpgroup_sync(10)` (只要求一个 warpgroup 内部到齐), **别拿**要求全员到齐的 `cta_sync()` 来凑。

## ④ 有些消费者 warpgroup 的线程漏掉了 next\_tile()

这里先解释一个角色: **tile scheduler (分块调度器)**。前面说过大矩阵会被切成一个个 `tile`; 调度器就是那个「派活的工头」, 负责告诉每个线程「下一块该处理哪个 `tile`」, 靠的就是 `next_tile()` (领下一块活) 这个调用。关键在于, 它给**每个线程各自记一份状态 (per-thread)**。所以一旦有线程跳过了 `next_tile()`, 它手里的「进度」就跟别的线程对不上号了, 后果往往是某些线程在那儿一直空转、不出来。

## ⑤ 搬数据 (TMA) 和乘法 (MMA) 对「要处理多少块」没数到一块去

矩阵乘法会在一个方向上把数据切成 `K_TILES` 块, 搬数据的循环和乘法的循环都得各跑这么多次、节奏对齐。要是乘法那边的循环次数手滑写成了 `K_TILES - 1` (少跑一次), 两边的进度对暗号 (也就是下面要讲的 `phase`) 就会一点点跑偏 / **drift**, 跑到第二个外层 `tile` 时一下就死锁。



## 注意

**注意:** 搬数据循环和算乘法循环, 都得老老实实迭代 `K_TILES` 次。哪怕少跑一次, 整条流水线的节奏就全乱套了。

## ⑥ 初始「相位 (phase)」设反了

先把 **phase(相位)** 讲清楚, 这是这种流水线里很巧妙的一个机制。

## 一句话先理解

**一句话先理解:** phase 就是生产者和消费者之间的「对暗号开关」, 在 0 和 1 之间来回翻; 靠它, 消费者才能分清「这是上一轮的旧数据, 还是这一轮的新数据」。

具体来说, 屏障的 `wait` 不是「等人到齐」那么简单, 它还会问一句「现在是不是我等的那个 phase?」。利用这一点, 可以巧妙地安排谁先动:

- 生产者从 `phase=1` 起步。这样它第一次 `wait` 直接就过, 马上就能动手生产 (因为还没人生产, 它当然要先干活)。
- 消费者从 `phase=0` 起步。这样它第一次 `wait` 会被卡住, 老老实实等生产者先产出东西来 (东西还没产出, 它当然得等)。

一个先干、一个先等, 流水线才转得起来。你要是让这俩从同一个 `phase` 起步, 那「谁先谁后」就乱了, 第一次交接当场就可能死给你看。

## 16.10 崩溃与 context 毒化

先分清「崩溃」和上一节的「死锁」: 死锁是大家在傻等、程序卡住但没报错; 崩溃则是程序直接出错挂掉, 典型表现是「非法内存访问」(碰了不该碰的显存地址) 或「XID 错误」(NVIDIA 驱动报出来的硬件级错误码)。常见的几个根子:

- 「申请内存」的步骤搞错了顺序。这里要管理一块可分配的内存池 (pool): `pool.alloc` 是「申请一块」, `pool.commit()` 是「我申请完了, 定下来」。规矩是所有申请都得在 `commit()` 之前做完。坑在于: 屏障的包装器内部也会偷偷帮你 `alloc` 一次, 容易让人忽略。所以分配顺序得严格按下面这个来:

```
“text tmem_addr → barrier → move_base_to(1024) → Asmem / Bsmem / Dsmem
→ commit() “
```

只要在 `commit()` (定稿) 之后还想再 `alloc` (追加申请), 事情就坏了。

- `tcgen05.alloc / tcgen05.dealloc` (申请 / 释放 Tensor Core 那块累加器内存 `TMEM`) 上挂了 lane 守卫。这两条指令有个硬性要求: 发它的那个 warp, 32 个 lane (线程) 必须一个不少地一起执行。你要是写成 `if lane_id == 0:` (只放 0 号线程进去), 那就是 未定义行为 / **undefined behavior**——意思是「规范没说这种情况会怎样, 实际表现完全没准, 可能这次没事下次就崩」, 是最难查的一类坑。

## 关键

**关键:** 对一下 16.11 节那个 Step 7 骨架——`tcgen05_alloc / tcgen05_dealloc` 该有 **warp** 守卫 (限定哪个 **warp** 来发), 但绝不能有 **lane** 守卫 (不能再往里挑某几个线程)。

- **释放 TMEM(tcgen05.dealloc)** 之前漏了 `cta_sync()`。后果是: 写结果那边还没把 TMEM 里的数据读完, 这边就抢先把 TMEM 释放了——于是读到的全是垃圾。加一句 `cta_sync()` (全员到齐再往下走) 就能保证「大家都读完了, 才允许释放」。
- **GMEM 或 SMEM 访问越界** (读写了超出数组范围的地址)。把问题缩到单个 tile, 查一查调度器算出来的 `m_idx / n_idx` (也就是「当前在处理第几行块、第几列块」的索引), 再确认当前矩阵的尺寸 (shape) 是 tile 大小的整数倍——尺寸要是除不尽, 边角那块就容易算出越界的地址。

## 16.11 Step 7 参考骨架

原文给了一份「能正常编译运行的 Step 7 kernel」的顶层结构。这东西特别值钱, 你完全可以拿它当「标准答案样板」——把出毛病的 kernel 跟它一段一段地对, 常常一眼就能看出哪儿少了一个守卫、或者哪儿多了一个守卫。下面我把它的骨架摘出来 (只留每一段的核心一两行, 省略了具体实现细节), 一段一段配上中文讲解。你不用看懂每个函数, 重点是体会「哪段代码该被哪种守卫包住」这个结构:

```
// (1) Barrier : , CTA 0
if (threadIdx.x < 1) {
 mbarrier_init(tma2mma[0..1], 1); // TMA->MMA, 1(TMA)
 mbarrier_init(mma2tma[0..1], 1); // MMA->TMA, 1
 mbarrier_init(mma2ld, 1); // MMA-> , 1
 mbarrier_init(ld2mma, 128); // ->MMA, 128(WGO)
}

// (2) TMEM :WGO warp0, warp lane (lane)
if (wg_id == 0 && warp_id == 0) tcgen05_alloc(..., 512);

// (3) fence + cta_sync, phase :producer=1, consumer=0

// (4) Warp
if (wg_id == 1 && warp_id == 3 && elect_sync) { /* TMA */ while(valid){ ... next_tile(); } }
if (wg_id == 1 && warp_id == 0 && elect_sync) { /* MMA */ while(valid){ ... next_tile(); } }
if (wg_id == 0) { /* WB */ while(valid){ ... next_tile(); } }

// (5) : warp, lane
cta_sync();
if (warp_id == 0) { tcgen05_relinquish_alloc_permit(); tcgen05_dealloc(..., 512); }
```

动手改算法之前, 这四点务必先核一遍:

1. **barrier init** 在顶层, 没躲在任何 `wg_id` 守卫里头。
2. `tcgen05_alloc / tcgen05_dealloc` 有 **warp** 守卫、但没有 **lane** 守卫——发指令那个 **warp** 的所有 **lane** 都得到场。
3. **TMA 循环** 和 **MMA 循环** 都跑 `K_TILES` 次 (对应 16.9 节 ⑤)。
4. **phase** 初始值是 `producer=1、consumer=0` (对应 16.9 节 ⑥)。

关键

**关键:** 这个骨架, 几乎就是前面所有死锁 / 崩溃检查项的「正确答案合集」。你就把它当成一份看得见摸得着的 checklist 来用——你的 kernel 每偏离骨架一处, 那一处就先打个问号。

16.12 结果错误 (Wrong Results)

结果算错了, 别瞎猜。先看错误长成什么「样子 / pattern」, 按样子分类, 再倒推原因:

| 错误形态                       | 大概率指向                                            |
|----------------------------|--------------------------------------------------|
| 整行条带 (whole row stripes) 错 | 生产者/消费者 phase、tile 索引、或角色归属 (role-ownership) 不匹配 |
| 输出 NaN                     | 描述符设置、操作数设置、或累加未初始化                              |
| 有限但有规律的错值                  | 消费者读到了旧 tile、部分写入的 tile, 或 store 尚未 drain 的数据    |

下面这几个是经典坑。先垫一个会反复出现的词:`elect_sync()`——它的作用是「从一个 warp 的 32 个线程里只选出一个来干某件事」。有些操作「只该一个人去做」, 如果 32 个人都去做就会出乱子。这正是下面第一个坑:

- `tcgen05.commit` 写到了 `elect_sync` 外面 (也就是没限定只让一个线程做)。这里 `commit` 是「提交一次, 告诉别人: 这一组活我干完了, 你可以接手了」。后果很微妙:32 个线程人人都提交了一次, 其中 31 个提交的是空的 (因为活其实只有一份), 而这些空提交会立马给屏障发出「干完了」的信号。结果就是: 算乘法 (MMA) 那边还没顾得上把 SMEM 里的数据读走, 搬数据 (TMA) 那边就以为「可以覆盖了」, 真把 SMEM 给覆盖掉了 → 算出来的数就错了。

注意

**注意:**`commit` 这种「本该由一个线程发」的操作, 一定要圈在 `elect_sync()` 里头, 保证真正去提交的就那么一个线程。

- 写出数据 (TMA store) 之前漏了 `fence.proxy_async("shared::cta")`。这里要引入 `fence` (内存栅栏) 这个概念:GPU 里不同的硬件部件走的是不同的「访问通道 / proxy」, 一个部件刚写好的数据, 另一个部件不一定马上就能看到;`fence` 就是一道「等一下, 把我刚才写的东西落实到位、让对方也能看见, 再往下走」的命令。漏了这道 `fence`, 线程刚写进 SMEM 的数据,TMA 引擎可能压根看不见, 于是搬出去的是旧数据。
- 写出数据之后漏了 `cp_async.bulk.commit_group() + wait_group(0)`。这里要引入 `drain` (排空 / 等写完): 数据「写出去」这个动作是异步的, 发出去之后不会立刻完成。`commit_group() + wait_group(0)` 合起来的意思就是「等这批写操作真正全部写完」。漏了它, 写还没写完, 下一个 tile 就过来把这块内存 (Dsmem) 抢去存别的东西了 → 数据被中途冲掉。
- 持久化 kernel / persistent kernel 在小尺寸 (比如  $1024\times1024$ ) 上偶尔挂掉。(persistent kernel 是一种「一次启动就一直驻留、自己不断领新活来干」的写法。) 这种现象特别能唬人——尺寸一大, 中间要算的轮数跟着变长, 反倒把那个时间窗很窄的竞争给盖住了, 看着一切正常。这时候重点去查 tile 之间的 phase 复位 (reset), 还有数据写出的 `commit / wait`。

关键

**关键:**「小尺寸偶尔挂、大尺寸看着没事」,这是一个明明白白的**竞争条件 / race condition** 信号——所谓竞争条件,就是「结果取决于两件事谁先谁后,而它俩的先后又没被保证」,所以有时对、有时错。**绝不能拿**「大尺寸看着没事」当成「大尺寸代码就没问题」的证据。

- `fence.after_thread_sync()` 一般不是你寻找的那剂解药。算乘法 (MMA) 完成时用的那个屏障,本身就已经带了「确保前面写的数据后面读得到」的保证 (术语叫 `release-acquire` 语义),所以大多数地方不用再额外加 `fence`。步骤 8 和 9 里加它,纯粹是为了在写回这一段上再稳一点——而且位置卡得很死: 在 `mma21d.wait` 之后、第一次从 `TMEM` 读数据 (`tcgen05.ld`) 之前。千万别手一顺,就把它也加到「搬数据 → 算乘法」那条边上去,那里不需要。

16.13 正确但慢 (Correct but Slow)

输出对是对了,可能性离预期差一大截——这通常意味着「该用的高速硬件没用上,或者三件事没真正并行起来」。这时候就掏出**前面那套「查生成 CUDA」的老办法**,一模一样的招数往下排:

| 线索                                              | 大概率区域                                                   | 第一步该查什么                                                  |
|-------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------|
| 生成的 CUDA 里搜不到 <code>cp.async.bulk.tensor</code> | 搬数据没用上 TMA(走了普通的慢搬法)                                    | 检查 <code>dispatch="tma"</code> 、显卡能力、操作数布局 (layout)      |
| 生成的 CUDA 里搜不到 <code>tcgen05</code>              | 矩阵乘法没用上 Blackwell 的 <code>Tensor Core</code> 指令 (走了慢算法) | 检查 <code>dispatch="tcgen05"</code> 、显卡能力、操作数布局           |
| 搬数据和算乘法没有重叠 (没并行)                               | 流水线开得太浅,或 <code>phase</code> 把生产者/消费者排成了一前一后串着跑         | 在生成的 CUDA 里检查 <code>wait / arrive / advance</code> 的先后顺序 |
| 小尺寸正确性好,但大尺寸特别慢                                 | 寄存器溢出、占用率太低,或暂存空间压力大                                    | 查编译器的资源报告; 减小 <code>tile</code> 、分块写回、或减少流水线层数           |

注意

**注意:** 几个性能术语顺手解释一下。**寄存器溢出 / register spill:** 每个线程能用的寄存器 (最快的那点存储) 有限,放不下就只能把多出来的临时变量挪到慢的内存里,一来一回就拖慢了。**占用率 / occupancy:** 衡量「一块 GPU 上同时塞了多少线程在跑」的指标——塞得太少,GPU 算力就闲着没吃饱。  
再强调一遍: 性能问题和正确性问题,其实用的是**同一套家伙**——存下生成的 CUDA, `grep` 关键路径,核对 `wait / arrive / advance` 的先后。区别只有一处: 这回你盯的是「该并行的有没有并行」「硬件资源够不够」,而不再是「交接对不对」。

16.14 提交一个好的 Issue

上面这些都查过了,故障还赖着不走,那就**先把问题缩到最小,再去 Apache TVM 的 GitHub 仓库提 issue**。一个像样的 issue,得带上这些东西:

- `tvm.__file__ / tvm.__version__` 的输出,加上 GPU 的计算能力;
- 能复现故障的**最小 shape**;
- 故障是哪一类: 编译器 / 死锁 / 崩溃 / 结果错 / 正确但慢;
- **最小的那段 kernel 或 notebook cell**,外带它的正确性检查;

- 你存下来的 `inspect_source("cuda")` 输出，或者一段能把可疑守卫 / `barrier` / `dispatch` 路径亮出来的最小代码片段。

关键

**关键:** 这一节其实就是整章方法的「收尾」。你前面填的四要素表、存的 CUDA、做的归类——这些刚好就是一个好 issue 要的全部料。换句话说，把调试纪律做扎实和把 issue 材料备齐，本来就是同一件事。

小结

这一章不是算法教程，而是一份专门伺候 `warp` 专门化异步 `kernel` 的调试操作手册。它的核心，一句话拎清：把 `kernel` 当成一组「交接」，用「角色 / 存储 / 交接 / 生命周期」四要素表把它写出来，再拿生成的 `CUDA` 一项一项去对。

整章的逻辑兜成了一个圈：

1. 先把环境排掉 (TVM 版本、GPU 是不是 Blackwell)，再跑最小正确性检查。
2. 生成的 `CUDA` 才算数：存盘、`grep` 关键字串、对着 `TIRx`→`CUDA` 映射表看，然后才回头读 `Python`。
3. 绝大多数运行期故障，都是交接坏了：到达计数对不上、`init` 被角色守卫坑了、集体操作被收窄、`phase` 跑偏或初值写反、存储提前被复用。
4. 先按四类症状归类 (死锁 / 崩溃 / 结果错 / 正确但慢)，对着各自的清单查，一次只改一个交接点，改完先重跑正确性。
5. `CUDA` 出错会毒化 `context`：一碰上非法内存访问，马上重启 `Python`。
6. 小尺寸偶尔挂，八成就是竞争条件，别让大尺寸那副「看着没事」的样子把你骗了。

最后再唠叨一句：把 16.11 节那个 Step 7 参考骨架记牢。它差不多就是所有检查项的「正确答案合集」，你的 `kernel` 每偏离它一处，都值得停下来，好好怀疑一下。

延伸阅读

- 原文章节:Debugging Warp-Specialized Kernels
- 前置章节: 本书「用 `Warp` 专门化和 `Cluster` 扩展 `GEMM`(Scaling `GEMM` with `Warp Specialization` and `Clusters`)」中的 `GEMM` 步骤 7-9，是本章调试方法的直接应用对象。
- 工具参考:TIRx 语言参考 (TIRx Language Reference), 用于核对 `API` 名称与 `dispatch` 路径。
- 提交 issue:Apache TVM GitHub 仓库。

术语对照

| 中文       | English             |
|----------|---------------------|
| 线程束      | warp                |
| 线程束组     | warpgroup           |
| Warp 专门化 | warp specialization |
| 角色守卫     | role guard          |
| 交接       | handoff             |

|            |                                |
|------------|--------------------------------|
| 生产者 / 消费者  | producer / consumer            |
| 异步屏障       | mbarrier (async barrier)       |
| 到达计数       | arrival count                  |
| 相位         | phase                          |
| 集体操作       | collective operation           |
| 共享内存       | SMEM (shared memory)           |
| 全局内存       | GMEM (global memory)           |
| 张量内存       | TMEM (tensor memory)           |
| 线程块        | CTA (cooperative thread array) |
| 非法内存访问     | illegal memory access          |
| 上下文毒化      | context poisoning              |
| 竞争条件       | race condition                 |
| 寄存器溢出      | register spill                 |
| 占用率        | occupancy                      |
| 持久化 kernel | persistent kernel              |
| 流水线深度      | pipeline depth                 |

# 第 17 章 · 编译器内部 (概览)

原文: [Compiler Internals](#)

## 本章要点

### 本章要点 (TL;DR)

- 先说清楚: 这一章是个**导航页** (就像一本书的目录页, 本身没多少干货, 只负责把你领到该去的地方)。它放在「参考与附录」里, 不是写给写 kernel 的人看的, 是写给想动手改 **TIRx** 编译器的人看的。
  - 这里先解释一个词:**kernel(核函数)**。它就是一段专门跑在 GPU(显卡) 上的计算程序。你平时写的代码跑在 CPU 上; 但有些活儿 (比如海量数字一起算) CPU 干太慢, 就把这段算法搬到 GPU 上跑, 这段搬过去的程序就叫 kernel。本书的主线就是教你写 kernel。
- 这章现在就指向一个有内容的子页面——**TIRx 下降流水线 / TIRx lowering pipeline**。它只回答一个问题: 你敲下 `tvm.compile(..., tir_pipeline="tirx")` 那一刻, 编译器内部到底干了些什么。(tvm.compile 你就理解成“把我写的高层代码编译成能在 GPU 上跑的东西”的那个按钮。)
- 「下降 / lowering」是啥? 说白了, 就是把你写的那些**高层、好读**的东西, 一层一层翻译成底层、贴近硬件的东西。打个比方, 就像把一句中文先翻成英文、再翻成机器码——每翻一层就更接近机器、离人更远一点。
  - 这里要“下降”的高层东西有三样, 先混个脸熟 (下文都会再细讲): **瓦片原语** (tile = 一个大矩阵切出来的小方块; 瓦片原语就是“一口气搬运 / 计算一整块小方块”的高层指令, 省得你一个数一个数地写)、**带 TileLayout 类型的缓冲区** (缓冲区 = 一块存数据的内存; TileLayout 是给这块内存标注的“摆放方式”)、**执行作用域 id** (GPU 上成千上万个干活的小工同时跑, 这个 id 用来回答“我是第几个工”)。
  - 它们最终被翻译成**两份函数: 主机函数 (host)**——跑在 CPU 上, 负责“发号施令、把活儿丢给 GPU”; **设备函数 (device)**——真正跑在 GPU 上的那段计算。最后由 CUDA 后端吐出源码 (CUDA 是 NVIDIA 显卡的官方编程语言, 你可以把它想成“GPU 版的 C++”)。
- 这套翻译的总入口叫 **LowerTIRx**, 它就干三件事: 把瓦片原语派发成具体的后端实现; 把布局算成真正的地址计算; 把作用域 id 落到 `blockIdx / threadIdx` 上 (这两个是 GPU 内置的变量, 后面会讲, 现在只要知道它们回答“我是第几个工”)。
- 这部分最值钱的地方在哪? 它会教你**只跑流水线的前几步, 然后一阶段一阶段地把 IR 打出来看** (IR = 编译器内部用的“中间代码”, 介于你写的高层代码和最终机器码之间的半成品)。这样编译器每一步干了啥, 你是亲眼看见的, 而不是靠脑补。

前置知识

**前置知识:** 读这一章前, 最好先懂 TIRx 的基础用法 (瓦片原语、缓冲区, 以及 `T.copy/T.gemm` 这种”拷一块数据 / 算一次矩阵乘”的高层写法, 见第 9 章), 还有 `host/device`(主机/设备, 即”CPU 端发令”与”GPU 端干活”这两边) 和 `kernel` 启动 (`launch`, 就是 CPU 喊一嗓子”GPU 开工”, 把一段计算正式发到 GPU 上跑起来) 这些概念。没把握的话, 先翻一下第 0 章·极简入门。本章会默认你已经认识这些词, 所以会反复用到它们而不再从外解释。

17.1 这一页是什么: 写给贡献者的入口

原文这一页的正文很短, 其实就是一个目录 (index)。它在书里归在「第五部分·参考与附录」, 任务很单纯: 把你领到「编译器内部」相关的那几个子页面去。

它跟前面 1-14 章不是一路货。前面那些章教你**怎么用** (怎么写 `kernel`), 这一章讲的是**编译器自己怎么转的** (你写的代码进了编译器之后被怎么处理)。换句话说, 前面是”用户手册”, 这一章是”发动机拆解图”。两边一对比就清楚了:

| 维度   | 主线章节 (Part I-IV)     | 本章「编译器内部」(Part V)      |
|------|----------------------|------------------------|
| 读者   | 想写高性能 GPU kernel 的人  | 想改 / 扩展 TIRx 编译器本身的贡献者 |
| 关注点  | 怎么用 API、怎么排布数据、怎么调性能 | kernel 源码在编译器里如何被一步步翻译 |
| 阅读时机 | 从头顺着读                | 用到了再翻                  |

关键

**关键:** 你就把「编译器内部」当成一份**实现笔记 / 贡献者文档**来看, 别当教程。不读它, 你照样能写出能跑的 TIRx kernel。可一旦你心里冒出一个念头——”我写的这行 `T.copy`, 最后到底被翻译成啥了?”——或者你想往编译器里塞一个新原语, 那这里就是你该来的地方。

「参考」部分一共三个页面, 分工很清楚。你想干啥, 就翻对应那一页:

| 你的需求                                                                                                       | 去哪一页                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 查某个 TIRx 语言特性                                                                                              | <b>TIRx 语言参考 (TIRx Language Reference)</b>                                                                                                |
| 了解编译器内部 (下降流水线)                                                                                            | <b>编译器内部 (Compiler Internals)</b> ← 本章                                                                                                    |
| 调试异步 GEMM(矩阵乘法; 深度学习里最核心、占用算力最多的运算就是它, 所以是 GPU 的”头号主角”)/ FlashAttention (一种把注意力算得又快又省内存的算子) 的挂起、崩溃、结果错误或变慢 | <b>调试 warp</b> (读作”沃普”; 一个 warp = 32 个线程绑在一起的”最小作业班”, 这 32 个线程必须步调一致地一起干活, 见第 0 章) <b>专业化 kernel</b> (Debugging Warp-Specialized Kernels) |

17.2 子页面一览: 目前只有「TIRx 下降流水线」

这一页眼下就列了一个条目:

- **TIRx lowering pipeline / TIRx 下降流水线**

下面给你画张图, 让你一眼看清它在整本书里待在哪儿, 跟旁边那两个参考页又是什么关系。图里最右边那个框你先扫一眼就行: 它说的是经过翻译后,`host` 函数最终变成 C/LLVM(给 CPU 跑的代码),`device` 函数最终变成 CUDA(给 GPU 跑的代码)——下文会反复讲到这条主线。



结构图

分组:

- 第五部分 · 参考与附录: 「TIRx 语言参考 / (查语言特性)」 「调试 warp 专业化 kernel / (排查异步 kernel 问题)」

连接关系:

- 子页面:TIRx 下降流水线 讲清楚 tvm.compile 内部 → host 函数 → C/LLVM / device 函数 → CUDA

<em> 图 17-1: 本章在「参考」部分的位置, 以及它指向的实质内容.</em></p>

注意

**注意:** 导航页本身一点技术细节都没有, 真东西全在子页面「TIRx 下降流水线」里。别急着翻过去——下面 17.3 先给你来一份速览, 看完你心里就有数: 要不要点进去深读。

17.3 速览:TIRx 下降流水线在做什么

这一节是子页面内容的**浓缩版导读**, 先帮你抓住主干, 再决定要不要点进去。想看每个 pass 一个一个怎么讲的, 翻本仓库的第 18 章笔记。

一句话定位

一句话先理解

**一句话先理解:** 你写的高层代码进了编译器后, 会像在流水线上一样, 经过一道又一道工序 (每道工序改一点点), 最后变成能在 GPU 上跑的源码。

先解释两个词, 后面会一直用:

- **模块 (module):** 你写好的那一整坨 TIRx 代码, 打包成的一个对象, 可以整体丢给编译器处理。把它想成”一个待编译的源文件”就行。
- **pass(译作”遍 / 处理趟”):** 编译器里一个只干一件事的小处理步骤。它读进当前代码、改一处、再吐出来。一条流水线就是把几十个这样的小步骤**排好顺序**串起来, 前一步的输出喂给后一步。这跟很多编译器的设计是一样的——把”大改写”拆成许多”小改写”, 每步可单独测试、单独看效果。

搞清这两个词, 这段就好懂了: 你一调用 `tvm.compile(mod, target, tir_pipeline="tirx")`, 后面就连着发生一串事: 你写好的 TIRx 模块 (`mod`), 被丢进一条**排好顺序**的 **pass 流水线** (也就是 `python/tvm/tirx/compilation_pipeline.py` 里那个 `tirx_pipeline`)。这条流水线把你那些高层写法翻成 `host + device` 两份函数, 再扔给 CUDA 后端生成源码。`target` 是”编译目标”, 意思是”我要给哪种硬件编”(这里就是 `"cuda"`, 即 NVIDIA 显卡)。

整体数据流

结构图

连接关系:

- authored TIRx (你写的 TIRx 模块)  $\xrightarrow{\text{BindTarget (绑定目标平台)}}$  tirx\_pipeline (模块级 pass 序列; 内部由 SplitHostDevice 拆出 host/device)
- tirx\_pipeline (模块级 pass 序列; 内部由 SplitHostDevice 拆出 host/device)  $\rightarrow$  host 函数
- tirx\_pipeline (模块级 pass 序列; 内部由 SplitHostDevice 拆出 host/device)  $\rightarrow$  device 函数
- host 函数  $\xrightarrow{\text{host finalize}}$  C / LLVM
- device 函数  $\xrightarrow{\text{device finalize}}$  CUDA

图 17-2: 编译先 `BindTarget` 绑定目标, 再跑模块级 `tirx_pipeline`(host/device 的拆分发生在流水线内部), 最后按函数种类分别做收尾 (finalization) 并渲染:host 出 C/LLVM,device 出 CUDA.

三个值得记住的关键阶段

整条流水线有十几个 pass。它们的名字你现在完全不用记, 但我顺手用大白话点一下, 免得你看到这些词发慌:**narrow dtype**(把数据用更省空间的小类型存, 省内存)、**向量化 vectorization**(把”一个一个数地算”改成”一次算一小批”, 更快)、**循环展开 loop unrolling**(把循环体复制几份铺开, 减少循环本身的开销)、**CSE**(公共子表达式消除, 算过一次的东西不重复算)、还有一堆 **fp8 / bf16 合法化**(fp8、bf16 都是比常规 32 位浮点更小的数字格式;”合法化”就是把它们改写成目标硬件真正支持的形式)。

这些你不用全记。想搞懂 lowering 到底是怎么回事, 盯死下面这三步就够了:

| 关键阶段      | 它把什么变成了什么                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LowerTIRx | 整个下降的核心。它里头分两半:TilePrimitiveDispatch 和 LowerTIRxCleanup。前半把每个瓦片原语 (copy 拷贝 / gemm 矩阵乘 / reduction 归约——归约就是”把一堆数合成一个”, 比如求和、求最大值) 换成你选定的那个后端的具体代码 (同一句 T.gemm, 在不同硬件上要翻成不同的底层实现, 这一步就是按你的硬件挑对实现)。后半跑 LayoutApplier, 干三件事:① 把 TileLayout 类型的缓冲区访问算成真正的内存地址——也就是把”我要第 (i, j) 个元素”这种坐标, 换算成”它在内存第几个字节”的真实地址 (公式 addr = data + elem_offset + layout.apply(coord): 起始地址 + 偏移 + 按布局把坐标折算成位置);② 把缓冲区展平 (把多维数组拉成一条直线的一维内存, 因为底层内存本来就是一条直线);③ 把执行作用域 id(T.cta_id / T.thread_id, 即代码里”我是第几个工”的高层写法) 经过 launch_thread 落到 blockIdx / threadIdx 上。blockIdx / threadIdx 是 GPU 内置的两个变量:GPU 干活时会同时拉起海量线程, 这些线程先分成一个个”线程块 (block)”, 每块里再装一堆线程;blockIdx 告诉每个线程”你属于第几块”,threadIdx 告诉它”你是块里第几个”。靠这两个编号, 每个线程才知道自己该处理数据的哪一小块。 |

|                 |                                                                                                                                                                                                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SplitHostDevice | 它在 launch_thread 这个边界上”下刀”，把一个 kernel 劈成两半。为什么要劈？因为一段 GPU 程序天然就有两部分职责：一部分在 CPU 上”指挥”，一部分在 GPU 上”干活”，编译到最后必须分成两份。劈出来的是 host 函数（跑在 CPU 上，负责算 grid / block——也就是”这次启动要开多少个线程块、每块多少线程”，算好后正式发起启动）和 device 函数（真正跑在 GPU 上的那段计算，在 CUDA 里就是带 __global__ 标记的那个函数体；__global__ 是 CUDA 用来标记”这个函数是给 GPU 跑的”的关键字）。 |
| MakePackedAPI   | 它把 host 函数改写成 TVM 规定的那套统一调用格式（packed-func ABI）。这里的 ABI 是”二进制层面的调用约定”——简单说就是”参数怎么传、按什么格式打包”的硬性规矩；TVM 要求所有函数都按同一套规矩对外暴露，这样上层才能用统一的方式调起它们。改完之后，这个 host 函数就是最终被调起来的那个”启动器”（你在 Python 里一调用，实际跑的就是它）。                                                                                                     |

关键

关键:LowerTIRx 一跑完，模块就”掉回”普通 TIR 了。啥意思？就是说瓦片原语没了,TileLayout 那层间接也没了，作用域 id 也变成真正的线程轴了。换句话说，高层那些花活儿到这一步全翻译完了。再往后的 pass, 无非是在普通 TIR 上做些常规优化和合法化，没别的了。

你能自己把每一步复现出来

子页面里最实用的一招，就是教你只手动跑流水线最前面那几步，然后一阶段一阶段把 IR(编译器的中间代码，前面解释过) 打出来看。为什么这么干？因为整条流水线一口气跑完，中间发生了啥你根本看不见，等于黑盒；而你手动一步一步跑，每跑一步打印一次，就能亲眼盯着代码被怎么改。书里那些 IR 片段，就是这么一步步生出来的。

下面这段代码我逐行讲给你听（你会编程，但这些函数是 GPU 编译器专有的，先不认识很正常）:

```
from tvm.tirx import transform as TT

target = tvm.target.Target("cuda")
(host llvm),
mod = TT.BindTarget(target.with_host("llvm"))(tvm.IRModule({"main": scale}))
mod = TT.LowerTIRx()(mod) # :
print(mod.script()) # TIRx IR,
```

逐行翻译:

- 1. from tvm.tirx import transform as TT——把那些 pass(每个 pass 都是一个能改写代码的工具) 所在的模块导进来，起个短名 TT, 方便后面写。
- 2. target = tvm.target.Target("cuda")——声明”我要编给 NVIDIA 显卡(cuda)”。
- 3. TT.BindTarget(target.with\_host("llvm"))(...)——BindTarget 是流水线的第一道 pass: 把”目标硬件”信息绑到模块上，顺便指定 host 那一半用 llvm 编 (LLVM 是个通用的编译器框架, 这里用它来生成 CPU 端代码)。注意写法 Pass( )(mod):TT.BindTarget(...) 先造出一

个 pass, 后面那个 (`tvm.IRModule({"main": scale})`) 才是把模块喂进去真正跑; `tvm.IRModule({"main": scale})` 就是把名叫 `scale` 的那个 kernel 包成一个模块。

4. `mod = TT.LowerTIRx()(mod)`——只跑核心那一步 `LowerTIRx`(就是上面表里讲的“派发瓦片原语 + 应用布局 + 落地线程轴”), 把结果存回 `mod`。
5. `print(mod.script())`——把当前模块以人类可读的形式打印出来。跟上一部打印的结果一对比, 你就能看见 `LowerTIRx` 到底改了啥。

要是你不关心中间过程, 只想直接看最后生成的 **CUDA 源码**长啥样, 那就别手动一步步跑了, 一把梭把整条流水线跑完:

```
, tirtx
exe = tvm.compile(tvm.IRModule({"main": scale}), target=target, tir_pipeline="tirtx")
print(exe.mod.imports[0].inspect_source()) # CUDA
```

逐行翻译:

1. `tvm.compile(..., tir_pipeline="tirtx")`——这就是开头说的那个“编译按钮”, `tir_pipeline="tirtx"` 指定走 `TIRx` 这条流水线。它会把上面所有 pass 一口气全跑完, 产出一个可执行对象 `exe`。
2. `exe.mod.imports[0].inspect_source()`——从产物里把 `device` 那一份 (也就是真正跑在 GPU 上的部分) 的源码掏出来打印。你会看到一段实打实的 `CUDA`(GPU 版 C++) 代码, 这就是你那段高层 `TIRx` 被翻译到最底层的最终样子。

#### 注意

**注意:**`LowerTIRx()(mod)` 这种“单跑一段”的玩法, 是你排查问题、摸清编译器脾气的利器。想看哪一步的结果, 就在那一步前后各 `print(mod.script())` 一次, 前后一对比, 变了什么一目了然。

## 17.4 建议的阅读顺序

1. 先看看你是不是这章的目标读者。要是你只想写写 kernel、调调性能, 这部分先跳过没关系。等哪天你心里冒出“怎么编译出来是这副样子”的疑问, 再回头看也来得及。
2. 从这一页 (本章) 进, 先把一件事搞清楚:「编译器内部」眼下就讲了下降流水线这一个主题, 别的还没有。
3. 认真读子页面「`TIRx` 下降流水线」(也就是本仓库第 18 章笔记)。把 pass 表按顺序过一遍, 重点啃这三个关键阶段:`LowerTIRx`(尤其是里头的 `TilePrimitiveDispatch` 和 `LayoutApplier`)、`SplitHostDevice`、`MakePackedAPI`。
4. 自己动手跑一遍。拿原文那个就一行的 `scale` kernel (`scale =` 把每个数乘上一个倍数, 够简单, 适合拿来观察), 自己走一遍 `BindTarget`  $\rightarrow$  `LowerTIRx`, 每跑一步就 `print(mod.script())`, 把脑子里的“概念”跟屏幕上的“真实 IR”一一对上。看一遍, 胜过读十遍。
5. 要用了再横跳。要排查异步 kernel 的问题, 就转去「调试 warp 专业化 kernel」; 只是查个语言特性, 就转去「`TIRx` 语言参考」。

小结

- 第 17 章自己就是「参考与附录」里的一个导航页, 写给编译器贡献者, 不是给写 kernel 的人看的。
- 它眼下就指向一个有内容的子页面:**TIRx 下降流水线**, 把 `tvm.compile(..., tir_pipeline="tirx")` 内部那一整套翻译过程讲清楚。
- 下降是怎么回事, 一句话: 把**瓦片原语** + **TileLayout 缓冲区** + **执行作用域 id**, 过一串排好顺序的 TIR pass, 翻成 **host(启动器)** 和 **device(\_\_global\_\_ 体)** 两份函数, 最后渲染成 CUDA。
- 最关键的三个阶段:**LowerTIRx**(派发原语 + 应用布局 + 落地线程轴)、**SplitHostDevice**(主机/设备拆分)、**MakePackedAPI**(打包 ABI)。
- 最实用的一招: **只跑流水线前几步, 再 `print(mod.script())`**, 这样你就能一阶段一阶段”看见”编译器到底在干什么。

延伸阅读

- 原文 (本导航页):<[https://mlc.ai/modern-gpu-programming-for-mlsys/tirx\\_guide/arch/index.html](https://mlc.ai/modern-gpu-programming-for-mlsys/tirx_guide/arch/index.html)>
- 子页面「TIRx 下降流水线」: 对应本仓库第 18 章笔记; 原书路径在同一 `tirx_guide/arch/` 目录下。
- 完整的 `tvm.tirx` Python API, 请参阅上游 TVM 官方文档。
- 相关参考页: 「TIRx 语言参考」「调试 warp 专业化 kernel」(均在本书「参考与附录」部分)。

术语对照

| 中文          | English                         |
|-------------|---------------------------------|
| 下降流水线       | lowering pipeline               |
| 瓦片原语        | tile primitive                  |
| 主机函数 / 设备函数 | host function / device function |
| 执行作用域 id    | execution-scope id              |
| 收尾 (阶段)     | finalization                    |
| 打包函数 ABI    | packed-func ABI                 |
| 布局应用器       | LayoutApplier                   |
| 编译目标        | target                          |



# 第 18 章 · TIRx Lowering 流水线

原文:[TIRx lowering pipeline](#)

## 本章要点

### 本章要点 (TL;DR)

- 你写的 TIRx 代码很” 高层”(只说大概要干啥, 不抠硬件细节), 但 GPU 这块芯片看不懂这种高层写法。TIRx 流水线 / tirx pipeline 就是一条” 翻译流水线”, 负责把你写的高层代码一步步翻译成 GPU 真能执行的样子。
- 整条路径其实很顺: 手写的 TIRx → (告诉它目标芯片是谁) → 跑一长串翻译步骤 → 把代码拆成 host(在 CPU 上跑的部分) 和 device(在 GPU 上跑的部分) → 两部分各自做收尾 → 最终吐出 C/LLVM 代码和 CUDA 代码。
- 这里先解释两个词: host(主机) = CPU 那边, 负责” 指挥” —— 准备数据、下命令让 GPU 开工; device(设备) = GPU 那边, 负责” 干苦力” —— 真正的大规模计算。一段任务往往是 CPU 先做准备、再把重活甩给 GPU, 所以代码天然分成这两半。
- 流水线一共 18 个 pass(pass = 编译器里的” 一道处理工序”, 每个 pass 读入代码、改一改、再交给下一个), 从最核心的 LowerTIRx 开始, 后面依次做: 合并线程绑定、化简、缓冲区扁平化、数据类型合法化、向量化、循环展开、消除重复表达式、拆分 host/device、生成调用接口等。这些名词现在看不懂没关系, 后面会一个个用大白话讲。
- 其中 LowerTIRx 自己又拆成两小步: 第一步把高层算子 (tile 原语) 展开成具体实现, 第二步把” 数据该摆在内存哪个位置” 的抽象描述算成真实的内存地址, 并把” 谁来执行这段活” 翻译成 GPU 听得懂的 blockIdx / threadIdx(后面会讲这俩是啥)。
- 一个很实用的点: 你可以只跑流水线的前半段 (比如只跑到 LowerTIRx), 然后用 mod.script() 把中间结果打印出来看一眼。本书里那些中间代码片段, 都是这么生出来的。

## 前置知识

前置知识: 这一章会用到几个词, 这里先一句话说明, 你心里有个底就行——

- TIR: TVM 这个编译框架里的一种” 中间表示”, 可以理解为介于” 你写的高层代码” 和” 最终机器码” 之间的一种代码形态。
- pass: 编译器里的一道处理工序 (上面解释过了)。
- host / device: CPU 端 / GPU 端 (上面解释过了)。
- grid / block / thread (网格 / 块 / 线程): GPU 干活时不是一个人埋头算, 而是召集成千上万个 thread (线程) 同时算; 这些线程会被分成一个个 block (块); 所有块合起来叫一个 grid (网格)。把它想成” 军队”: 一个士兵是 thread, 一个班是 block, 整支队伍是 grid。

没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 17 章。本章会默认你已经认识这几个词, 后

面用到时不再从头解释。

## 18.1 这条流水线解决什么问题

先看看我们手上拿着什么。在 TIRx 里, 你写的代码站得很“高”——只说大意, 不抠细节。

### 一句话先理解

**一句话先理解:** 你要写的是一个 **kernel**。kernel 就是“一段要在 GPU 上跑的函数”, 但它的特别之处是: 它不是被调用一次执行一次, 而是被成千上万个线程**同时各跑一遍** (每个线程算其中一小块数据)。这是 GPU 干活的基本方式——人海战术。

描述一个 kernel, 你拿三样东西就够了:

- 用 **tile 原语 / tile primitive** 说清楚“要算什么”。tile 原语就是一组现成的“积木操作”, 比如 **copy**(搬数据)、**gemm**(矩阵乘——也就是两个矩阵相乘; 它是深度学习里出现最频繁、最吃算力的运算, 所以 GPU 里它是绝对主角)、**reduction**(把一堆数归并成一个, 比如求和、求最大值)。你直接拿这些积木拼, 不用自己从头写循环。
- 用带 **TileLayout** 类型的缓冲区说清楚“数据怎么摆”。**缓冲区**你就理解成“一块数组/内存”; **TileLayout** 则是给这块数据贴的一张“摆放说明”, 描述数据在内存里按什么规律排列 (后面会讲为什么 GPU 很在乎这个)。
- 用 **执行域 id / execution-scope id** (像 `T.cta_id`、`T.thread_id`) 说清楚“谁来干”。说白了就是给每个线程发个工号, 让它知道“我是第几号, 该负责哪一小块”。

这么写当然省事, 可问题来了: GPU 根本看不懂这套高层积木。GPU 后端 (就是最后把代码变成 GPU 能跑的东西的那个部件) 只认**最朴素的 TIR**——也就是: 具体到每根线程轴、把多维数组压成一维的内存访问、以及它原生就支持的那几种数据类型。换句话说, 它只吃“嚼碎了的、贴着硬件的”代码, 不吃你那套高层抽象。

那怎么办? 这就得靠 **lowering(下降) 流水线**了。先说“lowering”这个词: 它字面是“降低/下降”, 在编译里专指“把高层、抽象的代码, 一级一级往低层、贴硬件的方向翻译”。为什么叫“下降”? 因为你可以把代码想象成有“楼层”: 越高层越抽象、越接近人的思维, 越低层越具体、越接近机器。lowering 就是带着代码一层层往下走楼梯, 直到走到硬件那一层。

所以这条流水线干的活, 就是把上面那套高层写法一步步“压”下去, 压成 GPU 跑得动的朴素形式。入口特别简单, 就一行:

```
tvm.compile(mod, target, tir_pipeline="tirx") # tirx
```

这一行背后发生了什么? 它把你手写的 TIRx 模块塞进 **tirx 流水线**, 过一遍那一长串处理步骤, 把它劈成 **host(主机, CPU 端)** 和 **device(设备, GPU 端)** 两类函数, 最后丢给 CUDA 后端去吐源码。(CUDA 是 NVIDIA 给自家 GPU 编程用的那套语言/工具, 你可以把“生成 CUDA 源码”理解成“生成 GPU 真正能编译执行的代码”。)

### 关键

**关键:** 流水线的定义在 `python/tvm/tirx/compilation_pipeline.py` 里, 函数名叫 `tirx_pipeline`。说白了, 看懂这条流水线, 就等于看懂了“高层的 tile 写法是怎么一步步变成



CUDA kernel 的”。它是整个 TIRx 编译栈的主干。

## 18.2 它处在编译流程的哪个位置

`tvm.compile` 干的活, 拆开看就是四步:

1. **绑定目标 / BindTarget**: 先跟模块交代清楚”咱们要往哪种硬件上编”——比如计算部分的目标是 `cuda(NVIDIA GPU)`, 指挥部分 (`host`) 用 `llvm`(一套通用的编译器后端, 这里就当它是”把代码编成 CPU 能跑的东西”的工具)。为什么要先说清楚目标? 因为后面每一步翻译, 都得先知道”最终是给谁跑的”, 才知道该往哪个方向翻——给 GPU 翻和给 CPU 翻, 做法完全不同。
2. **跑 tirx 流水线**: 就是 18.3 节要一个个列出来的那一长串步骤。
3. **给 host / device 函数各做各的收尾 (finalization)**: `finalization` 就是”末段收尾工序”。注意, `host` 和 `device` 用的是两套不一样的收尾步骤——因为 CPU 端和 GPU 端要处理的事情根本不一样。
4. **把每个 device 函数交给 CUDA 代码生成器**, 生成最终给 GPU 跑的源码。

那为什么 `host` 和 `device` 要分开收尾? 别急, 下面这张图先把整条主干画出来, 看完你就明白了。

### 结构图

连接关系:

- `tirx_pipeline` / (18 个模块级 pass)  $\rightarrow$  `host func` / (主机启动器)
- `tirx_pipeline` / (18 个模块级 pass)  $\rightarrow$  `device func` / (设备 kernel)
- `host func` / (主机启动器)  $\xrightarrow{\text{host finalize / 主机收尾}}$  C / LLVM
- `device func` / (设备 kernel)  $\xrightarrow{\text{device finalize / 设备收尾}}$  CUDA 源码

### 注意

**注意:** 这里有个容易误会的点——`host` 和 `device` 不是一上来就分成两家的。流水线跑到 `SplitHostDevice` 这一步 (也就是 18.3 节的第 15 个 pass) 之前, 整个 kernel 一直是一个函数, CPU 部分和 GPU 部分还混在一起; 到了这一步, 它才被”劈”成 `host` 和 `device` 两半, 然后各回各家、各做各的收尾。收尾为什么要”按函数种类分着跑”, 原因就在这——分家之后, 两边要做的事不一样了。

下面这张图讲的是同一件事, 只是画得更紧凑、横着排:

### 结构图

连接关系:

- `authored TIRx`  $\xrightarrow{\text{BindTarget}}$  `tirx_pipeline`
- `tirx_pipeline`  $\rightarrow$  `host func`
- `tirx_pipeline`  $\rightarrow$  `device func`
- `host func`  $\xrightarrow{\text{host finalize}}$  C / LLVM
- `device func`  $\xrightarrow{\text{device finalize}}$  CUDA

18.3 流水线的 18 个 pass(逐个看)

tirx\_pipeline 会严格照下面这个顺序，一个接一个地把这些步骤施加上去——就像一条工厂流水线，工件从第一个工位传到最后一个工位，每个工位干一道活。其中有几个步骤能用 PassContext 的开关关掉 (PassContext 你就理解成“编译时的一个配置开关箱”，用到了会说)。先拿一张表整体扫一眼，不用记，有个印象就够了，关键的几个后面会单独展开讲。

| #  | Pass 名称             | 它做什么                                                | 备注                          |
|----|---------------------|-----------------------------------------------------|-----------------------------|
| 1  | LowerTIRx           | 核心下降——把 tile 原语和布局抽象解析掉 (详见 18.4 节)                 | 流水线的“重头戏”                   |
| 2  | UnifyThreadBinding  | 合并等价的线程轴绑定，保证每个 threadIdx / blockIdx 轴只声明一次         | 去重                          |
| 3  | StmtSimplify        | 语句级算术化简 (由 arith 分析器 / arith analyzer 完成)           | 第一次化简                       |
| 4  | LowerTIRxOpaque     | 把剩余的不透明 (opaque)TIRx 结构下降为朴素 TIR                    | 收尾性下降                       |
| 5  | FlattenBuffer       | 把多维的 BufferLoad / BufferStore 扁平化为一维                | 多维 → 1D                     |
| 6  | BF16ComputeLegalize | 把 bfloat16 计算改写成合法形式 (上转为 f32 再算)                   | 计算合法化                       |
| 7  | NarrowDataType(32)  | 在能证明安全的地方，把算下标、循环计数用的数字从 64 位缩成 32 位                | 省资源、更快                      |
| 8  | VectorizeLoop       | 把标了 T.vectorized 的循环变成“一次处理一批数据”的向量操作               | 配 tir.disable_vectorize 时跳过 |
| 9  | UnrollLoop          | 展开标了 T.unroll 的循环 (以及次数很小的循环)——把循环体直接复制几份铺开         | 循环展开                        |
| 10 | StmtSimplify        | 再次化简——此时向量化/展开已暴露出新的常量                              | 第二次化简                       |
| 11 | CommonSubexprElim   | 公共子表达式消除 (CSE)，把重复子表达式提到临时变量                        | 配 tir.disable_cse_tir 时跳过   |
| 12 | FP8ComputeLegalize  | 把 float8 计算改写成合法形式                                  | 计算合法化                       |
| 13 | VerifyMemory        | 安全闸门：检查 host 端代码没有直接解引用 device 内存                   | 校验，不改写                      |
| 14 | AnnotateEntryFunc   | 把那个唯一的 PrimFunc 标记为模块入口                             | 标注                          |
| 15 | SplitHostDevice     | 在 launch_thread 边界把每个 kernel 拆成 host 函数 + device 函数 | 关键分水岭                       |
| 16 | MakePackedAPI       | 把 host 函数改写成打包函数 (packed-func)ABI，即 TVM 调用的启动器      | 生成调用接口                      |
| 17 | FP8StorageLegalize  | 合法化 float8 的存储 (打包进受支持的容器类型)                        | 存储合法化                       |
| 18 | BF16StorageLegalize | 合法化 bfloat16 的存储                                    | 存储合法化                       |

## 几个”为什么这么排”的小细节

这串步骤的排法可不是随手摆的。挑几个最值得琢磨的点说说：

先解释这里会反复出现的两个优化词，后面看着就不懵了：

- **向量化 (vectorize)**: GPU(和现代 CPU) 支持”一条指令同时处理一批数据”——比如本来要分 4 次给 4 个数各加 1, 向量化之后一次就把这 4 个数一起加完。把它想成”打包批处理”, 能少发好几条指令, 自然更快。
- **循环展开 (unroll)**: 把一个循环体直接复制几份铺平, 省掉”判断要不要再转一圈”的开销。比如一个跑 4 次的循环, 展开后就变成把循环体原地写了 4 遍、没有循环了。代价是代码变长, 所以一般只对次数小的循环这么干。

好, 接着看排序里的讲究:

- **化简 (StmtSimplify) 为什么来两遍?**(第 3、第 10) 不是手抖写重了。第一遍, 是把 LowerTIRx 翻完之后留下的一堆零碎算术清理干净; 第二遍, 是专门收拾向量化(第 8)和循环展开(第 9)之后又冒出来的那些常量和能化简的结构(展开循环时, 原来的循环变量会被换成一个个具体数字, 于是又出现了一批可以提前算好的式子)。说白了就是: **先把东西摊开, 再统一打扫一遍**——编译器排步骤, 常用这个套路。
- **数据类型合法化为什么拆成”计算”和”存储”两拨, 中间还隔得老远?** 先说”数据类型合法化”是干嘛的: `bfloat16`、`float8` 是两种”位数很少”的小号浮点数(分别是 16 位、8 位; 用得少的位数换更省内存、更快, 深度学习里很流行)。但很多 GPU 并不直接支持拿它们做加减乘除, 所以编译器得把这些运算”改写”成芯片真支持的形式——这就叫合法化 (legalize, 即”改成合法、能跑的样子”)。它又分两种: **计算合法化**(第 6、第 12, 改的是”怎么算”, 通常是先转成普通的 32 位浮点算完再说)放在前头, 因为改写计算会带出新的算术, 得留给后面的化简、向量化接着收拾; **存储合法化**(第 17、第 18, 改的是”怎么在内存里存放这些小数”)则压到**最后**, 等 host/device 拆完才动手——存储这事更贴近”数据在内存里到底怎么摆”的物理细节, 属于收尾性质的活, 搁最后最合适。
- **NarrowDataType(32)(第 7) 是个性能优化**。先补一句背景: 程序里算数组下标、数循环次数, 用的都是整数; 整数有 64 位和 32 位之分, 位数越大越占地、越慢。GPU 上这类算下标的活极多, 只要 32 位装得下, 就能省下宝贵的**寄存器 (register)**——线程私有的、速度最快的一小块存储, 相当于 CPU 的寄存器, 但 GPU 上每个线程各有一份、数量很有限, 所以特别金贵)、跑得更快。但它守着一底线: **只有在能证明绝对安全的时候才缩**。不然万一数字超出 32 位能表示的范围(溢出), 下标就算错了, 直接读写到不该碰的内存(越界), 程序就崩了。这就是典型的”能优化就大胆优化, 但兜底必须有、还得证得出来”。
- **VerifyMemory(第 13) 只检查、不改代码**。它是一道安全闸门 (safety gate)。这里要先理解一件 GPU 的硬规矩: CPU 和 GPU 各有各的内存, CPU 的代码不能直接去读写 GPU 的内存(反之亦然), 想交换数据得走专门的通道。VerifyMemory 就趁 host/device 还没拆开, 盯着一件事: host(CPU) 代码有没有违规地直接去碰 device(GPU) 的内存。为什么非得放在 SplitHostDevice(分家那一步) 前面? 就为了**早点把问题拦下来**——在还能看清全貌的时候就把违规揪出来, 别等分完家、出了事才追悔。
- **有两个步骤能用开关关掉**: VectorizeLoop(对应开关 `tir.disable_vectorize`) 和 CommonSubexprElim(对应 `tir.disable_cse_tir`), 都能靠 PassContext 跳过。调试的时候, 或者你想亲眼对比一下”

开和关到底差在哪”，这俩开关特别趁手。

Finalization(收尾)pass——按函数种类分别跑

前面提过,SplitHostDevice 把函数劈成 host 和 device 两类。劈完之后,收尾阶段会给它俩各上一套不一样的步骤(下表里 → 表示”先做这个再做那个”):

| 函数种类       | 收尾 pass 序列                                      | 说明                                                                           |
|------------|-------------------------------------------------|------------------------------------------------------------------------------|
| host(主机)   | LowerTVMBuiltin → LowerIntrin                   | 先把 tvvm_* 这类 TVM 自己的内建函数 (builtin) 翻成实际实现,再把”针对具体硬件的特殊指令”(intrinsic) 落到目标平台上 |
| device(设备) | LowerWarpMemory → StmtSimplify<br>→ LowerIntrin | 先把”warp 范围内共用的缓冲区”翻成 shuffle 指令(下面解释),再化简一遍,最后同样把硬件特殊指令落地                    |

这里出现了两个词:builtin(内建函数) 指 TVM 框架提供的一些标准辅助函数;intrinsic(内建指令) 指某款硬件特有的、能一条指令干一件特殊事的指令。把它们”下降”就是把这些占位的高层调用,换成目标平台上真正能跑的东西。

关键

关键:LowerWarpMemory 这一步是 device 端独有的,这里得先把 warp 讲清楚。GPU 里那一大堆线程,执行时并不是各自完全独立,而是 32 个一组捆在一起、迈同一步同时执行,这一组 32 个线程就叫一个 warp(线程束)。打个比方:warp 就像一个 32 人的方阵,口令一响,32 个人必须同时迈同一只脚——这是 GPU 硬件层面的执行单位。

既然一个 warp 里 32 个线程总在一起行动,它们之间常常需要互相交换数据。LowerWarpMemory 干的,就是把”这 32 个线程共用的一块小缓冲区”翻成 shuffle 指令。shuffle 是个啥?说白了,就是同一个 warp 里的线程,在寄存器 (register——前面讲过,线程私有、最快的那层存储) 这一层直接把数据递给隔壁线程,不用先写进 共享内存 (SMEM)、再读出来绕一大圈。

顺便把内存的”快慢分层”也点一下 (这是 GPU 的一个核心常识):GPU 的存储是分层的,越靠近线程越快但越小——寄存器最快 (线程私有)、共享内存 SMEM 次之 (同一个 block 里的线程共用的一小块片上内存,可以理解成一块”由你手动管理的高速缓存”)、全局内存最慢但最大 (所有线程都能访问的大仓库)。所以”能在寄存器层面直接互递,就别绕道 SMEM”——这就是 shuffle 快的原因,也是 GPU 上很常见的高效数据交换招数。host 端没有这一步,因为 CPU 上压根就没有 warp 这回事。

18.4 深入 LowerTIRx: 核心下降是怎么发生的

第 1 个步骤 LowerTIRx 是整条流水线的”重头戏”,绝大部分翻译活儿都堆在这里。它自己其实又是两个小步骤接在一起跑,代码在 src/tirx/transform/lower\_tirx.cc:

```
LowerTIRx pass
LowerTIRx = Sequential([TilePrimitiveDispatch, LowerTIRxCleanup])
```

逐行说一下这段在干嘛:`Sequential([...])` 的意思是”把方括号里这几个 pass 串成一条小流水线, 从左到右依次执行”。所以这一行就是说:`LowerTIRx` = 先跑 `TilePrimitiveDispatch`, 再跑 `LowerTIRxCleanup`。这两步分工很清楚:

## (1) `TilePrimitiveDispatch`——把高层算子换成具体实现

### 一句话先理解

一句话先理解: 你写 `gemm` 的时候, 只是写了”这里要做个矩阵乘”, 就像写下一个函数名; 但这个函数还没有”身体”, 它具体怎么算、用哪条 GPU 指令, 得这一步来填。

你写的每个 `copy`、`gemm`、`reduction`(它们在底层都是一种叫 `TilePrimitiveCall` 的东西, 就是”调用一个 tile 原语”), 其实只是个”调用”, 并没有真身。这一步干的, 就是把每一个这样的调用, 换成它在当前这块 GPU 上的真实函数体。具体两件事:

- **变体选择 / variant selection:** 同一个算子, 换块 GPU 可能有好几种写法 (不同型号的 GPU, 擅长的指令不一样)。这一步挑出最合适当前硬件的那一种。比方说 `gemm`, GPU 架构不同, 跑得最快的指令路径就可能不同——新卡上可能有专门加速矩阵乘的指令, 老卡上就只能用普通做法。
- **代码生成 / codegen:** 把上一步选中的那个实现, 展开成真正的、一条条的 TIR 语句。

打个程序员熟悉的比方: 这就像把一个函数调用做 **inline 内联展开**——不再”调用某函数”, 而是把那个函数的代码体直接原地铺开, 而且铺的是”这台机器上跑得最快的那一版”。

## (2) `LowerTIRxCleanup`——用 `LayoutApplier` 把抽象落地

这一步会跑 `LayoutApplier`(布局施加器)。前面那些虚的、抽象的东西, 到这儿就得全变成实打实的细节。它做三件事:

1. **把布局访问换成真实的地址计算:** 回想一下, 前面你访问一个带 `TileLayout` 的缓冲区时, 只写了个坐标 (比如”我要第 (3, 5) 个元素”), 并没有说”这个元素在内存的第几个字节”。但内存其实是一条一维的长带子, 只认”从头数第几个”这种线性地址。所以这一步要做的, 就是把你写的坐标, 按那张”摆放说明”(TileLayout) 算成具体的内存地址。核心公式是:

“`text addr = data + elem_offset + layout.apply(coord)`”

公式不用背, 记住是三样东西相加就行: 基址 `data`(这块数据从内存哪儿开头)+ 元素偏移 `elem_offset`(这块缓冲区在更大那块内存里的起点)+ `layout.apply(coord)`(把你给的坐标喂给那张摆放说明, 算出”再往后挪多少个”)。三样加起来, 就是这个元素在内存里的确切落点。一句话:`TileLayout` 那层”绕来绕去”的抽象, 在这一步被彻底拆光, 变成干干净净的地址加法。

1. **缓冲区扁平化:** 把多维缓冲区 (比如二维的”矩阵”) 压成一维 (一条长带子), 这样就能直接按上面算出来的地址去取数。
1. **把执行域 id 变成真实线程轴:** 还记得前面给每个线程发的”工号”(T.cta\_id、T.thread\_id) 吗? 它们现在靠 `launch_thread` 翻成 GPU 真正认识的内置变量 `blockIdx`、`threadIdx`。这俩是 GPU 里每个线程都能读到的”我是谁”——`blockIdx` 告诉线程”我属于第几个 block”, `threadIdx` 告诉它”我在 block 里是第几号”。线程就是靠这两个号, 算出自己该处理哪一块数据的。

下面这张图, 把 `LowerTIRx` 这两步”吃进去什么、吐出来什么”画清楚了:

### 结构图

连接关系:

- 手写 TIRx: / tile 原语 + TileLayout 缓冲区 + 执行域 id → TilePrimitiveDispatch / 变体选择 + codegen / (原语 → 具体实现)
- TilePrimitiveDispatch / 变体选择 + codegen / (原语 → 具体实现) → LowerTIRx-Cleanup / LayoutApplier / 布局 → 地址算术 / 缓冲区扁平化 / scope id → blockIdx/threadIdx
- LowerTIRxCleanup / LayoutApplier / 布局 → 地址算术 / 缓冲区扁平化 / scope id → blockIdx/threadIdx → 朴素 TIR: / 无 tile 原语 · 无 TileLayout 间接 · scope id 已解析为线程轴

### 关键

**关键:**LowerTIRx 一跑完, 模块就成了朴素 TIR——tile 原语没了, TileLayout 那层抽象没了, 执行域 id 也全落成了真实线程轴。后面那 17 个步骤对着的, 都是这种”已经落了地”的 TIR。也正因为前面把 TIRx 特有的那些东西都消化干净了, 后面才能直接拿 TVM 现成的通用步骤(化简、向量化、CSE 这些) 来用, 不用再单独为 TIRx 另写一套。

## 18.5 一个完整的小例子:scale kernel 的下降全程

前面概念铺得有点多, 这里拿一个最简单的例子, 把它们都串起来——一个只有一行计算的缩放 (scale)kernel, 功能简单到一句话: **把数组里每个数乘以 2**。我们就看它怎么一路从高层 TIRx 变成 CUDA。

### 起点: 手写的 TIRx

```
@T.prim_func
def scale(A_ptr: T.handle, B_ptr: T.handle):
 A = T.match_buffer(A_ptr, (256,), "float32") # "256 float32 " A
 B = T.match_buffer(B_ptr, (256,), "float32") # , B
 T.device_entry() # : GPU
 bx = T.cta_id([1]) # 1 block();bx
 tx = T.thread_id([256]) # 256 ;tx
 B[tx] = A[tx] * T.float32(2.0) # : tx B tx
```

逐行说一下这些没见过的写法:@T.prim\_func 是个装饰器, 标明”下面这个函数是一段 TIR 代码”(装饰器你写 Python 应该熟, 就是给函数加个标签/包装)。T.match\_buffer 把一个裸句柄 (T.handle, 可以理解成一个指向内存的指针) 绑成一个有形状、有类型的数组——这里是 256 个 float32(32 位浮点数)。T.cta\_id([1]) / T.thread\_id([256]) 就是前面说的”发工号”: 一共 1 个块、256 个线程。

最关键的是最后一行:B[tx] = A[tx] \* 2.0。注意这里**没有循环**! 因为有 256 个线程, 每个线程的 tx 不一样 (0 到 255), 它们**同时各算一个**——0 号线程算 B[0], 1 号算 B[1]……256 个数一瞬间全算完。这就是 GPU”人海战术”的写法: 你只写一个线程该干的活, 硬件帮你复制 256 份并行跑。

看一眼此刻的状态:bx、tx 还是**抽象的工号 (执行域 id)**, 还没变成 GPU 真认识的线程轴;A、B 也还是带类型、带形状的数组视图。接下来, 咱们就盯着这几样东西, 看它们怎么一步步落地。

经过 LowerTIRx 之后:scope id 变成真实线程轴, 布局已施加

```
with T.launch_thread("blockIdx.x", 1) as blockIdx_x: # cta_id blockIdx.x
 threadIdx_x = T.launch_thread("threadIdx.x", 256) # thread_id threadIdx.x
 bx: T.let = blockIdx_x # bx
 tx: T.let = threadIdx_x # ,tx threadIdx_x
 B_1[threadIdx_x] = A_1[threadIdx_x] * T.float32(2.0) # A_1/B_1 ()
```

T.launch\_thread("threadIdx.x", 256) 这行的意思是:”启动 256 个线程, 每个线程有个内置编号 threadIdx.x, 从 0 到 255”。这正是前面讲的 blockIdx / threadIdx——线程读自己的编号、对号入座算自己那份。

对比起点, 变了三处:(1) 抽象工号被 launch\_thread 裹成了 GPU 真认识的线程轴;(2) A、B 这两个数组被压成了一维视图 A\_1、B\_1(就是前面说的”缓冲区扁平化”);(3) 而那行核心计算逻辑一动没动, 还是”乘以 2”。前面念叨的那个”落地”, 这会儿全都兑现了——抽象的东西变实了, 要算的事没变。

经过 SplitHostDevice + MakePackedAPI 之后: 一个函数裂成两个

```
@I.ir_module
class Module:
 def main(...): # host: API (grid/block, launch)
 ...
 def scale_kernel(...): # device: GPU __global__
```

原来那个 scale 函数, 现在劈成了两个:

- **main(host):** 在 CPU 上跑的”指挥官”。它负责把 grid(网格, 所有块的总集合) 和 block(块, 一组线程) 的规模配置算好——也就是”开几个块、每块几个线程”——然后下命令让 GPU 启动这个 kernel。它对外用的是 TVM 的打包函数调用接口 (一种标准的”怎么调这个 kernel”的约定)。
- **scale\_kernel(device):** 真正在 GPU 上跑的那部分, 马上就要被生成成 \_\_global\_\_ 函数。(\_\_global\_\_ 是 CUDA 里的一个关键字, 标记”这个函数是从 CPU 那边发起、在 GPU 上执行的入口函数”——你之后在 CUDA 代码里看到它, 就知道”这是一个 GPU kernel 的入口”。)

终点: CUDA 后端生成源码

最后一步, CUDA 后端把 scale\_kernel 生成成 \_\_global\_\_ 函数, 那行核心计算就成了地道的 CUDA:

```
B_ptr[threadIdx.x] = A_ptr[threadIdx.x] * 2.0f;
```

看, 绕了这么一大圈, 落地后其实朴素得出奇——就是普通的数组下标 + 乘法。threadIdx.x 是 GPU 自动塞给每个线程的编号, 2.0f 里的 f 表示”这是个单精度 (32 位) 浮点常量”。这一个语句会被 256 个线程同时执行, 每个线程用自己的 threadIdx.x 取一个数、乘 2、写回去——256 个数就这么并行算完了。

下面这张竖着排的时间线, 把”这一行代码从头走到尾”完整串了一遍:

### 结构图

连接关系:

- 手写  $\text{TIRx} / \text{B}[\text{tx}] = \text{A}[\text{tx}] * 2.0 / (\text{tx} \text{ 是抽象 thread\_id}) \xrightarrow{\text{LowerTIRx}}$  朴素  $\text{TIR} / \text{B\_1}[\text{threadIdx\_x}] = \text{A\_1}[\text{threadIdx\_x}] * 2.0 / (\text{scope id 已落地, 缓冲区已扁平})$
- 朴素  $\text{TIR} / \text{B\_1}[\text{threadIdx\_x}] = \text{A\_1}[\text{threadIdx\_x}] * 2.0 / (\text{scope id 已落地, 缓冲区已扁平}) \xrightarrow{\text{SplitHostDevice} + \text{MakePackedAPI}}$  拆成两个函数 /  $\text{main}(\text{host 启动器}) + \text{scale\_kernel}(\text{device})$
- 拆成两个函数 /  $\text{main}(\text{host 启动器}) + \text{scale\_kernel}(\text{device}) \xrightarrow{\text{CUDA codegen}}$  CUDA 源码 /  $\text{B\_ptr}[\text{threadIdx.x}] = \text{A\_ptr}[\text{threadIdx.x}] * 2.0f$

## 18.6 自己动手复现每个阶段

这条流水线有个特别好用的地方: 你用不着一口气跑完, 完全可以只跑前面一段, 然后用 `mod.script()` 把中间结果打印出来瞧瞧。本书里所有 IR 片段, 都是这么来的。学 lowering 也好, 调 lowering 也好, 这都是最趁手的家伙。

只跑到 `LowerTIRx`, 看看下降后的 `TIRx IR` 长啥样

```
from tvm.tirx import transform as TT

target = tvm.target.Target("cuda") # :CUDA(NVIDIA GPU)
(host llvm), scale
mod = TT.BindTarget(target.with_host("llvm"))(tvm.IRModule({"main": scale}))
mod = TT.LowerTIRx()(mod) # : tile +
print(mod.script()) #
```

逐行讲: 每个 `pass` 用起来都是“`Pass()(mod)`”这种形式——把它当成一个函数, 喂进去一个模块、吐出来一个改过的模块。所以 `TT.LowerTIRx()(mod)` 就是“拿 `LowerTIRx` 这一步处理 `mod`”。最后的 `mod.script()` 是个特别好用的方法: 它把当前模块以“可读的伪代码”打印出来, 你就能亲眼看到这一步之后代码长啥样。

### 注意

**注意:** `target.with_host("llvm")` 的意思, 是给 CUDA 目标再额外配一个 host 目标 (这里选 LLVM)。为啥非配不可? 因为后面 `SplitHostDevice` 会拆出一个 host 函数, 那个函数是要在 CPU 上跑的, 得有个东西负责把它编成 CPU 能执行的代码——这就是 host 目标 (LLVM) 的活。

一口气编译整个模块, 直接看生成的 CUDA

```
exe = tvm.compile(
 tvm.IRModule({"main": scale}),
 target=target,
 tir_pipeline="tirx", # tirx ,
```



```
)
print(exe.mod.imports[0].inspect_source()) # CUDA
```

这里 `tvm.compile(...)` 就是本章开头那行入口，会把整条 18 个 pass 的流水线全跑完；最后 `inspect_source()` 帮你把 CUDA 后端真正吐出来的那段 GPU 源码亮出来看。

这两段对照着用：前一段看”逻辑落地后的中间态”，后一段看”最终的 CUDA 长啥样”。两头一对照，一个 kernel 从高层 DSL 到 CUDA 源码，中间每一跳你就全看明白了。

提示

提示：想搞清楚某一个步骤到底干了啥？有个简单招——把流水线分别截到这个步骤的”前”和”后”，各打印一次，然后一 diff，哪儿改了一目了然。再配上 18.3 节那几个开关（比如把向量化或者 CSE 关掉），你就能精准抓出某个变换到底起了什么作用。

小结

本章把 **TIRx Lowering 流水线**从入口 `tvm.compile(..., tir_pipeline="tirx")` 一路讲到了 CUDA 源码。回头捋一遍：

- **整体位置**: `BindTarget` → `tirx_pipeline` → `host/device` → → C/LLVM    CUDA。  
记住一个关键：`host` 和 `device` 是在 `SplitHostDevice` 这一步分的家，分完之后各走各的收尾。
- **18 个步骤的排法是有讲究的**：核心下降 `LowerTIRx` 打头阵；中间穿插着两轮化简、分两拨做的数据类型合法化（计算 / 存储）、偏性能的类型收窄和向量化/展开，还有那道只看不改的安全闸门 `VerifyMemory`；`SplitHostDevice` 则是把单函数劈成 `host/device` 的分水岭。
- **LowerTIRx 是整章的核心**：`TilePrimitiveDispatch` 把高层算子换成硬件上的具体实现，`LayoutApplier` 把 `TileLayout` 抽象算成 `addr = data + elem_offset + layout.apply(coord)` 这样的真实地址，顺带把执行域 id 落成 `blockIdx / threadIdx`。它一跑完，模块就成了朴素 TIR。
- **可复现**：整条流水线都支持”只跑前面一段 + `mod.script()` 看中间结果”。这既是书里那些 IR 片段的来源，也是你以后调 lowering 的标准做法。

说到底，这条流水线最值钱的地方在于：它把”你写的声明式 tile DSL”和”GPU 真正跑的 CUDA kernel”之间那道看着挺玄乎的鸿沟，拆成了一串**看得懂、能单跑、能逐段检查**的机械步骤。玄乎劲儿一散，剩下的全是你能动手验证的细节。

延伸阅读

- 原文：[TIRx lowering pipeline —Modern GPU Programming for ML](#) Sys
- 流水线定义源码：`python/tvm/tirx/compilation_pipeline.py(tirx_pipeline)`
- `LowerTIRx` 实现源码：`src/tirx/transform/lower_tirx.cc`

术语对照

| 中文      | English  | 说明                     |
|---------|----------|------------------------|
| 下降 / 降级 | lowering | 把高层结构逐步翻译为低层、贴近硬件形式的过程 |

|          |                                        |                                                                         |
|----------|----------------------------------------|-------------------------------------------------------------------------|
| 流水线      | pipeline                               | 一条有序的编译 pass 序列                                                         |
| tile 原语  | tile primitive                         | 高层算子调用，如 <code>copy</code> / <code>gemm</code> / <code>reduction</code> |
| 执行域 id   | execution-scope id                     | 如 <code>T.cta_id</code> / <code>T.thread_id</code> , 描述”谁执行”            |
| 主机 / 设备  | host / device                          | CPU 端启动器与 GPU 端内核                                                       |
| 收尾       | finalization                           | 拆分后对 host/device 分别施加的末段 pass                                           |
| 打包函数 ABI | packed-func ABI                        | TVM 调用 kernel 所用的标准调用接口                                                 |
| 公共子表达式消除 | CSE (Common Subexpression Elimination) | 把重复子表达式提取为临时变量                                                          |
| 数据类型合法化  | data type legalize                     | 把 bf16/fp8 等改写为硬件可执行的合法形式                                               |
| 类型收窄     | narrow data type                       | 在安全前提下把索引/循环类型降到 32 位                                                   |
| 布局施加器    | LayoutApplier                          | 把 <code>TileLayout</code> 解析为物理地址算术的组件                                  |
| 线程束作用域内存 | warp-scoped memory                     | 由 <code>LowerWarpMemory</code> 下降为 <code>shuffle</code> 的缓冲区            |
| 线程束      | warp                                   | GPU 中一组 (通常 32 个) 被捆在一起、迈同一步同时执行的线程                                     |
| 寄存器      | register                               | 线程私有、速度最快的一小块存储，数量有限很金贵                                                 |
| 共享内存     | SMEM (shared memory)                   | 同一个块内线程共用的片上内存，可当成”手动管理的高速缓存”，比寄存器慢、比全局内存快                              |
| 块 / 线程块  | CTA (Cooperative Thread Array)         | 一组线程，即 CUDA 里的 <code>block</code>                                       |
| 安全闸门     | safety gate                            | 只校验不改写的 pass，如 <code>VerifyMemory</code>                                |

# 第 19 章 · TIRx 语言参考 (概览)

原文:[TIRx Language Reference](#)

## 本章要点

### 本章要点 (TL;DR)

- 先说说几个名词,免得你一上来就懵。**GPU**(图形处理器,现在也是跑 AI 计算的主力芯片)上跑的一段程序,叫**内核 / kernel**(就是”在 GPU 上执行的那段代码”,别想成操作系统内核)。**TIRx** 是一种用来写这种内核的语言——你用 Python 风格写它,它再被翻译成 GPU 真正认识的指令。**设备 / device** 在 GPU 语境里就是指 GPU 本身(对应的”主机 / host”指的是 CPU)。
- 一句话:这一章是「TIRx 语言参考」那一组页面的**导读**。它自己不讲细节,只告诉你一件事——写 TIRx 内核的时候,想查某个特性到底**怎么写、是什么意思**,该翻哪一页。
- 这些参考页是从第二部分的「TIRx 入门」里单独抠出来的。说白了,它们是**速查手册 / reference**(像 API 文档,用来查的),不是**教程 / tutorial**(像新手指南,用来从头学的)。
- TIRx 语言被拆成了五块:解析器工具、数据类型与表达式、缓冲区与内存、控制流、CUDA C++/PTX 内建函数。这些词现在看不懂没关系,正文会一个一个用大白话讲清楚。
- 这五块刚好对着你写内核时一层层往下走的五个关注点:**元编程** → **标量数据** → **内存对象** → **控制结构** → **直达硬件**。建议先照这个顺序扫一遍,之后写代码时哪儿卡了再回来查。
- 它属于”参考与附录”这一部分的子页面。主线内容都在 Parts I-IV,这一组只管”把细节列清楚”。

## 前置知识

**前置知识:**这一章默认你已经认识几个 GPU 的基础词:**线程**(GPU 同时跑成千上万个,每个干一小份活)、**warp / CTA / grid**(线程被一层层打包成的组,后面正文也会再提)、**共享内存和寄存器**(GPU 的两层存储)。如果这些词对你完全陌生,强烈建议先翻第 0 章·极简入门,再回来读这章会顺很多。本章每个词第一次出现时也会顺手解释,所以即便不熟也能跟下来,只是会慢一点。

## 19.1 这一页是什么

这一页是一组**语言参考**的入口。你写 TIRx 内核会用到的所有语言特性,原文都从第二部分那篇偏教学的「TIRx 入门」里抽了出来,整理成几页速查文档。

为什么要这么拆?打个比方:学一门编程语言,刚开始你看的是”新手教程”——它手把手领你走,适合**第一次理解概念**。可等你已经在写代码了,只想确认某个函数的一个细节,再翻教程就太慢了,这时候你要的是”API 参考手册”——精确、零废话、一查就有。**TIRx** 也一样。比如这些很具体的小问题:**T.unroll**(后面会讲,大意是”把循环摊平”)到底是完全摊平,还是只是顺序跑一遍?动态共

享内存（一种 GPU 上的存储，后面会讲）是不是只能分配一块？某个函数最后选中的到底是哪个线程？——把这些答案做成手册式的参考页，就是为了让你**遇到问题一查就有**。

关键

**关键：**这一组页面是拿来”按需查”的，不是拿来”从头读到尾”的。建议先快速扫一遍，心里有个数——每页大概讲什么（本章就帮你干这件事），之后写代码卡住了，再精确翻回对应那页。

19.2 五个子页面分别讲什么

原文列了五个子页面。下面一个一个说它们讲什么，各配一个最典型的例子。看完你心里就有谱了：**碰到哪类问题，该去翻哪一页**。

| 子页面                       | 一句话定位                                                    | 你会在这里查到                                                  |
|---------------------------|----------------------------------------------------------|----------------------------------------------------------|
| 解析器工具 (Parser utilities)  | <b>解析期 (parse-time) 的元编程工具</b> ——也就是”代码还没真跑、只是在被翻译时”动手脚  | 把 Python 值内联进 IR、内联函数、打包解析期状态                            |
| 数据类型与表达式                  | 每个表达式的 <b>低层 dtype 与高层 type</b> (两套类型信息)                 | dtype 到 CUDA 类型的映射、向量类型、指针 (handle)                      |
| 缓冲区与内存 (Buffers & memory) | 内核里所有 <b>内存对象</b> （各种存储空间）的声明与语义                         | 各种 scope(global/shared/local/tmem)、视图、标量、let、张量内存 (TMEM) |
| 控制流 (Control flow)        | if / 循环家族 / while(就是程序的判断和循环)                            | 四种循环、统一 (uniform)vs 发散 (divergent) 控制流的坑                 |
| CUDA C++/PTX 内建函数         | 直达硬件的 <b>两个逃生口 (escape hatches)</b> ——当高层工具不够用时，绕过去直接调底层 | T.cuda.* / T.ptx.* 内建、内联原始 CUDA、同步语义                     |

下面一块块展开说。表里那些没解释的名词，正文里会逐个讲。

解析器工具：解析期的元编程

一句话先理解

**一句话先理解：**这一页讲的工具，全都在”代码被翻译的那一瞬间”干活，翻译完就没它们的事了——它们不会变成 GPU 真正执行的指令。

先解释一个贯穿全页的关键词：**解析期 / parse-time**。你写的 TIRx 代码，长得像 Python，但它不会被 Python 直接执行——而是先被**解析 (parse)** 一遍，翻译成一种叫 IR(中间表示，可以理解成”编译器内部用的、更接近机器的代码形式”)的东西，之后才轮到 GPU 跑。”解析期”就是指这个”正在翻译、还没真跑”的阶段。

那为什么需要在解析期动手脚？因为有些事你希望**提前**办掉、别拖到运行时。比如把一个固定 4 次的循环直接摊平成 4 行直白代码（这叫**展开 / unroll**，好处是省掉循环的判断开销），或者把一个函数整个铺进调用处。这些在翻译时就办完最划算。这一整套”在翻译期动手脚”的技巧，统称**元编程 / metaprogramming**(字面意思：写”操纵代码本身”的代码)。这一页讲的就是干这些事的几个工具：

- **T.meta\_var(x)**: 你在 Python 里先把一个值算好（比如算出循环要跑 4 次），再用 T.meta\_var 把这个值标成”解析期常量”(原文叫**元值 / meta value**)，直接塞进正在生成的代码里。它最妙

的一点是：拿这种值去写普通的 Python for 循环，这循环会在解析期就**自动展开**成几行直线代码。看例子：

```
n = T.meta_var(4) # n , 4
for j in range(n): # n ,
 acc[0] = acc[0] + A[tx, j] # 4 acc[0] += A[tx,0]...A[tx,3]
```

逐行说：第一行把 4 这个数标成” 翻译期就定死的常量”；第二行用它写普通 Python 循环——关键就在这，因为次数在翻译期已知，翻译器会把循环体复制 4 遍、把 j 分别填成 0/1/2/3，生成的代码里压根没有” 循环” 这个结构了。

- **@T.inline**: 一个**装饰器** (Python 里 @ 开头、加在函数定义上方那种修饰符)。给函数加上它，意思是” 每次调用这个函数，就把函数体原地摊开”——也叫**内联 / inline**。结果是生成的代码里看不到真正的” 函数调用”，只看到被摊平的函数体。这样做省掉了函数调用本身的开销。
- **@T.meta\_class**: 也是个装饰器，加在普通 Python 类上，意思是” 这个类的实例算一个解析期元值”。它的字段里可以装缓冲区和标量 (后面会讲这两个词)。用处是：把内核**流水线** (pipeline, 即把多步操作首尾重叠起来跑、像工厂流水线，见第 0 章) 所需要的一堆状态 (各种屏障、累加器、临时区视图) **打包成一个对象**，省得你在内核里到处撒一地零散的局部变量。这里提一句**累加器 / accumulator**: 就是一个” 一边算一边把结果累进去” 的变量，比如算总和时那个不断 += 的变量。
- **T.constexpr**: 把某个内核参数标成” 编译期就定死的常量”，再由 **@T.jit**(即时编译的装饰器) 的 **.specialize(...)** 在编译时把具体值烘焙进去。细节看「TIRx 入门」。

注意

注意：这一整页就一个关键词——**解析期**。这些工具弄出来的东西，你在最终生成的内核里是看不见的：要么被内联了 (函数体摊平了)，要么被展开了 (循环摊平了)，要么被当成常量烘进去了。

数据类型与表达式:dtype 与 type 两套体系

一句话先理解

一句话先理解:TIRx 里每个值都同时背着**两层类型信息**——一层管” 它在内存里到底是哪几个二进制位”，另一层管” 它逻辑上是个什么东西 (普通数? 指针?)”。

为什么要分两层? 打个比方：一个内存地址 (指针) 和一个整数，在硬件底层可能都是” 一串 64 位的二进制”，光看你位分不出谁是谁。所以除了” 底层是什么位”，还得有一层” 逻辑上这是什么”。这两层就是：

- **低层 dtype**(data type 的缩写，即” 数据类型”)：写作 **PrimExpr.dtype**，管的是” 它到底是几位、什么格式的数”。常见的有：**float32**(32 位浮点，最常用)、**float16 / bfloat16**(16 位浮点，占一半空间、算得更快、精度差点)、**int32**(32 位整数)、**bool**(布尔)、更省位的低精度格式 **float8\_e4m3fn / float4\_e2m1fn**(8 位 / 4 位浮点, AI 推理里为了省内存和提速会用)、向量形式 **float32x4**(把 4 个 float32 捆成一组，后面细说)，还有指针 **handle**。每个 dtype 都对应一个 CUDA(NVIDIA 的 GPU 编程语言) 里的具体类型。

- **高层 dtype**: 普通标量写成 `PrimType(dtype)`, 指针写成 `PointerType(PrimType(dtype), scope)(scope)` 是”这个指针指向哪块内存”, 后面缓冲区那节会讲)。这一层你**只在跟指针打交道时才用得上**, 平时写普通数值基本不用管。

这一页最该记的, 是这张 **dtype → CUDA 类型映射表** (左边是 TIRx 里的写法, 右边是它翻成 CUDA 后对应的类型名):

| TIRx dtype | CUDA 类型     |
|------------|-------------|
| float32    | float       |
| float16    | half        |
| bfloat16   | nv_bfloat16 |
| int32      | int         |
| uint8      | uchar       |
| bool       | signed char |
| float32x4  | float4(向量)  |
| handle     | T*(指针)      |

**向量 dtype(比如 float32x4) 是干嘛的?** 先说”为什么有它”:GPU 搬数据时, 一次搬 16 字节 (= 4 个 float32) 比分 4 次、每次搬 4 字节要快得多——就像搬箱子, 一次抱一摞肯定比来回跑四趟省事。把 4 个 float32 捆成一组 的类型, 就是 **float32x4**。它有两种玩法。一种是直接声明一个能装 4 个 float 的寄存器 (**寄存器 / register** 是每个线程私有的、速度最快的那层存储, 类比 CPU 寄存器但 GPU 是每个线程各有一份, 见第 0 章): 写 `T.alloc_local((1,), "float32x4")`, 然后用 `v[0]` 取出整组。另一种是配合 `vload/vstore`(向量读 / 向量写), 把这 16 字节当成**一次宽访问**一口气搬过去。补一句: 向量 dtype 不是非得跟 `vload` 一起用——随便哪个缓冲区或标量都能带上它。

**再说指针 (handle)**。每个缓冲区 (buffer, 下一节细讲, 先理解成”一块带说明书的数据”) 都有个 **data** 指针, 指向它真正的存储起点。这个指针是个**不可变 / immutable** 的变量 (**Var**)——”不可变”就是一旦定下来就不能改指向别处, 跟一般编程里的 `const / final` 一个意思。所以怎么拿到它有讲究, 常规两种办法: 要么让 `alloc_buffer`(分配缓冲区的函数) 在分配时顺手把它定义好; 要么用 `decl_buffer(data=ptr)` 复用 一个现成的指针。但有一种情况要特殊处理: 如果你想让缓冲区指向**另一个算出来的指针** (比如用 `T.ptx.map_shared_rank` 算出”另一个 CTA 里某块共享内存的地址”)——这里 **CTA / 线程块**是一组线程的打包单位,**cluster** 是更上一层、由几个 CTA 凑成的协作小队, 均见第 0 章), 那就得先用 `T.let` 把这个算出来的地址绑成一个固定的指针变量, 再拿去用。

缓冲区与内存: 内核里所有”内存对象”的字典

一句话先理解

一句话先理解: 缓冲区 / **buffer** 你可以理解成”一块数据 + 一张说明书”——数据存在某个地方, 说明书告诉你它的形状、类型、怎么算出每个元素的地址。内核里几乎所有东西都是缓冲区, 所以这是五页里**最厚、回头查得最勤**的一页。

它讲清楚三件事: 两套声明缓冲区的 API、几种**内存空间 / scope**(数据放在 GPU 的哪一层存储里), 还有一堆”看着像数据、其实只是说明书 (元数据)”的视图和标量概念。先解释 **scope**:GPU 的存储分好几层, 有的快但小、每个线程私有, 有的慢但大、所有线程共享 (为什么要分层? 因为又快又大又便宜的存储不存在, 只能分层取舍, 详见第 0 章)。scope 就是指”这块数据放在哪一层”。

**两个根 API**(API 即”提供给你调用的函数接口”):

| API                                                | 干什么                                                                                                                                                                  |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>T.alloc_buffer(shape, dtype, scope=ê)</code> | 真正分配一块新存储（就像 <code>malloc</code> ，问系统要一块空间）； <code>T.alloc_shared</code> / <code>T.alloc_local</code> 只是它分别带上 <code>scope="shared"</code> / <code>"local"</code> 的简写 |
| <code>T.decl_buffer(shape, dtype, data=ê)</code>   | 不分配新存储，只在一块已有的存储上再开一个”说明书”（原文叫视图 / <code>view</code> ）。用来给同一块数据起别名，或换一种方式解读它                                                                                         |

这俩的区别很关键：`alloc_buffer` 是”申请新地皮”，`decl_buffer` 是”在已有地皮上再画一张图纸”。

内存空间（`scope`）一共这几种：

| scope        | 简写                          | 对应内存                                                                    |
|--------------|-----------------------------|-------------------------------------------------------------------------|
| "global"     | (默认)                        | 设备全局内存——GPU 上最大但最慢的那层，所有线程都能访问                                          |
| "shared"     | <code>T.alloc_shared</code> | 静态共享内存——一个线程块内部共用的高速小存储，大小在编译时定死（对应 CUDA 的 <code>__shared__</code> ）    |
| "shared.dyn" | (用 <code>pool</code> )      | 动态共享内存——也是共享内存，但大小推迟到启动内核时才定                                            |
| "local"      | <code>T.alloc_local</code>  | 每线程私有的寄存器——最快、最小                                                        |
| "tmem"       | (TMEM <code>pool</code> )   | Blackwell(NVIDIA 较新一代 GPU 架构) 上的张量内存 (TMEM)，是 Tensor Core 专用的片上内存，后面单独讲 |

提示

**提示：**”共享内存”为什么是”高速缓存”那种角色？你可以把它类比成一块由你手动管理的高速缓存——CPU 的缓存是硬件自动帮你填的，而 GPU 的共享内存是你自己决定往里放什么、什么时候放。它比全局内存快很多，但只在一个线程块内部共享、容量很小，所以得精打细算地用。

这一页有几个反复出现的要点，先记在心里：

- 缓冲区说白了就是”挂在一块存储上的一张说明书”。同样是逻辑上访问 `B[i,j]`，你只要改一下 `B` 的 `layout`(布局，即”元素在内存里怎么排”) 或 `elem_offset`(起始偏移)，它实际算出来的内存地址就完全变了：按行排（行主序）时是 `i*8+j`，按列排（列主序）时是 `j*4+i`，带个偏移就再加 `64⋯⋯`可底下那块真正的数据从头到尾压根没动过。换句话说，缓冲区改的是”怎么读这块数据”，不是数据本身。
- 动态共享内存，整个内核只能分配一块。为什么？这是硬件的限制。所以你能把这唯一的一块当成一片”竞技场 / arena”(一大块连续空地)，其他所有想用动态共享内存的缓冲区，都得用 `decl_buffer(data=arena.data, elem_offset=ê)` 在这片空地里划一块视图出来用。也就是说，写两遍 `alloc_buffer(scope="shared.dyn")` 是错的。好在 `T.SMEMPool` 帮你把”在这片空地里依次划块、记好谁占了哪段”这件事自动化了（原理就是个 `bump` 分配器——每划一块就把游标往后推一截，简单直接）。
- 标量 / `scalar` 其实就是”单元素寄存器缓冲区”的语法糖。这里标量就是”单个的数”(相对于一整组数组而言)。你写 `phase: T.int32 = 0`，跟你老老实实写 `T.alloc_local((1,), "int32")` 再每次用 `[0]` 去取，翻译完得到的内部结构一模一样。前者无非是个语法糖，让你少写那个 `[0]`。

- **let 和标量是两码事，别搞混。**T.let 是一个不可变绑定——翻译下去就是个普通的 C 变量 (不是数组)，绑定后值不能改。正因为它不可变、值固定，编译器的”算术分析器”才能把它的一些已知事实 (比如”它一定是 0-63 之间””它一定能被 4 整除”) **传播到每一个用到它的地方**，顺手帮你化简地址计算、砍掉用不上的越界检查、判断能不能做向量化。而一个**可变的标量**呢？它本质上每次用都是一次”去内存里读一下”，编译器没法假设它的值，上面这些优化好处一个都享受不到。所以：值不会变的就用 **let**，会变的才用标量。
- **张量内存 / TMem 是个特例，门槛最高。**它必须用 T.ptx.tcgen05.alloc/dealloc 显式地申请和释放 (而且要 warp 内所有线程动作一致地调，即 warp-uniform)；更特别的是它**不能像普通数组那样直接用下标读写**——所有读写都得走 tcgen05 这一族专门指令 (mma/ld/st/cp)。每个用到的张量都得用 decl\_buffer(scope="tmem", allocated\_addr=< >, layout=ê ) 开一个视图。这一整套繁琐流程,T.TEMPpool 都替你封装好了，平时直接用它就行。

这一页末尾还有一张 **Buffer 方法表** (data / ptr\_to / vload / vstore / view / local / permute / access\_ptr)。这些方法绝大多数都是**编译期**的”换种读法”：要么改改地址算式，要么递给你一个指针，本身**不产生任何运行时操作** (不会真的搬数据)。

控制流:if、循环家族、while

一句话先理解

**一句话先理解：**控制流就是程序里的判断和循环 (if、for、while 这些)。这一页本身很短，因为 TIRx 这部分长得跟普通编程几乎一样；但有一个 GPU 特有的大坑，这一页就是来敲打你的。

普通部分很简单:Python 的 if/else 直接翻成 GPU 的 if/else,break/continue 照常能用。真正值得专门查的只有两件事：

循环有四种风味：

| 写法               | 含义                                                    |
|------------------|-------------------------------------------------------|
| T.serial(n)      | 普通的顺序循环 (你写普通 range 就会变成它；底层编译器 ptxas 之后仍可能自行决定要不要展开) |
| T.unroll(n)      | <b>完全展开</b> ——把循环体复制 n 遍摊成直线代码，生成的代码里没有循环结构           |
| T.vectorized(n)  | 向量化循环——让循环体里的操作一次处理一组数据 (回想前面 float32x4 那种”一次搬一摞”)    |
| T.grid(*extents) | 一次写出多层嵌套循环 (省得手写好几层 for 套 for)                        |

**统一 / uniform 控制流 vs 发散 / divergent 控制流**——这是最容易踩的坑，也是这一页真正想敲打你的地方。

先讲清这两词。GPU 上线程不是各干各的，而是被打包成 **warp**(一个 warp = 32 个线程，见第 0 章)一起跑；同一个 warp 里的 32 个线程，**必须步调一致地执行同一条指令**——你可以想象成”32 个人组成一个方阵，必须齐刷刷地迈同一步”。

- **统一控制流：**一个 if 的条件，对一组线程来说**要么全为真、要么全为假**，大家走同一条路，方阵不散。
- **发散控制流：**同一组线程里，有的进 if、有的不进，方阵就**散了**——这本身有性能代价，但更要命的是下面这个死锁问题。



## 关键

**关键:** 先记住一句话——**集合操作 / collective op**(需要“一群线程一起到齐才能完成”的操作), 必须让它要同步的每一个线程都“齐刷刷地”执行到, 一个都不能少。举个最常见的: `T.cuda.cta_sync()` 会翻成 CUDA 的 `__syncthreads()`, 意思是“线程块里所有线程都在这儿等齐了再一起往下走”(这种“大家在某一行等齐”的机制叫**屏障 / barrier**)。问题来了: 要是你把它塞进 `if wg_id == 0:` 这种只有一部分线程才进得去的分支里, 那没进这个分支的线程永远走不到这一行, 而进了的那些又在傻等所有人——结果就是大家互相干等、谁也动不了, 这就是**死锁 / deadlock**。(这里 **warpgroup** 是比 **warp** 再大一层的打包单位, 常见是 4 个 **warp** = 128 线程一组。)如果你的本意只是“让某一个 **warpgroup** 内部自己同步一下”, 那就别用管整个线程块的 `cta_sync()`, 改用 **warpgroup** 范围的 `T.cuda.warpgroup_sync(id)`。

同样的坑也出现在**屏障初始化**上。**mbarrier**(memory barrier, 放在共享内存里、用来让线程和异步引擎互相等待的一种硬件屏障——“异步引擎”指 GPU 上那些能后台自己搬数据的硬件单元)的 `.init()` 翻下去会自带一个“只让一个线程执行”的守卫 (`if (threadIdx.x < 1)`)。你要是再把它套进另一个发散分支里, 很可能这个屏障**压根没被初始化**, 结果就是莫名其妙的内核启动失败。

最后一个实用小技巧: 如果你只是想做“二选一取个值”(cond 成立取 a, 否则取 b), 而不是真要让线程走不同的分支, 那就别写 `if`, 改用 `T.if_then_else(cond, a, b)`。它翻下去就是 C 的三元运算符 (`cond ? a : b`), 所有线程都把两个值都算一下、再各自挑一个, **不会让方阵散开** (不引入控制流发散)。

## CUDA C++/PTX 内建函数: 两个直达硬件的逃生口

## 一句话先理解

**一句话先理解:** 当 TIRx 提供的高层工具满足不了你时, 这一页给你两条“绕过去、直接调底层硬件”的后门, 叫**逃生口 / escape hatch**。

先解释为啥需要逃生口。TIRx 平时帮你封装好了很多常用操作 (比如对 **tile**——把大矩阵切出来的小方块——做的各种现成动作)。但硬件能做的事太多, 封装不可能面面俱到。一旦遇到没被封装的功能, 你就得有办法“绕过封装、直接命令硬件”。这一页给的两条路是:

1. **调后端内建函数。**用 `T.cuda.* / T.ptx.*` 这两组现成函数 (**内建函数 / intrinsic** 指“语言/编译器直接提供、能映射到一两条硬件指令的特殊函数”)。这里能调到的东西很全: 线程同步、**mbarrier**、归约, 以及 PTX 那大家子底层操作——比如成块搬数据的 **cp\_async / TMA**(Tensor Memory Accelerator, 一个专门负责“整块整块搬运张量数据”的硬件引擎)、给 Tensor Core 喂数的 **ldmatrix/stmatrix**、做 **MMA**(matrix multiply-accumulate, 即“矩阵乘法”, 是 Tensor Core 这种专算矩阵乘法的硬件单元执行的核心指令——AI 计算里绝大部分算力都花在矩阵乘上, 所以这是主角)的指令、Blackwell 上的 **tcgen05.\***、原子加 **atomic\_add**、栅栏 **fence** 等等。(PTX 可以理解成 NVIDIA GPU 的一种“准汇编”——比 CUDA 更底层、更贴近硬件。)
2. **直接内联一段原始 CUDA 代码。**连内建都没有的功能, 就用 `T.cuda.func_call(name, *args, source_code=ê, return_type=ê)`, 把一段你手写的 `__device__` (标记“这是 GPU 上运行的函数”的 CUDA 关键字) 源码原样塞进去, 再把调用接上。这是最后的兜底手段。

举个具体例子。假设 **warp** 里 32 个线程各自手上有一个数, 你想把这 32 个数加成一个总和、让每个线程都拿到这个总和 (这叫**全归约 / all-reduce**)。GPU 上有个高效办法: 让同一个 **warp** 里

的线程直接互相交换寄存器里的值 (这个操作叫 **warp shuffle**, 省得绕道共享内存)。核心代码就一行:

```
warp shuffle "": 2~i ,
v[0] += T.tvm_warp_shuffle_xor(0xFFFFFFFF, v[0], i[0], 32, 32)
```

逐部分解释这一行:`T.tvm_warp_shuffle_xor(...)` 是”和我按异或规则配对的那个线程, 把它的 `v[0]` 拿过来”。第一个参数 `0xFFFFFFFF` 是个 32 位掩码, 32 个 1 表示”warp 里 32 个线程全部参与”;`v[0]` 是要交换的值; `i[0]` 是这一轮的配对距离; 后两个 32 是 warp 宽度等参数。把它加回自己的 `v[0]`, 循环几轮 (距离 `16→8→4→2→1`) 后, 每个线程手里就都是 32 个数的总和了——这种”每轮折半”的配对方式形状像蝴蝶, 所以叫蝶式归约。这一行最终会翻成 CUDA 的 `__shfl_xor_sync(0xFFFFFFFF, v_ptr[0], i_ptr[0], 32)`。

这一页还专门讲了**四类同步语义**。这四类在写高性能内核时反复出现, 而且一旦用错, 通常不会报错给你看, 而是静默地算出错误结果、或者直接死锁——所以特别值得单独记一记:

| 机制                  | 关键语义 / 坑                                                                                                                                                                                                                                                                                                                |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mbarrier 相位 / phase | 这里 <b>相位 / phase</b> 是个会在 0/1 之间来回翻的”轮次标记”。 <code>try_wait(bar, phase)</code> 的意思是”一直卡在这等, 直到屏障内部的相位 <b>不等于</b> 我传进去的 <b>phase</b> 为止”(即等到进入下一轮)。坑在哪: 要是你在一个循环里反复用同一个屏障, 每次等完都得手动把自己本地记的相位翻一下 ( <code>phase ^= 1</code> , 异或 1 就是 <code>0↔1</code> 翻转); 忘了翻, 下一次等待会 <b>立刻就返回</b> (因为相位没对上), 于是异步引擎可能读到一块只写了一半的内存, 数据就坏了 |
| 选举 / election       | <code>T.ptx.elect_sync()</code> 的作用是”从一个 warp 的活跃线程里挑出一个来”。坑在于: 它挑中的是 <b>某一个活跃 lane</b> (lane 即”线程在 warp 内的编号 0-31”), 但不保证是 <b>0 号</b> , 也不是”每个 CTA 只挑一个”。所以你要是想精确地”全局只让一个线程干某事”, 还得在外面再套一层 warp 级守卫, 比如 <code>if warp_id == 0:</code>                                                                                  |
| 具名 warpgroup 屏障     | <code>cta_sync()</code> 要求整个 CTA (线程块) 都到齐; 但如果你给不同 warpgroup 分了不同的活 (叫 warpgroup 特化), 就该改用 <code>warpgroup_sync(id)</code> 只同步组内。硬件总共只提供 16 个带编号的屏障 (ID 0-15), 所以不同 warpgroup 必须 <b>用不同的 ID</b> (比如 <code>warpgroup_sync(wg_id + 10)</code> ), 否则两组用了同一个编号会互相干扰                                                        |
| 栅栏 / fence          | <b>栅栏 / fence</b> 用来强制规定”谁先谁后”——典型场景是”生产者线程先写好数据, 消费者 (往往是后台的异步引擎) 再去读”。它确保写一定排在读前面, 不会乱序。具体有 <code>fence.proxy_async("shared::cta")</code> 、 <code>fence.mbarrier_init().tcgen05.fence.after_thread_sync()</code> , 各管一条排序关系                                                                                           |

注意

注意: 这四类同步是 **GEMM** (通用矩阵乘法——前面说过, 矩阵乘是 AI 计算的主角, 所以这是最核心的内核) 和 **FlashAttention** (一种高效计算注意力的算法, Transformer 大模型里的关键内核) 里的”常客”。它们管的是**后台异步引擎和并行线程组之间的配合**, 一旦用错, 通常**不报错**, 而是直接**静默地算错结果、或者死锁**——这也正是作者要单独写一章「Warp-Specialized 内核调试」的原因。

19.3 建议的阅读顺序

这五页虽说是速查手册，但作者排它们的顺序其实藏着一条线索：写内核的时候，你的关注点会从”代码怎么生成”一层一层往下走到”硬件怎么执行”。所以第一次速览，照下面这个顺序过一遍最顺：

结构图

连接关系：

- 2. 数据类型与表达式 / 标量 / dtype / 指针 → 3. 缓冲区与内存 / 内存对象 / scope / 视图
- 3. 缓冲区与内存 / 内存对象 / scope / 视图 → 4. 控制流 / if / 循环 / 同步守卫
- 4. 控制流 / if / 循环 / 同步守卫 → 5. CUDA/PTX 内建 / 直达硬件 + 同步语义

1. **解析器工具**：先搞明白”代码被翻译时能做哪些元编程”，因为后面好多代码都靠它来展开循环、内联函数。
2. **数据类型与表达式**：把”单个数据”的底子打牢——dtype、向量类型、指针。
3. **缓冲区与内存**：进到”数据存在哪、怎么读”这一层。这页分量最重，也是你回头查得最勤的一页。
4. **控制流**：有了数据和内存，再来看怎么把执行流程组织起来，尤其要记牢”集合操作必须让每个线程齐刷刷到达，否则死锁”这个坑。
5. **CUDA C++/PTX 内建**：最后是直达硬件的两个逃生口和四类同步语义——写 GEMM/FlashAttention 的时候你也最常翻回这一页。

关键

**关键**：速览过一遍以后，就别再”从头读到尾”了。它真正的用法是：代码写到哪儿卡住了，就跳回对应那页查精确语义。

小结

本章是「TIRx 语言参考」这组页面的导读 (TIRx 即”写 GPU 内核用的那门语言”)。核心就三点：

- 它是从「TIRx 入门」里抽出来的**语言速查手册**——作用是”查精确写法和语义”，像 API 文档，不是再把新手教程重讲一遍。
- 它把 TIRx 语言切成五块，刚好对着写内核的五个层面：**解析器工具 (代码翻译期的元编程)**→ **数据类型与表达式 (单个数据 / dtype / 指针)**→ **缓冲区与内存 (数据存哪、怎么读 / scope / 视图)**→ **控制流 (if / 循环 / 同步守卫)**→ **CUDA/PTX 内建 (绕到底层直达硬件 + 四类同步语义)**。
- 它的用法是**按需查**：先按上面的顺序速览一遍，在脑子里画一张地图；之后写代码碰到具体问题，再精确跳回对应那页。其中你回头翻得最多的是两页——「缓冲区与内存」(最厚)和「CUDA/PTX 内建」(同步语义的坑最多、用错还不报错)。

延伸阅读

- 原文页面：[TIRx Language Reference —Modern GPU Programming for ML](#)Sys
- 前置教程：第二部分的「TIRx 入门 / Introduction to TIRx」和「TIRx 布局 API / TIRx Layout API」。

- 完整的 `tvm.tirx` Python API: 见上游 TVM / Apache TVM 的官方文档。
- 相关实践: 想看四类同步语义的实际用法, 可以读 Part III 的 GEMM 内核, 以及「Warp-Specialized 内核调试」那一章。

术语对照

| 中文          | English                 |
|-------------|-------------------------|
| 设备内核        | device kernel           |
| 解析器工具       | parser utilities        |
| 解析期         | parse-time              |
| 元编程         | metaprogramming         |
| 元值          | meta value              |
| 内联函数        | inline function         |
| 数据类型 / 标量类型 | dtype                   |
| 向量类型        | vector dtype            |
| 指针          | handle / pointer        |
| 缓冲区         | buffer                  |
| 内存空间        | scope                   |
| 视图          | view                    |
| 静态共享内存      | static shared memory    |
| 动态共享内存      | dynamic shared memory   |
| 竞技场         | arena                   |
| 寄存器         | register                |
| 标量          | scalar                  |
| 不可变绑定       | let / immutable binding |
| 张量内存        | tensor memory (TMEM)    |
| 控制流         | control flow            |
| 统一控制流       | uniform control flow    |
| 发散控制流       | divergent control flow  |
| 集合操作        | collective op           |
| 死锁          | deadlock                |
| 内建函数        | intrinsic               |
| 逃生口         | escape hatch            |
| 相位          | phase                   |
| 选举          | election                |
| 栅栏          | fence                   |

## 第 20 章 · 解析器工具 (Parser utilities)

原文: [Parser utilities](#)

### 本章要点

#### 本章要点 (TL;DR)

先别管那几个工具名, 我们把这一章要解决的大问题用一句话说清楚: 你怎么用 Python 这门你熟的语言, 去”批量生成”一段要在显卡 (GPU) 上跑的程序? 这件事就叫元编程 / **metaprogramming**——简单说就是”写程序去写程序”。本章四个工具, 都是 TIRx 提供给你的元编程小开关。

- 本章讲四个工具: `T.meta_var`、`@T.inline`、`@T.meta_class`、`T.constexpr`。它们有个共同点: 都在**解析时** / **parse time** 干活。这里先解释一个贯穿全章的词——「**解析** / **parse**」就是”把你写的代码读进来、翻译成另一种内部表示”的过程 (就像编译器读你的源码、转成它内部的语法树)。在 TIRx 里, 这一步是把你写的 Python (叫 TVMScript) 翻译成一种叫 TIRx IR 的内部结构。所以”解析时干活”, 意思是这些工具出力的那一刻, 是**翻译代码的瞬间**, 而不是 GPU 真正跑代码的时候。说白了, 它们就是 TIRx 的元编程入口。
- `T.meta_var(x)`: 把一个你用 Python 当场算好的值直接塞进 IR, 当编译期常量使。最典型的玩法, 是让一个普通的 Python for 循环在解析阶段就被**展开** / **unroll**。
- `@T.inline`: 定义一个特殊的函数, 它会在**每个调用点就地展开**。换句话说, 生成的 IR 里压根看不到函数调用, 只剩替换进去的函数体。它遵循 Python 的 LEGB 作用域和延迟绑定 / late binding 规则。
- `@T.meta_class`: 给一个普通 Python 类贴个「解析期元对象」的标记。它的实例字段能装下 buffer、标量这些 IR 对象, 于是你就能把一组相关的分配和状态**打包成一个对象**, 不用再拎着一堆零散的局部变量到处传。
- `T.constexpr`: 标记一个**编译期的 kernel 参数**, 由 `@T.jit` 的 `.specialize(...)` 在编译时把它「烤」进 kernel。本章只点个名, 细节看《TIRx 入门》。

### 前置知识

**前置知识:** 这一章对 GPU 硬件的要求其实很低——只要你会写代码、懂”编译器把源码翻译成中间表示”这种事, 就能读懂大半。出现的几个 GPU 名词 (buffer、共享内存等) 我都会在第一用到时当场讲明白, 所以你**不用提前补 GPU 课**。不过如果你想看得更顺, 可以先翻一下第 0 章·极简入门建立点直觉, 以及第 9 章 TIRx 基础了解 TVMScript 的写法。

这里先把三个会反复出现的词钉一下:

- **TVMScript**: 你实际写出来的那段 Python 代码 (带一堆 `@T` 开头的装饰器)。它长得像 Python, 但其实是用来描述 GPU 程序的。
- **TIRx IR**: 解析器把你的 TVMScript 翻译之后得到的内部结构。IR = Intermediate Representation (中间表示), 你可以理解成”编译器内部用的、规规矩矩的程序树”。

- **kernel**: 在 GPU 上跑的那个函数/程序的统称。后面说“kernel”就是指“要在显卡上执行的这段代码”。

## 一、先搞清楚：什么叫「解析期工具」

### 一句话先理解

**一句话先理解**: 你的代码从“写出来”到“在 GPU 上真跑起来”，中间要过好几道关；本章这四个工具，全都在**最前面那道关（翻译关）**里干活，根本到不了 GPU。先把这条时间线在脑子里摆清楚，后面就全顺了。

你写好一段 TIRx kernel, 到它真正在 GPU 上跑起来, 中间得过三道关:

### 结构图

连接关系:

- 你写的 TVMScript / (带 @T 装饰的 Python)  $\xrightarrow{\text{① 解析 (parse)}}$  TIRx IR / (结构化中间表示)
- TIRx IR / (结构化中间表示)  $\xrightarrow{\text{② 下降 (lower) / 编译}}$  GPU 上执行的机器码 / (SASS / PTX)
- GPU 上执行的机器码 / (SASS / PTX)  $\xrightarrow{\text{③ 运行 (runtime)}}$  GPU 上执行的机器码 / (SASS / PTX)

- **① 解析阶段 / parse time**: 解析器读你写的那段 Python(也就是 TVMScript), 把它**翻译成 TIRx IR**。这里有个特别要紧、也特别反直觉的点: 这一步是在**普通的 Python 进程**里跑的——换句话说, **此刻你的代码是被当成“数据”在处理的, 而不是被当成程序在执行**。打个比方: 你写的 TVMScript 就像一张设计图纸, 解析阶段是有人在“读图纸、抄成正式蓝图”, 还远没到“照着蓝图盖房子”那一步。正因为这时候还在 Python 里, 所以你能用普通 Python 的能力 (算数、循环、对象) 去影响翻译结果——这就是后面所有把戏的根基。
- **② 下降 / 编译阶段**: LowerTIRx 这类 **pass**(可以理解成“对 IR 做一轮变换处理的步骤”, 编译器里一道道工序) 上场, 把 IR 一步步落实现成具体的 GPU 指令。
- **③ 运行阶段**: 代码真正在 GPU 上跑。

### 关键

**关键**: 本章这四个工具, **全都活在阶段 ①**。它们不会变成 GPU 上的任何一条指令。它们只在「Python 翻译成 IR」这一瞬间动手——要么把一个 Python 值塞进 IR, 要么趁翻译的时候提前把循环或函数展开掉, 要么把一堆状态打成一个包。

说白了, 这就是开头讲的「元编程」: 拿 Python 这门你写得顺手的语言 (术语叫“宿主语言 / host language”, 意思是“用来生成别的代码的那门语言”), 去操控、去生成另一门语言 (TIRx IR) 的代码。你只要记住一句话——「这些事全发生在解析器里、还没到 GPU」, 后面四节就都顺了。

那它们各自治什么病? 主要是下面这三类:

| 工具            | 解决的痛点                                               |
|---------------|-----------------------------------------------------|
| T.meta_var    | 想把 Python 算出来的常量直接用进 IR，又不想多写一个没用的临时变量；想让循环在解析期展开   |
| @T.inline     | 想把一段重复逻辑抽成函数复用，又不希望 IR 里真出现一次函数调用                   |
| @T.meta_class | kernel 里状态太多（屏障、累加器、scratch view……），不想把十几个局部变量一路穿下去 |
| T.constexpr   | 想要一个编译期就固定下来的 kernel 参数（如 tile 尺寸），让编译器据此特化         |

## 二、T.meta\_var 一把 Python 值 inline 进 IR

### 2.1 它在做什么

#### 一句话先理解

一句话先理解：`T.meta_var(x)` 就是告诉解析器“`x` 是我在 Python 里早就算好的一个普通数，你直接把这个数填进生成的代码里，别把它当成 GPU 程序里的一个变量”。

`T.meta_var(x)` 其实就是在跟解析器交代一句：「`x` 是个编译期的 meta 值，你直接把它 inline 进 IR 就行，别当成普通的『脚本变量』去解析。」这里有两个词先解释一下：“inline 进 IR” 就是”把这个值的字面数字直接嵌进生成的代码里”(类似你把常量 `4` 直接写死在代码里，而不是先 `n = 4` 再用 `n`)；“meta 值” 就是”只在翻译阶段存在、翻译完就没了的值”。

这里的 `x`，是你在 Python 这一层就已经算好的值，比如一个 Python `int`(整数)。那加不加 `T.meta_var`，到底差在哪？对比一下你就懂了：

- 不加：解析器可能把 `n = 4` 看成 IR 里的一次变量赋值，于是真给你生成一个 IR 变量节点。
- 加了 `T.meta_var(4)`：这下 `n` 就只是个 Python 整数 `4`。后面哪儿用到 `n`，解析器就把 `4` 这个字面值直接填进去——IR 里根本没有 `n` 这么个变量。

```
n = T.meta_var(4) # n Python int 4, inline
for j in range(n): # meta ->
 acc[0] = acc[0] + A[tx, j]
```

逐行说一下这段在干嘛（里面有几个 GPU 味儿的名字，我先点一下）：

- `n = T.meta_var(4)`：把整数 `4` 标记成 meta 值，`n` 从此就是个普通 Python 整数。
- `for j in range(n)`：一个普通的 Python `for` 循环，转 `4` 圈。
- `acc[0] = acc[0] + A[tx, j]`：这是循环体。`A` 和 `acc` 是 **buffer**——你可以先理解成”GPU 上的一块带类型、带形状的数组/内存”(细节见第 0 章，这里不用深究)。`tx` 是当前线程的编号（GPU 上同时有成千上万个线程在跑，`tx` 就是”我是第几个”）。整句的意思是：把 `A` 里这一行的若干元素，一个个累加到 `acc[0]` 上。

### 2.2 最重要的副作用：驱动循环展开

先解释”展开 / unroll”这个词：把一个循环拆掉，改写成它每一圈的语句一条条排开。比如 `for i in range(3): f(i)` 展开后就是 `f(0)`；`f(1)`；`f(2)`；，循环没了，只剩三条平铺的语句。（为什么要

这么干?GPU 上循环每转一圈都有点额外开销——判断要不要继续、跳转回去——展开掉就省了这些,而且展平后编译器还更容易做进一步优化。这是 GPU 编程里很常见的提速手段,你先有个印象即可。)

**T.meta\_var** 最值钱的本事,就是能让循环在**解析期直接被展开**。

为什么?关键全在 **range(n)** 里那个 **n** 身上。**n** 是个**实打实的 Python 值** (不是 GPU 程序里的变量),所以 **for j in range(n)** 不过是一个最普通的 **Python for 循环**。解析器翻译的时候,会老老实实把这个 Python 循环跑一遍:转一圈,就吐一条对应的 IR 语句。这么一圈一条,循环就被摊平了。换句话说,循环是在”翻译你代码的那个 Python 进程里”跑掉的,GPU 那边压根没看见循环。

还拿上面那段代码举例,它等价于解析器生成了这么一段 IR(意思是这样):

```
acc[0] = acc[0] + A[tx, 0]
acc[0] = acc[0] + A[tx, 1]
acc[0] = acc[0] + A[tx, 2]
acc[0] = acc[0] + A[tx, 3]
```

下面这张图,把「普通的运行期循环」和「meta 值驱动的解析期展开」并排一放,差别一眼就出来了:

#### 结构图

分组:

- **普通 IR 循环变量**: 「for j in range(N) / N 是 IR 变量」「IR 里保留一个 / 真实的 for 循环节点」「运行期在 GPU 上 / 循环执行 N 次」
- **meta\_var 驱动展开**: 「n = T.meta\_var(4) / n 是 Python int」「for j in range(n) / 就是 Python for」「解析器跑一遍 Python 循环 / 逐圈生成 IR 语句」「IR 里没有循环, / 只剩 4 条展开后的语句」

连接关系:

- IR 里保留一个 / 真实的 for 循环节点 → 运行期在 GPU 上 / 循环执行 N 次
- for j in range(n) / 就是 Python for → 解析器跑一遍 Python 循环 / 逐圈生成 IR 语句
- 解析器跑一遍 Python 循环 / 逐圈生成 IR 语句 → IR 里没有循环, / 只剩 4 条展开后的语句

#### 关键

**关键**: 一个 **for** 会不会在解析期展开? 判断起来特别简单——就看它的**循环上界** (还有循环里用到的那些值) 是不是 **meta 值**。而 **T.meta\_var** 干的,正是把一个 Python 值「升格」成 meta 值的那个开关。上界一旦是 meta 值,这个 **for** 就退回成一次普通的 Python 迭代,解析器照着它一条一条把语句铺出来就完事了。

## 2.3 为什么这么设计

- **省掉那种「用一次就扔」的局部变量**: 很多时候你不过是想用一个 Python 算出来的常量 (比如 **tile\_k = 64**。这里的 **tile** 是 GPU 编程里的高频词——因为一个大矩阵往往太大、放不进 GPU



的快速内存，人们就把它切成一个个小方块分批处理，每个小方块就叫一个 `tile`，见第 0 章)，犯不着在 IR 里专门给它立一个变量节点。`T.meta_var` 能让这个值不留痕迹地融进 IR。

- **把 Python 和 IR 的元编程打通：**它把 Python 算出来的结果，干干净净地注入到 IR 的生成过程里。循环展开只是最常见的一种用法而已。说到底，只要你想让「解析器把某个值当成已知常量来处理」，都能用它。

## 三、@T.inline — 内联函数

### 3.1 它在做什么

#### 一句话先理解

一句话先理解：`@T.inline` 是个装饰器 (Python 里那种写在函数上面、用来给函数加特殊行为的 `@xxx`)。被它贴上的函数，在翻译时会被”拆开抄到调用它的地方”，最后生成的代码里根本没有这个函数，只剩它的函数体。

先给个直觉：`@T.inline` 让一个函数「写的时候像函数，出来的时候像手写」。它修饰的函数，函数体会在每个调用点就地铺开 (这个动作就叫 **inline**，内联)。换句话说，生成的 IR 里一个函数调用都看不到，看到的只是把函数体「抄」过去、再把形参 (函数定义里的参数名) 换成实参 (调用时真正传进来的东西) 之后的那几行语句。

```
@T.inline
def add_into(acc, x):
 acc[0] = acc[0] + x

add_into(acc, A[tx, j]) # -> acc[0] = acc[0] + A[tx, j]
```

调一次 `add_into(acc, A[tx, j])`，效果跟你亲手写 `acc[0] = acc[0] + A[tx, j]` 一模一样：形参 `acc` 换成实参 `acc`，形参 `x` 换成实参 `A[tx, j]`，然后整个函数体铺到调用点上，就这么回事。

### 3.2 作用域规则：LEGB + 延迟绑定

原文专门强调：`@T.inline` 遵循 Python 的 LEGB 词法作用域，而且采用延迟绑定 / **late binding**。这俩词听着唬人，其实都是 Python 里现成的规矩，一个个拆开看就明白了：

- **LEGB：**就是 Python 找名字的标准顺序——Local(局部)→ Enclosing(闭包外层)→ Global(模块全局)→ Built-in(内建)。内联函数体里用到的名字，就照这个顺序一层层往外找。
- **延迟绑定：**名字是真正用到它的那一刻才去解析，不是定义的时候就钉死。
- **形参会盖住外层的同名变量 (shadowing)：**正因为走 LEGB，内联函数的形参名要是碰巧跟外层某个变量撞名了，那在函数体里，这个名字指的就是形参 (Local 这一层级优先级最高)，外层那个同名的就被挡在外面了。

#### 注意

**注意：**这条规矩在实战里很要紧。举个例子：外层有个变量叫 `x`，你的内联函数形参也叫 `x`，那函数体里的 `x` 用的是传进来的实参，不是外层那个 `x`。这正是它和朴素宏 (macro) 的区别——宏只是把函数体原封不动粘到调用点，一不留神就会误碰外层的同名变量。`@T.inline` 有作用域规则兜底，行为更像一个真函数，只是省掉了运行期的调用开销而已。

下面这张图, 把「@T.inline」和「真实的函数调用」摆一块儿看:

#### 结构图

连接关系:

- 写法相同, 都长得像函数 / `add_into(acc, A[tx, j])` → @T.inline 版 / 解析期就地展开
- 写法相同, 都长得像函数 / `add_into(acc, A[tx, j])` → 真实函数调用版 / IR 里保留一个 call 节点
- @T.inline 版 / 解析期就地展开 → IR: `acc[0] = acc[0] + A[...]` / (无 call, 无栈帧)
- 真实函数调用版 / IR 里保留一个 call 节点 → IR: `call add_into(acc, A[...])` / (运行期有调用开销)

### 3.3 为什么这么设计

- **既能复用, 又零开销**: 你想把重复的逻辑抽成函数, 让代码好读、好复用; 可又不想在底层 kernel 里背上函数调用的开销。这里要说一下: 在普通 CPU 程序里一次函数调用很便宜, 你平时根本不会在意; 但在 GPU kernel 里, 函数调用往往是要尽量避免的——它会带来额外开销, 还常常挡住编译器的优化 (很多 GPU 编译器干脆默认就把函数全 inline 掉)。@T.inline 正好两头都顾上——写着像函数, 出来像手写。
- **比宏靠谱**: 它不是死板的文本替换, 而是带着 Python 作用域语义的展开。所以同名遮蔽它能处理对, 宏式展开里那些经典的坑也就绕过去了。

## 四、@T.meta\_class — 解析期状态对象

### 4.1 它在做什么

#### 一句话先理解

一句话先理解: @T.meta\_class 让你能用一个普通的 Python 类, 把 kernel 里一堆零散的内存块和状态”打包成一个对象”, 之后只传这一个对象就行, 不用拎着十几个变量到处跑。

先说它治什么病: kernel 里东西一多, 你就得拎着一大把零散的局部变量满世界跑。@T.meta\_class 就是来帮你「装箱」的。

具体来说, 它给一个普通 Python 类贴个标记: 这个类的实例是解析期的 meta 值。它最关键的能耐是——实例的字段 (field) 能装下 buffer (一块带类型和形状的连续内存区, 见第 0 章)、标量这些 IR 对象。这么一来, 你就能把一组相关的分配 (allocation) 和状态打包进一个对象, 然后在 kernel 体里像用普通 buffer 那样, 直接用它的字段。

```
@T.meta_class
class State:
 def __init__(self, smem):
 self.acc = T.alloc_local([1], "float32") # local buffer
 self.buf = T.decl_buffer([64], "float16", smem, scope="shared.dyn") # SMEM

s = State(smem.data)
s.acc[0] = T.float32(0.0) # buffer
... s.buf[i] ...
```

先逐行看懂这段 (里面两个分配函数是 GPU 特有的, 我解释一下):

1. `@T.meta_class`: 贴在类上的标记, 告诉解析器“State 的实例是解析期的状态容器”。
2. `self.acc = T.alloc_local([1], "float32"):T.alloc_local` 是“申请一小块线程私有内存”。GPU 内存是分等级的:local 这种是**每个线程自己独占、速度最快**的一小块 (类比 CPU 的寄存器); 这里申请了能装 1 个 float32 的格子, 存进字段 `self.acc`。
3. `self.buf = T.decl_buffer([64], "float16", smem, scope="shared.dyn"):T.decl_buffer` 是“在一块已有内存上声明一个 buffer 视图”。`scope="shared.dyn"` 里的 **shared** 指的是**共享内存 (SMEM)**——同一组线程能一起读写的一块高速内存, 你可以把它想成程序员手动管理的高速缓存 (详见第 0 章)。这行的意思是: 在传进来的那块共享内存 `smem` 上, 声明一个能放 64 个 float16 的视图, 存进字段 `self.buf`。
4. 后面 `s.acc[0] = ...、s.buf[i]`: 拿到对象 `s` 之后, 直接用它的字段, 跟用普通数组一模一样。

重点在于:State 本身就是个再普通不过的 Python 类——`__init__` 里调用 TIRx 的分配 API, 把结果存进字段。`@T.meta_class` 这个装饰器干的, 是让解析器把 State 的实例当成「解析期的状态容器」, 于是字段才装得下这些 GPU 内存对象、在 kernel 体里也能被正确引用。

#### 4.2 为什么这么设计: 别再「穿线」一堆局部变量

这里的“穿线”是个比喻: 就像把一颗颗珠子串到一根线上, 你得把同一批变量从外层函数一路当参数传进内层函数、再传进更内层……传得到处都是。

一个真正快的 kernel, 流水线上的状态多得吓人。**流水线 / pipeline 也先解释一下**:GPU 上“从内存搬数据”很慢、“算数据”很快, 如果先搬完再算, 算的时候就在干等。聪明的做法是让两件事**错开、重叠着跑**——这一批数据在算的同时, 下一批数据已经在搬了, 就像工厂流水线一样不停工 (详见第 0 章)。但这么搞, 要维护的中间状态就多得吓人:

- 好几个 **mbarrier**(屏障, 让线程互相等齐、对上步调的同步原语);
- 一堆累加器 (**accumulator**, 边算边把结果累加进去的那块寄存器/buffer);
- 各种 **scratch** 视图 / 中间 **buffer**(临时用来暂存中间结果的草稿空间);
- 还有阶段计数、**phase** 标志这类标量。

不打包的话, 你就得把这十样东西全当局部变量, 一路从外层「穿」到内层, 再从内联函数传给下一个内联函数。结果呢? 参数列表越拖越长, 又容易出错, 又难读。`@T.meta_class` 的思路是: 这些逻辑上本来就是一伙的状态, 干脆全收进一个对象里。

##### 结构图

分组:

- 不打包: 穿线一堆局部变量: 「kernel 体」「acc」「buf」「barrier0」「barrier1」「phase」「参数列表越来越长」
- 打包: 一个 **meta\_class** 实例: 「kernel 体」      「s = State(...)」「s.acc」「s.buf」「s.barriers」「s.phase」「接口干净」

连接关系:

- kernel 体 → acc
- kernel 体 → buf
- kernel 体 → barrier0

- kernel 体  $\rightarrow$  barrier1
- kernel 体  $\rightarrow$  phase
- phase  $\xrightarrow{\text{逐个传进各内联函数}}$  参数列表越来越长
- $s = \text{State}(\dots) \rightarrow s.\text{acc}$
- $s = \text{State}(\dots) \rightarrow s.\text{buf}$
- $s = \text{State}(\dots) \rightarrow s.\text{barriers}$
- $s = \text{State}(\dots) \rightarrow s.\text{phase}$
- $s = \text{State}(\dots) \xrightarrow{\text{只传一个对象}}$  接口干净

### 关键

**关键:** `@T.meta_class` 给你的是**条理**, 不是性能。它本身不会冒出任何运行期对象——实例只活在解析期。它治的是「kernel 代码怎么组织」这个病: 把散落一地的流水线状态拢成一个有名字的整体, 让 kernel 体读起来像在操作一台清清楚楚的状态机, 而不是在一堆裸 buffer 里转晕。

## 4.3 和 `@T.inline` 的搭配

`@T.meta_class` 和 `@T.inline` 是天生一对。你可以写一组 `@T.inline` 的「方法式」辅助函数, 让它们都接收同一个 `State` 实例 (比如 `add_into(s, ...)`、`advance(s)`)。内联展开之后, 这些函数既复用了逻辑, 又共享了同一份状态, 接口还特别干净。

## 五、`T.constexpr` —编译期 kernel 参数

### 一句话先理解

**一句话先理解:** `T.constexpr` 让 kernel 的某个参数在编译时就被定死成一个常量 (而不是每次调用才传), 好让编译器照着这个固定值把代码优化得更狠。

`T.constexpr` 用来标记一个**编译期的 kernel 参数**。这个参数会被 `@T.jit` 的 `.specialize(...)` 在编译时「烤」进 (bake in) kernel。这里有两个词解释一下: `@T.jit` 是把 kernel 即时编译 (JIT = Just-In-Time) 的装饰器; `.specialize(...)` 叫“特化”, 意思是“给某个/某些参数填上具体值, 生成一个专门针对这些值的 kernel 版本”。“烤进 / bake in”就是个形象说法——把这个值当成常量焊死进生成的代码里, 拿不出来了。

原文在本章只点了个名, 扔下一句话: 完整细节看《TIRx 入门 (Introduction to TIRx)》。不过照着本书一贯的路子, 我们先吧它的定位讲清楚:

- 普通的 kernel 参数, 是**运行期**才定的——每次调用都能传不同的值。
- 而打了 `T.constexpr` 标记的参数, 在 `.specialize(...)` 那一刻就定死了。这相当于跟编译器说: 「在这个特化 (specialize) 出来的 kernel 版本里, 这个值就是个常量。」
- 既然成了编译期常量, 编译器就敢放手做更狠的优化: **常量折叠** (提前把含常量的算式算出结果)、**循环展开** (就是 2.2 讲的那个)、**砍掉永远走不到的分支**, 样样都行; 还能拿它去驱动数据布局和调度上的决策。这也正是它跟本章前几个工具气味相投的地方——都是「把某个值在更早的阶段就钉死, 好让后面生成的代码更优」。

注意

**注意:**`T.constexpr` 起作用的时机，跟前三个工具不太一样。`T.meta_var`、`@T.inline`、`@T.meta_class` 都在**解析期**动手；而 `T.constexpr` 挂在 `@T.jit` 的 `.specialize(...)` 这套编译期特化机制上。但骨子里的精神是一回事——「决策能提前就提前」，只是落地的位置不同罢了。具体语义以《TIRx 入门》为准，本章只是把它收进「解析器 / 编译期工具」这一家子儿点个名。

六、四个工具的横向对比

把这四件工具塞进同一张表，谁负责干啥就一目了然了：

| 工具                         | 作用对象         | 生效时机                                | 核心效果                                                  | 典型场景                                  |
|----------------------------|--------------|-------------------------------------|-------------------------------------------------------|---------------------------------------|
| <code>T.meta_var(x)</code> | 一个 Python 值  | 解析期                                 | 把值 <code>inline</code> 进 IR；让 <code>for</code> 在解析期展开 | 用 Python 常量驱动循环展开                     |
| <code>@T.inline</code>     | 一个函数         | 解析期                                 | 函数体在每个调用点就地展开,IR 里无 <code>call</code>                 | 抽取可复用逻辑且零调用开销                         |
| <code>@T.meta_class</code> | 一个 Python 类  | 解析期                                 | 实例字段可装 <code>buffer</code> /标量，打包状态                   | 聚合屏障/累加器/ <code>scratch</code> 等流水线状态 |
| <code>T.constexpr</code>   | 一个 kernel 参数 | 编译期<br>( <code>.specialize</code> ) | 把参数烤成编译期常量                                            | <code>tile</code> 尺寸等编译期固定的配置         |

这四个工具其实共用一条思路，一句话就能串起来：

能提前确定的东西，就趁早定下来——不管是值 (`meta_var` / `constexpr`)、函数展开 (`inline`)，还是状态怎么组织 (`meta_class`)。这样既让 IR 更干净、生成的 GPU 代码更快，又不用丢掉 Python 这门宿主语言的元编程表达力。

小结

- 本章这四个工具，都是 TIRx 的**解析期 / 编译期元编程入口**。它们干的都是「在 TVMScript → TIRx IR 这条翻译链上做文章」，本身不对应 GPU 上的任何一条运行期指令。
- `T.meta_var(x)`: 把 Python 算出来的值升格成编译期 `meta` 值，再 `inline` 进 IR。最常见的效果，是让 `for j in range(n)` 这类循环在解析期就被**展开**成一条条铺平的语句。
- `@T.inline`: 函数体在每个调用点就地展开,IR 里看不到函数调用。它遵循 Python 的 **LEGB** 词法作用域和延迟绑定，形参能正确盖住外层的同名变量，所以比朴素的宏更靠谱。
- `@T.meta_class`: 把普通 Python 类的实例变成「解析期状态容器」，字段能装下 `buffer` / 标量。你可以拿它**打包**屏障、累加器、`scratch` 视图这些流水线状态，省得在 kernel 体里拎着一堆零散局部变量到处穿。它和 `@T.inline` 一搭，就能写出特别干净的「带状态辅助函数」。
- `T.constexpr`: 标记编译期的 kernel 参数，由 `@T.jit` 的 `.specialize(...)` 烤进 kernel，细节看《TIRx 入门》。
- 全章就一条主心骨：**能提前定的值、展开、状态组织，就趁早定下来**。这样既留住了 Python 的元编程能力，又让生成的 IR 和 GPU 代码更精简、更快。

延伸阅读

- 原文页面:[Parser utilities —Modern GPU Programming for ML](#)Sys
- `T.constexpr` 与 `@T.jit` 的 `.specialize(...)` 细节: 见本书第二部分「TIRx 入门 (Introduction to TIRx)」。
- 元编程与循环展开的更广背景: 可结合本书 TIRx 语言参考的其余章节一起看。

术语对照

| 中文          | English                |
|-------------|------------------------|
| 解析期 / 解析时   | parse time             |
| 元编程         | metaprogramming        |
| 元值 / meta 值 | meta value             |
| 内联 (就地展开)   | inline                 |
| 循环展开        | unroll                 |
| 词法作用域       | lexical (LEGB) scoping |
| 延迟绑定        | late binding           |
| 遮蔽 (同名覆盖)   | shadowing              |
| 编译期常量       | compile-time constant  |
| 特化          | specialize             |
| 累加器         | accumulator            |
| 屏障          | mbarrier / barrier     |
| 中间暂存 (视图)   | scratch (view)         |

## 第 21 章 · 数据类型与表达式

原文:[Data types and expressions](#)

### 本章要点

#### 本章要点 (TL;DR)

别急,下面这几条现在看不懂很正常,它们是这一章的”地图”。读完全章你再回来扫一眼,就全串起来了。先混个脸熟:

- 我们这一章聊的是 **TIRx** 这套东西怎么给”数据”贴类型标签。(TIRx 是个工具,你先把它理解成”一个帮你把程序翻译成显卡能跑的代码的翻译器”,细节后面讲。)
- 在 TIRx 里, **每个表达式身上都挂着两层类型标签**: 下面一层叫 **dtype**, 管的是”这堆数据是什么二进制位”(几个字节、是浮点还是整数); 上面一层叫 **type**, 管的是”这是个普通的值, 还是一个指针”。
- **dtype** 跟显卡 C++ 代码里的类型一一对得上, 比如 `float32`  $\rightarrow$  `float`、`float16`  $\rightarrow$  `half`、`bfloat16`  $\rightarrow$  `nv_bfloat16`、`float32x4`  $\rightarrow$  `float4`。最后翻译出来的代码用哪个 C++ 类型, 就由它说了算。
- **向量 dtype**(像 `float32x4`, 意思是”4 个浮点数捆成一包”) 是头等公民。后面会讲到, 用它能做”一次搬一大块数据”的加速。但记住一句: 一个数据是不是”向量”, 是 **dtype** 自己决定的, 跟用什么指令搬它没关系。
- 上面那层 **type**, 大多数时候平平无奇 (就是个标量值), 你基本不用管。它真正有戏, 是碰到**指针**的时候。
- 想让一个数据结构指向一个**临时算出来的地址**? 不能直接塞, 得先用一个叫 **T.let** 的办法给这个地址”起个名字”, 绕一道。

### 前置知识

**前置知识:** 这一章会反复提到几个 GPU(也就是显卡) 世界的基本概念——寄存器、共享内存、线程块、TIRx 等等。别担心, 这一章每个词第一次出现时, 我都会当场用大白话讲清楚”它是什么、为什么需要它”, 你不需要提前懂。如果你想先建立一个整体印象, 可以翻一下第 0 章 · 极简入门; 但即使不翻, 直接往下读也完全没问题。

## 21.0 为什么要把”类型”拆成两层?

### 一句话先理解

**一句话先理解:** 同样是”类型”, TIRx 比你平时写的 C++ 多记了一层信息, 因为它的目标不是直接运行, 而是要被翻译成显卡代码, 翻译就得记得更细。

先把背景交代清楚, 不然后面全是云里雾里。

先说几个名词, 都是大白话:

- **GPU**: 就是显卡。它和 CPU 最大的不同是:CPU 擅长一件一件事飞快地干,GPU 擅长成千上万件小事同时干。做 AI(尤其是矩阵乘法) 就特别吃这种”同时干一大堆” 的能力, 所以现在跑模型都靠它。
- **CUDA**: 英伟达 (NVIDIA) 显卡上写程序用的那套语言/工具, 长得很像 C++。你写一段 CUDA 代码, 显卡就能跑。可以把它理解成” 显卡专用的 C++”。
- **kernel(核函数)**: 一段会被丢到显卡上、由成千上万个线程同时跑的函数。后面我们说” 一个 kernel”, 你就想成” 一段跑在显卡上的程序”。

好, 背景有了。现在说为什么类型要两层。咱们从平时写代码聊起。

你写 `float x;` 的时候, 类型就一个维度, 够用了。编译器一看到 `float`, 啥都知道了: 占 4 个字节、是浮点、能拿来做浮点运算——一句话就交代清楚了。

可我们这一章的主角 **TIRx** 不太一样。它不是给人直接运行的代码, 而是一种**中间表示 (IR, intermediate representation)**。

一句话先理解

**一句话先理解**:IR 就是程序” 翻译过程中的半成品”——还没变成最终代码, 但已经不是你写的原始代码了, 夹在中间, 专门给翻译器 (编译器) 看的。

打个比方: 你要把一篇中文翻译成英文, 但你不是一步到位, 而是先把它整理成一份结构清清楚楚的” 提纲”(谁是主语、谁是动词、修饰关系), 再照着提纲译成英文。这份提纲就相当于 **IR**。它比原文啰嗦, 但正因为信息更齐全, 翻译器照着它干活才不容易出错。

那 **TIRx** 具体是什么? 它是 **TVM**(一个把 AI 模型编译成高效代码的开源框架) 里的一个组件, 专门用来**描述一个要跑在显卡上的 kernel**, 最后把它翻译 (术语叫 **lower**, 见 21.1.2) 成 **CUDA** 源码。它要把两件事都顾好:

1. 底层每个数据到底占多少位、怎么摆, 得控制得死死的 (因为最后要生成精确的显卡代码, 差一个字节都不行);
2. 在这份” 提纲” 里, 一眼就能看出” 这是个普通的值, 还是一个指针”(指针就是 C/C++ 里那个” 存的不是数据本身, 而是数据所在地址” 的东西)。

光靠一层 `float` 这种类型, 这两件事顾不全。于是 **TIRx** 干脆把类型劈成了两层:

| 层级 | 名称                 | 回答的问题                | 典型取值                                                                                                            |
|----|--------------------|----------------------|-----------------------------------------------------------------------------------------------------------------|
| 底层 | <code>dtype</code> | ” 这是什么位?”(标量/向量元素类型) | <code>float32</code> 、 <code>float16</code> 、 <code>int32</code> 、 <code>float32x4</code> 、 <code>handle</code> |
| 高层 | <code>type</code>  | ” 这是标量还是指针?”         | <code>PrimType(dtype)</code> 、 <code>PointerType(PrimType(dtype), scope)</code>                                 |

关键

**关键**: 先记住一句话——`dtype` 管” 数据长啥样”(多少位、带不带符号、是不是向量),`type` 管” 这数据是拿来干嘛的”(是个能直接参与运算的值, 还是个指向某块存储的指针)。这两件事一分开, 编译器既能精确地生成底层代码, 又能顺手对指针做点语义检查。

下面这张图就是这个两层结构的概貌:



### 结构图

连接关系:

- TIRx 表达式 `PrimExpr`  $\rightarrow$  `.dtype` 低位类型
- TIRx 表达式 `PrimExpr`  $\rightarrow$  `type` 高层语义类型
- `.dtype` 低位类型  $\rightarrow$  标量: `float32` / `int32` / `bool` ...
- `.dtype` 低位类型  $\rightarrow$  向量: `float32x4` ...
- `.dtype` 低位类型  $\rightarrow$  指针: `handle`
- `type` 高层语义类型  $\rightarrow$  `PrimType(dtype)` / (绝大多数表达式是标量)
- `type` 高层语义类型  $\rightarrow$  `PointerType(PrimType(dtype), scope)` / (只有指针才用)

## 21.1 表达式的 dtype

咱们先讲下面那层——`dtype`, 它最好懂, 基本就是你熟悉的”数据类型”。

在 TIRx 里, 你写下的每一个表达式 (就是任何能算出一个值的东西, 比如 `a + b`、`x[3]`, 术语叫 `PrimExpr`, 原始表达式) 身上, 都挂着一个 `.dtype` 属性, 告诉你这个表达式里每个元素是什么类型。常见的就这么几类, 大部分你写代码都见过:

- **普通浮点**: `float32`(单精度, 4 字节)、`float16`(半精度, 2 字节)、`bfloat16`(也是 2 字节, 但精度分配不一样)。位数越少占内存越少、算得越快, 但精度也越低——AI 计算里经常为了快而用低位数的浮点。
- **整数**: `int32`(32 位有符号整数)、`uint8`(8 位无符号整数)。
- **布尔**: `bool`。
- **低精度浮点**: `float8_e4m3fn`、`float4_e2m1fn` 这一类。这些是专门为 AI 计算造出来的”超窄”浮点格式 (8 位甚至 4 位), 主要图一个省内存、算得快, 代价是精度更低。名字里其实就把位数怎么分写明了: `e4m3` 就是”4 位拿来表示指数 + 3 位拿来表示尾数”。现在不懂这些细节没关系, 知道”它们是更省位的浮点”就够。
- **指针**: `handle`, 说白了就是个 C/C++ 里的指针——存的不是数据本身, 而是”数据待在哪个地址”。这个后面 21.3 节会专门讲。
- **向量形式**: 像 `float32x4`, 意思是”把 4 个 `float32` 打成一包, 当成一个整体用”。这一包对应到显卡代码里是个叫 `float4` 的类型。为什么要打包? 后面 21.1.1 会讲, 简单说就是”一次搬一大包比一个一个搬快”。

### 注意

**注意:** 每个 `dtype` 到了”翻译成显卡代码”那一步 (术语叫 `codegen`, 代码生成), 都会变成一个”对得上号的 CUDA 类型”。换句话说, `dtype` 不是个虚头巴脑的标签, 而是和最终生成的 C++ 类型一一咬合的、实打实的物理类型——它说是 4 字节, 生成出来就真是 4 字节。

### 21.1.1 一个例子: 用各种 dtype 分配 buffer

光看列表不解渴, 上个例子。原文有个示范 `kernel`, 拿好几种 `dtype` 各分配了一块 `buffer`(缓冲区, 你就理解成”一块存数据的内存区域, 类似一个数组”)——有的放在寄存器里, 有的放在共享内存里,

顺序还演了一次” 向量化访存”(就是上面说的” 一次搬一大包”)。

一句话先理解

一句话先理解:GPU 上的存储不是只有一种内存, 而是分好几层, 越靠近计算单元的越快但越小。下面代码里会出现两种, 你先记住这个对比表, 看代码就不懵了:

| 存储                            | 大白话                                            | 谁能看到    | 用什么开                |
|-------------------------------|------------------------------------------------|---------|---------------------|
| 寄存器 (register)                | 最快的一小块私有存储, 类似 CPU 寄存器, 但显卡上每个线程各有自己的一份, 别人看不见 | 单个线程私有  | T.alloc_local(...)  |
| 共享内存 (shared memory, 简称 SMEM) | 一块速度很快, 需要你手动管理的存储, 有点像” 程序员能手动控制的高速缓存”        | 同一组线程共用 | T.alloc_shared(...) |

(这里又冒出” 线程” 和” 一组线程”——线程就是” 一条独立执行的指令流”, 显卡靠成千上万条线程同时跑来加速; 那” 一组线程” 是什么、为什么要分组, 看完代码下面第 2 点就懂了。)

核心片段在下面, 中文注释我都加好了:

```
@T.prim_func
def dtypes(A_ptr: T.handle, O_ptr: T.handle):
 A = T.match_buffer(A_ptr, (256,), "float32") # 256 float32 buffer
 O = T.match_buffer(O_ptr, (256,), "float32")
 T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([64]) # + CTA bx tx

 f16 = T.alloc_local((1,), "float16") # :float16
 bf16 = T.alloc_local((1,), "bfloat16") # :bfloat16
 i32 = T.alloc_local((1,), "int32") # :int32
 u8 = T.alloc_local((1,), "uint8") # :uint8
 b1 = T.alloc_local((1,), "bool") # :bool
 sm = T.alloc_shared((64,), "float16") # (SMEM)tile:64 float16

 v = T.alloc_local((1,), "float32x4") # : dtype (float4)
 v[0] = A.vload([tx * 4], dtype="float32x4") # 4 float32
 O.vstore([tx * 4], v[0]) # 4 float32
 # ... (f16/bf16/i32/u8/b1/sm) ...
```

这段代码里有几个 GPU 专有的写法, 你大概率没见过, 我一条条喂到嘴边:

1. T.alloc\_local((1,), dtype):alloc\_local 就是” 开一块寄存器存储”。前面说了, 寄存器是每个线程自己的一小块私有超快存储, 别人看不到。(1,) 是它的形状, 意思是” 我只要 1 个元素”(就像声明一个长度为 1 的数组)。所以 f16 = T.alloc\_local((1,), "float16") 读作:” 给我开一个 float16 的寄存器变量”。
2. T.alloc\_shared((64,), "float16"):alloc\_shared 开的是一块共享内存。(64,) 表示”64 个元素”。这块地方, 是同一个 CTA 里所有线程一块儿用的。
  - 这里得讲清楚 CTA(线程块) 是什么、为什么有它: 显卡有成千上万条线程, 不可能让它们全互相通信 (太乱、太慢), 于是把线程分成一个个” 小组”, 每组就叫一个 CTA。同一个 CTA 里的线程才能共用共享内存、才能方便地配合干活。所以共享内存是” 按 CTA 分配、给一个 CTA 内的线程共享” 的。打个比方: 整个工厂 (显卡) 几万个工人 (线程), 太多了管不过来,

就编成一个个班组 (CTA), 每个班组有一张共用的工作台 (共享内存), 组内的人可以在这张台子上交换东西。

- 顺带提一句代码开头的 `bx = T.cta_id(...)` 和 `tx = T.thread_id(...):cta_id` 是”我是第几个 CTA(班组)”, `thread_id` 是”我在组里是第几个线程 (工人)”。每条线程靠这俩编号知道自己该处理哪份数据。

3. `v = T.alloc_local((1,), "float32x4")`: 这是本节最要紧的一行。注意 `dtype` 是 `float32x4`(向量), 所以它开出来的是一个 `float4` 寄存器。写 `v[0]` 就能把这个向量整体取出来。这里有个特别容易绊倒人的点: 形状是 `(1,)`, 看着只有”一个元素”, 但这”一个元素”本身就是个 4 宽的向量 (里面装着 4 个 `float`)。所以它实际占的是 4 个 `float` 的位置。

4. `A.vload(..., dtype="float32x4")` 和 `0.vstore(..., dtype="float32x4")`: `vload` 是”向量加载”, `vstore` 是”向量写回”。这两行把内存里挨着的 4 个 `float32` 当成一次 16 字节的搬运整体读/写 (4 个 `float × 4 字节 = 16 字节`)。

- 为啥要这么干? 这是 GPU 上的一个经典提速套路。显卡每发一条”读内存”的指令是有固定开销的; 如果一次只搬 4 字节, 要搬完一大堆数据就得发巨多条指令, 开销摊下来很亏。改成一次搬 16 字节, 指令数直接降到四分之一, 内存带宽就用得更满。这就叫**向量化访存**——用更宽的单次搬运, 换更少的指令次数。

### 21.1.2 它生成什么样的 CUDA?

TIRx 这份”提纲”写完了, 咱们肯定想看看它最后被翻译成的真实显卡代码长啥样。这一步术语叫 **lower(降级 / 下降)**——就是把高层的 IR 一步步降到低层的真实代码。上面那段 TIRx 会被 lower 成下面这堆 CUDA(C++ 风格)。为了让重点更扎眼, 原文把没关系的部分都删了:

```
“c++ half f16_ptr[1]; // float16 -> half nv_bfloat16 bf16_ptr[1]; // bfloat16 -> nv_bfloat16
int i32_ptr[1]; // int32 -> int uchar u8_ptr[1]; // uint8 -> uchar signed char b1_ptr[1]; //
bool -> signed char __shared__ alignas(64) half sm_ptr[64]; // 共享 float16, 带 64 字节对齐
float4 v_ptr[1]; // float32x4 -> float4 (向量)
```

```
v_ptr[0] = (float4)(A_ptr + tx * 4); // 向量化加载: reinterpret 成 float4 再解引用 (float4)(O_ptr
+ tx * 4) = v_ptr[0]; // 向量化写回: 同样 reinterpret 成 float4
```

```

:
- 'half' 'nv_bfloat16' 'uchar' CUDA , bfloat16 , TIRx
↳ dtype
- '__shared__' CUDA , " " 'alignas(64)' 64
↳ (,)
- *(float4*)(...) C/C++ " ": 'float' , "
↳ 'float4' (4 float) " , '*' , / 16

> ** **: , CUDA : ** " float
↳ " " 4 float ", ** 'float4' 16
↳ , " " 'float32x4' dtype , ,

 "TIRx dtype -> CUDA " , :

TIRx ('alloc_local' / 'alloc_shared')	CUDA
'alloc_local'((1,), "float16")	'half f16_ptr[1];
```

```

`alloc_local((1,), "bfloat16")`	`nv_bfloat16 bf16_ptr[1];`
`alloc_local((1,), "int32")`	`int i32_ptr[1];`
`alloc_local((1,), "uint8")`	`uchar u8_ptr[1];`
`alloc_local((1,), "bool")`	`signed char b1_ptr[1];`
`alloc_shared((64,), "float16")`	`__shared__ alignas(64) half sm_ptr[64];`
`alloc_local((1,), "float32x4")`	`float4 v_ptr[1];`

21.1.3 dtype " ", vload

, : " " `vload` ,

> ** **: , dtype ** ** , `vload`/`vstore`
↪ `T.alloc_local((1,), "float32x4")` , `float4`
↪ , `vload`/`vstore` " "
↪ dtype, **, buffer , dtype**, `vload`
↪

** (orthogonality)**
↪ , : ,
↪ " " " "
↪ , (`vload`); ,
↪ buffer ,

21.1.4 dtype -> CUDA

, :

TIRx dtype	CUDA	
`float32`	`float`	
`float16`	`half`	
`bfloat16`	`nv_bfloat16`	brain float,8 7
`int32`	`int`	32
`uint8`	`uchar`	8
`bool`	`signed char`	`signed char`
`float32x4`	`float4`	(4xfloat32)
`handle`	`T*` ()	, `T`
dtype	CUDA	`<` `>x<N>` -> CUDA `<` `><N>`

> ** **: , `bool` `signed char`, C++ `bool`? : GPU
↪ 1 , (`float8_e4m3fn` `float4_e2m1fn`
↪), CUDA ,

21.2 dtype type ?

`dtype` , `type` ? :

- **`dtype`** : " ()"
- **`type`** : " , "

(" " " " : ** ** ,
↪ `3.14`; ** ,)

`type` ? :

```

```
- ** **, `type` `PrimType(dtype)`(`PrimType` " ") `float32`
↳ ,`type` `PrimType("float32")`, " float32 "
- ** **, `type` `PointerType(PrimType(dtype), scope)`(`PointerType` " ")
↳ `dtype` " ",`scope` " "(`shared`
↳ `global`)

> ** **: , `type` , `PrimType(dtype)`, **`type`
↳ , ** ? dtype
↳ " " " , `type`

``mermaid
graph LR
 subgraph SCALAR_CASE[" "]
 S["float32 " --> SP["type = PrimType('float32')"]]
 end
 end
 subgraph PTR_CASE[" "]
 P[" shared uint64 " --> PP["type = PointerType(PrimType('uint64'), 'shared')"]]
 end
 end
```

21.3 指针 (handle)

接着上节说——指针就是 **type** 系统真正能耍开的地方, 这节好好讲。

在 TIRx 里, 一个指针露面时, 它的 dtype 是 **handle**(通用指针)。但光看 dtype 你只知道” 这是个指针”, 并不知道它指向什么、住在哪——这些” 身份证信息” 是上层的 **PointerType** 替它保管的。这节聊的是个挺实在的事: 一个指针, 你怎么正经地拿到手、再用起来。

21.3.1 一条核心规矩:buffer 的 data 是不可变的 Var

一句话先理解

一句话先理解: 每个 buffer(那块存数据的内存区) 底下都垫着一根指针, 告诉它” 我管的数据从哪个地址开始”。这根指针有个硬规矩——一旦定了就不能改。这条规矩是后面所有麻烦的根源, 先记住它。

每个 buffer 都有个 **data** 字段, 它就是这个 buffer 底下垫着的那根指针 (指向这块数据的起始地址)。关于 **data**, 记住两点就够:

- 1. 它是个**指针类型**的 **Var**。这里的 **Var** 就是” 变量”(variable)——在 IR 里, 一个有名字、能被引用的东西就叫一个 **Var**。你可以把它理解成一个” 具名的盒子”。
- 2. 它**改不了 (immutable, 不可变)**——这根指针第一次定下来之后, 就不许再指向别处了。

关键

**关键:** 干嘛先把这条规矩摆出来? 因为下面那几种” 取指针” 的写法, 全是被这条规矩逼出来的。规矩特别简单:**data** 只认 **Var**(具名的变量), 不收表达式 (临时算出来、还没名字的东西)。所以, 你别想着把一个” 现场算出来的地址表达式” 随手塞给 **buffer** 当 **data**——必须先把它” 安顿” 成一个有名字的 **Var**, 才行。这就好比某个登记表只接受” 已经登记在册的住户”, 你不能填一个” 正在路上还没落户的人”。

21.3.2 拿到指针的三种方式

把上面那条规矩（data 必须是 Var）揣兜里，取指针就有三条路可走，各管各的场景。先看表，下面再逐一讲：

| 方式      | API(怎么写)                     | 它做了什么                                      | 什么时候用                           |
|---------|------------------------------|--------------------------------------------|---------------------------------|
| 分配新存储   | T.alloc_buffer(...)          | 开一块新存储，顺手就把 data 指针定义好了                    | 你需要一块全新的内存                      |
| 复用已有指针  | T.decl_buffer(..., data=ptr) | 不开新存储，而是在一个已有的指针 Var(就叫 ptr) 上面声明一个 buffer | 指针你已经有了，只是想换一个 buffer ”视角” 去访问它 |
| 绑定指针表达式 | 先 T.let 成 Var，再 decl_buffer  | 先把一个临时算出来的指针表达式绑定成一个有名字的 Var，再拿它去声明 buffer | 指针不是现成的，而是现场算出来的（比如跨 CTA 的远程地址） |

简单解读一下这三条：第一条最常见，要新内存就用它，它一步到位；第二条是”我已经有指针了，只想换个数组视图来读写同一块内存”；第三条最绕，专门对付”指针是算出来的”这种刁钻情况——也正是它逼出了 T.let 这个写法，下一节细讲。

不管你走哪条，终点都一样:data 最后必须是个合法的 Var。下面这张流程图，就是把三条路怎么殊途同归到这个终点画了出来：

结构图

连接关系：

- 我手上有什么？什么都没有，需要新存储→ T.alloc\_buffer(...) / 分配存储并定义 data
- 我手上有什么？已有指针 Var→ T.decl\_buffer(..., data=ptr)
- 我手上有什么？只有一个指针表达式→ 先 T.let 绑定成 Var
- 先 T.let 绑定成 Var → 再 T.decl\_buffer(..., data= 那个 Var)
- T.alloc\_buffer(...) / 分配存储并定义 data → data 是合法的 Var
- T.decl\_buffer(..., data=ptr) → data 是合法的 Var
- 再 T.decl\_buffer(..., data= 那个 Var) → data 是合法的 Var

21.3.3 绑定指针表达式：访问远程 CTA 的共享内存

三条路里头，第三条最有意思，也最值得掰开讲。它对付的是这么一种情况：你手上的指针不是现成的、有名字的 Var，而是当场算出来的一个表达式（还没名字）。这时你绕不开——得先用 T.let 给它”起个名字”、变成一个 Var，然后才好递给 decl\_buffer。

那什么时候会碰上”算出来的指针”呢？原文举的典型场景是**集群（cluster）**里的分布式共享内存。这一句信息量有点大，我拆开讲清楚：

- 前面说过，共享内存只在一个 CTA(线程块) 内部共享。可有时候我们希望”几个 CTA 之间也能互相读写彼此的共享内存”，光靠基本机制做不到。
- 于是新一代显卡引入了 **cluster(集群)** 这个概念：把几个 CTA 再打成一个更大的”团”，团里的 CTA 可以互相够到对方的共享内存。这种”跨 CTA 互相访问的共享内存”就叫**分布式共享内存 (distributed shared memory)**。打个比方：原来每个班组 (CTA) 只能用自己的工作台（共享内存）；现在几个班组组成一个车间 (cluster)，车间里的班组之间也能伸手去够隔壁那张工作台了。

- 但要够到”隔壁班组那张工作台上的某个位置”，你得先**算出它的地址**。这个换算就靠一个叫 `T.ptx.map_shared_rank` 的操作（它底层对应显卡的一条低级指令 `mapa`，你不用记）。它干的活是：把”我自己 CTA 里某个共享内存位置的地址”，换算成”集群里另一个 CTA 上同一个位置的地址”。
- 关键点来了：这个换算出来的结果，**天生就是个临时算出来的地址表达式**，还没名字。所以它正好撞上 21.3.1 那条规矩，只能走”先 `T.let` 起名字，再 `decl_buffer`” 这第三条路。

下面就是这段代码。它一上来扯进来好几个底层名词，别慌——你只需要抓住一条主线：**把一个算出来的地址，一步步变成一个能正常读写的 buffer**。

```
from tvn.ir.type import PointerType, PrimType

1) map_shared_rank(PTX mapa) CTA ()
reinterpret handle, T.let PointerType Var "ptr"
ptr: T.let[T.Var(name="ptr", dtype=PointerType(PrimType("uint64")))] = \
 T.reinterpret("handle", T.ptx.map_shared_rank(mbar.ptr_to([0]), 0))

2) ptr Var, buffer
CTA mbarrier(uint64), shared
remote_mbar = T.decl_buffer([1], "uint64", data=ptr, scope="shared")
```

这段代码从里到外是四步，咱们一行一行拆（从最内层往外读）：

1. `T.ptx.map_shared_rank(mbar.ptr_to([0]), 0)`：这是最里面的一步，就是上面讲的”换算地址”。它把当前 CTA 里某个 `mbarrier` 的地址，换算成集群里目标 CTA(0 是目标 CTA 的编号) 上同一个位置的地址。
  - 这里又冒出个新词 **mbarrier**：它是放在共享内存里的一个”同步计数器”，作用是让一群线程互相等——典型用法是”等数据全部搬到齐了，大家再一起往下走”。它内部就是用一个 `uint64` 存的，所以后面你会看到处处是 `uint64`。这个机制本章不展开，你知道”它是个共享内存里的同步用的小变量”就够。
  - 这一步吐回来的，是一个**地址表达式**（临时算出来的，没名字）。
2. `T.reinterpret("handle", ...):reinterpret` 是”按另一种类型重新解释这串二进制位”（类似 C++ 的 `reinterpret_cast`）。这里把上一步的结果重新解释成 `handle`，也就是”通用指针”这个 `dtype`。到这儿它还是个没名字的表达式。
3. `ptr: T.let[T.Var(..., dtype=PointerType(PrimType("uint64")))] = ...` 这步就是关键的”起名字”。前面反复念叨 `data` 必须是 `Var`（有名字的变量），所以这儿用 `T.let` 把那个表达式绑定到一个叫 `ptr` 的指针 `Var` 上。`T.let` 你就理解成”把右边算出来的东西，起个名字叫 `ptr`，以后就能用 `ptr` 引用它”。
  - 这个 `ptr` 的上层 `type` 写成 `PointerType(PrimType("uint64"))`，意思是”一个指向 `uint64` 的指针”（因为 `mbarrier` 就是 `uint64` 存的）。
  - 留意一个小细节：这里的 `PointerType` 只填了”指向什么类型”，没填 `scope`（它在哪层存储）。作用域 `"shared"` 故意留给了下一步的 `decl_buffer`——两边各管一摊，分工清楚。
4. `remote_mbar = T.decl_buffer([1], "uint64", data=ptr, scope="shared")`：最后一步，在那个合法的指针 `Var(ptr)` 上面声明出一个 `buffer`：1 个元素、类型 `uint64`、作用域 `shared`。打这儿起，你每读写一次 `remote_mbar`，实际碰到的就是远端那个 CTA 的共享内存了。

## 注意

**注意：**这段代码别被它唬住。它一口气塞进来好几个底层概念——地址换算、集群里的分布式共享内存、`mbarrier`、`reinterpret`。但本章真正要你记住的不是这些机制本身，而是一条规则：指针表达式不能直接拿来当 `data`，必须先用 `T.let` 绑成有名字的 `Var`。这条想通了，你就明白这段代码为啥非得绕这么一圈、套个 `T.let` 了——不是作者故意复杂，是规矩逼的。

最后拿一张图收个尾，看看一个“远程共享地址表达式”是怎么一步步熬成一个能用的远程 `buffer` 的：

## 结构图

连接关系：

- 远程 CTA 的共享地址 / (PTX `mapa`, 表达式)  $\xrightarrow{\text{reinterpret}(\text{"handle"})}$  通用指针 `handle` / (仍是表达式)
- 具名指针 `Var` / (`PointerType`, 必须的具名化)  $\xrightarrow{\text{decl\_buffer}(\text{data}=\text{ptr})}$  `remote\_mbar` / (`uint64`, `scope="shared"`) / 最终可读写的远程 `buffer`

## 小结

这章没多长，但 `TIRx` 类型系统的三根顶梁柱算是立住了。如果开头的 `TL;DR` 你当时没看懂，现在再回头扫一眼，应该顺了：

1. **类型分两层：**下面那层 `dtype` 管“是什么位”（几个字节、什么格式），它跟显卡代码里的标量 / 向量类型一一对得上；上面那层 `type` 管“是值还是指针”（要么是普通值 `PrimType`，要么是指针 `PointerType`）。是普通值时上层没啥可说的；只有碰到指针，上层才扛起两件要紧事——指向谁、在哪层存储。
2. **向量 `dtype` 是“正交”的：**像 `float32x4` 这种“打包”类型是头等公民，声明个寄存器就直接到手一个 4 宽向量；“向量化访存”（一次搬一大包来提速）只是它众多用法里的一种。一句话：向量这个属性是 `dtype` 自己带的，跟用什么指令搬它（`vload/vstore`）不捆在一起。
3. **拿指针得守规矩：**`buffer` 底下的 `data` 指针是个改不了的、有名字的 `Var`。三条取指针的路——直接开新存储（`alloc_buffer`）、复用现成指针（`decl_buffer(data=ptr)`）、给算出来的地址先起名字再用（先 `T.let` 再 `decl_buffer`）——说到底都是为了守住同一条底线：**`data` 必须是有名字的 `Var`，不能是临时算出来的表达式**。指针要是现场算出来的（比如跨 CTA 去够别的块的共享内存地址），那就先用 `T.let` 给它起个名字。

把这三条串一块儿你就咂摸出味儿了：`TIRx` 的类型设计自始至终就奔着一个目标——既要把握底层 `CUDA` 代码精确地生成出来，又要在 `IR` 这层就把“值 vs 指针”这种语义差别拎得明明白白，还能拿来做校验。

## 延伸阅读

- 原文 (英文): [Data types and expressions — Modern GPU Programming for ML Sys](#)

## 术语对照



| 中文       | English                   | 备注                            |
|----------|---------------------------|-------------------------------|
| 数据类型     | dtype                     | 低层级类型                         |
| 高层类型     | type                      | PrimType / PointerType        |
| 原始表达式    | PrimExpr                  | TIRx 表达式基类                    |
| 原始类型     | PrimType                  | 标量的高层类型                       |
| 指针类型     | PointerType               | 指针的高层类型, 含元素 dtype 与作用域       |
| 句柄 / 指针  | handle                    | 通用指针 dtype                    |
| 向量 dtype | vector dtype              | 如 float32x4 → float4          |
| 向量化访存    | vectorized memory access  | vload / vstore                |
| 共享内存     | SMEM (shared memory)      | CTA 内共享                       |
| 寄存器      | register                  | alloc_local                   |
| 作用域      | scope                     | 如 "shared"、"global"           |
| 不可变      | immutable                 | buffer 的 data 指针              |
| 降级 / 下降  | lower                     | IR → CUDA 代码生成                |
| 集群       | cluster                   | 多 CTA 组成的协作单元                 |
| 分布式共享内存  | distributed shared memory | 跨 CTA 访问彼此 SMEM               |
| 低精度浮点    | low-precision float       | float8_e4m3fn、float4_e2m1fn 等 |



## 第 22 章 · 缓冲区与内存

原文: [Buffers and memory](#)

### 本章要点

#### 本章要点 (TL;DR)

别被下面这些词吓到, 每一个本章后面都会从零讲明白, 这里只是先让你扫一眼全貌。

- **缓冲区 (buffer)** 的本质就是「指针 + 元数据」: 你可以先把缓冲区理解成「一块数据 + 一张说明书」。同样一句 `B[i, j]` 的下标访问, 因为说明书 (我们叫它元数据: 布局 `layout`、偏移 `elem_offset`) 不一样, 最终算出来的内存地址就完全不同。
- 创建缓冲区只有两个根本 API: `T.alloc_buffer`(真的去申请一块新存储) 和 `T.decl_buffer`(在已有的一块存储上, 再贴一张新说明书, 这叫一个视图 `view`)。 `alloc_shared` / `alloc_local` 都只是 `alloc_buffer` 的简写。
- `scope`(作用域 / 内存空间) 决定了这块数据放在 GPU 的哪种内存里: `global`(所有线程都能访问的大内存) / `shared`(一小组线程共用的高速内存, 静态版) / `shared.dyn`(高速内存的动态版) / `local`(单个线程私有的寄存器, 最快) / `tmem`(新一代显卡上专门给矩阵乘用的张量内存)。
- **共享内存有静态与动态两种**: 静态的大小在写代码、编译时就定死了; 动态的是一整块在「启动 kernel (运行 GPU 程序) 那一刻」才定大小的区域, 我们叫它 `arena`(竞技场), 整个 GPU 程序只能有这么一块, 你想用的其它缓冲区都是从它里面切出来的视图。 `T.SMEMPool` 这个工具能自动帮你算「谁放在这块大区域的第几个字节」这种繁琐账。
- **标量 (scalar, 就是单个数, 比如一个计数器)** 其实是「只有一个元素的寄存器数组」的语法糖; 而 `T.let` 是不可变绑定 (起个名字绑定一个算好就不再变的值), 会变成普通的 C 局部变量, 编译器能放开手脚优化它。 **张量内存 (TMEM)** 脾气特殊: 必须你亲手申请和释放, 还不能直接用地址访问, 只能通过专门的 `tcgen05` 指令来读写。

### 前置知识

**前置知识**: 这一章会用到几个 GPU 的基本概念, 我先用一两句话给你交个底, 后文每个都会展开讲, 所以读不懂也别慌。

- **GPU 为什么有「内存层级」?** 因为 GPU 上同时跑着成千上万个小工 (线程) 在算数。给每个线程都配一份「跑得最快但极小」的内存 (寄存器), 给一小组线程配一块「快、稍大、能互相传东西」的内存 (共享内存), 再给所有线程配一块「慢但巨大」的内存 (全局内存)。越快越小、越慢越大, 这就是「层级」。本章主角就是教你怎么在 TIRx 里申请和使用这几层内存。
- **寄存器 / 共享内存 / 全局内存**就是上面这三层, 从快到慢、从小到大。
- **静态 vs 动态共享内存、TMEM** 是本章后面会专门讲的词, 现在不认识完全没关系。
- 至于 TIRx 的基本写法 (`T.prim_func`、`T.match_buffer` 这类装饰器和函数), 有个印象就

行,看到时我会顺带解释。

没把握的话,先翻一下第 0 章·极简入门,以及第 9 章 TIRx。

### 一句话先理解

**一句话先理解:** 在 TIRx 里,一个缓冲区 (buffer) 不是装数据的盒子,而是「一根指针 + 一张说明书」。说明书改一改,同一句下标就指向不同的地方。

这一章讲的是 TIRx 的**内存模型**,在整个「语言参考」里算是骨干。这里先解释两个词:**TIRx** 你就理解成「一种用 Python 写、专门用来生成 GPU 代码的语言」;**内存模型**就是「这门语言怎么描述数据放在哪、怎么访问」。

想读懂这一章,你首先要抓住一句话:在 TIRx 里,一个缓冲区 / buffer **并不是一个真的装着数据的「盒子」**。

打个比方。在普通编程里你可能习惯把数组想象成「一排带格子的柜子,数据就躺在格子里」。但在 TIRx 里,缓冲区更像是贴在一根**指针**(指针就是「某块内存从哪个地址开始」的记号,跟 C 语言里的指针是一回事)上的一张**标签**——标签上写着「这块数据有多大、是几维的、每个元素多大、按什么顺序摆放、该怎么从下标算出地址」。真正的数据躺在指针指向的那块内存里,而缓冲区本身只是这张标签。这些写在标签上的描述信息,我们统称**元数据**(metadata,就是「描述数据的数据」)。

为什么要这么设计?说白了,就是图个省事和高效:这样一来,绝大多数缓冲区方法干的活儿,无非是在**编译期**(也就是「把你的代码翻译成 GPU 能跑的机器码」那个阶段,程序还没真正运行)改改这张标签——换个形状、换种读法而已。它们影响的只是「下标最后怎么翻译成内存地址」,等程序真跑起来时,一条多余的指令都不会多出来。这个道理一旦想透,后面所有 API 为什么长成那个样子,你立马就全懂了。

## 一、缓冲区的统一心智模型

先混个脸熟:在 TIRx 里,缓冲区来来回回就这么几种玩法。这几行代码里有些函数你不认识没关系,先看右边的中文,知道「能拿缓冲区干哪几类事」就够了。

- **包装参数指针:** `T.match_buffer(ptr, shape, dtype, ...)`, 把外面 (比如 GPU 程序的调用方) 传进来的一根裸指针,包装成一个有形状、有类型的缓冲区,这样你才能用 `A[i, j]` 这种舒服的下标去访问它。
- **下标访问:** `A[i, j]`, 读写某个元素,跟你平时用二维数组一模一样。
- **切片:** `A[m0:m0+BM, 0:BK]`, 取出一块子区域 (一个矩形小块), 得到一个 `BufferRegion`(缓冲区区域 / buffer region, 就是「缓冲区里的一块范围」)。
- **取指针:** `A.ptr_to([i, j])` 拿某个元素的内存地址,或者 `A.data` 拿这块存储最开头的那根原始指针。

这里有一句最要紧的话:只要不是 `tmem`(那个脾气特殊的张量内存,本章最后才讲),所谓「声明一个缓冲区」,说到底就是「给一根指针配上一个布局 layout」。

这里插一句解释 **布局 / layout** 是啥:它就是「数据在内存里按什么顺序摆」的规则。同样一个二维数组,你可以一行一行地铺 (行主序),也可以一列一列地铺 (列主序),摆法不同,`A[i, j]` 这个元素实际落在内存第几个位置就不同——布局管的就是这件事。

「指针 + 布局」这两样凑齐了，随便你写哪个下标，最后都能算出一个内存地址来。这个地址怎么算，原文给了个公式，值得一记：

```
addr(buffer[coord]) = buffer.data + elem_offset + layout.apply(coord, shape=shape)["m"]
```

公式看着挺唬人，但拆开其实就三块加起来：「起点 + 挪一段 + 按坐标算偏移」。打个比方，这就像在停车场找车位：先有停车场大门口（起点），再走到你那一排的入口（挪一段），最后数到第几个车位（按坐标算）。一个个说：

| 组成部分                                       | 大白话                                                                                                                                          |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>buffer.data</code>                   | 起点指针，也叫基址 / base pointer，就是这块存储从哪个内存地址开始                                                                                                     |
| <code>elem_offset</code>                   | 在起点的基础上再往后挪几个元素；它的用处是，让一个「视图」从 <code>data</code> 的某个位置开始，而不是非得从最开头开始                                                                         |
| <code>layout.apply(coord, ...)["m"]</code> | 把多维坐标 <code>coord</code> （比如 <code>[i, j]</code> ）换算成一维的「第几个元素」； <code>"m"</code> 就是这个换算结果里的「元素偏移」那一项（ <code>layout</code> 还会算别的项，这里我们只关心偏移） |

符号你不用死记，记住这条思路就够了：从哪儿开始 + 往后挪多少 + 坐标算出多少，三者一加就是地址。

关键

关键：同样一句 `B[i, j]`，就因为缓冲区的元数据不一样，编译出来的地址算术能差出十万八千里。元数据是编译期就钉死的，到了运行时，剩下的不过是对裸指针做加减乘罢了。

下面我用一组对照例子，把这套心智模型给你坐实。先说明一下：行主序 (row-major) 就是「一行一行地铺」，第 0 行铺完接着铺第 1 行，这是最常见的摆法 (C 语言、NumPy 默认都是它)。在一个有 8 列的数组里，`A[i, j]` 这个元素就落在第 `i*8 + j` 个位置——你想啊，前面 `i` 整行各占 8 个，再加上这一行里的第 `j` 个，正好。

我们对一块 `4×8` (4 行 8 列) 的区域干同一件事 `B[i, j] = A[i, j] + 1` (把 `A` 的每个元素加 1 存到 `B`)，但只动 `B` 的声明方式，看看地址会跟着怎么变。`A` 我们让它一直是行主序，所以 `A[i,j]` 的下标永远是 `i*8 + j`，雷打不动——这样你就能专心盯着 `B` 一个看了：

```
from tvm.tirx.layout import TileLayout, S

1) (row-major):
B = T.match_buffer(p, (4, 8), "float32")
2) (column-major): TileLayout stride (1, 4)
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, 8):(1, 4)]))
3) (shifted view): 64
B = T.match_buffer(p, (4, 8), "float32", elem_offset=64)
4) stride 16: 16 8
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, 8):(16, 1)]))
```

还是那句 `B[i, j]`，这四种声明生成的 CUDA 下标，各是各的样：

“`c++ B_ptr[((i / 8) + j)] = ...; // 行主序: i8 + j B_ptr[((j / 4) + i)] = ...; // 列主序: j4 + i B_ptr[((i / 8) + j) + 64]] = ...; // elem_offset=64: i8 + j + 64(整体平移 64) B_ptr[((i / 16) + j)] = ...; // 行 stride 16: i16 + j(每行间隔 16)`

```

, : ** , `B[i, j]` ** `B[i, j]`
↪ `j*4 + i`(`, 4, `j` 4);`elem_offset=64` 64; stride
↪ 16, 16 (8) , ,
↪

> ** `TileLayout(S[(shape):(strides)])` : / stride (stride) ?
↪ 1, 8 , `j` 1 1
↪ , 1; `i` 1 (8), 8 `S[(4,8):(1,4)]`
↪ : 0 () 1 1 () 4 1 4 1 1
↪ , ,

: API

API, : ** **

> ** `alloc_buffer` ;`decl_buffer`
↪ ,

2.1 `T.alloc_buffer`

```python
T.alloc_buffer(shape, dtype, scope=..., ...)

```

它会实打实地去申请一块新存储 (用术语说, 是在 IR 里吐出一个 `AllocBuffer` 节点;IR 就是「程序的中间表示」, 你可以理解成编译过程中代码的内部样子), 然后把一个 `Buffer` 对象还给你。它有两个常用的简写:

- `T.alloc_shared(...)` 就是 `scope="shared"`(放进共享内存) 的 `alloc_buffer`。
- `T.alloc_local(...)` 就是 `scope="local"`(放进寄存器) 的 `alloc_buffer`。

(`scope` 这个词指「这块内存放在哪一层」, 马上 2.3 会列全。)

2.2 `T.decl_buffer` ——在已有指针上声明视图

```
T.decl_buffer(shape, dtype, data=..., ...)
```

它一块新存储都不申请, 只是在一根现成的指针 `data` 上「再开一个视图」。说白了, 就是给同一块存储起个别名 (alias, 同一块内存的另一个名字), 或者换种眼光重新解读 (reinterpret) 它——比方说「我想把某块大内存里中间那一段单独当成一个数组来用」。这在后面讲的内存池、张量内存里会反复用到。

关键

关键:`decl_buffer` 有个小例外——要是你传了 `data=None`(意思是「我没有现成指针给你」), 它手头就没存储可用了, 这时它会退一步, 像 `alloc_buffer` 那样自己掏腰包申请一块。所以你可以把 `alloc_buffer` 看成 `decl_buffer(data=None)` 的特例。

归根结底, 这两者的差别就一处: 那根 `data` 指针 (它在 `TIRx` 里是个不可变的 `Var`, `Var` 你就理解成「一个一旦绑定就不能再改的变量名」) 究竟从哪儿来:

API	data 指针从哪来
alloc_buffer	它自己造出这个 data(这次分配的产物)
decl_buffer	它接收一个已经存在的 data(从外面传进来)

注意

注意: 要是你手里是个**指针表达式** (而不是现成的 **Var**), 想拿它来撑起缓冲区, 那得先把它绑定成一个 **Var** 才行 (具体怎么绑, 见原书「数据类型与表达式」那章)。

2.3 共享的描述符参数

这两个 API 共用同一套描述参数。挑几个最常打交道的看看:

参数	含义
dtype	元素类型, 如 "float32"、"float16"、"float4_e2m1fn" 等
shape	逻辑形状 (各维 extent 组成的元组)
layout	物理映射 (TileLayout); "default" 即稠密行主序
elem_offset / byte_offset	把一个视图放到 data 的某个偏移处
allocated_addr	携带一个 预先指定的地址 (用于张量内存)
align	数据指针的对齐字节数

还有个 **scope** 参数, 它说了算的是: 这块缓冲区到底落在 GPU 的**哪一层内存**里。前面前置知识里说过 GPU 内存分快慢层级, 这张表就是把每一层对应到一个 **scope** 名字。这条线索本章后面几节会反反复复出现, 你先认个脸:

Scope	快捷方式	对应内存 (大白话)
"global"	(默认)	全局内存 (global memory): 最大但最慢, 所有线程都能访问, 数据从外面进来一般先落这里
"shared"	T.alloc_shared	静态共享内存: 一小组线程共用的高速便签纸, 大小编译时定死 (对应 C++ 里的 __shared__)
"shared.dyn"	(走内存池)	动态共享内存: 同样是高速便签纸, 但大小启动时才定 (下文细讲)
"local"	T.alloc_local	寄存器 (registers): 每个线程私有、最快, 放它自己的临时小数据
"tmem"	(走 TMem 池)	张量内存 (tensor memory): 新一代 Blackwell 显卡上专给矩阵乘用的特殊内存

下面这段把四种典型用法摆在一块儿, 你对照着看:

```
A = T.match_buffer(A_ptr, (M, K), "float16", align=16) #
As = T.alloc_shared((BM, BK), "float16") # (tile)
acc = T.alloc_local((4,), "float32") # 4 ,
view = T.decl_buffer((BM, BK), "float16", data=As.data) # , As
```

逐行说一下 (这里 **M K BM BK** 都是表示尺寸的数字): 第一行把调用方传进来的裸指针 **A_ptr** 包成一个 **M×K** 的 **float16**(16 位浮点数) 缓冲区; 第二行真的去共享内存里要了一块 **BM×BK** 的新地

方；第三行在寄存器里要了 4 个 `float32`(32 位浮点数) 的格子；第四行没要新内存，只是把第二行那块 `As` 的指针 (`As.data`) 拿来，贴一张新说明书重新看它。`tile`(小方块) 这个词在 GPU 编程里很常见，指「从一个大矩阵里切出来的一小块」，后面会多次出现。

三、共享内存 (shared memory)

一句话先理解

一句话先理解：共享内存是「一小组线程能共用的一块高速便签纸」，比全局内存快得多，但容量很小。线程之间想互相传数据，把它当中转站最划算。

先说说**为什么需要共享内存**，不然你不知道它解决什么问题。全局内存又大又慢，如果每个线程算东西都反复去全局内存读同一份数据，会慢得要命。于是 GPU 给「一组协作的线程」配了一小块又快又近的内存，叫共享内存：大家先把要反复用的数据从全局内存搬一份到这里，之后就在这块快内存上来回读写，省下大量慢访问。它有点像你给一组同事配了一块共用白板——比每人都跑去档案室翻文件快多了。

共享内存 / shared memory(往下简称 SMEM) 有两种「口味」：**静态**和**动态**。怎么分？就看一点——它的大小是**编译时**(写代码翻译阶段) 就定死了，还是**每次启动 kernel 的时候**(运行时) 才现定。这里的 **kernel** 就是「一段在 GPU 上跑的程序」，你每次让 GPU 干活就叫「启动 (launch) 一个 kernel」。除此之外 TIRx 还配了个内存池 (pool) 帮手，专门收拾动态这种麻烦事，后面会讲。

结构图

连接关系：

- 共享内存 SMEM → 静态 `static / scope=shared`
- 共享内存 SMEM → 动态 `dynamic / scope=shared.dyn`
- 静态 `static / scope=shared` → 编译期定大小 / `__shared__ T x[N]`
- 动态 `dynamic / scope=shared.dyn` → launch 时定大小 / `extern __shared__ arena[]`
- 动态 `dynamic / scope=shared.dyn` → `T.SMEMPool / 自动 bump 分配 + 偏移记账`

3.1 静态共享内存

最省事的共享缓冲区就是**静态**这种——用 `T.alloc_shared`(也就是 `scope="shared"`)，大小在编译时就钉死。

它最经典的套路是三步走：**先把数据从全局内存搬进共享内存 → 拿 `cta_sync` 让整组线程都「看见」这次写入 → 再读回来用。**

中间这步 `cta_sync` 很关键，先解释一下**为什么需要它**。GPU 上一组线程是同时在跑的，但它们的快慢并不完全一致。假设线程 A 往共享内存写了个数，线程 B 想读这个数——可万一 B 跑得快，在 A 还没写完就来读了呢？那读到的就是垃圾。`cta_sync` 就是一道「集合栅栏」：所有线程跑到这一行都得停下等齐，等大家都到了再一起往下走。这样就保证「等到栅栏之后，前面所有人写的东西，大家都能看到了」。

这里再解释两个词：**block / CTA** 指「一组会协作、并且共享同一块共享内存的线程」(block 是俗称,CTA 是它的正式名 Cooperative Thread Array);`cta_sync` 里的 `cta` 就是指它。


```

@T.prim_func
def smem_demo(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (128,), "float32")
    B = T.match_buffer(B_ptr, (128,), "float32")
    T.device_entry()
    bx = T.cta_id([1])
    tx = T.thread_id([128])
    sm = T.alloc_shared((128,), "float32") #          128    float
    sm[tx] = A[tx] #
    T.cuda.cta_sync() #          :
    B[tx] = sm[tx] * T.float32(2.0) #          ,    2,

```

逐一讲一下，因为这里有几个 GPU 专有的写法你没见过：

- `@T.prim_func` 是个装饰器，意思是「下面这个函数是一段要在 GPU 上跑的 kernel」。
- `T.handle` 是参数类型，你就理解成「一根从外面传进来的裸指针」。
- `T.device_entry()` 标记「从这里开始就是设备 (GPU) 上执行的代码了」。
- `tx = T.thread_id([128])` 拿到当前线程的编号。这是 GPU 编程的精髓：同一段代码会被 128 个线程同时跑，每个线程的 `tx` 不一样 (0 到 127)。所以 `sm[tx] = A[tx]` 这一句，128 个线程一起执行，效果就是 128 个格子被各自的线程并行搬好了——这就是 GPU 「一条代码，千军万马同时跑」的并行方式。
- `bx = T.cta_id([1])` 类似地拿到「当前这一组 (block) 的编号」，这个例子里只有一组，用不上。

它会老老实实变成一段普普通通的 C++/CUDA 代码，核心是一个 `__shared__` 数组 (`__shared__` 就是 CUDA 里声明共享内存的关键字)：

```

“c++ __shared__ alignas(64) float sm_ptr[128]; // 对应 T.alloc_shared sm_ptr[tx] =
A_ptr[tx]; __syncthreads(); // 对应 T.cuda.cta_sync() B_ptr[tx] = sm_ptr[tx] * 2.0f;

```

```

### 3.2

**          ?**
↳          ,          ,

** **      (`scope="shared.dyn",dyn    dynamic          )          , **      kernel
↳ **,      `sharedMemBytes`(          )          ,          :

> **      **:      kernel **          **,          arena /          (arena
↳          )          ,          `T.decl_buffer`(          arena          `data`
↳          ,          `elem_offset`          ),          ,
↳
↳          :          ,          ,          ,

```python
arena = T.alloc_buffer((128,), "float32", scope="shared.dyn") # (128 float)
arena decl , 0 , 64
As = T.decl_buffer((64,), "float32", data=arena.data, scope="shared.dyn") # 64
Bs = T.decl_buffer((64,), "float32", data=arena.data, elem_offset=64, scope="shared.dyn") # 64
As[tx] = A[tx]
Bs[tx] = B[tx]

```

```
T.cuda.cta_sync()
C[tx] = As[tx] + Bs[tx]
```

读这段的关键:**arena** 是唯一真正申请的那一块 (128 个 float)。**As** 和 **Bs** 都没有申请新内存, 它们都拿 **arena.data**(竞技场的起始指针) 当 **data**, 区别只在 **elem\_offset**:**As** 不偏移 (从第 0 个开始),**Bs** 偏移 64(从第 64 个开始)。于是 **As** 占了前半截 0~63,**Bs** 占了后半截 64~127, 正好把竞技场分两半用。

这两个视图, 共用的是同一块用 **extern \_\_shared\_\_** 声明的竞技场 (**extern \_\_shared\_\_** 是 CUDA 里声明「动态共享内存」的固定写法, 中括号里不写大小, 正是因为大小要等启动时才填; 下面为了看得清楚, 我把这块竞技场起名叫 **smem**)。你会看到 **As** 直接用 **smem[tx]**,**Bs** 则是 **smem[tx + 64]**——那个 +64 就是 **Bs** 的 **elem\_offset** 变出来的:

```
“c++ extern __shared__ __align__(64) float smem[]; // 唯一的动态共享 arena
smem[tx] = A_ptr[tx]; // As ——偏移 0 处的视图
smem[tx + 64] = B_ptr[tx]; // Bs ——偏移 64 处的视图
__syncthreads(); C_ptr[tx] = smem[tx] + smem[tx + 64];
```

```

 : , arena, :

```mermaid
flowchart TD
    subgraph StaticGroup["static __shared__"]
        S1["sm[128]"]
        S2[""]
    end
    end
    subgraph DynGroup["dynamic arena (launch)"]
        Arena["extern __shared__ float smem[]"]
        Arena --> V1["As (offset 0, 64)"]
        Arena --> V2["Bs (offset 64, 64)"]
    end
    end
```

图注: 静态共享内存是几块各自独立的小数组; 动态共享内存只有一整块 **arena** (**smem[]**, 大小在 **launch** 时确定),**As** 落在偏移 0、**Bs** 落在偏移 64, 两个加起来正好填满 0~128。

注意

注意: 千万别手一抖写了两次 **alloc_buffer(scope="shared.dyn")**, 那是错的——动态共享内存只许分配一块。一句话把两者拎清: 静态共享内存编译时就定好大小 (**__shared__ T x[N]**); 动态共享内存呢, 是独此一块、启动时才定大小的 **arena**, 你想要的各个视图, 都靠 **decl** 在它内部不同偏移上声明出来。

动态共享大小是怎么被传进去的?

这里有个特别容易绕晕的点: 既然大小是启动时才定的, 那 **sharedMemBytes** 到底是谁填进去的? 答案——根本轮不到你手动填,TVM(就是 TIRx 背后那套编译器框架)会照着你 **shared.dyn** 分配的大小, 自动把它算出来。

(顺带说两个词:**host** 端指「CPU 这边的代码」, 负责发号施令、启动 GPU 程序;**device** 端 / 设备端指「GPU 那边真正干活的代码」。一次 GPU 计算, 总是 **host** 端把 GPU 程序连同参数一起

递过去启动。**lowering / 下降**则是编译里的一步，指「把高层写法一层层翻译成更底层、更接近机器的形式」。）

整个流程是这样：

1. 其实呢，竞技场多大，编译时就已经心里有数了（上面例子里是 128 个 float，每个 float 4 字节，合计 512 字节）。
2. 到了 lowering 这一步，TVM 会往设备 kernel 的启动参数清单（`tirx.kernel_launch_params`）里塞一个 `"tirx.use_dyn_shared_memory"` 标记，意思是「这个 kernel 要用动态共享内存，记得把大小补上」。
3. host 端的启动器一看到这个标记，就把总字节数算好（512），当成**最后一个**启动参数递进去。

```
# kernel : use_dyn_shared_memory
"tirx.kernel_launch_params": ["blockIdx.x", "threadIdx.x", "tirx.use_dyn_shared_memory"]

# host (... , gridDim.x, blockDim.x, dyn_shared_bytes):
T.call_packed("dyn_kernel", A.data, B.data, C.data, 1, 64, 512)
```

等跑起来，这个 512 就被填进真正启动 GPU 程序的那个调用（`cuLaunchKernelEx`, CUDA 的内核启动函数）的 `sharedMemBytes` 参数里。从头到尾一条龙，全自动，你啥都不用管——你只管写 `shared.dyn` 分配，字节数 TVM 替你数。

3.3 内存池语法糖：`T.SMEMPool`

前面那种做法——手动 `decl` 一堆视图、自己掰手指头算「这块从第几个开始、那块从第几个开始」——又烦人又容易出错（算错一个偏移，两块视图就会打架、互相覆盖）。`T.SMEMPool`（SMEM 内存池）就是来替你包这个脏活的：竞技场里「谁占哪一段」的记账，它给你**全自动**搞定。

它的工作原理叫**碰撞分配器 / bump allocator**。这个名字听着玄，其实特别简单：想象一把尺子，有个游标（cursor）从 0 开始。你每问它要一块内存，它就把游标往后推相应的长度，然后告诉你「你那块就从刚才游标停的地方开始」。一块接一块地往后顶，所以叫「碰撞 / bump」。视图怎么放、偏移多少，全是它在背后帮你记，你一行都不用自己写。

除了最基本的 `alloc`（要一块）/ `commit`（收工，定下总大小），它还附送几个贴心的小帮手：

- **逐块指定 `align=`**：某一块缓冲区想单独设个对齐（对齐 = 要求这块地址是某个数的整数倍，某些硬件指令有这种要求）？可以。
- **`alloc_mma` 帮手**：自动给你配好一个 MMA 专用的特殊布局。这里两个词解释下：**MMA**（matrix-multiply-accumulate，矩阵乘累加）是 GPU 里专门飞快算矩阵乘法的硬件单元（叫 Tensor Core）干的活，矩阵乘是深度学习里最核心、最频繁的运算；**swizzle**（可以理解成「故意把数据错位摆放」）是为了避开一种叫 bank 冲突（多个线程同时挤到共享内存的同一个小口子上，导致排队变慢）的性能问题。这些细节你现在不用懂，知道「`alloc_mma` 替你把矩阵乘需要的特殊摆法都安排好了」即可。
- **`move_base_to`**：把游标往回倒一倒，这样后面再要的内存就会**盖在前面那块上、复用同一片空间**（适合「前一块用完了、不要了，把地方让给后一块」的场景，省内存）。

```
pool = T.SMEMPool() # ( )
As = pool.alloc(BM, BK, "float16", align=128) # ,
Bs = pool.alloc(BK, BN, "float16", align=128) # ,
Cs = pool.alloc_mma((BM, BN), "float16") # ,
```

```
pool.commit()                                # : ,
# pool.move_base_to(offset)                  , alloc
```

对比一下: 用了内存池, 你只管一句句 `pool.alloc(...)`, 完全不用关心「这块该放第几个字节」——而前面手写 `decl_buffer` 时, 那个 `elem_offset=64` 是你自己算出来的。内存池把这种容易算错的账全接管了。

注意

注意: 后面要讲的 **TMEM 池** (`T.TEMPpool`), 其实就是叠在一个 `SMEmpool` 上面的。所以你现在把 `SMEmpool` 这套 bump 分配的路数搞明白, 待会儿看 `TMEM` 会顺畅很多。

`SMEmpool` 怎么做 bump 分配, 看一眼图就明白了:

结构图

连接关系:

- base (游标 cursor 从 0 开始) $\rightarrow$ As
- As $\xrightarrow{\text{alloc 向右推进, 按 align 对齐}}$ Bs
- Bs $\rightarrow$ Cs
- Cs $\rightarrow$ cursor (commit() 时即总大小)

图注:`SMEmpool` 是个碰撞分配器, 游标从 0 起步, 每 `alloc` 一次就往右推进一段 (顺带按 `align` 对齐); 到 `commit()` 那一刻, 游标停在哪儿, 内存池就有多大;`move_base_to(off)` 能把游标倒回去, 让后面的分配覆盖、复用前面那块空间。

四、寄存器 (registers)

一句话先理解

一句话先理解: 寄存器是每个线程私有的、GPU 上最快的存储, 放线程自己算东西时的临时小数据。它跟 CPU 寄存器是同一个概念, 只不过在 GPU 上是每个线程各有一份。

如果你写过底层代码, 大概知道 CPU 有寄存器——CPU 算数时, 操作数得先在寄存器里才能算, 寄存器是离运算符最近、最快的那一小撮存储。GPU 也一样, 而且因为同时跑着成千上万个线程, 每个线程都独立拥有自己的一份寄存器, 互不干扰。线程做循环计数、累加中间结果这类「只给自己用、马上要算」的小数据, 放寄存器最划算。

想要这种存储, 就用 `T.alloc_local(shape, dtype)` (也就是 `scope="local"`) 来分配。它每个线程一份、谁也不碍着谁, 最后会变成一个蹲在寄存器里的本地数组:

```
r = T.alloc_local((4,), "float32") # 4 float
for k in T.unroll(4):              # T.unroll 4
    r[k] = A[tx, k]                # A 4
# ... r[0]-r[3] ...
```

这里 `T.unroll(4)` 解释一下: 展开 (unroll) 就是把 `for k in range(4)` 这种循环, 直接摊成 `r[0]=...; r[1]=...; r[2]=...; r[3]=...` 四行写死的代码。为什么要展开? 因为这样一来, 下

标 **k** 在编译时就是确定的常数 (0/1/2/3), 编译器一看「啊这就是固定的几个格子」, 才好把它们干脆塞进寄存器, 而不是放进慢一些的内存。

```
“c++ alignas(64) float r_ptr[4]; // 每线程私有, 驻留寄存器 r_ptr[0] = A_ptr[tx 4 + 0];
r_ptr[1] = A_ptr[tx 4 + 1]; // ...
```

```
> **      :      `alignas(64)`      ?**(`alignas(64)`      C++      64
↳      ,      )      **      **      64      CUDA
↳      ,      64      **      **      ,      `local`
↳
>
↳      ?      ,      64      **      **      (      `r[0]`~`r[3]`),      GPU
↳      (nvcc/ptxas)      ,      (      )
↳      (      / SROA      ,
↳      )      **      **      (      spill /      )      ,
↳

### 4.1      (scalar):

      **      (scalar)**      :      ,      `i`
↳ `sum`,

      :      TIRx      ,**      **
↳      ,      1      `local`      ,      `[0]`      :

```python
phase = T.alloc_local((1,), "int32") # 1
phase[0] = 0
while phase[0] < 4:
 acc = acc + A[tx, phase[0]]
 phase[0] += 1
```

可满屏都是 `phase[0]`, 本来一个计数器, 却处处带着个 `[0]`, 看着实在别扭。于是「标量」就是为这事加的一层**语法糖** (语法糖 = 更顺手的写法, 背后干的还是同一件事)——让你**直接拿名字** (`phase` 而不是 `phase[0]`) 读写的单元元素寄存器缓冲区:

```
phase: T.int32 = 0 # ()
while phase < 4:
 acc = acc + A[tx, phase]
 phase += 1

s = T.local_scalar("int32") # ; (s = ..., s[0])
acc: T.float32 = 0.0 #
```

关键

关键: 这两种写法可不只是「长得像」——它们解析出来的内部结构压根就是同一个。这层糖在 `parser`(解析器, 就是把你写的代码读进来、转成内部结构的那一步) 阶段就化没了:`phase: T.int32` 就是那个单元元素 `local` 缓冲区,`phase / phase += 1` 就是 `phase[0] / phase[0] += 1`。

怎么证明它俩真的一样？把这两种写法的 `kernel` 各转成内部结构，用 `tvm.ir.assert_structural_equal`（一个「断言两段 IR 结构完全相同」的检查函数）一比，直接通过；反过来，负责把内部结构打印成代码的 `printer`，甚至会主动把显式的 `alloc_local + [0]` 重新打印成标量那种简洁写法。换句话说，解析一结束，两者就再无分别——最后都变成同一句 C 代码 `alignas(64) int phase_ptr[1]`；要是你想显式指定这个标量放哪层内存，就用 `T.local_scalar`（寄存器）/ `T.shared_scalar`（共享内存）/ `T.alloc_scalar`。

注意

**注意：**那为啥不干脆用一个普通变量（`Var`）？因为 TIRx 的 `Var` 是不可变的——它只是个一次性绑定，绑了就不能再改（下一节的 `T.let` 吐出来的就是这玩意）。可标量要的恰恰是**可变**：循环计数器要一遍遍加 1，累加器（`accumulator`，就是「边算边把结果累加进去」的那个变量，比如求和时的 `sum`）要一次次更新。这种「反复重新赋值」的需求，不可变的 `Var` 干不了，所以只能靠一个「能反复写入」的单元元素缓冲区来撑——这就是标量背后非得是个缓冲区的原因。

4.2 let: 不可变绑定

一句话先理解

**一句话先理解：**`let` 就是「给一个算好就不再变的值起个名字」，它不是变量、不占内存，纯粹是个名字。

`T.let` 绑定是**不可变的**——它对应一个 `LetStmt`（`let` 语句），干的事就是「给一个值起个名字」，而不是开一块缓冲区。它跟上一节的标量正好相反：标量是「能反复改的盒子」，`let` 是「绑一次就钉死的名字」。它特别适合用来定义那种算一次就再也不变的派生常量（比如由几个尺寸乘出来的总数）：

```
n: T.let = M * K # (LetStmt)
half: T.let[T.int32] = N // 2 # ...
```

它会下降成一个**普通的 C 标量变量**——注意，不是缓冲区，没有数组，也没有 `[0]` 那一套。拿 `half: T.let = m * 2` 举个例子（`m` 是个运行时才知道的值）：

“`c++ int half = m * 2; // let -> 一个类似 const 的局部变量`”

```

 , (simplifier) , (CSE)
↳ , `m * 2` , `half`

> ** : ?**
↳ , ,
>
> (,): ** **
↳) `let` , ,
↳ ** , **
↳ ? (**)
↳
>
```

```

> ** ** : ('buf[0]`), ,]
↳ , :`let` ** ** ,
↳ (CSE, ,
↳) ; ,

 `let` , :

| | scalar | `let` | |
|---|---|---|---|
| | ** ** () | ** ** () |
| | `local` (load/store) | `LetStmt`, C |
| | (1) | |
| | / | () | ** ** (,) |
| | | (`M*K`) |

(tensor memory,TMEM)

> ** **:TMEM
↳ ,

 Blackwell NVIDIA GPU (Hopper Ampere ;
↳) ** / tensor memory(TMEM)** (Tensor
↳ Core) `alloc` ,TMEM ,
↳ , , :

1. ** ** ,TMEM ,
↳ `T.ptx.tcgen05.alloc` ()/ `tcgen05.dealloc` () **warp
↳ (warp-uniform)** **warp**::GPU , **32
↳ ** (), 32 warp warp , warp
↳ 32 ,

2. ** TMEM ** : ,
↳ `T.decl_buffer(..., scope="tmem", ...)` TMEM

3. **`allocated_addr` ** (TMEM)
↳ ? ,`T.alloc_buffer(scope="tmem")`
↳ ** ** `alloc_buffer` `allocated_addr`, `decl_buffer`
4. ** ** `sm[tx]` ,TMEM `tcgen05`
↳ (`mma` `ld` `st` `cp`)

 TMEM warp , `decl`
↳ , warp `if warp_id == alloc_warp:`,
↳ warp alloc/dealloc (1 warp):

```python
addr = T.alloc_shared((1,), "uint32")      #
if warp_id == alloc_warp:      # tcgen05.alloc      warp
    T.ptx.tcgen05.alloc(T.address_of(addr), n_cols=512, cta_group=cta_group)
acc = T.decl_buffer((CTA_M, 512), "float32", scope="tmem",
                    allocated_addr=0, layout=tmem_layout) # 0
# ...      acc      gemm_async / copy_async      ...
if warp_id == alloc_warp:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=cta_group) #
    T.ptx.tcgen05.dealloc(addr, n_cols=512, cta_group=cta_group)

```

把这段读一遍你就明白它有多繁琐：先用一个共享内存小槽 `addr` 存放系统分配给你的基址；然后只让 `alloc_warp` 那个 warp 调 `tcgen05.alloc` 真正预留 512 列；接着用 `decl_buffer` 在列 0 处声明出 `acc` 这块视图 (`allocated_addr=0` 就是「从第 0 列开始」)；中间把 `acc` 交给矩阵乘去用；

最后又只让那个 warp 先交还许可、再 `dealloc` 释放。

这里头的列偏移、还有 `tmem_layout` (矩阵乘数据通路要求的一种特殊摆法, 细节不用懂), 通得你自己操心。看到这儿你大概已经在皱眉头了——别急, 下面要讲的内存池, 自动帮你发的正是这一整套烦人的序列。

TMEM 手动管理的生命周期:

时序图 (按时间顺序)

1. `alloc_warp` 单一 warp → 张量内存 TMEM: `tcgen05.alloc(addr, n_cols=512)` 【warp 统一】
2. 备注: `decl_buffer(scope=tmem, allocated_addr=0)` / `acc` = 列 0 处的视图
3. kernel 主体 → 张量内存 TMEM: 通过 `tcgen05 mma/ld/st/cp` 读写 (不可直接寻址)
4. `alloc_warp` 单一 warp → 张量内存 TMEM: `relinquish_alloc_permit` 交还分配许可
5. `alloc_warp` 单一 warp → 张量内存 TMEM: `tcgen05.dealloc(addr, n_cols=512)`

5.1 TMEM 池: `T.TMEMPpool`

`T.TMEMPpool` 把上面那一大摊子活儿全给你打包了——warp 统一的 `alloc/dealloc`、列偏移的 `bump` 分配、外加数据通路布局, 一站全搞定:

```
tmem_addr = pool.alloc((1,), "uint32") # ,
tmem_pool = T.TMEMPpool(pool, total_cols=512, cta_group=cta_group,
                          tmem_addr=tmem_addr) # TMEM , 512
acc = tmem_pool.alloc((CTA_M, 512), "float32") # , allocated_addr
tmem_pool.commit() # : tcgen05.alloc
# ... acc ...
tmem_pool.dealloc() # tcgen05.dealloc
```

对比上一段手写版, 你瞬间能感到轻松: 那些 `if warp_id == alloc_warp.relinquish_alloc_permit`、列偏移记账, 现在全藏进了 `commit()` / `dealloc()` 里, 你只管 `alloc` 和用。

注意

注意: 用 `TMEMPpool` 之前, 你得先递给它一个现成的 `SMEMPpool`。为啥前面说过了——它是「叠」在共享内存池上头的, 得借共享内存里的一个小槽, 来存放分配到的基址。想看它怎么真刀真枪地用, 翻原书第三部分那个 GEMM (通用矩阵乘法, GPU 上最核心的计算) kernel 就行。

六、缓冲区方法 (Buffer APIs)

咱们绕回开头那句心智模型: 缓冲区不过是指针上的一层「说明书」(元数据)。正因为如此, 它的绝大多数方法都只是编译期的 `reshape` (换个形状看) / `reinterpret` (换种解读)——要么换个算下标的方式, 要么递给你一个指针, 等程序真跑起来时, 这些方法自己一条多余指令都不会冒出来。这就是为什么改形状、改布局在 GPU 上几乎「免费」: 你改的只是那张编译期的说明书。常用的方法先扫一眼, 看不懂的列后面会挑重点逐个讲:

方法	它是什么
B.data	原始数据指针 (一个 Var); 打印成 B_ptr
B.ptr_to([i, j])	指向某元素的有类型指针 (即 address_of); 打印成 &B_ptr[$\hat{e}$]
B.vload([i], dtype="float32x4") / B.vstore([i], v)	向量化 load / store; 打印成 $\ast(\text{float4}\ast)(\text{B_ptr} + \hat{e})$
B.view(*shape, layout= $\hat{e}$)	用新的形状/布局重新解释同一块存储 (无拷贝)
B.local(*shape, layout= $\hat{e}$)	取出调用线程在某 local 缓冲区里的私有寄存器切片
B.permute(*dims)	轴重排后的视图 (即转置布局)
B.access_ptr(mask, $\hat{e}$)	带掩码的访问指针 (tvm_access_ptr 内建, 用于把一块区域传给某 intrinsic

下面分几类, 挑关键的用法挨个看看。

6.1 指针:ptr_to / data

想把**某个元素的地址**递给一个 intrinsic 或者内联函数, 就用 ptr_to; 而 data 给你的是整块存储的基址指针:

```
B[tx] = T.cuda.func_call("ld", A.ptr_to([tx]), source_code=SRC, return_type="float32")

“c++ B_ptr[tx] = ld(&A_ptr[tx]); // ptr_to([tx]) -> &A_ptr[tx]; A.data -> A_ptr

### 6.2      :`vload` / `vstore`

    **      **      ,      1      4      ,
↪      ,      ( )
↪      ,      ,      **      /      (wide
↪ transfer)** `vload` ( ) / `vstore` ( )      :

``python
B.vstore([tx * 4], A.vload([tx * 4], dtype="float32x4"))
```

dtype="float32x4" 就是「一次按 4 个 float32 打包」的意思 (x4 = 四个一组)。所以这一句的效果是: 从 A 里一口气读 4 个 float, 再一口气写进 B。对应到 CUDA 代码, 就是把指针强转成 float4(CUDA 内置的「4 个 float 捆一起」类型) 再一次性读写:

“c++ (float4)(B_ptr + tx 4) = (float4)(A_ptr + tx 4); // 一条指令搬 4 个 float

```
### 6.3 reshape /      :`view` / `permute`

    **      **      ,      `A.view(64, 4)`
↪      256      `64x4` (64 4 )      ( NumPy reshape); `A.permute(1, 0)`
↪      ,      **      ** ( ):

``python
A2 = A.view(64, 4); y = A2[tx, 0] + A2[tx, 3] # A2[tx, j] -> A_ptr[tx*4 + j]
At = A.permute(1, 0); z = At[i, j] # At[i, j] -> A_ptr[j*4 + i]
```

“c++ A2_ptr[tx 4] / +3 // view: 行主序 64x4 的下标 At_ptr[(j 4) + i] // permute: 步长互换 (转置)


```

|         | BufferRegion |
|   | view |
|   | layout / TileLayout |
|   | stride |
|         | elem_offset |
|         | shared memory (SMEM) |
|         | static shared memory |
|         | dynamic shared memory |
|   (           ) | arena |
|         | bump allocator |
|   | registers |
|   | scalar |
|         | let / immutable binding |
|         | tensor memory (TMEM) |
|         | CSE (common-subexpression elimination) |
|         | SROA (scalar replacement of aggregates) |
| warp   | warp-uniform |

```


第 23 章 · 控制流

原文:Control flow

本章要点

本章要点 (TL;DR)

- TileIR(TIR-X) 里的控制流就三大类:if/else 分支、循环家族 (T.serial / T.unroll / T.vectorized / T.grid), 还有 while。每一类都会被「直白地」降级 (lower) 成对应的 CUDA C++ 写法。
- 想在两个值之间二选一, 用 T.if_then_else(cond, a, b)。它降级成三元运算符 ? :, 不产生分支、也不引入控制流发散 (divergence), 做 ReLU 这类逐元素取舍很合适。
- 一定要分清统一控制流 (uniform control flow) 和发散控制流 (divergent control flow): 逐线程的 if tx < 128 用来做普通计算没问题; 但集体操作 (collective op)(比如 __syncthreads()) 必须被所有参与同步的线程统一到达, 否则会死锁。
- T.cuda.cta_sync()(= __syncthreads()) 绝不能放进任何线程级或 warpgroup 级发散的分支。只想同步单个 warpgroup 时, 改用 T.cuda.warpgroup_sync(id)。还有 mbarrier.init() 这种「单线程守护」式的初始化, 同样不能嵌进发散分支, 否则屏障可能根本没被初始化。
- while 靠一个可变标量计数器实现, 底层降级成 while(1) + break, 而这个计数器其实是个单元素寄存器缓冲区 (register buffer)。

前置知识

前置知识: 读这一章前, 你最好对 GPU 的「线程层级」和「同步」这两件事有点概念。别紧张, 这里先用一段话给你铺个底, 后文每个词第一次出现时还会再讲一遍。

- **GPU 为什么要搞一堆线程?** CPU 擅长一件事接一件事快速地干; GPU 反过来, 它的强项是「同一件事, 同时对成千上万份数据各干一遍」。所以 GPU 编程里, 你写的一段代码会被几万个「线程 (thread)」同时跑, 只不过每个线程处理的数据不一样。你可以把线程理解成「一个干活的小工」, GPU 一次性派出几万个小工。
- **这些线程是分层组织的**, 从小到大大致是: lane(一个线程在它小队里的编号) → warp(32 个线程绑成的「方阵」, 硬件调度的最小单位) → warpgroup(几个 warp 再打包) → CTA / thread block(一个「线程块」, 这一块线程能共享一小块高速内存、还能互相等)。这些词后文都会在第一次用到时当场解释, 你现在只要知道「线程是分层的, 层越大管的线程越多」就够了。
- **「同步」是什么?** 就是「大家在某个点上互相等一等, 等齐了再一起往下走」。多线程一起干活时, 常常需要先确保「所有人都把上一步做完了」, 才能安全地进下一步——这就是同步的意义。

想看更系统的讲解, 可以先翻一下第 0 章·极简入门; 但只看上面这三条, 也足够把本章读下来

了。

先交代一下背景。本书在讲一门叫 **TileIR(也写作 TIR-X)** 的语言——它是一种「专门用来写 GPU 程序的 Python 风格小语言」(行话叫 DSL, domain-specific language, 领域专用语言)。你用 Python 的语法写 TileIR, 工具链再把它翻译成真正能在 GPU 上跑的代码。这个「翻译成更底层代码」的过程, 行话叫**降级 (lowering)**——就像高级语言被编译成汇编一样, 从「人好懂的写法」一层层落到「机器好执行的写法」。

那它最后翻译成什么? 是 **CUDA C++**(NVIDIA 给 GPU 用的 C++ 方言) 和 **PTX**(一种比 CUDA C++ 更靠近硬件的「GPU 汇编」)。本章不要求你会写它们, 但会拿它们来给你看「你写的 TileIR 到底变成了啥」。

这一章干的事很简单: 把 TileIR 里和**控制流**(就是 **if**、循环、**while** 这些决定「代码往哪走、走几遍」的语句) 有关的每一个「原语 (primitive, 可以理解成这门语言提供的内置积木)」, 一个一个跟它降级出来的 CUDA C++/PTX 对上号。

这事为什么值得单独讲? 因为 TileIR 不是个「黑盒」——你写的每一条控制流语句, 几乎都能一对一地对应到一段 CUDA。换句话说, 看着 TileIR 代码, 你脑子里基本就能预判出底层会生成什么。这对调试和优化都很有用。

篇幅不长, 但里头有一节——**统一控制流 vs 发散控制流**——是 GPU 编程里最容易翻车、最容易把程序卡死的地方, 我们会重点讲透它。

下面先把各种控制流积木一个个过一遍, 再单独深挖「发散」这个大坑。

一、if / else: 分支

这是最省心的一种: 你在 TileIR 里写的 Python **if / else**, 几乎原封不动就变成 CUDA 的 **if / else**, 中间什么花活都没有。

一句话先理解

一句话先理解: GPU 上几万个线程跑的是同一段代码, 但每个线程有自己的「编号」。**if** 最常见的用法, 就是**拿这个编号做判断, 让不同编号的线程走不同的路**——这样虽然代码只有一份, 行为却能各不相同。

先认识两个编号:**threadIdx.x**(简称 **tx**) 是**线程在它所在线程块里的编号**; **lane** 则是一个线程在它所属那个 **32 人 warp** 里的小编号 (**0~31**)。拿编号当条件, 限定「哪些线程才能执行某段代码」, 这种套路有个行话叫**守护 (guard)**——就像门口的保安, 按编号放行。

来看个最常见的例子: 按线程号把活儿分成两半。

```
# TileIR      :
if tx < 128:
    A[tx] = A[tx] * T.float32(2.0) #          128      :          2
else:
    A[tx] = A[tx] + T.float32(1.0) #          :          1
```

这里 **A[tx]** 的意思是「第 **tx** 号线程去处理数组 **A** 里第 **tx** 个元素」——每个线程认领一格, 互不干扰。**T.float32(2.0)** 就是「32 位浮点数 2.0」的写法。

降级后的 CUDA C++ 跟它几乎一模一样, 一行对一行:

```
“c++ if (((int)threadIdx.x) < 128) { A_ptr[tx] = A_ptr[tx] * 2.0f; } else { A_ptr[tx] = A_ptr[tx] + 1.0f; }
```

```

                                :`tx`      `threadIdx.x`(CUDA                                ),`T.float32(2.0)`
↪ `2.0f`(C++      `f`      32      ),

                                ,32      **warp(      )**, warp      **

↪      **      32      32      lane      :
↪ `if`,      **      warp      32      lane      **(`tx < 128`,      )
↪      , GPU      **      (divergence)**

      ?      :

-      ,      ,      :**
↪      (      ,      ),      **
↪      ,      ,
-      (      ),

###      :`T.ptx.select_sync()`

GPU      ,**      ,      **      :
- **TMA(Tensor Memory Accelerator) **:      ,
↪
- **MMA(      ,Matrix Multiply-Accumulate)**:GPU      (
↪      , GPU      )      , warp

      warp
↪      warp      32      lane **      **,      32      ,
↪

      :**      warp      32      lane      ,      **
↪ `T.ptx.select_sync()`(`select =      )      :

```python
if T.ptx.select_sync():
 ... # lane (TMA / MMA)
```

它怎么做到的？在一个同步点上，它从当前还活跃的 lane 里确定性地点名一个（每次点的都是同一个，不随机），给这个被点中的 lane 返回 true，其余所有 lane 一律返回 false。把它当 if 的条件，就只有那一个 lane 能进 if 体——这就是「只让一个线程发射」的标准写法。

关键

关键:elect\_sync() 选的是「一个 warp 内部的代表」，它管的范围也就到这个 warp 里面为止。这个 warp 里所有活跃的 lane 都会执行到这条 elect\_sync()，只不过被点中的那个拿到 true、其他人拿到 false，选举在 warp 内部就办完了。请特别注意：它的「全员」最多到一个 warp，不是整个线程块。

这里顺带把一个更大的单位讲清楚:CTA / thread block(线程块)——它是「能跑在同一个 SM 上、能共享一小块高速内存 (SMEM)、还能互相同步等待」的一组线程 (SM 是 GPU 上的一个「计算核心区」，你可以先把它当成「一个班级的教室」)。一个线程块里通常有好几个 warp。elect\_sync() 只在「一个 warp」这个小圈子里选代表，跟后面要讲的「线程块级别的

操作必须全班到齐」完全是两码事，千万别混。

表达式级选择: `T.if_then_else`，无分支

前面的 `if/else` 是「跑两段不同的代码」。但很多时候你根本不需要那么大动干戈，你只是想**在两个值里挑一个**——比如「这个数大于 0 就保留它，否则用 0」。这种「二选一取值」，用 `T.if_then_else(cond, a, b)` 才对（读法：条件 `cond` 成立就取 `a`，否则取 `b`）。

它好在哪？它**根本不生成 `if` 分支，而是降级成 C 语言的三元运算符 `cond ? a : b`**（三元运算符就是一行写出「条件成立取前者、否则取后者」的紧凑写法）。

“`c++` // 例如 ReLU: 大于 0 就取原值，否则取 0，全程没有分支 `O_ptr[tx] = (A_ptr[tx] > 0.0f) ? A_ptr[tx] : 0.0f;`

```
(ReLU
,
0,
,
)

> **
> - `if`/`else`: ** `if`? ** :
↳ `if`/`else` () warp lane
↳ , ** ,
> - : `select` (,PTX `selp`) ** 32 lane
↳ ** , lane
>
> : clamp() ReLU `max(0, x)`
↳ `T.if_then_else`
:

```mermaid
graph TD
    Q{ }
    Q -->| " | A["if / else<br/>-> / predication<br/>warp" ]
    Q -->| " | B["T.if_then_else(cond, a, b)<br/>-> ? :<br/>" ]
    style A fill:#ffe2e2,stroke:#c0392b
    style B fill:#e2ffe2,stroke:#27ae60
```

二、统一控制流 vs 发散控制流 (核心)

这一节是全章的核心，也是 GPU 程序**死锁 (deadlock)** 最常冒出来的地方。死锁就是「大家互相干等、谁也走不了，程序永远卡住」——后面会看到它是怎么发生的。

一句话先理解

一句话先理解:GPU 上有两类截然不同的操作。一类是「各干各的」，线程之间互不相干；另一类是「**必须全员一起到齐才能进行**」（比如同步：大家在一个点上互相等）。这一节的全部要害就是：**第二类操作，绝对不能藏在「只有一部分线程会进去」的 `if` 里**，否则没到齐的那些线程会让大家永远等下去。

先把两类操作掰开说清楚：

这类操作	含义	例子	把它放进「只有部分线程会进的分支」会怎样
普通逐线程工作	每个线程独立算自己那份, 谁也不等谁	<code>A[tx] = ...</code> 这种逐元素运算	没问题, 顶多是前面讲的那点发散开销
集体操作 (collective op)	一组线程必须「一起」到达、共同参与 (比如同步)	<code>T.cuda.cta_sync()</code> (就是 CUDA 的 <code>__syncthreads()</code>)、各种屏障操作	可能死锁, 或者行为完全没准 (未定义)

(「集体操作」这个词记住就行: 顾名思义, 它是「集体活动」, 讲究的就是人到齐。)
核心规则, 一句话:

集体操作必须被「它要同步的每一个线程」都统一 (uniformly) 地走到——绝不能把它藏在「只有一部分线程才会进去」的发散分支里。「统一地走到」的意思是: 要么所有相关线程都执行到它, 要么都不执行, 不能有人到、有人没到。

经典死锁: 把 `cta_sync()` 放进发散分支

`T.cuda.cta_sync()` 降级成 CUDA 的 `__syncthreads()`, 这是最常用的同步操作。它的含义是: 「这个线程块里的每一个线程, 都在这儿等着, 等所有人都到了, 再一起往下走」。所以它有个铁打的前提——线程块里的每一个线程都必须走到这一行, 缺一个都不行。

现在看个反面教材: 把它塞进一个按 `warpgroup` 划分的分支里。先解释下 `warpgroup`: 它就是「若干个连续的 `warp` 再打包成的一组」(在 Hopper 这代 GPU 之后, 常用它来给线程分工——比如让 `warpgroup 0` 专门搬数据、`warpgroup 1` 专门算)。这里假设 `wg_id` 是「当前线程属于第几个 `warpgroup`」。

```
#           :           !
if wg_id == 0:
    T.cuda.cta_sync() #           warpgroup 0
...
```

问题出在哪?`cta_sync()` 要的是「整个线程块所有线程到齐」, 可这个 `if` 只放 `warpgroup 0` 的线程进来。于是:`warpgroup 0` 的线程进来了、停在屏障前等; 而 `warpgroup 1、2、3` 的线程压根不进这个 `if`, 它们继续往后跑了。结果 `warpgroup 0` 苦苦等着「全块到齐」, 可剩下那几个 `warpgroup` 永远不会来——大家就这么干等到天荒地老, 整个 `kernel`(`kernel` = 你提交给 GPU 跑的那段程序) 永久卡死。这就是死锁。

下面这张图把这个坑画出来了:

结构图

连接关系:

- 线程块 (CTA) = 4 个 `warpgroup` → `wg0`
- 线程块 (CTA) = 4 个 `warpgroup` → `wg1`
- 线程块 (CTA) = 4 个 `warpgroup` → `wg2`
- 线程块 (CTA) = 4 个 `warpgroup` → `wg3`
- `wg0` → `if wg_id == 0` ?

- `wg1` → `if wg_id == 0` ?
- `wg2` → `if wg_id == 0` ?
- `wg3` → `if wg_id == 0` ?
- `if wg_id == 0` ? $\xrightarrow{\text{是 (仅 wg0)}}$ `cta_sync()` / 等待全部线程到齐
- `if wg_id == 0` ? $\xrightarrow{\text{否 (wg1/wg2/wg3)}}$ 跳过整个 `if` 体 / 各自继续 / 退出
- `cta_sync()` / 等待全部线程到齐 → 永久阻塞: / `wg1/wg2/wg3` 永远不来 / → 死锁

注意

注意: 死锁的病根不在「同步」本身, 而在「只有一部分线程会走到同步点」。集体操作天生就带着「全员都得到场」的约定, 而发散分支恰恰把这个约定给破坏了。

正确做法: 用 `warpgroup` 作用域的同步

上面那个需求其实很合理——你本来就只想让 `warpgroup 0` 自己内部对一下表、互相等一等, 根本没打算惊动别的 `warpgroup`。那问题不在「想同步」, 而在「用错了工具」: 你用的 `cta_sync()` 要求的是「全块到齐」, 范围太大了。

正确做法是换一个「作用范围更小」的同步工具——只管一个 `warpgroup` 的那种:

```
if wg_id == 0:
    T.cuda.warpgroup_sync(id)    #                warpgroup,                warpgroup
```

`T.cuda.warpgroup_sync(id)` 立的屏障只在「一个 `warpgroup` 内部」生效, 它要的「全员」就只算这一个 `warpgroup` 的线程。所以把它放进 `if wg_id == 0:` 里反而**完全安全**——因为一个 `warpgroup` 的线程会**整整齐齐**一起进这个分支 (要么全进、要么全不进), 不会出现「有人到、有人没到」, 屏障自然能凑齐。

关键

关键区别 (这是本节最该记住的一张对照):

- `cta_sync()`: 它要的「全员」= **整个线程块**的所有线程 → 所以**不能**放进只有部分线程会进的发散分支。
- `warpgroup_sync(id)`: 它要的「全员」= **单个 warpgroup**的所有线程 → 所以**可以**放进「按 `warpgroup` 整齐划分」的分支。

一句话: 选哪个同步工具, 就看「你真正要同步的范围」和「这个分支把线程切成多大块」**对不对得上**。范围对得上就安全, 对不上就死锁。

顺带一提, `warpgroup` 级同步最常出场的场合, 是「用 **warp 专化 (warp specialization)**——也就是给不同 `warp` 分派不同工种——和 **cluster** 来加速 GEMM」这类高级场景。这里有两个新词:**cluster** 是「多个线程块再打包成的更大一组, 能跨块协作」(Hopper 引入的); GEMM (General Matrix Multiply, 通用矩阵乘法) 则是深度学习里最核心、最吃算力的运算, 可以说 GPU 一大半力气都花在它身上。细节本章不展开, 感兴趣可看相关章节和「CUDA C++/PTX intrinsics (内建函数)」那一节。

同样的坑：屏障初始化 (mbarrier.init)

发散这个坑，不光「同步」这一步会踩，给屏障做初始化那一步同样危险，而且更隐蔽。

先认识 mbarrier: 它是一种内存屏障 (memory barrier), 专门用来给「异步操作」做「等结果到齐」的。啥叫异步操作? 比如前面提的 TMA 拷贝——你下完搬运命令，硬件在后台慢慢搬，你的线程不用傻等，可以先去干别的; 等真要用那批数据时，就靠 mbarrier 来确认「搬完了没有」。所以用之前，得先把这个屏障「初始化」一下 (类似用一个对象前先 new 出来、设好初值)。

关键点来了:mbarrier 的 .init()(初始化) 操作，会降级成一个只让单个线程去做的形式——这种「只放一个线程进来干」的写法，前面讲过，叫单线程守护:

```
“c++ if (threadIdx.x < 1) { // .init() 的本质: 只让 0 号线程去初始化屏障 / 初始化 mbarrier
/ }
```

```
        ,`.init()`` **                ** `if (threadIdx.x < 1)`(` 0          )
↪      **          `.init()``                **,      :

``python
if some_cond:      #          :          some_cond
    mbarrier.init() #          0
```

就可能闹出一个特别坑的局面: 负责初始化的那个 0 号线程，恰好不满足外层的 some_cond，根本没进来。

那结果呢? 屏障压根没人初始化。后面任何对它的「等待 / 到达」操作，行为全是「未定义」的 (意思是: 可能崩、可能卡、可能算错，完全没准)，最常见的表现就是莫名其妙的启动失败 (unspecified launch failures)——程序一跑就挂，还看不出为啥。

结构图

连接关系:

- mbarrier.init() / 内部展开为 if(threadIdx.x < 1) → 0 号线程进入了外层分支吗?
- 0 号线程进入了外层分支吗? $\xrightarrow{\text{是}}$ 屏障被正确初始化
- 0 号线程进入了外层分支吗? $\xrightarrow{\text{否}}$ 屏障未初始化 / 后续等待 → 未定义/启动失败

经验法则: 凡是带「集体语义」或「单线程守护语义」的原语——同步、屏障初始化、TMA/MMA 发射准备这些——都得保证它们待在**所有相关线程都会统一走到**的位置。要么搁在所有线程都会经过的直线代码里，要么搁在「按同样粒度统一划分」的分支里。一旦把它们跟普通的逐元素发散分支搅在一块，基本就是必出问题。

三、循环家族

TileIR 的循环有**四种口味**。它们干的事都是「重复跑几遍」，区别在于「**怎么个跑法、最后生成什么样的底层代码**」。你平时写的那个普通 Python range, 默认对应的就是第一种 T.serial:

写法	它表达的意思	最后生成什么 / 用在哪
----	--------	--------------

T.serial(n)	老老实实一轮接一轮地跑	生成普通的 for 循环;ptxas(GPU 的编译器,管最后一步生成机器码) 有可能自作主张地展开它
T.unroll(n)	完全展开	在编译期就把循环整个铺平成一条条直线语句,运行时不再有「循环」这个结构
T.vectorized(n)	向量化循环	生成「一条指令处理多个元素」的访存/计算(下面会解释什么叫向量化)
T.grid(*extents)	嵌套循环	一行就写出好几层嵌套的 for

这里有两个新词先点一下:「**展开 (unroll)**」指「把循环按次数复制成一串重复语句,从此不再循环」;「**向量化 (vectorize)**」指「让一条指令一次处理一批数据,而不是一个一个来」——后面都会展开讲。

另外,break(提前跳出循环)/ continue(跳过本轮、进下一轮) 在这些循环里都照常能用。

T.grid: 一次写出嵌套循环

T.grid(8, 8) 其实就等于「外层 i 跑 0~7、内层 j 跑 0~7」这两层嵌套 for, 只是写起来比手写两层嵌套清爽得多:

```
for i, j in T.grid(8, 8):
    B[i, j] = T.max(A[i, j], T.float32(0.0)) # 8x8 ReLU
```

这段的意思是: 对一张 8×8 的表里每一格 (i, j), 取 A 这一格和 0 之间的较大值, 写进 B(T.max 就是取较大值)——这正是二维版的 ReLU。

它降级成标准的双层 for。这里有个底层细节值得一看: 二维坐标 (i, j) 被「拉平」成了一维下标 i * 8 + j。为什么? 因为内存其实是一条**一维的长条**, 没有真正的「二维」; 要在一条长条上找到第 i 行第 j 列, 就得算「跳过前面 i 整行 (每行 8 个), 再往后数 j 个」, 也就是 i * 8 + j:

“c++ for (int i = 0; i < 8; ++i) for (int j = 0; j < 8; ++j) B_ptr[i * 8 + j] = max(A_ptr[i * 8 + j], 0.0f);

```
### `T.serial` vs `T.unroll`: vs

, : ** **

| | `T.serial(4)`(` `) | `T.unroll(4)`(` ` , 4 `) |
| --- | --- | --- |
| | `for (int k=0; k<4; ++k)`<br/>` stmt(k);` |
↳ `stmt(k=0);`<br/>`stmt(k=1);`<br/>`stmt(k=2);`<br/>`stmt(k=3);` |
| | `ptxas` | IR , |

- `T.serial(n)`: , , `ptxas`
- `T.unroll(n)`: IR ** ** 4 : (
↳ ), ** ** ( 4
↳ ) , ( )

: ** (register)** CPU
↳ , GPU ** ** , , `T.unroll`
↳ : , , ** **
↳ , , ,
```

```

> **      **:
↳ (      ,ILP);      ,      TileIR
↳      **      **,
↳      ,
##      `while`

`while`      ,      **,      **(
↳ `i`,      `+1`,      64      )      ,

↳      ,      TileIR      :      ,      **      1

↳      **      (      ;      / Buffers and memory      ):

```python
i: T.int32 = 0
while i < 64:
 A[i] = A[i] + T.float32(1.0)
 i += 1

```

它降级的路子有点出人意料: 它**不直接**生成 `while (i < 64)`, 而是生成一个 `while (1)` 的「死循环」(`while(1)` 的条件永远成立, 本来会无限转), 再在循环体一进来就先检查条件, 不满足就 `break` 跳出去。等于把「停不停」的判断从循环头搬到了循环体开头。计数器 `i` 则被实现成一个长度为 1 的寄存器数组 `i_ptr[1]` (下标固定用 `[0]`):

```

“c++ int i_ptr[1]; i_ptr[0] = 0; while (1) { if (!(i_ptr[0] < 64)) { break; } // 条件不满足
就提前退出 A_ptr[i_ptr[0]] = A_ptr[i_ptr[0]] + 1.0f; i_ptr[0] = i_ptr[0] + 1; // 计数器自增
}

```

```

> ** ? ?** , IR TileIR
↳ (IR) , : ** ** (SSA,
↳ ,)
>
> , ? **: ,
↳ ** 1 , `i += 1`
↳ `i_ptr[0]` : `while`
↳ , ** `(i: T.int32 = 0)`
↳

> ** **: `while(1) + break` `while(cond)` ** **,
↳ , , (PTX, SASS
↳ GPU) ,

##

- TileIR : `if`/`else` `while`, CUDA C++ TileIR
↳ ,
- ** , `T.if_then_else` **: , , ReLU
↳ clamp
- ** , vs **: (`cta_sync`()`,) (
↳ `mbarrier.init()`,), ** **,
↳ , , `warpgroup_sync(id)`
↳

```

```

- , / /
↪ ** **, () `T.unroll` `T.serial` ,
↪
- `while` , `while(1) + break` IR
↪

##

- :[Control flow Modern GPU Programming for MLSys](https://mlc.ai/modern-gpu-programming-for-mlsys |
↪ /tirx_guide/language_reference/cuda/control_flow.html)
- (): Scaling GEMM with Warp Specialization and Clusters (warp cluster
↪ GEMM) CUDA C++/PTX intrinsics () Buffers and memory ()

##

| | English | |
| --- | --- | --- |
| | control flow | `if` / / `while` |
| () | divergence / divergent control flow | warp lane |
| | uniform control flow | |
| | collective operation | () |
| | guard | |
| () | elect (`elect_sync`) | warp lane |
| | predication | |
| | unroll | |
| | vectorize | |
| | deadlock | |
| | CTA / thread block | `__syncthreads()` |
| | warp | 32 lane / |
| warpgroup | warpgroup | warp |
| | mbarrier | |
| | register buffer | 1 |

```

## 第 24 章 · CUDA C++ / PTX intrinsics

原文: CUDA C++ / PTX intrinsics

### 本章要点

#### 本章要点 (TL;DR)

- 先说人话: 在 GPU 上写程序, 代码不叫「函数」, 叫**内核 / kernel**——就是一段「会被成千上万个线程同时各跑一份」的代码。平时我们写内核, 用的是高层的 **tile 原语 / tile primitive**(tile = 把一个大矩阵切成的一小块; 原语 = 框架替你封好的、现成的高级操作), 很省心。可有时候, 你脑子里想要的那个硬件动作, 这层高级封装根本表达不出来。这时候有两条「逃生通道」让你绕过封装、直接命令硬件: 一条是**调用后端内建函数 / backend intrinsic**(intrinsic = 编译器/框架内置的、一一对应某条硬件指令的特殊函数), 也就是 `T.cuda.* / T.ptx.*` 这一堆命名空间; 另一条是**内联一段原始 CUDA 源码**(CUDA = 英伟达写 GPU 程序用的那门 C++ 方言)。
- `T.cuda.* / T.ptx.*` 干的活其实很朴素: 把 GPU 硬件提供的那些底层操作, 原封不动地端到你面前, 让你在 Python 风格的代码里直接调。这里头有线程之间「对表/等齐」用的同步, 有专门等异步搬运完成的 `mbarrier`, 有把一堆数汇总成一个数的归约, 还有 PTX(可以理解成 GPU 的「汇编语言」) 那套搬数据和做矩阵乘法的家族 (`cp_async`、`TMA`、`ldmatrix`、`tcgen05` 这些名字先混个脸熟, 后面会逐个讲)。
- 写 GEMM(矩阵乘法, 深度学习里最核心的运算)、Flash Attention(一种高效的注意力算法, 内部全是矩阵乘和数据搬运) 这类内核, 你会反复碰到 **四类同步机制**: `mbarrier` **相位 / phase**、**单线程选举 / election**、**具名 warpgroup 屏障 / named barrier**、**栅栏 / fence**。它们管两件事: 一是协调成百上千个并行线程, 二是协调那些「自己在后台默默干活的硬件搬运引擎」。坑在哪? 用错了它通常不报错, 而是直接给你悄悄算错数据, 或者干脆卡死。
- 这四个里头, `mbarrier` 的相位坑藏得最深。同一个 `mbarrier` 要在循环里反复用, 你本地那个「相位记号」每转一圈都得手动翻一下 (`phase ^= 1`, 意思是 0 变 1、1 变 0)。一旦忘了翻, 后面的等待会立刻返回——根本没真等, 于是放任硬件去读那块「才写了一半」的内存。
- 万一连内建函数都凑不齐, 还有最后一招: `T.cuda.func_call(..., source_code=..., return_type=...)`。你给它一段原始的 GPU 函数源码, 它原样塞进生成的代码里, 还顺手帮你把调用点接好。

### 前置知识

**前置知识:** 这一章会大量用到 GPU 的几个基础概念——**线程层级** (warp / CTA / warpgroup, 简单说就是「线程怎么分组」)、**异步引擎** (能在后台自己搬数据/算矩阵、不占用线程的专用硬件, 比如 TMA、异步 MMA)、以及**屏障 / 同步** (让一群线程「在某个点上对齐、互相等一等」的机制)。这些词本章第一次出现时都会当场用大白话讲一遍, 所以你没有 GPU 背景也能读,

别担心。要是想更系统地打底, 可以先翻第 0 章 · 极简入门。

## 24.1 为什么需要「逃生通道」

### 一句话先理解

**一句话先理解:** 高级封装 (tile 原语) 能帮你搞定大部分情况, 但 GPU 新硬件出能力的速度太快, 封装总有跟不上时候——这一章就是教你「封装不够用时怎么自己上」。

这本教材从头到尾的主线, 就是用高层的 **tile 原语**来描述 GPU 内核。你只管声明数据怎么摆放 (数据布局)、写「一个小方块 (tile) 上的计算」这一层逻辑, 剩下的全交给框架——它会自动把你的代码**降低 / lower**(编译领域的术语: 把高层、抽象的写法一步步翻译成更底层、更贴近硬件的形式)成高效的 PTX。这套抽象用起来很舒服, 常见的那些套路 (GEMM、注意力 attention、归约……) 它基本都能包圆。

可问题来了: 硬件更新得太快, 封装跟不上。这里先认两个名词——英伟达每隔一两年出一代新 GPU 架构, 会起个代号:**Hopper** 是其中一代 (数据中心常用的 H100 就属于它),**Blackwell** 是更新的一代。Hopper 这代刚带来 **TMA**(Tensor Memory Accelerator, 张量内存加速器——一个专门负责「在后台批量搬运大块数据」的异步引擎, 搬的时候不占用线程),**Blackwell** 又甩出个新的矩阵乘法引擎 **tcgen05**。每出一波这种新能力,tile 原语都**还没来得及封装**, 或者**压根就不打算封** (因为太底层、太专用, 封了也没几个人用)。遇上这种情况, 你总不能被抽象「锁死」、干瞪眼吧。这一章讲的, 就是怎么绕过抽象、直接命令硬件, 一共两条路:

### 结构图

连接关系:

- 你的内核 (prim\_func) → 高层路径: tile 原语
- 你的内核 (prim\_func) → 逃生通道 1: T.cuda.\* / T.ptx.\* (后端内建函数)
- 你的内核 (prim\_func) → 逃生通道 2: T.cuda.func\_call(source\_code=...) (内联原始 CUDA 源码)
- 高层路径: tile 原语  $\xrightarrow{\text{框架自动降低}}$  PTX / SASS
- 逃生通道 1: T.cuda.\* / T.ptx.\* (后端内建函数) → PTX / SASS
- 逃生通道 2: T.cuda.func\_call(source\_code=...) (内联原始 CUDA 源码) → PTX / SASS

### 关键

**关键:** 这两条通道是来「补缺」的, 不是来「顶替」tile 原语的。优先级顺序很清楚, 从上到下越下越底层: **能用 tile 原语就用 tile 原语; 原语够不着, 才下沉到内建函数; 内建函数也没有, 才退到内联源码**。记住一句话: 越往下沉, 框架帮你兜的底越少, 出错的风险越得你自己扛——尤其是同步这块 (为什么同步这么容易出事,24.3 节会让你彻底明白)。



24.2 调用后端内建函数 (backend intrinsics)

T.cuda.\* 和 T.ptx.\* 这两个命名空间 (都来自 `tvm.backend.cuda` 这个模块) 干的事很直白: 把 GPU 硬件提供的那些底层操作原封不动端给你, 让你在代码里直接喊。你能拿到的东西包括: **线程同步** (让一群线程在某个点上互相等齐)、**mbarrier**(一种专门等异步搬运完成的「计数门闩」, 后面 24.3.1 细讲)、**归约** (reduction——把一组数汇成一个值, 最常见的就是求和), 还有 PTX 那套**数据搬运**和 **MMA**(Matrix-Multiply-Accumulate, 矩阵乘加; 这是 GPU 里一块叫 **Tensor Core** 的专用电路干的活——Tensor Core 你可以理解成「专门做小矩阵乘法的硬件加速器」, 深度学习快快在它身上) 家族。

24.2.1 常见入口一览

在看代码前, 先把几个「线程怎么分组」的名词一次说清楚——这是读懂全章的地基, 你会写代码但没碰过 GPU, 所以这几句务必看进去:

- **线程 / thread**:GPU 上最小的执行单位。一份内核代码会被海量线程同时各跑一遍, 只是每个线程处理数据的不同部分。
- **warp(线程束)**:GPU 把线程**每 32 个绑成一束**一起调度, 这一束就叫一个 warp。关键特性: 同一个 warp 里这 32 个线程**必须迈同步** (同一时刻执行同一条指令)。打个比方, 就像 32 个人组成的方阵, 口令一响必须一起迈左脚, 没法你走你的我走我的。
- **CTA / 线程块 (thread block)**: 若干个 warp 再凑成一个 CTA(CTA 是 CUDA 里的叫法, 和「线程块」是一回事)。同一个 CTA 里的线程能共享一块高速内存、能互相等齐。
- **warpgroup(线程束组)**:4 个 warp(也就是 128 个线程) 再抱成一团, 叫一个 warpgroup。这是 Hopper 这代新引入的概念, 后面讲「让不同线程组各干各的活」时会用到。

层级关系串起来就是: **线程** → **32 个凑成 warp** → **几个 warp 凑成 warpgroup** → **若干 warp/warpgroup 凑成 CTA**。

好, 现在看最常用的那几类入口长啥样 (右边注释先扫一眼, 具体每行下面表格里逐个解释):

```
T.cuda.cta_sync() # CTA() , __syncthreads()
T.cuda.warp_sync() # warp , __syncwarp()
T.cuda.warpgroup_sync(8) # warpgroup (, 24.3.3)
T.cuda.cta_sum(val, num_warps, scratch.ptr_to[0])) # CTA

bar = T.alloc_shared((1,), "uint64") # mbarrier
T.ptx.mbarrier.init(bar.data, 1) # mbarrier,
T.ptx.mbarrier.try_wait(bar.data, phase) # (, 24.3.1)
```

这里有个反复出现的词「**屏障 / barrier**」, 先解释清楚: 屏障就是代码里的一个「集合点」——线程跑到这儿会停下等, 直到「该到的线程全到齐了」才一起放行。为啥需要它? 因为线程是各跑各的、快慢不一, 你要是不设集合点, 跑得快的线程可能去读跑得慢的线程「还没写完」的数据, 就出错了。`cta_sync` 是「整个 CTA 集合」, `warp_sync` 是「一个 warp 内集合」, 粒度不同而已。

下面一个一个说:

内建函数	作用	对应的底层硬件操作
T.cuda.cta_sync()	让整个 CTA(线程块) 的线程在此集合等齐	__syncthreads()
T.cuda.warp_sync()	让一个 warp 内的 32 个线程集合等齐	__syncwarp()

T.cuda.warpgroup_sync(id)	让一个 warpgroup 内的线程集合等齐 (用具名屏障实现)	具名屏障 (bar.sync)
T.cuda.cta_sum(val, num_warps, scratch)	CTA 级求和归约 (需要一块叫 scratch 的临时共享内存做中转)	多步 warp shuffle + 共享内存
T.ptx.mbarrier.init(bar, n)	初始化 mbarrier, 告诉它「要等 n 次到达」	mbarrier.init
T.ptx.mbarrier.try_wait(bar, phase)	按相位等屏障翻转, 非阻塞 (等不到立刻返回)	mbarrier.try_wait.parity

表里「对应的底层硬件操作」一列, 像 `__syncthreads()` 这种带双下划线的, 就是 CUDA C++ 里真正的底层函数名;`T.cuda.*` 基本是它们的「一一对应的别名」, 框架会把你写的 `T.cuda.cta_sync()` 直接翻译成 `__syncthreads()`。

注意

**注意:** 这里要补两个概念。第一, **共享内存 / SMEM(shared memory)**: 每个 CTA 都有一小块位于芯片上、特别快的内存, 这个 CTA 里的所有线程都能读写它——你可以把它类比为「一块由你手动管理的高速缓存」, 数据放进去后大家共用, 比从大而慢的全局显存反复取要快得多。第二,**mbarrier** 说白了就是 SMEM 里的一个 64 位整数 (`uint64`)。所以你得先用 `T.alloc_shared((1,), "uint64")` 在共享内存里给它腾块地方, 再把 `.data`(也就是指向这块内存的指针) 递给 `init / try_wait`。它专门干一件事: **等异步引擎** (前面说过, 就是 TMA 拷贝、异步 MMA 这类「在后台自己干活的硬件」) 把活干完再通知你。到了 Hopper/Blackwell 这代, **异步流水线** (让「搬数据」和「算数据」重叠起来同时跑、互不干等, 从而榨干硬件) 的同步全靠它, 算得上是顶梁柱级别的机制。

24.2.2 完整可运行示例:warp 内 all-reduce

一句话先理解

**一句话先理解:** 我们想让一个 warp 里 32 个线程各自手上的数, 加在一起, 而且让每个线程最后都拿到这个总和。难点在于: 线程之间默认不能直接读对方的变量, 得用一种叫 **shuffle** 的特殊硬件指令来交换数据。

先说清楚要解决的问题。一个 warp 有 32 个线程, 假设每个线程手上有一个数, 我们想求这 32 个数的总和——这种「把一堆数汇成一个数」的操作就叫**归约 / reduction**。麻烦在于: 每个线程的局部变量是它自己私有的, 别的线程看不见。那线程之间怎么交换数据?

这就要引入 **shuffle(线程束内洗牌)** 这个 warp 专属的硬件指令。它的能力很特别: **让同一个 warp 里的线程, 直接读取另一个线程寄存器里的值, 完全不经过内存, 所以极快**。下面这个例子用的是它的一个变种 `T.tvw_warp_shuffle_xor`, 来实现「warp 内全归约求和」。要搞懂 shuffle, 这个例子再合适不过了。咱们先看代码, 然后一点点把思路捋清楚 (代码里有些 GPU 专有的函数, 看不懂别慌, 下面逐行解释):

```
@T.prim_func
def warp_reduce(A_ptr: T.handle):
 A = T.match_buffer(A_ptr, (32,), "float32", align=16)
```

```
T.device_entry()
:CTA / warp / lane (0..31)
cta_id = T.cta_id([1]); warp_id = T.warp_id([1]); lane_id = T.lane_id([32])
v = T.alloc_local((1,), "float32"); i = T.alloc_local((1,), "int32")
v[0] = T.float32(31 - lane_id) # lane ()
i[0] = 16 # = 32/2
while i[0] >= 1:
 # : i lane (butterfly)
 v[0] += T.tvmm_warp_shuffle_xor(0xFFFFFFFF, v[0], i[0], 32, 32)
 i[0] = i[0] // 2 # :16 -> 8 -> 4 -> 2 -> 1
 A[lane_id] = v[0] # , 32 lane
```

先逐行解释一下上面这段代码在干嘛 (只挑关键行):

- 1. `cta_id / warp_id / lane_id`: 取当前线程的「三层身份证号」。这里要认识一个新词 **lane(通道)**——它就是「一个线程在自己 warp 里的编号」, 范围 0~31。同一个 warp 里 32 个线程, lane 号分别是 0 到 31。后文说「lane 5」你就理解成「这个 warp 里的第 6 个线程」。
- 2. `v[0] = 31 - lane_id`: 给每个 lane 塞一个不同的初值 (纯粹为了好验证结果)。`alloc_local` 申请的是线程私有的**局部变量**, 每个线程一份、互不干扰——类比 CPU, 就像每个线程有自己的局部变量。
- 3. `i[0] = 16`: 这是一会儿要讲的「蝶形归约」的初始步长, 等于 32 的一半。
- 4. `while i[0] >= 1` 循环: 核心就在这一句 `v[0] += T.tvmm_warp_shuffle_xor(...)`——每个 lane 跟「相距 `i` 个位置」的那个 lane 交换数据并相加, 然后把步长 `i` 折半 ( $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ), 循环 5 次。
- 5. `A[lane_id] = v[0]`: 把结果写回。循环跑完后, 32 个 lane 手里的 `v[0]` 全都等于那个总和。

**核心思路: 蝶形归约 / butterfly reduction.** 先给个直觉: 它就是「两两配对、交换相加, 然后步长一层层折半」这么个求和办法。为什么这么折腾、不直接一个线程把 32 个数加一遍? 因为那样太慢 (要 32 步, 而且只有那一个线程在干活, 其余 31 个干瞪眼)。蝶形归约让所有线程一起动手, **只用 5 步** ( $\log 32 = 5$ ) 就把 32 个数加完, 而且 32 个 lane 个个都攒齐了总和。

`__shfl_xor_sync`(就是 `T.tvmm_warp_shuffle_xor` 对应的底层指令) 妙就妙在配对的方式上。每个 lane 都跟「自己 lane 号**按位异或** `i`」算出来的那个 lane 换数据 (异或 = XOR, 二进制位运算; 不熟也没关系, 看下面的配对表更直观)。`i` 一路取 16, 8, 4, 2, 1, 信息 5 步就传遍整个 warp。最后的效果是:**32 个 lane 全都拿到同一个总和**, 而不是只有 lane 0 一家拿到——这就是为什么它叫「all-reduce / 全归约」(all = 人人有份)。如果只让 0 号拿到总和, 那才叫普通的 reduce。

warp 内 32 个 lane 的蝶形归约 (以步长 `i` 的变化为例):

步长 i	配对 (lane $\rightleftharpoons$ 异或 i 得到的 lane)
i=16	lane 0 $\rightleftharpoons$ lane 16, lane 1 $\rightleftharpoons$ lane 17, ..., lane 15 $\rightleftharpoons$ lane 31
i=8	lane 0 $\rightleftharpoons$ lane 8, lane 1 $\rightleftharpoons$ lane 9, ...
i=4	lane 0 $\rightleftharpoons$ lane 4, ...
i=2	lane 0 $\rightleftharpoons$ lane 2, ...
i=1	lane 0 $\rightleftharpoons$ lane 1, ...

5 步走完, 所有 lane 持有相同的最终和 (all-reduce, 不是单点 reduce)。

这个 `T.tvmm_warp_shuffle_xor`, 框架会把它**原样翻译 (降低)** 成对应的 CUDA 底层函数 `__shfl_xor_sync` 基本就是一对一的直译:

“c++ // 框架生成的 C++, 几乎是一行直译  $v\_ptr[0] = v\_ptr[0] + \_\_shfl\_xor\_sync(0xFFFFFFFF, v\_ptr[0], i\_ptr[0], 32);$

```
> ** **: , `0xFFFFFFFF` **lane / lane mask** 32
↳ lane, 1 lane `0xFFFFFFFF` 32 1,
↳ 32 lane , `_sync` : lane
↳ ** , , () ? **Volta**
↳ GPU , ** , warp 32 ,Volta
↳ ** ** , `_sync` shuffle
↳ ; `_sync` , shuffle `_sync`

24.2.3

shuffle `T.ptx.*` / `T.cuda.*` ,
↳ , , :

| | | |
| --- | --- | --- |
| `cp_async`(LDGSTS) | (= ,) |
↳ tile |
| `cp_async.bulk.tensor`(TMA) | TMA() | Hopper tile |
| `ldmatrix` / `stmatrix` | Tensor Core , / / |
↳ Tensor Core / |
| `tcgen05.*` | Blackwell 5 Tensor Core | Blackwell GEMM |
| `atomic_add` | (=) |
| `fence` | (, 24.3.4) |

** / (global memory)**:GPU (
↳ 24GB), , ,
↳
↳ ** (asynchronous)**: , , ,
↳ ,

> ** **: , ** GPU ** `cp_async`
↳ **Ampere**(Hopper) ,TMA(`cp_async.bulk.tensor`) Hopper ,`tcgen05.*` Blackwell
↳ , , Ampere `tcgen05`
↳ , , `tvm.backend.cuda` API

24.3 (synchronization semantics)

, , GEMM Flash Attention , ** **
↳ :
↳ ** ** (/)

? bug ** ** bug
↳ , , ** ** (**silent
↳ corruption / **: , ,), ** **(**deadlock /
↳ **: , ,) ,

24.3.1 mbarrier (Mbarrier Phases)

> ** **:mbarrier ,
↳ ,
```

```

 :mbarrier , ** /
↪ phase bit** 0 / 1
↪ : , ** ** (0 1, 1 0) , / phase

`T.ptx.mbarrier.try_wait(bar, phase)` ? :

> ** (), `phase` **

: `phase`, ** ** ; , **

↪ ** , , `phase` ^ 1`, ~~~ , 0/1
↪ , `phase` (`phase` ^ 1`, ~~~ , 0/1
↪)

** ** ** mbarrier**(
↪), , ** `phase`
↪ **(`phase` ^ 1`) ? ,
↪ , ? :

- `try_wait(bar, 0)` 0 1, 0
↪ , ,
- `phase` 1, `try_wait(bar, 0)` : 1 ,
↪ 0 1, 0
↪ ** **! ** **,
- : () , () **
↪ (half-written memory)** ,
↪ : ,

(: ** / producer** , ; ** /
↪ consumer** , ,)

(`phase`):

| | | phase | | | `phase` ^ 1` |
| --- | --- | --- | --- |
| 0 | 0 | 0 | `try_wait(bar, 0)` | 1 | phase = 1 |
| 1 | 1 | 1 | `try_wait(bar, 1)` | 0 | phase = 0 |
| 2 | 0 | 0 | `try_wait(bar, 0)` | 1 | ... |

(phase):

| | | phase | | |
| --- | --- | --- | --- |
| 1 | 1 | 0(1) | `try_wait(bar, 0)` | 1, !=0 -> ! ->
↪ -> |

> ** **: ** ** , `phase`
↪ (0 1), ; , , mbarrier
↪ Building a Tiled GEMM ,

24.3.2 (Election)

> ** **: () ** ** ,
↪ `elect_sync()` warp , CTA
↪

```

```

 , warp 32 , TMA 32
 , : ,
 `T.ptx.select_sync()` : ** warp lane** , lane
 , , ** :
- ** lane 0** , , 0
- ** CTA , warp ** warp , CTA
- ** lane / active lanes** (lane; `if`
↳ lane) ,
: ** CTA ** (TMA MMA),
↳ `select_sync()` ?** ** warp , CTA
↳ warp, CTA ? **warp / warp-level
↳ guard** `if` , :
```python
if warp_id == 0: # : 0 warp
    if T.ptx.select_sync(): # : warp lane
        # : CTA
        Tx.gemm_async(...) # GEMM
        # tcgen05.commit(...) # Blackwell MMA

```

结构图

分组:

- **CTA (多个 warp):** 「warp 0」 「warp 1 (被守卫挡掉)」 「warp 2 ... (被守卫挡掉)」

连接关系:

- warp 0 $\xrightarrow{\text{if warp_id == 0}}$ 只剩 warp 0 $\rightarrow$ lane0 ... lane31
- lane0 ... lane31 $\xrightarrow{\text{select_sync() 选 1 个}}$ 整个 CTA 恰好 1 个线程

看懂这段代码的逻辑: `warp_id == 0` 这第一道守卫, 把整个 CTA 里除了 0 号 warp 之外的所有 warp 全挡在门外, 于是只剩 0 号 warp 这 32 个线程能进来; 接着 `select_sync()` 第二道守卫, 在这 32 个里头再选出唯一一个。两道叠在一起, 整个 CTA 里就恰好剩一个线程能执行大括号里的代码了。

注意

注意: 为啥非得「恰好一个线程」不可? 因为 `gemm_async` / `tcgen05.commit` 这类指令, 本质是向异步引擎下一道命令, 同一条命令发一遍就够了。要是放任好几个线程一起发, 轻则重复发起、白费功夫, 重则直接「未定义行为」(undefined behavior——意思是结果完全不可预测, 可能算错、可能崩、可能这次对下次错)。所以「先用 `if warp_id == 0` 守一道, 再用 `select_sync()` 挑一个», 就是把「一大堆线程在跑」稳稳收成「单个线程发令」的**标准套路**, 以后遇到这种场景照抄就行。

24.3.3 具名 warpgroup 屏障 (Named Warpgroup Barriers)

一句话先理解

一句话先理解:cta_sync()(就是 __syncthreads()) 要求「全 CTA 一个不少都到齐」才放行;可一旦不同的线程组在跑不同的代码,这种「全员到齐」的要求就会把自己卡死。这一节讲的是一种「只让某一小组内部对齐、别管其他组」的屏障。

先回顾一下:T.cuda.cta_sync() 对应底层的 __syncthreads(), 它非得 CTA 里每个线程都跑到这一句、到齐了才放行。要是内核里所有线程跑的都是同一套代码,那大家迟早都会到这一句,没问题。

可一旦你用上 warp 特化 / warp specialization, 情况就变了。先解释这个词:warp 特化就是「让不同的线程组各干各的专职活」,而不是所有线程干一样的事。最典型的分工就是——一组 warpgroup(还记得吗,warpgroup = 4 个 warp 抱团) 当生产者专管把数据搬进共享内存,另一组 warpgroup 当消费者专管拿这些数据去算矩阵乘 (MMA)。这就好比流水线上分工:有人专门上料,有人专门加工。为什么要这么分工? 因为搬数据和算数据可以同时进行、互不干等,这样硬件利用率最高。

但分工带来一个麻烦:不同的 warpgroup 现在走的是不同的代码路径了。这时候你要是把 cta_sync() 塞进某一个 warpgroup 的分支里头,就会直接死锁。道理不难想:别的 warpgroup 走的是另一条岔路,压根就不会执行到这句 cta_sync(); 可 __syncthreads() 又死活非要「等齐 CTA 里所有线程」。于是——到了这句的线程在傻等那些永远不会来的线程,大家你等我、我等你,整个程序就僵死了。

硬件早就给这事备了解法:16 个具名屏障 / named barriers, 编号 0~15(「具名」= 每个屏障有自己的编号,你点名用哪个就同步哪一拨人)。T.cuda.warpgroup_sync(10) 就是用 10 号这个屏障,只让某一个 warpgroup 内部的线程互相等齐,别的 warpgroup 它一概不管:

```
#      warpgroup      ID,
T.cuda.warpgroup_sync(wg_id + 10)  #      wg_id=0      10,wg_id=1      11 ...
```

结构图

分组:

- 必须全员到齐,放进分支会死锁:「warp0」「warp1」「... warpN」

连接关系:

- cta_sync() (__syncthreads) → 必须全员到齐,放进分支会死锁
- warpgroup_sync(10) $\xrightarrow{\text{用屏障 ID 10}}$ 仅生产者 warpgroup 内部对齐
- warpgroup_sync(11) $\xrightarrow{\text{用屏障 ID 11}}$ 仅消费者 warpgroup 内部对齐

对比	cta_sync()	warpgroup_sync(id)
同步范围	整个 CTA 的所有线程	单个 warpgroup 内的线程
底层	__syncthreads()	16 个具名屏障之一
放进 warpgroup 分支里	死锁	安全 (其它 warpgroup 不受影响)
ID 管理	无需	必须给不同 warpgroup 分配不同 ID

关键

关键: 具名屏障的编号是**有限的硬件资源**, 拢共就 **16 个** (0~15)。这里顺带认一个词——**寄存器 / register**: 它是每个线程私有的、速度最快的一小块存储, 类比 CPU 的寄存器, 但 GPU 上是「每个线程各有一份」, 数量很少很金贵。具名屏障的稀缺程度和寄存器类似, 所以你得**规划着用**, 千万别让两个不同的 warpgroup 撞到同一个屏障号上 (撞了就会互相误等、出错)。原文那句 `warpgroup_sync(wg_id + 10)` 是个小巧的手法: 拿 warpgroup 自己的编号 (`wg_id`) 去算屏障号, 这样 0 号组用 10、1 号组用 11……每个组自动分到一个**独一无二、不打架**的号。实战怎么用, 见原文「Scaling GEMM with Warp Specialization and Clusters」那章。

24.3.4 栅栏 (Fences)

一句话先理解

一句话先理解: 屏障管的是「人等齐了没」; 栅栏管的是「**读写动作的先后顺序乱没乱**」。这是两回事, 别混。

栅栏 / fence 管的是**定序 / ordering**: 它保证「先写、后读」这个顺序不会被打乱。为什么需要它? 因为现代硬件为了快, 会偷偷调整读写指令的实际执行顺序, 或者把刚写的**数据先攒在某个缓存里**没及时让别人看见。平时这没事, 但当「写的一方」和「读的一方」是两套不同的硬件机制时, 就可能出现「我明明先写了, 你却读到了旧值」的怪事。栅栏就是插在中间的一道命令:「**到此为止, 前面的写必须全部生效、对后面的读可见, 谁也不许乱序**」。

别拿它跟屏障弄混。再强调一遍区分: **屏障**管的是「等一群线程都到齐这个点」; **栅栏**管的是「我写下的东西, 后面的读者一定看得见, 而且看到的顺序没乱」。

原文列了三种栅栏:

栅栏	它保证的定序
<code>T.ptx.fence.proxy_async("shared::cta")</code>	让「线程写进共享内存的东西」, 能被 异步代理 (async proxy) (比如 TMA store、MMA) 随后读到。普通线程访存和异步引擎访存走的是不同的「代理」, 得靠这道栅栏把两边接上。
<code>T.ptx.fence.mbarrier_init()</code>	保证 mbarrier 的 初始化 排在前列, 任何对这个屏障的到达 (arrive) 或等待 (wait) 都在它之后。不然别的线程可能在屏障还没建好的时候就去用了。
<code>T.ptx.tcgen05.fence.after_thread_sync()</code>	在 tcgen05 写回 (writeback) 这条边上, 加一道 偏保守的定序栅栏 。原文说它出现在「Step 8 和 Step 9」, 而且 在 TMA→MMA 这条路径上用不着 。

注意

注意: 要搞懂 `fence.proxy_async`, 关键是「**代理 / proxy**」这个词。在 Hopper/Blackwell 上, 普通线程去读写内存, 和**异步引擎 (TMA/MMA)** 去读写内存, 走的是两条各管各的**通道**, 硬件管这两条通道叫两个不同的「代理」。麻烦在于: 这两条通道**各算各的可见性顺序**, **谁也不自动知道对方写了啥**。所以哪怕你 `__syncthreads()` 让所有线程都对齐了, 也**保证不了**「普通线程写进共享内存的东西, 异步引擎能正确看到」——因为对齐的是线程, 管不着另一条代理通道。`fence.proxy_async("shared::cta")` 干的就是把这两条代理通道接通, 在「CTA 范围

的共享内存」(就是 "shared::cta" 这个参数的意思) 里硬把顺序定死: 线程先写的, 异步引擎一定后读、且读得到。

关键

关键: 第三种栅栏 `tcgen05.fence.after_thread_sync()`, 原文白纸黑字说它「偏保守 / conservative」(意思是它定的顺序比实际需要的更严, 稳是稳, 但代价大), 而且「在 TMA→MMA 这条路径上用不到」。这给我们提了个重要提醒: **栅栏不是白来的, 它有性能开销**——每插一道, 硬件就得停下来「等前面的写都生效」, 会拖慢速度。所以**不能见缝就往里插**。该加的地方 (`tcgen05` 写回那条边) 加上, 不该加的地方 (TMA→MMA) 就省掉——这样既保证正确, 又不白白搭上性能。一句话: 栅栏要「按需精准投放」, 不是越多越安全。

24.4 内联原始 CUDA(Inlining raw CUDA)

一句话先理解

一句话先理解: 这是最后的兜底招——前面那些封装好的入口都没有你要的东西时, 你可以直接写一段原始的 GPU C++ 函数, 塞进去让它原样生效。

万一你要的功能**连内建函数都没有**, 别急, 还有最后一招: 用 `T.cuda.func_call` 把一段 GPU 设备函数源码**原样塞进去**, 调用它的地方框架会帮你自动接好。

先认一个关键字: `__device__`。在 CUDA C++ 里, 函数前面加 `__device__` 修饰, 表示「这个函数是跑在 GPU 上、给 GPU 线程调用的」(对应地, 普通 CPU 函数就没有这个修饰)。还有个常搭配的 `__forceinline__`, 意思是「强烈要求编译器把这个函数的代码直接展开到调用处」, 省去函数调用开销。下面例子里这两个你都会看到。

```
SRC = r"""
__device__ __forceinline__ float my_relu(float x) { return x > 0.f ? x : 0.f; }
"""

@T.prim_func
def k(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")
    B = T.match_buffer(B_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([256])
    #           :       +       +       +
    B[tx] = T.cuda.func_call("my_relu", A[tx], source_code=SRC, return_type="float32")
```

上面这段代码读法: SRC 里写了一个叫 `my_relu` 的 GPU 函数 (就是个 ReLU: 大于 0 返回原值, 否则返回 0); 然后在内核里, 通过 `T.cuda.func_call("my_relu", A[tx], ...)` 调用它, 把结果存到 `B[tx]`。

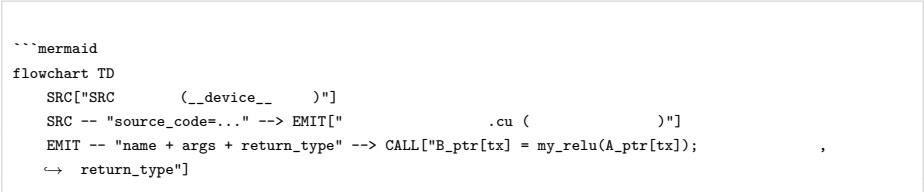
`func_call` 有四个要紧的参数:

- **name:** 你要调用的那个设备函数叫啥名字 (这里是 "my_relu")。注意, 它必须跟 `source_code` 里定义的函数名**一字不差地对上**, 否则框架找不到。
- ***args:** 递给这个函数的实参, 这里就是 `A[tx]` (当前线程负责的那个元素)。

- `source_code=...`: 一段源码字符串, 框架会把它一字不改地 (verbatim) 塞进生成的代码文件里。
- `return_type=...`: 返回类型。框架靠这个信息知道「这次调用返回的是个什么类型的值」, 好把结果接回到表达式里, 这里是 `"float32"`(32 位浮点数)。

降低之后的 C++ 大概是这副样子——源码原样落地, 调用点自动接上:

```
“c++ __device__ __forceinline__ float my_relu(float x) { return x > 0.f ? x : 0.f; }  
// ... B_ptr[tx] = my_relu(A_ptr[tx]); // 调用点由框架自动生成
```



注意

注意: 这是逃生通道里最底下的一档, 动手前先想清楚。源码「一字不改地塞进去」听着方便, 但反过来说就是: 除了帮你接一下返回类型, 框架几乎啥检查、啥兜底都不给你做了。拼写有没有错、函数签名对不对、`__device__` / `__forceinline__` 这类修饰加没加、有没有越界——全得你自己写对, 错了它也不拦你, 直接交给底层编译器去崩。所以一条铁律: 但凡 `T.cuda.*` / `T.ptx.*` 内建函数能办的事, 就别退到这一层来。

小结

- 本章给了你两条直达硬件的逃生通道。它俩定位一样: 高层的 `tile` 原语够不着时, 拿来补缺。
 - 后端内建函数 `T.cuda.*` / `T.ptx.*`——同步、`mbarrier`、归约、`shuffle`, 加上 `cp_async/TMA/ldmatrix/t` 这些搬数据和做矩阵乘的家族, 基本就是一条「直接调用 GPU 底层硬件操作」的直通车。
 - 内联原始 `CUDA T.cuda.func_call(source_code=...)`——连内建函数都没有时的最后兜底, 把一段 `__device__ GPU` 函数源码一字不改地塞进去。
- 本章最大的雷区, 是同步语义——再提醒一遍, 这类错误通常不报错, 而是悄悄算错或者卡死, 所以最值得花心思。四个机制, 每个都各自带着一个「最容易踩」的坑:

机制	一句话记忆	最容易踩的坑
<code>mbarrier</code> 相位	<code>try_wait</code> 等的是相位「翻转」	循环复用时忘了 <code>phase ^= 1</code> → 不等就返回 → 读半写内存
单线程选举	<code>elect_sync()</code> 是「每 warp 一个」	误以为是 <code>lane 0</code> / 每 CTA 一个; 需配 <code>if warp_id == 0</code> 守卫
具名 <code>warpgroup</code> 屏障	只同步本 <code>warpgroup</code> , 共 16 个	<code>cta_sync()</code> 放进 <code>warpgroup</code> 分支 → 死锁; ID 撞车 → 互相干扰
栅栏	定序「写在前、读在后」	跨代理 (proxy) 不加栅栏 → 异步引擎读到旧值; 乱加 → 损性能

- 最后送你一个统一的心智模型: 逃生通道越往下沉, 框架帮你兜的底就越少。从 `tile` 原语 → 内建函数 → 内联源码, 这一路下来, 「写对」的担子是一级一级往你自己肩上压的。尤其是异步引

擎和线程同步的先后顺序, 千万别凭感觉瞎写——一定对着实战章节里给好的相位跟踪表和栅栏摆放规则来, 照着抄, 别自己发明。

延伸阅读

- 原文链接:[CUDA C++/PTX intrinsics](#)
- 原文交叉引用的实战章节 (用于把本章机制落地):
 - **Building a Tiled GEMM** ——完整走一遍 mbarrier 相位跟踪表, 以及 `elect_sync()` + warp 守卫发起 `gemm_async / tcgen05.commit`。
 - **Scaling GEMM with Warp Specialization and Clusters** ——具名 warpgroup 屏障在 warp 特化下的用法与 ID 规划。
- `tvm.backend.cuda` 后端 API 参考——`T.cuda.* / T.ptx.*` 全部内建函数的完整清单。

术语对照

中文	English
后端内建函数	backend intrinsic
tile 原语	tile primitive
线程束	warp
线程束组	warpgroup
共享内存	SMEM(shared memory)
线程块	CTA(thread block)
蝶形归约	butterfly reduction
全归约	all-reduce
通道 (lane)	lane
通道掩码	lane mask
相位	phase(mbarrier phase)
单线程选举	election(<code>elect_sync</code>)
具名屏障	named barrier
栅栏	fence
异步代理	async proxy
张量内存加速器	TMA(Tensor Memory Accelerator)
静默数据损坏	silent corruption
死锁	deadlock
warp 特化	warp specialization

术语对照表 (全书总表)

本表汇总《现代 GPU 编程·面向 ML 系统 (中文精读笔记)》各章出现的核心术语, 去重整理为「英文—中文—简要说明」三列。

同一概念在不同章节可能有略微不同的中文译法 (如 *warp specialization* 既见「Warp 专门化」也见「warp 专精」「warp 特化」), 本表取常用译法并在说明中兼顾。返回首页:README。

A · 架构与执行模型

英文	中文	简要说明
SIMT (Single Instruction Multiple Threads)	单指令多线程	GPU 的基本执行范式: 一条指令同时驱动 warp 内多个线程。
warp / lane	线程束 / 通道	32 个线程组成一个 warp, warp 内每个线程占一条 lane (由 laneid 标识)。
warpgroup	线程束组 / warp 组	4 个 warp (128 线程) 组成一组, 是新一代 MMA 的调度单位。
CTA (Cooperative Thread Array)	协作线程阵列 / 线程块	即 thread block, 共享同一块 SMEM 的线程集合。
cluster / thread block cluster	集群 / 线程块集群	多个 CTA 组成的集群, 可跨 CTA 共享内存与协作。
scope	作用域 (谁来执行)	指定一段代码由哪一层级 (线程 / warp / warpgroup / CTA) 执行。
occupancy	占用率	SM 上活跃 warp 数与硬件上限之比, 影响延迟隐藏能力。
overlap	重叠	让数据搬运与计算同时进行, 是性能优化的核心杠杆。
persistent kernel	持久化核 / 常驻核	一个 CTA 常驻 SM 循环领取多个 tile, 摊薄启动开销。
grid scheduler / grid stride	网格调度器 / 网格步长	网格层面分发工作的调度器, 及持久化核循环领取 tile 的步长。
work-stealing	工作窃取	空闲 SM 主动抢占未完成 tile, 改善尾部负载均衡。
critical path	关键路径	决定内核总耗时最长依赖链。
tail behavior	尾部行为	网格末尾 SM 负载不均导致的性能损耗现象。

B · 性能模型

英文	中文	简要说明
roofline model	屋顶线模型	以内存带宽与算力吞吐为上界, 用算术强度定胜负的性能模型。
arithmetic intensity (AI)	算术强度	每搬运一字节数据所执行的浮点运算数 (FLOP/Byte)。

memory-bound / compute-bound	内存受限 / 算力受限	性能瓶颈分别落在带宽端或算力端的两种状态。
ridge point	脊点	屋顶线上内存受限与算力受限的分界点。
fusion	算子融合	合并多个算子以减少中间结果的访存往返。
blocking for reuse	为复用而分块	把数据切成 tile 留在片上反复使用，提高算术强度。

C · 数据布局 (Layout)

英文	中文	简要说明
data layout	数据布局	逻辑张量到物理存储位置的映射规则。
shape / stride	形状 / 步长	张量各维的大小，以及沿各维移动一格对应的物理偏移。
view	视图	不复制数据，仅改变 shape/stride 的张量重解释。
tile / tiling	瓦片 / 分块	把大张量切成可放入片上内存的小块。
named axes (TLane / TCol)	命名轴	与硬件对应的具名逻辑轴（如 TMEM 的 Lane/Col）。
affine map	仿射映射	用线性公式描述逻辑坐标到物理地址的映射。
forward mapping	前向映射	一个逻辑坐标映射到一组物理坐标的方向。
register fragment	寄存器片段	MMA 操作数 / 结果在各线程寄存器中分布的小块。
replication / broadcast	复制 / 广播	同一份数据被多个线程持有的布局模式。
sharding / shard	分片	把数据切分给不同线程 / 单元各持一份。
offset	偏移	布局起始位置相对基址的位移。
iter (extent, stride, axis)	迭代器三元组	描述一维遍历的范围、步长与所属轴。
atom	原子单元	布局可重复拼贴的最小基本块。
swizzle	混排 / 地址混洗	对 SMEM 地址做 XOR 重排以避免 bank 冲突。
swizzle atom	swizzle 原子	swizzle 模式可重复的最小单元。
bank conflict	bank 冲突	多线程同周期访问同一 SMEM bank 导致的串行化。
global memory coalescing	全局内存合并访问	warp 内连续地址合并成少量内存事务的高效访问。
warpx4 broadcast	warpx4 广播	一份数据广播给 4 个 warp 共享的布局。
TileLayout	仿射布局	TIRx 的主力布局对象，用 shard/replica/offset 描述仿射映射。
SwizzleLayout / ComposeLayout	换序布局 / 组合布局	XOR 换序布局，以及把多个布局组合成新布局。
tensor layout model	张量布局模型	贯穿全书的「逻辑坐标 → 物理坐标集合」统一模型。

D · Tensor Core 与 MMA

英文	中文	简要说明
Tensor Core	张量核 (固定功能单元)	专做矩阵乘累加的硬件单元，非通用 ALU。
MMA (Matrix-Multiply-Accumulate)	矩阵乘累加	Tensor Core 执行的核心操作 $D = A \cdot B + C$ 。
accumulator	累加器	存放 MMA 累加结果的寄存器 / TMEM 区域。
tcgen05	第 5 代 Tensor Core 指令族	Blackwell 的 MMA 指令集，结果写入 TMEM。
matrix descriptor	矩阵描述符	告诉 MMA 操作数在 SMEM 中布局 / 主序的元数据。
major mode (K-major / MN-major)	主序 (K 主序 / MN 主序)	操作数在内存中沿 K 还是 MN 维连续。
block-scaled MMA / scale factor	块缩放 MMA / 缩放因子	低精度 MMA 按块附带缩放因子以保持数值范围。
epilogue	收尾阶段 / 尾声阶段	MMA 后做缩放、加偏置、类型转换并写回的阶段。

E · 内存层级与特殊内存

英文	中文	简要说明
TMEM (Tensor Memory)	张量内存	Blackwell 专供 Tensor Core 累加的二维 (Lane $\times$ Col) 片上内存。
Lane / Col (TLane / TCol)	TMEM 二维坐标轴	TMEM 的两个寻址维度。
allocated_addr (column offset)	列偏移地址	TMEM 分配返回的列方向起始地址。
DSMEM (Distributed Shared Memory)	分布式共享内存	集群内一个 CTA 可访问另一 CTA 的 SMEM。
static shared memory	静态共享内存	编译期确定大小的 SMEM。
dynamic shared memory / arena	动态共享内存 / 竞技场	运行时按需从竞技场分配的 SMEM。
bump allocator (SMEMPool)	碰撞分配器	顺序推进指针的轻量 SMEM 分配器。
buffer	缓冲区	指针之上附加 shape/stride/layout 元数据的抽象。
metadata over a pointer	指针之上的元数据	buffer 的本质: 裸指针 + 布局信息。
elem_offset	元素偏移	buffer 起始元素相对基指针的偏移量。
register buffer	寄存器缓冲区	驻留在寄存器中的小型缓冲区。
scalar	标量	单元素本地缓冲区的语法糖。

F · 异步搬运与协调

英文	中文	简要说明
TMA (Tensor Memory Accelerator)	张量内存加速器	异步批量在全局与共享内存间搬运 tile 的硬件引擎。
tensor-map descriptor	张量映射描述符	描述全局张量布局供 TMA 寻址的对象。
tile coordinates	tile 坐标	TMA 按整块定位时使用的块级坐标。
mbarrier (memory barrier)	内存屏障 / 异步屏障	到达计数 + 相位 + 字节计数三合一的异步同步原语。

arrival counter / arrival count	到达计数器 / 到达计数	记录已到达 barrier 的参与者数量。
phase / phase bit	相位 / 相位位	barrier 在两个相位间翻转，用于区分世代。
arrive / wait	到达 / 等待	参与者声明到达，以及阻塞等待 barrier 满足。
expect_tx / arrive.expect_tx	期望字节预算	提前声明本世代期望接收的字节数。
complete-tx / tx-count / byte count	完成事务 / 字节计数	异步引擎回报已完成的字节 / 事务数。
byte-count completion	字节数完成	以累计字节数达到预算作为完成判据 (区别于到达计数)。
try_wait / try_cancel / query_cancel	等待释放 / 请求 / 查询指令	非阻塞探测 barrier，以及 CLC 的请求 / 查询新 tile。
commit group / wait group	提交组 / 等待组	把一批异步操作打包提交并按组等待完成。
commit	提交	把 MMA 工作组提交给硬件执行。
double buffering / ring buffer	双缓冲 / 环形缓冲	用多份缓冲让搬运与计算重叠。
multi-stage buffering	多级缓冲	超过两级的流水缓冲。
stage	流水级	流水线中的一个阶段。
ownership transfer	所有权转移	缓冲区在生产者与消费者间交接使用权。
async proxy	异步代理	区分同步 / 异步访存的内存代理，需配 fence 保证可见性。

G · CLC 与调度

英文	中文	简要说明
Cluster Launch Control (CLC)	集群启动控制	让常驻集群运行时动态领取新输出 tile 的机制。
tile scheduler / l2_group_size	tile 调度器 / L2 分组大小	决定 tile 领取顺序的调度器，及为 L2 复用而分组的大小。
output tile	输出分块	一个 CTA / 集群负责计算的结果块。
sentinel	哨兵值	标记「无更多工作」等特殊状态的占位值。
predicate	谓词	控制指令是否生效的布尔条件。

H · Warp 专门化与调试

英文	中文	简要说明
warp specialization	Warp 专门化 / 专精	不同 warp 各司其职 (如生产者搬运 vs 消费者计算)。
handoff	交接	一段数据 / 资源在角色间移交的事件，是调试建模的核心。
role guard	角色守卫	用条件分支把代码限定给特定角色 warp 执行。
collective operation / collective op	集体操作 / 集合操作	必须由作用域内全体参与者一起执行的操作。
context poisoning	上下文毒化	在错误作用域调用集体操作污染后续状态的错误。

race condition	竞争条件	因缺乏同步导致结果依赖执行时序的缺陷。
silent corruption	静默数据损坏	不报错却产生错误结果的隐蔽缺陷。
deadlock	死锁	参与者互相等待、谁也无法前进的僵局。

I · TIRx 语言与控制流

英文	中文	简要说明
TIRx (Tensor IR neXt)	张量 IR neXt	IR 层级的 GPU 编程 DSL，显式表达 <code>scope/layout/dispatch</code> 。
dispatch	派发	选择某操作走哪条具体硬件路径。
layout	布局	数据 <code>tile</code> 住在哪、怎么排 (见上方 <code>Layout</code> 专节)。
dtype	数据类型	低层位级类型 (如 <code>float32x4</code>)。
type / PrimType / PointerType	高层类型 / 指针类型	IR 中的高层类型系统。
PrimExpr	原始表达式	TIRx 的基础表达式节点。
handle / pointer	句柄 / 指针	通用指针 <code>dtype</code> 。
vector dtype	向量类型	如 <code>float32x4</code> $\rightarrow$ <code>float4</code> 的打包向量。
vectorized memory access	向量化访存	用 <code>vload</code> / <code>vstore</code> 一次搬多个元素。
let / immutable binding	不可变绑定	一次性赋值、不可重赋的命名值。
scalar (sugar)	标量 (语法糖)	1 元素本地缓冲区的便捷写法。
uniform vs divergent control flow	统一 vs 发散控制流	<code>warp</code> 内是否所有线程走同一分支。
elect_sync / election	选举单线程发射 / 选举	在 <code>warp</code> 内推选一个线程执行单线程操作。
T.if_then_else / ternary	表达式级选择 / 三元运算符	表达式而非语句层面的条件选择。
cta_sync / __syncthreads	线程块同步	CTA 内全体线程同步屏障。
wargroup_sync / named wargroup barrier	wargroup 同步 / 具名 wargroup 屏障	wargroup 作用域的 (可具名) 同步。
mbarrier.init	内存屏障初始化	初始化一个 <code>mbarrier</code> 对象。
fence	栅栏	强制内存可见性 / 顺序的屏障。
unroll / serial / vectorized / grid	完全展开 / 顺序 / 向量化 / 嵌套循环	循环的不同执行修饰符。
intrinsic	内建函数	直接映射到底层指令的函数。
backend intrinsic	后端内建函数	暴露 CUDA C++ / PTX 能力的内建。
butterfly reduction	蝶形归约	warp 内用 <code>shuffle</code> 做对数级归约的模式。
escape hatch	逃生口	绕过高层抽象、直达底层指令的机制。

J · 解析期 / 元编程

英文	中文	简要说明
parser utilities	解析器工具	在解析阶段生效的一组元编程辅助。
parse time / parse-time metaprogramming	解析期 / 解析期元编程	在源码被解析为 IR 时执行的元编程。
metaprogramming	元编程	用代码生成 / 操控代码的技术。
meta value / meta_var	元值	仅存在于解析期、不进入运行时的值。
inline (@T.inline)	内联 (就地展开)	把函数体在调用处直接展开。

meta_class (@T.meta_class)	元类	解析期可参数化展开的类模板。
constexpr / compile-time constant	编译期常量	编译期即可求值的常量。
specialize	特化	为特定参数生成专门版本的代码。
unroll	循环展开	解析期把循环体复制多份。
lexical (LEGB) scoping	词法作用域	按代码书写层次解析名字的规则。
late binding	延迟绑定	名字到值的绑定推迟到使用时确定。
shadowing	遮蔽 (同名覆盖)	内层同名变量遮住外层变量。

K · 编译器内部与下降

英文	中文	简要说明
lowering / lowering pipeline	下降 / 降级 / 下降流水线	把高层 IR 逐步转成低层代码的多 pass 流程。
tirx pipeline	TIRx 流水线	<code>tvm.compile(tir_pipeline="tirx")</code> 内部的下降流水线。
LowerTIRx	TIRx 核心下降 pass	流水线中最关键的下降阶段。
tile primitive	tile 原语 / 瓦片原语	以 tile 为操作单位的高层算子原语。
TilePrimitiveDispatch	tile 原语派发	把 tile 原语映射到具体硬件实现的子阶段。
LayoutApplier	布局应用器 / 施加器	把 TileLayout 实际落到缓冲区上的子阶段。
execution-scope id	执行域 id	标识某段代码所属执行作用域的标记。
SplitHostDevice	host/device 拆分	把混合代码拆成主机与设备两个函数。
host function / device function	主机函数 / 设备函数	分别在 CPU 与 GPU 上运行的函数。
packed-func ABI	打包函数 ABI	TVM 主机 / 设备调用约定。
finalization	收尾 (阶段)	下降流水线末端的整理 / 渲染阶段。
warp-scoped memory	线程束作用域内存	归属于单个 warp 的内存。
safety gate	安全闸门	拦截非法 IR、保证下降正确性的检查。
SROA / CSE	聚合体标量替换 / 公共子表达式消除	两类标准编译优化。

L · 参考与生态

英文	中文	简要说明
Compiler Internals	编译器内部实现	面向贡献者的编译器架构文档。
TIRx Language Reference	TIRx 语言参考	TIRx 语言特性的权威说明。
TIRx Layout API	TIRx 布局 API	把布局记号具体化为编译器对象的 API。
Debugging Warp-Specialized Kernels	Warp-Specialized 内核调试	异步专门化内核的调试操作手册。
TIRx native level	TIRx 原生层	TIRx 最底层、最贴近硬件的抽象层。
upstream TVM	上游 TVM	完整 <code>tvm.tirx</code> API 所在的上游项目文档。

M · 关键算法

英文	中文	简要说明
GEMM	通用矩阵乘	全书贯穿的核心计算示例。
Flash Attention	闪存注意力	分块 + 在线 softmax、避免物化大注意力矩阵的算法。